Answers to

Theory of Computation

Jim Hefferon
https://hefferon.net/computation

About these answers

A person who provides answers to text exercises has decisions to make: some exercises or all? or maybe all exercises to instructors only? complete worked answers or summaries? if summaries, what about exercises involving proofs?

I do generate the complete answers. I find that when I write a book, if I write out the complete answers then that makes the questions better. As I work through them, I am checking that the section contains all the needed prerequisites and also that the question is clear. I often go back to adjust either the question or the section body. Besides, I enjoy the process and there may as well be one person having a good time.

However, there is something to be considered in furnishing them. The point of many exercises is for students to do them, and perhaps struggle. Providing the answers risks that some students will just read the question and then read the answer and so proceed with the sense that they understand, which they do not. But weighed against that is the worry that providing incomplete answers or hints, or an incomplete list of answers such as odd only, can leave an honestly-trying person with no way to get what it is that they didn't see.

I am sometimes asked to provide for download the answers to only some exercises, and to reserve the rest for instructors. Putting aside the increased burden on me of email with instructors, for something that I give away, this is impractical. I have no way of verifying when someone emails me that they are who they claim to be. In addition, this request assumes that people are using the text in the framework of a college and university system, while I find that many readers are outside of that system, and just want to learn. Finally, I'll also remark that the people asking this have a different idea than I do of the relationship between undergraduates and the Internet. My experience has been that, as with a traditionally published Calculus text, any student who looks for the complete answer document will be able to find it within minutes. So the belief that somehow the other texts that students have do not give them access to all exercises seems to me to be mistaken.

In short, I provide complete answers to all exercises, including proofs. I perceive that the potential for help outweighs the potential for harm.

Note: This PDF matches the book version compiled on 2025-Jun-17. If you put that book version and this answers in the same directory then clicking on an exercise number in the book will take you to its answer here, and clicking on the answer number will take you back to the book.

Jim Hefferon
University of Vermont

Cover: Flauros, the sixty-fourth demon and the Duke of Hell, who gives true answers of all things past, present, and future. Although, you should verify what he says for yourself. (Unless you can get him inside a triangle and then you're good, although it is unclear whether that is a triangle chalked out on the floor, or of roads, or rivers, or planets.) From *Dictionnaire Infernal*, Jacques Albin Simon Collin de Plancy, 1863.

Chapter I: Mechanical Computation

I.1.11 It is like a program in that it is single-purpose, that is, just as we may have a program that doubles its input and does nothing else, so also we may have such a Turing machine. It is unlike a program in that it is hardware.

It is like the computers we have on our desks today in that it mechanically converts the input into the output. A Turing machine is unlike such a computer in that it is purpose-built, for a single job.

I.1.12 The definition describes the machine. Further description of the machine's action, which involves the definitions of configurations and the transitions, follows Definition 1.4. In some ways it is an extension of Definition 1.4.

Restated, as A Bauer says in a comment to this question on Stack Exchange, why is the road not part of the car?

I.1.13 It is not a question of determining whether a particular mathematical statement is true. Obviously, if $x = 2$ then we can determine whether $x > 3$. Similarly, we can determine the truth or falsity of $\forall x \in \mathbb{N}\ [x^2 > 3]$. Rather, the *Entscheidungsproblem* asks for an algorithm that inputs any mathematical statement at all, and outputs '*T*' or '*F*'.

I.1.14

(A) Here is the sequence of tapes traced out in the course of the computation of the predecessor of 2.

Step	*Machine*
0	1 1 q_0
1	1 1 q_0
2	1 1 q_0
3	1 1 q_1
4	1 q_1
5	1 q_2
6	1 q_2
7	1 q_3

(B) This is the tape diagram sequence for the sum of 2 and 2.

Step	*Machine*
0	1 1 1 1 q_0
1	1 1 1 1 q_0
2	1 1 1 1 q_0
3	1 1 1 1 q_1
4	1 1 1 1 1 q_1
5	1 1 1 1 1 q_2
6	1 1 1 1 1 q_2
7	1 1 1 1 1 q_2
8	1 1 1 1 1 q_2
9	1 1 1 1 1 q_3
10	1 1 1 1 1 q_3
11	1 1 1 1 q_4
12	1 1 1 1 q_5

(C) This is the predecessor computation.

$$\langle q_0, 1, \varepsilon, 1\rangle \vdash \langle q_0, 1, 1, \varepsilon\rangle \vdash \langle q_0, \mathsf{B}, 11, \varepsilon\rangle \vdash \langle q_1, 1, 1, \varepsilon\rangle \vdash \langle q_1, \mathsf{B}, 1, \varepsilon\rangle \vdash \langle q_2, 1, \varepsilon, \varepsilon\rangle$$
$$\vdash \langle q_2, \mathsf{B}, \varepsilon, 1\rangle \vdash \langle q_3, 1, \varepsilon, \varepsilon\rangle$$

There is no instruction for state q_3 so $\langle q_3, 1, \varepsilon, \varepsilon\rangle$ is a halting configuration.

The sum of 2 and 2 is a little longer.

$$\langle q_0, 1, \varepsilon, 1\mathtt{B}11\rangle \vdash \langle q_0, 1, 1, \mathtt{B}11\rangle \vdash \langle q_0, \mathtt{B}, 11, 11\rangle \vdash \langle q_1, \mathtt{B}, 11, 11\rangle \vdash \langle q_1, 1, 11, 11\rangle$$
$$\vdash \langle q_2, 1, 11, 11\rangle \vdash \langle q_2, 1, 1, 111\rangle \vdash \langle q_2, 1, \varepsilon, 1111\rangle \vdash \langle q_2, \mathtt{B}, \varepsilon, 11111\rangle$$
$$\vdash \langle q_3, \mathtt{B}, \varepsilon, 11111\rangle \vdash \langle q_3, 1, \varepsilon, 1111\rangle \vdash \langle q_4, \mathtt{B}, \varepsilon, 1111\rangle \vdash \langle q_5, 1, \varepsilon, 111\rangle$$

I.1.15

(A) To show that some operation on negative numbers (or any mathematical object, for that matter) can be computed, name a representation for them with strings over a finite alphabet and then show how to use a Turing machine to do mechanical computations on those strings. For instance, to show that multiplication of two integers is mechanical, we could use the alphabet $\Sigma = \{\mathtt{B}, \mathtt{0}, \ldots \mathtt{9}, \mathtt{+}, \mathtt{-}\}$ and represent each integer as a pair $\langle$sign, natural number$\rangle$. Making a Turing machine that has four cases, for the four possible pairs of signs, is tedious but perfectly possible.

(B) If by "problem" we mean that the machine fails to halt, this is not the source of the problem. Rather, this machine is a straightforward infinite loop.

(C) We could give a machine states that are not sequential. The only limitation is that we use the convention that the initial state is q_0. For instance, this machine fits the definition $\mathcal{P} = \{q_0\mathtt{B}\mathtt{B}q_{50},\ q_0\mathtt{1}\mathtt{1}q_{50},\ q_{50}\mathtt{1}\mathtt{1}q_1,\ q_{50}\mathtt{1}\mathtt{1}q_1\}$, and on any input reaches state q_{50} after one step.

I.1.16 The halting state is q_4 and the working states are q_0, q_1, q_2 and q_3.

I.1.17 The trace is not too interesting.

Step	*Machine*
0	q_0
1	q_0
2	q_0
3	q_0

Step	*Machine*
4	q_0
5	q_0
6	q_0
7	q_0

Step	*Machine*
8	q_0
9	q_0
10	q_0

I.1.18

(A) The result of running the mystery machine on input 11 is this.

Step	*Machine*
0	1 1 q_0
1	1 1 q_1

Step	*Machine*
2	1 1 q_0
3	1 1 q_4

(B) On input 1011 it gives this.

Step	*Machine*
0	1 0 1 1 q_0
1	1 0 1 1 q_1

Step	*Machine*
2	1 0 1 1 q_2
3	1 0 1 1 q_3

(C) The input 110 gives this.

Step	*Machine*
0	1 1 0 q_0
1	1 1 0 q_1
2	1 1 0 q_0
3	1 1 0 q_0
4	1 1 0 q_4

(D) With the input 1101 the machine gives this sequence of tapes.

Step	*Machine*
0	1 1 0 1 q_0
1	1 1 0 1 q_1
2	1 1 0 1 q_0
3	1 1 0 1 q_0
4	1 1 0 1 q_1
5	1 1 0 1 q_4

(E) This is what happens when the initial tape is empty.

Step	*Machine*
0	q_0
1	q_4

I.1.19 This is the transition table.

Δ	B	0	1
q_0	Bq_4	Rq_0	Rq_1
q_1	Bq_4	Rq_2	Rq_0
q_2	Bq_4	Rq_0	Rq_3
q_3	–	–	–

I.1.20 The machine $\mathcal{P}_{\text{blankones}} = \{q_0\text{BB}q_2,\ q_0\text{1B}q_1,\ q_1\text{BR}q_0,\ q_1\text{11}q_0\}$ does this on a tape with two 1's.

Step	*Machine*
0	1 1 q_0
1	1 q_1
2	1 q_0
3	 q_1
4	 q_0
5	 q_2

I.1.21

(A) This machine does the job; note that the head never moves.

$$\mathcal{P}_{\text{singlebitnot}} = \{q_0\text{BB}q_1,\ q_0\text{01}q_1,\ q_0\text{10}q_1\}$$

This shows input $\sigma = \texttt{0}$.

Step	*Machine*
0	0 q_0
1	1 q_1

(B) This machine does 'and'. Note that the states are not numbered sequentially; sometimes making a gap is convenient when you are writing the machine. For instance, here state q_{100} is used as the halting state, since it was clear there would not be a hundred states and so this one would be available. Similarly, q_{10} is for what the machine does if the first bit is 0 while q_{20} is for the case where the first bit is 1.

$$\begin{aligned}\mathcal{P}_{\text{singlebitand}} = \{\,&q_0\mathtt{BB}q_{100},\ q_0\mathtt{0B}q_{10},\ q_0\mathtt{1B}q_{20},\ q_{10}\mathtt{BR}q_{11},\ q_{10}\mathtt{00}q_{100},\ q_{10}\mathtt{11}q_{100},\\ &q_{11}\mathtt{BB}q_{100},\ q_{11}\mathtt{00}q_{100},\ q_{11}\mathtt{10}q_{100},\ q_{20}\mathtt{BR}q_{21},\ q_{20}\mathtt{00}q_{100},\ q_{20}\mathtt{10}q_{100},\\ &q_{21}\mathtt{BB}q_{100},\ q_{21}\mathtt{00}q_{100},\ q_{21}\mathtt{11}q_{100}\,\}\end{aligned}$$

This shows what the machine does on input $\sigma = \mathtt{01}$.

Step	*Machine*
0	0 1 q_0
1	1 q_{10}

Step	*Machine*
2	1 q_{11}
3	0 q_{100}

(C) This is the 'or' machine. As in the prior item, the states are not numbered sequentially. Note also that this machine is basically the same as the 'or' machine.

$$\begin{aligned}\mathcal{P}_{\text{singlebitor}} = \{\,&q_0\mathtt{BB}q_{100},\ q_0\mathtt{0B}q_{10},\ q_0\mathtt{1B}q_{20},\ q_{10}\mathtt{BR}q_{11},\ q_{10}\mathtt{00}q_{100},\ q_{10}\mathtt{11}q_{100},\\ &q_{11}\mathtt{BB}q_{100},\ q_{11}\mathtt{00}q_{100},\ q_{11}\mathtt{11}q_{100},\ q_{20}\mathtt{BR}q_{21},\ q_{20}\mathtt{00}q_{100},\ q_{20}\mathtt{10}q_{100},\\ &q_{21}\mathtt{BB}q_{100},\ q_{21}\mathtt{01}q_{100},\ q_{21}\mathtt{11}q_{100}\,\}\end{aligned}$$

Here is what it does on input $\sigma = \mathtt{01}$.

Step	*Machine*
0	0 1 q_0
1	1 q_{10}

Step	*Machine*
2	1 q_{11}
3	1 q_{100}

I.1.22 This machine uses q_0 to slide to the right end, appends the 01, and uses q_2 to slide back to the left.

$$\mathcal{P}_{\text{appendo1}} = \{\,q_0\mathtt{B0}q_1,\ q_0\mathtt{0R}q_0,\ q_0\mathtt{1R}q_0,\ q_1\mathtt{B1}q_2,\ q_1\mathtt{0R}q_1,\ q_1\mathtt{1L}q_2,\ q_2\mathtt{BR}q_3,\ q_2\mathtt{0L}q_2,\ q_2\mathtt{1L}q_2\,\}$$

This example shows what the machine does, on input $\sigma = 11$.

Step	*Machine*
0	1 1 q_0
1	1 1 q_0
2	1 1 q_0
3	1 1 0 q_1

Step	*Machine*
4	1 1 0 q_1
5	1 1 0 1 q_2
6	1 1 0 1 q_2
7	1 1 0 1 q_2

Step	*Machine*
8	1 1 0 1 q_2
9	1 1 0 1 q_2
10	1 1 0 1 q_3

I.1.23 This machine works.

$$\mathcal{P}_{\text{constantthree}} = \{ q_0\mathsf{BB}q_2,\ q_0\mathsf{1B}q_1,\ q_1\mathsf{BR}q_0,\ q_1\mathsf{11}q_0,\ q_2\mathsf{B1}q_2,\ q_2\mathsf{1R}q_3,\ q_3\mathsf{B1}q_3,\ q_3\mathsf{1R}q_4,\ q_4\mathsf{B1}q_4,\ q_4\mathsf{11}q_5,\ q_5\mathsf{BL}q_6,\ q_5\mathsf{1L}q_6,\ q_6\mathsf{BB}q_7,\ q_6\mathsf{1L}q_7 \}$$

Here is the machine acting on an input of 2.

Step	*Machine*
0	1 1 q_0
1	1 q_1
2	1 q_0
3	q_1
4	q_0

Step	*Machine*
5	q_2
6	1 q_2
7	1 q_3
8	1 1 q_3
9	1 1 q_4

Step	*Machine*
10	1 1 1 q_4
11	1 1 1 q_5
12	1 1 1 q_6
13	1 1 1 q_7

I.1.24 This machine computes successor.

$$\mathcal{P}_{\text{successor}} = \{ q_0\mathsf{B1}q_1,\ q_0\mathsf{1L}q_0 \}$$

An example is this tape sequence for input 3.

Step	*Machine*
0	1 1 1 q_0
1	1 1 1 q_0
2	1 1 1 1 q_1

I.1.25

(A) We begin with the head at the start of a sequence of 1's. Erase that first 1, then slide to the end of the sequence, and past a blank. Put two 1's, and then slide back to the start of the initial sequence. Iterate that. The machine has nine instructions and alphabet $\Sigma = \{1, \mathsf{B}\}$.

$$\mathcal{P}_{\text{doubler}} = \{ q_0\mathsf{BB}q_9,\ q_0\mathsf{1B}q_1,\ q_1\mathsf{BR}q_2,\ q_1\mathsf{1R}q_2,\ q_2\mathsf{BR}q_3,\ q_2\mathsf{1R}q_2,\ q_3\mathsf{B1}q_4,\ q_3\mathsf{1R}q_3,\ q_4\mathsf{BB}q_4,\ q_4\mathsf{1R}q_5,\ q_5\mathsf{B1}q_6,\ q_5\mathsf{11}q_6,\ q_6\mathsf{BL}q_7,\ q_6\mathsf{1L}q_6,\ q_7\mathsf{BR}q_8,\ q_7\mathsf{1L}q_7,\ q_8\mathsf{BR}q_9,\ q_8\mathsf{11}q_0 \}$$

Here is the sequence of tapes for the input 2. It shows the blanks because there can be two of them, which can be confusing. It is in halves just to allow a page break in the middle.

Step	*Machine*
0	1 1 q_0
1	B 1 q_1
2	B 1 q_2
3	B 1 B q_2
4	B 1 B B q_3

Step	*Machine*
5	B 1 B 1 q_4
6	B 1 B 1 B q_5
7	B 1 B 1 1 q_6
8	B 1 B 1 1 q_6
9	B 1 B 1 1 q_6

Step	*Machine*
10	B 1 B 1 1 q_7
11	B 1 B 1 1 q_7
12	B 1 B 1 1 q_8
13	B 1 B 1 1 q_0
14	B B B 1 1 q_1

This is the second half.

Step	*Machine*
15	B B B 1 1 q_2
16	B B B 1 1 q_3
17	B B B 1 1 q_3
18	B B B 1 1 B q_3
19	B B B 1 1 1 q_4

Step	*Machine*
20	B B B 1 1 1 B q_5
21	B B B 1 1 1 1 q_6
22	B B B 1 1 1 1 q_6
23	B B B 1 1 1 1 q_6
24	B B B 1 1 1 1 q_6

Step	*Machine*
25	B B B 1 1 1 1 q_6
26	B B B 1 1 1 1 q_7
27	B B B 1 1 1 1 q_8
28	B B B 1 1 1 1 q_9

(b) To double a number represented in binary just append a `0` to the right side. The machine has three instructions and $\Sigma = \{\texttt{0}, \texttt{1}, \texttt{B}\}$.

$$\mathcal{P}_{\text{doublerbinary}} = \{q_0\texttt{B}\texttt{B}q_3,\ q_0\texttt{0}\texttt{0}q_3,\ q_0\texttt{1}\texttt{R}q_1,\ q_1\texttt{B}\texttt{0}q_2,\ q_1\texttt{0}\texttt{R}q_1,\ q_1\texttt{1}\texttt{R}q_1,\ q_2\texttt{B}\texttt{R}q_3,\ q_2\texttt{0}\texttt{L}q_2,\ q_2\texttt{1}\texttt{L}q_2\}$$

On input 6 it gives this tape sequence.

Step	*Machine*
0	1 1 0 q_0
1	1 1 0 q_1
2	1 1 0 q_1
3	1 1 0 q_1

Step	*Machine*
4	1 1 0 0 q_2
5	1 1 0 0 q_2
6	1 1 0 0 q_2
7	1 1 0 0 q_2

Step	*Machine*
8	1 1 0 0 q_2
9	1 1 0 0 q_3

I.1.26 This Turing machine works.

$$\mathcal{P}_{\text{odd}} = \{q_0\texttt{B}\texttt{B}q_{100},\ q_0\texttt{1}\texttt{R}q_1,\ q_1\texttt{B}\texttt{L}q_{100},\ q_1\texttt{1}\texttt{B}q_2,\ q_2\texttt{B}\texttt{L}q_3,\ q_2\texttt{1}\texttt{L}q_3,\ q_3\texttt{B}\texttt{B}q_4,\ q_3\texttt{1}\texttt{B}q_4,\ q_4\texttt{B}\texttt{R}q_5,\ q_4\texttt{1}\texttt{1}q_5,\ q_5\texttt{B}\texttt{R}q_0,\ q_5\texttt{1}\texttt{1}q_0\}$$

For instance, on input 3 it does this

Step	*Machine*
0	1 1 1 q_0
1	1 1 1 q_1
2	1 1 q_2

Step	*Machine*
3	1 1 q_3
4	1 q_4
5	1 q_5

Step	*Machine*
6	1 q_0
7	1 q_1
8	1 q_{100}

and on 4 it does this.

Step	*Machine*
0	1 1 1 1 q_0
1	1 1 1 1 q_1
2	1 1 1 q_2
3	1 1 1 q_3
4	1 1 q_4

Step	*Machine*
5	1 1 q_5
6	1 1 q_0
7	1 1 q_1
8	1 q_2
9	1 q_3

Step	*Machine*
10	q_4
11	q_5
12	q_0
13	q_{100}

I.1.27 While this Turing machine is moving right, if it hits a comma then it replaces it with a 1 and closes up to the left. That is, it scans left to the start of the string, erases the first 1, and then starts moving right again.

$$\mathcal{P}_{\text{addany}} = \{q_0\mathsf{BL}q_{10},\ q_0\mathsf{1R}q_0,\ q_0\mathsf{,1}q_1,\ q_1\mathsf{BR}q_2,\ q_1\mathsf{1L}q_1,\ q_1\mathsf{,L}q_1,\ q_2\mathsf{BR}q_0,\ q_2\mathsf{1B}q_2,\ q_{10}\mathsf{BR}q_{100},\ q_{10}\mathsf{1L}q_{10},\ q_{10}\mathsf{,L}q_{10}\}$$

For instance, on input 11,1,11 it does this.

Step	*Machine*
0	1 1 , 1 , 1 1 q_0
1	1 1 , 1 , 1 1 q_0
2	1 1 , 1 , 1 1 q_0
3	1 1 1 1 , 1 1 q_1
4	1 1 1 1 , 1 1 q_1
5	1 1 1 1 , 1 1 q_1

Step	*Machine*
6	1 1 1 1 , 1 1 q_1
7	1 1 1 1 , 1 1 q_2
8	1 1 1 , 1 1 q_2
9	1 1 1 , 1 1 q_0
10	1 1 1 , 1 1 q_0
11	1 1 1 , 1 1 q_0

Step	*Machine*
12	1 1 1 , 1 1 q_0
13	1 1 1 1 1 1 q_1
14	1 1 1 1 1 1 q_1
15	1 1 1 1 1 1 q_1
16	1 1 1 1 1 1 q_1
17	1 1 1 1 1 1 q_1

(It is split into two sets of tables to give the formatter a place to put a page break.)

Step	*Machine*
18	1 1 1 1 1 1 q_2
19	1 1 1 1 1 q_2
20	1 1 1 1 1 q_0
21	1 1 1 1 1 q_0
22	1 1 1 1 1 q_0

Step	*Machine*
23	1 1 1 1 1 q_0
24	1 1 1 1 1 q_0
25	1 1 1 1 1 q_0
26	1 1 1 1 1 q_{10}
27	1 1 1 1 1 q_{10}

Step	*Machine*
28	1 1 1 1 1 q_{10}
29	1 1 1 1 1 q_{10}
30	1 1 1 1 1 q_{10}
31	1 1 1 1 1 q_{10}
32	1 1 1 1 1 q_{100}

I.1.28 No. Given a configuration $C = \langle q, s, \tau_L, \tau_R \rangle$, one preceding configuration is $\hat{C} = C$ with $\mathcal{I} = q\,s\,s\,q$.

I.1.29

(A) The machine can flip the first bit, slide right and flip the second bit, etc. The fact that we know the number of bits in advance means that we can just have four groups of states: first flip and slide, second flip and slide, etc. (We could do an arbitrary number of bits but it would be a little harder.) At the end we slide left to position the head at the left end of the non-blank characters.

Although the question says that you need not produce a machine, here is one. In this machine, state q_0 flips the first bit and then q_1 slides right. The next flip and slide combination is q_2 and q_3, etc., until the machine gets to the fourth bit. Then it moves the head back to the start with q_7.

$$\begin{aligned}\mathcal{P}_{\text{bitwiseNOT}} = \{\,&q_0\texttt{BB}q_1,\ q_0\texttt{01}q_1,\ q_0\texttt{10}q_1,\ q_1\texttt{BR}q_2,\ q_1\texttt{0R}q_2,\ q_1\texttt{1R}q_2,\ q_2\texttt{BB}q_3,\ q_2\texttt{01}q_3,\ q_2\texttt{10}q_3,\\ &q_3\texttt{BR}q_4,\ q_3\texttt{0R}q_4,\ q_3\texttt{1R}q_4,\ q_4\texttt{BB}q_5,\ q_4\texttt{01}q_5,\ q_4\texttt{10}q_5,\ q_5\texttt{BR}q_6,\ q_5\texttt{0R}q_6,\ q_5\texttt{1R}q_6,\\ &q_6\texttt{BB}q_7,\ q_6\texttt{01}q_7,\ q_6\texttt{10}q_7,\ q_7\texttt{BR}q_8,\ q_7\texttt{0L}q_7,\ q_7\texttt{1L}q_7\,\}\end{aligned}$$

For instance, on input 0110 it does this.

Step	*Machine*
0	0 1 1 0 q_0
1	1 1 1 0 q_1
2	1 1 1 0 q_2
3	1 0 1 0 q_3
4	1 0 1 0 q_4

Step	*Machine*
5	1 0 0 0 q_5
6	1 0 0 0 q_6
7	1 0 0 1 q_7
8	1 0 0 1 q_7

Step	*Machine*
9	1 0 0 1 q_7
10	1 0 0 1 q_7
11	1 0 0 1 q_7
12	1 0 0 1 q_8

(B) The machine has two cases, depending on what it finds for b_3. If $b_3 = \texttt{0}$ then it moves right, copying the next bit into the current one, and then at the end it appends a 0. Likewise, if $b_3 = 1$ then it moves right, copying, and then at the end it appends a 1. As with the prior item, because we know in advance that there are four bits we can just have three groups of states.

(C) One implementation is to expect two four-bit strings, $a_3a_2a_1a_0$ and $b_3b_2b_1b_0$, separated by a blank. The machine reads a_3 and goes into one of two states, r_0 or r_1, depending on whether $a_3 = \texttt{0}$ or $a_3 = 1$. From state r_0 it slides past the blank separator, then reads b_3, and changes it to the logical 'and' of b_3 and 0. Similarly, from state r_1 it slides past the blank separator, reads b_3, and changes it to the logical 'and' of b_3 and 1. Repeat that for a total of four iterations.

I.1.30

(A) Under the convention that the machine starts with its head under the first non-blank character if there are any, $\mathcal{P} = \{q_0\mathtt{BB}q_1,\ q_0\mathtt{1}1q_0\}$ will halt only if the input is blank.
(B) The machine $\mathcal{P} = \{q_0\mathtt{BB}q_0,\ q_0\mathtt{1}1q_1\}$ will fail to halt only if the input is blank.
(C) The machine $\mathcal{P} = \{q_0\mathtt{BR}q_1,\ q_0\mathtt{1R}q_1,\ q_1\mathtt{BB}q_1,\ q_1\mathtt{1}1q_2\}$ will halt if and only if the second character is 1. (Trying the same idea for the first character won't work because of our convention that the first character is blank only if all characters are blank, so there is only one input string with a blank first character.)

I.1.31 For this machine, if the first character is 0 then it puts a 1 in its place, marking that the string is in the language. Otherwise, it puts a 0. Then it blanks the remaining characters, and returns to the first character.

$$\mathcal{P}_{\text{starts with zero}} = \{q_0\mathtt{0}1q_1,\ q_0\mathtt{1}0q_1,\ q_0\mathtt{B}0q_{10},\ q_1\mathtt{0R}q_2,\ q_1\mathtt{1R}q_2,\ q_2\mathtt{0B}q_3,\ q_2\mathtt{1B}q_3,\ q_2\mathtt{BB}q_4,\ q_3\mathtt{BR}q_2,\ q_4\mathtt{BL}q_4,\ q_4\mathtt{0}9q_{10},\ q_4\mathtt{1}1q_{10}\}$$

This is the action of the machine on 01101.

Step	*Machine*
0	0 1 1 0 1 q_0
1	1 1 1 0 1 q_1
2	1 1 1 0 1 q_2
3	1 B 1 0 1 q_3
4	1 B 1 0 1 q_2
5	1 B B 0 1 q_3
6	1 B B 0 1 q_2
7	1 B B B 1 q_3
8	1 B B B 1 q_2
9	1 B B B B q_3
10	1 B B B B B q_2
11	1 B B B B B q_4
12	1 B B B B B q_4
13	1 B B B B B q_4
14	1 B B B B B q_4
15	1 B B B B B q_4
16	1 B B B B B q_4
17	1 B B B B B q_{10}

I.1.32 For this machine, if the first character is not a 0 then it puts a 0 in its place, marking that the string as not in the language. If the first character is 0, then it checks whether the second character is 1. If so, it leaves it alone, otherwise it puts 0, marking that the input is not in the language. Then it blanks the remaining characters, and returns to the single character marking whether the input is a member.

$$\mathcal{P}_{\text{starts with 01}} = \{q_0\mathtt{0B}q_1,\ q_0\mathtt{1}0q_3,\ q_0\mathtt{B}0q_{10},\ q_1\mathtt{BR}q_2,\ q_2\mathtt{0R}q_4,\ q_2\mathtt{1R}q_4,\ q_2\mathtt{B}0q_{10},\ q_3\mathtt{0R}q_4,\ q_4\mathtt{0B}q_5,\ q_4\mathtt{1B}q_5,\ q_4\mathtt{BB}q_6,\ q_5\mathtt{BR}q_4,\ q_6\mathtt{00}q_{10},\ q_6\mathtt{1}1q_{10},\ q_6\mathtt{BL}q_6\}$$

This is the machine on 01101.

Step	*Machine*
0	0 1 1 0 1 q_0
1	B 1 1 0 1 q_1
2	B 1 1 0 1 q_2
3	B 1 1 0 1 q_4
4	B 1 B 0 1 q_5
5	B 1 B 0 1 q_4

Step	*Machine*
6	B 1 B B 1 q_5
7	B 1 B B 1 q_4
8	B 1 B B B q_5
9	B 1 B B B B q_4
10	B 1 B B B B q_6

Step	*Machine*
11	B 1 B B B B q_6
12	B 1 B B B B q_6
13	B 1 B B B B q_6
14	B 1 B B B B q_6
15	B 1 B B B B q_{10}

I.1.33 The main thing with this machine is that there could be zero-many 0's. That is, the machine should accept a string that starts with 1. On the other hand, if the input string is 000, without a 1, then it is not in the language.

With that, the machine reads past initial 0's, if any, blanking them out. If it find a 1 then it leaves it alone, marking that the input is in the language. Then it blanks the remaining characters, and returns to the single character marking whether the input is a member. Otherwise it puts 0, marking that the input is not in the language.

$$\mathcal{P}_{\text{contains 1}} = \{q_0\texttt{0B}q_1,\ q_0\texttt{1R}q_2,\ q_0\texttt{B0}q_{10},\ q_1\texttt{BR}q_0,\ q_2\texttt{0B}q_3,\ q_2\texttt{1B}q_3,\ q_2\texttt{BL}q_4,\ q_3\texttt{BR}q_2,\ q_4\texttt{00}q_{10},\ q_4\texttt{11}q_{10},\ q_4\texttt{BL}q_4\}$$

This is the machine with input 00110.

Step	*Machine*
0	0 0 1 1 0 q_0
1	B 0 1 1 0 q_1
2	B 0 1 1 0 q_0
3	B B 1 1 0 q_1
4	B B 1 1 0 q_0

Step	*Machine*
5	B B 1 1 0 q_2
6	B B 1 B 0 q_3
7	B B 1 B 0 q_2
8	B B 1 B B q_3
9	B B 1 B B B q_2

Step	*Machine*
10	B B 1 B B B q_4
11	B B 1 B B B q_4
12	B B 1 B B B q_4
13	B B 1 B B B q_{10}

I.1.34 The idea of this machine is that it starts with two groups of 1's. The head blanks the first 1 in the first group, then moves over and blanks the last 1 in the second group. At that point the relation of less-than-or-equal exists between the original two groups of 1's if and only if it exists between what remains of the two groups. Iterating the machine ends with one of two cases: the first string is empty (in which case we must blank out what's left of the second string, if any, and print a 1), and the second string is empty but the first is not (in this case we blank out what's left of the first string). The first case is handled by instructions

starting with q_{50} and the second with instructions starting with q_{75}.

$$\begin{aligned}\mathcal{P}_{\text{leq}} = \{\,&q_0\mathsf{BR}q_{50},\ q_0\mathsf{1B}q_1,\ q_1\mathsf{BR}q_2,\ q_1\mathsf{11}q_2,\ q_2\mathsf{BR}q_3,\ q_2\mathsf{1R}q_2,\ q_3\mathsf{BL}q_{75},\ q_3\mathsf{1R}q_4,\ q_4\mathsf{BL}q_5,\ q_4\mathsf{1R}q_4,\\ &q_5\mathsf{BB}q_6,\ q_5\mathsf{1B}q_6,\ q_6\mathsf{BL}q_7,\ q_6\mathsf{11}q_7,\ q_7\mathsf{BL}q_8,\ q_7\mathsf{1L}q_7,\ q_8\mathsf{BR}q_0,\ q_8\mathsf{1L}q_8,\\ &q_{50}\mathsf{BR}q_{51},\ q_{50}\mathsf{11}q_{51},\ q_{51}\mathsf{B1}q_{100},\ q_{51}\mathsf{1B}q_{52},\ q_{52}\mathsf{BR}q_{51},\ q_{52}\mathsf{11}q_{51},\\ &q_{75}\mathsf{BL}q_{76},\ q_{75}\mathsf{11}q_{76},\ q_{76}\mathsf{BL}q_{100},\ q_{76}\mathsf{1B}q_{77},\ q_{77}\mathsf{BL}q_{76},\ q_{77}\mathsf{1L}q_{76}\,\}\end{aligned}$$

For instance, on inputs 2 and 3 it does this.

Step	*Machine*
0	1 1 1 1 1 q_0
1	B 1 1 1 1 q_1
2	B 1 1 1 1 q_2
3	B 1 B 1 1 1 q_2
4	B 1 B 1 1 1 q_3

Step	*Machine*
5	B 1 B 1 1 1 q_4
6	B 1 B 1 1 1 q_4
7	B 1 B 1 1 1 B q_4
8	B 1 B 1 1 1 B q_5
9	B 1 B 1 1 B B q_6

Step	*Machine*
10	B 1 B 1 1 B B q_7
11	B 1 B 1 1 B B q_7
12	B 1 B 1 1 B B q_7
13	B 1 B 1 1 B B q_8
14	B 1 B 1 1 B B q_8

(This tape sequence is split in two to leave a place for a line break.)

Step	*Machine*
15	B 1 B 1 1 B B q_0
16	B B B 1 1 B B q_1
17	B B B 1 1 B B q_2
18	B B B 1 1 B B q_3
19	B B B 1 1 B B q_4
20	B B B 1 1 B B q_4

Step	*Machine*
21	B B B 1 1 B B q_5
22	B B B 1 B B B q_6
23	B B B 1 B B B q_7
24	B B B 1 B B B q_7
25	B B B 1 B B B q_8
26	B B B 1 B B B q_0

Step	*Machine*
27	B B B 1 B B B q_{50}
28	B B B 1 B B B q_{51}
29	B B B B B B B q_{52}
30	B B B B B B B q_{51}
31	B B B B 1 B B q_{100}

I.1.35 The strategy is to blank out the first character, slide to the end, and then blank out the final character (if it matches the first), and then repeat. When the string is empty, the machine puts a 1 on the tape, showing success.

The machine below remembers that the first character was an a by moving to state q_{20}. It remembers a b by moving to state q_{30}. Starting in q_{40} it moves back to the start of the remaining input string. State q_{60} is for the cases where the machine detects that the input was not a palindrome, while state q_{80} is for when the machine finds it is one. Finally, states $q_1, \ldots q_4$ handle the case that the input string (either the initial string or

what is left of the initial string after some end-trimming) has one character.

$$\begin{aligned}\mathcal{P}_{\text{pal}} = \{\, & q_0\mathsf{BB}q_{80},\ q_0\mathsf{a}\mathsf{B}q_1,\ q_0\mathsf{b}\mathsf{B}q_3,\ q_1\mathsf{BR}q_2,\ q_1\mathsf{a}\mathsf{R}q_2,\ q_1\mathsf{b}\mathsf{R}q_2,\ q_2\mathsf{BL}q_{80},\ q_2\mathsf{aa}q_{20},\ q_2\mathsf{bb}q_{20},\\ & q_3\mathsf{BR}q_4,\ q_3\mathsf{a}\mathsf{R}q_4,\ q_3\mathsf{b}\mathsf{R}q_4,\ q_4\mathsf{BL}q_{80},\ q_4\mathsf{aa}q_{30},\ q_4\mathsf{bb}q_{30},\\ & q_{20}\mathsf{BL}q_{21},\ q_{20}\mathsf{a}\mathsf{R}q_{20},\ q_{20}\mathsf{b}\mathsf{R}q_{20},\ q_{21}\mathsf{BB}q_{60},\ q_{21}\mathsf{a}\mathsf{B}q_{40},\ q_{21}\mathsf{bb}q_{60},\\ & q_{30}\mathsf{BL}q_{31},\ q_{30}\mathsf{a}\mathsf{R}q_{30},\ q_{30}\mathsf{b}\mathsf{R}q_{30},\ q_{31}\mathsf{BB}q_{60},\ q_{31}\mathsf{aa}q_{60},\ q_{31}\mathsf{b}\mathsf{B}q_{40},\\ & q_{40}\mathsf{BL}q_{41},\ q_{40}\mathsf{a}\mathsf{L}q_{41},\ q_{40}\mathsf{b}\mathsf{L}q_{41},\ q_{41}\mathsf{BR}q_0,\ q_{41}\mathsf{a}\mathsf{L}q_{41},\ q_{41}\mathsf{b}\mathsf{L}q_{41},\\ & q_{60}\mathsf{BB}q_{100},\ q_{60}\mathsf{a}\mathsf{B}q_{61},\ q_{60}\mathsf{b}\mathsf{B}q_{61},\ q_{61}\mathsf{BL}q_{60},\ q_{61}\mathsf{aa}q_{60},\ q_{61}\mathsf{bb}q_{60},\\ & q_{80}\mathsf{B1}q_{100},\ q_{80}\mathsf{a1}q_{100},\ q_{80}\mathsf{b1}q_{100}\,\}\end{aligned}$$

As an example, this is the action of the machine on abba.

Step	*Machine*
0	a b b a q_0
1	b b a q_1
2	b b a q_2
3	b b a q_{20}
4	b b a q_{20}
5	b b a q_{20}
6	b b a q_{20}
7	b b a q_{21}
8	b b q_{40}
9	b b q_{41}
10	b b q_{41}
11	b b q_{41}

(The tape sequence is in two parts to leave room for a page break.)

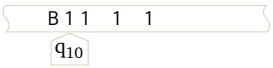
Step	*Machine*
12	b b q_0
13	b q_3
14	b q_4
15	b q_{30}
16	b q_{30}
17	b q_{31}
18	 q_{40}
19	 q_{41}
20	 q_0
21	 q_{80}
22	1 q_{100}

A second example is the action of the machine on aba.

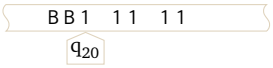
Step	*Machine*
0	a b a q_0
1	b a q_1
2	b a q_2
3	b a q_{20}
4	b a q_{20}
5	b a q_{20}
6	b a q_{21}
7	b q_{40}
8	b q_{41}
9	b q_{41}
10	b q_0
11	 q_3
12	 q_4
13	 q_{80}
14	1 q_{100}

I.1.36 The strategy is to make two copies of the starting block of 1's, then move the head to the start of the first of the two copies. This is an example initial configuration.

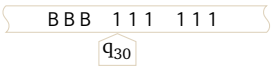
The machine strips the leading 1, moves right, puts in the 1 that will start the first copy block, and then puts in the 1 to start the second block. With that, the head moves back to the start.



Now comes the complication. The machine strips the leading 1, then moves right and puts in a 1 at the end of the first copy block. But the place where it must put this 1 is the blank separating the two copy blocks. So it must then move right and put in a blank before moving to the end of the second copy block. At that end it puts two 1's, the first to mark the stripped leading 1 and the second to mark the inserted blank. With that, the head moves back to the start.



The machine iterates the prior sequence of steps until it finds that the leading sequence of 1's is now blank. To finish, it moves the head to the start of the first copy block.

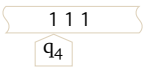


I.1.37 Let the machine have the same configuration at two different steps $C(t) = C(\hat{t})$ with $t < \hat{t}$. By determinism, $C(t+1) = C(\hat{t}+1)$. Likewise, $C(t+i) = C(\hat{t}+i)$ for $i = 2, \ldots$ This means that the machine cycles every $\hat{t} - t$ steps, because $i = \hat{t} - t$ gives that $C(t + (\hat{t} - t)) = C(\hat{t} + (\hat{t} - t))$ but $C(t + (\hat{t} - t)) = C(\hat{t})$.

This sufficient condition is not necessary, since we can make a machine that never repeats its configuration but also never halts. One way is to make a machine that writes a 1 to the tape, then moves right, and then repeats.

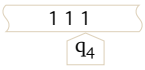
I.1.38 Consider this configuration, with the machine in state q_5.

It could have arrived at that configuration after this configuration and instruction.



$q_4 1 \mathrm{R} q_5$

or this



$q_4 1 \mathrm{L} q_5$

or for that matter, this.

$q_4 1 1 q_5$

I.2.2 Church's Thesis is an assertion that 'mechanically computable' (by a discrete and deterministic machine) is equivalent to 'computable by a Turing machine'. It is not a theorem because we cannot derive it from the traditional axioms for mathematics.

In some ways it is more like a definition than a theorem. But it is not a usual definition, where as a convenience we take a word so that we don't have to keep saying a phrase over and over, as where we define 'bachelor' as 'unmarried man' or 'linearly dependent' as '$x_0 = a_1 x_1 + \cdots a_n x_n$'. Rather, it is more like Calculus's definition of the derivative in that it expresses the introduction of a fundamental concept.

I.2.3

(A) As with a program, a Turing Machine can implement an algorithm. But an algorithm may be implemented by more than one Turing Machine. For instance, we can do a shell sort of a list of five numbers with two different Turing Machines, if only by renaming some of the states. In the words of M Davis (Davis 2016), "[Church's Thesis] says nothing about what algorithms ARE. It is about what algorithms can and can not DO." *Comment:* there is no widely agreed-upon definition of algorithm, which is odd as algorithms are a central study in Computer Science.

(B) A Turing Machine is a single-purpose device. For instance, we may write a Turing Machine to multiply two of its inputs. But today we use "computer" to mean a general-purpose, that is, programmable, device. *Comment:* we will later see that there are general-purpose Turing Machines, so the distinction is blurry, one of connotation rather than of precise definition.

I.2.4 Anything that can be done on the modern computers that we have in our pockets can be done, in principle, on an old-timey-style mechanical device. For more detail, see Extra B; the low level construction details covered there can be done with gears and levers. So the phrase 'mechanically computable' is not about the exact implementation, it covers any way of computing that is discrete and deterministic, and physically realizable.

I.2.5

(A) The tape, as given in the definition, is not infinite. It is unbounded.

For an analogy, consider the dictionary definition of a book. In that definition there is no upper limit on the number of pages. But no book, no actual bound collection of pages, has infinitely many. So also, no halting Turing machine computation uses infinitely much tape.

While it is true that you can write a Turing machine that does not halt and that just keeps using more and more tape, at no step of the calculation is the amount of tape infinite. It never runs out — it is unbounded — but it is never infinite.

(B) It is a model. A model works by discarding some aspects of the situation that it models. See (Wikipedia contributors 2016). (Besides, whether the universe is finite — either just currently finite, or finite into any unbounded future — is not clear. In any event it is not a mathematical question.)

I.2.6 Church's Thesis is the third.

The first is mistaken because Church's Thesis speaks only to discrete and deterministic mechanisms. The second is mistaken because Church's Thesis does not speak to the capabilities, or limitations, of human computers. There are people who posit that humans can do more than mechanisms can do, and people who speculate that there may be machines that that can do more than people can, even in principle, but Church's Thesis speaks to an an equivalence between what can be done by *any* discrete and deterministic mechanism and what can be done by this one kind, Turing machines. The last misses the mark for much the same reason as the second.

I.2.7 (From (Andreas Blass 2015).) The first benefit that we get from this thesis is that it lets us connect formal mathematical theorems to real-world issues of computability. For example, the theorem that the word problem for groups is Turing-undecidable has the real-world interpretation that no algorithm can solve all instance of the word problem.

The second benefit is in mathematics itself, specifically in computability theory. Published proofs that something is Turing-computable almost never proceed by exhibiting a Turing-machine program, or indeed a program in any actual computing language. Sometimes, if the matter is simple enough, they provide some sort of pseudo-code. Most often, though, they merely give an informal description of an algorithm. It is left to the reader to see that this actually does give an algorithm (in the intuitive sense) and therefore, by the Church-Turing thesis, could be simulated by a Turing machine. The usual situation is that, although experts in Turing-machine programming would (if they existed) be able to routinely convert the intuitive algorithm into a Turing machine program, the program would be too large and complicated to be worth writing down.

I.2.8 Lance Fortnow says it perfectly, "Computation is about process, about the transitions made from one state of the machine to another. Computation is not about the input and the output, point A and point B, but the journey. . . . You can feed a Turing machine an infinite digits of a real number (Siegelmann), have computers interact with each other (Wegner-Goldin), or have a computer that perform an infinite series of tasks (Denning) but in all these cases the process remains the same, each step following Turing's model."

(Fortnow 2010).

I.2.9

(A) Yes, this is clear. To compute $f \circ g\,(x)$ we compute $y = g(x)$ and then compute $f(y)$.

(B) The composition $f \circ g$ may be computable, or may be not computable. On the one hand, the constant function $g(x) = 0$ is computable, and the composition $f \circ g\,(x)$ equals $f(0)$. Because $f(0)$ is fixed, not variable, this result is computable.

On the other hand, if $g(x) = x$ then the composition is $f \circ g\,(x) = f(x)$ is not computable.

I.2.10 We can simulate a ternary machine using an ordinary programming language, that is, on a binary machine. We can simulate a binary machine on a Turing Machine. Thus we can simulate a ternary machine on a Turing machine. So, anything that can be computed on a ternary machine can be computed on a Turing machine.

The converse is even easier: to simulate a binary machine on a ternary machine just pick a trit and avoid using it.

I.2.11 The role of Church's Thesis here is that it allows us to avoid exhibiting Turing machines as sets of four-tuples to do these computations. Instead we can describe how to do them inside a usual programming environment.

(A) Let f_0 be computed by the Turing machine $\mathcal{P}_0$ and let f_1 be computed by TM_1. For $x \in \mathbb{N}$ this will compute $h(x)$: run $\mathcal{P}_0$ on input x for one step, then $\mathcal{P}_1$ on x for a step, then $\mathcal{P}_0$ for another step, etc., until each halts. When that happens, if it ever does, output 1.

(B) As with the prior item, for $x \in \mathbb{N}$ compute the function on input x by: run $\mathcal{P}_0$ on x for one step, then $\mathcal{P}_1$ on x for a step, then $\mathcal{P}_0$ for another step, etc. The difference here is that if either halts then output 1.

(C) Suppose f is computed by $\mathcal{P}$. Fix $x \in \mathbb{N}$. Run $\mathcal{P}$ on input 0 for a step. Then run $\mathcal{P}$ on input 0 for an additional step and run $\mathcal{P}$ on 1 for a step. After that, run $\mathcal{P}$ on input 0 for another step, run $\mathcal{P}$ on 1 for another step, and start a run of $\mathcal{P}$ on input 2. In this way we cycle among the machine running on different inputs.

Whenever a computation of $\mathcal{P}$ on some input i halts, check whether what is left on its tape equals x. If so, output 1.

(D) Let f_0 be computed by the Turing machine $\mathcal{P}_0$ and let f_1 be computed by TM_1. For $x \in \mathbb{N}$, run $\mathcal{P}_0$ on input x until it halts. Suppose the output is y. Run $\mathcal{P}_1$ on y until it halts, and then the output is $h(x)$. Since each function f_0 and f_1 was given as total, that output will appear eventually, and so h is also total.

(E) This is the same as the prior item except that the function h will be partial since there is some input for which $\mathcal{P}_0$ does not halt.

I.2.12 Here is a procedure that is intuitively mechanically computable, and so by Church's Thesis is computable. Let the input be n. For $i = 0$ compute $f(i)$ and see if it equals n. If so, output 0. If not we iterate: take $i = 1$ and compute $f(1)$. If so, output 0. Otherwise, repeat until the procedure outputs some value; if it never does then we are in the second case of h's specification.

I.2.13 This procedure is intuitively mechanically computable and so by Church's Thesis is computable. Let the input be n. Run the Turing machine computing f on input n until it halts, if it ever does. If so then run the Turing machine computing $g(n)$ until it halts, if ever. After that, if after that ever comes, output 1. Otherwise we are in the second case of h's specification.

I.2.14 If there is a root then this procedure will find it eventually. If there is no root then this procedure will just run forever, in an unbounded search.

I.2.15 First, looping and failing to halt are not the same thing — looping forever is one way to fail to halt but not the only way. Putting that aside, one way for a multitape machine to fail to halt is to ignore all of the tapes but the first, and fail to halt in the way that a single tape machine fails to halt. (There are other ways for a multitape machine to fail to halt, but this shows that it is not harder to make such a machine halt.)

I.2.16 The strategy is to move right on tape 0, writing the complementary bit onto tape 1 until the tape 0 head hits a blank. Then the machine moves left on tape 1. (Below, rather than write the pair of characters as $\langle \mathtt{x}, \mathtt{y} \rangle$ we just have $\mathtt{xy}$.)

Δ	00	01	0B	10	11	1B	B0	B1	BB
q_0	00, q_1	01, q_1	01, q_1	10, q_1	11, q_1	10, q_1	B0, q_1	B1, q_1	LL, q_2
q_1	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	LL, q_2
q_2	0L, q_2	0L, q_2	0R, q_3	1L, q_2	1L, q_2	1R, q_3	BL, q_2	BL, q_2	BR, q_3

(Many of the input pairs cannot happen. For example, if the machine is in state q_0 then the input 00 cannot happen.)

I.2.17 The strategy is to move right on the two tapes, writing the logical and onto tape 1 until on both tapes the heads hit blanks. Then the machine moves left on tape 1. (Below, instead of writing the character pair as $\langle$x, y$\rangle$, we just write xy.)

Δ	00	01	0B	10	11	1B	B0	B1	BB
q_0	00, q_1	00, q_1	0B, q_1	10, q_1	11, q_1	1B, q_1	BB, q_1	BB, q_1	LL, q_2
q_1	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	LL, q_2
q_2	LL, q_2	LL, q_2	0R, q_3	LL, q_2	LL, q_2	1R, q_3	BL, q_2	BL, q_2	BR, q_3

(Many of these input pairs cannot happen. For example, if the machine is in state q_1 then the input B0 cannot happen.)

I.2.18 Perhaps predictably, we end with nothing since there is no bill still in our possession.

I.3.10 'Primitive recursive' is a collection of functions, while 'primitive recursion' is a way to combine functions.

I.3.11 There the definition gives 1 for all y, including $y = 0$.

I.3.12

(A) $f(0) = 0$, $f(1) = f(2 \cdot 1 - 2) = f(0) = 0$

(B) Expanding the definition gives $f(2) = f(2 \cdot 2 - 2) = f(2)$, leaving the value undefined.

I.3.13

(A) Following the definition gives this; for instance, $F(1) = F(1 - 1) = F(0) = 42$.

n	0	1	2	3	4	5	6	7	8	9	10
$t(n)$	42	42	42	42	42	42	42	42	42	42	42

(B) This

$$F(y) = \begin{cases} g(\,) & \text{– if } y = 0 \\ h(F(z), z) & \text{– if } y = \mathcal{S}(z) \end{cases}$$

works where g is the no-input constant function $g(\,) = 42$ and $h(a, b) = a$.

By the way, since F is constant we can construct it as a primitive recursive function without using the schema of primitive recursion. Use composition: $F(y) = \mathcal{S}(\mathcal{S}(\ldots Z(y))) = \mathcal{S}^{42} \circ Z\,(y)$.

I.3.14 One way is to note that it is the composition plus_three $= \mathcal{S} \circ (\mathcal{S} \circ \mathcal{S})\,(x)$.

I.3.15 Starting with this

$$\text{is\_zero}(y) = \begin{cases} 1 & \text{– if } y = 0 \\ 0 & \text{– if } y = \mathcal{S}(z) \end{cases}$$

we take g to be the zero input function composition $g(\,) = \mathcal{S}(Z(\,))$ and take h to be the two-input zero function $h(a, b) = Z(a, b) = 0$.

I.3.16

(A) These are the first eleven values; obviously this is the sequence of squares.

n	0	1	2	3	4	5	6	7	8	9	10
$t(n)$	0	1	4	9	16	25	36	49	64	81	100

(B) Because s is a function of one input, the bookkeeping says that we need to describe it in this way.

$$s(y) = \begin{cases} g(\,) & \text{– if } y = 0 \\ h(s(z), z) & \text{– if } y = \mathcal{S}(z) \end{cases}$$

So g is a function of no arguments and is therefore constant, $g(\,) = Z(\,) = 0$. For h we want $h(s(z), z) = s(z) + (2 \cdot (z+1) - 1) = s(z) + (2z + 1)$. So we take $h(a, b) = \text{plus}(a, \mathcal{S}(\text{product}(\mathcal{S}(\mathcal{S}(Z(\,))), b)))$.

I.3.17

(A) Here are the first eleven triangular numbers.

n	0	1	2	3	4	5	6	7	8	9	10
$t(n)$	0	1	3	6	10	15	21	28	36	45	55

(B) Because t is a function of one input, the bookkeeping is that we need to describe it as here.

$$t(y) = \begin{cases} 0 & \text{– if } y = 0 \\ h(t(z), z) & \text{– if } y = \mathcal{S}(z) \end{cases}$$

So g is a function of no arguments and therefore is a constant function, $g(\,) = Z(\,) = 0$. As to h, observe that for $n > 0$ we have $t(n) = t(n-1) + n$, because there is one dot in row 1, two dots in row 2, etc. So we want $h(t(z), z) = t(z) + (z + 1)$ and putting that into the format required gives $h(a, b) = \text{plus}(a, \mathcal{S}(b))$.

I.3.18

(A) These are the first six values, suggesting that $d(n) = n^3$.

n	0	1	2	3	4	5
$d(n)$	0	1	8	27	64	125

(B) The given d is a function of one input so the arity bookkeeping has g be a function of no inputs and h be a function of two.

$$d(y) = \begin{cases} g(\,) & \text{– if } y = 0 \\ h(d(z), z) & \text{– if } y = \mathcal{S}(z) \end{cases}$$

Take g to be the constant $g(\,) = Z(\,) = 0$. To get $h(d(z), z) = d(z) + 3(z+1)^2 + 3(z+1) + 1$ only requires some (perhaps a little tedious) translation into low-level functions. Write $d(z) + 3(z+1)^2 + 3(z+1) + 1$ as $s + t + u + v$. Then $h(a, b) = \text{plus}(s, \text{plus}(t, \text{plus}(u, v)))$ where $s(a, b) = a$, and $u(a, b) = \text{product}(\mathcal{S}(\mathcal{S}(\mathcal{S}(Z(\,)))), \text{plus}(b, \mathcal{S}(Z(\,))))$, and $v(a, b) = \mathcal{S}(Z(\,))$, and $t(a, b)$ is this.

$$\text{product}(\mathcal{S}(\mathcal{S}(\mathcal{S}(Z(\,)))), \text{product}(\text{plus}(b, \mathcal{S}(Z(\,))), \text{plus}(b, \mathcal{S}(Z(\,)))))$$

I.3.19

(A) Here are the ten values.

n	1	2	3	4	5	6	7	8	9	10
$h(n)$	1	3	7	15	31	63	127	255	511	1023

(B) This is a function of one input so the arity bookkeeping gives that we want this.

$$H(y) = \begin{cases} g(\,) & \text{– if } y = 0 \\ h(H(z), z) & \text{– if } y = \mathcal{S}(z) \end{cases}$$

Take g to be the constant function $g(\,) = 1 = \mathcal{S}(Z(\,))$. To get $h(H(z), z) = 2 \cdot H(z) + 1$ take $h(a, b) = \mathcal{S}(\text{product}(a, \mathcal{S}(\mathcal{S}(Z(\,)))))$.

I.3.20 The derivation for factorial is this.

$$\text{fact}(y) = \begin{cases} g(\,) & \text{if } y = 0 \\ h(\text{fact}(z), \mathcal{S}(z)) & \text{if } y = \mathcal{S}(z) \end{cases}$$

Here g is a function of no inputs $g(\,) = 1 = \mathcal{S}(Z(\,))$ and h is a function of two variables $h(a, b) = \text{product}(a, \mathcal{S}(b))$.

I.3.21

(A) Assume both are greater than zero. We know what to do if $a > b > 0$. For the $a < b$ case we can switch the order of the two, $\gcd(a, b) = \gcd(b, a)$, because for both sides of the equation we just find all of the common divisors and take the maximum. And for $a = b$, the greatest common divisor is a.

$$\gcd(a, b) = \begin{cases} 0 & \text{– if } a = b = 0 \\ a & \text{– if } b = 0 \text{ and } a \neq 0 \\ b & \text{– if } a = 0 \text{ and } b \neq 0 \\ a & \text{– if } a = b \\ \gcd(b, a) & \text{– if } a < b \\ \gcd(a - b, b) & \text{– if } a > b \end{cases}$$

For fun we can write a program (it collapses some of the above cases).

```
(define (gcd-first a b)
  (cond
    [(zero? a) b]
    [(zero? b) a]
    [(> b a) (gcd-first b a)]
    [else (gcd-first (- a b) b)]))
```

(B) These are easy to compute by hand but here they came from the program

```
> (gcd-first 28 12)
4
> (gcd-first 104 20)
4
> (gcd-first 300009 25)
1
```

(C) Again, for fun we did this as Racket script.

```
(define (gcd-second a b)
  (if (zero? b)
      a
      (gcd-second b (remainder a b))))
```

Here is the result.

```
> (gcd-second 28 12)
4
> (gcd-second 104 20)
4
> (gcd-second 300009 25)
1
```

I.3.22 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$.

(A) The constant function that always returns zero, $\mathcal{C}_0(\vec{x})$, is included the definition of primitive recursion, Definition 3.8, as $\mathcal{Z}(\vec{x}) = 0$. The function that always returns 1 is then $\mathcal{C}_1(\vec{x}) = \mathcal{S}(Z(\vec{x}))$, the function that returns 2 is $\mathcal{C}_2(\vec{x}) = \mathcal{S}(\mathcal{S}(Z(\vec{x})))$, etc. Because successor, zero, and composition are all permitted in the construction of a primitive recursive function, each $\mathcal{C}_k$ is primitive recursive.

(B) Use proper subtraction to define $\max(\{x, y\}) = y + (x \dot{-} y)$. If $x \geq y$ then it reduces to $\max(\{x, y\}) = y + (x - y) = x$, while if $x < y$ then it is $\max(\{x, y\}) = y + 0 = y$. The parts of the right-hand side of $\max(\{x, y\}) = y + (x \dot{-} y)$ are all primitive recursive, so the max function is also primitive recursive. Similarly $\min(\{x, y\}) = x + (y \dot{-} \max(\{x, y\}))$.

(C) Let $\text{absdiff}(x, y) = (x \dot{-} y) + (y \dot{-} x)$. If $x \geq y$ then only the second term is zero and $\text{absdiff}(x, y) = x - y$, as desired. The $y > x$ case is similar. Since this expression is made from primitive recursive parts put together using primitive recursive combiners, $\text{absdiff}(x, y)$ is therefore primitive recursive.

I.3.23 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners, and therefore the function is primitive recursive.

(A) This primitive recursion defines sign.

$$\text{sgn}(y) = \begin{cases} 0 & \text{– if } y = 0 \\ 1 & \text{– if } y = \mathcal{S}(z) \end{cases}$$

Note that in the first clause we write 0 as an abbreviation for $Z(y)$, and we write 1 to abbreviate $\mathcal{S}(Z(y))$.

(B) A primitive recursion will do.

$$\text{negsign}(y) = \begin{cases} 1 & \text{– if } y = 0 \\ 0 & \text{– if } y = \mathcal{S}(z) \end{cases}$$

But also note that $\text{negsign}(y) = 1 - \text{sgn}(y)$, that is, $\text{negsign}(y) = 1 \dot{-} \text{sgn}(y)$.

(C) The first is $\text{lessthan}(x, y) = \text{sgn}(y \dot{-} x)$ while the other is $\text{greaterthan}(x, y) = \text{sgn}(x \dot{-} y)$.

I.3.24 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners and therefore the function is primitive recursive.

(A) We have $\text{not}(x) = 1 \dot{-} x$. For the two-input ones, $\text{and}(x, y) = \text{sgn}(x) \cdot \text{sgn}(y) = \text{product}(\text{sgn}(x), \text{sgn}(y))$ and $\text{or}(x, y) = \text{sgn}(x + y)$ (an alternative is $\text{or}(x, y) = 1 \dot{-} ((1 \dot{-} x) \cdot (1 \dot{-} y))$).

(B) $\text{equal}(x, y) = \text{negsign}(\text{lessthan}(x, y) + \text{greaterthan}(x, y))$

I.3.25 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners, and therefore the function is primitive recursive.

(A) $\text{notequal}(x, y) = \text{sign}(\text{lessthan}(x, y) + \text{greaterthan}(x, y))$

(B) The first is $m(x) = 7 \cdot \text{equal}(x, 1) + 9 \cdot \text{equal}(x, 5) + 2 \cdot (\text{and}(\text{notequal}(x, 1)), \text{notequal}(x, 5))$. The second is similar, $n(x, y) = 7 \cdot \text{equal}(x, 1) \cdot \text{equal}(y, 2) + 9 \cdot \text{equal}(x, 5) \cdot \text{equal}(y, 5)$.

I.3.26 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners and therefore the function is primitive recursive.

(A) Since g is given as primitive recursive, the function $h(a, b) = a + g(b)$ is also primitive recursive. With that, this primitive recursion will do the bounded sum.

$$f(y) = \begin{cases} 0 & \text{– if } y = 0 \\ h(f(z), z) & \text{– if } y = \mathcal{S}(z) \end{cases}$$

An example is called for. We will expand $f(4)$.

$$\begin{aligned} f(4) &= h(f(3), 3) = f(3) + g(3) \\ &= h(f(2), 2) + g(3) = f(2) + g(2) + g(3) \\ &= h(f(1), 1) + g(2) + g(3) = f(1) + g(1) + g(2) + g(3) \\ &= h(f(0), 0) + g(1) + g(2) + g(3) = f(0) + g(0) + g(1) + g(2) + g(3) = g(0) + \cdots + g(4) \end{aligned}$$

(B) This is similar to bounded sum.

(C) Consider the bounded product function.

$$P_p(\vec{x}, m) = p(\vec{x}, 0) \cdot p(\vec{x}, 1) \cdots p(\vec{x}, m - 1) = \prod_{0 \leq i < m} p(\vec{x}, i)$$

There are two things to note here. First, if $P_p(\vec{x}, m) = 1$ then $P_p(\vec{x}, i) = 1$ for all $i < m$. Second, if there is an m where $P_p(\vec{x}, m) = 0$ then $P_p(\vec{x}, n) = 0$ for all $m \leq n$. Thus the sequence $P_p(\vec{x}, 0)$, $P_p(\vec{x}, 1)$, $P_p(\vec{x}, 2)$, ... consists of a number of 1's followed by all 0's. The initial 1's go until the first n for which the predicate $p(\vec{x}, n)$ gives 0.

Next consider the bounded sum function.

$$S_{P_p}(\vec{x}, m) = \sum_{0 \leq i < m} P_p(\vec{x}, m)$$

The prior paragraph gives that $S_{P_p}(\vec{x}, m)$ is the least number n with $n < m$ such that $p(\vec{x}, n) = 0$.

Now define $M(\vec{x}, m)$ by cases.

$$M(\vec{x}, m) = \begin{cases} m & \text{– if } S_{P_p}(\vec{x}, m) = 0 \\ S_{P_p}(\vec{x}, m) & \text{– otherwise} \end{cases}$$

I.3.27 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners and therefore the function is primitive recursive.

(A) We can define U with a primitive recursion.

$$U(\vec{x}, m) = \begin{cases} 1 & \text{– if } m = 0 \\ p(\vec{x}, z) \cdot U(\vec{x}, z) & \text{– if } m = \mathcal{S}(z) \end{cases}$$

For instance, here is the computation of $U(\vec{x}, 3)$.

$$\begin{aligned} U(\vec{x}, 3) &= p(\vec{x}, 2) \cdot U(\vec{x}, 2) \\ &= p(\vec{x}, 2) \cdot p(\vec{x}, 1) \cdot U(\vec{x}, 1) \\ &= p(\vec{x}, 2) \cdot p(\vec{x}, 1) \cdot p(\vec{x}, 0) \cdot U(\vec{x}, 0) \\ &= p(\vec{x}, 2) \cdot p(\vec{x}, 1) \cdot p(\vec{x}, 0) \cdot 1 \end{aligned}$$

(B) First an example with $m = 3$. Consider this.

$$\hat{E}(\vec{x}, 3) = 1 \dot{-} \left[(1 \dot{-} p(\vec{x}, 2)) \cdot (1 \dot{-} p(\vec{x}, 1)) \cdot (1 \dot{-} p(\vec{x}, 0)) \right]$$

If any $p(\vec{x}, i)$ is 1 then the entire expression in square brackets is 0, and so $\hat{E}$ is 1. If all of the $p(\vec{x}, i)$ are 0 then the expression in square brackets is 1 and so $\hat{E}$ is 0.

Consequently, take $E(\vec{x}, m) = 1 \dot{-} \hat{E}(\vec{x}, m)$ where the latter function is below.

$$\hat{E}(\vec{x}, m) = \begin{cases} 0 & \text{– if } m = 0 \\ (1 \dot{-} p(\vec{x}, z)) \cdot \hat{E}(\vec{x}, z) & \text{– if } m = \mathcal{S}(z) \end{cases}$$

(C) This follows straight from the definition on noting that one suitable bound on the quotient k is the dividend y, that is, $\text{divides}(x, y) = \exists d \leq y \, [y = x \cdot d]$.

(D) As in the prior item, we need only find a suitable bound. Here, $\text{prime}(y) = 1$ if and only if both $1 < y$ and there is no $x < y$ with $x > 1$ and $\text{divides}(x, y)$, We have verified that all of those components are primitive recursive.

I.3.28 These answers use familiar notation. For instance, we write $x + y$ instead of $\text{plus}(x, y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners and therefore the function is primitive recursive.

(A) The table is straightforward.

a	0	1	2	3	4	5	6	7
$\text{rem}(a, 3)$	0	1	2	0	1	2	0	1

(B) The relationship is not true when $\text{rem}(a,3)=2$.

(C) This follows from the prior item: item (1) is 0, item (2) is also 0, and item (3) is $\text{rem}(z,3)+1$.

$$\text{rem}(a,3)=\begin{cases}0 & \text{– if } a=0\\ 0 & \text{– if } a=S(z) \text{ and } \text{rem}(z,3)+1=3\\ \text{rem}(z,3)+1 & \text{– if } a=S(z) \text{ and } \text{rem}(z,3)+1\neq 3\end{cases}$$

(D) Here is the explicit definition given in the form for primitive recursion.

$$\text{rem}(a,3)=\begin{cases}0 & \text{– if } a=0\\ \text{product}(\mathcal{S}(\text{rem}(z,3)),\ \text{notequal}(\mathcal{S}(\text{rem}(z,3)),3) & \text{– if } a=\mathcal{S}(z)\end{cases}$$

(E) Change the 3's to b's.

$$\text{rem}(a,b)=\begin{cases}0 & \text{– if } a=0\\ \text{product}(\mathcal{S}(\text{rem}(z,b)),\ \text{notequal}(\mathcal{S}(\text{rem}(z,b)),b) & \text{– if } a=\mathcal{S}(z)\end{cases}$$

I.3.29 These answers use familiar notation. For instance, we write $x+y$ instead of $\text{plus}(x,y)$. We will also omit explicit mention that the expressions are made from primitive recursive parts put together using primitive recursive combiners and therefore the function is primitive recursive.

(A) The division is easy.

a	0	1	2	3	4	5	6	7	8	9	10
$\text{div}(a,3)$	0	0	0	1	1	1	2	2	2	3	3

(B) One way to say it is that $\text{div}(a+1,3)=\text{div}(a,3)$ except when $\text{rem}(a+1,3)=0$.

(C) Based on the prior item, this is the explicit definition given in the form for primitive recursion (again, the order of the two inputs is not important).

$$\text{div}(a,3)=\begin{cases}0 & \text{– if } a=0\\ \text{plus}(\text{div}(z,3),\ \text{equal}(\text{rem}(\mathcal{S}(z),3),0)) & \text{– if } a=\mathcal{S}(z)\end{cases}$$

(D) Replace the 3's with b's.

$$\text{div}(a,b)=\begin{cases}0 & \text{– if } a=0\\ \text{plus}(\text{div}(z,b),\ \text{equal}(\text{rem}(\mathcal{S}(z),b),0)) & \text{– if } a=\mathcal{S}(z)\end{cases}$$

I.3.30 This starts with bounded minimization.

$$f(x,y)=\lfloor x/y\rfloor=\min_{t\leq x}[(t+1)\cdot y>x]$$

For instance, $f(9,2)=\lfloor 9/2\rfloor=\min_{t\leq 9}[(t+1)\cdot 2>9]$ gives $t+1=5$, so $t=4$. Another example is that $f(8,2)=\lfloor 8/2\rfloor=\min_{t\leq 8}[(t+1)\cdot 2>9]$ also gives $t+1=$ and so $t=4$. Putting it in terms of previously defined functions, for instance changing the multiplication dot to a call to the product function, is routine.

I.3.31

(A) $G(\langle 3,1\rangle)=2^4\cdot 3^2=144$, $G(\langle 2,2,2\rangle)=2^3\cdot 3^3\cdot 5^3=27\,000$.

(B) The first two are $A_0=\langle 3,2,1\rangle$ and $A_1=\langle 1,0\rangle$. For the third, the prime factorization is $343=7^3$. Since there is no factor of 2 (or 3 or 5), there is no such sequence A_2.

(C) If it did not use successor then $A_0\langle 1,0\rangle$ and $A_1=\langle 1\rangle$ would have the same encoding.

(D) The first is easy, $\bar{F}(0)=G(\langle\,\rangle)=1$. Because $F(0)=1$ we have $\bar{F}(1)=G(\langle\langle\rangle F(0)\rangle)=2^2=4$. Then $F(1)=1$ so $\bar{F}(2)=G(\langle\langle\rangle F(0),F(1)\rangle)=2^2\cdot 3^2=36$. Finally, $F(2)=2$ so $\bar{F}(3)=G(\langle\langle\rangle F(0),F(1),F(2)\rangle)=2^2\cdot 3^2\cdot 5^3=4500$.

I.3.32 This is an implementation.

```
(define (M x)
   (if (<= x 100)
       (M (M (+ x 11)))
       (- x 10)))
```

(A) The output values are that $M(x) = 91$ if $x \in \{0, \ldots 101\}$, and otherwise $M(x) = x - 10$.
(B) That this function is primitive recursive follows from Exercise 3.25 and Exercise 3.22.

I.3.33 Every initial function in Definition 3.8 is total. Every primitive recursive function is derived from the initial functions using a finite number of applications of function composition and primitive recursion. Both of these combiners yield total function outputs from total function inputs. (Technically, the last two sentences use induction.)

I.4.10 A total recursive function is any computable function that converges on all inputs, that is, it is computed by a machine that halts on all inputs. A primitive recursive function is one that can be derived with the schema in Definition 3.8.

Every primitive recursive function is a total recursive function. The converse does not hold — there are total recursive functions that are not primitive recursive, such as Ackermann's.

I.4.11 The value $\mathcal{H}_4(2,0) = 1$ is immediate from the definition. Next, $\mathcal{H}_4(2,1) = \mathcal{H}_3(2, \mathcal{H}_4(2,0))$, giving $\mathcal{H}_2(2, \mathcal{H}_3(2,0)) = \mathcal{H}_1(2, \mathcal{H}_2(2,0)) = 2$.

The last three are similar but tedious so a script is the best way to proceed. This is a copy of the straightforward transcription of the definition of $\mathcal{H}$ given in the section body.

```
(define (H n x y)
  (cond
      [(= n 0) (+ y 1)]
      [(and (= n 1) (= y 0)) x]
      [(and (= n 2) (= y 0)) 0]
      [(and (> n 2) (= y 0)) 1]
      [else (H (- n 1) x (H n x (- y 1)))]))
```

These calls give the answers.

```
> (H 4 2 0)
1
> (H 4 2 1)
2
> (H 4 2 2)
4
> (H 4 2 3)
16
> (H 4 2 4)
65536
```

The last one takes some time; below, the first number is milliseconds of CPU time.

```
cpu time: 155726 real time: 157987 gc time: 53317
65536
```

So it takes 156 seconds, about three minute (on a standard business laptop). Here is the output value summary.

y	0	1	2	3	4
$\mathcal{H}(4,2,y)$	1	2	4	16	65 536

I.4.12 A script yields that $\mathcal{H}_4(3,3) = 7\,625\,597\,484\,987$. Dividing by $60 \cdot 60 \cdot 24 \cdot 365.25$ gives the number of years as about a quarter million, 241 640.

I.4.13 We have that $\mathcal{H}_3(3,3) = 27$ and $\mathcal{H}_2(2,2) = 4$ so the ratio is 6.75.

I.4.14 We can get the values with Lemma 4.2, or the Racket procedure given in the section body.

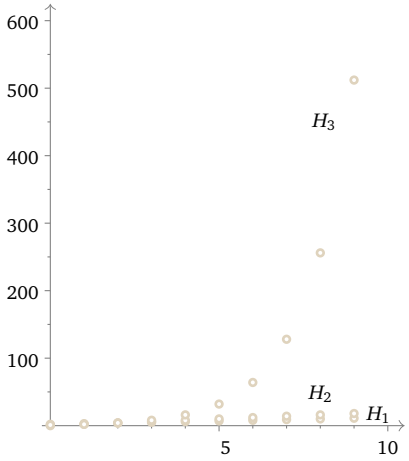
```
> (for ([y (in-range 10)])
    (printf "(~a,~a),\n" y (H 1 2 y)))
(0,2),
(1,3),
(2,4),
(3,5),
(4,6),
(5,7),
```

```
(6,8),
(7,9),
(8,10),
(9,11),
```

This summarizes the output.

y	0	1	2	3	4	5	6	7	8	9
$\mathcal{H}_1(2,y)$	2	3	4	5	6	7	8	9	10	11
$\mathcal{H}_2(2,y)$	0	2	4	6	8	10	12	14	16	18
$\mathcal{H}_3(2,y)$	1	2	4	8	16	32	64	128	256	512

Here is a graph.



I.4.15 Writing a small script makes the job of computing easier. Here is the table of values.

	$y=0$	$y=1$	$y=2$	$y=3$	$y=4$	$y=5$
$k=0$	1	2	3	4	5	6
$k=1$	2	3	4	5	6	7
$k=2$	3	5	7	9	11	13
$k=3$	5	13	29	61	125	253

I.4.16 This Racket program illustrates the definition.

```
(define (g x y)
  (if (= 100 (+ x y))
      0
      1))

(define (f x)
  (define (f-helper y)  ; value of x inherited from enclosing defn of f
    (if (= 0 (g x y))
        y
        (f-helper (+ 1 y))))

  (f-helper 0))
```

Racket will compute the values.

```
 > (f 0)
100
> (f 1)
99
> (f 50)
50
> (f 100)
0
```

The computation of `(f 101)` has to be terminated as it runs forever.

(A) $f(0) = 100$
(B) $f(1) = 99$
(C) $f(50) = 50$
(D) $f(100) = 0$
(E) This is not defined.

This is another description of f.

$$f(x) = \begin{cases} 100 - x & \text{– if } x \leq 100 \\ \uparrow & \text{– otherwise} \end{cases}$$

I.4.17

This Racket program illustrates the definition.

```
(define (g-mult x y)
  (if (= 100 (* x y))
      0
      1))

(define (f-mult x)
  (define (f-helper y)  ; value of x inherited from enclosing defn of f-mult
    (if (= 0 (g-mult x y))
        y
        (f-helper (+ 1 y))))

  (f-helper 0))
```

Racket will compute some of the values.

```
> (f 1)
100
> (f 50)
2
> (f 100)
1
```

(A) The value of $f(0)$ is undefined because there is no y where $0 \cdot y$ equals 100.
(B) $f(1) = 100$
(C) $f(50) = 2$
(D) $f(100) = 1$
(E) This is not defined because there is no y where $101 \cdot y$ equals 100.

I.4.18 As stated, the largest known prime Fermat number is $F_4 = 65\,537$.

(A) $f(0) = 5$
(B) $f(1) = 17$
(C) We don't know if this is defined as we don't know of any Fermat primes larger than F_{50}.
(D) We don't know if this is defined because we don't know of any Fermat primes larger than F_{F_4}.

I.4.19 For each input x to the function f, search for the first y where $y^3 \geq x^2$.

(A) $f(0) = 0$
(B) $f(1) = 1$
(C) $f(50) = 14$ since $50^2 = 2500$ while $14^3 = 2744$.
(D) $f(100) = 22$
(E) $f(x) = \lceil x^{2/3} \rceil$ where the 'ceiling' notation denotes taking the first integer greater than or equal to $x^{2/3}$

I.4.20 Recall that the number $d \in \mathbb{N}$ divides the number $n \in \mathbb{N}$ if there is a $k \in \mathbb{N}$ with $k \geq 1$ such that $d \cdot k = n$.

(A) We have notrelprime$(0, 0) = 0$ as the number $d = 2$ divides them both. It divides $n = 0$ because there is a $k \in \mathbb{N}$ such that $2 \cdot k = 0$, namely $k = 0$.
(B) The only number that divides 1 is 1 itself, so $f(1)\uparrow$.
(C) $f(2) = 2$
(D) $f(3) = 3$
(E) $f(4) = 2$
(F) $f(42) = 2$

(G) If $x = 0$ then $f(x) = 0$. If $x = 1$ then $f(x)$ is undefined. Otherwise $f(x) = p$ where p is the smallest prime that divides x

I.4.21 This Racket script helps with the calculations.

```
(define (g x y)
  (ceiling (- (/ (add1 x) (add1 y))
              1)))

(define (f x)
  (define (f-helper y)
    (if (= 0 (g x y))
        y
        (f-helper (add1 y))))

    (f-helper 0))
```

(A) Here is the output.

```
> (f 0)
0
> (f 1)
1
> (f 2)
2
> (f 3)
3
> (f 4)
4
> (f 5)
5
> (f 6)
6
```

The explanation of those values comes from this table of the relevant values of $g(x, y)$.

$g(x,y)$	$y=0$	$y=1$	$y=2$	$y=3$	$y=4$	$y=5$	$f(x)$
$x=0$	0						0
$x=1$	1	0					1
$x=2$	2	1	0				2
$x=3$	3	1	1	0			3
$x=4$	4	2	1	1	0		4
$x=5$	5	2	1	1	1	0	5

(B) We will argue that $f(x) = x$. Fix x. To have $g(x, y) = 0$ we need that $(x + 1)/(y + 1)$ is less than or equal to 1. The first y giving that is x.

I.4.22 No. All the functions in that collection would be total, they would all halt on all inputs. But some computable functions are not total — some effective procedures do not halt on some inputs.

I.4.23 The proof of Lemma 4.2 shows that $\mathcal{H}_1(x, y) = x + y$. The definition of $\mathcal{H}$ gives this.

$$\mathcal{H}_2 = \begin{cases} 0 & \text{– if } y = 0 \\ \mathcal{H}_1(x,\ \mathcal{H}_2(x, y-1)) & \text{– otherwise} \end{cases}$$

We will use induction to show that $\mathcal{H}_2(x, y) = x \cdot y$. The base step is $y = 0$, and in this case $x \cdot y = 0$, which matches the first clause. Now the inductive step. Suppose that $\mathcal{H}_2(x, y) = x \cdot y$ for $y = 0$, $y = 1$, ... $y = k$. For $y = k+1$, the 'otherwise' clause applies (as y can't be zero) so $\mathcal{H}_2(x, y) = \mathcal{H}_1(x, x \cdot k) = x + x \cdot k = x \cdot (1 + k) = x \cdot y$.

The other equality is similar. The definition of $\mathcal{H}$ gives this.

$$\mathcal{H}_3 = \begin{cases} 1 & \text{– if } y = 0 \\ \mathcal{H}_2(x,\ \mathcal{H}_3(x, y-1)) & \text{– otherwise} \end{cases}$$

We will use induction to show that $\mathcal{H}_3(x, y) = x^y$. For the $y = 0$ base step we have $x^0 = 1$, which matches the first clause. For the inductive step assume the inductive hypothesis that $\mathcal{H}_3(x, y) = x^y$ for

$y = 0$, $y = 1$, ... $y = k$. Consider $y = k + 1$. As y can't be zero the 'otherwise' clause applies, giving $\mathcal{H}_3(x, y) = \mathcal{H}_2(x, x^k) = x \cdot x^k = x^{1+k} = x^y$.

I.4.24 In each application of the recursion, either the first argument decreases or else the first argument remains the same and the third argument decreases. Further, each time that the third argument reaches zero, the first argument decreases, so it eventually reaches zero as well. If the first argument is zero then the computation terminates.

I.4.25 The function remtwo just toggles.

y	0	1	2	3	4	5	...
remtwo(y)	0	1	0	1	0	1	...

(A) The associated primitive recursion is also simple

$$\text{remtwo}(y) = \begin{cases} 0 & \text{– if } y = 0 \\ 1 \dot{-} \text{remtwo}(z) & \text{– if } y = S(z) \end{cases}$$

(As we often do, for clarity here we use abbreviations. For instance, instead of the function $Z(\,)$ we write the number 0 and we similarly abbreviate $\mathcal{S}(Z(\,))$ with 1.)

(B) Let $f(n) = \mu y \left[y + \text{remtwo}(n) = 0 \right]$. If n is even then remtwo(n) = 0 and $y = 0$ suffices. If n is odd then the remtwo(n) = 1 and there is no natural number y such that $y + 1 = 0$.

I.4.26 We can compute $f(x)$ by running the machine until it halts. This is the computation with $x = 0$.

Step	*Machine*
0	q_0
1	1 q_1
2	1 q_1
3	1 q_2

Here is the computation with input $x = 1$.

Step	*Machine*
0	1 q_0
1	1 q_0
2	1 1 q_1
3	1 1 q_1
4	1 1 q_1
5	1 1 q_2

The same for $x = 2$.

Step	*Machine*
0	1 1 q_0
1	1 1 q_0
2	1 1 q_0
3	1 1 1 q_1
4	1 1 1 q_1
5	1 1 1 q_1
6	1 1 1 q_1
7	1 1 1 q_2

This is 3.

Step	*Machine*
0	1 1 1 q_0
1	1 1 1 q_0
2	1 1 1 q_0
3	1 1 1 q_0

Step	*Machine*
4	1 1 1 1 q_1
5	1 1 1 1 q_1
6	1 1 1 1 q_1

Step	*Machine*
7	1 1 1 1 q_1
8	1 1 1 1 q_1
9	1 1 1 1 q_2

This is 4.

Step	*Machine*
0	1 1 1 1 q_0
1	1 1 1 1 q_0
2	1 1 1 1 q_0
3	1 1 1 1 q_0

Step	*Machine*
4	1 1 1 1 q_0
5	1 1 1 1 1 q_1
6	1 1 1 1 1 q_1
7	1 1 1 1 1 q_1

Step	*Machine*
8	1 1 1 1 1 q_1
9	1 1 1 1 1 q_1
10	1 1 1 1 1 q_1
11	1 1 1 1 1 q_2

Finally, this is the computation for $f(5)$.

Step	*Machine*
0	1 1 1 1 1 q_0
1	1 1 1 1 1 q_0
2	1 1 1 1 1 q_0
3	1 1 1 1 1 q_0
4	1 1 1 1 1 q_0

Step	*Machine*
5	1 1 1 1 1 q_0
6	1 1 1 1 1 1 q_1
7	1 1 1 1 1 1 q_1
8	1 1 1 1 1 1 q_1
9	1 1 1 1 1 1 q_1

Step	*Machine*
10	1 1 1 1 1 1 q_1
11	1 1 1 1 1 1 q_1
12	1 1 1 1 1 1 q_1
13	1 1 1 1 1 1 q_2

This table summarizes.

x	0	1	2	3	4	5
$f(x)$	3	5	7	9	11	13

I.4.27 To compute $f(x)$ we can run the machine until it halts. This is the computation with $x = 0$,

Step	*Machine*
0	q_0
1	1 q_2

and this is the computation with input $x = 1$.

Step	*Machine*
0	1 q_0
1	1 q_1
2	1 1 q_2

This is the same for $x = 2$.

Step	*Machine*
0	1 1 q_0
1	1 1 q_1
2	1 1 1 q_2

The $x = 3$ computation does this.

Step	*Machine*
0	1 1 1 q_0
1	1 1 1 q_1
2	1 1 1 1 q_2

This is $x = 4$.

Step	*Machine*
0	1 1 1 1 q_0
1	1 1 1 1 q_1
2	1 1 1 1 1 q_2

Finally, this is the computation for $f(5)$.

Step	*Machine*
0	1 1 1 1 1 q_0
1	1 1 1 1 1 q_1
2	1 1 1 1 1 1 q_2

The table below summarizes.

x	0	1	2	3	4	5
$f(x)$	2	3	3	3	3	3

For this machine the running time is constant. The machine in the prior exercise does the same job but its running time is a linear function of the input size.

I.4.28 One way to compute $f(x)$ for various values is to run the machine until it halts.

(A) This is the computation with $x = 0$.

Step	*Machine*
0	q_0
1	q_1
2	q_2

Step	*Machine*
3	q_3
4	q_4

(B) This is $x = 1$.

Step	*Machine*
0	1 q_0
1	1 q_1
2	1 q_2

Step	*Machine*
3	1 q_3
4	1 q_4

(C) This is the same for $x = 2$.

Step	*Machine*
0	1 1 q_0
1	1 1 q_1
2	1 1 q_2

Step	*Machine*
3	1 1 q_3
4	1 1 q_4

(D) The table summarizes.

x	0	1	2
$f(x)$	4	4	4

This machine ignores its input, moves right two squares and back left two, and then halts. So its running time is constant, $f(x) = 4$.

I.4.29 These are not too hard to compute by hand but a small program is also handy.

```
;; 3n+1 function
(define (H n)
  (if (even? n)
      (/ n 2)
      (+ (* 3 n) 1)))

(define (C n)
  (define (C-helper n k)
    (if (= 1 n)
        k
        (C-helper (H n) (add1 k))))
```

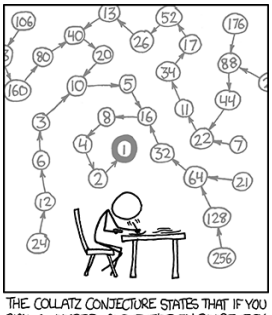


FIGURE 1, FOR QUESTION I.4.29: Courtesy xkcd.com

```
(C-helper n 0))
```

(A) The script gives this.

```
> (H 4)
2
> (H (H 4))
1
> (H (H (H 4)))
4
```

So we have this.

k	0	1	2	3
$H^k(4)$	4	2	1	4

(B) The script says this.

```
> (C 4)
2
```

This matches the result of the prior item, that after the second application of H the result is 1.

(C) Here are the successive values.

```
> (H 5)
16
> (H (H 5))
8
> (H (H (H 5)))
4
> (H (H (H (H 5))))
2
> (H (H (H (H (H 5)))))
1
```

Thus $C(5) = 5$, which agrees with the program's result.

```
> (C 5)
5
```

(D) The successive values are 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, and 1, so $C(11) = 14$.

(E) Here are the values.

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$C(n)$	0	1	7	2	5	8	16	3	19	6	14	9	9	17	15	4	12	20	20

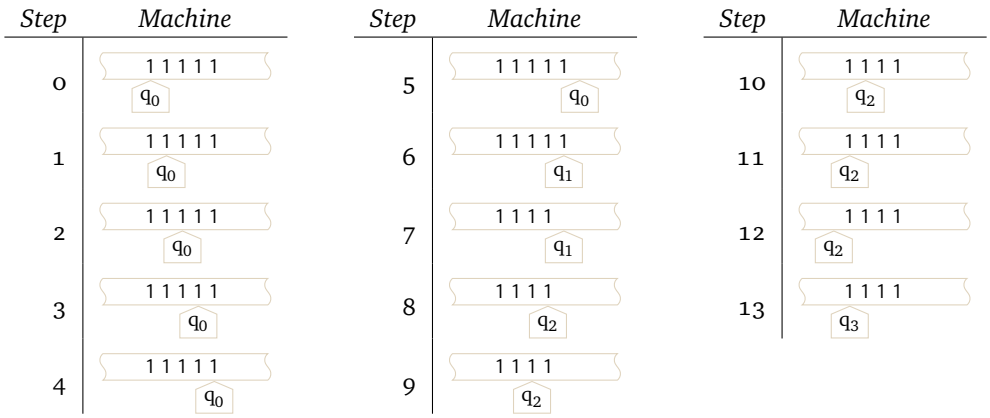
Figure 1 on page 32 shows that XKCD has something to say about the Collatz conjecture.

I.4.30 If D were primitive recursive then it would be some f_i. But then $D(i) = f_i(i) + 1 = D(i) + 1$, which is impossible.

I.A.1 This command line run starts with five 1's.

```
$ ./turing-machine.rkt -f machines/pred.tm -c "1" -r "1111"
step 0: q0: *1*1111
step 1: q0: 1*1*111
step 2: q0: 11*1*11
step 3: q0: 111*1*1
step 4: q0: 1111*1*
step 5: q0: 11111*B*
step 6: q1: 1111*1*B
step 7: q1: 1111*B*B
step 8: q2: 111*1*BB
step 9: q2: 11*1*1BB
step 10: q2: 1*1*11BB
step 11: q2: *1*111BB
step 12: q2: *B*1111BB
step 13: q3: B*1*111BB
step 14: HALT
```

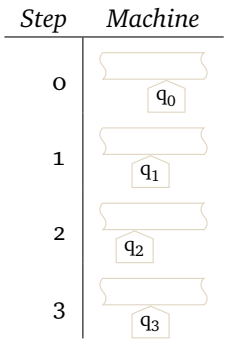
Here it is in the tape graphic format.



This is the command line invocation for an empty tape,

```
$ ./turing-machine.rkt -f machines/pred.tm -c " "
step 0: q0: *B*
step 1: q1: *B*B
step 2: q2: *B*BB
step 3: q3: B*B*B
step 4: HALT
```

and the matching sequence of tape diagrams.



I.A.2 This simulates 1 + 2.

```
$ ./turing-machine.rkt -f machines/pred.tm -c "1" -r " 11"
step 0: q0: *1* 11
step 1: q0: 1*B*11
step 2: q1: 1*B*11
step 3: q1: 1*1*11
step 4: q2: 1*1*11
step 5: q2: *1*111
```

```
step 6: q2: *B*1111
step 7: q3: *B*1111
step 8: q3: B*1*111
step 9: q4: B*B*111
step 10: q5: BB*1*11
step 11: HALT
```

and this is the matching tape diagram.

Step	*Machine*
0	1 1 1 / q_0
1	1 1 1 / q_0
2	1 1 1 / q_1
3	1 1 1 1 / q_1

Step	*Machine*
4	1 1 1 1 / q_2
5	1 1 1 1 / q_2
6	1 1 1 1 / q_2
7	1 1 1 1 / q_3

Step	*Machine*
8	1 1 1 1 / q_3
9	1 1 1 / q_4
10	1 1 1 / q_5

The command line for 0 + 2

```
$ ./turing-machine.rkt -f machines/pred.tm -c " " -r "11"
```

gives these tape diagrams.

Step	*Machine*
0	1 1 / q_0
1	1 1 / q_1
2	1 1 1 / q_1

Step	*Machine*
3	1 1 1 / q_2
4	1 1 1 / q_2
5	1 1 1 / q_3

Step	*Machine*
6	1 1 1 / q_3
7	1 1 / q_4
8	1 1 / q_5

The command line for 0 + 0

```
$ ./turing-machine.rkt -f machines/pred.tm -c " "
```

gives this sequence of tape diagrams.

Step	*Machine*
0	/ q_0
1	/ q_1
2	1 / q_1

Step	*Machine*
3	1 / q_2
4	1 / q_2
5	1 / q_3

Step	*Machine*
6	1 / q_3
7	/ q_4
8	/ q_5

I.A.3 This ten-instruction machine

$$\mathcal{P}_{\text{addthree}} = \{q_0\mathsf{1}\mathsf{R}q_0,\ q_0\mathsf{B}\mathsf{B}q_1,\ q_1\mathsf{B}\mathsf{1}q_1,\ q_1\mathsf{1}\mathsf{R}q_2,\ q_2\mathsf{B}\mathsf{1}q_2,\ q_2\mathsf{1}\mathsf{R}q_3,\ q_3\mathsf{B}\mathsf{1}q_3,\ q_3\mathsf{1}\mathsf{1}q_4,\ q_4\mathsf{1}\mathsf{L}q_4,\ q_4\mathsf{B}\mathsf{R}q_5\}$$

matches the source for the simulator.

```
0 1 R 0
0 B B 1
1 B 1 1
1 1 R 2
2 B 1 2
2 1 R 3
3 B 1 3
3 1 1 4
4 B R 5
4 1 L 4
```

This command line for input 4

```
$ ./turing-machine.rkt -f machines/addthree.tm -c "1" -r "111"
```

gives these tape diagrams.

Step	*Machine*
0	1 1 1 1 q_0
1	1 1 1 1 q_0
2	1 1 1 1 q_0
3	1 1 1 1 q_0
4	1 1 1 1 q_0
5	1 1 1 1 q_1
6	1 1 1 1 1 q_1

Step	*Machine*
7	1 1 1 1 1 q_2
8	1 1 1 1 1 1 q_2
9	1 1 1 1 1 1 q_3
10	1 1 1 1 1 1 1 q_3
11	1 1 1 1 1 1 1 q_4
12	1 1 1 1 1 1 1 q_4
13	1 1 1 1 1 1 1 q_4

Step	*Machine*
14	1 1 1 1 1 1 1 q_4
15	1 1 1 1 1 1 1 q_4
16	1 1 1 1 1 1 1 q_4
17	1 1 1 1 1 1 1 q_4
18	1 1 1 1 1 1 1 q_4
19	1 1 1 1 1 1 1 q_5

The command line for 0

```
$ ./turing-machine.rkt -f machines/addthree.tm -c " "
```

gives this sequence of tape diagrams.

Step	*Machine*
0	q_0
1	q_1
2	1 q_1
3	1 q_2

Step	*Machine*
4	1 1 q_2
5	1 1 q_3
6	1 1 1 q_3
7	1 1 1 q_4

Step	*Machine*
8	1 1 1 q_4
9	1 1 1 q_4
10	1 1 1 q_4
11	1 1 1 q_5

I.A.4

(A) Something like this will do.

$$\mathcal{P}_0 = \{q_{10}\texttt{0R}q_{10},\ q_{10}\texttt{1R}q_{10},\ q_{10}\texttt{BB}q_{100}\}$$

(B) This will back up, blanking out cells.

$$\mathcal{P}_1 = \{q_{20}\texttt{0B}q_{21},\ q_{20}\texttt{1B}q_{21},\ q_{20}\texttt{BB}q_{100},\ q_{21}\texttt{0L}q_{20},\ q_{21}\texttt{1L}q_{20},\ q_{21}\texttt{BL}q_{20}\}$$

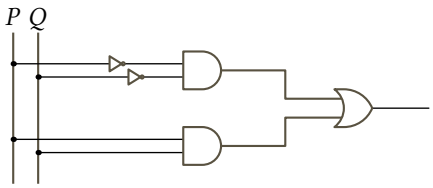


FIGURE 2, FOR QUESTION I.B.1: Circuit for P XOR Q.

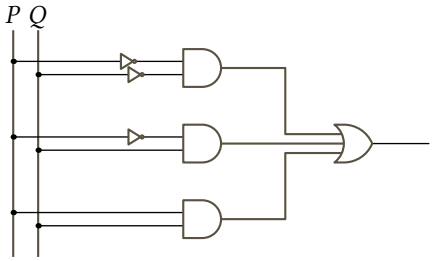


FIGURE 3, FOR QUESTION I.B.2: Circuit for the implication operation.

(C) Here, roughly q_0 is where none of the substring is matched, q_1 is where `0` has been seen and the machine is looking for the `1`, and q_2 is where `01` has been seen and the machine is looking for the `0`. Then the four states q_3, q_4, q_5, and q_6 mark a failure (they end with q_6 writing `0` on a blank tape). The four states q_7, q_8, q_9, and q_{10} mark a success.

$$\begin{aligned}\mathcal{P}_{\text{decideo1o}} = \{&q_0\texttt{0R}q_1,\ q_0\texttt{1R}q_0,\ q_0\texttt{BB}q_3,\ q_1\texttt{0R}q_1,\ q_1\texttt{1R}q_2,\ q_1\texttt{BB}q_3,\ q_2\texttt{0R}q_7,\ q_2\texttt{11}q_0,\ q_2\texttt{BB}q_3,\\ &q_3\texttt{0R}q_3,\ q_3\texttt{1R}q_3,\ q_3\texttt{BL}q_4,\ q_4\texttt{0B}q_5,\ q_4\texttt{1B}q_5,\ q_4\texttt{BB}q_6,\\ &q_5\texttt{0L}q_4,\ q_5\texttt{1L}q_4,\ q_5\texttt{BL}q_4,\ q_6\texttt{00}q_{100},\ q_6\texttt{10}q_{100},\ q_6\texttt{B0}q_{100},\\ &q_7\texttt{0R}q_7,\ q_7\texttt{1R}q_7,\ q_7\texttt{BL}q_8,\ q_8\texttt{0B}q_9,\ q_8\texttt{1B}q_9,\ q_8\texttt{BB}q_{10},\\ &q_9\texttt{0L}q_8,\ q_9\texttt{1L}q_8,\ q_9\texttt{BL}q_8,\ q_{10}\texttt{01}q_{100},\ q_{10}\texttt{11}q_{100},\ q_{10}\texttt{B1}q_{100}\}\end{aligned}$$

I.B.1

(A) Here is the table.

P	Q	P XOR Q
0	0	0
0	1	1
1	0	1
1	1	0

The statement is $(\neg P \wedge \neg Q) \vee (P \wedge Q)$.

(B) Figure 2 on page 36 gives the circuit.

I.B.2

(A) The definition gives this input/output behavior

P	Q	$P \to Q$
0	0	1
0	1	1
1	0	0
1	1	1

so a Boolean expression is $(\neg P \wedge \neg Q) \vee (\neg P \wedge Q) \vee (P \wedge Q)$.

(B) Figure 3 on page 36 gives the circuit.

I.B.3 The statement is $(\neg P \wedge Q) \vee (P \wedge Q)$. Figure 4 on page 37 gives the circuit.

I.B.4 The two circuits are in Figure 5 on page 37.

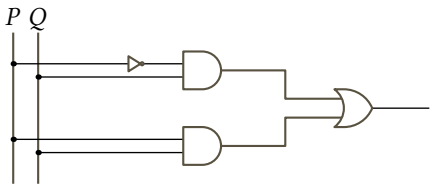


FIGURE 4, FOR QUESTION I.B.3: Circuit for the unnamed operation (we could call it Q).

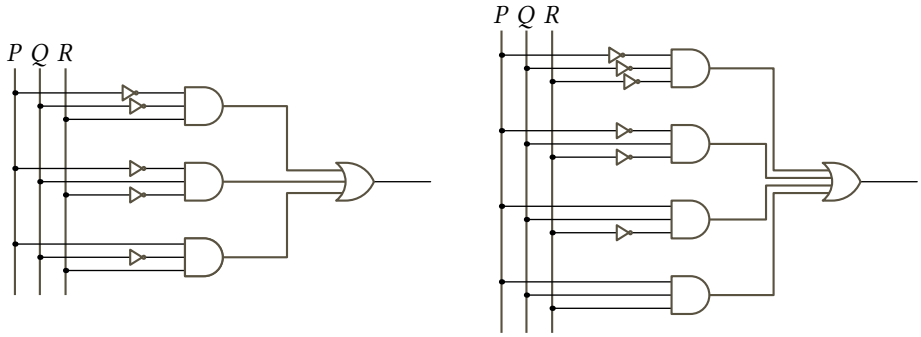


FIGURE 5, FOR QUESTION I.B.4: Circuits for the two unnamed operation tables.

(A) In the expression, each clause matches a line of the table with a 1 output.

$$(\neg P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge R)$$

(B) The clauses match the 1 lines.

$$(\neg P \wedge \neg Q \wedge \neg R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R).$$

I.B.5 Here is the input/output behavior.

P	Q	R	
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

That table immediately gives the Disjunctive Normal Form expression.

$$(\neg P \wedge \neg Q \wedge \neg R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge R) \vee (P \wedge Q \wedge R)$$

Figure 6 on page 38 gives the circuit.

I.B.6 Here a line has a 1 in the output column if and only if two or three of the inputs are 1's.

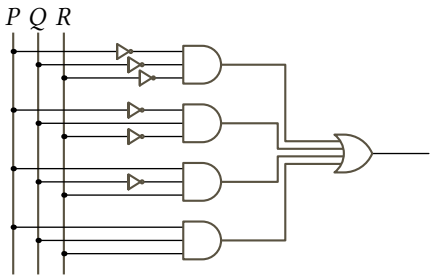


FIGURE 6, FOR QUESTION I.B.5: Circuit for $P = R$.

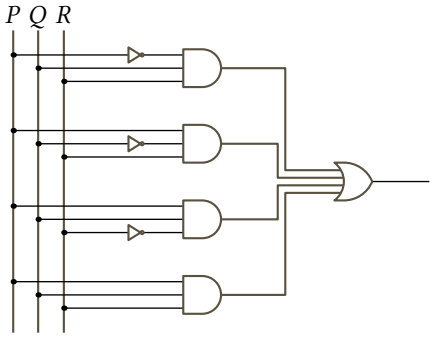


FIGURE 7, FOR QUESTION I.B.6: Circuit for the three-input majority operator.

P	Q	R	
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

This is a Disjunctive Normal Form expression.

$$(\neg P \wedge Q \wedge R) \vee (P \wedge \neg Q \wedge R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R)$$

Figure 7 on page 38 gives the circuit.

I.B.7

(A) Here a line has a 1 in the output column if and only if three or four of the inputs are 1's.

P	Q	R	S	
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

This DNF expression contains all the clauses where at least three of the variables are without negations.

$$(\neg P \wedge Q \wedge R \wedge S) \vee (P \wedge \neg Q \wedge R \wedge S) \vee (P \wedge Q \wedge \neg R \wedge S) \vee (P \wedge Q \wedge R \wedge \neg S) \vee (P \wedge Q \wedge R \wedge S)$$

(B) The DNF expression must consist of the clauses where at least three of the variables are not negated. There are $\binom{5}{3} = 10$ such clauses with exactly three non-negated variables, $\binom{5}{4} = 5$ clauses with one negated variable, and $\binom{5}{5} = 1$ with no negated variables.

$$\begin{aligned}
&(\neg P \wedge \neg Q \wedge R \wedge S \wedge T) \vee (\neg P \wedge Q \wedge \neg R \wedge S \wedge T) \vee (\neg P \wedge Q \wedge R \wedge \neg S \wedge T) \vee (\neg P \wedge Q \wedge R \wedge S \wedge \neg T) \\
&\quad \vee (P \wedge \neg Q \wedge \neg R \wedge S \wedge T) \vee (P \wedge \neg Q \wedge R \wedge \neg S \wedge T) \vee (P \wedge \neg Q \wedge R \wedge S \wedge \neg T) \\
&\quad \vee (P \wedge Q \wedge \neg R \wedge \neg S \wedge T) \vee (P \wedge Q \wedge \neg R \wedge S \wedge \neg T) \vee (P \wedge Q \wedge R \wedge \neg S \wedge \neg T) \\
&\vee (\neg P \wedge Q \wedge R \wedge S \wedge T) \vee (P \wedge \neg Q \wedge R \wedge S \wedge T) \vee (P \wedge Q \wedge \neg R \wedge S \wedge T) \\
&\quad \vee (P \wedge Q \wedge R \wedge \neg S \wedge T) \vee (P \wedge Q \wedge R \wedge S \wedge \neg T) \\
&\vee (P \wedge Q \wedge R \wedge S \wedge T)
\end{aligned}$$

I.B.8

(A) Here is the tabular version. The carrys are in the second, third, and fifth columns from the right.

$$\begin{array}{ccccc}
^{1} & 1 & ^{1}0 & ^{1}1 & 1 \\
+ & 1 & 0 & 0 & 1 \\
\hline
1 & 0 & 1 & 0 & 0
\end{array}$$

(B) The definition gives these two input/output behaviors. What is called the Least significant bit here is often called a 'full adder'. (Also common is a 'half adder' which does not take a Carry in, so it only has two inputs.)

Carry in I	P	Q	Least significant bit L	Carry out C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

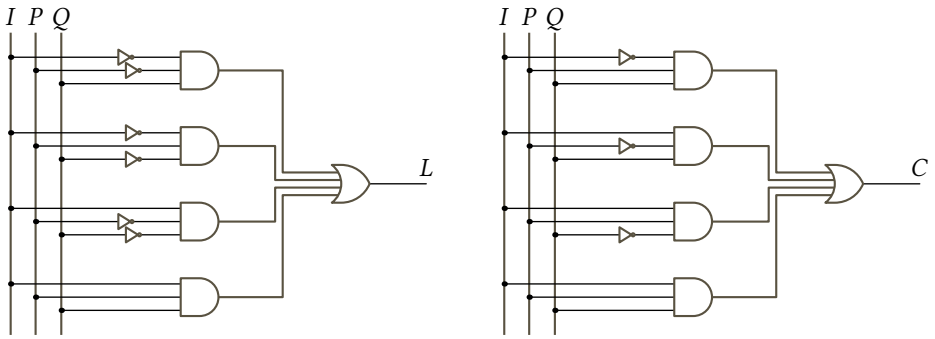
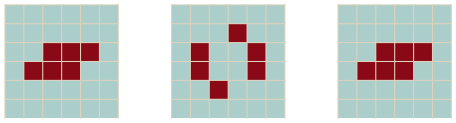


FIGURE 8, FOR QUESTION I.B.8: Partial sum and carry circuits for one addition column.

A Boolean expression for L is $(\neg I \wedge \neg P \wedge Q) \vee (\neg I \wedge P \wedge \neg Q) \vee (I \wedge \neg P \wedge \neg Q) \vee (I \wedge P \wedge Q)$. For C a DNF expression is $(\neg I \wedge P \wedge Q) \vee (I \wedge \neg P \wedge Q) \vee (I \wedge P \wedge \neg Q) \vee (I \wedge P \wedge Q)$.

(C) Figure 8 on page 40 gives the circuits.

I.C.4 The tub is a still life. The toad oscillates with a period of two.



I.C.5 No. For instance, the discussion of the simplest nontrivial pattern gives two unequal 3×3 gameboards where the next generation is entirely dead. So from a dead board you cannot tell what was the prior board.

Also, because there are pairs of boards that go next to the same board by counting there must also be some boards with no predecessor, with zero-many boards that come before. Such a board is a 'Garden of Eden'. For these, were we to try running the clock backward, it is not a question of choosing which of several boards lead to the current one — there is no such board.

I.C.6

(A) For 3×3 each cell is either alive or dead so there are 2^9 grids. For $n \times n$ it is 2^{n^2}.

(B) With $2^9 = 512$ many 3×3 gameboards, we use a program.

```
; Input a natural number less than 2^9, output a grid with associated 0's and 1's
; For instance, n=28 is 11100 in binary so this returns the vector of vectors [[0 0 0] [0 1 1] [1 0 0]]
(define (number->three-by-three-grid n)
  (if (>= n (expt 2 9))
      (error "input too large; must be less than 2^9")
      (let* ([bitstr (number->string n 2)]
             [pad-bitstr (make-string (- 9 (string-length bitstr)) #\0)]
             [length-9-bitstr (string-append pad-bitstr bitstr)]
             [input-style-bitstr (string-replace (string-replace length-9-bitstr "0" ".") "1" "*")]
             [list-of-lines (list (substring input-style-bitstr 0 3)
                                  (substring input-style-bitstr 3 6)
                                  (substring input-style-bitstr 6 9))])
        (parse-lines-to-grid list-of-lines))))

; Test if a grid is empty
(define (grid-empty? g)
  (let ([size (grid-size g)])
    (grid-equal? g (grid-create (first size) (second size)))))

; Count the number of 3x3 grids that are alive after the specified number of generations
(define (number-alive generations)
  (let ([counter 0])
    (for ([n (in-range (expt 2 9))])
      (displayln (~a "n=" n "\ngrid=" (grid->string (number->three-by-three-grid n))))
      (let* ([initial-u (universe (number->three-by-three-grid n) (list 0 0))]
             [list-of-universes (run initial-u generations)]
             [ending-grid (universe-grid (last list-of-universes))])
        (when (not (grid-empty? ending-grid))
          (set! counter (add1 counter))
          (displayln " not blank"))))
```

```
    (displayln (~a "Number of 3x3 boards surviving is " counter))))
```

Running the program gives this.

```
> (number-alive 1)
  ... lots of lines ...
n=510
grid=***
***
**.
 not blank
n=511
grid=***
***
***
 not blank
Number of 3x3 boards surviving is 461
```

(C) Again using the program gives this.

```
> (number-alive 10)
  ... lots of lines ...
n=510
grid=***
***
**.
 not blank
n=511
grid=***
***
***
 not blank
Number of 3x3 boards surviving is 249
```

I.D.8 Rewrite $2^{65\,536}$ as $(10^{\log_{10} 2})^{65\,536}$. That is about $(10^{0.3010299957})^{65\,536} \approx 10^{19728.30}$. So there are 19 729 digits.

I.D.9 This is about uniformity. Imagine that we have an enumeration of the primitive recursive functions f_0, f_1, etc. Suppose that level zero is $\mathcal{A}_0 = f_9$, and level one is $\mathcal{A}_1 = f_{121}$, and $\mathcal{A}_2 = f_{800}$, etc. What we have is that the function associating the subscript on the $\mathcal{A}$ with the subscript on the f is not primitive recursive. That is, in this imagined scenario, $\mathcal{A}_i = f_{g(i)}$ but g is not primitive recursive.

It is a little like saying that all the layers of a cake are delicious, but the cake was made where the flavor of layer 1 was picked with no regard for the flavor of layer 0, perhaps coffee cake and hummingbird cake.

I.D.10

(A) We can get this function as a composition $f(y) = \mathcal{S}(\mathcal{S}(y))$. The successor functions are level 1. Composition adds two to that so overall f is level 3.

(B) We have this.

$$\text{pred}(y) = \begin{cases} \mathcal{Z}(x) = 0 & \text{– if } x = 0 \\ \mathcal{I}_0(z) = z & \text{– if } y = \mathcal{S}(z) \end{cases}$$

The zero function is level zero, as is the projection function. Primitive recursion adds three so the predecessor function is level 3.

I.D.11 In the definition of $\mathcal{A}$, evaluation of the $k = 0$ clause clearly terminates. In each of the other two clauses, either k decreases or k does not decrease but y does. When y reaches zero then k decreases, so it eventually reaches zero as well, which we have already noted gives a terminating evaluation.

I.D.12

(A) We will verify this statement by induction on k (the 'in general' follows immediately from it).

$$\mathcal{A}(k, y+1) > \mathcal{A}(k, y) \qquad (*)$$

In the $k = 0$ base step, $\mathcal{A}(0, y+1) = y + 2$ while $\mathcal{A}(0, y) = y + 1$, so $(*)$ is clear.

For the inductive step assume that $(*)$ holds for $k = 0, 1, \ldots n$ and consider $k = n + 1$. Then the definition of $\mathcal{A}$ gives the equality in $\mathcal{A}(n+1, y+1) = \mathcal{A}(n, \mathcal{A}(n+1, y)) > \mathcal{A}(n+1, y)$, while the inequality follows from Lemma D.2.**??**.

(B) We will prove this statement by induction on y.

$$\mathcal{A}(k+1,y) \geq \mathcal{A}(k,y+1) \qquad (*)$$

The second clause of $\mathcal{A}$'s definition is $\mathcal{A}(k+1,0) = \mathcal{A}(k,1)$ so the $y=0$ base step is immediate.

The inductive step starts with the inductive hypothesis that $(*)$ holds for $y = 0, 1, \ldots n$. Consider the $y = n+1$ case, involving $\mathcal{A}(k+1,n+1)$. The third clause of $\mathcal{A}$'s definition is that $\mathcal{A}(k+1,n+1) = \mathcal{A}(k,\mathcal{A}(k+1,n))$. Then $\mathcal{A}(k+1,n) \geq \mathcal{A}(k,n+1) \geq (n+1)+1$ holds first by the inductive hypothesis and second by Lemma D.2.**??**. With that, by the 'in general' clause of the prior exercise, Lemma D.2.**??**, $\mathcal{A}(k,\mathcal{A}(k+1,n)) \geq \mathcal{A}(k,(n+1)+1)$, and therefore $\mathcal{A}(k+1,n+1) \geq \mathcal{A}(k,(n+1)+1)$, as desired.

(C) The statement here is that $\mathcal{A}(k,y) > k$. Repeatedly applying the prior item, Lemma D.2.**??**, shows this.

$$\mathcal{A}(k,y) = \mathcal{A}(k-1,y+1) = \mathcal{A}(k-2,y+2) = \cdots = \mathcal{A}(0,y+(k+1))$$

Then the first clause of the definition is that $\mathcal{A}(0,y+(k+1)) = y+(k+1)$, which is greater than k.

(D) For $\mathcal{A}(k+1,y) > \mathcal{A}(k,y)$, start with Lemma D.2.**??** and then apply Lemma D.2.**??**.

$$\mathcal{A}(k+1,y) \geq \mathcal{A}(k,y+1) > \mathcal{A}(k,y)$$

The 'in general' follows on iterating.

(E) This is the statement.

$$\mathcal{A}(k+2,y) > \mathcal{A}(k,2y) \qquad (*)$$

We will do induction on y. For the base step of $y=0$ we use the 'in general' part of the prior item, Lemma D.2.**??**, to get $\mathcal{A}(k+2,y) = \mathcal{A}(k+2,0) > \mathcal{A}(k,0) = \mathcal{A}(k,2y)$.

For the inductive step assume that $(*)$ holds for $y = 0, \ldots n$ and consider the $y = n+1$ case. The third clause in the definition of $\mathcal{A}$ is that $\mathcal{A}(k+2,n+1) = \mathcal{A}(k+1,\mathcal{A}(k+2,n))$. The inductive hypothesis is that $\mathcal{A}(k+2,n) > \mathcal{A}(k,2n)$ and so the 'in general' clause of Lemma D.2.**??** gives this.

$$\mathcal{A}(k+2,n+1) = \mathcal{A}(k+1,\mathcal{A}(k+2,n)) > \mathcal{A}(k+1,\mathcal{A}(k,2n))$$

Apply Lemma D.2.**??** to that.

$$\mathcal{A}(k+2,n+1) > \mathcal{A}(k+1,\mathcal{A}(k,2n)) \geq \mathcal{A}(k,\mathcal{A}(k,2n)+1)$$

Now, by Lemma D.2.**??**, $\mathcal{A}(k,2n) \geq 2n+1$ and so $\mathcal{A}(k,2n)+1 \geq 2n+2 = 2(n+1)$. Thus $\mathcal{A}(k+2,n+1) > \mathcal{A}(k,2(n+1))$, as required.

I.E.2 Here is the code.

```
# swap-two.loop  Input x,y and output y,x
r2 = r0
r0 = r1
r1 = r2
```

And here is a sample invocation.

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/swap-two.loop -p "1 2 3" -o 2
2 1
```

I.E.3 We can get the effect of the first with two increments in a row, and we can get the effect of the second by first zeroing and then incrementing.

I.E.4 The rotate left code is an adaptation of the rotate right code.

```
# rotate-shift-left.loop  input x,y,z, output y,z,x
r3 = r0
r0 = r1
r1 = r2
r2 = r3
```

Here is a sample invocation.

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/rotate-shift-left.loop -p "1 2 3" -o 3
2 3 1
```

Putting the two together gives this.

```
# rotate-shift-right
r3 = r2
r2 = r1
r1 = r0
r0 = r3
# now rotate-shift-left
r3 = r0
r0 = r1
r1 = r2
r2 = r3
```

Here is a sample invocation.

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/concatenate-rotations.loop -p "1 2 3" -o 3
1 2 3
```

I.E.5 Here is a program.

```
# power.loop  Return r0 ^ r1
r2 = r1   # save the inputs in higher registers
r1 = r0   #
r0 = 0
r0 = r0 + 1 # cover the r1=0 case
r3 = 0       # Initialize
r3 = r3 + 1  # to 1
loop r2
  r0 = 0
  loop r3
    loop r1
      r0 = r0 + 1
      end
    end
  r3 = r0
  end
```

And here are two sample invocations.

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/power.loop -p "3 2"
9
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/power.loop -p "3 0"
1
```

I.E.6 The first does this; observe that it runs five times.

```
 > (for ([i (in-range 5)])
    (set! i 1)
    (displayln (~a "i=" i)))
i=1
i=1
i=1
i=1
i=1
```

The second does this.

```
> (for ([i (in-range 5)])
    (displayln (~a "i=" i))
    (set! i 1))
i=0
i=1
i=2
i=3
i=4
```

Chapter II: Background

II.1.20 To show that the map is one-to-one we will show that $g(n) = g(\hat{n})$ can only happen if $n = \hat{n}$. So let $g(n) = g(\hat{n})$, giving $3n = 3\hat{n}$. Divide by three to get $n = \hat{n}$.

To show that the map is onto we will show that every member of the codomain $y \in \{3k \mid k \in \mathbb{N}\}$ is the image of some member of the domain. Because y is a multiple of three, $y/3$ is a natural number. Since $y = g(y/3)$ we have that y is indeed the image of a member of the domain.

II.1.21 This is a type error. It is not the sets that are one-to-one or onto; rather, it is a correspondence f between the sets. For $D = \{n^2 \mid n \in \mathbb{N}\}$ and $C = \{n^3 \mid n \in \mathbb{N}\}$ the natural such correspondence $f : D \to C$ is $f(n) = n^{3/2}$.

II.1.22

(A) The function $f : \mathbb{Z} \to \mathbb{Z}$ given by $f(x) = 2x$ is one-to-one but not onto. Verifying that it is one-to-one is routine: assume that $f(x) = f(\hat{x})$ for $x, \hat{x} \in \mathbb{Z}$, so that $2x = 2\hat{x}$, and then cancel the 2's to get that $x = \hat{x}$. To show that it is not onto we need only name one codomain element $y \in \mathbb{Z}$ that is not the image of any domain element. One such is $y = 1$, since the image of every domain element is even.

(B) The function $g : \mathbb{Z} \to \mathbb{Z}$ given by $g(x) = 2x - 1$ is also one-to-one but not onto. The one-to-one argument works just as it did in the prior item: assume that $g(x) = g(\hat{x})$ for $x, \hat{x} \in \mathbb{Z}$ so that $2x - 1 = 2\hat{x} - 1$. Add 1 to both sides and cancel the 2's to get $x = \hat{x}$. This function is not onto because the codomain element $y = 2$ is not the image of any domain element, since it is even but the number $2x - 1$ is odd for any input integer x.

(C) They are not inverse. For instance, $f \circ g\,(1) = f(g(1)) = f(1) = 2$.

II.1.23

(A) The function $f : \mathbb{N} \to \mathbb{N}$ given by $f(n) = n + 1$ is one-to-one but not onto. To check that it is one-to-one suppose that $f(n_0) = f(n_1)$ for $n_0, n_1 \in \mathbb{N}$, so that $n_0 + 1 = n_1 + 1$, and then subtract 1 from both sides to get $n_0 = n_1$. It is not onto because the codomain element $0 \in \mathbb{N}$ is not the image of any domain element.

(B) The map $f : \mathbb{Z} \to \mathbb{Z}$ given by $f(n) = n + 1$ is a correspondence — it is both one-to-one and onto. The verification that it is one-to-one goes: assume that $f(n_0) = f(n_1)$ so that $n_0 + 1 = n_1 + 1$, and subtract 1 to conclude that $n_0 = n_1$. The verification that it is onto goes: fix some codomain element $y \in \mathbb{Z}$ and note that $x = y - 1$ is a domain element which satisfies that $f(x) = y$.

(C) The function $f : \mathbb{N} \to \mathbb{N}$ defined by $f(n) = 2n$ is one-to-one but not onto. For one-to-one, assume that $n_0, n_1 \in \mathbb{N}$ are such that $f(n_0) = f(n_1)$. Then $2n_0 = 2n_1$ and cancelling the 2's gives that $n_0 = n_1$. It is not onto because the codomain element $y = 1$ is not the image of any domain element, since all such images are even.

(D) The function $f : \mathbb{Z} \to \mathbb{Z}$ defined by $f(n) = 2n$ is one-to-one but not onto. The one-to-one verification is as in the prior item: suppose that $f(n_0) = f(n_1)$ for some $n_0, n_1 \in \mathbb{Z}$, so that $2n_0 = 2n_1$, and cancel the 2's to conclude that $n_0 = n_1$. Also as in the prior item, this map is not onto because the codomain element $y = 1$ is not in the range of the function, as all of the range elements are even.

(E) The function $f : \mathbb{Z} \to \mathbb{N}$ given by $f(n) = |n|$ is not one-to-one but it is onto. It is not one-to-one because the two domain elements $n_0 = -1$ and $n_1 = 1$ map to the same output, $f(n_0) = 2 = f(n_1)$. To show that f is an onto map consider a codomain element $y \in \mathbb{N}$, and observe that if we take the domain element $x \in \mathbb{Z}$ such that $x = y$ then $f(x) = y$.

II.1.24

(A) This function is a correspondence. To verify that it is one-to-one suppose that $f(q_0) = f(q_1)$ for $q_0, q_i \in \mathbb{Q}$, so that $q_0 + 3 = q_1 + 3$, and subtract 3 from both sides to get that $q_0 = q_1$. For onto, consider a codomain element $y \in \mathbb{Q}$ and observe that the domain element $q \in \mathbb{Q}$ given by $q = y - 3$ satisfies that $f(q) = y$.

(B) This function is not a correspondence. It is not onto because the codomain element $1/2 \in \mathbb{Q}$ is not the image of any domain element $z \in \mathbb{Z}$, since each such image $f(z) = z + 3$ is an integer. (This function is

one-to-one: suppose that $f(z_0) = f(z_1)$, so that $z_0 + 3 = z_1 + 3$, and subtract 3 from both sides to conclude that $z_0 = z_1$. But because it is not onto, it is not a correspondence.)

(C) This is not a correspondence. It is not even a function, because it is not well-defined — the two $q_0 = 1/2$ and $q_1 = 2/4$ are equal, $q_0 = q_1$, but they are not associated with the same $|a \cdot b|$'s since $|1 \cdot 2| = 2$ and $|2 \cdot 4| = 8$.

II.1.25

(A) This set is finite. It has the same cardinality as $I = \{0, 1, 2\}$, as witnessed by the function $f : I \to \{1, 2, 3\}$ given by $f(x) = x + 1$, which is one-to-one and onto by inspection.

(B) The set $S = \{0, 1, 4, 9, 16, \ldots\}$ of perfect squares is infinite. There is no correspondence between some $I = \{0, 1, \ldots\ n - 1\}$ and S since there is no onto function — for any $f : I \to S$ the set $\{f(0), f(1), \ldots\ f(n-1)\}$ cannot equal S because it has a largest element but S has no largest element.

(C) The set of primes is infinite. The argument from the prior item applies.

(D) By the Fundamental Theorem of Arithmetic, a fifth degree polynomial has at most five real number roots. Thus this set has cardinality at most five, and so is finite.

II.1.26

(A) The map $f : \{0, 1, 2\} \to \{3, 4, 5\}$ given by $f(0) = 3$, $f(1) = 4$, and $f(2) = 5$, that is, $f(x) = x + 3$, is a correspondence. By inspection each element of the codomain is associated with one and only one element of the domain.

(B) The function $f(x) = x^3$ is a correspondence between these sets. For each perfect cube there is one and only one associated integer cube root.

II.1.27

(A) The function f given by $0 \mapsto \pi$, $1 \mapsto \pi + 1$, $3 \mapsto \pi + 2$, and $7 \mapsto \pi + 3$ is both one-to-one and onto, by inspection.

(B) Let $E = \{2n \mid n \in \mathbb{N}\}$ and $S = \{n^2 \mid n \in \mathbb{N}\}$. Consider the map $f : E \to S$ given by $f(x) = (x/2)^2$. To show that f is one-to-one, suppose that $f(x_0) = f(x_1)$. Then $(x_0/2)^2 = (x_1/2)^2$ and because these are natural numbers their square roots are equal, $x_0/2 = x_1/2$. Thus $x_0 = x_1$ and the map is one-to-one.

For onto, consider $y \in S$. Because y is a perfect square there is a natural number $n \in \mathbb{N}$ so that $y = n^2$. Let $x = 2n$. Clearly x is even and $f(x) = y$.

(C) The second interval is 2/3-rds the width of the first, so consider the function $f(x) = (2/3) \cdot x - (5/3)$. It is one-to-one because $f(x_0) = f(x_1)$ implies that $(2/3) \cdot x_0 - (5/3) = (2/3) \cdot x_1 - (5/3)$, and straightforward algebra gives that $x_0 = x_1$. To show that f is onto, fix $y \in (-1 \,..\, 1)$. Let $x = (y + (5/3)) \,/\, (2/3)$ (this came from solving $y = (2/3)x - (5/3)$ for x). When y is an element of the codomain $(-1 \,..\, 1)$ then this x is an element of the domain $(1 \,..\, 4)$, and more algebra shows that $f(x) = y$.

II.1.28 The problem statement is ambiguous as to which set is the domain and which is the codomain. We will prove that $f : (0 \,..\, 1) \to (1 \,..\, \infty)$ given by $f(x) = 1/x$ is one-to-one and onto.

For one-to-one assume that $f(x_0) = f(x_1)$, where $x_0, x_1 \in (0 \,..\, 1)$. Then $1/x_0 = 1/x_1$ and cross-multiplication gives $x_1 = x_0$. For onto suppose that $y \in (1 \,..\, \infty)$ and note that if $x = 1/y$ then $f(x) = y$ and also $x \in (0 \,..\, 1)$.

II.1.29 Call the sets $D = \{1, 2, 3, 4, \ldots\}$ and $C = \{7, 10, 13, 16 \ldots\}$. We will show that the function $f : D \to C$ described by the formula $f(x) = 3(x - 1) + 7$ gives a correspondence between the sets. This function is one-to-one since $f(x_0) = f(x_1)$ implies that $3(x_0 - 1) + 7 = 3(x_1 - 1) + 7$, and subtracting 7, dividing by 3, and adding 1 gives that $x_0 = x_1$. It is onto because if y is an element of the codomain C then it has the form $y = 3n + 4$ for $n \in \mathbb{N}^+$. That equation gives $y = 3(n - 1) + 7$. Some algebra gives $(y - 7)/3 = n - 1$ for $n \in \mathbb{N}^+$ and we have that $n = ((y - 7)/3) + 1$ is an element of $\mathbb{N}^+ = D$ with $f(n) = y$.

II.1.30

(A) The obvious map is this.

x	0	1	2	3	4	5	6	7	8	9
$f(x)$	48	49	50	51	52	53	54	55	56	57

It is one-to-one by inspection, meaning that looking over the association shows that never do two different domain elements map to the same codomain element. Similarly, it is also onto by inspection.

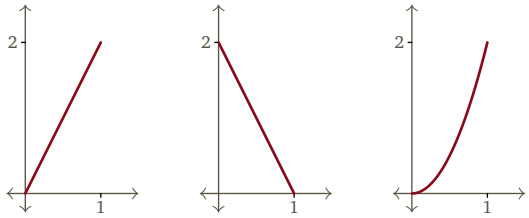


FIGURE 9, FOR QUESTION II.1.32: The correspondences $f_0, f_1, f_2 : (0\,..\,1) \to (0\,..\,2)$ given by $f_0(x) = 2x$, $f_1(x) = 2-2x$, and $f_2(x) = 2x^2$.

(B) The inverse map associates the same pairs of elements in the prior item's table, but the domain and codomain swap places so here A is the domain and C is the codomain.

y	48	49	50	51	52	53	54	55	56	57
x	0	1	2	3	4	5	6	7	8	9

As in the prior item, this map is one-to-one and onto by inspection.

II.1.31 For each pair of sets we could define a function whose domain is the first set, or a function whose domain is the second set. We choose whichever is convenient.

(A) Write the even numbers as $E = \{\,2n \mid n \in \mathbb{N}\,\}$. Consider the function $f : \mathbb{N} \to E$ defined by $f(m) = 2m$. To verify that f is one-to-one, assume that two inputs yield the same output $f(m) = f(\hat{m})$, giving $2m = 2\hat{m}$, and divide by 2 to get that therefore the two inputs are the same, $m = \hat{m}$.

To verify that f is onto, consider a member of the codomain $y \in E$, so that $y = 2n$ for some $n \in \mathbb{N}$. Now, $n = y/2$ has the property that $f(n) = y$. Note that n is an element of the domain $\mathbb{N}$ because as a member of the codomain, y is even, and thus dividing by two yields a natural number.

(B) Write the odd numbers as $M = \{\,2n+1 \mid n \in \mathbb{N}\,\}$. Consider $g : \mathbb{N} \to M$ given by $n \mapsto 2n+1$. To verify that g is one-to-one, assume that $g(m_0) = g(m_1)$, so that $2m_0 + 1 = 2m_1 + 1$, subtract 1, and divide by 2, to conclude that $m_0 = m_1$.

To verify that g is onto, start with an element of the codomain $y \in M$. Then it has the form $y = 2k+1$ for some natural number k. That means there is a natural number k that maps under g to y, namely $k = (y-1)/2$.

(C) One answer is to cite the two prior exercise parts and that the inverse of a correspondence is also a correspondence, to get that the composition $g \circ f^{-1}$ is a correspondence from the even numbers to the odds.

A direct answer is to consider the map $p : E \to M$ given by $p(n) = n+1$. Verify that p is one-to-one and onto as in the prior two items.

II.1.32 The natural correspondence $f_0 : (0\,..\,1) \to (0\,..\,2)$ is $f_0(x) = 2x$. For the other two there are many choices but two more correspondences $f_0, f_1, f_2 : (0\,..\,1) \to (0\,..\,2)$ are $f_1(x) = 2 - 2x$ and $f_2(x) = 2x^2$. Figure 9 on page 47 shows the three graphs. They are different functions because they return different values on the input $x = 1/3$.

Each function is one-to-one and onto because the figure shows that every horizontal line at $y \in (0\,..\,2)$ intercepts its graph in one and only one point.

For an algebraic verication that they are correspondences we can use f_1 as a model. It is one-to-one because if $f(x_0) = f(x_1)$ then $2 - 2x_0 = 2 - 2x_1$, and then subtracting 2 and dividing by -2 yields that $x_0 = x_1$. It is onto because if $y \in (0\,..\,2)$ then the number $x = (y-2)/(-2)$ is an element of $(0\,..\,1)$ and has the property that $f(x) = 2 - 2 \cdot ((y-2)/-2) = y$.

II.1.33 The function

$$\hat{f}(n) = \begin{cases} 1 & \text{– if } n = 0 \\ 0 & \text{– if } n = 1 \\ n^2 & \text{– if } n > 1 \end{cases}$$

is different than Example 1.8's function f because they give different outputs for the input 0. That $\hat{f}$ is onto is clear, as every perfect square is the output associated with some input. That $\hat{f}$ is a one-to-one function is also

clear, although it is a little messier to write down. Assume $\hat{f}(x_0) = \hat{f}(x_1)$ and then there are four cases: both x_0 and x_1 are members of the set $\{0, 1\}$, only the first one is a member, only the second is a member, or neither is a member. All four cases are easy.

II.1.34 To check that f is one-to-one suppose that $f(x_0) = f(x_1)$, so that $c^{x_0} = c^{x_1}$. Take the logarithm base c of both sides, giving $x_0 = x_1$.

For onto, consider a codomain element $y \in (0 \,..\, \infty)$. The number $x = \log_c(y)$ is defined and is an element of $\mathbb{R}$, the domain of f. Of course, $f(x) = f(\log_c(y)) = c^{\log_c(y)} = y$.

II.1.35 The two $P_2 = \{2^k \mid k \in \mathbb{N}\} = \{1, 2, 4, 8, \ldots\}$ and $P_3 = \{3^k \mid k \in \mathbb{N}\} = \{1, 3, 9, \ldots\}$ are subsets of $\mathbb{N}$. The natural correspondence $g\colon P_2 \to P_3$ is to associate elements having the same exponent, $g(2^k) = 3^k$. This function is one-to-one because if $g(2^{k_0}) = g(2^{k_1})$ then $3^{k_0} = 3^{k_1}$, and taking the logarithm base 3 of both sides shows that the inputs are the same, $k_0 = k_1$. This function is onto because if $y \in P_3$ then it has the form $y = 3^k$, and is the image of $2^k \in P_2$.

The obvious generalization is that if two natural numbers $a, b \in \mathbb{N}$ are greater than 1 then $P_a = \{a^k \mid k \in \mathbb{N}\}$ and $P_b = \{b^k \mid k \in \mathbb{N}\}$ are sets of natural numbers that have the same cardinality. The argument is as in the prior paragraph.

II.1.36 Of course, for each item there are many possible answers.

(A) One is $f_0\colon \mathbb{N} \to \mathbb{N}$ given by $f_0(n) = n + 1$. It is one-to-one because if $f_0(n) = f_0(\hat{n})$ then $n + 1 = \hat{n} + 1$ and so $n = \hat{n}$. It is not onto because no $n \in \mathbb{N}$ maps to 0.

A second one is $f_1\colon \mathbb{N} \to \mathbb{N}$ given by $f_1(n) = n^2$. It is one-to-one because if $f_1(n) = f_1(\hat{n})$ then $n^2 = \hat{n}^2$ and because these are natural numbers they are nonnegative, so $n = \hat{n}$. This function is not onto because no natural number maps to 2.

(B) This function

$$f_0(x) = \begin{cases} x/2 & \text{– if } x \text{ is even} \\ (x-1)/2 & \text{– if } x \text{ is odd} \end{cases} \qquad \begin{array}{r|cccccc} x & 0 & 1 & 2 & 3 & 4 & 5 & \ldots \\ \hline f_0(x) & 0 & 0 & 1 & 1 & 2 & 2 & \ldots \end{array}$$

is onto because every $y \in \mathbb{N}$ is the image of $x = 2y$. It is not one-to-one because $f_0(1) = f_0(0)$.

A second one is $f_1(x) = \lfloor\sqrt{x}\rfloor$ (recall that the floor operation, $\lfloor n \rfloor$, gives the largest integer less than or equal to n).

x	0	1	2	3	4	5	6	7	8	9	10	...
$f_1(x)$	0	1	1	1	2	2	2	2	2	3	3	...

It is onto because every $y \in \mathbb{N}$ is the image of $x = y^2$. It is not one-to-one because $f_1(1) = f_1(2)$.

(C) One such map is $f_0(x) = 0$. It is not one-to-one because $f_0(0) = f_0(1)$. It is not onto because no natural number input maps to 1.

A non-constant such function is the square of the prior item's second one, $f_1(x) = (\lfloor\sqrt{x}\rfloor)^2$.

x	0	1	2	3	4	5	6	7	8	9	10	...
$f_1(x)$	0	1	1	1	4	4	4	4	4	9	9	...

It is not one-to-one because $f_1(1) = f_1(2)$. It is not onto because no number maps to 2, as 2 is not a square.

(D) The natural function that is both one-to-one and onto is the identity function, $f_0(x) = x$. It is one-to-one because if $f_0(n) = f_0(\hat{n})$ then $n = \hat{n}$, just straight from the definition of f_0. It is onto because any codomain element $y \in \mathbb{N}$ is the image under f_0 of itself, $f_0(y) = y$.

Another correspondence swaps even and odd numbers.

$$f_1(x) = \begin{cases} x + 1 & \text{– if } x \text{ is even} \\ x - 1 & \text{– if } x \text{ is odd} \end{cases} \qquad \begin{array}{r|cccccc} x & 0 & 1 & 2 & 3 & 4 & 5 & \ldots \\ \hline f_1(x) & 1 & 0 & 3 & 2 & 5 & 4 & \ldots \end{array}$$

Observe first that $f_1\colon \mathbb{N} \to \mathbb{N}$; that is, if $x \in \mathbb{N}$ then $f_1(x) \in \mathbb{N}$. To check that f_1 is one-to-one suppose that $f_1(n) = f_1(\hat{n})$. There are two cases. If n is odd then $f_1(n) = n - 1$ is even, and so $f_1(\hat{n})$ is even also,

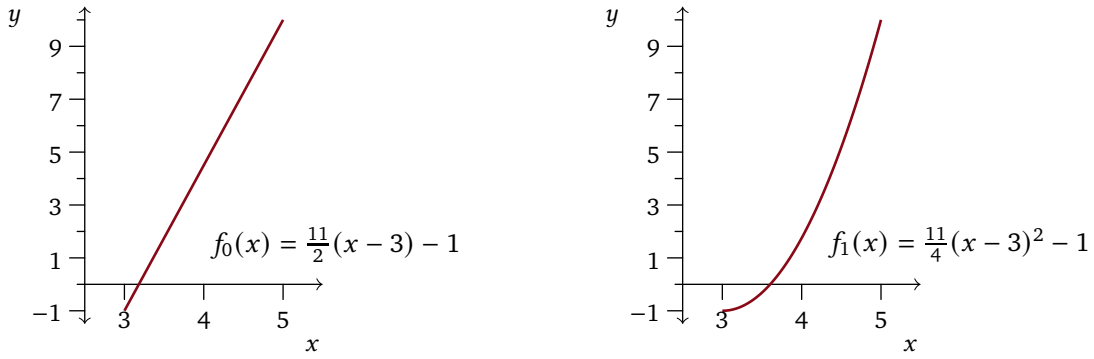


FIGURE 10, FOR QUESTION II.1.37: Two correspondences between $(3\,..\,5)$ and $(-1\,..\,11)$.

implying that $\hat{n}$ is odd, and therefore $n-1 = \hat{n}-1$, giving that $n = \hat{n}$. The even case is similar. To check that f_1 is onto consider a codomain point $y \in \mathbb{N}$. Here also there are two cases and we will only do the odd one, so suppose that y is odd. Then $y = f_1(x)$ where $x = y-1$, and x is an element of the domain $\mathbb{N}$ because if y is odd then $y - 1 \geq 0$.

II.1.37 Let $D = (3\,..\,5)$ and $C = (-1\,..\,10)$.

One correspondence is $f_0(x) = (11/2)(x-3) - 1$. Whether $f_0 : D \to C$ is not obvious so we start with that: if $3 < x < 5$ then subtracting 3, multiplying by $11/2$, and subtracting 1 from all terms gives $-1 < f_0(x) < 10$. Next we show that this map is onto. If $y \in C$ then $y = f_0(x)$ where $x = (2/11)(y+1) + 3$. To verify that this input x is an element of the domain D, start with $-1 < y < 10$, add 1 to all terms, multiply by $2/11$, and add 3, ending with $3 < x < 5$. Finally, we show that the map f_0 is one-to-one. If $f_0(x) = f_0(\hat{x})$ then $(11/2)(x-3) - 1 = (11/2)(\hat{x}-3) - 1$ and adding 1 to both sides, multiplying by $2/11$, and finally adding 3, gives that $x = \hat{x}$.

Another correspondence is $f_1(x) = (11/4)(x-3)^2 - 1$. As with f_0 we start by verifying that $f_1 : D \to C$: if $3 < x < 5$ then subtracting 3, squaring, multiplying by $11/4$, and subtracting 1 from all terms gives $-1 < f_1(x) < 10$. To show that f_1 is onto assume that $y \in C$. Then $y = f_1(x)$ where $x = \sqrt{(4/11)(y+1)} + 3$. To check that $x \in D$, start with $-1 < y < 10$ and add 1 to all terms, multiply by $4/11$, square (this doesn't change the direction of the $<$'s because all numbers are nonnegative), and add 3, ending with $3 < x < 5$. To show that this map is one-to-one, assume $f_1(x) = f_1(\hat{x})$ so that $(11/4)(x-3)^2 - 1 = (11/4)(\hat{x}-3)^2 - 1$. To both sides add 1, multiply by $4/11$, take the square root (which is unambiguous because the x's are all greater than zero) and then add 3, giving that $x = \hat{x}$.

Figure 10 on page 49 shows that the two functions are unequal. It also gives an alternative demonstration that each is a correspondence because for each, every horizontal line at $y \in (-1\,..\,10)$ intersects the graph one and only one point.

II.1.38 In each case we produce a function from one set to the other and verify that it is both one-to-one and onto. For the third item, note that just because a set appears first in the question does not mean that it must be the domain of the correspondence.

(A) Where $S_4 = \{4k \mid k \in \mathbb{N}\}$ and $S_5 = \{5k \mid k \in \mathbb{N}\}$, let $f : S_4 \to S_5$ be $f(x) = (5/4)x$. It is one-to-one because $f(x_0) = f(x_1)$ implies that $(5/4)x_0 = (5/4)x_1$, so $x_0 = x_1$. It is onto because if we consider a codomain element $y = 5k$ for some $k \in \mathbb{N}$ then clearly $y = f(4k)$.

(B) These two sets are the same set, that is, the contain the same numbers. They correspond via the identity function.

(C) Write $T = \{0, 1, 3, 6, \ldots\} = \{n(n+1)/2 \mid n \in \mathbb{N}\}$ (these are called the triangular numbers). Consider the function $f : \mathbb{N} \to T$ defined by $f(n) = n(n+1)/2$. This map is one-to-one because the continuous function $\hat{f} : \mathbb{R} \to \mathbb{R}$ given by $\hat{f}(x) = x(x+1)/2$ is a parabola with roots at 0 and -1 and so the part in the positive numbers, with $x \geq 0$, has that a horizontal line at y will intersect the graph in at most one point. The fact that $\hat{f}$ is one-to-one gives that its restriction f is also one-to-one. The function f is onto by definition, since T is defined as the range of that function.

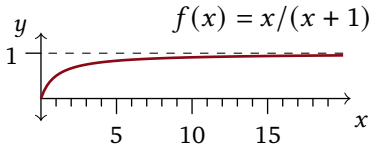


FIGURE 11, FOR QUESTION II.1.39: A correspondence between $(0\,..\,\infty)$ and $(0\,..\,1)$.

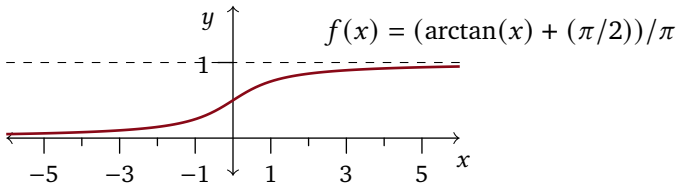


FIGURE 12, FOR QUESTION II.1.42: A correspondence between $\mathbb{R}$ and $(0\,..\,1)$.

II.1.39

(A) One is $f(x) = 2x$. This map is one-to-one because if $f(x) = f(\hat{x})$ then $2x = 2\hat{x}$ and so $x = \hat{x}$. This map is onto because where $0 < y < 2$, the number $x = y/2$ has the properties that $f(x) = y$ and $0 < x < 1$.

(B) One is $f(x) = (b-a)\cdot x + a$. Note that $f\colon (0\,..\,1) \to (a\,..\,b)$ since if $0 < x < 1$ then multiplying by $b-a$ gives $0 < x < (b-a)$ (the $<$'s maintain their direction because $a < b$), and adding a gives $a < x < b$. Further, this function is one-to-one: because $b - a \neq 0$ this line does not have slope 0 and so the function passes the horizontal line test (for an algebraic argument, start with $f(x) = f(\hat{x})$, subtract a and divide by $b-a$, and conclude that $x = \hat{x}$). Finally, to show that this function is onto, fix $y \in (a\,..\,b)$. Note that $f(x) = y$ where $x = (y-a)/(b-a)$. What remains is to check that $x \in (0\,..\,1)$: starting from $a < y < b$, subtract a and divide by $b-a$ to get $0 < (y-a)/(b-a) < 1$ (as earlier, note that the $<$'s maintain their direction because $b - a > 0$).

(C) One is $f(x) = x + a$. This function is one-to-one because $f(x) = f(\hat{x})$ implies that $x + a = \hat{x} + a$ and so $x = \hat{x}$. It is onto because if $y \in (a\,..\,\infty)$ then $y = f(x)$ where $x = y - a$ (and $x \in (0\,..\,\infty)$).

(D) Similar triangles gives $x/(x+1) = f(x)/1$. Figure 11 on page 50 shows that it is a correspondence because for each $y \in (0\,..\,1)$, the horizontal line at that height intercepts the function's graph in exactly one point.

II.1.40 Consider the function $f\colon \mathbb{N} \to S$ given by $f(m) = \sqrt[m+2]{2} = 2^{1/(m+2)}$. It is one-to-one because if $f(m) = f(\hat{m})$ then $2^{1/(m+2)} = 2^{1/(\hat{m}+2)}$. Take the logarithm of both sides $\lg(2^{1/(m+2)}) = \lg(2^{1/(\hat{m}+2)})$ to get $(1/(m+2))\cdot\lg(2) = (1/(\hat{m}+2))\cdot\lg(2)$, and so $1/(m+2) = 1/(\hat{m}+2)$. Thus $m = \hat{m}$.

This function is onto because if $y \in S$ then $y = 2^{1/n}$ for some $n \in \mathbb{N}$ with $n \geq 2$. Thus, $y = 2^{1/(x+2)} = f(x)$ for $x \in \mathbb{N}$.

II.1.41 Recall that just as every natural number has a finite decimal representation, and that this representation is unique, so also every natural number has a unique binary representation.

(A) View each string of bits σ starting with 1 as the binary representation of a natural number n. Then $f\colon B \to \mathbb{N}$ given by $\sigma \mapsto n - 1$ is clearly a correspondence.

(B) Prefix each string $\tau \in \mathbb{B}^*$ with 1, so $\sigma = 1^\frown\tau$. Now the prior argument applies.

II.1.42 The tangent function is a correspondence from $(-\pi/2\,..\,\pi/2)$ to $\mathbb{R}$ so its inverse, arctangent, is a correspondence in the other direction. Thus, $g(x) = (\arctan(x) + (\pi/2))/\pi$ gives a correspondence from $\mathbb{R}$ to $(0\,..\,1)$. See Figure 12 on page 50, which shows that g is one-to-one and onto because for each element y of the codomain $(0\,..\,1)$, the horizontal line at y intercepts the function's graph in exactly one point.

II.1.43

(A) The map $g\colon I_0 \to I_1$ given by $g(x) = x\cdot(r_1/r_0)$ is onto because if $y \in I_1 = [0\,..\,2\pi r_1)$ then $y = g(x)$ where $x = y\cdot(r_0/r_1)$, and $0 \leq y < 2\pi r_1$ implies that $0 \leq y\cdot(r_0/r_1) < 2\pi r_0$. To show that this map is one-to-one suppose $g(x) = g(\hat{x})$. Then $x\cdot(r_1/r_0) = \hat{x}\cdot(r_1/r_0)$ and so $x = \hat{x}$.

(B) The map $g(x) = a + x\cdot(b-a)$ is a correspondence. To verify that it is one-to-one, suppose that $g(x) = g(\hat{x})$

so that $a+x\cdot(b-a)=a+\hat{x}\cdot(b-a)$. Subtract a and divide by $b-a$ to conclude that $x=\hat{x}$. This map is also onto because $y\in[a\,..\,b)$ is the image of $x=(y-a)/(b-a)$, and $a\leq y<b$ implies that $0\leq y-a<b-a$, and so $0\leq(y-a)/(b-a)<1$.

(C) By the prior item there are correspondences $f\colon[0\,..\,1)\to[a\,..\,b)$ and $g\colon[0\,..\,1)\to[c\,..\,d)$. Then the composition $g\circ f^{-1}$ has domain $[a\,..\,b)$, codomain $[c\,..\,d)$, and is a correspondence.

(This is also easy to check by considering the map $f\colon[a\,..\,b)\to[c\,..\,d)$ given by $f(x)=c+(x-a)\cdot(d-c)/(b-a)$.)

II.1.44 Suppose for contradiction that f is not one-to-one. Then there are $x,\hat{x}\in D$ such that $f(x)=f(\hat{x})$ but $x\neq\hat{x}$. But one or the other of these two is smaller; without loss of generality suppose that $x<\hat{x}$. Then $x<\hat{x}$ but $f(x)=f(\hat{x})$, which contradicts that the function is strictly increasing.

Yes, the same argument works for $D\subseteq\mathbb{N}$.

II.1.45

(A) Let $\mathcal{E}=\{\,2n\;\big|\;n\in\mathbb{N}\,\}$. The natural correspondence is $f\colon\mathbb{N}\to\mathcal{E}$ given by $n\mapsto 2n$. This map is one-to-one because if $f(n)=f(\hat{n})$ then $2n=2\hat{n}$ and so $n=\hat{n}$. This map is onto because if $y\in\mathcal{E}$ then $y=2k$ for some $k\in\mathbb{N}$ and setting $n=k$ gives that $y=f(n)$.

(B) Let $T=\{\,n\in\mathbb{N}\;\big|\;n>4\,\}$. Consider $f\colon\mathbb{N}\to T$ given by $n\mapsto n+5$. This is one-to-one because if $f(n)=f(\hat{n})$ then $n+5=\hat{n}+5$ and $n=\hat{n}$. It is onto because if $y\in T$ then $y=f(x)$ where $x=y-5$, and observe that $y\in T$ implies that $y-5\in\mathbb{N}$. So f is a correspondence.

II.1.46 We will show that no matter what is the cardinality of the finite set $S\subset\mathbb{N}$, there is a correspondence $f\colon\mathbb{N}\to\mathbb{N}-S$. We will use induction, at each step taking the finite set to have one element more than at the prior step.

Before the proof we give an illustration. Suppose that we have a five-element set $S_5=\{2,5,8,10,11\}$ and this correspondence $f_5\colon\mathbb{N}\to\mathbb{N}-S_5$.

n	0	1	2	3	4	5	6	7	...
$f_5(n)$	0	1	3	4	6	7	9	12	...

Now we add a sixth element, so let $S=S_5\cup\{9\}$. We want a correspondence $f\colon\mathbb{N}\to\mathbb{N}-S$. The natural approach is find that $9=f_5(6)$ and define f in this way.

$$f(n)=\begin{cases}f_5(n) & \text{– if } n<6\\ f_5(n+1) & \text{– if } n>6\end{cases}$$

n	0	1	2	3	4	5	6	...
$f(n)$	0	1	3	4	6	7	12	...

The induction argument's base case is $S=\varnothing$ and here the identity function $f(n)=n$ is clearly a one-to-one and onto map from $\mathbb{N}$ to $\mathbb{N}-S$.

For the inductive step take $k\in\mathbb{N}$, assume that the statement is true for any subset of $\mathbb{N}$ whose cardinality is less than or equal to k, and consider $S\subset\mathbb{N}$ with $|S|=k+1$. Fix any $s\in S$ and let $S_k=S-\{s\}$. By the inductive hypothesis there is a correspondence $f_k\colon\mathbb{N}\to\mathbb{N}-S_k$. Set $\hat{n}$ to be such that $f_k(\hat{n})=s$ and define $f\colon\mathbb{N}\to\mathbb{N}-S$ in this way.

$$f(n)=\begin{cases}f_k(n) & \text{– if } n<\hat{n}\\ f_k(n+1) & \text{– if } n>\hat{n}\end{cases}$$

The function $f\colon\mathbb{N}\to\mathbb{N}-S$ is one-to-one because it is a restriction of f_k, which is a one-to-one map from $\mathbb{N}$ to $\mathbb{N}-\hat{S}$. Similarly, f is onto because f_k is onto, from $\mathbb{N}$ to $\mathbb{N}-S_k$. Thus f is a correspondence between $\mathbb{N}$ and $\mathbb{N}-S$, and so the two have the same cardinality.

II.1.47

(A) The inverse function $f^{-1}\colon C\to D$ is given by $f^{-1}(\text{Spades})=0$, $f^{-1}(\text{Hearts})=1$, $f^{-1}(\text{Clubs})=2$, and $f^{-1}(\text{Diamonds})=3$. By inspection it is both one-to-one and onto.

(B) Suppose that $f\colon D\to C$ is a correspondence. Consider the binary relation, the set of ordered pairs, $f^{-1}=\{\,\langle y,x\rangle\;\big|\;f(x)=y\,\}$. We will show that f^{-1} is a function, which means that we will show that it is well-defined, that each input $y\in C$ is associated with one and only one output $x\in D$.

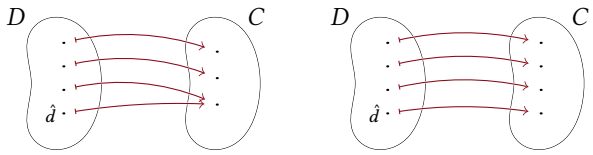


FIGURE 13, FOR QUESTION II.1.49: A finite function's domain has at least as many elements as its range.

Each $y \in C$ is associated with at least one x because the function f is onto. Each $y \in C$ is associated with at most one x because f is one-to-one.

(C) To show that $f^{-1}: C \to D$ is one-to-one suppose that $f^{-1}(y) = f^{-1}(\hat{y})$. Then there is a $x \in D$ such that $f(x) = y$ and $f(x) = \hat{y}$. This contradicts that f is a function, because a function must be well-defined.

To show that f^{-1} is onto consider a member of its codomain, $\hat{x} \in D$. By the definion of a function there is a $\hat{y} \in C$ with $f(\hat{x}) = \hat{y}$, and then $\hat{x} = f^{-1}(\hat{y})$.

II.1.48 One direction is straightforward: no finite set has the same cardinality as a proper subset of itself, by Lemma 1.5.

For the other direction, fix an infinite set S, to show that it has the same cardinality as a proper subset of itself. (*Remark:* as elsewhere, we feel free here to use the Axiom of Choice.) Choose any element $s_0 \in S$. Then $S - \{s_0\}$ is not empty (or else S is not infinite) so choose $s_1 \in S - \{s_0\}$. Continuing in this way gives $\{s_0, s_1, s_2 \ldots\}$. Then this is a correspondence from S to $S - \{s_0\}$.

$$f(x) = \begin{cases} s_{i+1} & \text{– if } x = s_i \text{ for some } i \in \mathbb{N} \\ x & \text{– otherwise} \end{cases}$$

II.1.49 Denote the function $f: D \to C$, and assume that D is finite.

(A) We will argue that the statement $|\text{dom}(f)| \geq |\text{ran}(f)|$ holds for all functions on finite domains by induction on the number of elements in D.

The base step is that the domain has no elements at all, $\text{dom}(f) = \varnothing$. The only function with an empty domain is the empty function. The empty function's range is empty, $\text{ran}(f) = \varnothing$, and thus $|\text{dom}(f)| \geq |\text{ran}(f)|$.

For the inductive step, fix $k \in \mathbb{N}$, assume that the statement is true for any function with a domain having 0 elements, or 1 elements, ... or k elements, and consider the $|D| = k + 1$ case.

Because $k + 1 > 0$ there is a $\hat{d} \in D$. Let $\hat{D} = D - \{\hat{d}\}$ and consider the restriction $f\restriction_{\hat{D}}: \hat{D} \to C$. Its domain has k many elements, so the induction hypothesis applies, giving $|\text{dom}(f\restriction_{\hat{D}})| \geq |\text{ran}(f\restriction_{\hat{D}})|$

To add back the element $\hat{d}$ there are two cases; see Figure 13 on page 52. The first case, on the left in the figure, is that the range of f is the same as the range of the restriction $f\restriction_{\hat{D}}$. The other case is that the range of f has one additional element. In either case, adding 1 to both sides ends the argument.

$$|\text{dom}(f)| = |\text{dom}(f\restriction_{\hat{D}})| + 1 \geq |\text{ran}(f\restriction_{\hat{D}})| + 1 \geq |\text{ran}(f)|$$

(B) Assume that the function is one-to-one. We will show that the domain and range have the same size by induction on the number of elements in the domain. This argument is like the prior item's except that it has only the single case shown on the right of Figure 13 on page 52.

The base step is that the function's domain has no elements at all, $\text{dom}(f) = \varnothing$. This function's range is empty and $|\text{dom}(f)| = |\text{ran}(f)|$.

For the inductive step, fix $k \in \mathbb{N}$, assume that the domain and range have the same size for any one-to-one function where the domain D has 0 elements, or 1 element, ... or k elements, and suppose that $|D| = k + 1$.

Because $k + 1 > 0$ there is a $\hat{d} \in D$. Let $\hat{D} = D - \{\hat{d}\}$. Consider the restriction function $f\restriction_{\hat{D}}: \hat{D} \to C$. The function f is one-to-one so the restriction is also one-to-one. The induction hypothesis applies, giving $|\text{dom}(f\restriction_{\hat{D}})| = |\text{ran}(f\restriction_{\hat{D}})|$. Adding 1 to both sides gives $|\text{dom}(f)| = |\text{dom}(f\restriction_{\hat{D}})| + 1 = |\text{ran}(f\restriction_{\hat{D}})| + 1 = |\text{ran}(f)|$.

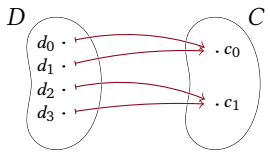


FIGURE 14, FOR QUESTION II.1.49: A finite function that is not one-to-one has more elements in its domain than its range.

(C) Figure 14 on page 53 illustrates the argument.

For the proof, let f be a function with a finite domain that is not one-to-one. Fix some numering of its domain, $D = \{d_0, d_1, \ldots d_n\}$. Consider

$$\hat{D} = \{ d_i \in D \;\big|\; i \text{ is minimal among the indices } j \text{ where } f(d_j) = f(d_i) \}$$

(in the illustration, $f(d_0) = f(d_1)$ and $f(d_2) = f(d_3)$) so this is the set $\{d_0, d_2\}$).

Because f is not one-to-one, $\hat{D}$ is a proper subset of D. The restriction $f\restriction_{\hat{D}} : \hat{D} \to C$ is one-to-one. By the prior item, its range has the same number of elements as its domain. The range of f is the same as the range of $f\restriction_{\hat{D}}$. But the domain of f has more elements than the domain of $f\restriction_{\hat{D}}$. Thus the domain of f contains more elements than its range.

(D) Let D and C be finite sets. If there is a correspondence $f : D \to C$ then the prior items show that the sets have the same number of elements.

For the other direction assume that the two have the same number of elements. Fix an ordering of each, $D = \{d_0, \ldots d_k\}$ and $C = \{c_0, \ldots c_k\}$. Then the map $f : D \to C$ defined by $f(d_i) = c_i$ is clearly a correspondence.

II.2.19 The table alternates between column 0 and column 1.

$n \in \mathbb{N}$	6	7	8	9	10	11	12
$f(n) \in \{0,1\} \times \mathbb{N}$	$\langle 0,3\rangle$	$\langle 1,3\rangle$	$\langle 0,4\rangle$	$\langle 1,4\rangle$	$\langle 0,5\rangle$	$\langle 1,5\rangle$	$\langle 0,6\rangle$

The formulas are $x(n) = (1 + (-1)^{n+1})/2$ and $y(n) = \lfloor n/2 \rfloor$.

II.2.20

(A) The pair before $\langle 50, 50\rangle$ is $\langle 49, 51\rangle$ and the pair after is $\langle 51, 49\rangle$. As a check they correspond to cantor$(50, 50) = 5\,100$, cantor$(49, 51) = 5\,099$, and cantor$(51, 49) = 5\,101$.

(B) The pair before $\langle 100, 4\rangle$ is $\langle 99, 5\rangle$ and the pair after is $\langle 101, 3\rangle$. As a check, the three correspond to cantor$(100, 4) = 5\,560$, cantor$(99, 5) = 5\,559$, and cantor$(101, 3) = 5\,561$.

(C) The pair before $\langle 4, 100\rangle$ is $\langle 3, 101\rangle$ and the pair after is $\langle 5, 99\rangle$. The three give cantor$(4, 100) = 5\,464$, cantor$(3, 101) = 5\,463$, and cantor$(5, 99) = 5\,465$.

(D) Before $\langle 0, 200\rangle$ is $\langle 199, 0\rangle$ while after is $\langle 1, 199\rangle$. These three correspond to cantor$(0, 200) = 20\,100$, cantor$(199, 0) = 20\,099$, and cantor$(1, 199) = 20\,101$.

(E) Before $\langle 200, 0\rangle$ is $\langle 199, 1\rangle$ while after is $\langle 0, 201\rangle$. They correspond to cantor$(200, 0) = 20\,300$, cantor$(199, 1) = 20\,299$, and cantor$(0, 201) = 20\,301$.

II.2.21

(A) The natural correspondence $f_T : \mathbb{N} \to T$ is $f_T(n) = 2n$. The natural correspondence $f_F : \mathbb{N} \to T$ is given by $f_F(m) = 5m$.

To verify that f_T is one-to-one start with $f_T(n_0) = f_T(n_1)$, which is $2n_0 = 2n_1$, and cancelling gives $n_0 = n_1$. To verify that f_T is onto take $y \in T$. Then $y = 2k$ by definition, and so $y = f_T(k)$. Checking that f_N is one-to-one and onto works the same way.

$n \in \mathbb{N}$	0	1	2	3	4	5	6	7	8	9
$f_T(n) \in T$	0	2	4	6	8	10	12	14	16	18
$f_F(n) \in F$	0	5	10	15	20	25	30	35	40	45

(B) A natural correspondence gives outputs in ascending order.

$n \in \mathbb{N}$	0	1	2	3	4	5	6	7	8	9
$f(n) \in T \cup F$	0	2	4	5	6	8	10	12	14	15

II.2.22 This gives the first few associations for a function $f : \mathbb{N} \to \mathbb{N} \times \{0, 1\}$.

n	0	1	2	3	4	5	...
$f(n)$	$\langle 0, 0\rangle$	$\langle 0, 1\rangle$	$\langle 1, 0\rangle$	$\langle 1, 1\rangle$	$\langle 2, 0\rangle$	$\langle 2, 1\rangle$	...

From that table a description sufficient for computing f is easy.

$$f(n) = \begin{cases} \langle \lfloor n/2 \rfloor, 0\rangle & \text{– if } n \text{ is even} \\ \langle \lfloor n/2 \rfloor, 1\rangle & \text{– if } n \text{ is odd} \end{cases}$$

That gives $f(0) = \langle 0, 0\rangle$, $f(10) = \langle 5, 0\rangle$, $f(100) = \langle 50, 0\rangle$, and $f(101) = \langle 50, 1\rangle$.

The above table makes it easy to find the formula for the inverse, $f^{-1}(i, j) = 2i + j$. Then $f^{-1}(2, 1) = 5$, $f^{-1}(20, 1) = 41$, and $f^{-1}(200, 1) = 401$.

II.2.23 Members of $\{0, 1, 2\} \times \mathbb{N}$ are pairs $\langle x, y\rangle$ where the first entry is either 0, or 1, or 2, and the second entry is a natural number. This gives the first few of the associations under the obvious correspondence.

Number	0	1	2	3	4	5	6	7	8	...
Pair	$\langle 0, 0\rangle$	$\langle 1, 0\rangle$	$\langle 2, 0\rangle$	$\langle 0, 1\rangle$	$\langle 1, 1\rangle$	$\langle 2, 1\rangle$	$\langle 0, 2\rangle$	$\langle 1, 2\rangle$	$\langle 2, 2\rangle$	...

The formula for the top to bottom association is this.

$$f(n) = \begin{cases} \langle 0, \lfloor n/3 \rfloor\rangle & \text{– if 3 divides } n \\ \langle 1, \lfloor n/3 \rfloor\rangle & \text{– if } n \text{ mod 3 is 1} \\ \langle 2, \lfloor n/3 \rfloor\rangle & \text{– if } n \text{ mod 3 is 2} \end{cases}$$

In short: $f(n) = \langle n \bmod 3, \lfloor n/3 \rfloor\rangle$. Thus $f(0) = \langle 0, 0\rangle$, $f(10) = \langle 1, 3\rangle$, $f(100) = \langle 1, 33\rangle$, and $f(1\,000) = \langle 1, 333\rangle$. The formula for the bottom to top function $f^{-1} : \{0, 1, 2\} \times \mathbb{N} \to \mathbb{N}$ is also easy from the above table: $f^{-1}(x, y) = x + 3y$. With that, $f^{-1}(1, 2) = 7$, $f^{-1}(1, 20) = 61$, and $f^{-1}(1, 200) = 601$.

II.2.24 This is the initial part of an enumeration f of $\{0, 1, 2, 3\} \times \mathbb{N}$.

Number	0	1	2	3	4	5	6	7	8	...
Pair	$\langle 0, 0\rangle$	$\langle 1, 0\rangle$	$\langle 2, 0\rangle$	$\langle 3, 0\rangle$	$\langle 0, 1\rangle$	$\langle 1, 1\rangle$	$\langle 2, 1\rangle$	$\langle 3, 1\rangle$	$\langle 0, 2\rangle$	...

The formula for the top to bottom association is $f(n) = \langle n \bmod 4, \lfloor n/4 \rfloor\rangle$, so $x(n) = n \bmod 4$ and $y(n) = \lfloor n/4 \rfloor$.

In general, for $k \in \mathbb{N}$ an enumeration f of $\{0, 1, 2, \ldots k\} \times \mathbb{N}$ is $f(n) = \langle n \bmod (k + 1), \lfloor n/(k + 1) \rfloor\rangle$.

II.2.25 Example 2.4's table shows up to $n = 6$.

Number	7	8	9	10	11	12	13	14	15	16
Pair	$\langle 1, 2\rangle$	$\langle 2, 1\rangle$	$\langle 3, 0\rangle$	$\langle 0, 4\rangle$	$\langle 1, 3\rangle$	$\langle 2, 2\rangle$	$\langle 3, 1\rangle$	$\langle 4, 0\rangle$	$\langle 0, 5\rangle$	$\langle 1, 4\rangle$

II.2.26 One way to compute this function is by brute force. Given n, a program can generate the pairs in ascending order $\langle 0, 0\rangle$, $\langle 0, 1\rangle$, $\langle 1, 0\rangle$, $\langle 0, 2\rangle$, ... , keeping count until it generates the matching one. Figure 15 on page 55 has Racket code to do this.

II.2.27 Corollary 2.13 gives that, because the set $A - B$ is a subset of A, it is countable. Similarly $B - A \subseteq B$ is countable. Then, by the same result, their union is also countable.

II.2.28 Where $S = \{a + bi \mid a, b \in \mathbb{Z}\}$, obviously the map $g : \mathbb{Z} \times \mathbb{Z} \to S$ given by $g(a, b) = a + bi$ is a correspondence. Lemma 2.8 says that $\mathbb{Z} \times \mathbb{Z}$ is countable so there is a correspondence $f : \mathbb{N} \to \mathbb{Z} \times \mathbb{Z}$. Then the composition $g \circ f : \mathbb{N} \to S$ is a correspondence.

II.2.29 The table below walks through the algorithm of that example. The second column gives the pairs in ascending order. The third column lists the rational number result of the search loop. For example, loop number $n = 1$ first generates $\langle 1, 0\rangle$ and rejects it since the denominator is zero, then it generates $\langle 0, 2\rangle$ and rejects it as the rational number $0 = 0/2$ has already been enumerated, and then it generates $\langle 1, 1\rangle$ and since $1 = 1/1$ is a rational that is so-far not enumerated, we have $f(1) = 1$.

```
;; Return the pair following p in Cantor's correspondence
(define (next-pair p)
  (let ((x (first p))
        (y (second p)))
    (if (= y 0)
        (list 0 (+ x 1))
        (list (+ x 1) (- y 1)))))

;; Find pair n by brute force
(define (brute-force-pairing n)
  (define (bfp-helper j pr)
    (if (zero? j)
        pr
        (bfp-helper (sub1 j) (next-pair pr))))

  (bfp-helper n (list 0 0)))
```

FIGURE 15, FOR QUESTION II.2.26: Racket code that inputs a number and finds the pair by brute force.

n	*pairs*	$f(n) \in \mathbb{Q}$
0	$\langle 0,0\rangle$, $\langle 0,1\rangle$	0
1	$\langle 1,0\rangle$, $\langle 0,2\rangle$, $\langle 1,1\rangle$	1
2	$\langle 2,0\rangle$, $\langle 0,3\rangle$, $\langle 1,2\rangle$	$1/2$
3	$\langle 2,1\rangle$	2
4	$\langle 3,0\rangle$, $\langle 0,4\rangle$, $\langle 1,3\rangle$	$1/3$
5	$\langle 2,2\rangle$, $\langle 3,1\rangle$	3
6	$\langle 4,0\rangle$, $\langle 0,5\rangle$, $\langle 1,4\rangle$	$1/4$
7	$\langle 2,3\rangle$	$2/3$
8	$\langle 3,2\rangle$	$3/2$
9	$\langle 4,1\rangle$	4
10	$\langle 5,0\rangle$, $\langle 0,6\rangle$, $\langle 1,5\rangle$	$1/5$
11	$\langle 2,4\rangle$, $\langle 3,3\rangle$, $\langle 4,2\rangle$, $\langle 5,1\rangle$	5

II.2.30

(A) The set $S - T$ is a subset of a countable set, so it is countable.

(B) The set $S - T$ cannot be finite, or else $T \cup (S - T) = S$ would be the union of two finite sets and therefore finite.

(C) Yes, the set $\mathbb{N}$ is countably infinite and the set of even numbers $2\mathbb{N} \subseteq \mathbb{N}$ is also infinite, and $\mathbb{N} - 2\mathbb{N}$, the set of odd numbers, is infinite.

II.2.31 An infinite set contains some a_0. Because it is infinite it has more than one element so it also contains some $a_1 \neq a_0$. Proceeding in this way we get the countably infinite set $\{a_0, a_1, \ldots\}$. (If this is a subset of $\mathbb{N}$ then we could make the choice of a_1 definite by taking it to be the smallest element that is larger than a_0, and proceed in that way.)

II.2.32

(A) The map $f\colon \mathbb{Z}^{n+1} \to \mathbb{Z}_n[x]$ given by $f(a_0, \ldots a_n) = a_n x^n + \cdots + a_1 x + a_0$ is clearly a correspondence. (We are not claiming that it is clear that different polynomials are different as functions, we are only claiming that they are different as polynomials.) Since Corollary 2.9 says that there is a correspondence $g\colon \mathbb{N} \to \mathbb{Z}^{n+1}$, we have a correspondence $f \circ g$ from $\mathbb{N}$ to $\mathbb{Z}_n[x]$.

(B) The set $\mathbb{Z}[x]$ is the union of the $\mathbb{Z}_i[x]$ over $i \in \mathbb{N}$, and the union of countably many countable sets is countable.

II.2.33 Let $f\colon \mathbb{N} \to S$ be a correspondence. Where we write $f(n)$ as s_n, the map $g\colon S \to S$ given by $s_n \mapsto s_{n+1}$ is one-to-one but not onto.

II.2.34 We will use that the function cantor$\colon \mathbb{N}^2 \to \mathbb{N}$ is a correspondence.

To verify that cantor$_3$ is onto, fix a codomain element $n \in \mathbb{N}$. Because cantor is onto there is a pair $\langle w, z\rangle \in \mathbb{N}^2$ such that cantor$(w, z) = n$. Also because cantor is onto, there is a pair $\langle x, y\rangle \in \mathbb{N}^2$ so that cantor$(x, y) = w$. Then cantor$_3(x, y, z) =$ cantor(cantor$(x, y), z)) = n$.

What's left is to verify that cantor_3 is one-to-one. Suppose that $\text{cantor}_3(a_0,b_0,c_0)=\text{cantor}_3(a_1,b_1,c_1)$. Then $\text{cantor}(\text{cantor}(a_0,b_0),c_0)=\text{cantor}(\text{cantor}(a_1,b_1),c_1)$. The function cantor is one-to-one so the prior sentence implies that $\text{cantor}(a_0,b_0)=\text{cantor}(a_1,b_1)$ and $c_0=c_1$. Again applying that cantor is one-to-one gives that $a_0=a_1$, $b_0=b_1$, along with the $c_0=c_1$.

II.2.35 A way to be confident about these is to write a small script.

```
sage: def cantor(x,y):
....:     return x+ ( (x+y)*(x+y+1)/2 )
....:
sage: def cantor3(x,y,z):
....:     return cantor(cantor(x,y),z)
....:
```

(A) $c_3(0,0,0)=0$, $c_3(1,2,3)=62$, $c_3(3,3,3)=402$

(B) $c_3(0,0,0)=0$, $c_3(0,0,1)=1$, $c_3(0,1,0)=2$, $c_3(0,0,2)=3$, $c_3(0,1,1)=4$, $c_3(1,0,0)=5$

(C) Substituting gives this.

$$\begin{aligned}c_3(x,y,z)&=\left(x+\frac{(x+y)(x+y+1)}{2}\right)+\left(\frac{\left(x+\frac{(x+y)(x+y+1)}{2}+z\right)\left(x+\frac{(x+y)(x+y+1)}{2}\right)+z+1}{2}\right)\\&=\frac{1}{8}\cdot\big((x+y+1)(x+y)+2x+2z+2\big)\cdot\big((x+y+1)(x+y)+2x+2z\big)\\&\quad+\frac{1}{2}\cdot(x+y+1)(x+y)+x\end{aligned}$$

That last is computed by machine, not by hand.

```
sage: x,y,z = var('x,y,z')
sage: cantor3(x,y,z)
1/8*((x + y + 1)*(x + y) + 2*x + 2*z + 2)*((x + y + 1)*(x + y) + 2*x + 2*z) + 1/2*(x + y + 1)*(x + y) + x
```

II.2.36 Plug $x=y=i$ into $\text{cantor}(x,y)=x+(x+y)(x+y+1)/2$ to get $\text{cantor}(i,i)=i+(2i)(2i+1)/2=2i^2+2i$. That's $2i(i+1)$. It is obviously a multiple of 2 but because the numbers i and $i+1$ are consecutive, one of them contributes another factor of 2. Hence the entire expression is a multiple of 4.

II.2.37 Fix b. For $n\in\mathbb{N}$, let S_N be the set of sequences that are equivalent to b because the two are equal from place N on. This set is finite because the only way some $\hat{b}\in S_N$ varies from b is in the first N places, so the set S_N has 2^N elements. (For example, if $b=\langle 0,1,0,1,0,1,\ldots\rangle$ and $N=1$ then the only element of S_N other than b is $\hat{b}=\langle 1,1,0,1,0,1,\ldots\rangle$.) Therefore the total number of equivalent sequences is the union of the finite sets S_N over the countably many N.

II.2.38 Let S_0, S_1, and S_2 be countably infinite and suppose that $f_0\colon\mathbb{N}\to S_0$, $f_1\colon\mathbb{N}\to S_1$ and $f_2\colon\mathbb{N}\to S_2$ are onto. Then $\hat{f}\colon\mathbb{N}\to S_0\cup S_1\cup S_2$ is given by

$$\hat{f}(n)=\begin{cases}f_0(n/3) & \text{– if } n \text{ is a multiple of 3, that is, } n \bmod 3=0\\ f_1((n-1)/3) & \text{– if } n \bmod 3=1\\ f_2((n-2)/3) & \text{– if } n \bmod 3=2\end{cases}$$

(briefly, $\hat{f}(n)=f_{n \bmod 3}(\lfloor n/3\rfloor)$). This tabular illustration

n	0	1	2	3	4	5	6	...
$\hat{f}(n)$	$f_0(0)$	$f_1(0)$	$f_2(0)$	$f_0(1)$	$f_1(1)$	$f_2(1)$	$f_3(0)$	...

shows why the function is onto. Lemma 2.12 applies, and so the union is countable.

II.2.39 A function $f\colon\{0,1\}\to\mathbb{N}$ is two pieces of information, $f(0)$ and $f(1)$. So we can take it to be a pair $\langle f(0),f(1)\rangle$. Restated, there is a natural correspondence between the set of functions $\{f\mid f\colon\{0,1\}\to\mathbb{N}\}$ and the set of pairs $\mathbb{N}\times\mathbb{N}$, the latter of which we know to be countable.

II.2.40 Because S is countable, Lemma 2.12 says that either it is empty or there is an onto function $g\colon\mathbb{N}\to S$. If S is empty then the range set is empty, so it is countable. Otherwise, the function $f\circ g\colon\mathbb{N}\to\text{ran}(f)$ is onto.

II.2.41 The Cartesian product of two finite sets is finite because the number of elements in the Cartesian product equals the product of the number of elements in the sets.

Next consider a finite set, $S = \{s_0, \ldots s_{m-1}\}$, and a countably infinite set, $T = \{t_0, t_1, \ldots\}$. We will show that $S \times T$ is countable (the $T \times S$ argument is similar). Enumerate it row-wise, so that where $n = d \cdot m + r$ is the usual quotient-remainder relationship then $f(n) = \langle s_r, t_d \rangle$. (For illustration, this initial portion of the $m = 3$ case

n	0	1	2	3	4	5	6	...
$f(n)$	$\langle s_0, t_0 \rangle$	$\langle s_1, t_0 \rangle$	$\langle s_2, t_0 \rangle$	$\langle s_0, t_1 \rangle$	$\langle s_1, t_1 \rangle$	$\langle s_2, t_1 \rangle$	$\langle s_0, t_2 \rangle$	...

has the first entries cycle $s_0, s_1, s_2, s_0, \ldots$ while the second entries ratchet up through the infinite set.) Clearly this function is a correspondence.

II.2.42

(A) The set of length 5 strings $\Sigma^5 = \{\langle s_0, \ldots s_4 \rangle \mid s_i \in \Sigma\}$ is the Cartesian product of five finite sets, each with 40 characters. So Σ^5 has 40^5 many members. (By the way, $40^5 = 102\,400\,000$.) Thus it is countable.

(B) The set $\Sigma^0 \cup \Sigma^1 \cup \cdots \cup \Sigma^5$ is the union of finitely many finite sets. So it is finite and therefore countable.

(C) The set $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \cdots$ is the union of countably many finite sets. By Corollary 2.13 it is countable.

(D) A program is a finite string. So the set of all programs is a subset of Σ^*. Corollary 2.13 says that this set is countable.

II.2.43

(A) These are easy to calculate by hand or with a script.

	$m = 0$	$m = 1$	$m = 2$	$m = 3$
$n = 0$	0	2	4	6
$n = 1$	1	5	9	13
$n = 2$	3	11	19	27
$n = 3$	7	23	39	55

(B) Every natural number greater than zero is a unique product of primes, $2^{e_2}3^{e_3} \cdots p^{e_p}$. So every such number is the product of a maximal power of two, 2^n, with an odd number, $2m+1$. Thus the pair $\langle n, m \rangle$ corresponds to $2^n \cdot (2m + 1)$, and therefore the map g with domain $\mathbb{N} \times \mathbb{N}$ given by $g(n, m) = 2^n \cdot (2m + 1) - 1$ is a correspondence, with range $\mathbb{N}$.

(C) This table shows that under the box enumeration $\langle 3, 4 \rangle$ corresponds with 19.

$y = 4$	16	17	18	19	20
$y = 3$	9	10	11	12	21
$y = 2$	4	5	6	13	22
$y = 1$	1	2	7	14	23
$y = 0$	0	3	8	15	24
	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$

II.2.44 Here is an onto map from $\mathbb{N}$ to $\mathbb{Q}$. Given a natural number input n, factor out the first three primes to get $n = 2^{e_2}3^{e_3}5^{e_5} \cdot k$ where k is not divisible by 2, 3, or 5. If e_2 is odd then the output is $-e_3/(e_5 + 1)$ while if e_2 is even then the output is $e_3/(e_5 + 1)$.

II.2.45

(A) $\text{cantor}(2, 1) = 8$

(B) Solving $8 = 1 + [(1 + y)(1 + y + 1)]/2$ leads to the quadratic equation $0 = y^2 - 3x - 12$. The quadratic formula gives $(-3 \pm \sqrt{57})/2$, two different solutions, $y \approx 2.27$ and $y \approx -5.27$. Figure 16 on page 58 shows the graph.

II.3.10 Infinite is different than exhaustive. The list $0, 2, 4, \ldots$ is infinite but does not exhaust all of the natural numbers.

II.3.11 The union is not the entire set of real numbers. It does not include irrational numbers. For instance, $\pi = 3.14\ldots$ is not in any of the sets and so is not in the union. You classmate is counting only numbers that end in zeros.

II.3.12 Cantor's Theorem says that the power set has more elements than the set.

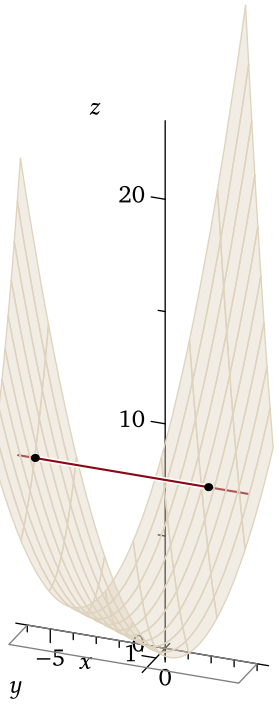


FIGURE 16, FOR QUESTION II.2.45: Cantor's unpairing function, with the line where $x = 1$ and $z = 8$.

(A) The set $\{0, 1, 2\}$ has three elements. Its power set has eight.

$$\mathscr{P}(\{0,1,2\}) = \{\{0,1,2\},\ \{0,1\},\ \{0,2\},\ \{1,2\},\ \{0\},\ \{1\},\ \{2\},\ \varnothing\}$$

(B) The set $\{0, 1\}$ has two elements while its power set $\mathscr{P}(\{0,1\}) = \{\{0,1\}, \{0\}, \{1\}, \varnothing\}$ has four.
(C) The set $\{0\}$ only has one element but its power set $\mathscr{P}(\{0\}) = \{\{0\}, \varnothing\}$ has two.
(D) The empty set has zero elements, while its power set $\mathscr{P}\varnothing = \{\varnothing\}$ has one.

II.3.13 The definition of '$\leq$' for cardinality is that $|A| \leq |B|$ if there is a one-to-one function from A to B.
(A) The function $f\colon S \to \hat{S}$ given by $f(1) = 11$, $f(2) = 12$, and $f(3) = 13$ is one-to-one by inspection.
(B) The function $f\colon T \to \hat{T}$ given by $f(1) = 11$, $f(2) = 12$, and $f(3) = 13$ is one-to-one by inspection.
(C) Again by inspection, the function $f\colon U \to \mathbb{O}$ where $f(1) = 1$, $f(2) = 3$, and $f(3) = 5$ is one-to-one.
(D) The function $f\colon \mathbb{E} \to \mathbb{O}$ given by $f(x) = x + 1$ is a one-to-one function. The verification is easy: $f(x_0) = f(x_1)$ implies that $x_0 + 1 = x_1 + 1$, and so $x_0 = x_1$.

II.3.14 The two differ on the set from which the elements are drawn.
(A) The set $S = \{n \in \mathbb{Z} \,\big|\, n + 3 < 5\} = \{\ldots, -2, -1, 0, 1\}$ is countable. The function $f\colon \mathbb{N} \to S$ defined by $n \mapsto -n + 1$ is a correspondence, as is easy to check.
(B) Uncountable; it is the set of real numbers $(-\infty \,..\, 2)$. (This set corresponds to $\mathbb{R}$ under the map $x \mapsto \ln(2-x)$.)

II.3.15
(A) Uncountable. (The function $x \mapsto \ln(x - 5)$ is a correspondence with $\mathbb{R}$.)
(B) Countable. (Because it is given as a subset of the naturals, and also because this set is $\{1, 2, 3\}$, which is finite.)
(C) This set is uncountable. (The set $S = [1\,..\,4) \subset \mathbb{R}$ has at least the same cardinality as $T = (1\,..\,4) \subset \mathbb{R}$ since the inclusion map $\iota\colon T \to S$ given by $x \mapsto x$ is one-to-one. The interval T has the same cardinality as $\mathbb{R}$. One way to see that is to start with the correspondence between the interval $(-\pi/2\,..\,\pi/2)$ and $\mathbb{R}$ given by $x \mapsto \tan(x)$. Adapt it to the case of domain T by resizing that interval by a factor of $\pi/3$ and then shifting it to get that $x \mapsto \tan(\,(\pi/3)\cdot(x-1) - (\pi/2)\,)$ is a correspondence.)
(D) Countable. (Any subset of $\mathbb{N}$ is countable.)

II.3.16 For the two element set $A_2 = \{0,1\}$, the power set, $\mathscr{P}(A_2) = \{\{0,1\},\{0\},\{1\},\{\}\}$, has four elements. Making a function from A_2 to $\mathscr{P}(A_2)$ involves picking where to send 0 and where to send 1. So there are $4^2 = 16$ different functions.

function f_i	$f_i(0)$	$f_i(1)$
f_0	$\{\}$	$\{\}$
f_1	$\{\}$	$\{0\}$
f_2	$\{\}$	$\{1\}$
f_3	$\{\}$	$\{0,1\}$
f_4	$\{0\}$	$\{\}$
f_5	$\{0\}$	$\{0\}$
f_6	$\{0\}$	$\{1\}$
f_7	$\{0\}$	$\{0,1\}$

function f_i	$f_i(0)$	$f_i(1)$
f_8	$\{1\}$	$\{\}$
f_9	$\{1\}$	$\{0\}$
f_{10}	$\{1\}$	$\{1\}$
f_{11}	$\{1\}$	$\{0,1\}$
f_{12}	$\{0,1\}$	$\{\}$
f_{13}	$\{0,1\}$	$\{0\}$
f_{14}	$\{0,1\}$	$\{1\}$
f_{15}	$\{0,1\}$	$\{0,1\}$

The three element set A_3 has $|\mathscr{P}(A_3)| = 8$. The number of functions from A_3 to $\mathscr{P}(A_3)$ is $8^3 = 512$. In general, where $|A| = n$ the power set has $|\mathscr{P}(A_3)| = 2^n$-many elements. The number of functions is $(2^n)^n = 2^{(n^2)}$. (As a check, $n = 2$ gives $2^4 = 16$ and $n = 3$ gives $2^9 = 512$.)

II.3.17

(A) These are the functions from S to T.

function f_i	$f_i(0)$	$f_i(1)$
f_0	10	10
f_1	10	11
f_2	11	10
f_3	11	11

The functions that are one-to-one are f_1 and f_2.

(B) These are the functions from S to T.

function f_i	$f_i(0)$	$f_i(1)$
f_0	10	10
f_1	10	11
f_2	10	12
f_3	11	10
f_4	11	11

function f_i	$f_i(0)$	$f_i(1)$
f_5	11	12
f_6	12	10
f_7	12	11
f_8	12	12

The functions that are one-to-one are f_1, f_2, f_3, f_5, f_6, and f_7.

II.3.18

(A) The answer is (iii) finite, since the union of two finite sets is finite. (All of these are true: (ii) countable or uncountable, (iii) finite, (iv) countable, and (v) finite, countably infinite, or uncountable. But the sharpest one, the one that is most specific, is (iii).)

(B) The sharpest is (iv).

(C) The sharpest is (i).

(D) The sharpest is (iv). (The intersection could be finite but there are cases where it is countable, such as $A = \mathbb{N}$ and $B = \mathbb{R}$. So that is the sharpest statement we can make in general is (iv).)

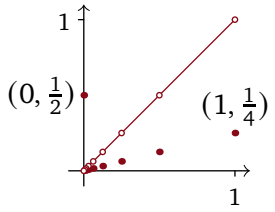
II.3.19 The question doesn't require a proof but here the parenthetical comments give a justification.

(A) They are all possible except for the last one. (An example of (i) and (ii) is $\mathrm{id}\colon \{0,1\} \to \{0,1\}$. An example of (iii) and (iv) is $\mathrm{id}\colon \mathbb{N} \to \mathbb{N}$. Lemma 2.12 rules out (v).)

(B) They are all possible. (An example of (i) and (ii) is $\mathrm{id}\colon \{0,1\} \to \{0,1\}$. An example of (iii) and (iv) is $\mathrm{id}\colon \mathbb{N} \to \mathbb{N}$. An example of (v) is $\iota\colon \mathbb{N} \to \mathbb{R}$ given by $\iota(x) = x$.)

II.3.20 Following the hint, consider this function.

$$f(x) = \begin{cases} 1/2 & \text{– if } x = 0 \\ 1/2^{n+2} & \text{– if } x = 1/2^n \text{ for } n \in \mathbb{N} \\ x & \text{– otherwise} \end{cases}$$



One way to argue that the function f is one-to-one and onto is to see that it satisfies the horizontal line test, that every horizontal line $y = t$ for $t \in (0\,..\,1)$ has one and only one associated input x.

We can expand. Consider first the case that the output $y \in (0\,..\,1)$ satisfies that $y \neq 1/2^n$ for $n \in \mathbb{N}^+$. Then y has exactly one associated input, namely $x = y$. The other case is that $y = 1/2^n$ for some $n \in \mathbb{N}^+$. Here also there is one and only one associated input x. Specifically, if $n = 1$ then that input is $x = 0$, which is in the domain $[0\,..\,1]$. If $n > 1$ then that input is $x = 1/2^{n-2}$, which is also an element of $[0\,..\,1]$.

II.3.21 Theorem 3.7 says that for every set, its cardinality is strictly less than that of its power set. So one set with a cardinality larger than $\mathbb{R}$ is $\mathscr{P}(\mathbb{R})$. (Another is $\mathscr{P}(\mathscr{P}(\mathbb{R}))$.)

II.3.22

(A) The set of finite bit strings, $\mathbb{B}^*$, is the union of the sets $\mathbb{B}^k = \{\, \langle b_0 b_1 \ldots b_{k-1} \rangle \;\big|\; k \in \mathbb{N} \,\}$. The set $\mathbb{B}^k$ is finite since it has 2^k many elements (when $k = 0$ it contains only the empty string), and so this is a countable union of countable sets.

(B) Suppose that we can count the set of infinite bit strings, meaning that it has an exhaustive enumeration $f_0, f_1, \ldots$ Let $g\colon \mathbb{B} \to \mathbb{B}$ be given by $g(0) = 1$ and $g(1) = 0$. Then the infinite bit string $F\colon \mathbb{N} \to \mathbb{B}$ defined by $F(i) = g(f_i(i))$ is not equal to any of the f_i, which contradicts that the enumeration is exhaustive.

II.3.23 Let S and T be such that $S \subseteq T$. Then the inclusion function, the map $\iota\colon S \to T$ given by $\iota(s) = s$, is clearly one to one. So $|S| \leq |T|$ by Definition 3.4.

II.3.24 We follow the hint. Call the collection of all such functions $S = \{\, f\colon \mathbb{N} \to \mathbb{N} \;\big|\; \text{ran}(f) \text{ is finite} \,\}$. Assume that S is countable so that there is a correspondence $h\colon \mathbb{N} \to S$. For all natural numbers i, the value of $h(i)$ is a function, which we will denote f_i.

Define $g\colon \mathbb{N} \to \mathbb{N}$ by $g(x) = 0$ if $x \neq 0$ and $g(x) = 1$ otherwise. Consider the function $k\colon \mathbb{N} \to \mathbb{N}$ given by $k(i) = g(\, f_i(i)\,)$ This map has a finite range, namely $\text{ran}(k) = \{0, 1\}$, but k is not equal to any $f_i \in S$ because they differ on the input i.

II.3.25 Example 2.10 shows that the set of rational numbers is countable. Suppose that the irrational numbers were also countable. Then because the union of two countable sets is countable, the real numbers would be countable. But the set of reals is not countable and so the set of irrationals is not countable either.

II.3.26

(A) Write the decimal expansion of z as $z = 0.z_0 z_1 z_2 \ldots$ It does not end in 9's as the output of g is never a 9. So z has a unique decimal expansion. For each q_i the decimal expansion of z differs from q_i's in that $z_i \cdot 10^{-(i+1)} \neq q_{i,i} \cdot 10^{-(i+1)}$, by the definition of g. Because their decimal expansions differ, and because neither ends in 9's, the two are different numbers.

Since z does not equal any rational number, it is irrational.

(B) If $d = \sum_{n \in \mathbb{N}} d_{n,n} \cdot 10^{-(n+1)}$ is rational then its decimal expansion repeats. That is, there is a decimal place such that past that place the expansion consists of a sequence of digits $d_j d_{j+1} \ldots d_{j+k}$ repeated again and again. But then the prior item's z would also repeat, in that past that same decimal place its expansion would consist of a sequence of digits $g(d_j)g(d_{j+1}) \ldots g(d_{j+k})$ repeated again and again. That would make z rational, which it isn't.

(C) In Theorem 3.1 we start with what purports to be a list of all reals and then produce a real that is not on that list. Here we start with what is a list of all rationals and produce something that is not a rational. That makes it still a real, just not a rational real.

II.3.27 Before the proof, as a motivation for the inductive step consider $S = \{0, 1, 2, 3\}$. These are the sixteen elements of $\mathscr{P}(S)$. The top line has the sets that do not contain 3 while the second line has the sets that do.

$$\begin{array}{cccccccc} \{\,\}, & \{0\}, & \{1\}, & \{2\}, & \{0,1\}, & \{0,2\}, & \{1,2\}, & \{0,1,2\}, \\ \{3\}, & \{0,3\}, & \{1,3\}, & \{2,3\}, & \{0,1,3\}, & \{0,2,3\}, & \{1,2,3\}, & \{0,1,2,3\} \end{array}$$

Each set on the bottom line is the same as the matching set on top, with an added 3.

We proceed by induction on $|S|$. The base step is that $|S| = 0$, so that S is empty. Then $\mathscr{P}(S) = \{\varnothing\}$ and $|\mathscr{P}(S)| = 1 = 2^0$, as required.

For the inductive step fix $k \in \mathbb{N}^+$, assume that the statement is true for $n = 0$, $n = 1$, ... $n = k$, and consider a set S where $|S| = k + 1$. Because $|S| = k + 1$, the set S has at least one element, $\hat{s} \in S$. The inductive hypothesis applies to $S - \{\hat{s}\}$, so $\mathscr{P}(S - \{\hat{s}\})$ has 2^k elements. As illustrated above, the elements of $\mathscr{P}(S)$ that do not contain $\hat{s}$ are in correspondence with the elements of $\mathscr{P}(S)$ that do contain $\hat{s}$. But the sets from $\mathscr{P}(S)$ that do not contain $\hat{s}$ are the exactly the sets in $\mathscr{P}(S - \{\hat{s}\})$. Thus the total number of elements of $\mathscr{P}(S)$ is $2 \cdot 2^k$, which equals 2^{k+1}.

II.3.28 Suppose that $f(s_i) = H$. We will argue that neither is $s_i \in H$ nor is $s_i \notin H$. If $s_i \in H$ then student s_i is a member of their own list, so s_i is not humble by the definition of humble, which violates the assumption of this case. On the other hand, if $s_i \notin H$ then s_i is not a member of their own list, so s_i is humble, which violates this case's assumption.

II.3.29 Suppose otherwise, that $\mathcal{C}$ is the set of all sets. Then $\mathscr{P}(\mathcal{C})$ would be a collection of sets so we would have $\mathscr{P}(\mathcal{C}) \subseteq \mathcal{C}$. That would give $|\mathscr{P}(\mathcal{C})| \leq |\mathcal{C}|$, which would contradict Cantor's Theorem.

II.3.30 One way is to take the diagonal bits two at a time; for instance changing 01 to 10, and changing any other two-bit sequence to 01.

II.3.31 In the prototype $1.000\ldots = 0.999\ldots$ we will write the two representations as $x_0 = 1$, $x_{-1} = 0$, $x_{-2} = 0$, ..., and $y_0 = 0$, $y_{-1} = 9$, $y_{-2} = 9$, ... Observe that the subscripts, the decimal places, are integers such as 0 and -1. They are not natural numbers.

(A) In $a + ar + ar^2 + ar^3 + \cdots = a/(1-r)$, take $a = 9$ and $r = 0.1$. The left side is $9 + 0.9 + 0.09 + 0.009 + \cdots = 9.999\,99\ldots$ The right side is $9/(1 - (0.1)) = 9/0.9 = 10$. Subtract 9 from both sides to get $0.999\,99\ldots = 1$.

(B) Consider a real number with two representations. Among the two there is a furthest left nonzero decimal place, $N \in \mathbb{Z}$. Consequently, write the two representations as $\vec{x} = \langle x_N, x_{N-1}, \ldots\rangle$ and $\vec{y} = \langle y_N, y_{N-1}, \ldots\rangle$. These are subject to the condition that $x_i, y_i \in \{0, 1, \ldots 9\}$ and to the condition that they add to the same number.

$$\sum_{i \leq N} x_i \cdot 10^i = \sum_{i \leq N} y_i \cdot 10^i \tag{$*$}$$

Rewrite $(*)$ as a difference.

$$0 = \Big(\sum_{i \leq N} x_i \cdot 10^i\Big) - \Big(\sum_{i \leq N} y_i \cdot 10^i\Big) = \sum_{i \leq N} (x_i - y_i) \cdot 10^i$$

Let the leftmost decimal place where $\vec{x}$ and $\vec{y}$ differ be the M-th place, so that $x_i = y_i$ for $i \in \{N, N-1, \ldots M+1\}$ and $x_M \neq y_M$. One of x_M or y_M is larger; without loss of generality suppose that $x_M > y_M$. Throwing away the initial zeroes, now the difference starts at the M-th place.

$$0 = \sum_{i \leq M} (x_i - y_i) \cdot 10^i = (x_M - y_M) \cdot 10^M + \sum_{i \leq (M-1)} (x_i - y_i) \cdot 10^i \tag{$**$}$$

On the right we've broken out the first term because we must show that $x_M = y_M + 1$ (as it does for the prototype $\vec{x} = \langle 1, 0, 0, \ldots\rangle$ and $\vec{y} = \langle 0, 9, 9, \ldots\rangle$). Consider the tail, $\sum_{i \leq (M-1)} (x_i - y_i) \cdot 10^i$. To make the sum come out to be zero this tail must offset $(x_M - y_M) \cdot 10^M$, which is a positive value since $x_M > y_M$. The largest offset that the tail can provide is if each of its $x_i - y_i$'s equals -9. The geometric series formula gives this.

$$-9 \cdot 10^{M-1} - 9 \cdot 10^{M-2} + \cdots = -9 \cdot 10^{M-1} \cdot \Big[1 + \frac{1}{10} + \frac{1}{100} + \cdots\Big] = \frac{-9 \cdot 10^{(M-1)}}{1 - (1/10)} = \frac{-9 \cdot 10^{(M-1)}}{9/10} = -10^M$$

Therefore $x_M - y_M = 1$, since the tail cannot offset a larger difference.

(C) Consider again the prior item's $(**)$, now that we have shown $x_M - y_M = 1$.

$$0 = 1 \cdot 10^M + \sum_{i \leq (M-1)} (x_i - y_i) \cdot 10^i$$

We want to show that every $x_i - y_i$ equals -9. We will just do the $i = M-1$ case, as the others are very similar.

If $x_{M-1} - y_{M-1} \geq -8$ then $(**)$ becomes this.

$$\begin{aligned} 0 &= 1 \cdot 10^M + (x_{M-1} - y_{M-1}) \cdot 10^{(M-1)} + \sum_{i \leq (M-2)} (x_i - y_i) \cdot 10^i \\ &\geq 10 \cdot 10^{(M-1)} - 8 \cdot 10^{(M-1)} + \sum_{i \leq (M-2)} (x_i - y_i) \cdot 10^i \\ &= 2 \cdot 10^{(M-1)} + \sum_{i \leq (M-2)} (x_i - y_i) \cdot 10^i \end{aligned}$$

As in the prior item the tail must offset the first term for the entire sum to come out negative. We get the largest tail by taking each $x_i - y_i$ to be -9. Again apply the sum of an infinite geometric series.

$$-9 \cdot 10^{M-2} - 9 \cdot 10^{M-3} + \cdots = -9 \cdot 10^{M-2} \cdot \left[1 + \frac{1}{10} + \frac{1}{100} + \cdots\right] = \frac{-9 \cdot 10^{(M-2)}}{1 - (1/10)} = \frac{-9 \cdot 10^{(M-2)}}{9/10} = -10^{M-1}$$

The tail is not enough to offset the $2 \cdot 10^{(M-1)}$ so $x_{M-1} - y_{M-1} \geq -8$ is not possible.

II.3.32

(A) These are the chains for S.

$s \in S$	*Chain*
0	... 0, a, 0, a ...
1	... 1, b, 1, b ...
2	... 2, d, 3, c, 2, d ...
3	—same as the chain for 2—

These are the chains for T.

$t \in T$	*Chain*
a	... a, 0, a, 0 ...
b	... b, 1, b, 1 ...
c	... c, 2, d, 3, c, 2 ...
d	—same as the chain for c—

(B) The function f is one-to-one because if $f(s) = f(\hat{s})$ then $s + 1 = \hat{s} + 1$, and so $s = \hat{s}$. The function g is one-to-one in the same way.

The chain associated with $0 \in S$ is $0, 1, 2, 3 \ldots$ It contains all of the elements of both sets, and so the chain is unique. It also has a first element, that is, there is no element of T that is $g^{-1}(0)$.

(C) If they are not already disjoint, if $S \cap T \neq \emptyset$, then we can just color all the elements of S green and all the elements of T gold and that will make them disjoint. More precisely, consider the set $\{0\} \times S$ of pairs $\langle 0, s \rangle$ along with the set $\{1\} \times T$ of pairs $\langle 1, t \rangle$. These are disjoint sets because any two elements differ in the first entry.

Obviously they correspond to the original sets S and T. So we can rephrase the result to a disjoint version that if both $|\{0\} \times S| \leq |\{1\} \times T|$ and $|\{1\} \times T| \leq |\{0\} \times S|$ then $|\{0\} \times S| = |\{1\} \times T|$.

(D) First we show that every element is in a unique chain. Fix some $x \in S \cup T$. Because we assume that the two sets are disjoint, the function operation that applies to x is determined, meaning that if $x \in S$ then for the chain we next consider $f(x)$ while if $x \in T$ then we are next considering $g(x)$. With that, all the elements of the chain to the right of x are determined. And, because the two functions are one-to-one, all of the element to the left are likewise also determined.

As to the four possibilities, either a chain is finite, that is, it repeats, or it is infinite. If it repeats then it is case (i). If it is infinite then either it does not have an initial element, and so is case (ii), or else it does. If it does have an initial element then either that element is from the set S, as in case (iii), or else that element is from the set T, as in case (iv).

(E) We will consider the correspondence for each chain type below. But first note that because each case defines h by using restrictions of the functions f and g, in each case h is well-defined. That is, never does any of these maps associate two different outputs with the same input.

A type (i) chain repeats after some number n of elements $s_0, t_0, s_1, t_1, \ldots, s_{n-1}, t_{n-1}, s_n = s_0$ where $n > 0$. (It cannot repeat after a single element — it cannot be that $s_0 = t_0$ — because the sets S and T are disjoint.) We can therefore take the first element in this loop to be a member of S. The function f makes these associations.

$$s_0 \mapsto t_0 \quad s_1 \mapsto t_1, \quad \ldots \quad s_{n-1} \mapsto t_{n-1}$$

This is clearly a correspondence among the elements of the chain. It is manifestly onto the set $\{t_0, t_1, \ldots t_{n-1}\}$ and it is one-to-one because the t_j's are distinct, or else the chain would be shorter.

A type (iii) chain

$$s,\ f(s),\ g(f(s)),\ f(g(f(s))),\ \ldots$$

associates even-indexed entries with the odd-indexed ones.

$$s \mapsto f(s) \quad g(f(s)) \mapsto f(g(f(s))), \quad \ldots$$

That is, we are defining h in a way that takes members of S to members of T Clearly this is a a correspondence.

A type (ii) chain is similar.

$$\ldots\ f^{-1}(g^{-1}(s)),\ g^{-1}(s),\ s,\ f(s),\ g(f(s)),\ f(g(f(s))),\ \ldots$$

The function h associates these, following the action of f.

$$\ldots \quad f^{-1}(g^{-1}(s)) \mapsto g^{-1}(s) \quad s \mapsto f(s) \quad g(f(s)) \mapsto f(g(f(s))), \quad \ldots$$

This association also is clearly a correspondence.

The one that takes some thought is a chain of type (iv).

$$t,\ g(t),\ f(g(t)),\ g(f(g(t)),\ \ldots$$

We want the correspondence function h to go from S to T. Consider these associations.

$$g(t) \mapsto t \quad g(f(t)) \mapsto f(t), \quad \ldots$$

Because the function g is one-to-one, its inverse is defined over its range. So taking these associations to define h gives a well-defined function. Like the chain types above, this also is clearly a correspondence.

II.4.6 A Universal Turing machine is a regular machine, in that it is just another machine in the list $\mathcal{P}_0$, $\mathcal{P}_1, \ldots$ Perhaps your friend meant something more like, "What does a Universal Turing machine do that a non-universal machine does not do?" Every machine does a job. Some machines expect a single input and then double it, some interpret their input as two numbers and then add them. A Universal Turing machine expects two inputs, e and x, and the output of this machine is the same as the output of the machine $\mathcal{P}_e$ on input x, including failing to halt if that machine fails to halt. It is universal in that any input-output behavior that can be mechanically produced can be produced by this one device.

II.4.7 Every general purpose computer, including any standard laptop or cell phone, is essentially a Universal Turing machine.

Comment: Some observers object to the contention that a general purpose computer is essentially a Turing machine by saying that a laptop does not have unboundedly much memory. That's true. But also true is that such a machine can be hooked up to an external memory, perhaps the Cloud or perhaps a device allowing it to make marks on a tape, so that it is not restricted to the installed RAM.

A person can go on to object that there are only so many elementary particles in the physical universe with which to make bits, and thus memory is not in the end unbounded. Perhaps that is true — it seems to match

current Physics, although we could worry back that we have not got enough time to exhaust what bits there are before the Big Bang becomes a Big Crunch, or whatever will happen.

However, in the section on Church's Thesis we noted that the definition of Turing machine makes precise the notion of intuitively computable. That is, the point of 'Turing machine' is that it is the right model in which to think about algorithms; it is a mathematical idealization. A laptop is a physical instantiation of the same notion.

It is true that in a sense they cannot be the same because that would be a type error — the type of one is 'mathematical object' while the other is 'physical object. In the same way, a carpenter's T-square is an instantiation of the mathematical ideal of a right angle. Asserting that T-squares are illegitimate in that no one ever will make one that is a perfect right angle to more decimal places than there are elementary particles in the universe seems to have lost track of the reasons why we find it useful to think about models. The essence of the instantiation is the idealization.

II.4.8 Yes. If $\mathcal{P}_{e_0}$ and $\mathcal{P}_{e_1}$ are universal then giving the first one the inputs e_1 and x will result in an input-output behavior exactly like the second. Likewise, giving $\mathcal{P}_{e_0}$ the inputs e_0 and x will result in the machine behaving just like itself.

II.4.9 The universal machine does not use its states to simulate the states of the other machine. Rather, it has codes for those and it manipulates those codes. In short, it does the simulation in software so the number of states in the universal machine is not relevant.

II.4.10 Yes, by Lemma 2.17 every computable function has infinitely many indices. So if $\mathcal{P}_e$ is universal, then the associated function ϕ_e has infinitely many unequal indices: $\phi_e = \phi_{e_0} = \phi_{e_1} = \cdots$. The machines $\mathcal{P}_{e_0}$, $\mathcal{P}_{e_1}$, ... are different but they have the same input/output behavior so all are universal.

II.4.11 We write $f_{i,a}$ for the function produced by freezing x_i to the value a.

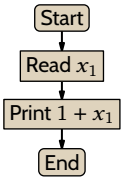
(A) The resulting one-variable function is $f_{0,4}(x_1) = 12 + 4x_1$.
(B) The one-variable function is $f_{0,5}(x_1) = 15 + 5x_1$.
(C) The one-variable function is $f_{1,0}(x_0) = 3x_0$.

II.4.12 We use $\hat{f}$ for the name of each function.

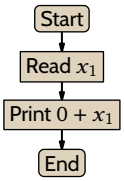
(A) The resulting two-variable function is $\hat{f}(x_1, x_2) = 1 + 2x_1 + 3x_2$.
(B) The result is the two-variable function $\hat{f}(x_1, x_2) = 2 + 2x_1 + 3x_2$.
(C) This gives $\hat{f}(x_1, x_2) = a + 2x_1 + 3x_2$.
(D) This results in the one-variable function $\hat{f}(x_2) = 11 + 3x_2$.
(E) The result is $\hat{f}(x_2) = a + 2b + 3x_2$.

II.4.13 From the machine's flowchart we have the associated function $\phi_{e_0}(x_0, x_1) = x_0 + x_1$.

(A) The two subscripts on $s_{1,1}$ mean that we will freeze the first input and leaves the second one as a variable. So $\phi_{s_{1,1}(e_0,1)}$ will be a one-input function. The arguments e_0 and 1 indicate that we are working on Turing machine e_0, the one whose flow chart is given in the problem statement, and that the input is frozen to be $x_0 = 1$. This flowchart gives a sense of the result.



The function is $\phi_{s_{1,1}(e_0,1)}(x_1) = 1 + x_1$.
(B) The values are $\phi_{s_{1,1}(e_0,1)}(0) = 1$, $\phi_{s_{1,1}(e_0,1)}(1) = 2$, and $\phi_{s_{1,1}(e_0,1)}(2) = 3$.
(C) The notation $s_{1,1}(e_0, 0)$ means that we work on Turing machine e_0, that we freeze the first input to $x_0 = 0$, and that we leave the second input as a variable. This flowchart gives the idea.

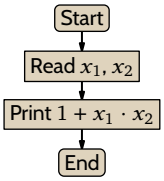


The function is $\phi_{s_{1,1}(e_0,1)}(x_1) = 0 + x_1 = x_1$.

(D) We have $\phi_{s_{1,1}(e_0,0)}(0) = 0$, $\phi_{s_{1,1}(e_0,0)}(1) = 1$, and $\phi_{s_{1,1}(e_0,0)}(2) = 2$.

II.4.14 The machine's flowchart gives that the associated function is $\phi_{e_0}(x_0, x_1, x_2) = x_0 + x_1 \cdot x_2$.

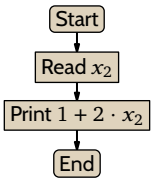
(A) The notation $s_{1,2}(e_0, 1)$ indicates that we work on $\mathcal{P}_{e_0}$, that we freeze the first input to $x_0 = 1$, and that we leave the second and third inputs as variables. This flowchart sketches the behavior.



The function is $\phi_{s_{1,2}(e_0,1)}(x_1, x_2) = 1 + x_1 \cdot x_2$.

(B) We have $\phi_{s_{1,2}(e_0,1)}(0, 1) = 1$, $\phi_{s_{1,2}(e_0,1)}(1, 0) = 1$, and $\phi_{s_{1,2}(e_0,1)}(2, 3) = 7$.

(C) The $s_{2,1}(e_0, 1, 2)$ says that we start with $\mathcal{P}_{e_0}$, that we freeze the first input to $x_0 = 1$ and the second input to $x_1 = 2$, and leave the third input as a variable. This outlines the result.



The function is $\phi_{s_{2,1}(e_0,1,2)}(x_2) = 1 + 2 \cdot x_2$.

(D) The values are $\phi_{s_{2,1}(e_0,1,2)}(0) = 1$, $\phi_{s_{2,1}(e_0,1,2)}(1) = 3$, and $\phi_{s_{2,1}(e_0,1,2)}(2) = 5$.

II.4.15 The machine's flowchart says that the associated function is this.

$$\phi_{e_0}(x_0, x_1) = \begin{cases} x_1 & \text{– if } x_0 \leq 1 \\ \uparrow & \text{– otherwise} \end{cases}$$

(A) $\phi_{s_{1,1}(e_0,0)}(x_1) = x_1$

(B) $\phi_{s_{1,1}(e_0,0)}(5) = 5$

(C) $\phi_{s_{1,1}(e_0,1)}(x_1) = x_1$

(D) $\phi_{s_{1,1}(e_0,1)}(5) = 5$

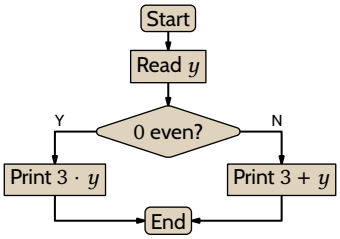
(E) $\phi_{s_{1,1}(e_0,2)}(x_1)\uparrow$

(F) $\phi_{s_{1,1}(e_0,2)}(5)\uparrow$

II.4.16 From the machine's flowchart we get that this is the associated function.

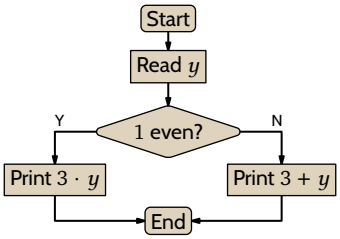
$$\phi_{e_0}(x_0, x_1, y) = \begin{cases} x_1 \cdot y & \text{– if } x_0 \text{ is even} \\ x_1 + y & \text{– otherwise} \end{cases}$$

(A) This function $\mathcal{P}_{s(e_0,0,3)}$ has $x_0 = 0$ so it is even.



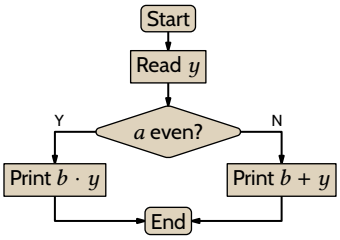
Thus the function is $\phi_{s(e_0,0,3)}(y) = 3y$. With that, $\phi_{s(e_0,0,3)}(5) = 15$.

(B) This function has $x_0 = 1$, which is odd. This flowchart sketches $\mathcal{P}_{s(e_0,1,3)}$.



The computed function is $\phi_{s(e_0,1,3)}(y) = 3 + y$ and consequently $\phi_{s(e_0,1,3)}(5) = 8$.

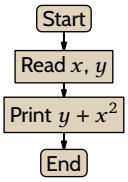
(C) This is the flowchart sketching $\mathcal{P}_{s(e_0,a,b)}$.



The distinction between this and the flowchart in the question statement is that here the machine does not read in x_0 or x_1. The two a and b are parameters, which are hard-coded into the program body. This is a family of functions parametrized by a and b, and the indices of these functions are uniformly computable from e_0, a, and b, using the *s-m-n* function s.

$$\phi_{s(e_0,a,b)}(y) = \begin{cases} b \cdot y & \text{– if } a \text{ is even} \\ b + y & \text{– otherwise} \end{cases}$$

II.4.17 Start with the function $\psi : \mathbb{N}^2 \to \mathbb{N}$ defined by $\psi(x, y) = y + x^2$. It is intuitively computable because it is computed by the machine sketched by this flowchart.

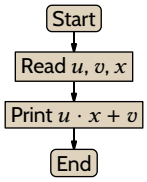


By Church's Thesis there is a Turing machine that computes it. Let that machine have index e_0, that is, suppose that $\psi(x, y) = \phi_{e_0}(x, y)$. By the *s-m-n* theorem there is a family of computable functions parametrized by n, $\phi_{s_{1,1}(e_0,n)}$, with the desired behavior, $\phi_{s_{1,1}(e_0,n)}(y) = y + n^2$. Take $g(n)$ to be $s_{1,1}(e_0, n)$ (the e_0 is not a variable, it is the fixed index of the above machine).

II.4.18 Start with the function $\psi(u, v, x) = ux + v$. It is intuitively computable as sketched by this flowchart.

```
;; Return the first character of the file (raise exception if file empty)
(define (first-char fn)
  (let ([contents (port->string (open-input-file fn) #:close? #t)])
    (string-ref contents 0)))
```

FIGURE 17, FOR QUESTION II.4.21: Racket code that returns the first character of a file.



By Church's Thesis there is a Turing machine that computes it. Let that machine have index e_0, that is, suppose that $\psi(u, v, x) = \phi_{e_0}(u, v, x)$. The *s-m-n* theorem gives a family of computable functions $\phi_{s_{2,1}(e_0,m,b)}$ parametrized by m and b with the desired behavior, $\phi_{s_{2,1}(e_0,m,b)}(x) = m \cdot x + b$. Take $g(m, b)$ to be $s_{2,1}(e_0, m, b)$ (note that e_0 is not a variable, it is the fixed index of the Turing machine from the flowchart).

II.4.19

(A) This gives the same result as $\phi_{e_1}(5)$, that is, it converges and gives 20.

(B) This returns 4 times the the value of cantor$(e_0, 5)$. We don't know the value of e_0 so that's the most that we can say.

(C) By the first item, this returns $\phi_{e_0}(20)$. Because pair$(20) = \langle 5, 0\rangle$ this is the value of $\phi_5(0)$, whatever that is.

II.4.20

(A) Because pair$(4) = \langle 1, 1\rangle$, this returns the same as $\phi_1(1)$, whatever that is.

(B) This gives the same result as $\phi_{e_1}(4)$, that is, it gives 6.

(C) This returns the value of $\phi_4(e_1)$, whatever that is.

(D) This returns $2 +$ cantor$(e_0, 4)$. The value of cantor$(e_0, 4)$ depends on e_0, which we don't know. The most we can say is that cantor$(e_0, 4) = e_0 + (e_0 + 4)(e_0 + 5)/2$.

II.4.21 Figure 17 on page 67 has Racket code. Here is what it does when run on its own source.

```
> (first-char "first-char.rkt")
#\#
```

II.4.22 Figure 18 on page 68 has some Racket code. Here is what it does when run on its own source.

```
jim@millstone:~/computing/src/scheme/background$ ./self-modifying.rkt
Counter=11
jim@millstone:~/computing/src/scheme/background$ ls -l self-modifying.rkt
-rw-rw-r-- 1 jim jim 1463 Aug 28 10:17 self-modifying.rkt
jim@millstone:~/computing/src/scheme/background$ chmod u+x self-modifying.rkt
jim@millstone:~/computing/src/scheme/background$ ./self-modifying.rkt
Counter=12
```

And, after bring run, the third line of the source now says `(define COUNTER 12)`.

II.5.7 We have shown that there are tasks that no computer can do. The Halting problem is a concrete example of such a task. (We will see many more.)

As to why this task is of any interest, the point of the Turing machine definition is to give a place on which to model running algorithms. Whether the algorithms we are modeling will halt, even when given an unbounded chance to do so, is an important question.

II.5.8 Wrong. There is such a function, but that function is not computable. That is, there is no Turing machine whose behavior is that function.

II.5.9

(A) The first is right. If we can find the store then we can get bread.

(B) The first is right. If we can exhibit a factor then we can show that the number is nonprime.

(C) The second.

```
#! /usr/bin/env racket
#lang racket
(define COUNTER 11)

;; Get list of file lines, where the counter defined in the third line is incremented
(define (intake fn)
  (let* ([contents (port->string (open-input-file fn) #:close? #t)]
         [lines-in (string-split contents "\n")]
         [third-line-in (string-split (third lines-in))]
         [later-lines-in (cdr (cdr (cdr lines-in)))]
         [counter (add1 (string->number (string-trim (third third-line-in) ")")))]
         [third-line-out (string-join (list "(define"
                                            "COUNTER"
                                            (number->string counter)
                                            ")")
                                      " "
                                      #:before-last "")]
         [lines-out (append (list (first lines-in)
                                  (second lines-in)
                                  third-line-out)
                            later-lines-in)])
    lines-out))

;; Output the strings in a list to the named file
(define (update fn str-list)
  (let ([out-port (open-output-file fn #:exists 'replace #:permissions #o664)])  ; permit rw-rw-r--
    (write-string (string-join str-list "\n") out-port)
    (close-output-port out-port)))

;; When this file is called from the command line it runs these commands
(update "self-modifying.rkt" (intake "self-modifying.rkt"))
(displayln (~a "Counter=" COUNTER))
```

FIGURE 18, FOR QUESTION II.4.22: Racket code that is self-modifying.

II.5.10 The unsolvability results do not say that you cannot prove that a particular machine does a particular thing. The given machine does indeed fail to halt on all inputs and it is indeed obvious. Instead the results are about uniformity: they say that no machine can take in any index e and correctly determine whether $\mathcal{P}_e$ does that thing.

II.5.11 First, to detect a loop we must look for a repeated configuration, not just a repeated state-character pair.

But beyond that, failing to halt and repeating a configuration are not the same thing. Machines can run forever without repeating a configuration. Here is one.

$$\mathcal{P} = \{\, q_0\mathsf{B}1q_0, q_0\mathsf{1R}q_0 \,\}$$

It keeps writing a 1 and then shifting right.

The Halting problem is not the problem of detecting whether or not a machine has an infinite loop. Rather, the Halting problem is to detect whether the machine will halt or fail to halt. It could fail to halt because there is a loop but there are other, more complicated, ways to fail to halt.

II.5.12 Sure, we could get the runtime environment to do that. However, 'fails to halt' and 'in an infinite loop' are not the same. Consider the code below.

```
;; Run forever without repeating a configuration
(define (go-up)
  (do ([i 0 (add1 i)])
    (#f (displayln "routine halt"))
    (displayln (~a "i=" i))))
```

The runtime environment described would not flag this routine as being in an infinite loop but it does not halt.

```
> i=0
> i=1
> i=2
> :
```

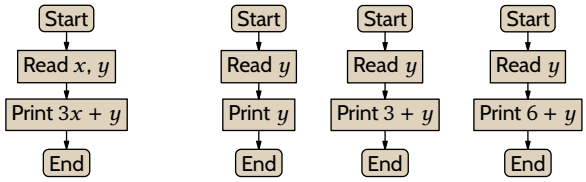


FIGURE 19, FOR QUESTION II.5.16: The machine associated with the function $f = \phi_{e_0}$ and the machines associated with $\phi_{s(e_0,x)}$ for $x = 0$, $x = 1$, and $x = 2$.

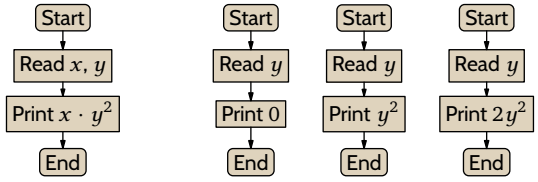


FIGURE 20, FOR QUESTION II.5.16: The machine associated with the function $f = \phi_{e_0}$ and machines associated with $\phi_{s(e_0,x)}$ for $x = 0$, $x = 1$, and $x = 2$.

It only stops when the user forces it to stop.

II.5.13 No, it is not an unsolvable problem. Either f does halt on all inputs or else it doesn't. That's two possibilities. So, as in the book's discussion of the Halting problem, overall the problem is solved by some computer program, either the one that prints 'yes' or the one that prints 'no'. (It is perhaps a unsettling that we don't know which of the two is true but nonetheless both are computable.)

II.5.14

(A) False. Such a function exists. Rather, we say that the problem is unsolvable because there is no such mechanically computable function.

(B) False. Church's Thesis asserts that Turing machines are a maximally-powerful model of computation. The existence of unsolvable problems is inherent in effective computation.

II.5.15 Any finite set is computable, so yes it is computable. There are subtleties in coflating 'computable' with 'knowable'.

II.5.16

(A) The function $f(x, y) = 3x + y$ is intuitively computable, as sketched on the left of Figure 19 on page 69. Church's Thesis gives that there is a Turing machine with that behavior; let that machine have index e_0, so that $f(x, y) = \phi_{e_0}(x, y)$ for all $x, y \in \mathbb{N}$. Applying the *s-m-n* Theorem to parametrize x gives this computable family: $\phi_{s(e_0,x)}(y) = 3x + y$, which for any x is a one-input function. Flowcharts sketching some associated machines are on the right side of Figure 19 on page 69.

(B) The function $f(x, y) = xy^2$ is intuitively computable, as in Figure 20 on page 69. Church's Thesis gives that there is a Turing machine $\mathcal{P}_{e_0}$ with that behavior, so that $f(x, y) = \phi_{e_0}(x, y)$ for all $x, y \in \mathbb{N}$. Applying the *s-m-n* Theorem to parametrize x gives $\phi_{s(e_0,x)}(y) = xy^2$, which is a family of one-input functions. Flowcharts sketching some associated machines are on the right side of Figure 20 on page 69.

(C) The function f is intuitively computable, as sketched on the left of Figure 21 on page 70. Church's Thesis gives that there is a Turing machine $\mathcal{P}_{e_0}$ with that behavior, so that $f(x, y) = \phi_{e_0}(x, y)$ for all $x, y \in \mathbb{N}$. Apply the *s-m-n* Theorem to parametrize x. The resulting family of one-input functions is this.

$$\phi_{s(e_0,x)}(y) = \begin{cases} x & \text{– if } x \text{ is odd} \\ 0 & \text{– otherwise} \end{cases}$$

Flowcharts sketching associated machines are on the right side of Figure 21 on page 70.

II.5.17 All three have the same answer. Running a Turing machine for a fixed number of steps is decidable (and in the third item, for any given input e, the number of steps is fixed).

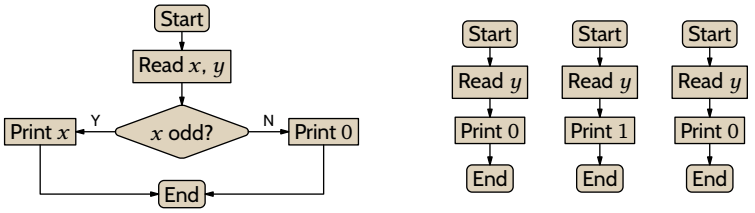


FIGURE 21, FOR QUESTION II.5.16: A sketch of the machine for the function $f = \phi_{e_0}$ and of the machines associated with $\phi_{s(e,x_0)}$ when $x = 0$, $x = 1$, and $x = 2$.

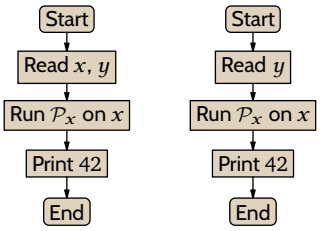


FIGURE 22, FOR QUESTION II.5.18: The machine associated with the function $\psi = \phi_e$ and the machine for $\phi_{s(e,x)}$.

II.5.18 We will show that this function is not mechanically computable.

$$\texttt{total\_decider}(e) = \begin{cases} 1 & \text{– if } \phi_e(y)\downarrow \text{ for all } y \\ 0 & \text{– otherwise} \end{cases}$$

The function

$$\psi(x,y) = \begin{cases} 42 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

is intuitively computable by the flowchart on the left of Figure 22 on page 70. Thus Church's Thesis says that there is a Turing machine that computes it; let that machine have index e_0, so that $\psi(x,y) = \phi_{e_0}(x,y)$. The *s-m-n* Theorem gives a family of one-input functions parametrized by x. The generic such machine is sketched on the right of the figure. Notice that $\phi_x(x)\downarrow$ if and only if $\texttt{total\_decider}(s(e_0,x)) = 1$.

Since s is computable, if we could mechanically compute `total_decider` then we could mechanically solve the Halting problem. We cannot mechanically solve the Halting problem, so we cannot mechanically compute `total_decider`.

II.5.19 We will show that this function is not mechanically computable.

$$\texttt{square\_decider}(e) = \begin{cases} 1 & \text{– if } \phi_e(y) = y^2 \text{ for all } y \\ 0 & \text{– otherwise} \end{cases}$$

First consider this function.

$$\psi(x,y) = \begin{cases} y^2 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

The flowchart on the left of Figure 23 on page 71 shows that ψ is intuitively computable. So by Church's Thesis there is a Turing machine that computes it. Let that machine have index e_0. Apply the *s-m-n* Theorem to get a family of functions parametrized by x. The matching machine is on the right of the figure. Then $\phi_x(x)\downarrow$ if and only if $\texttt{square\_decider}(s(e_0,x)) = 1$.

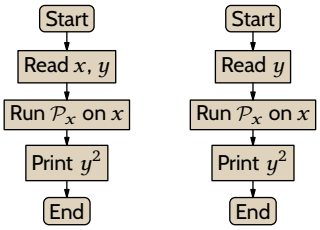


FIGURE 23, FOR QUESTION II.5.19: The machine associated with the function $\psi = \phi_{e_0}$ and the machine associated with $\phi_{s(e_0,x)}$.

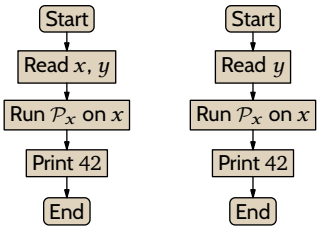


FIGURE 24, FOR QUESTION II.5.20: The machine associated with the function $\psi = \phi_{e_0}$ and the machine for $\phi_{s(e_0,x)}$.

Because of that equivalence, since the function s is computable, if we could mechanically compute `square_decider` then we could mechanically solve the Halting problem. But we can't mechanically solve the Halting problem and hence we can't mechanically compute `square_decider`.

II.5.20 We will show that this is not computable.

$$\texttt{same\_value\_decider}(e) = \begin{cases} 1 & \text{– if } \phi_e(y) = \phi_e(y+1) \text{ for some } y \\ 0 & \text{– otherwise} \end{cases}$$

The function

$$\psi(x,y) = \begin{cases} 42 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

is intuitively computable by the flowchart on the left of Figure 24 on page 71. So by Church's Thesis there is a Turing machine that computes ψ; let that machine have index e_0, so $\psi(x,y) = \phi_{e_0}(x,y)$. Apply the *s-m-n* Theorem to get a family of functions parametrized by x, the $\phi_{s(e_0,x)}$, where the associated machines are sketched on the right of the figure. These machines produce the function with constant output 42 if and only if the machine gets through the middle box. That is, $\phi_x(x)\downarrow$ if and only if $\texttt{same\_value\_decider}(s(e_0,x)) = 1$.

Since the function s is computable, by the prior sentence if we could mechanically compute `same_value_decider` then we could mechanically solve the Halting problem. But that we cannot do.

II.5.21 We will show that this function is not computable; no Turing machine has this input-output behavior.

$$\texttt{diverges\_on\_five\_decider}(e) = \begin{cases} 1 & \text{– if } \phi_e(5)\uparrow \\ 0 & \text{– otherwise} \end{cases}$$

The function

$$\psi(x,y) = \begin{cases} 42 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

is intuitively computable by the flowchart on the left of Figure 25 on page 72. So Church's Thesis says that there is a Turing machine that computes it. Let that machine have index e_0, meaning that $\psi(x,y) = \phi_{e_0}(x,y)$.

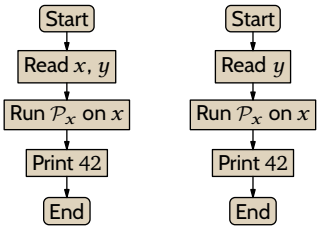


FIGURE 25, FOR QUESTION II.5.21: The machine associated with $\psi = \phi_{e_0}$ and the machine for $\phi_{s(e_0,x)}$.

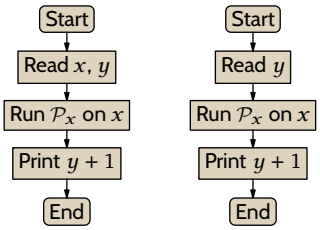


FIGURE 26, FOR QUESTION II.5.23: Machine associated with $\psi = \phi_{e_0}$ and machine associated with $\phi_{s(e_0,x)}$.

The *s-m-n* Theorem gives a family of functions parametrized by x, whose machines are sketched on the right of the same figure. Then $\phi_x(x)\downarrow$ if and only if `diverges_on_five_decider`$(s(e_0, x)) = 0$. (We could also state it as: $\phi_x(x)\downarrow$ if and only if $1 -$ `diverges_on_five_decider`$(s(e_0, x)) = 1$.) Because the function s is mechanically computable, this means that if we could mechanically compute `diverges_on_five_decider` then we could mechanically solve the Halting problem. But we cannot mechanically solve the Halting problem, and therefore we cannot mechanically compute `diverges_on_five_decider`.

II.5.22 We want to show that this function is not computable.

$$\texttt{diverge\_on\_odds\_decider}(e) = \begin{cases} 1 & \text{– if } \phi_e(y)\uparrow \text{ for all odd } y \in \mathbb{N} \\ 0 & \text{– otherwise} \end{cases}$$

The argument given in the prior item will do again here.

II.5.23 We will show that no Turing machine has this input-output behavior.

$$\texttt{successor\_decider}(e) = \begin{cases} 1 & \text{– if } \phi_e(y) = y + 1 \text{ for all } y \in \mathbb{N} \\ 0 & \text{– otherwise} \end{cases}$$

The function

$$\psi(x, y) = \begin{cases} y + 1 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

is intuitively computable by the flowchart on the left of Figure 26 on page 72. So Church's Thesis says that there is a Turing machine that computes it. Let that machine have index e_0, meaning that $\psi(x, y) = \phi_{e_0}(x, y)$. The *s-m-n* Theorem gives a family of functions parametrized by x, with machines sketched on the right of that figure. Then $\phi_x(x)\downarrow$ if and only if `successor_decider`$(s(e_0, x)) = 1$. The function s is mechanically computable, so if we could mechanically compute `successor_decider` then we could mechanically solve the Halting problem. But we can't do that and hence we cannot mechanically compute `successor_decider`.

II.5.24 We will show that no Turing machine has this input-output behavior.

$$\texttt{diverges\_on\_twice\_input\_decider}(e) = \begin{cases} 1 & \text{– when for some } y \in \mathbb{N}, \text{ both } \phi_e(y)\uparrow \text{ and } \phi_e(2y)\uparrow \\ 0 & \text{– otherwise} \end{cases}$$

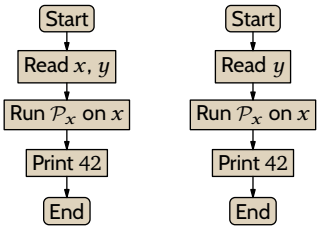


FIGURE 27, FOR QUESTION II.5.24: Outline of the machine for $\psi = \phi_{e_0}$ and the machine for $\phi_{s(e_0,x)}$.

The function

$$\psi(x,y) = \begin{cases} 42 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

is intuitively computable by the flowchart on the left of Figure 27 on page 73. So Church's Thesis says that there is a Turing machine that computes it. Let that machine have index e_0. Apply the *s-m-n* Theorem to get a family of functions parametrized by x, whose machines are sketched on the right of the figure. Then $\phi_x(x)\downarrow$ if and only if `diverges_on_twice_input_decider`$(s(e_0,x)) = 0$. (We could instead write $\phi_x(x)\downarrow$ if and only if $1 -$ `diverges_on_twice_input_decider`$(s(e_0,x)) = 1$.) So if we could effectively compute `converges_on_twice_input_decider` then we could effectively solve the Halting problem. But we cannot effectively solve the Halting problem, so we cannot effectively compute `converges_on_twice_input_decider`.

II.5.25 The second is solvable: just run the machine for a thousand steps. The first is unsolvable. For a proof we can use the blanks of the prior exercise: (1) `halts_on_153`, (2) $\phi_e(153)\downarrow$, (3) 42, (4) `Print 42`.

II.5.26 This problem is solvable. Because x and y can't be bigger than c, we just write a program that inputs cantor(x,y) for $0 \leq x, y \leq c$, applies the unpairing function to get x and y, and then plugs them into $ax + by$, to see if the result equals c. That's a bounded search and it halts.

II.5.27

(A) Unsolvable. (Verification is like many other exercises in this section.)

(B) Unsolvable. (Verification is like many other of these exercises.)

(C) Cannot tell; whether this is solvable or not depends on the specifics of the way Turing machines are counted, that is, it depends on the details of $\mathcal{P}_4$.

(D) Solvable. From e we can determine the set of five-tuples $\mathcal{P}_e$. It is finite so we can just search for the desired instruction.

II.5.28

(A) (1) `same_output_as_input`, (2) there exists $y \in \mathbb{N}$ such that $\phi_e(y) = y$, (3) y, (4) `Print y`.

(B) (1) `outputs_42`, (2) there exists $y \in \mathbb{N}$ such that $\phi_e(y) = 42$, (3) 42, (4) `Print 42`.

(C) (1) `ever_adds_2`, (2) there exists $y \in \mathbb{N}$ such that $\phi_e(y) = y + 2$, (3) $y + 2$ (4) `Print y + 2`.

II.5.29 Suppose that we could compute K_0. That is, suppose that there is a Turing machine $\mathcal{P}_{e_0}$ such that $\phi_{e_0}(\langle e,x\rangle) = 1$ if $\phi_e(x)\downarrow$ and $\phi_{e_0}(\langle e,x\rangle) = 0$ if $\phi_e(x)\uparrow$. Then we could solve the Halting problem, we could determine membership in K, just by applying $\mathcal{P}_{e_0}$ to the input $\langle e,e\rangle$.

II.5.30

(A) One such machine takes input e, uses a Universal Turing machine to run $\mathcal{P}_e$ on input e, and then exits. Figure 28 on page 74 has a flowchart. To see that computing the domain of the associated computable function is not mechanically computable, observe that if we could compute its domain then we could compute the solution to the Halting problem.

(B) For any Turing machine $\mathcal{P}_e$ and any fixed input y, The answer is either "yes it halts on y" or "no it does not." We can write two programs, one for each, and one of the programs is right.

II.5.31

(A) This is solvable. The index e is computationally equivalent to the source because we are using machine numberings that are acceptable in the sense defined on page **??**. So given e, start by translating it to the

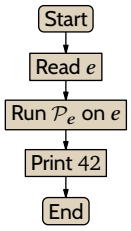


FIGURE 28, FOR QUESTION II.5.30: A single machine whose domain is undecidable.

set of Turing machine instructions, $\mathcal{P}_e$. Finish by counting the number of states that appear in that Turing machine source.

(B) This is not solvable; this function

$$\texttt{halts\_on\_empty\_decider}(e) = \begin{cases} 1 & \text{– if } \mathcal{P}_e \text{ halts when started on an empty tape} \\ 0 & \text{– otherwise} \end{cases}$$

is not mechanically computable. For, consider this function.

$$\psi(x) = \begin{cases} 42 & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

It is intuitively mechanically computable and so by Church's Thesis there is a Turing machine that computes it. Let the index of that machine be e_0. Apply the *s-m-n* theorem to parametrize x, giving a uniformly computable family of functions $\phi_{s(e_0,x)}$. (The $\phi_{s(e_0,x)}$'s are functions of no input; that's fine as the associated Turing machine simply does not read any input from the tape.)

Then $\phi_x(x)\downarrow$ if and only if $\texttt{halts\_on\_empty\_decider}(s(e_0,x)) = 1$. If `halts_on_empty_decider` were mechanically computable then we could mechanically solve the Halting problem. Therefore `halts_on_empty_decider` is not mechanically computable.

(C) This is solvable. From the index e, use a Universal Turing machine to run $\mathcal{P}_e$ on input e for one hundred steps. By the end either it has halted or it hasn't.

II.5.32 Yes. If K were finite then it would be computable, since any finite set is computable. For one thing, we can decide membership in a finite set with a sequence of if-then-else statements.

II.5.33 False; it is indeed infinite but it is uncountably infinite. There are uncountably infinitely many problems, that is, functions from $\mathbb{N}$ to $\mathbb{N}$. But there are only countably many Turing machines and hence countably many solvable problems. Therefore there are uncountably many unsolvable problems.

II.5.34 Either $\mathcal{P}_e$ does halt on all inputs, or it doesn't. So there are two possibilities, both of which are computable (either the answer is 'yes' or the answer is 'no').

Note the contrast with the unsolvable problem: given $e \in \mathbb{N}$ decide whether $\mathcal{P}_e$ halts on all inputs. As asked, this exercise does not look for an algorithm that is uniform in the index e.

II.5.35 Consider a Turing machine that, for any input, ignores the input and just loops, searching for an even number greater than two that is not the sum of two primes. If on any input, that Turing machine halts then the conjecture is false. Otherwise the conjecture is true. So if we could solve the Halting problem then we could apply it to this machine and settle the question.

XKCD's take on Goldbach is Figure 29 on page 75.

II.5.36 We can write a program that takes input, ignores it, and performs a loop that searches for a solution larger than 7, halting if it finds one. With a solution to the Halting problem we could check whether that program halts, and thereby settle the problem.

II.5.37 The set of functions $f : \mathbb{N} \to \mathbb{N}$ is the disjoint union of two subsets: the functions that are computable and the functions that are not. We know that the set of computable functions is countable. Because the union of two countable sets is countable, if the set of non-computable functions were also countable then there

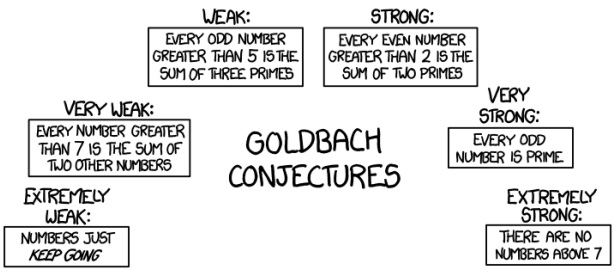


FIGURE 29, FOR QUESTION II.5.35: Goldbach's conjectures. (Courtesy xkcd.com)

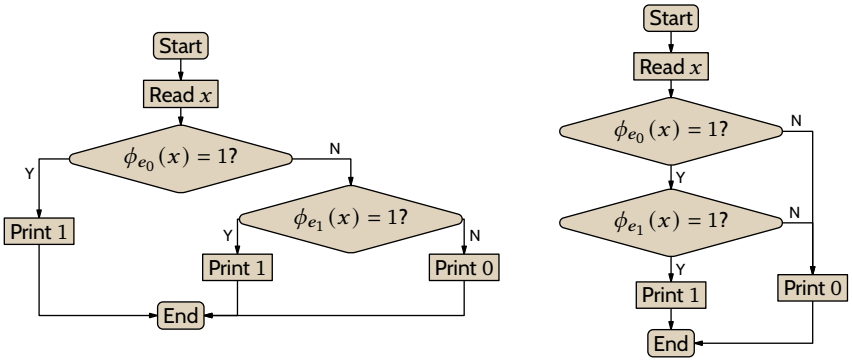


FIGURE 30, FOR QUESTION II.5.39: Sketches of machines to decide union and intersection of decidable languages.

would be countably many functions overall, which there are not. Thus the set of non-computable functions is not countable.

II.5.38 Fix an element $k \in K$. The range of the following function $f\colon \mathbb{N}^2 \to \mathbb{N}$ is K.

$$f(x,n) = \begin{cases} x & \text{– if } \mathcal{P}_x \text{ with input } x \text{ halts in } n \text{ steps} \\ k & \text{– otherwise} \end{cases}$$

Clearly the function is computable and converges on all input pairs $x, n \in \mathbb{N}^2$. For all elements of K there is an n where x appears in the range. And, for the same reason, only the elements of K appear in the range.

II.5.39 Start with decidable languages $\mathcal{L}_0$ and $\mathcal{L}_1$. Because they are decidable, there are Turing machines $\mathcal{P}_{e_0}$ and $\mathcal{P}_{e_1}$ that decide membership.

$$\phi_{e_0}(x) = \begin{cases} 1 & \text{– if } x \in \mathcal{L}_0 \\ 0 & \text{– if } x \notin \mathcal{L}_0 \end{cases} \qquad \phi_{e_1}(x) = \begin{cases} 1 & \text{– if } x \in \mathcal{L}_1 \\ 0 & \text{– if } x \notin \mathcal{L}_1 \end{cases}$$

Note that these machines halt on all inputs.

(A) A sketch of the machine that decides membership in $\mathcal{L}_0 \cup \mathcal{L}_1$ is on the left in Figure 30 on page 75. A number is an element of the union if it is a member of either set. So, given x, this machine first runs $\mathcal{P}_{e_0}$ on x. If that returns 1 then x is a member of the union and so the new machine outputs 1. Otherwise the new machine runs $\mathcal{P}_{e_1}$ on x and returns the result.

(B) A sketch of the machine that decides membership in $\mathcal{L}_0 \cap \mathcal{L}_1$ is on the right in the same figure. A number is an element of the intersection if it is a member of both sets. So, given x, this machine first runs $\mathcal{P}_{e_0}$ on x. If that returns 0 then x is not a member of the intersection, so the new machine outputs 0. Otherwise the new machine runs $\mathcal{P}_{e_1}$ on x and returns the result.

(C) Complement is straightforward. For the complement of $\mathcal{L}_0$ the new machine just runs $\mathcal{P}_{e_0}$ and subtracts the result from 1.

II.6.10 Rice's Theorem does not say that everything is impossible. For instance, it does not say that writing a routine that implements $f(x) = x^2$ is impossible.

What it does says is impossible is to write a routine that, given source code, will correctly decide whether that source, no matter how clever or for that matter obfuscated, implements f. (Such a decider is of interest because we could apply it to the routine in the first paragraph to know for sure that it works correctly in all cases.)

II.6.11 No, it is not an index set. We can write two Turing machines with the same behavior, say, on any input x they return output x. One of them takes only a few steps to do this (the trivial Turing machine $\mathcal{P} = \{\ \}$ does it in no steps), while the other takes 101 steps. The index of the second set is a member of $\mathcal{I}$ but the index of the first set is not.

II.6.12 The set of indices $e \in \mathbb{N}$ such that $\varnothing \subseteq \{x \mid \phi_e(x)\downarrow\}$ is all of $\mathbb{N}$. This problem is solvable, although the solution is trivial in that for any e, the answer is 'yes'.

II.6.13

(A) True, this is about the input-output behavior of the machine. If machines have the same input/output behavior and one halts on input 5 then the other must also.

(B) False, we can make two machines with the same behavior but where one has four instructions and the other does not. For instance we can take the four-instruction machine and make a new one by adding unreachable code, dummy instructions that start with states not referenced by the the original four instructions.

(C) True. If two machines compute the same function and the function of the first is $x \mapsto 2x$ then that is also the function of the second.

II.6.14 In each case we will argue that there are two machines, $\mathcal{P}$ and $\hat{\mathcal{P}}$, with the same input/output behavior, $\phi = \hat{\phi}$, but where one machine has the property and the other does not.

(A) The trivial Turing machine $\mathcal{P} = \{\ \}$ halts in zero steps and has the behavior $\phi(x) = x$. We can easily make another machine with the same behavior but which takes more than one hundred steps.

$$\hat{\mathcal{P}} = \{q_0\mathsf{BB}q_1, q_0\mathsf{1}\mathsf{1}q_1, q_1\mathsf{BB}q_2, q_1\mathsf{1}\mathsf{1}q_2, \ldots\ q_{100}\mathsf{BB}q_{101}, q_{100}\mathsf{1}\mathsf{1}q_{101}\}$$

(B) As with the prior item, the trivial machine uses zero tape cells on input 0. This machine visits one hundred and one cells on input 0.

$$\hat{\mathcal{P}} = \{q_0\mathsf{BL}q_1, q_0\mathsf{1}\mathsf{1}q_{102}, q_1\mathsf{BL}q_2, q_1\mathsf{1L}q_2, \ldots\ q_{100}\mathsf{BL}q_{101}, q_{100}\mathsf{1L}q_{101}\}$$

Both machines have the same behavior, $\phi(x) = x$.

(C) The trivial machine does not contain any instruction with state q_{10}. The machine

$$\hat{\mathcal{P}} = \{q_0\mathsf{BB}q_1, q_0\mathsf{1}\mathsf{1}q_1, q_1\mathsf{BB}q_2, q_1\mathsf{1}\mathsf{1}q_2, \ldots\ q_{10}\mathsf{BB}q_{11}, q_{10}\mathsf{1}\mathsf{1}q_{11}\}$$

has the same behavior and contains an instruction with that state.

II.6.15 There are many ways to describe the empty set. One is $\{e \mid \mathcal{P}_e$ solves the Halting problem$\}$.

II.6.16 One way to describe the set of all indices, $\mathbb{N}$, is $\{e \mid$ on input 3 machine $\mathcal{P}_e$ either halts or does not$\}$.

II.6.17 Notation like $\phi_e(x) = y$ means that $\phi_e(x)$ converges and gives the output value y.

(A) $\{e \in \mathbb{N} \mid \phi_e(7) = 7\}$

(B) $\{e \in \mathbb{N} \mid \phi_e(e) = e\}$

(C) $\{e \in \mathbb{N} \mid$ there is a y where $\phi_{2e}(y) = 7\}$

(D) $\{e \in \mathbb{N} \mid \phi_e(7)\downarrow$ and is prime$\}$

II.6.18 Let $\mathcal{I} = \{e \in \mathbb{N} \mid \phi_e$ is total$\}$. We will show that it is a nontrivial index set.

We first show that $\mathcal{I}$ is not the empty set. Consider the program sketched on the left of Figure 31 on page 77. By Church's Thesis, there is a Turing machine fitting this sketch. It has an index and that index is an element of $\mathcal{I}$. So $\mathcal{I}$ is not empty.

Next we show that $\mathcal{I}$ is not all of $\mathbb{N}$. Consider the program sketched on the right of the same figure. By Church's Thesis, there is a Turing machine fitting this sketch. Its index is not an element of $\mathcal{I}$ and so $\mathcal{I}$ is not all of $\mathbb{N}$.

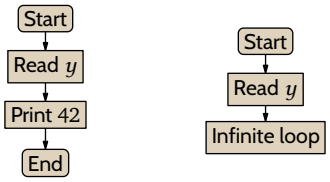


FIGURE 31, FOR QUESTION II.6.18: The machines used to show that the set $\mathcal{I}$ is not trivial.

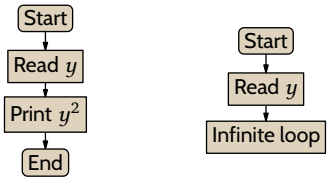


FIGURE 32, FOR QUESTION II.6.19: The machines used to show that the set $\mathcal{I}$ of indices of the squarer function is not trivial.

To finish, we show that $\mathcal{I}$ is an index set. So suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, we know that $\phi_e(a)\downarrow$ for all $a \in \mathbb{N}$. Because $\phi_x \simeq \phi_{\hat{e}}$ we therefore know that $\phi_e(a)\downarrow$ for all $a \in \mathbb{N}$. Thus, $\hat{e} \in \mathcal{I}$ and consequently $\mathcal{I}$ is an index set.

Therefore, Rice's theorem shows that the problem of determining totality, the problem of computing whether, given e, the Turing machine $\mathcal{P}_e$ halts on all inputs, is unsolvable.

II.6.19 Let $\mathcal{I} = \{ e \in \mathbb{N} \mid \phi_e(y) = y^2 \text{ for all } y \}$. We will show that it is a nontrivial index set.

We first show that $\mathcal{I}$ is not the empty set. Consider the program sketched on the left of Figure 32 on page 77. By Church's Thesis there is a Turing machine matching this sketch. It has an index, and that index is an element of $\mathcal{I}$. So $\mathcal{I}$ is not empty.

Next we show that $\mathcal{I}$ is not all of $\mathbb{N}$. Consider the program sketched on the right of the same figure. By Church's Thesis there is a Turing machine fitting this sketch. Its index is not an element of $\mathcal{I}$ and so $\mathcal{I}$ is not all of $\mathbb{N}$.

To finish we show that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ we know that $\phi_e(y) = y^2$ for all $y \in \mathbb{N}$. Because $\phi_e \simeq \phi_{\hat{e}}$ we therefore know that $\phi_{\hat{e}}(y) = y^2$ for all $y \in \mathbb{N}$. Thus, $\hat{e} \in \mathcal{I}$ and consequently $\mathcal{I}$ is an index set.

With those, Rice's theorem shows that the problem of deciding, given e, whether the Turing machine $\mathcal{P}_e$ squares its input, is unsolvable.

II.6.20 We will show that $\mathcal{I} = \{ e \in \mathbb{N} \mid \phi_e(y) = \phi_e(y+1) \text{ for some } y \in \mathbb{N} \}$ is a nontrivial index set.

We first show that $\mathcal{I}$ is not empty. Consider the program sketched on the left of Figure 33 on page 78. It is intuitively computable so by Church's Thesis there is a Turing machine fitting this sketch. That Turing machine's index is an element of $\mathcal{I}$. So $\mathcal{I}$ is not empty.

Next we argue that $\mathcal{I} \neq \mathbb{N}$. Consider the program outlined on the right of the figure. By Church's Thesis, there is a Turing machine fitting this. Its index is not an element of $\mathcal{I}$ and so $\mathcal{I}$ is not all of $\mathbb{N}$.

To finish, we show that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, there exists $y \in \mathbb{N}$ so that $\phi_e(y) = \phi_e(y+1)$. Because $\phi_e \simeq \phi_{\hat{e}}$, we know that $\phi_{\hat{e}}(y) = \phi_{\hat{e}}(y+1)$ for the same y. Thus, $\hat{e} \in \mathcal{I}$ also. Therefore $\mathcal{I}$ is an index set.

Consequently, by Rice's theorem, the problem of determining membership in $\mathcal{I}$ is unsolvable.

II.6.21 We will show that $\mathcal{I} = \{ e \in \mathbb{N} \mid \phi_e(5)\uparrow \}$ is a nontrivial index set.

First we show that $\mathcal{I} \neq \varnothing$. Consider the program sketched on the left of Figure 34 on page 78. It is intuitively computable, so by Church's Thesis there is a Turing machine fitting this sketch. That machine's index is an element of $\mathcal{I}$.

Next we argue that $\mathcal{I} \neq \mathbb{N}$. Consider the program outlined on the right of that figure. By Church's Thesis,

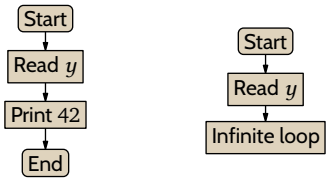


FIGURE 33, FOR QUESTION II.6.20: The machines used to show that the set $\mathcal{I}$ is not trivial.

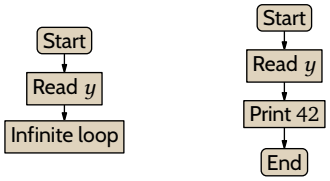


FIGURE 34, FOR QUESTION II.6.21: Outlines of the Turing machines used to show that $\mathcal{I}$ is not trivial.

there is a Turing machine fitting this. Its index is not an element of $\mathcal{I}$.

To finish, we show that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, we know that $\phi_e(5)\!\uparrow$. But $\phi_e \simeq \phi_{\hat{e}}$ so we have that $\phi_{\hat{e}}(5)\!\uparrow$ also. Thus, $\hat{e} \in \mathcal{I}$. Therefore $\mathcal{I}$ is an index set.

With those, Rice's theorem says that the problem of determining membership in $\mathcal{I}$ is unsolvable.

II.6.22 We will show that $\mathcal{I} = \{\, e \in \mathbb{N} \;\big|\; \phi_e(y)\!\uparrow \text{ for all odd } y \,\}$ is a nontrivial index set.

First we show that $\mathcal{I} \neq \varnothing$. Consider the program sketched on the left of Figure 35 on page 79. It is intuitively computable, so by Church's Thesis there is a Turing machine fitting this sketch. That machine's index is an element of $\mathcal{I}$.

Next we verify that $\mathcal{I} \neq \mathbb{N}$. Consider the program outlined on the right of the same figure. By Church's Thesis, there is a Turing machine fitting this. Its index is not an element of $\mathcal{I}$.

To finish we show that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ we know that $\phi_e(y)\!\uparrow$ for all odd inputs y. Because $\phi_e \simeq \phi_{\hat{e}}$ we know also that $\phi_{\hat{e}}(y)\!\uparrow$ for all odd y. Thus, $\hat{e} \in \mathcal{I}$, and therefore $\mathcal{I}$ is an index set.

With those, Rice's theorem gives that the problem of determining membership in $\mathcal{I}$ is unsolvable.

II.6.23 We will show that $\mathcal{I} = \{\, e \in \mathbb{N} \;\big|\; \phi_e(x) = x + 1 \,\}$ is a nontrivial index set.

First we show that $\mathcal{I} \neq \varnothing$. Consider the program sketched on the left of Figure 36 on page 79. It is intuitively computable, so by Church's Thesis there is a Turing machine fitting this sketch. That machine's index is an element of $\mathcal{I}$.

Next we show that $\mathcal{I} \neq \mathbb{N}$. Consider the program outlined on the right of that figure. By Church's Thesis, there is a Turing machine matching this. Its index is not an element of $\mathcal{I}$.

We finish by showing that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, we know that $\phi_e(x) = x + 1$ for all inputs x. Because $\phi_e \simeq \phi_{\hat{e}}$ we also know that $\phi_{\hat{e}}(x) = x + 1$ for all x. Thus, $\hat{e} \in \mathcal{I}$. Therefore $\mathcal{I}$ is an index set.

Consequently, Rice's theorem gives that the problem of determining membership in $\mathcal{I}$ is unsolvable.

II.6.24 We will show that $\mathcal{I} = \{\, e \in \mathbb{N} \;\big|\; \text{both } \phi_e(x)\!\uparrow \text{ and } \phi_e(2x)\!\uparrow \text{ for some } x \,\}$ is a nontrivial index set.

First we show that $\mathcal{I} \neq \varnothing$. Consider the program sketched on the left of Figure 37 on page 80. It is intuitively computable, so by Church's Thesis there is a Turing machine fitting this sketch. That machine's index is an element of $\mathcal{I}$.

Next we argue that $\mathcal{I} \neq \mathbb{N}$. Consider the program outlined on the right of Figure 37 on page 80. By Church's Thesis, there is a Turing machine fitting this. Its index is not an element of $\mathcal{I}$.

To finish, we show that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, we know that for some input x we have both $\phi_e(x)\!\uparrow$ and $\phi_e(2x)\!\uparrow$. Because $\phi_e \simeq \phi_{\hat{e}}$ we know that $\phi_{\hat{e}}(x)\!\uparrow$ and $\phi_{\hat{e}}(2x)\!\uparrow$, for the same x. Thus, $\hat{e} \in \mathcal{I}$. Therefore $\mathcal{I}$ is an index set.

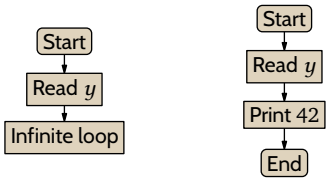


FIGURE 35, FOR QUESTION II.6.22: The machines used to show that $\mathcal{I}$ is not trivial.

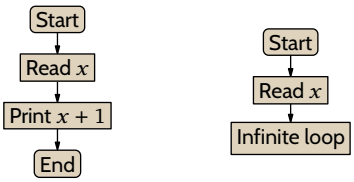


FIGURE 36, FOR QUESTION II.6.23: The machines used to show that $\mathcal{I}$ is not trivial.

Consequently, by Rice's theorem, the problem of determining membership in $\mathcal{I}$ is unsolvable.

II.6.25 The index set in the first item, is the complement of the one from **??**. A later exercise, **??**, shows that the complement of an index set is also an index set. But we will verify that the given problem is unsolvable without reference to those exercises.

(A) The problem is: given an index e, decide whether $\phi_e(y)\!\uparrow$ for some $y \in \mathbb{N}$. To show that this is unsolvable we will verify that $\mathcal{I} = \{e \in \mathbb{N} \;\big|\; \phi_e(y)\!\uparrow \text{ for some } y \in \mathbb{N}\}$ is a nontrivial index set.

The set $\mathcal{I}$ is not empty because we can write a program that that does not halt on any input and then by Church's Thesis there is a Turing machine with that behavior. That machine's index is a member of $\mathcal{I}$.

The set is not equal to $\mathbb{N}$ because we can write a program that returns its input and then by Church's Thesis there is a Turing machine with that behavior. That machine's index is not a member of $\mathcal{I}$.

We finish verifying that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ we know that $\phi_e(y)\!\uparrow$ for at least one $y \in \mathbb{N}$. Because the two have the same behavior, $\phi_{\hat{e}}(y)\!\uparrow$ for at least one y also. Therefore $\hat{e} \in \mathcal{I}$ and $\mathcal{I}$ is an index set.

(B) Let $\mathcal{I} = \{e \in \mathbb{N} \;\big|\; \phi_e(y)\!\downarrow \text{ for some } y \in \mathbb{N}\}$. We will verify that it is a nontrivial index set.

The set $\mathcal{I}$ is not empty because we can write a program that that returns 0 on all inputs. Church's Thesis gives a Turing machine with that behavior. That machine's index is a member of $\mathcal{I}$.

The set $\mathcal{I}$ is not equal to $\mathbb{N}$ because we can write a program that fails to halt on any input, and by Church's Thesis there is a Turing machine with that behavior, and that machine's index is not a member of $\mathcal{I}$.

With $\mathcal{I}$ as nontrivial, we finish by verifying that it is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ we know that $\phi_e(y)\!\downarrow$ for at least one $y \in \mathbb{N}$. The two have the same behavior and thus $\phi_{\hat{e}}(y)\!\downarrow$ for at least one y also. Therefore $\mathcal{I}$ is an index set.

II.6.26 Each answer just gives the numbered fill-ins.

(A) (1) $\phi_e(x)\!\downarrow$ for all inputs x that are a multiple of 5. (2) and (3) Figure 38 on page 80. (4) $\phi_e(5k)\!\downarrow$ for all $k \in \mathbb{N}$. (4) $\phi_{\hat{e}}(5k)\!\downarrow$ for all $k \in \mathbb{N}$ also.

(B) (1) $\phi_e(x) = 7$ for some input x. (2) and (3) Figure 39 on page 81. (4) $\phi_e(x) = 7$ for some input $x \in \mathbb{N}$. (4) $\phi_{\hat{e}}(x) = 7$ for the same x.

II.6.27 Every finite set is computable. With a finite set we can easily write a routine that acts as its characteristic function; there are just a finite number of cases. (Church's Thesis says that if there is such a program then there is such a Turing machine.)

II.6.28

(A) We will show that this is a nontrivial index set: $\mathcal{I} = \{e \in \mathbb{N} \;\big|\; \mathcal{P}_e \text{ accepts an infinite language}\}$.

First we verify that $\mathcal{I}$ is nonempty. We can write a Turing machine to output 1 on all inputs and this machine accepts $\mathcal{L} = \mathbb{B}$, which is an infinite set. Thus the index of this Turing machine is an element of $\mathcal{I}$.

Second we verify that $\mathcal{I}$ does not equal $\mathbb{N}$. We can write a Turing machine to output 0 on all inputs. This

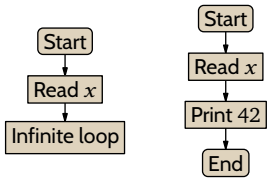


FIGURE 37, FOR QUESTION II.6.24: The machines used to show that $\mathcal{I}$ is not trivial.

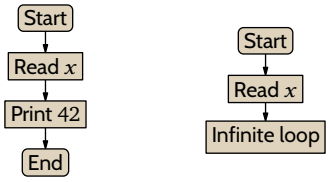


FIGURE 38, FOR QUESTION II.6.26: The machines used to argue that $\mathcal{I}$ is not trivial.

machine accepts the empty subset of $\mathbb{B}$, which is not an infinite set. Thus the index of this Turing machine is not an element of $\mathcal{I}$.

Finally, to verify that $\mathcal{I}$ is an index set, suppose that $e \in \mathcal{I}$ and that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ the language that $\mathcal{P}_e$ accepts is infinite. Because $\phi_e \simeq \phi_{\hat{e}}$ the set of bit strings for which $\mathcal{P}_{\hat{e}}$ outputs 1 equals the set for which $\mathcal{P}_e$ outputs 1, and so $\mathcal{P}_{\hat{e}}$ also accepts an infinite language. Therefore $\hat{e} \in \mathcal{I}$, and $\mathcal{I}$ is an index set.

(B) We will show that $\mathcal{I} = \{ e \in \mathbb{N} \mid \mathcal{P}_e \text{ accepts } \texttt{101} \}$ is a nontrivial index set. First, $\mathcal{I}$ is nonempty because we can write a Turing machine to output 1 on all inputs, and that machine accepts the string 101. The index of that machine is an element of $\mathcal{I}$.

Second, $\mathcal{I}$ does not equal $\mathbb{N}$ because we can write a Turing machine to output 0 on all inputs, and that machine does not accept the string 101. The index of that machine is not an element of $\mathcal{I}$.

Finally, we verify that $\mathcal{I}$ is an index set. Suppose that $e \in \mathcal{I}$ and that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ the machine $\mathcal{P}_e$ accepts 101. Because $\phi_e \simeq \phi_{\hat{e}}$, the machine $\mathcal{P}_{\hat{e}}$ also accepts 101. Thus $\hat{e} \in \mathcal{I}$ and $\mathcal{I}$ is an index set.

II.6.29 We will use Rice's Theorem so consider $\mathcal{I} = \{ e \mid \mathcal{P}_e \text{ accepts every } \sigma \in \mathbb{B}^* \}$.

First, we can write a Turing machine that halts on all inputs and outputs 1, and the index of this machine is a member of $\mathcal{I}$, so that is not the empty set.

Second, we can write a Turing machine that halts on all inputs and outputs 0, and the index of that machine is not a member of $\mathcal{I}$, so $\mathcal{I} \neq \mathbb{N}$.

To show that $\mathcal{I}$ is an index set, suppose that $e \in \mathcal{I}$ and that the number $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, machine $\mathcal{P}_e$ accepts all bitstrings. Because $\phi_e \simeq \phi_{\hat{e}}$, machine $\mathcal{P}_{\hat{e}}$ also accepts all bitstrings. Thus $\hat{e} \in \mathcal{I}$, and $\mathcal{I}$ is a nontrivial index set.

II.6.30 We can produce two Turing machines, $\mathcal{P}_e$ and $\mathcal{P}_{\hat{e}}$ that have the same behavior, $\phi_e \simeq \phi_{\hat{e}}$, but $e \in \mathcal{I}$ while $\hat{e} \notin \mathcal{I}$. Here are two such machines.

$$\mathcal{P}_e = \{ q_0\mathsf{BB}q_0, q_0\mathsf{11}q_0, q_1\mathsf{BB}q_1, q_1\mathsf{11}q_1 \} \qquad \mathcal{P}_{\hat{e}} = \{ q_0\mathsf{BB}q_0, q_0\mathsf{11}q_0 \}$$

The first has the unreachable state of q_1.

II.6.31 This problem is not solvable. We cannot tell anything by whether the machine 'goes on', that is, does not stop immediately. It can go on to do quite complex things and then stop eventually, or do complex things and never stop.

For more, we can reduce the Halting problem to this problem. Consider this $\psi \colon \mathbb{N}^2 \to \mathbb{N}$.

$$\psi(x,y) = \begin{cases} 42 & \text{– if machine } \mathcal{P}_x \text{ halts on } x \text{ and } y = \varepsilon \\ \uparrow & \text{– otherwise} \end{cases}$$

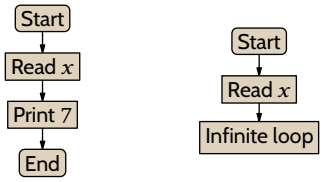


FIGURE 39, FOR QUESTION II.6.26: The machines used to argue that $\mathcal{I}$ is not trivial.

(We take this machine to have tape alphabet $\Sigma = \{\mathsf{B}, 1\}$.) This is intuitively computable so Church's Thesis gives that there is a Turing machine with this behavior, and we can assume it has index e_0.

Apply the *s-m-n* Theorem to parametrize x, giving a family of machines and associated computable functions as here.

$$\phi_{s(e_0,x)}(y) = \begin{cases} 42 & \text{– if machine } \mathcal{P}_x \text{ halts on } x \text{ and } y = \varepsilon \\ \uparrow & \text{– otherwise} \end{cases}$$

Then $\phi_x(x)\downarrow$ if and only if the function $\phi_{s(e_0,x)}$ halts only on $y = \varepsilon$. So if we could check the latter mechanically then we could mechanically solve the Halting problem, which of course we cannot.

II.6.32 The Padding Lemma, I.**??**, says that every computable function has infinitely many indices. So if the set contains an $e \in \mathbb{N}$ then it must contain the infinitely many indices $\hat{e}$ such that $\phi_e \simeq \phi_{\hat{e}}$.

II.6.33 We will call each set $\mathcal{I}$.

(A) Suppose that $e \in \mathcal{I}$ and suppose that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$ the machine $\mathcal{P}_e$ halts on at least five inputs, that is, the function ϕ_e converges on at least five inputs. Because $\phi_e \simeq \phi_{\hat{e}}$, the function $\phi_{\hat{e}}$ also halts on at least five inputs, namely, the same five. Thus $\hat{e} \in \mathcal{I}$ also.

(B) Suppose that $e \in \mathcal{I}$ and suppose also that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, the function ϕ_e is one-to-one. Because $\phi_e \simeq \phi_{\hat{e}}$, the function $\phi_{\hat{e}}$ is also one-to-one. Thus $\hat{e} \in \mathcal{I}$.

(C) Suppose that $e \in \mathcal{I}$ and also that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, the function ϕ_e is either total or else $\phi_e(3)\uparrow$. In the case that ϕ_e is total, because $\phi_e \simeq \phi_{\hat{e}}$, the function $\phi_{\hat{e}}$ is also total. In the case that $\phi_e(3)\uparrow$, because they have the same behavior, $\phi_{\hat{e}}(3)\uparrow$ also. In either case, $\hat{e} \in \mathcal{I}$.

II.6.34

(A) The relation $\simeq$ is reflexive because a Turing machine $\mathcal{P}_e$ has the same behavior as itself: $\phi_e \simeq \phi_e$. It is symmetric because if $\mathcal{P}_e$ has the same behavior as $\mathcal{P}_{\hat{e}}$ then the converse also holds. Transitivity is just as clear.

(B) Each equivalence class is a set of integers $e \in \mathbb{N}$. Two integers are together in a class, $e, \hat{e} \in \mathcal{C}$ if the input-output behaviors of the associated Turing machines $\mathcal{P}_e$ and $\mathcal{P}_{\hat{e}}$ are the same, $\phi_e \simeq \phi_{\hat{e}}$. (As an example, one class contains the indices of all Turing machines that double the input, $\mathcal{C} = \{ e \in \mathbb{N} \mid \phi_e(x) = 2x \text{ for all } x \}$.)

(C) We follow the hint. Suppose that $\mathcal{I}$ is an index set, that e is an element of $\mathcal{I}$, and that it belongs to the equivalence class $\mathcal{C}$ of the relation $\simeq$. Let $\hat{e}$ be another element of the class $\in \mathcal{C}$, so that $\phi_{\hat{e}} \simeq \phi_e$. Because $\mathcal{I}$ is an index set, $\hat{e} \in \mathcal{I}$ also. Thus having one element of $\mathcal{C}$ as an element of $\mathcal{I}$ implies that every element of C is in $\mathcal{I}$.

II.6.35

(A) Let $\mathcal{I}$ be an index set and consider the complement, $\mathcal{I}^{\mathsf{c}}$. Suppose that $e \in \mathcal{I}^{\mathsf{c}}$ and suppose also that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. We will show that $\hat{e} \in \mathcal{I}^{\mathsf{c}}$.

Either $\hat{e} \in \mathcal{I}$ or $\hat{e} \in \mathcal{I}^{\mathsf{c}}$. If it were to be that $\hat{e} \in \mathcal{I}$ then because $\phi_e \simeq \phi_{\hat{e}}$ and because $\mathcal{I}$ is an index set, we would have that $e \in \mathcal{I}$ also. This contradicts the assumption that $e \in \mathcal{I}^{\mathsf{c}}$. Therefore $\hat{e} \in \mathcal{I}^{\mathsf{c}}$.

(B) Fix a collection $\mathcal{I}_j$ of index sets and consider the union $\mathcal{I} = \cup_j \mathcal{I}_j$. Suppose that $e \in \mathcal{I}$ and suppose also that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. We will show that $\hat{e} \in \mathcal{I}$.

The number e is an element of the union $\cup_j \mathcal{I}_j = \mathcal{I}$ so it is an element of some $\mathcal{I}_{j_0}$. Because $\phi_e \simeq \phi_{\hat{e}}$ and because $\mathcal{I}_{j_0}$ is an index set, the number $\hat{e}$ is an element of $\mathcal{I}_{j_0}$, and is therefore an element of the union $\mathcal{I}$. Thus $\mathcal{I}$ is an index set.

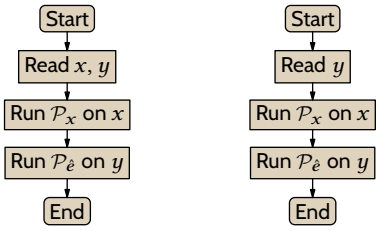


FIGURE 40, FOR QUESTION II.6.36: Sketches of machines for the proof of half of Rice's Theorem.

(C) It is closed under intersection. Let $\mathcal{I}_j$ be a collection of index sets and let $\mathcal{I} = \cap_j \mathcal{I}_j$. Suppose that $e \in \mathcal{I}$, and also that $\hat{e} \in \mathbb{N}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. We will show that $\hat{e} \in \mathcal{I}$.

Since e is an element of the intersection, $\cap_j \mathcal{I}_j = \mathcal{I}$, it is an element of each set $\mathcal{I}_j$. Because $\phi_e \simeq \phi_{\hat{e}}$ and because $\mathcal{I}_j$ is an index set, the number $\hat{e}$ is an element of each $\mathcal{I}_j$ also. As $\hat{e}$ is an element of each $\mathcal{I}_j$, it is an element of the intersection $\mathcal{I}$. Thus $\mathcal{I}$ is an index set.

II.6.36 The proof half in the section fixes an index $e_0 \in \mathbb{N}$ so that $\phi_{e_0}(y)\uparrow$ for all inputs $y \in \mathbb{N}$. It then does the $e_0 \notin \mathcal{I}$ case, so what's left to do here is the $e_0 \in \mathcal{I}$ case.

Because $\mathcal{I}$ is nontrivial it does not equal $\mathbb{N}$. So there is some $\hat{e} \notin \mathcal{I}$. Since $\mathcal{I}$ is an index set, $\phi_{e_0} \neq \phi_{\hat{e}}$. Thus there is a y such that $\phi_{\hat{e}}(y)\downarrow$.

Consider the flowchart on the left of Figure 40 on page 82. By Church's Thesis there is a Turing machine with that behavior; call it $\mathcal{P}_e$. Apply the *s-m-n* theorem to parametrize x, resulting in the uniformly computable family of functions $\phi_{s(e,x)}$, whose computation is outlined on the right of Figure 40 on page 82. We've constructed the machines so that if $\phi_x(x)\uparrow$ then $\phi_{s(e,x)} \simeq \phi_{e_0}$ and thus $s(e,x) \in \mathcal{I}$. Further, if $\phi_x(x)\downarrow$ then $\phi_{s(e,x)} \simeq \phi_{\hat{e}}$ and thus $s(e,x) \notin \mathcal{I}$. Therefore if $\mathcal{I}$ were mechanically computable, so that we could effectively check whether $s(e,x) \in \mathcal{I}$, then we could mechanically solve the Halting problem. Of course, that's impossible.

II.7.7 Half right. It is right that a set is computably enumerable if it can be enumerated by a Turing machine. However, the "but is not computable" is wrong. For instance, the set of even numbers is computably enumerable, since it is clearly effectively listable, and is also computable in that its characteristic function is computable.

II.7.8 The idea of 'effectively listable' is that a machine process gives this as its set of outputs. Some Turing machines never halt, no matter the input, so these machines list the empty set.

II.7.9 For each part we will produce a computable total function $f : \mathbb{N} \to \mathbb{N}$ that has the given set as range.

(A) The identity function, $f(x) = x$, works.

(B) The function $f(x) = 2x$ comes to mind.

(C) $f(x) = x^2$

(D) This will do.

$$f(x) = \begin{cases} 5 & \text{– if } x = 0 \\ 7 & \text{– if } x = 1 \\ 11 & \text{– otherwise} \end{cases}$$

II.7.10 For each item we produce an effective function $f : \mathbb{N} \to \mathbb{N}$.

(A) The association $n \mapsto n$-th prime is a function. Clearly we can write a program that computes it, so by Church's Thesis it is effective.

(B) To compute this function we can use brute force: given n, searching through the natural numbers, 0, 1, ... , checking each to see if its digits appear in non-increasing order. Output the n-th such number.

II.7.11 Yes to all four. For the first and fourth, we can write a Turing machine whose input/output behavior is to act as the identity function $x \mapsto x$, and this machine lists all natural numbers. So the set of all natural numbers is computably enumerable.

For the second and third, there are Turing machines that halt on no inputs, so the empty set is computably enumerable.

II.7.12 The first is computable, since to decide if a number e is a member of this set we can run a simulation for twenty steps. The second is computably enumerable but not computable. It is computably enumerable because we can dovetail computations for the indices $e \in \mathbb{N}$. It is not computable because clearly if we could solve it then we could solve the problem of deciding membership in $\{e \bigm| \phi_e(4)\!\downarrow\}$, but we cannot solve that problem by Rice's Theorem.

II.7.13 Recall that for sets 'decidable' means the same as 'computable', namely that the characteristic function is a total computable function. Also, 'semidecidable' means the same as 'computably enumerable'.

(A) This set is computable, since we can just run the machine for a hundred steps. Because it is computable, it is also computably enumerable.

(B) This set is also computable, again since we can just run the machine for a hundred steps. Because it is computable, it is also computably enumerable.

II.7.14 The second is true by Lemma 7.5. The first is false; an example is that the set $\mathbb{N}$ is computably enumerable but its proper subset K is not computable.

II.7.15 The function is an enumeration but unless it is computable, it is not a computable enumeration. An example of a countable set that is not computably enumerable is K^c. It is countable because it is a subset of the countably set $\mathbb{N}$, and it is not computably enumerable by Corollary 7.6.

II.7.16 Briefly, we dovetail. To get the enumeration, first run $\mathcal{P}_0$ on input 5 for one step. Then run $\mathcal{P}_0$ on input 5 for a second step and $\mathcal{P}_1$ on input 5 for one step. Then run $\mathcal{P}_0$ for a third step, along with $\mathcal{P}_1$ for its second step and $\mathcal{P}_2$ on input 5 for one step. Continue in this way. The enumerating function $f\colon \mathbb{N} \to \mathbb{N}$ is: $f(0)$ is the index of the first such computation to halt, $f(1)$ is the index of the second, etc.

II.7.17 There are only countably many Turing machines to do the enumeration.

II.7.18 No. The empty set is computably enumerable but finite.

II.7.19

(A) The set is computable. We are given an e and want to decide whether $\mathcal{P}_e$ contains such an instruction. Indices are source-equivalent, meaning that there is a program that takes in the index e and puts out the set of instructions for $\mathcal{P}_e$. We can then decide whether e is in the set by scanning the set of instructions for a q_4.

(B) This set is computably enumerable but not computable. To show it is computably enumerable, use dovetailing on the indices $e \in \mathbb{N}$ to run $\mathcal{P}_e$'s on input 4. That is, first run $\mathcal{P}_0$ on input 4 for one step. Next run $\mathcal{P}_0$ on input 4 for one more step and also run $\mathcal{P}_1$ on input 4 for one step. In continuing this process, when a computation halts, enumerate its e into the set. As to this set being not computable, Rice's Theorem easily shows that this is not a computable set.

(C) Computable. Given e, decide if it is in the set by running $\mathcal{P}_e$ on 4 for the hundred steps.

II.7.20 To do the enumeration, dovetail. Start by running $\mathcal{P}_0$ on input 2 for one step. Then run $\mathcal{P}_0$ on input 2 for a second step and $\mathcal{P}_1$ on input 2 for one step. Next, run $\mathcal{P}_0$ for a third step, along with $\mathcal{P}_1$ for its second step and $\mathcal{P}_2$ on input 2 for one step. Continue in this way. Then let $f(0)$ be the index of the first such computation to halt and output 4, let $f(1)$ be the index of the second, etc.

II.7.21 There are of course many correct answers. One answer is to take K as the first set. For the second, take K with its smallest element omitted and for the third, take K with the smallest two elements omitted.

A perhaps less wiseguy-ish answer is to take K along with some sets that we used as examples in the prior two sections, such as $\{e \in \mathbb{N} \bigm| \phi_e(3)\!\downarrow\}$ and $\{e \in \mathbb{N} \bigm| \text{there is } x \in \mathbb{N} \text{ so that } \phi_e(x) = 7\}$. The latter two are computably enumerable via a dovetailing process.

II.7.22

(A) Enumerate it by dovetailing. The difference from some of the dovetail examples is that here there are two numbers, e and x. So recall Cantor's pairing function.

Number	0	1	2	3	4	5	6	...
Pair	$\langle 0,0\rangle$	$\langle 0,1\rangle$	$\langle 1,0\rangle$	$\langle 0,2\rangle$	$\langle 1,1\rangle$	$\langle 2,0\rangle$	$\langle 0,3\rangle$	...

For the initial time through the dovetailing loop find the initial pair in the table, $\langle 0,0\rangle$ (we take the first number to be e and the second to be x), and run $\mathcal{P}_0$ on input 0, for a single step. In the next time through the loop, run $\mathcal{P}_0$ on input 0 for a second step and also run (because $\langle 0,1\rangle$ is the second pair) $\mathcal{P}_0$ on input 1 for one step. For the next time through, run $\mathcal{P}_0$ on input 0 for its third step, and $\mathcal{P}_0$ on input 1 for its second

step, and $\mathcal{P}_1$ on input 0 for one step. The enumerating function outputs $\langle e, x\rangle$ when $\mathcal{P}_e$ halts on x.

(B) This is immediate from Lemma 7.3.

II.7.23 No, for two reasons.

First, it is not the case that the collection of computable sets and the collection of computably enumerable sets are disjoint. Rather, the collection of computable sets is a subset of the collection of computably enumerable sets, by Lemma 7.5.

Furthermore, the collection $\{W_e \mid e \in \mathbb{N}\}$ is countable, since the elements in that collection, the W_e's are indexed by the natural numbers. Since the number of subsets of $\mathbb{N}$ is uncountable (because the power set of $\mathbb{N}$ has cardinality greater than $\mathbb{N}$), there are more subsets of $\mathbb{N}$ than there are countably enumerable sets. Therefore there are subsets of $\mathbb{N}$ that are not computably enumerable.

II.7.24 The computable sets are also computably enumerable so we will have shown that Tot is neither computable nor computably enumerable if we have shown that it is not computably enumerable.

Looking for a contradiction, assume that Tot $= \{e \mid \phi_e(x)\downarrow \text{ for all } x\}$ is computably enumerable, enumerated by the function $f\colon \mathbb{N} \to \mathbb{N}$. Following the hint, that gives a table like the one starting section 5 on the Halting problem, because the i, j entry $\phi_{f(i)}(j)$ is sure to halt. Diagonalize: consider $h\colon \mathbb{N} \to \mathbb{N}$ given by $h(i) = 1 + \phi_{f(i)}(i)$. That function is effective, total, and unequal to any member of Tot, which is a contradiction.

II.7.25 Rice's theorem easily shows that the set $S = \{e \mid \phi_e(3)\uparrow\}$ is not computable. Clearly the complement, $S^c = \{e \mid \phi_e(3)\downarrow\}$, is computably enumerable — just dovetail and enumerate elements as they converge. If S were also computably enumerable then by Lemma 7.5 it would be computable, which would be a contradiction.

II.7.26 Let the infinite computably enumerable set be W and fix a computable enumeration $\phi(0), \phi(1), \ldots$ We will define an infinite $S \subseteq W$ that is computable.

Let the first element of S be $s_0 = \phi(0)$. Take the second element s_1 to be the first member of the enumeration that is larger than s_0; such a number must eventually appear because W is infinite. In general s_{i+1} is the first element of the enumeration that is larger than s_i (as with s_1, one must eventually appear because W is infinite).

The set S is computably enumerable because we have just described a mechanical procedure to enumerate it. It is computable because, given a number $n \in \mathbb{N}$, to decide whether it is an element of S just wait until either it is enumerated into S or a number larger than it is enumerated into S, which shows $n \notin S$.

II.7.27

(A) We can argue by Church's Thesis: compute steps(e) by running $\mathcal{P}_e$ on input e (using a Universal Turing machine), and if it ever halts then output the number of steps.

(B) We claim that if steps were total in addition to being computable then we could solve the Halting problem. The reason is that to decide whether $e \in K$, first compute steps(e). Then run $\mathcal{P}_e$ on input e for steps(e)-many steps. If this computation has not halted after that many steps then it will never halt.

(C) This is the same as the prior argument, but with f in place of steps.

II.7.28 Suppose that S is computable, so that its characteristic function

$$\mathbb{1}_S(x) = \begin{cases} 1 & \text{– if } x \in S \\ 0 & \text{– if not} \end{cases}$$

is a computable function. To enumerate S in increasing order, run $\mathbb{1}_S(0), \mathbb{1}_S(1), \ldots$, each of which must converge because the function is computable, and then the first element of S is the smallest, the second element is the second smallest, etc.

More formally, define the enumerating function by: where s_0 is the smallest element of S then $f(0) = s_0$, and where we have values for $f(0), \ldots\ f(k)$ then the value for $f(k+1)$ is the smallest x such that $x > f(k)$ and $\mathbb{1}_S(x) = 1$. Because S is infinite, there is always such an x.

For the other direction, assume that S is an infinite set that is enumerable in increasing order, by the computable function h. Given x, to decide whether $x \in S$, compute $h(0), h(1), \ldots$ until the output is x (and so $x \in S$) or the output is larger than x (and so $x \notin S$).

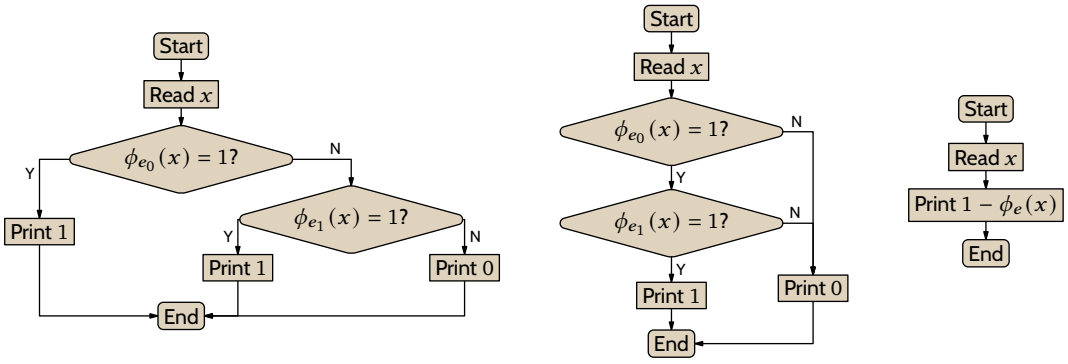


FIGURE 41, FOR QUESTION II.7.30: Machines to decide union, intersection, and complement of computable sets.

II.7.29 Consider the set of ordered pairs $S = \{ \langle a, b \rangle \mid a \in K \text{ and } b \in K^{\mathsf{c}} \}$. It is not computably enumerable, or enumerating it would enumerate its second entries, but K^{c} is not computably enumerable. Similarly its complement is not computably enumerable or enumerating it would enumerate the complement of S's first entries, again impossible because K^{c} is not computably enumerable.

II.7.30

(A) No. The set $\mathbb{N}$ is computable but its subset K is not.

(B) Yes. Let $S_0 \subseteq \mathbb{N}$ and $S_1 \subseteq \mathbb{N}$ be computable via the functions ϕ_{e_0} and ϕ_{e_1}. A sketch of the machine to compute the union is in Figure 41 on page 85.

(C) Yes. A sketch of the machine to compute the intersection is in the same figure.

(D) Yes. A sketch of the machine to compute the complement is also in that figure.

II.7.31

(A) No. The set $\mathbb{N}$ is computably enumerable. The complement of the Halting problem set, K^{c}, is a subset of $\mathbb{N}$ and is not computably enumerable.

(B) Yes. Let $W_{e_0}, W_{e_1} \subseteq \mathbb{N}$ be computably enumerable, via $\phi_{e_0}, \phi_{e_1} : \mathbb{N} \to \mathbb{N}$. Dovetail enumerations of the two sets. That is, run $\mathcal{P}_{e_0}$ and $\mathcal{P}_{e_1}$ on input 0 for a step each. Then, run $\mathcal{P}_{e_0}$ and $\mathcal{P}_{e_1}$ on input 0 for a second step, and also run $\mathcal{P}_{e_0}$ and $\mathcal{P}_{e_1}$ on input 1 for a step each. Continue in this way, and when a number is enumerated into either of W_{e_0} or W_{e_1}, enumerate it into their union.

(C) Yes. As in the prior item, dovetail enumerations of the two sets. When a number has been enumerated into both of W_{e_0} and W_{e_1} then enumerate it into their intersection.

(D) No. The Halting problem set K is computably enumerable but its complement is not.

II.7.32 We must show that if a set S is the range of a total computable function or is empty, then S is the domain of a partial computable function.

We first do the left to right implication. If S is empty then it is the domain of the partial computable function that diverges on all inputs. Otherwise S is the range of a total computable f; we will describe a partial computable g with domain S. To compute g: given the input $x \in \mathbb{N}$, enumerate $f(0), f(1), \ldots$ If x ever appears in this stream then halt the computation of g (with some generic output). If x never appears then the computation of $g(x)$ never halts. The domain of g is S.

For the other implication, assume that S is the domain of a partial computable function $\hat{g}$. We must show that S is either empty or is the range of a total computable function. The domain of a partial computable function can be empty; in the case that S is empty then we are done. Otherwise fix some $s_0 \in S$ and we will produce a total computable $\hat{f}$ whose range is S. Given $n \in \mathbb{N}$, run the computations of $\hat{g}(0), \hat{g}(1), \ldots \hat{g}(n)$, each for n-many steps. Possibly some of these computations halt. Define $\hat{f}(n)$ to be the least k where the computation of $\hat{g}(k)$ halts within n steps and where $k \notin \{\hat{f}(0), \hat{f}(1), \ldots \hat{f}(n-1)\}$. If no such k exists then define $\hat{f}(n) = s_0$ (this makes $\hat{f}$ a total function).

We finish by verifying that $\hat{f}$ is the desired enumeration. If $t \notin S$ then $\hat{g}(t)$'s computation never halts and so t is never enumerated by $\hat{f}$. If $t \in S$ then eventually the computation of $\hat{g}(t)$ must halt, in some number of

steps, n_t. The number t is then queued for output by $\hat{f}$ in that it will be enumerated as, at most, $\hat{f}(n_t + t)$.

II.8.17 We do not assume that there exists an in-principle physically realizable device that can correctly answer 'yes' or 'no' in a finite time to questions of the form, "Does $\mathcal{P}_e$ halt on input e?" But it is OK to have an argument with the phrase, "Assuming that we had a Halting problem oracle, then we could . . . "

Comment. It is worth a mention that the concept of an oracle, whether as described in this section or as extended to ideas such as oracles that can can solve cryptographic problems, or oracles that can sort number lists in linear time, is very commonly used to address in-practice problems.

II.8.18 A decider is a Turing machine, whose computed function is the characteristic function of some set. An oracle is a set, used as an addition to a Turing machine that the machine can query.

II.8.19 First of all, they are fascinating and fun. Beyond that, they help us understand the relationships among problems. This will include trying to understand which are easy and which are hard.

In the Complexity chapter we will expand on the idea to help understand how many everyday, practical, problems are related.

II.8.20 Their answer captures the spirit of the reason for considering oracle computations, but it is lacking in two ways. The first is that we can use a computable set, such as the empty set, as an oracle. (Although, it is true that with a computable oracle we cannot solve any problems we could not solve without that oracle.)

The second lack is that their answer doesn't refer to the mechanism at all. The oracle set doesn't solve the problems. We might informally say that, but for a definition we should say that we give a Turing machine access to answers about membership in the oracle and then the input-output behavior of that machine is the solution to the problem.

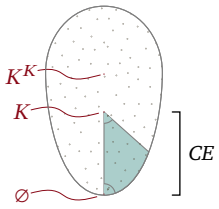
II.8.21 We take the first question to ask: is there a most-powerful oracle, one that solves every problem? The answer is no. Given a set $X \subseteq \mathbb{N}$, the problem of deciding membership in K^X, the relativized Halting problem for that oracle, is not solvable from X.

The second question asks whether, given a set $S \subseteq N$, there is an oracle that suffices to determine membership in S. The answer is yes: the oracle S will do.

II.8.22 Each oracle can only compute answers to some problems. (There are countably many $\mathcal{P}_e^X$ but uncountably many subsets of $\mathbb{N}$.) So for each oracle set there are problems that it cannot solve. In particular, it cannot solve the relativized Halting problem for that set.

II.8.23 In an oracle computation that halts there are indeed only finitely many calls to the oracle. But the calls when the computation's input is 0 may well differ from the calls when the computation's input is 1.

II.8.24 It is above K.



II.8.25 Recall that the notation for 'A is Turing-reducible to B' is $A \leq_T B$.

(A) This is false. For instance, the empty set is reducible to the Halting problem set, $\varnothing \leq_T K$, but while a decider for $\varnothing$ is trivial, there is no computable decider for K.

The correct statement is the opposite: if $A \leq_T B$ then we can use a decider for B to decide questions about membership in the set A.

(B) False. As in the prior item, if A is the empty set then it is computable. Then $\varnothing \leq_T K$ but K is not decidable. What's right is the other way around: if B is decidable then A is decidable also.

(C) True. If $A \leq_T B$ then for some index $e \in \mathbb{N}$, the function ϕ_e^B is the characteristic function of A, $\phi_e^B = \mathbb{1}_A$. With this function, if we could compute B then we could use it to compute A. So if A is uncomputable then B must be uncomputable also.

II.8.26 See Figure 42 on page 87.

(A) The flowchart on the left is straightforward.

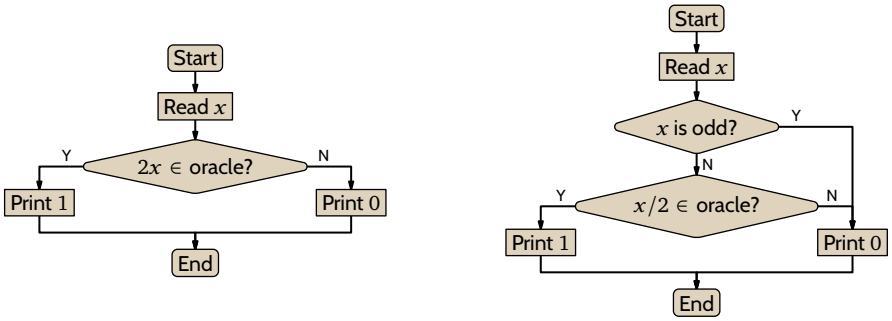


FIGURE 42, FOR QUESTION II.8.26: Showing that $B \leq_T 2B$ and $2B \leq_T B$.

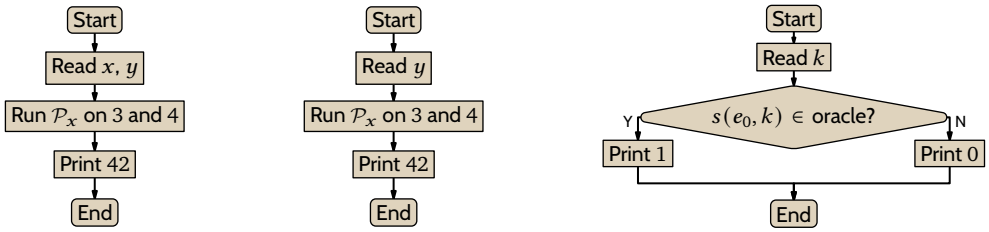
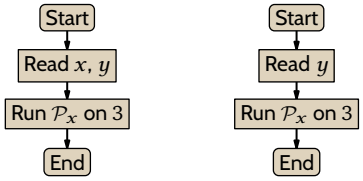


FIGURE 43, FOR QUESTION II.8.28: The set $\{x \mid \phi_x(3)\downarrow \text{ and } \phi_x(4)\downarrow\}$ Turing-reduces to K.

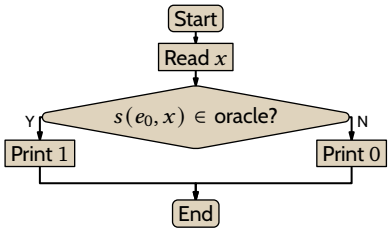
(B) For the flowchart on the right, notice that if the input x is odd then this machine must reply "no," since no odd number can be an element of $2B$.

II.8.27 Start with the function $\psi : \mathbb{N} \to \mathbb{N}$ below.

$$\psi(x,y) = \begin{cases} \mathcal{P}_x(3) & \text{– if } \mathcal{P}_x(3)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$



By Church's Thesis the first flowchart gives that ψ is mechanically computable. Let it have index e_0. Apply the *s-m-n* theorem to parametrize x, resulting in the machine sketched on the right, with index $s(e_0, x)$. The behavior of this machine does not depend on its input y so $\phi_{s(e_0,x)}(s(e_0,x))\downarrow$ if and only if $\phi_x(3)\downarrow$. Thus, given x, we can test whether $x \in A$ via this machine by using a K oracle.



II.8.28 On the left of Figure 43 on page 87 is a flowchart sketching a program. By Church's Thesis there is a Turing machine that computes it. Let the index of that machine be e_0. Apply the *s-m-n* theorem to get a family of machines parametrized by x. The x-th member of that family is shown in the middle. Clearly if it halts for any input then it halts for all inputs. Hence $\mathcal{P}_x$ halts on inputs 3 and 4 if and only if $s(e_0, x) \in K$. Restated, there is a computable function $x \mapsto s(e_0, x)$ so that $x \in \{e \mid \phi_e(3)\downarrow \text{ and } \phi_e(4)\downarrow\}$ if and only if $s(e_0, x) \in K$. Therefore $\{e \mid \phi_e(3)\downarrow \text{ and } \phi_e(4)\downarrow\} \leq_T K$.

Now we make that oracle machine using the computable function $x \mapsto s(e_0, x)$. See the chart on the right of the figure.

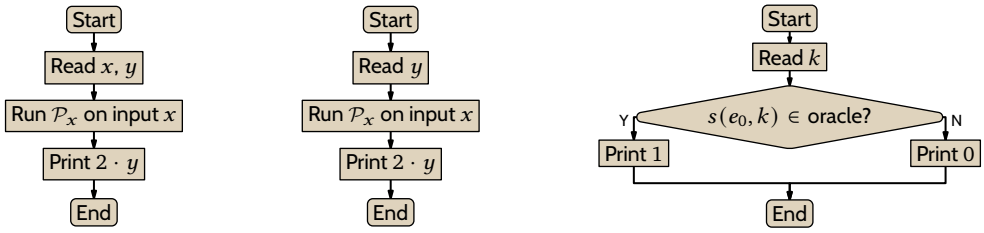


FIGURE 44, FOR QUESTION II.8.30: Showing that $K \leq_T \{e \mid \phi_e(y) = 2y \text{ for all input } y\}$.

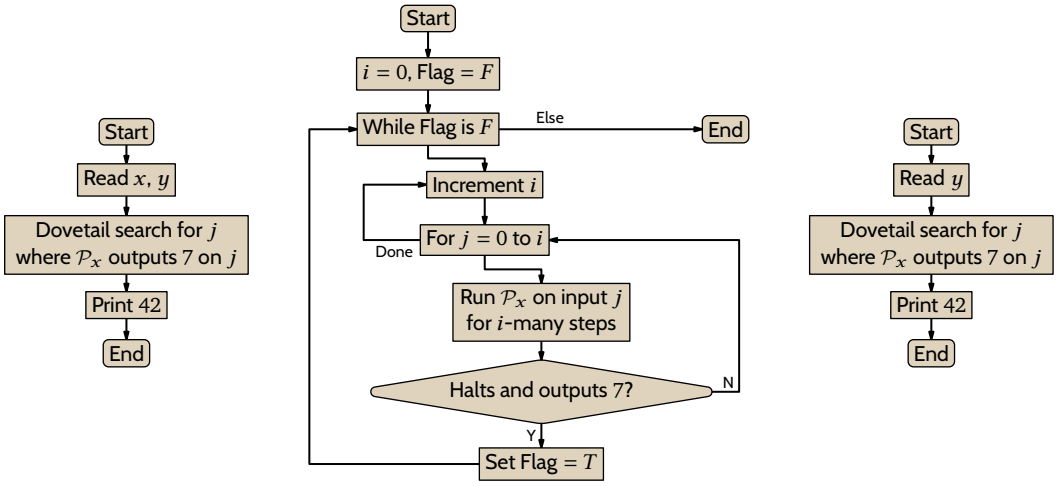


FIGURE 45, FOR QUESTION II.8.31: Computing $\{x \mid \phi_x(j) = 7 \text{ for some } j\}$ with a K oracle.

II.8.29 By definition, $K_0 = \{\langle e, x\rangle \mid \phi_e(x)\downarrow\}$. So $e \in S$ if and only if $\langle e, 3\rangle \in K_0$.

II.8.30 Let $D = \{e \mid \phi_e(y) = 2y \text{ for all inputs } y\}$. Consider this function.

$$\psi(x, y) = \begin{cases} 2y & \text{– if } \phi_x(x)\downarrow \\ \uparrow & \text{– otherwise} \end{cases}$$

By the flowchart on the left of Figure 44 on page 88, Church's Thesis gives that there is a Turing machine whose input/output behavior is ψ. Let that Turing machine have index e_0, so that $\phi_{e_0} = \psi$.

Apply the *s-m-n* theorem to get a family of functions $\phi_{s(e_0,x)}$ outlined in the middle of the figure. Then $x \in K$ if and only if $s(e_0, x) \in D$. On the right of the figure is an oracle machine giving that $K \leq_T D$, as required.

II.8.31

(A) Let $\mathcal{I} = \{e \mid \phi_e(j) = 7 \text{ for some } j\}$. This set is not empty because one element is an index of a Turing machine that outputs 7 for all inputs. This set is not all of $\mathbb{N}$ because one non-element is an index of a Turing machine that fails to halt on all inputs.

To see that $\mathcal{I}$ is an index set, suppose that $e \in \mathcal{I}$ and that $\hat{e}$ is such that $\phi_e \simeq \phi_{\hat{e}}$. Because $e \in \mathcal{I}$, there is an input j so that $\phi_e(j) = 7$. Because $\phi_e \simeq \phi_{\hat{e}}$, for the same j we have $\phi_{\hat{e}}(j) = 7$ and therefore $\hat{e} \in \mathcal{I}$ also.

(B) On the left of Figure 45 on page 88 is a flowchart sketching a program. The second chart gives extra detail on what's involved in 'dovetail'. By Church's Thesis there is a Turing machine that computes this flowchart. Let the index of that machine be e_0. Apply the *s-m-n* theorem to get a family of machines parametrized by x, shown in the third flowchart. Clearly it halts for all inputs y if and only if $\mathcal{P}_x$ outputs a 7 for any input. Thus, $\mathcal{P}_x$ ever outputs a 7 if and only if $s(e_0, x) \in K$. That means there is a computable function, $x \mapsto s(e_0, x)$, where $x \in \{e \mid \phi_e(j) = 7 \text{ for some } j\}$ if and only if $s(e_0, x) \in K$.

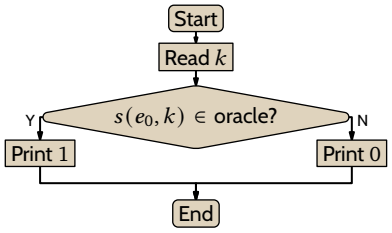


FIGURE 46, FOR QUESTION II.8.31: Oracle machine for computing $\{e \mid \phi_e(j) = 7 \text{ for some } j\}$ from K.

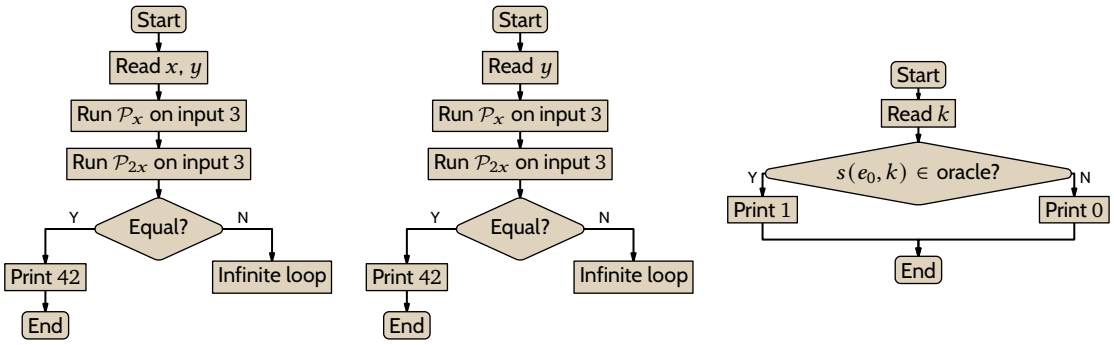


FIGURE 47, FOR QUESTION II.8.32: The set $\{x \mid \phi_x(3)\!\downarrow = \phi_{2x}(3)\!\downarrow\}$ is T-equivalent to K.

To see that $\{e \mid \phi_e(j) = 7 \text{ for some } j\} \leq_T K$, consider the oracle machine in Figure 46 on page 89. If we plug a K oracle into this machine then by the prior paragraph it acts as the characteristic function of $\{e \mid \phi_e(j) = 7 \text{ for some } j\}$.

II.8.32 On the left of Figure 47 on page 89 is a sketch of a computation. By Church's Thesis there is a Turing machine with the same behavior. Let the index of that machine be e_0. Apply the *s-m-n* theorem to get a family of machines parametrized by x, whose x-th member, $\mathcal{P}_{s(e_0,x)}$, is shown in the middle. Clearly it halts for all inputs y if and only if $x \in S$. In particular, it halts for $y = s(e_0, x)$ if and only if $x \in S$. Thus $s(e_0, x) \in K$ if and only if $x \in S$. With that it is easy to see that plugging oracle K into the the oracle machine on the right makes it act as the characteristic function of S. Thus $S \leq_T K$.

II.8.33 Consider the routine sketched on the left of Figure 48 on page 90. By Church's Thesis there is a Turing machine with that behavior. Let its index be e_0. Apply the *s-m-n* theorem to get a family of machines $\mathcal{P}_{s(e_0,x)}$, as shown in the middle. Observe that $x \in K$ if and only if $s(e_0, x) \in$ Tot. On the right is an oracle machine, which when we plug in a Tot oracle will act as the characteristic function of K. Thus $K \leq_T$ Tot.

II.8.34

(A) For contradiction suppose that the function steps has a computable extension g, which by the definition of extensible must be total. Then we could solve the Halting problem by running $\mathcal{P}_x$ on input x for $g(x)$-many steps. If the computation of $\phi_x(x)$ hasn't halted by then, it will never halt.

(B) Consider the routine sketched on the left of Figure 49 on page 90. By Church's Thesis there is a Turing machine with that behavior. Let its index be e_0. Apply the *s-m-n* theorem to parametrize x, giving a family of routines like the one shown in the middle, $\mathcal{P}_{s(e_0,x)}$. If $x \notin K$ then $\phi_{s(e_0,x)}(y)$ is undefined for all inputs y, and this function is a member of Ext, since for instance it is extended by the function that always returns 0, which is obviously computable. If $x \in K$ then $\phi_{s(e_0,x)}(y) = \text{steps}(y)$ for all inputs y, and by the first item this is not a member of Ext. Thus $k \in K$ if and only if $s(e_0, k) \notin$ Ext. The oracle machine on the right, when hooked up to an Ext oracle will act as the characteristic function of K. Therefore $K \leq_T$ Ext.

II.8.35 The computation will proceed in exactly the same way, so it will come to the same result.

II.8.36 The oracle machine is in Figure 50 on page 90. That machine, when hooked up to an A^c oracle, will act as the characteristic function of A.

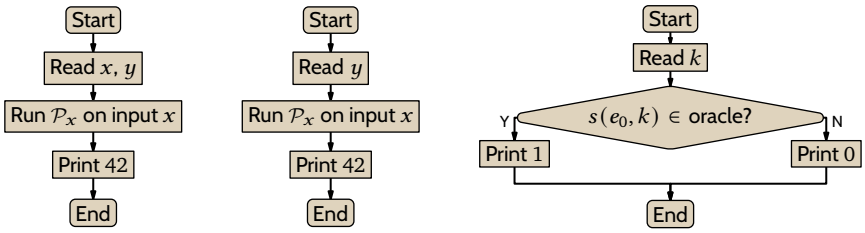


FIGURE 48, FOR QUESTION II.8.33: Computing K with a Tot oracle.

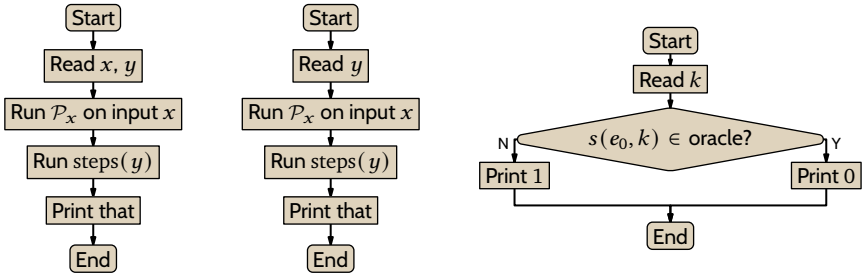


FIGURE 49, FOR QUESTION II.8.34: Computing K with a Ext oracle.

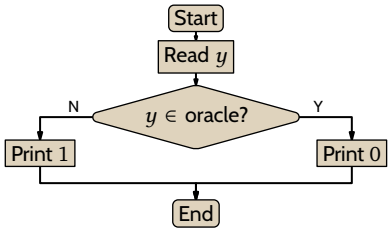


FIGURE 50, FOR QUESTION II.8.36: Computing A from an A^{c} oracle.

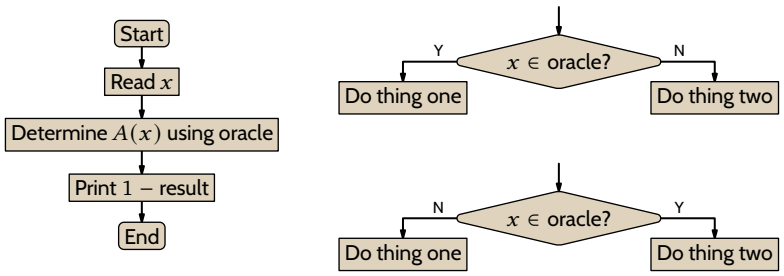


FIGURE 51, FOR QUESTION II.8.42: Oracle machines relating to the complement.

II.8.37 By Lemma 8.5 the only sets reducible to $\emptyset$ are computable. But K is not computable.

II.8.38 Any subset of $\mathbb{N}$ can be an oracle so there are uncountably many oracles.

II.8.39 There are only countably many programs that can involve an `oracle` call. That is, there are only countably many oracle-enhanced Turing machines, enumerated by the index e in $\mathcal{P}_e^X$.

II.8.40 One is $C = \{\text{cantor}(0,a) \bigm| a \in A\} \cup \{\text{cantor}(1,b) \bigm| b \in B\}$. Clearly from this set we can answer questions about membership in each of A and B.

II.8.41

(A) No. The Halting problem set K is a subset of the natural numbers $\mathbb{N}$ but $K \not\leq_T \mathbb{N}$, because the Halting problem is unsolvable.

(B) No. The Halting problem set K is not Turing reducible to $\mathbb{N} = K \cup K^{\mathsf{c}}$, but $K \subseteq K \cup K^{\mathsf{c}}$.

(C) No. The Halting problem set K is not Turing reducible to $\emptyset$, but $\emptyset = K \cap K^{\mathsf{c}}$.

(D) Yes. We can write this oracle-using program: read the input x and if it is an element of the oracle then output 0, otherwise output 1.

II.8.42 Refer to Figure 51 on page 91.

(A) True. This is shown in the flowchart on the left. Basically, interchange outputs of 'yes' and 'no'.

(B) True. This is shown in the middle flowchart. Replace all oracle calls like those on top with the ones on the bottom (the only difference is interchanging the 'Y' and 'N').

(C) True. Combine the first two items.

II.8.43

(A) The natural approach is to relativize Definition 7.1 and define that a set B is computably enumerable in the set A if B is the range of an A-computable function. That is, B is c.e. in A if $B = \{\phi_e^A(x) \bigm| x \in \mathbb{N}\}$ for some index e (that function may be a partial function).

(B) As in Lemma 8.5, we can output the elements of $\mathbb{N}$ without even referring to the oracle. The function $\phi^A(x) = x$ is computable, and does not need to ever refer to the oracle.

(C) Dovetail. First run the computation of $\mathcal{P}_0^A$ on input 0 for one step. Then run the computation of $\mathcal{P}_0^A$ on input 0 for a second step, and the computation of $\mathcal{P}_1^A$ on input 1 for one step. Next run the computation of $\mathcal{P}_0^A$ on input 0 for its third step, the computation of $\mathcal{P}_1^A$ on input 1 for its second step, and the computation of $\mathcal{P}_2^A$ on input 2 for one step. Continuing in this way, over time some of these halt. When a computation $\mathcal{P}_e^A(e)$ halts, enumerate the index e into K^A. (So in the enumerating function, $\phi^A(i)$ is the i-th index that is listed in this way.)

II.9.6 In everyday programming we can easily write two different source codes with the same input/output behavior, that is, computing the same function. Similarly, saying that the two are the same function, that $\phi_{g(v)} = \phi_{\phi_v(v)}$, is different than saying that they are produced by the same Turing machine, that the indices are the same.

II.9.7

(A) Apply the Fixed Point Theorem to the total computable function $f(x) = x + 7$.

(B) Apply the Fixed Point Theorem to the total computable function $f(x) = 2x$.

II.9.8 No matter what numbering scheme we use, (as long as it is acceptable) there is a Turing machine that computes the same function as the machine whose index is five more, so that there exists $k \in \mathbb{N}$ with

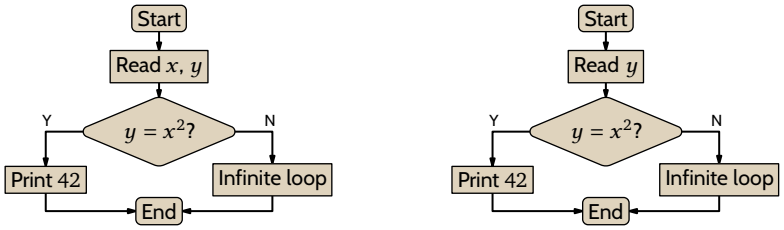


FIGURE 52, FOR QUESTION II.9.10: Flowcharts sketching the machine for $\psi = \phi_{e_0}$ and the machine for $\phi_{s(e_0,x)}$.

$\phi_k = \phi_{k+5}$.

The natural generalization is that for any acceptable numbering scheme and any constant $c \in \mathbb{N}$, there is a Turing machine that computes the same function as the machine whose index is c greater. That is, there exists a $k \in \mathbb{N}$ so that $\phi_k = \phi_{k+c}$.

II.9.9

(A) For any (acceptable) numbering scheme for Turing machines, there is one that computes the same function as the machine with three times its index. That is, there exists a $k \in \mathbb{N}$ so that $\phi_k = \phi_{3k}$.

(B) For any acceptable numbering scheme, there is a Turing machine that computes the same function as the machine whose index is the square. That is, there exists a $k \in \mathbb{N}$ so that $\phi_k = \phi_{k^2}$.

(C) Let $f : \mathbb{N} \to \mathbb{N}$ be this function.

$$f(x) = \begin{cases} 1 & \text{– if } x = 5 \\ 0 & \text{– otherwise} \end{cases}$$

For any acceptable numbering for Turing machines there will be a $k \in \mathbb{N}$ so that $\phi_k = \phi_{f(k)}$. (For example, the Padding Lemma says that there are many j where $\phi_j = \phi_0$.)

(D) If the function is $f(x) = 42$ then the Fixed Point Theorem gives that for any acceptable numbering there is a $k \in \mathbb{N}$ so that $\phi_k = \phi_{f(k)} = \phi_{42}$. This is not interesting because we already know by the Padding Lemma that there are infinitely many such indices k.

II.9.10

(A) Consider the function on the left below. It is computed by the program sketched on the left of Figure 52 on page 92. Church's Thesis gives that there is a Turing machine with this behavior. Let that machine's index be e_0.

$$\psi(x,y) = \begin{cases} 42 & \text{– if } y = x^2 \\ \uparrow & \text{– otherwise} \end{cases} \qquad \phi_{s(e_0,x)}(y) = \begin{cases} 42 & \text{– if } y = x^2 \\ \uparrow & \text{– otherwise} \end{cases}$$

Apply the *s-m-n* Theorem to get the family of functions on the right.

(B) The number e_0 is fixed because it is the index of a Turing machine that computes ψ. So define $g : \mathbb{N} \to \mathbb{N}$ by $g(x) = s(e_0, x)$.

(C) The function g is total and computable because s is total and computable. Apply the Fixed Point Theorem with g to get an $m \in \mathbb{N}$ with $\phi_m = \phi_{g(m)} = \phi_{s(e_0,m)}$. By construction that last function halts only on input $y = m^2$. Thus $W_m = \{ m^2 \}$.

II.9.11

(A) We can write a program in a standard programming language, such as Racket, that inputs n and outputs the n-th prime.

```
(require math/number-theory)
> (nth-prime 5)
13
> (nth-prime 666)
4987
```

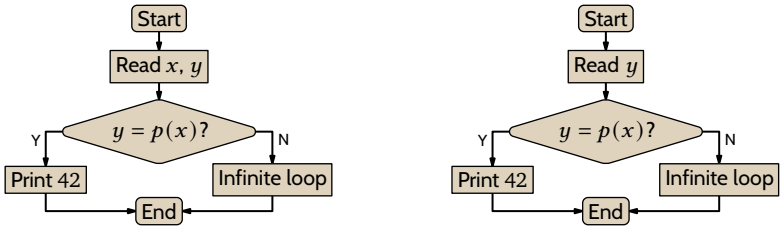


FIGURE 53, FOR QUESTION II.9.11: Flowcharts outlining the machine $\mathcal{P}_m$ that halts only on the m-th prime number.

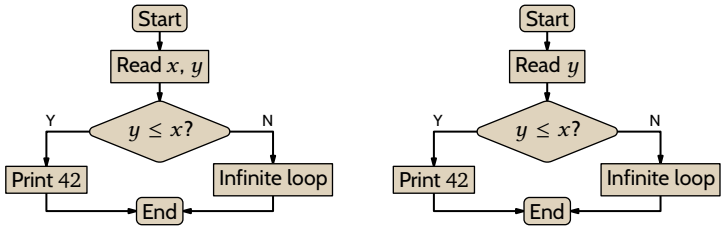


FIGURE 54, FOR QUESTION II.9.14: A sketch of the machine for $\psi = \phi_{e_0}$, and the machine for $\phi_{s(e_0,x)}$.

(B) Consider the function on the left side below, computed by the machines sketched on the left of Figure 53 on page 93. By Church's Thesis that function has an index; let that index be e_0.

$$\psi(x,y) = \begin{cases} 42 & \text{– if } y = p(x) \\ \uparrow & \text{– otherwise} \end{cases} \qquad \phi_{s(e_0,x)}(y) = \begin{cases} 42 & \text{– if } y = p(x) \\ \uparrow & \text{– otherwise} \end{cases}$$

Apply the *s-m-n* Theorem to get the uniformly computable family of functions on the right, parametrized by x. Observe that $\mathcal{P}_{s(e_0,x)}$ halts only on the x-th prime.

(C) The number e_0 is fixed so $s(e_0, x)$ is a function only of the variable x. To emphasize that, define $g\colon \mathbb{N} \to \mathbb{N}$ by $g(x) = s(e_0, x)$. The Fixed Point Theorem gives a $m \in \mathbb{N}$ with $\phi_m = \phi_{g(m)} = \phi_{s(e_0,m)}$, which halts only when the input is the m-th prime.

II.9.12 Consider the function on the left below. By Church's Thesis it is computable and so has an index; let that index be e_0.

$$\psi(x,y) = \begin{cases} 42 & \text{– if } y = 10^x \\ \uparrow & \text{– otherwise} \end{cases} \qquad \phi_{s(e_0,x)}(y) = \begin{cases} 42 & \text{– if } y = 10^x \\ \uparrow & \text{– otherwise} \end{cases}$$

Apply the *s-m-n* Theorem to get the uniformly computable family of functions on the right. Observe that $\phi_{s(e_0,x)}$ converges only on $y = 10^x$. The number e_0 is fixed so define $g\colon \mathbb{N} \to \mathbb{N}$ by $g(x) = s(e_0, x)$. The Fixed Point Theorem gives $m \in \mathbb{N}$ with $\phi_m = \phi_{g(m)} = \phi_{s(e_0,m)}$, and so $W_m = \{10^m\}$.

II.9.13 Consider the function $f\colon \mathbb{N} \to \mathbb{N}$ defined by $f(x) = y$ where all of the digits in y are one greater than the digits in x, except that a 9 in x changes to a 0 in y. Thus $f(35) = 46$, and $f(29) = 30$, and $f(99) = 0$. Clearly this function is computable and total. Apply the Fixed Point theorem.

II.9.14 Consider the function on the left below. It is computed by the machine sketched on the left in Figure 54 on page 93, so Church's Thesis gives that there is a Turing machine index for this machine. Let that index be e_0.

$$\psi(x,y) = \begin{cases} 42 & \text{– if } y \leq x \\ \uparrow & \text{– otherwise} \end{cases} \qquad \phi_{s(e_0,x)}(y) = \begin{cases} 42 & \text{– if } y \leq x \\ \uparrow & \text{– otherwise} \end{cases} \qquad (*)$$

Apply the *s-m-n* theorem to get the family of machines sketched on the right, which compute the family of

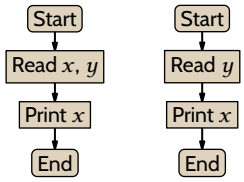


FIGURE 55, FOR QUESTION II.9.15: Flowcharts for ϕ_{e_0} and $\phi_{s(e_0,x)}$, for the machine that outputs its own index.

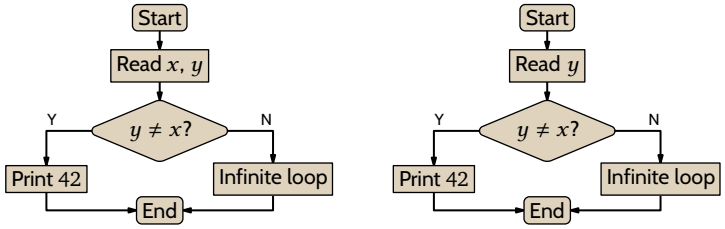


FIGURE 56, FOR QUESTION II.9.17: Sketches for $\psi = \phi_{e_0}$ and for $\phi_{s(e_0,x)}$.

functions given on the right of $(*)$. Observe that $\phi_{s(e_0,x)}$ converges only for inputs y that are less than or equal to x.

Because e_0 is the index of the machine on the left, it is fixed. So $s(e_0, x)$ is not a function of two variables, it is a function of the single variable x. Let $g\colon \mathbb{N} \to \mathbb{N}$ be defined by $g(x) = s(e_0, x)$. Then g is a computable and total function of one variable. By the Fixed Point Theorem Theorem 9.1, the function g has a fixed point so that there is a $n \in \mathbb{N}$ with $\phi_n = \phi_{g(n)} = \phi_{s(e_0,n)}$. Then $W_n = \{0, 1, \ldots n\}$.

II.9.15 See the flowcharts in Figure 55 on page 94. On the left is a computation that outputs its first input, $\phi_{e_0}(x, y) = x$. By Church's Thesis there is a Turing machine with that behavior; let it have index e_0. On the right we've applied the *s-m-n* theorem to parametrize that first input, resulting in $\phi_{s(e_0,x)}(y) = x$. Now let $f(x) = s(e_0, x)$. It is total and computable so the Fixed Point theorem applies. Hence there is a natural number k such that $\phi_k = \phi_{f(k)} = \phi_{s(e_0,k)}$. Since $\phi_{s(e_0,k)}(y) = k$ for all y, we are done.

II.9.16 Yes, we can take $f(x) = x$.

II.9.17

(A) The function on the left below is computed by the machine sketched on the left of Figure 56 on page 94. Church's Thesis gives that it has an index; let that index be e_0.

$$\psi(x,y) = \begin{cases} 42 & \text{– if } y \neq x \\ \uparrow & \text{– otherwise} \end{cases} \qquad \phi_{s(e_0,x)}(y) = \begin{cases} 42 & \text{– if } y \neq x \\ \uparrow & \text{– otherwise} \end{cases}$$

The *s-m-n* theorem parametrizes x to get the family of functions on the right. The number e_0 is fixed so $s(e_0, x)$ is a function of one variable. Consequently, define $g\colon \mathbb{N} \to \mathbb{N}$ by $g(x) = s(e_0, x)$. The Fixed Point Theorem gives $m \in \mathbb{N}$ with $\phi_m = \phi_{g(m)} = \phi_{s(e_0,m)}$, and so $W_m = \mathbb{N} - \{m\}$.

(B) No such computably enumerable W_m exists. By definition W_m is the set $\{x \mid \phi_m(x)\downarrow\}$. It can't also be the set $\{x \mid \phi_m(x)\uparrow\}$.

II.9.18

(A) Consider the function $\psi\colon \mathbb{N}^2 \to \mathbb{N}$ given by: $\psi(x, y) = 42$ if $y = x + 1$ and $\psi(x, y)\uparrow$ otherwise. It is computed by the machine on the left of Figure 57 on page 95. So by Church's Thesis there is a Turing machine with that behavior. Let that be machine $\mathcal{P}_{e_0}$. Apply the *s-m-n* Theorem to get the machine on the right. which computes the function $\phi_{s(e_0,x)}(y)$ with the property that it converges only on $x + 1$. Let $g\colon \mathbb{N} \to \mathbb{N}$ be given by $g(x) = s(e_0, x)$. By the Fixed Point Theorem there is an $m \in \mathbb{N}$ where $\phi_m = \phi_{g(m)} = \phi_{s(e_0,m)}$, which has domain $\{m + 1\}$.

(B) Consider the function $\psi(x, y) = 2x$. It is computed by the machine sketched on the left of Figure 58 on page 95. So Church's Thesis gives that there is a machine with this behavior. Let its index be e_0. Apply the

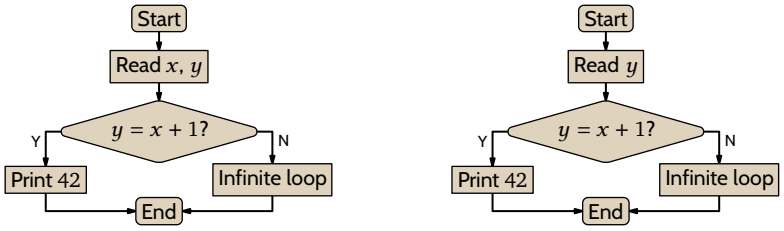


FIGURE 57, FOR QUESTION II.9.18: Flowcharts for a function ϕ_m whose domain is $\{m+1\}$.

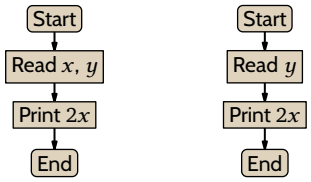


FIGURE 58, FOR QUESTION II.9.18: Flowcharts for a function ϕ_m whose range is $\{2m\}$.

s-m-n theorem to get the family of machines on the right of that figure. Define $g\colon \mathbb{N} \to \mathbb{N}$ by $g(x) = s(e_0, x)$ (the number e_0 is a constant so $s(e_0, x)$ is a function of one variable). The Fixed Point Theorem gives a number m such that $\phi_m = \phi_{g(m)}$, and since the range of $\phi_{g(m)}$ is the singleton set $\{2m\}$, we are done.

II.9.19 By Corollary 9.3 there is an index e satisfying this.

$$\phi_e(y) = \begin{cases} \downarrow & \text{– if } y = e \\ \uparrow & \text{– otherwise} \end{cases} \qquad (*)$$

This means that $e \in K$.

Now, by the Padding Lemma there exists $\hat{e} \neq e$ such that $\phi_{\hat{e}} \simeq \phi_e$. Then $(*)$ gives that $\phi_{\hat{e}}(\hat{e})\uparrow$, and so $\hat{e} \neq K$. Therefore K is not an index set.

II.9.20

(A) The set F is finite so the set of functions computed by indices that are members of F is finite. But there are infinitely many partial computable functions, so there must be ones that are not computed by any index in F.

(B) There are two possibilities for the fixed point x. If $x \in F$ then $h(x) = e_0$. Since e_0 is an index for g, which has no indices in F, it cannot be that $\phi_{h(x)} = \phi_{e_0} = \phi_x$.

The other case is $x \neq F$. Then $\phi_{h(x)} = \phi_{f(x)}$, but $\phi_{f(x)} \neq \phi_x$ because $x \neq F$, and so $\phi_{h(x)} \neq \phi_x$.

II.A.1 One is to put the passengers from bus B_0 in rooms 0, 100, 200, etc. Then put the passengers from B_1 in rooms 1, 101, 201, etc. In general, put $b_{i,j}$ in room $i + 100j$.

II.A.2 First move the current guests, assigning the person now in room n to room $2n$. That leaves free the odd-numbered rooms. Then Person j from bus B_i goes into $f(p_{i,j}) = 2 \cdot \text{cantor}(i, j) + 1$.

II.A.3 Each person is a triple, $\langle i, j, k\rangle$ = ⟨floor number, space number for the bus, person number on that bus⟩. The natural way to assign them rooms is to clear the odd-numbered rooms and then put $p_{i,j,k}$ into $2 \cdot \text{cantor}(\text{cantor}(i, j), k) + 1$.

II.A.4 No, the power set $\mathscr{P}(\mathbb{R})$ has more members than does $\mathbb{R}$.

II.D.3 The Δ function has $n \cdot 2$ inputs and $(n+1) \cdot 2 \cdot 2$ outputs. So there are $(2n)^{4(n+1)}$ machines.

II.D.4 The Racket code in Figure 59 on page 96 computes it. The values are 6, 16, 34, 64, 114, 196, 334, 564, 946, 1584, 2646, 4416, 7366, and 12284.

II.D.5 The exercise does not ask us to produce a computable function; indeed, there is no such computable function.

Define $f(0)$ to be $1 + \phi_0(0)$ if $\phi_0(0)$ converges, otherwise define $f(0) = 1$. (Note that f is not a computable

```
(define (g n)
  (let ([i (modulo n 3)])
    (cond
      [(= i 0) (/ (+ (* n 5) 18) 3)]
      [(= i 1) (/ (+ (* n 5) 22) 3)]
      [else -1])))

; Show values computed while finding Busy Beaver 5.
;  r  values; usually initially use r=0
;  N  cap on number of values to compute; use N=10 for instance
(define (BB5 r N)
  (let ([val (g r)])
    (if (> val 0)
        (begin
          (writeln (number->string val))
          (BB5 val (- N 1)))
        (writeln "done"))))
```

FIGURE 59, FOR QUESTION II.D.4: Racket code producing values found during computation of BB(5).

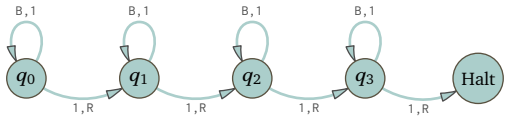
function.) Similarly define $f(1)$ to be the maximum of $1 + f(0)$, and $1 + \phi_0(1)$, and $1 + \phi_1(1)$ where if either of the last two diverges then we don't include it in the maximum. Do the same for all n.

More formally, define $F(e, x)$ to be $\phi_e(x)$ if it converges and to be 0 otherwise. Then $f(i)$ is the maximum of $\{F(0, i), \ldots F(i, i)\}$ plus 1.

II.D.6 Following the hint, note that there are uncountably many functions with output in the set $\{0, 1\}$, that is, where every output is less than or equal to 1. There are only countably many Turing machines, so there are only countably many computable functions, and so there are only countably many computable functions with output bounded by 1. Therefore there is a uncomputable function with output bounded by 1.

II.D.7

(A) This is best answered with an example: here is $\mathcal{M}_4$.



(B) The first is true because $f(m)$ is a summand in the expression for $F(m)$, and similarly the second is true because m^2 is a summand. The third holds because all of the summands for $m > 0$ are positive.

(C) If started on a blank tape this machine will first produce j-many 1's, then produce $F(j)$-many 1's, and finally will leave the tape with $F(F(j))$-many 1's. Thus its productivity is $F(F(j))$.

(D) Since F is strictly increasing, $F(j + 2n_F) < F(F(j))$. Combining gives $f(j + 2n_F) < F(j + 2n_F) < F(F(j)) \leq \Sigma(j + 2n_F)$, as required.

II.E.1

```
> (cantor-pairing 42)
'(6 2)
```

II.E.2

```
> (cantor-pairing 666)
'(0 36)
```

II.E.3

```
> (cantor-unpairing 3 0)
9
> (for ([i '(0 1 2 3 4 5 6 7 8 9)])
    (display (cantor-pairing-omega i)) (newline))
()
(0)
(0 0)
(1)
(0 1)
(0 0 0)
```

```
(2)
(1 0)
(0 0 1)
(0 0 0 0)
```

Chapter III: Languages, Grammars, and Graphs

III.1.17

(A) Five are `000`, `001`, `010`, `011`, and `100`.

(B) These five work: `0`, `1`, `00`, `11`, and `000`. (Is the empty string in that language? It perhaps is arguable, but certainly it is not the case that its first and last strings differ.)

III.1.18 No. By definition, a language is a set of strings and also by definition, a string is of finite length. For some real numbers the decimal representation does not make a finite string; one is $\pi = 3.14\ldots$ and another is $1/3 = 0.33\ldots$

III.1.19 It is both.

III.1.20 If $\beta = \langle b_0, \ldots, b_{i-1} \rangle$ is a string then $\beta^\frown \beta^R = \langle b_0, \ldots, b_{i-1} \rangle^\frown \langle b_{i-1}, \ldots, b_0 \rangle$ is clearly a palindrome. (This holds for the empty string also.) There are palindromes not of that form. One is $\gamma = \texttt{010}$, which cannot be decomposed into $\gamma = \beta^\frown \beta^R$, since it has odd length.

III.1.21

(A) These are the members of $\mathcal{L}_0{}^\frown \mathcal{L}_1$, where in the first line the strings start with an empty string, the second line's strings start with a single a, and so forth.

ε, b, bb, bbb,
a, ab, abb, abbb
aa, aab, aabb, aabbb
aaa, aaab, aaabb, aaabbb

(B) These are the members of $\mathcal{L}_1{}^\frown \mathcal{L}_0$.

ε, a, aa, aaa,
b, ba, baa, baaa
bb, bba, bbaa, bbaaa
bbb, bbba, bbbaa, bbbaaa

(C) We have this.

$$\mathcal{L}_0^2 = \mathcal{L}_0{}^\frown \mathcal{L}_0 = \{ \varnothing, \text{a}, \text{aa}, \text{aaa}, \text{aaaa}, \text{aaaaa}, \text{aaaaaa} \}$$

(D) There are many different correct answers, but a natural ten are ε, a, aa, ... a^9.

III.1.22

(A) The language consists of any number of a's followed by a single b. Thus five members are b, ab, aab, aaab, and aaaab.

(B) The language consists of a number of a's followed by the same number of b's. Thus five members are ε, ab, aabb, aaabbb, and aaaabbbb.

(C) The language is a set of strings of 1's followed by `0`'s, where there is one more `0` than `1`. Five members are `0`, `100`, `11000`, `1110000`, and `111100000`.

(D) The language consists of strings from $\mathbb{B}^*$ where there is a string of 1's, then a string of `0`'s, then a single `1`. The number of `0`'s is twice the number of leading 1's. Five members are `1`, `1001`, `1100001`, $1^3\texttt{0}^6\texttt{1}$, and $1^4\texttt{0}^8\texttt{1}$.

III.1.23

(A) $\mathcal{L}^2 = \{ \text{aa}, \text{aab}, \text{aba}, \text{abab} \}$

(B) We first enter into the set strings starting with aa, then strings starting with aab, etc. The result is $\mathcal{L}^3 = \{\,\mathsf{aaa}, \mathsf{aaab}, \mathsf{aaba}, \mathsf{aabab}, \mathsf{abaa}, \mathsf{abaab}, \mathsf{ababa}, \mathsf{ababab}\,\}$.

(C) $\mathcal{L}^1 = \mathcal{L} = \{\,\mathsf{a}, \mathsf{ab}\,\}$

(D) $\mathcal{L}^0 = \{\,\varepsilon\,\}$

III.1.24

(A) $\mathcal{L}_0 \frown \mathcal{L}_1 = \{\,\mathsf{ab}, \mathsf{abb}, \mathsf{abb}, \mathsf{abbb}\,\}$

(B) $\mathcal{L}_1 \frown \mathcal{L}_0 = \{\,\mathsf{ba}, \mathsf{bab}, \mathsf{bba}, \mathsf{bbab}\,\}$

(C) $\mathcal{L}_0^2 = \{\,\mathsf{aa}, \mathsf{aab}, \mathsf{aba}, \mathsf{abab}\,\}$

(D) $\mathcal{L}_1^2 = \{\,\mathsf{bb}, \mathsf{bbb}, \mathsf{bbbb}\,\}$

(E) $\mathcal{L}_0^2 \frown \mathcal{L}_1^2 = \{\,\mathsf{aabb}, \mathsf{aabbb}, \mathsf{aabbbb}, \mathsf{aabbbbb}, \mathsf{ababb}, \mathsf{ababbb}, \mathsf{ababbbb}, \mathsf{abababbb}\,\}$

III.1.25

(A) The minimum is three, which happens when the second is a subset of the first. The maximum is five, which happens when the two are disjoint.

(B) The minimum is zero, which happens when the two are disjoint. The maximum is two, which happens when the second is a subset of the first.

(C) The maximum is six, as with $\mathcal{L}_0 = \{\,\mathsf{a}, \mathsf{b}, \mathsf{c}\,\}$ and $\mathcal{L}_1 = \{\,\mathsf{x}, \mathsf{y}\,\}$, which gives $\mathcal{L}_0 \frown \mathcal{L}_1 = \{\,\mathsf{ax}, \mathsf{ay}, \mathsf{bx}, \mathsf{by}, \mathsf{cx}, \mathsf{cy}\,\}$. The minimum is four, as with $\mathcal{L}_0 = \{\,\varepsilon, \mathsf{a}, \mathsf{aa}\,\}$ and $\mathcal{L}_1 = \{\,\varepsilon, \mathsf{a}\,\}$, which gives $\mathcal{L}_0 \frown \mathcal{L}_1 = \{\,\varepsilon, \mathsf{a}, \mathsf{aa}, \mathsf{aaa}\,\}$.

(D) The minimum is three, as when $\mathcal{L}_1 = \{\,\varepsilon, \mathsf{a}\,\}$ gives $\mathcal{L}_1^2 = \{\,\varepsilon, \mathsf{a}, \mathsf{aa}\,\}$. The maximum is four, an example of which is $\mathcal{L}_1 = \{\,\mathsf{a}, \mathsf{b}\,\}$ giving $\mathcal{L}_1^2 = \{\,\mathsf{aa}, \mathsf{ab}, \mathsf{ba}, \mathsf{bb}\,\}$.

(E) The only number is 2. The two elements of $\mathcal{L}_1$ cannot reverse to be the same element, or else they would be the same element before the reversal. (This also holds if one of the elements is the empty string.)

(F) The intersection can be infinite, as with $\mathcal{L}_0 = \{\,\varepsilon, \mathsf{a}, \mathsf{b}\,\}$ and $\mathcal{L}_1 = \{\,\varepsilon, \mathsf{a}\,\}$, where $\mathsf{a}^n \in \mathcal{L}_0{}^* \cap \mathcal{L}_1{}^*$ for all $n \in \mathbb{N}$. The smallest that the intersection can be is size 1, as when $\mathcal{L}_0 = \{\,\mathsf{a}, \mathsf{b}, \mathsf{c}\,\}$ and $\mathcal{L}_1 = \{\,\mathsf{x}, \mathsf{y}\,\}$ and the only element of $\mathcal{L}_0{}^* \cap \mathcal{L}_1{}^*$ is the empty string.

III.1.26 It is the empty set, $\varnothing^* = \varnothing$. By definition, $\mathcal{L}^* = \{\,\sigma_0 \frown \cdots \frown \sigma_{k-1} \;\big|\; k \in \mathbb{N} \text{ and } \sigma_0, \ldots, \sigma_{k-1} \in \mathcal{L}\,\}$. If the language is empty then the condition '$\sigma_0, \ldots, \sigma_{k-1} \in \mathcal{L}$' is never met.

III.1.27 No. Let $\mathcal{L} = \{\,\mathsf{a}, \mathsf{b}\,\}$ and let $k = 2$. Then the k-th power of the language is $\mathcal{L}^k = \{\,\mathsf{aa}, \mathsf{ab}, \mathsf{ba}, \mathsf{bb}\,\}$ while the language of k-th powers is $\{\,\sigma^2 \;\big|\; \sigma \in \mathcal{L}\,\} = \{\,\mathsf{aa}, \mathsf{bb}\,\}$.

III.1.28 Yes, but only in an edge case. If $\mathcal{L} = \varnothing$ then $\mathcal{L}^* = \varnothing$ while $(\mathcal{L} \cup \{\,\varepsilon\,\})^* = \{\,\varepsilon\,\}$.

We will show that if $\mathcal{L} \neq \varnothing$ then the two sets are equal, $\mathcal{L}^* = (\mathcal{L} \cup \{\,\varepsilon\,\})^*$. The containment $\mathcal{L}^* \subseteq (\mathcal{L} \cup \{\,\varepsilon\,\})^*$ is clear.

For the other direction, that $(\mathcal{L} \cup \{\,\varepsilon\,\})^* \subseteq \mathcal{L}^*$, assume that $\sigma \in (\mathcal{L} \cup \{\,\varepsilon\,\})^*$ where $\sigma = \sigma_0 \frown \cdots \frown \sigma_{n-1}$ for some $n \in \mathbb{N}$ and all of the σ_i are elements of $\mathcal{L} \cup \{\,\varepsilon\,\}$. If for all $i \in \{\,0, \ldots n-1\,\}$ the string σ_i equals ε then $\sigma = \varepsilon$, and is an element of $\mathcal{L}^*$ because $\mathcal{L} \neq \varnothing$ and $\varepsilon = \rho^0$ for any $\rho \in \mathcal{L}$. Otherwise there is at least one $i \in \{\,0, \ldots n-1\,\}$ such that $\sigma_i \neq \varepsilon$. In this case, omitting from the string σ those σ_j that equal the empty string gives that $\sigma \in \mathcal{L}^k$ for some $k \leq n$, and so $\sigma \in \mathcal{L}^*$.

III.1.29 For any language we refer to its alphabet as Σ.

(A) Consider unequal strings $\sigma_0 \neq \sigma_1$. They can't both be the empty string, and if one of them is the empty string but the other is not then clearly $\sigma_0^2 \neq \sigma_1^2$ because one of those two is empty while the other is not. So now suppose that neither of the two strings σ_0 and σ_1 is empty. If they are different lengths then their squares are also different lengths. The case remaining is that they are nonempty equal-length strings, $\sigma_0 = \langle s_{0,0}, s_{0,1}, \ldots\ s_{0,n}\rangle$ and $\sigma_1 = \langle s_{1,0}, s_{1,1}, \ldots\ s_{1,n}\rangle$ for some $n \in \mathbb{N}$. Fix the smallest index i where $s_{0,i} \neq s_{1,i}$. For that same i we have that σ_0^2 and σ_1^2 differ on index i, and are therefore unequal.

The prior paragraph implies that if $\mathcal{L}$ has k-many different elements $\mathcal{L} = \{\,\sigma_0, \ldots \sigma_{k-1}\,\}$ then the strings $\sigma_0^2, \ldots, \sigma_{k-1}^2$ are different and so $\mathcal{L}^2$ has at least k-many different elements.

(B) The language $\mathcal{L} = \{\,\varepsilon\,\}$ has $\mathcal{L}^2 = \{\,\varepsilon\,\}$.

(C) Let $\mathcal{L} = \{\,\sigma_0, \sigma_1, \ldots\ \sigma_{k-1}\,\}$ for $k \in \mathbb{N}^+$ (we take these strings to all be different from each other). Then the list $\sigma_0 \frown \sigma_0$, $\sigma_0 \frown \sigma_1$, $\ldots\ \sigma_{k-1} \frown \sigma_{k-1}$ has length k^2. Hence the largest number of elements that $\mathcal{L}^2$ can have is k^2.

(D) Consider the language $\mathcal{L} = \{\,\sigma_0, \sigma_1, \ldots\ \sigma_{k-1}\,\}$ where the k-many strings have length one and are unequal, so each consists of a single symbol that is different from the symbols used in any of the others. Clearly all of the members of the list $\sigma_0 \frown \sigma_0$, $\sigma_0 \frown \sigma_1$, $\ldots\ \sigma_{k-1} \frown \sigma_{k-1}$ are unequal and so $\mathcal{L}^2$ has k^2-many elements.

III.1.30 Recall that $\mathcal{L}^k = \{\, \sigma_0 \frown \cdots \frown \sigma_{k-1} \;\big|\; \sigma_i \in \mathcal{L} \,\}$ and that $\mathcal{L}^* = \{\, \sigma_0 \frown \cdots \frown \sigma_{k-1} \;\big|\; k \in \mathbb{N} \text{ and } \sigma_0, \ldots, \sigma_{k-1} \in \mathcal{L} \,\}$. If $\mathcal{L} = \emptyset$ then by definition $\mathcal{L}^* = \{\,\varepsilon\,\}$. If $\mathcal{L} = \emptyset$ then by definition $\mathcal{L}^0 = \{\,\varepsilon\,\}$, while for any $k > 0$ we have $\mathcal{L}^k = \emptyset$.

So what remains is the argument for $\mathcal{L} \neq \emptyset$. We first show that $\mathcal{L}^k \subseteq \mathcal{L}^*$. If $k = 0$ then because we are assuming $\mathcal{L} \neq \emptyset$, we can take $\sigma \in \mathcal{L}$ and find $\sigma^0 = \varepsilon$. Thus $\mathcal{L}^0 = \{\,\varepsilon\,\}$ and $\varepsilon \in \mathcal{L}^*$. Now suppose that $k > 0$ and fix $\sigma \in \mathcal{L}^k$, so that $\sigma = \sigma_0 \frown \cdots \frown \sigma_{k-1}$. Then the definition of $\mathcal{L}^*$ gives that $\sigma \in \mathcal{L}^*$.

We finish by showing that $\mathcal{L}^* \subseteq \mathcal{L}^0 \cup \mathcal{L}^1 \cup \cdots$. Fix $\sigma \in \mathcal{L}^*$ (recall that $\mathcal{L} \neq \emptyset$), so that $\sigma = \sigma_0 \frown \cdots \frown \sigma_{k-1}$ for some $k \in \mathbb{N}$. Note that $\sigma \in \mathcal{L}^k$ (even if $k = 0$). Thus $\mathcal{L}^* \subseteq \mathcal{L}^0 \cup \mathcal{L}^1 \cup \cdots$.

III.1.31 The set of concatenations $\sigma_1 \frown \sigma_0$ where $\sigma_1 \in \mathcal{L}_1$ and $\sigma_0 \in \mathcal{L}_0$ is empty, because there are no σ_0 members of $\mathcal{L}_0$.

III.1.32

(A) They are ε, a, aa, b, and bb.

(B) The number of elements in the union is the sum of the number of elements in the first language plus the number of elements in the second. Because the alphabets are disjoint there is no overlap between the two languages.

(C) A string in the union of the languages may involve characters that are in either alphabet. So the union of a language over Σ_0 and a language over Σ_1 is a language over $\Sigma_0 \cup \Sigma_1$.

(D) To be in the intersection of the languages, a string must involve only characters that are in both alphabets. So the intersection of a language over Σ_0 with a language over Σ_1 is a language over $\Sigma_0 \cap \Sigma_1$.

III.1.33

(A) Yes, but it has nothing to do with the language being finite. Our definition is that an alphabet is finite. (Appendix A has a review.)

(B) Because the language is finite, either the language is empty and for B we can use 0, or else there is a longest string and for B we can take the length of that string.

(C) Let $\mathcal{L}_0, \ldots \mathcal{L}_{k-1}$ be finite languages over Σ^* for some $k \in \mathbb{N}$. Then the union is also finite because it contains at most $|\mathcal{L}_0| + \cdots + |\mathcal{L}_{k-1}|$ elements.

(D) Let $\mathcal{L}_0, \ldots \mathcal{L}_{k-1}$ be finite languages over Σ^*. Then the intersection contains at most $\min(|\mathcal{L}_0|, \ldots |\mathcal{L}_{k-1}|)$-many elements, so it is finite also. The number of strings in the concatenation $\mathcal{L}_0 \frown \mathcal{L}_1 \frown \cdots \frown \mathcal{L}_{k-1}$ is at most the product $|\mathcal{L}_0| \cdot |\mathcal{L}_0| \cdots |\mathcal{L}_{k-1}|$.

(E) If $\Sigma = \{\, \mathtt{a}, \mathtt{b} \,\}$ then the complement of the finite language $\mathcal{L} = \{\,\}$ is $\mathcal{L}^c = \Sigma^*$, which is infinite since it contains $\mathtt{a}^n$ for all n. For the same alphabet, $\mathcal{L} = \{\, \mathtt{a}, \mathtt{b} \,\}$ is finite but $\mathcal{L}^*$ is infinite.

III.1.34 In $\hat{\mathcal{L}}$ all strings are of even length, so it contains all of the even-length palindromes. The language $\mathcal{L}$ contains all palindromes, of any length.

III.1.35 The prefixes are ε, a, b, ab, bb, aba, bba, abaa, abaab, and abaaba.

III.1.36

(A) Concatenating $\sigma_0 \frown \cdots \frown \sigma_{m-1} \in \mathcal{L}^m$ with $\tau_0 \frown \cdots \frown \tau_{n-1} \in \mathcal{L}^n$ gives $\sigma_0 \frown \cdots \frown \sigma_{m-1} \frown \tau_0 \frown \cdots \frown \tau_{n-1} \in \mathcal{L}^{m+n}$.

(B) By definition $\mathcal{L}_0 \frown \mathcal{L}_1 = \{\, \sigma_0 \frown \sigma_1 \;\big|\; \sigma_0 \in \mathcal{L}_0 \text{ and } \sigma_1 \in \mathcal{L}_1 \,\}$. If one of the two is the empty set then the condition is never satisfied, so the result is the empty set.

(C) For any string $\sigma \in \mathcal{L}^*$ we have that $\sigma^0 = \varepsilon$. Thus, if there are any strings in the language then when raised to the zero power they all give the empty string, so the result is $\mathcal{L}^0 = \{\,\varepsilon\,\}$.

(D) If $\mathcal{L} = \emptyset$ gave that $\mathcal{L}^0 = \emptyset$ then for any language $\hat{\mathcal{L}}$, extending the formula for the prior item results in $\hat{\mathcal{L}} = \hat{\mathcal{L}}^1 = \hat{\mathcal{L}}^{1+0} = \hat{\mathcal{L}} \frown \emptyset = \emptyset$ (the final equality follows from the second item), which is nonsense.

III.1.37

(A) Consider the set $S = \Sigma \times \cdots \times \Sigma$ (we are avoiding calling this Σ^n because the two differ, in that the Cartesian product is set of n-tuples while Σ^n is a subset of Σ^*). It is the Cartesian product of countably many countable sets and so is countable by Chapter Two's Corollary 2.9. So there is a one-to-one and onto function $g\colon \mathbb{N} \to S$. The function concat$\colon S \to \Sigma^n$ that concatenates its arguments together $f(\sigma_0, \ldots \sigma_{n-1}) = \sigma_0 \frown \cdots \frown \sigma_{n-1}$ is clearly onto. So the composition $f \circ g\colon \mathbb{N} \to \Sigma^n$ is onto. Then by Chapter Two's Lemma 2.12, Σ^n is countable.

(B) The set $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \cdots$ is the countable union of countable sets and so by Chapter Two's Corollary 2.13 is a countable set.

III.1.38 We are asked to verify that any two parenthesizations of concatenation yield the same string. To prevent confusion between the two forms, for this argument only denote $\{\sigma_0 \frown \cdots \frown \sigma_{k-1} \bigm| \sigma_i \in \mathcal{L}\}$ as ${}^k\mathcal{L}$.

We first show that any element of ${}^k\mathcal{L}$ is an element of $\mathcal{L}^k$. We do a simple induction argument. The base cases are that ${}^0\mathcal{L}$ and $\mathcal{L}^0$ both equal $\{\varepsilon\}$ by definition, and that $\sigma_0 \in \mathcal{L}^1$ and $\sigma_0 \in {}^1\mathcal{L}$ for any $\sigma_0 \in \mathcal{L}$. The inductive hypothesis is that $\sigma_0 \frown \cdots \frown \sigma_{i-1} \in {}^i\mathcal{L}$ implies that $\sigma_0 \frown \cdots \frown \sigma_{i-1} \in \mathcal{L}^i$, for $i \in \{0, \ldots j\}$. For the $i = j+1$ inductive step assume that we have a concatenation of $j+1$-many strings, $\sigma_0 \frown \cdots \frown \sigma_{j-1} \frown \sigma_j \in {}^{j+1}\mathcal{L}$. Apply associativity of the string concatenation operation to rewrite it as $(\sigma_0 \frown \cdots \frown \sigma_{j-1}) \frown \sigma_j$, and by the definition of $\mathcal{L}^{j+1}$ in this exercise, $(\sigma_0 \frown \cdots \frown \sigma_{j-1}) \frown \sigma_j \in \mathcal{L}^{j+1}$.

The other direction is that for any k, any element of $\mathcal{L}^k$ is an element of ${}^k\mathcal{L}$. Here also we do induction. Again the $k = 0$ and $k = 1$ base cases are that ${}^0\mathcal{L}$ and $\mathcal{L}^0$ equal $\{\varepsilon\}$ by definition, and that $\sigma_0 \in \mathcal{L}^1$ and $\sigma_0 \in {}^1\mathcal{L}$ for any $\sigma_0 \in \mathcal{L}$. The inductive hypothesis is that if $\sigma \in \mathcal{L}^i$ then $\sigma \in {}^i\mathcal{L}$ for $i \in \{0, \ldots k\}$. Any element of $\mathcal{L}^j$ has the form $(\sigma_0 \frown \cdots \frown \sigma_{j-1}) \frown \sigma_j$. By associativity of string concatenation we can write it without the parentheses $\sigma_0 \frown \cdots \frown \sigma_{j-1} \frown \sigma_j$ (more precisely, any way of parenthesizing gives the same resulting string), which is an element of ${}^{j+1}\mathcal{L}$.

III.1.39 The statement is true. As motivation for the argument, the natural language to try for a counterexample is $\mathcal{L} = \{\mathtt{a}, \mathtt{aa}, \ldots\}$. However, $\mathcal{L} \frown \mathcal{L}$ is a proper subset of $\mathcal{L}$ since it does not contain $\mathtt{a}$.

With that, consider the $\mathcal{L} \neq \varnothing$ case, aiming to show that $\varepsilon \in \mathcal{L}$. The language is not empty so it contains at least one element. From among all such strings, let the shortest length be $\ell \in \mathbb{N}$. If $\ell > 0$ then any element of $\mathcal{L} \frown \mathcal{L}$ has length at least 2ℓ, which is strictly bigger than ℓ when $\ell > 0$. But $\mathcal{L} \frown \mathcal{L} = \mathcal{L}$ by the exercise assumption, giving a contradiction. The other possibility is that $\ell = 0$, which means that $\varepsilon \in \mathcal{L}$.

III.1.40 Every language is countable since it is a subset of Σ^*, which is countable because Σ is finite by definition of an alphabet. But there are uncountably many reals.

III.1.41

(A) For any sets, languages or not, union and intersection are commutative.

(B) We have $\mathcal{L} \frown \{\varepsilon\} = \{\sigma \frown \varepsilon \bigm| \sigma \in \mathcal{L}\} = \{\sigma \bigm| \sigma \in \mathcal{L}\} = \mathcal{L}$.

(C) Where $\Sigma = \{\mathtt{a}, \mathtt{b}\}$ take $\mathcal{L}_0$ to be the single-string set $\{\mathtt{a}\}$ and similarly let $\mathcal{L}_1 = \{\mathtt{b}\}$. Then $\mathcal{L}_0 \frown \mathcal{L}_1$ is also a single-string set, $\{\mathtt{ab}\}$, while $\mathcal{L}_1 \frown \mathcal{L}_0 = \{\mathtt{ba}\}$.

(D) This is immediate from the fact that string concatenation is associative: the i-th entry of $(\sigma_0 \frown \sigma_1) \frown \sigma_2$ is the same as the i-th entry of $\sigma_0 \frown (\sigma_1 \frown \sigma_2)$.

(E) This is immediate from the fact that for strings $(\sigma_0 \frown \sigma_1)^{\mathsf{R}}$ equals ${\sigma_1}^{\mathsf{R}} \frown {\sigma_0}^{\mathsf{R}}$.

(F) A string is an element of $(\mathcal{L}_0 \cup \mathcal{L}_1) \frown \mathcal{L}_2$ if and only if it has the form $\sigma \frown \tau$ where $\tau \in \mathcal{L}_2$, and $\sigma \in \mathcal{L}_0$ or $\sigma \in \mathcal{L}_1$. That's true if and only if both $\tau \in \mathcal{L}_2$ and $\sigma \in \mathcal{L}_0$, or both $\tau \in \mathcal{L}_2$ and $\sigma \in \mathcal{L}_1$. In turn that holds if and only if $\sigma \in (\mathcal{L}_0 \frown \mathcal{L}_2) \cup (\mathcal{L}_1 \frown \mathcal{L}_2)$. The right-distributive argument goes the same way.

(G) For any language $\mathcal{L}$ the definition gives $\varnothing \frown \mathcal{L} = \{\sigma_0 \frown \sigma_1 \bigm| \sigma_0 \in \varnothing \text{ and } \sigma_1 \in \mathcal{L}\}$. The condition '$\sigma_0 \in \varnothing$' is never satisfied so this set is empty. The same goes for $\mathcal{L} \frown \varnothing$.

(H) For any strings, no matter how the concatenation is parenthesized we can remove the parentheses. For instance, $((\sigma_0 \frown \sigma_1) \frown \sigma_2) \frown \sigma_3$ equals $((\sigma_0 \frown (\sigma_1 \frown \sigma_2)) \frown \sigma_3$ because the i-th elements of the two are equal. (We can easily show this statement by induction.) From this, the result is immediate.

III.2.11

(A) We've adopted the convention that the start symbol is the head of the first listed rule so it is ⟨*expr*⟩.

(B) The terminals are a, and b, . . . , and z.

(C) The nonterminals are ⟨*expr*⟩, ⟨*term*⟩, and ⟨*factor*⟩.

(D) There are twenty seven rewrite rules on the final line. The other two lines have a total of four, so that makes thirty one.

(E) Two are `a+b` and `j*h+k*m`.

(F) Three are `a+`, and `+`, and `**`.

III.2.12

(A) By our convention that it is the head of the first listed rule, the start symbol is ⟨*sentence*⟩.

(B) The terminals are `the`, `young`, `caught`, `man`, and `ball`.

(C) The nonterminals are ⟨*sentence*⟩, ⟨*noun phrase*⟩, ⟨*verb phrase*⟩, ⟨*article*⟩, ⟨*noun*⟩, ⟨*adjective*⟩, and ⟨*verb*⟩.

(D) Three are `the ball caught the young man`, `the ball caught the man`, and `the man caught the man`.

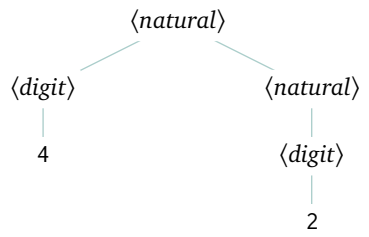


FIGURE 60, FOR QUESTION III.2.13: Parse tree for 42.

(E) There is no derivation for the length one string `ball` because it is a ⟨*noun*⟩ and those must be preceded by an ⟨*adjective*⟩. Similarly there is no derivation for `man man man`. A third is that there is no derivation for ε because every derivation that finishes must do so with non-empty terminals.

III.2.13

(A) The alphabet is $\Sigma = \{\texttt{0}, \ldots \texttt{9}\}$. The terminals are `0`, ... `9`. The nonterminals are ⟨*natural*⟩ and ⟨*digit*⟩. The start symbol is ⟨*natural*⟩.

(B) For the production

⟨*natural*⟩ → ⟨*digit*⟩

the head is ⟨*natural*⟩ while the body is ⟨*digit*⟩. For the production

⟨*natural*⟩ → ⟨*digit*⟩⟨*natural*⟩

the head is ⟨*natural*⟩ while the body is ⟨*digit*⟩⟨*natural*⟩. For the production

⟨*digit*⟩ → `0`

the head is ⟨*digit*⟩ and the body is `0`. The remaining nine productions are similar.

(C) The two metacharacters are → and |.

(D) Here is a derivation.

⟨*natural*⟩ ⇒ ⟨*digit*⟩⟨*natural*⟩
⇒ `4`⟨*natural*⟩
⇒ `4`⟨*digit*⟩
⇒ `42`

For the associated parse tree, see Figure 60 on page 103.

(E) Here is a derivation.

⟨*natural*⟩ ⇒ ⟨*digit*⟩⟨*natural*⟩
⇒ ⟨*digit*⟩⟨*digit*⟩⟨*natural*⟩
⇒ ⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩
⇒ ⟨*digit*⟩`9`⟨*digit*⟩
⇒ ⟨*digit*⟩`93`
⇒ `993`

The parse tree is Figure 61 on page 104.

(F) We are given a string and try to find a way, starting at the start symbol, to end at that string. Keeping on expanding is not the game.

(G) Here is one answer.

⟨*integer*⟩ → `+`⟨*natural*⟩ | `-`⟨*natural*⟩ | ⟨*natural*⟩
⟨*natural*⟩ → ⟨*digit*⟩ | ⟨*digit*⟩⟨*natural*⟩
⟨*digit*⟩ → `0` | `1` | `2` | `3` | `4` | `5` | `6` | `7` | `8` | `9`

(H) Yes, the grammar of the prior item allows us to derive both `+0` and `-0`. This is a derivation for `+0`.

⟨*integer*⟩ ⇒ `+`⟨*natural*⟩
⇒ `+`⟨*digit*⟩
⇒ `+0`

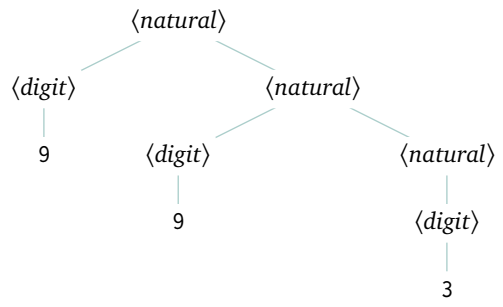


FIGURE 61, FOR QUESTION III.2.13: Parse tree associated with the derivation of 993.

By the way, here is a grammar that does not allow strings such as +0 in its language.

⟨*integer*⟩ → 0 | ⟨*nonzero natural*⟩ | ⟨*sign*⟩⟨*nonzero natural*⟩
⟨*nonzero natural*⟩ → ⟨*nonzero digit*⟩⟨*digits*⟩
⟨*digits*⟩ → ⟨*digits*⟩⟨*digit*⟩ | ⟨*digit*⟩
⟨*sign*⟩ → + | -
⟨*digit*⟩ → 0 | ⟨*nonzero digit*⟩
⟨*nonzero digit*⟩ → 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

III.2.14

(A) Here is a derivation of 'the car hit a wall'.

⟨*sentence*⟩ ⇒ ⟨*subject*⟩ ⟨*predicate*⟩ ⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*predicate*⟩
⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*verb*⟩ ⟨*direct object*⟩ ⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun*⟩
⇒ the ⟨*noun*⟩ ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun*⟩ ⇒ the car ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun*⟩
⇒ the car hit ⟨*article*⟩ ⟨*noun*⟩ ⇒ the car hit a ⟨*noun*⟩
⇒ the car hit a wall

(B) Here is a derivation of 'the car hit the wall'.

⟨*sentence*⟩ ⇒ ⟨*subject*⟩ ⟨*predicate*⟩ ⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*predicate*⟩
⇒ the ⟨*noun*⟩ ⟨*predicate*⟩ ⇒ the car ⟨*predicate*⟩
⇒ the car ⟨*verb*⟩ ⟨*direct object*⟩ ⇒ the car hit ⟨*direct object*⟩
⇒ the car hit ⟨*article*⟩ ⟨*noun*⟩ ⇒ the car hit the ⟨*noun*⟩
⇒ the car hit the wall

(C) This is one for 'the wall hit a car'.

⟨*sentence*⟩ ⇒ ⟨*subject*⟩ ⟨*predicate*⟩ ⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*predicate*⟩
⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*verb*⟩ ⟨*direct object*⟩ ⇒ ⟨*article*⟩ ⟨*noun*⟩ ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun*⟩
⇒ ⟨*article*⟩ ⟨*noun*⟩ hit ⟨*article*⟩ ⟨*noun*⟩ ⇒ ⟨*article*⟩ wall hit ⟨*article*⟩ ⟨*noun*⟩
⇒ ⟨*article*⟩ wall hit ⟨*article*⟩ car ⇒ ⟨*article*⟩ wall hit a car
⇒ the wall hit a car

III.2.15

(A) This is a derivation for 'dog bites man'.

⟨*sentence*⟩ ⇒ ⟨*subject*⟩ ⟨*predicate*⟩ ⇒ ⟨*article*⟩ ⟨*noun1*⟩ ⟨*predicate*⟩
⇒ ⟨*article*⟩ ⟨*noun1*⟩ ⟨*verb*⟩ ⟨*direct object*⟩ ⇒ ⟨*article*⟩ ⟨*noun1*⟩ ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun2*⟩
⇒ ε ⟨*noun1*⟩ ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun2*⟩ ⇒ ε dog ⟨*verb*⟩ ⟨*article*⟩ ⟨*noun2*⟩
⇒ ε dog bites ⟨*article*⟩ ⟨*noun2*⟩ ⇒ ε dog bites ε ⟨*noun2*⟩
⇒ ε dog bites ε man = dog bites man

(B) The start symbol ⟨*sentence*⟩ expands to ⟨*subject*⟩⟨*predicate*⟩. As to the first of those, ⟨*subject*⟩ expands to ⟨*article*⟩ ⟨*noun1*⟩, and ⟨*noun1*⟩ must expand to either of the terminals dog or flea. However, man is derived only from ⟨*noun2*⟩, which is derived only from ⟨*direct-object*⟩, which in turn is derived only from ⟨*predicate*⟩. So man cannot be the first word of the sentence, since it must be preceded by either dog or flea.

III.2.16 There is no nonterminal $\langle dog|flea\rangle$ or $\langle man|dog\rangle$. Your friend is confusing nonterminals with terminals.

III.2.17 The outermost operation is the $*$, so we go for that first. Next we get the parentheses and the $+$. Here is a derivation.

$$\begin{aligned}
&\langle expr\rangle \\
&\quad\Rightarrow \langle term\rangle * \langle factor\rangle \\
&\quad\Rightarrow \langle factor\rangle * \langle factor\rangle \\
&\quad\Rightarrow (\ \langle expr\rangle\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ \langle term\rangle + \langle expr\rangle\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ \langle term\rangle + \langle term\rangle\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ \langle factor\rangle + \langle term\rangle\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ a + \langle term\rangle\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ a + \langle factor\rangle\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ a + b\ ) * \langle factor\rangle \\
&\quad\Rightarrow (\ a + b\ ) * c
\end{aligned}$$

III.2.18

(A) First the leftmost derivation.

$$\begin{aligned}
S &\Rightarrow T\,b\,U \Rightarrow a\,T\,b\,U \Rightarrow a\,a\,T\,b\,U \Rightarrow a\,a\,\varepsilon\,b\,U = a\,a\,b\,U \\
&\Rightarrow a\,a\,b\,a\,U \Rightarrow a\,a\,b\,a\,b\,U \Rightarrow a\,a\,b\,a\,b\,\varepsilon = a\,a\,b\,a\,b
\end{aligned}$$

Next the rightmost derivation.

$$\begin{aligned}
S &\Rightarrow T\,b\,U \Rightarrow T\,b\,a\,U \Rightarrow T\,b\,a\,b\,U \Rightarrow T\,b\,a\,b\,\varepsilon = T\,b\,a\,b \\
&\Rightarrow a\,T\,b\,a\,b \Rightarrow a\,a\,T\,b\,a\,b \Rightarrow a\,a\,\varepsilon\,b\,a\,b = a\,a\,b\,a\,b
\end{aligned}$$

(B) This is the leftmost derivation.

$$\begin{aligned}
S &\Rightarrow T\,b\,U \Rightarrow \varepsilon\,b\,U = b\,U \Rightarrow b\,a\,U \\
&\Rightarrow b\,a\,a\,U \Rightarrow b\,a\,a\,b\,U \Rightarrow b\,a\,a\,b\,\varepsilon = b\,a\,a\,b
\end{aligned}$$

Here is the rightmost.

$$\begin{aligned}
S &\Rightarrow T\,b\,U \Rightarrow T\,b\,a\,U \Rightarrow T\,b\,a\,a\,U \Rightarrow T\,b\,a\,a\,b\,U \\
&\Rightarrow T\,b\,a\,a\,b\,\varepsilon = T\,b\,a\,a\,b \Rightarrow \varepsilon\,b\,a\,a\,b = b\,a\,a\,b
\end{aligned}$$

(C) The start symbol expands to something that includes the terminal b.

III.2.19

(A) Here is a derivation of aabb,

$$S \Rightarrow aABb \Rightarrow aaBb \Rightarrow aabb$$

of aaabb,

$$S \Rightarrow aABb \Rightarrow aaABb \Rightarrow aaaBb \Rightarrow aaabb$$

and of aabbb.

$$S \Rightarrow aABb \Rightarrow aABb \Rightarrow aaBb \Rightarrow aaBbb \Rightarrow aabbb$$

(B) The production in the first line shows that every derived string must begin with a and end with b. So three strings that cannot be derived are a, ba, and ε, the empty string.

(C) $\mathcal{L} = \{\, a^n b^m \;\big|\; n, m \geq 2 \,\}$

III.2.20 Here is one.

$$\begin{aligned}
S &\longrightarrow TU \\
T &\longrightarrow aTb \mid \varepsilon \\
U &\longrightarrow bUa \mid \varepsilon
\end{aligned}$$

Here is a derivation of $a^2b^{(2+1)}a^1 = aabbba$.

$$S \Rightarrow TU \Rightarrow aTbU \Rightarrow aaTbbU \Rightarrow aa\varepsilon bbU = aabbU \Rightarrow aabbbUa \Rightarrow aabbb\varepsilon a = aabbba$$

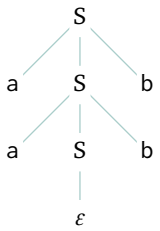


FIGURE 62, FOR QUESTION III.2.21: Parse tree for a^2b^2.

III.2.21 Figure 62 on page 106 shows the tree.

III.2.22 We will use induction to show that in the course of a derivation all of the results are of either the form a^k S b^k or of the form a^kb^k. By the definition of the language generated by a grammar, only strings without any nonterminals are in the language so that will give the verification.

The induction is on the number of derivation steps, that is, on the number of rule applications. The base step is that there are zero-many rule applications. In this case the string is 'S', which is the $k = 0$ instance of the desired form.

For the inductive step assume that the statement is true where the number of rule applications is $n = 0$, $n = 1, \ldots$ $n = k$ and consider the $n = k + 1$ case. We get the string for the $k + 1$ case by applying a rule to a string after the k case. The inductive hypothesis is that such a string has one of two forms, a^k S b^k or a^kb^k. The rules don't apply to the latter string so consider the former one.

Applied to a^k S b^k the first rule gives a^{k+1} S b^{k+1}, which has the right form. The second rule gives $a^{k+1}b^{k+1}$, again of the right form.

III.2.23 This grammar has no terminating derivations and so it generates the empty language, $\mathcal{L} = \emptyset$.

III.2.24 Here is one.

⟨*identifier*⟩ → ⟨*letter*⟩ | ⟨*letter-or-digit-string*⟩
⟨*letter-or-digit-string*⟩ → ⟨*letter*⟩ ⟨*letter-or-digit-string*⟩
| ⟨*digit*⟩ ⟨*letter-or-digit-string*⟩
| ε
⟨*letter*⟩ → a | ... | z | A | ... | Z
⟨*digit*⟩ → 0 | ... | 9

III.2.25

⟨*identifier*⟩ → ⟨*letter*⟩ ⟨*second*⟩
⟨*second*⟩ → ⟨*letter-or-digit*⟩ ⟨*third*⟩ | ε
⟨*third*⟩ → ⟨*letter-or-digit*⟩ ⟨*fourth*⟩ | ε
⟨*fourth*⟩ → ⟨*letter-or-digit*⟩
⟨*letter-or-digit*⟩ → ⟨*letter*⟩ | ⟨*digit*⟩
⟨*letter*⟩ → A | ... | Z
⟨*digit*⟩ → 0 | ... | 9

III.2.26 It is the empty language, $\mathcal{L} = \emptyset$.

III.2.27

(A) Figure 63 on page 107 shows one derivation.

(B) In the grammar's first rule, the only way to have a multiple clause expression is to join the clauses with ∧'s.

III.2.28

(A) ⟨*string*⟩ → ⟨*letter*⟩ ⟨*string*⟩ | ε
⟨*letter*⟩ → a | ... | z

(B) ⟨*string1*⟩ → ⟨*digit*⟩ ⟨*string2*⟩
⟨*string2*⟩ → ⟨*digit*⟩ ⟨*string2*⟩ | ε
⟨*digit*⟩ → 0 | ... | 9

$$\begin{aligned}
\langle \mathit{CNF} \rangle &\Rightarrow \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \wedge \langle \mathit{CNF} \rangle \\
&\Rightarrow \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \langle \mathit{Literal} \rangle \vee \langle \mathit{Disjunction} \rangle\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \langle \mathit{Literal} \rangle \vee \langle \mathit{Literal} \rangle\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \langle \mathit{Variable} \rangle \vee \langle \mathit{Literal} \rangle\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \langle \mathit{Literal} \rangle\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \langle \mathit{Variable} \rangle\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Literal} \rangle \vee \langle \mathit{Disjunction} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Literal} \rangle \vee \langle \mathit{Literal} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \langle \mathit{Variable} \rangle \vee \langle \mathit{Literal} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \texttt{x1} \vee \langle \mathit{Literal} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \texttt{x1} \vee \langle \mathit{Variable} \rangle\ \texttt{)} \\
&\Rightarrow \texttt{(}\ \texttt{x0} \vee \neg\, \texttt{x1}\ \texttt{)} \wedge \texttt{(}\ \texttt{x1} \vee \texttt{x2}\ \texttt{)}
\end{aligned}$$

FIGURE 63, FOR QUESTION III.2.27: Derivation for $(x_0 \vee \neg x_1) \wedge (x_1 \vee x_2)$.

III.2.29

(A) From this start of a derivation we can easily get to the address.

$$\begin{aligned}
\langle \mathit{postal\ address} \rangle &\Rightarrow \langle \mathit{name} \rangle\ \texttt{EOL}\ \langle \mathit{street\ address} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{personal\ part} \rangle\ \langle \mathit{last\ name} \rangle\ \langle \mathit{opt\ suffix} \rangle\ \texttt{EOL}\ \langle \mathit{street\ address} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{first\ name} \rangle\ \langle \mathit{last\ name} \rangle\ \langle \mathit{opt\ suffix} \rangle\ \texttt{EOL}\ \langle \mathit{street\ address} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \langle \mathit{last\ name} \rangle\ \langle \mathit{opt\ suffix} \rangle\ \texttt{EOL}\ \langle \mathit{street\ address} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \langle \mathit{opt\ suffix} \rangle\ \texttt{EOL}\ \langle \mathit{street\ address} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \varepsilon\ \texttt{EOL}\ \langle \mathit{street\ address} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&= \langle \mathit{char\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{house\ num} \rangle\ \langle \mathit{street\ name} \rangle\ \langle \mathit{apt\ num} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{street\ name} \rangle\ \langle \mathit{apt\ num} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \varepsilon\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{street\ name} \rangle\ \langle \mathit{apt\ num} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&= \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{street\ name} \rangle\ \langle \mathit{apt\ num} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \langle \mathit{apt\ num} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \varepsilon\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&= \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{town} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{town\ name} \rangle\ \texttt{,}\ \langle \mathit{state\ or\ region} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{char\ string} \rangle\ \texttt{,}\ \langle \mathit{state\ or\ region} \rangle \\
&\Rightarrow \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{digit\ string} \rangle\ \langle \mathit{char\ string} \rangle\ \texttt{EOL}\ \langle \mathit{char\ string} \rangle\ \texttt{,}\ \langle \mathit{char\ string} \rangle
\end{aligned}$$

(B) The problem is the 'B' in `221B`; it is not part of a digit string.

(C) Some possible modifications are to allow more than one line for the address, to allow addresses without a house number, and to allow names or addresses with non-ASCII characters.

Comment: in practice the best thing to do may well be just to use a blob of characters. See for instance `https://github.com/kdeldycke/awesome-falsehood?tab=readme-ov-file#postal-addresses`.

III.2.30

(A) These two rules suffice: $S \;\longrightarrow\; 11S \mid \varepsilon$.

(B) As with the prior item, this is straightforward: $S \;\longrightarrow\; 111S \mid \varepsilon$.

III.2.31

(A) This is a sentence of length one.

$$\langle \mathit{sentence} \rangle \;\Rightarrow\; \texttt{buffalo}\ \langle \mathit{sentence} \rangle \;\Rightarrow\; \texttt{buffalo}\ \varepsilon = \texttt{buffalo}$$

Here is a sentence of length one.

$$\begin{aligned}
\langle \mathit{sentence} \rangle &\Rightarrow \texttt{buffalo}\ \langle \mathit{sentence} \rangle \;\Rightarrow\; \texttt{buffalo buffalo}\ \langle \mathit{sentence} \rangle \\
&\Rightarrow \texttt{buffalo buffalo}\ \varepsilon = \texttt{buffalo buffalo}
\end{aligned}$$

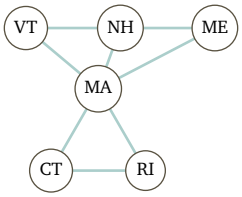


FIGURE 64, FOR QUESTION III.3.16: Adjancency relation among the New England states.

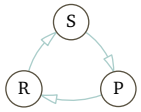


FIGURE 65, FOR QUESTION III.3.16: Relation of 'beats' among Rock, Paper, and Scissors.

And, a sentence of length three.

$$\begin{aligned}\langle \textit{sentence}\rangle &\Rightarrow \texttt{buffalo}\ \langle \textit{sentence}\rangle \Rightarrow \texttt{buffalo buffalo}\ \langle \textit{sentence}\rangle\\ &\Rightarrow \texttt{buffalo buffalo buffalo}\ \langle \textit{sentence}\rangle \Rightarrow \texttt{buffalo buffalo buffalo}\ \varepsilon\\ &= \texttt{buffalo buffalo buffalo}\end{aligned}$$

(B) In English, as a noun, 'buffalo' refers to the American bison or to a city in New York State. As a verb it refers to fooling or confusing someone. So, the sentence of length one could be a command instructing the listener to fool someone. The sentence of length two could be a command instructing that same listener to fool all of a city in upstate New York. The length three sentence could exhort American bison in particular to confuse that entire city. (Perhaps it is clearer with punctuation: "Buffalo, buffalo Buffalo!")

III.2.32

(A) This is a derivation of `(a . b)`.

$$\begin{aligned}\langle \textit{s expression}\rangle &\Rightarrow \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)} \Rightarrow \texttt{(}\ \langle \textit{atomic symbol}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \langle \textit{atomic symbol}\rangle\ \texttt{.}\ \langle \textit{atomic symbol}\rangle\ \texttt{)} \Rightarrow \texttt{(}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{.}\ \langle \textit{atomic symbol}\rangle\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \langle \textit{letter}\rangle\ \varepsilon\ \texttt{.}\ \langle \textit{atomic symbol}\rangle\ \texttt{)} = \texttt{(}\ \langle \textit{letter}\rangle\ \texttt{.}\ \langle \textit{atomic symbol}\rangle\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \langle \textit{atomic symbol}\rangle\ \texttt{)} \Rightarrow \texttt{(}\ \texttt{a .}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \langle \textit{letter}\rangle\ \varepsilon\ \texttt{)} = \texttt{(}\ \texttt{a .}\ \langle \textit{letter}\rangle\ \texttt{)} \Rightarrow \texttt{(}\ \texttt{a . b}\ \texttt{)}\end{aligned}$$

(B) This is a derivation of `(a . (b . c))`.

$$\begin{aligned}\langle \textit{s expression}\rangle &\Rightarrow \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \langle \textit{atomic symbol}\rangle\ \texttt{.}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{.}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a}\ \langle \textit{atomic part}\rangle\ \texttt{.}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a}\ \varepsilon\ \texttt{.}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)} = \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{s expression}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{atomic symbol}\rangle\ \texttt{.}\ \langle \textit{s expression}\rangle\ \texttt{)}\ \texttt{)} \Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{atomic symbol}\rangle\ \texttt{.}\ \langle \textit{atomic symbol}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{.}\ \langle \textit{atomic symbol}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{.}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{letter}\rangle\ \varepsilon\ \texttt{.}\ \langle \textit{letter}\rangle\ \langle \textit{atom part}\rangle\ \texttt{)}\ \texttt{)} = \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{letter}\rangle\ \texttt{.}\ \langle \textit{letter}\rangle\ \langle \textit{atomic part}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{letter}\rangle\ \texttt{.}\ \langle \textit{letter}\rangle\ \varepsilon\ \texttt{)}\ \texttt{)} = \texttt{(}\ \texttt{a .}\ \texttt{(}\ \langle \textit{letter}\rangle\ \texttt{.}\ \langle \textit{letter}\rangle\ \texttt{)}\ \texttt{)}\\ &\Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \texttt{b .}\ \langle \textit{letter}\rangle\ \texttt{)}\ \texttt{)} \Rightarrow \texttt{(}\ \texttt{a .}\ \texttt{(}\ \texttt{b . c}\ \texttt{)}\ \texttt{)}\end{aligned}$$

III.3.16

(A) Figure 64 on page 108 shows the graph. The abbreviations are: VT is Vermont, NH is New Hampshire, ME is Maine, MA is Massachusetts, CT is Connecticut, and RI is Rhode Island.

(B) The graph is a loop, Figure 65 on page 108.

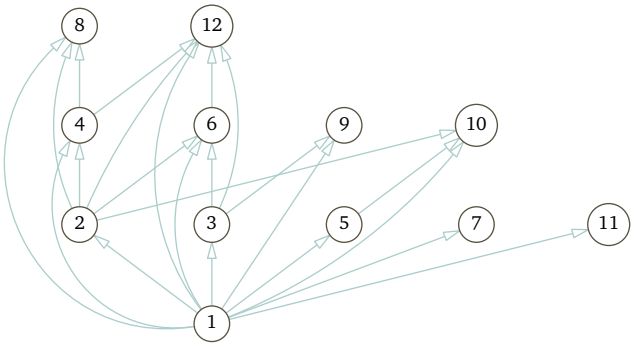


FIGURE 66, FOR QUESTION III.3.16: Divisibility relation among small numbers.

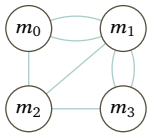


FIGURE 67, FOR QUESTION III.3.16: Bridge connection relation between land masses.

(C) The graph is Figure 66 on page 109.
(D) This graph is Figure 67 on page 109.

III.3.17

	Vertices can repeat?	*Edges can repeat?*	*Can be closed?*	*Can be open?*
Walk	Y	Y	Y	Y
Trail	Y	N	Y	Y
Circuit	Y	N	Y	N
Path	N	N	Y	Y
Cycle	N	N	Y	N

Note that a path could have equal first and last vertices, but no other vertices can be equal. A cycle must have that its first and last vertices are equal, but no other vertices repeat.

III.3.18 For example, the given information implies that taking Calculus I precedes taking Calculus III. But because it is implied by other edges, for visual design reasons we omit that edge from the prerequisite structure graph in Figure 68 on page 110.

III.3.19

(A) A picture is given in Figure 69 on page 110.
(B) Here is the matrix.

$$\mathcal{M}(\mathcal{G}) = \begin{array}{c} \\ v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccc} v_0 & v_1 & v_2 & v_3 & v_4 & v_5 \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \end{array}$$

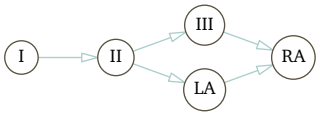


FIGURE 68, FOR QUESTION III.3.18: Prerequisite structure among courses shown with a minimum number of edges.

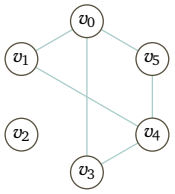


FIGURE 69, FOR QUESTION III.3.19: A graph with some edges.

(C) With six vertices, these are fifteen ways to choose four of them. This table includes all the allowed edges.

Line	*Vertices*	*Max edge set*
0	v_0, v_1, v_2, v_3	v_0v_1, v_0v_3
1	v_0, v_1, v_2, v_4	v_0v_1, v_0v_3, v_1v_4
2	v_0, v_1, v_3, v_4	$v_0v_1, v_0v_3, v_1v_4, v_3v_4$
3	v_0, v_2, v_3, v_4	v_0v_3, v_3v_4
4	v_1, v_2, v_3, v_4	v_1v_4, v_3v_4
5	v_0, v_1, v_2, v_5	v_0v_1, v_0v_5
6	v_0, v_1, v_3, v_5	v_0v_1, v_0v_3, v_0v_5
7	v_0, v_2, v_3, v_5	v_0v_3, v_0v_5
8	v_1, v_2, v_3, v_5	
9	v_0, v_1, v_4, v_5	$v_0v_1, v_0v_5, v_1v_4, v_4v_5$
10	v_0, v_2, v_4, v_5	v_0v_5, v_4v_5
11	v_1, v_2, v_4, v_5	v_1v_4, v_4v_5
12	v_0, v_3, v_4, v_5	$v_0v_3, v_0v_5, v_3v_4, v_4v_5$
13	v_1, v_3, v_4, v_5	v_1v_4, v_3v_4, v_4v_5
14	v_2, v_3, v_4, v_5	v_3v_4, v_4v_5

(Figure 69 on page 110 is handy in checking the final column entries.) There are three such graphs: $G_2 = \{v_0, v_1, v_3, v_4\}$ and $G_9 = \{v_0, v_1, v_4, v_5\}$ and $G_{12} = \{v_0, v_3, v_4, v_5\}$.

(D) It is the same three as in the prior item.

III.3.20

(A) See Figure 70 on page 111.

(B) See Figure 71 on page 111.

(C) It is $\binom{n}{2} = n(n+1)/2$.

III.3.21 See Figure 72 on page 111.

III.3.22

(A) There are ten vertices, $\mathcal{N} = \{v_0, \ldots v_9\}$. There are fifteen edges.

$$\mathcal{E} = \{v_0v_1, v_0v_4, v_0v_5, v_1v_2, v_1v_6, v_2v_3, v_2v_7, v_3v_4, v_3v_8, v_4v_9, v_5v_7, v_5v_8, v_6v_8, v_6v_9, v_7v_9\}$$

(B) The walk $\langle v_0v_5, v_5v_7\rangle$ has length two. The walk $\langle v_0v_1, v_1v_2, v_2v_7\rangle$ has length three.

(C) A closed walk is $\langle v_4v_0, v_0v_1, v_1v_2, v_2v_3, v_3v_4\rangle$. An open one is $\langle v_4v_9, v_9v_6, v_6v_8, v_8v_3, v_3v_2\rangle$.

(D) One cycle is $\langle v_5v_0, v_0v_1, v_1v_6, v_6v_8, v_8v_5\rangle$.

(E) Yes, this graph is connected since we can find a path between any two vertices. (We see this by inspection, although we could list all pairs and give a path between them, if there is some doubt.)

III.3.23 With only two colors, a cycle must alternate colors. But if we go around the outside five nodes then the initial node and the fifth node would have the same color.

III.3.24

(A) See Figure 73 on page 111.

(B) The chromatic number is three; see the right-hand graph in the same figure. The chromatic number cannot be less than three because, for example, *ABC* is a triangle.

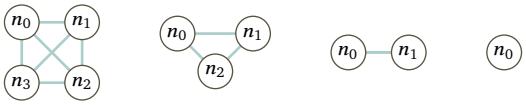


FIGURE 70, FOR QUESTION III.3.20: Complete graphs K_4, K_3, K_2, and K_1.

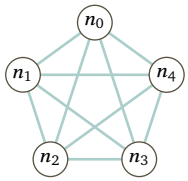


FIGURE 71, FOR QUESTION III.3.20: Complete graph K_5.

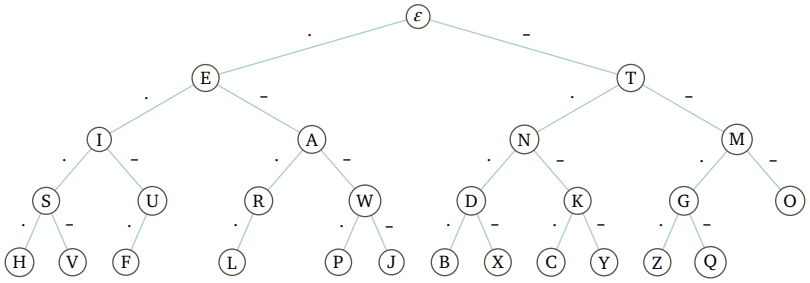


FIGURE 72, FOR QUESTION III.3.21: Prefix relation for Morse code representations of ASCII letters.

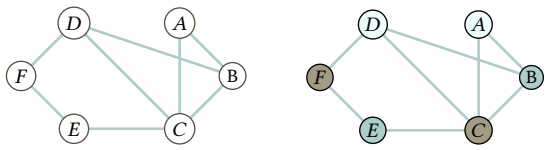


FIGURE 73, FOR QUESTION III.3.24: Relation of incompatibility between fish species.

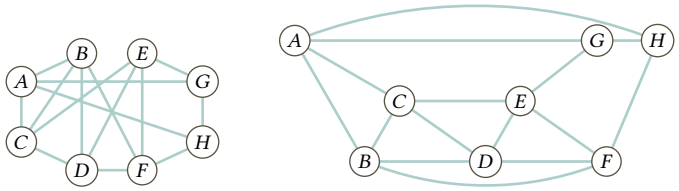


FIGURE 74, FOR QUESTION III.3.25: Drawings of the same graph, with edge crossings and without.

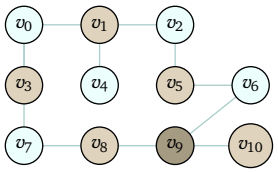


FIGURE 75, FOR QUESTION III.3.26: Cell tower three-coloring.

(C) It is the number of fish tanks needed to safely keep all of the fish. This person can keep A and D in one tank, B and E in another, and C and F in the third.

III.3.25

(A) On the left of Figure 74 on page 112 is the drawing with the vertices connected by the given edges.

(B) See the right side of Figure 74 on page 112 for the planar drawing.

III.3.26 The minimal number of frequencies is three. Figure 75 on page 112 gives a three-coloring. As to showing that no two-coloring suffices, note first that the graph has no triangles so that arguement is not possible. Instead, starting by coloring v_6 with color 0 and tracing around: v_5 is color 1, then v_2 is color 0, then v_1 is color 1, then v_0 is color 0, then v_3 is color 1, then v_7 is color 0, then v_8 is color 1, and then v_9 is color 0, which leaves v_6 and v_9 in the same color.

III.3.27 The degree sequence is $\langle 3, 3, 2, 2, 2, 2, 2, 2, 2, 1, 1\rangle$.

III.3.28 The directed graph is Figure 76 on page 113. Every type is compatible with itself so every vertex is shown with a loop. The type O− can donate to everyone, as we see from its row below, so it is a universal donor. The type AB+ is a universal receptor, as we see from its column.

Here is the graph's adjacency matrix.

	O−	O+	A−	A+	B−	B+	AB−	AB +
O−	1	1	1	1	1	1	1	1
O+	0	1	0	1	0	1	0	1
A−	0	0	1	1	0	0	1	1
A+	0	0	0	1	0	0	0	1
B−	0	0	0	0	1	1	1	1
B+	0	0	0	0	0	1	0	1
AB−	0	0	0	0	0	0	1	1
AB+	0	0	0	0	0	0	0	1

III.3.29 The graph in Example 3.2 has this degree sequence: $\langle 4, 4, 3, 3, 2\rangle$. The vertices v_1 and v_2 have degree 4, the vertices v_3 and v_4 have degree 3, and the vertex v_0 has degree 2.

In Example 3.4 the graph on the left has degree sequence $\langle 3, 3, 3, 3\rangle$. The graph on the right has $\langle 3, 3, 3, 3, 3, 3, 3, 3\rangle$ (there are eight 3's).

III.3.30 Both of these graphs are complete, meaning that every possible node-to-node connection is there

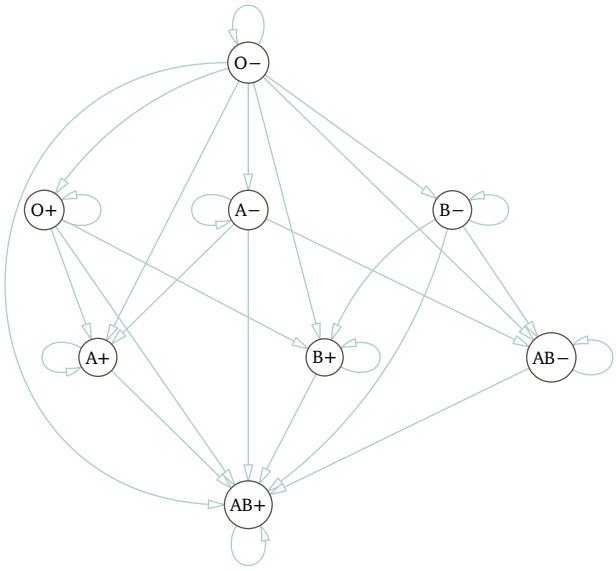


FIGURE 76, FOR QUESTION III.3.28: Compatibility for blood types.

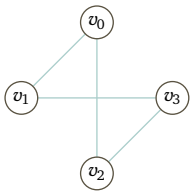


FIGURE 77, FOR QUESTION III.3.31: Graph associated with the given matrix.

except that there are no loops. Here are the adjacency matrices.

$$\begin{array}{c} \\ u_0 \\ u_1 \\ u_2 \\ u_3 \end{array} \begin{array}{c} \begin{array}{cccc} u_0 & u_1 & u_2 & u_3 \end{array} \\ \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \end{array} \qquad \begin{array}{c} \\ v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \\ v_7 \end{array} \begin{array}{c} \begin{array}{cccccccc} v_0 & v_1 & v_2 & v_3 & v_4 & v_5 & v_6 & v_7 \end{array} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{pmatrix} \end{array}$$

III.3.31 The simple graph is Figure 77 on page 113.

III.3.32 If the tree is empty then the coloring is trivial Otherwise fix a vertex. Give it one color, and give its neighbors the second color. Assign the first color to all the vertices that are two away from the initial vertex. In general, where a vertex is k away from the initial vertex, give it the first color if k is even and the second color if k is odd. Trees don't have loops, so this assignment is well-defined.

III.3.33 There is more than one correct answer but we will just give one.

(A) The function is suggested by the names of the vertices: $a \mapsto A$, $b \mapsto B$, etc. By inspection it is one-to-one and onto, so it is a correspondence.

(B) The edges on the left are: ax, ay, az, bx, by, bz, cx, cy, and cz. The edges on the right are: AX, AY, AZ, BX, BY, BZ, CX, CY, and CZ. Under the mapping the edge ax is associated with AX, the edge ay is

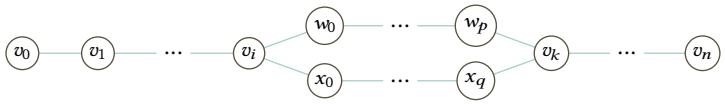


FIGURE 78, FOR QUESTION III.3.35: Two distinct paths makes a cycle.

associated with AY, etc. Again by inspection, this is a correspondence between the edges.

III.3.34 For both graphs the degree sequence is $\langle 3, 3, 3, 3, 3, 3\rangle$.

III.3.35

(A) A path is a trail with distinct vertices (except that possibly the starting vertex equals the ending vertex). This trail meets that criteria.

(B) The vertex B appears twice, not both at the start and end.

(C) Figure 78 on page 114 illustrates. By definition every tree is connected, so there is a path. Suppose for contradiction that there are two unequal paths from v_0 to v_n. Then there is some vertex v_i where the paths diverge and also a vertex v_k where they come together again. But that makes a cycle from v_i, around the one path to v_k, and back by the other path to return to v_i. That's a contradiction because trees do not have cycles.

III.3.36 There are three nodes of degree 2 and four nodes of degree 1. The degree sequence is $\langle 2, 2, 2, 1, 1, 1, 1\rangle$.

III.3.37

(A) A breadth first traversal is the sequence $\langle A, B, C, D, E, F, G\rangle$.

(B) A depth-first traversal is $\langle A, B, D, E, C, F, G\rangle$.

III.3.38

(A) For each edge we choose two vertices. Thus the number of potential edges is $\binom{n}{2} = n \cdot (n-1)/2$.

(B) For each edge, we either include it or leave it out. So with n vertices there are $2^{n\cdot(n-1)/2}$ graphs. (Some of these graphs may be isomorphic but they are different in that they involve different vertices being connected.)

(C)

n	0	1	2	3	4	5	6
$2^{n\cdot(n-1)/2}$	1	1	2	8	64	1024	32768

III.3.39

(A) By the definition of graph isomorphism, Definition 3.15, if two graphs $\mathcal{G}$ and $\hat{\mathcal{G}}$ are isomorphic then there is a correspondence between their vertices. They therefore have the same number of vertices.

(B) The definition of graph isomorphism states that the edges correspond. So isomorphic graphs have the same number of edges.

(C) Suppose that $\mathcal{G} = \langle \mathcal{N}, \mathcal{E}\rangle$ and $\hat{\mathcal{G}} = \langle \hat{\mathcal{N}}, \hat{\mathcal{E}}\rangle$ are isomorphic via the function f. Let vertex $v \in \mathcal{N}$ have degree k. Then in the set $\mathcal{E}$ the vertex v occurs k times (counting once for edges in which the vertex occurs once and twice for loops on v).

Now consider the vertex $f(v)$ and the set $\hat{\mathcal{E}}$. Because f gives a correspondence among edges, the vertex must occur k times in the set (here also counting once for edges in which the vertex occurs once and twice for loops on v). So $f(v)$ has degree k.

(D) This is the same as the prior item, but instead of a single vertex v we include all of the vertices $v_0, \ldots v_n$.

(E) There are a number of the prior items that we could apply. The simplest is that they have a different number of vertices. Another is that the degree sequences differ since on the left the degree sequence is $\langle 3, 3, 3, 3\rangle$ while on the right it is $\langle 4, 4, 4, 4, 4, 4, 4, 4\rangle$.

(F) Clearly they have the same degree sequence, $\langle 3, 1, 1, 1, 1, 1\rangle$. Suppose that the two are isomorphic via the correspondence f. Then f must associate the two degree-3 vertices. The graph on the left has two unequal length 2 paths that start with its degree 3 vertex. The graph on the right does not have two, it only has one.

III.3.40

(A) The vertices are $\mathcal{N}_0 = \{v_0, \ldots v_7\}$. These are the edges.

$$\mathcal{E}_0 = \{v_0v_1, v_0v_2, v_0v_4, v_1v_3, v_2v_3, v_2v_6, v_4v_5, v_4v_6, v_5v_7, v_6v_7\}$$

The vertices of $\mathcal{G}_1$ are $\mathcal{N}_1 = \{n_0, \ldots n_7\}$. These are the edges.

$$\mathcal{E}_1 = \{n_0n_4, n_0n_5, n_1n_4, n_1n_5, n_2n_3, n_2n_6, n_3n_4, n_3n_7, n_5n_6, n_6n_7\}$$

(B) The degree sequences are equal. Each is $\langle 3, 3, 3, 3, 2, 2, 2, 2\rangle$.

(C) These are the images of edges from the left-hand graph.

edge of $\mathcal{G}_0$	v_0v_1	v_0v_2	v_0v_4	v_1v_3	v_2v_3	v_2v_6	v_4v_5	v_4v_6	v_5v_7	v_6v_7
image edge	n_6n_2	n_6n_7	n_6n_5	n_2n_3	n_7n_3	n_7n_0	n_5n_1	n_5n_0	n_1n_4	n_0n_4

That list has no n_3n_4, although $\mathcal{G}_1$ has such an edge. Further, that list shows n_7n_0, although $\mathcal{G}_1$ has no such edge. So the given correspondence between vertices does not induce a correspondence between edges. This map is not a graph isomorphism.

(D) The graph $\mathcal{G}_0$ has a cycle of four degree 3 vertices. But $\mathcal{G}_1$ has no such cycle.

III.3.41

(A) There is a length 2 walk from v_i to v_j if and only if there is some v_k where both v_iv_k and v_kv_j are edges. Thus the number of such walks is $m_{i,1}m_{1,j} + m_{i,2}m_{2,j} + \cdots m_{i,n}m_{n,j}$, where each $m_{a,b}$ is an entry from $\mathcal{M}(\mathcal{G})$. That sum is the the i, j entry of $\big(\mathcal{M}(\mathcal{G})\big)^2$.

(B) The base step holds by the definition of $\mathcal{M}(\mathcal{G})$.

For the inductive step assume the hypothesis that the statement is true for $n = 2, \ldots, n = p$ and consider the statement for $n = p + 1$. There is a length $p + 1$ walk from v_i to v_j if and only if there is some v_k where there is a length p walk from v_i to v_k, and also v_kv_j is an edge. By the inductive hypothesis the number of such walks is $q_{i,1}m_{1,j} + q_{i,2}m_{2,j} + \cdots + q_{i,n}m_{n,j}$, where each $q_{i,b}$ is an entry from $\big(\mathcal{M}(\mathcal{G})\big)^p$ and $m_{b,j}$ is an entry from $\mathcal{M}(\mathcal{G})$. That summation is the i, j entry of $\big(\mathcal{M}(\mathcal{G})\big)^{p+1}$.

III.3.42

(A) We will keep track of a set R of reachable vertices. To start, put q_0 into a set R_0. For the first step find all vertices connected to q_0, and enter them into the set $R_1 \supseteq R_0$. At step $k + 1$ put all vertices connected to any vertex in R_k into a set $R_{k+1} \supseteq R_k$. When there are no new vertices then we are done; this must happen after finitely many steps because the graph is assumed to have a finite number of vertices. The result is the set R. The unreachable vertices are those that are not members of R.

(B) For the graph on the left these are the steps.

step k	R_k
0	$\{w_0\}$
1	$\{w_0, w_1\}$
2	$\{w_0, w_1, w_2\} = R$

Here are the steps for the graph on the right.

step k	R_k
0	$\{w_0\}$
1	$\{w_0, w_1\}$
2	$\{w_0, w_1, w_3\} = R$

III.3.43 Fix a vertex v_0. The graph is connected, so for every other vertex there is a path reaching it that starts at v_0.

For each of v_0's neighbors, there is a set of vertices that can be reached from v_0 via a path through that neighbor. There are infinitely many vertices so there must be a neighbor (unequal to v_0) where the set of vertices that are reachable in that way is infinite. Pick such a neighbor and call it v_1.

Now iterate: by choice of v_1 there are infinitely many vertices reachable by a path starting with the edge v_0v_1. Because v_1 has finitely many neighbors, there is a v_2 adjacent to v_1 (and unequal to either v_0 or v_1), through which there are paths to infinitely many of the graph's vertices. In this way we get a path containing infinitely many vertices

III.A.4

⟨*zip*⟩ ::= ⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩[-⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩]
⟨*digit*⟩ ::= `0` | ... | `9`

III.A.5 When we add a character at the front we also add it at the back.

⟨*palindrome*⟩ ::= `a` ⟨*palindrome*⟩ `a` | `b` ⟨*palindrome*⟩ `b` | ... `z` ⟨*palindrome*⟩ `z` | ⟨*letter*⟩ | ε
⟨*letter*⟩ ::= `a` | ... `z`

Note that this includes ε as a palindrome.

III.A.6 Here is one way to do it.

⟨*course code*⟩ ::= ⟨*dept*⟩ ⟨*space*⟩ ⟨*course-number*⟩
⟨*dept*⟩ ::= ⟨*two-letter*⟩ | ⟨*three-letter*⟩
⟨*two-letter*⟩ ::= ⟨*letter*⟩⟨*letter*⟩
⟨*three-letter*⟩ ::= ⟨*two-letter*⟩⟨*letter*⟩
⟨*course-number*⟩ ::= ⟨*digit*⟩⟨*digit*⟩⟨*digit*⟩
⟨*digit*⟩ ::= `0` | ... `9`
⟨*letter*⟩ ::= `A` | ... `Z`

The nonterminal ⟨*space*⟩ produces a hard-to-show blank space.

We could also put the department names in on a case-by-case basis, as here.

⟨*dept*⟩ ::= `MA` | `PSY` | ...

This has the advantage of being more true, of not allowing department names that are nonsensical. But it has the maintenance disadvantage that if a department code changes then the description must also change. (Enforcing truths in a grammar often comes at a price.)

III.A.7

(A) This will do.

⟨*pointfloat*⟩ ::= ⟨*intpart*⟩ ⟨*fraction*⟩ | ⟨*fraction*⟩ | ⟨*intpart*⟩ .

(B) Recursion replaces the + operator.

⟨*intpart*⟩ ::= ⟨*intpart*⟩ ⟨*digit*⟩ | ⟨*digit*⟩

For ⟨*exponent*⟩, list the cases.

⟨*exponent*⟩ ::= `e` ⟨*intpart*⟩ | `e+` ⟨*intpart*⟩ | `e-` ⟨*intpart*⟩ | `E` ⟨*intpart*⟩ | `E+` ⟨*intpart*⟩ | `E-` ⟨*intpart*⟩

III.A.8

(A) This is a grammar for standard notation.

⟨*roman-numeral*⟩ ::= ⟨*thousands*⟩ ⟨*hundreds*⟩ ⟨*tens*⟩ ⟨*ones*⟩
⟨*thousands*⟩ ::= `M*`
⟨*hundreds*⟩ ::= `C` | `CC` | `CCC` | `CCCC` | `D` | `DC` | `DCC` | `DCCC` | `DCCCC` | ε
⟨*tens*⟩ ::= `X` | `XX` | `XXX` | `XXXX` | `L` | `LX` | `LXX` | `LXXX` | `LXXXX` | ε
⟨*ones*⟩ ::= `I` | `II` | `III` | `IIII` | `V` | `VI` | `VII` | `VIII` | `VIIII` | ε

(B) This is a grammar for Roman numerals in subtractive notation.

⟨*roman-numeral*⟩ ::= ⟨*thousands*⟩ ⟨*hundreds*⟩ ⟨*tens*⟩ ⟨*ones*⟩
⟨*thousands*⟩ ::= `M*`
⟨*hundreds*⟩ ::= `CM` | `CD` | (`D` | ε) (`C` | ε | `CC` | `CCC`)
⟨*tens*⟩ ::= `XC` | `XL` | (`L` | ε) (`X` | ε | `XX` | `XXX`)
⟨*ones*⟩ ::= `IX` | `IV` | (`V` | ε) (`I` | ε | `II` | `III`)

III.A.9

(A) Here is a derivation. As an abbreviation we have, for instance, put in multiple ⟨*statement*⟩'s on the third line instead of putting them in one at a time.

⟨*program*⟩ ⇒ { ⟨*statement-list*⟩ }
⇒ { ⟨*statement*⟩ ; ⟨*statement*⟩ ; ⟨*statement*⟩ ; }
⇒ { ⟨*data-type*⟩ ⟨*identifier*⟩ ; ⟨*statement*⟩ ; ⟨*statement*⟩ ; }
⇒ { `int` ⟨*identifier*⟩ ; ⟨*statement*⟩ ; ⟨*statement*⟩ ; }
⇒ { `int` ⟨*letter*⟩ ; ⟨*statement*⟩ ; ⟨*statement*⟩ ; }
⇒ { `int` `A` ; ⟨*statement*⟩ ; ⟨*statement*⟩ ; }

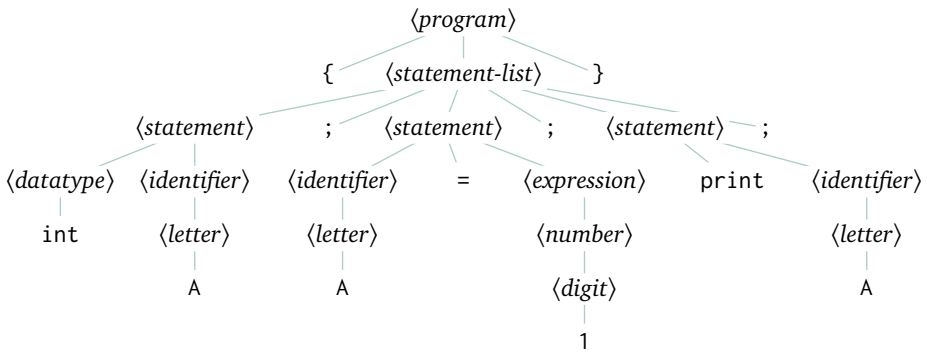


FIGURE 79, FOR QUESTION III.A.9: Parse tree for C-like language.

⇒ { int A ; ⟨*identifier*⟩ = ⟨*expression*⟩ ; ⟨*statement*⟩ ; }
⇒ { int A ; ⟨*letter*⟩ = ⟨*expression*⟩ ; ⟨*statement*⟩ ; }
⇒ { int A ; A = ⟨*expression*⟩ ; ⟨*statement*⟩ ; }
⇒ { int A ; A = ⟨*number*⟩ ; ⟨*statement*⟩ ; }
⇒ { int A ; A = ⟨*digit*⟩ ; ⟨*statement*⟩ ; }
⇒ { int A ; A = 1 ; ⟨*statement*⟩ ; }
⇒ { int A ; A = 1 ; print ⟨*identifier*⟩ ; }
⇒ { int A ; A = 1 ; print ⟨*letter*⟩ ; }
⇒ { int A ; A = 1 ; print A ; }

Figure 79 on page 117 shows the tree.

(B) Yes, because all derivations must go from ⟨*program*⟩ through ⟨*statement-list*⟩, which forces them to have curly braces.

III.A.10 In this partial derivation we abbreviate by, for instance, putting in multiple ⟨*s-expression*⟩'s for a ⟨*list*⟩ at one time, instead of one at a time.

⟨*s-expression*⟩ ⇒ ⟨*list*⟩
⇒ (⟨*s-expression*⟩ ⟨*s-expression*⟩ ⟨*s-expression*⟩)
⇒ (⟨*s-expression*⟩ ⟨*list*⟩ ⟨*s-expression*⟩)
⇒ (⟨*s-expression*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (⟨*atomic-symbol*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (⟨*letter*⟩ ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (⟨*letter*⟩ ⟨*letter*⟩ ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (⟨*letter*⟩ ⟨*letter*⟩ ⟨*letter*⟩ ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (⟨*letter*⟩ ⟨*letter*⟩ ⟨*letter*⟩ ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (c ⟨*letter*⟩ ⟨*letter*⟩ ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (c o ⟨*letter*⟩ ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (c o n ⟨*letter*⟩ ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (c o n s ⟨*atom-part*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (c o n s ⟨*empty*⟩ (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)
⇒ (c o n s " " (⟨*s-expression*⟩ ⟨*s-expression*⟩) ⟨*s-expression*⟩)

Expanding the remaining nonterminals to letters is tedious, so we will stop here.

III.A.11

(A) Here we will leverage the BNF notation to, for instance, substitute for multiple nonterminals, instead of putting them in one at a time.

⟨*format-spec*⟩ ⇒ 0 ⟨*width*⟩ ⟨*type*⟩
⇒ 0 ⟨*integer*⟩ ⟨*type*⟩
⇒ 0 ⟨*digit*⟩ ⟨*type*⟩

$\Rightarrow$ `0` `3` ⟨*type*⟩
$\Rightarrow$ `0` `3` `f`

(B) ⟨*format-spec*⟩ $\Rightarrow$ ⟨*sign*⟩ `#` `0` ⟨*width*⟩ ⟨*type*⟩
$\Rightarrow$ `+` `#` `0` ⟨*width*⟩ ⟨*type*⟩
$\Rightarrow$ `+` `#` `0` ⟨*integer*⟩ ⟨*type*⟩
$\Rightarrow$ `+` `#` `0` ⟨*digit*⟩ ⟨*type*⟩
$\Rightarrow$ `+` `#` `0` `2` ⟨*type*⟩
$\Rightarrow$ `+` `#` `0` `2` `X`

III.B.1

(A) Here is the code.

```
          [ne (node-add-child! nb "e")]
          [nf (node-add-child! nc "f")]
          [ng (node-add-child! nc "g")]
          [nh (node-add-child! ng "h")]
          [ni (node-add-child! ng "i")]
          )
    t))

;; void --> DAG of nodes
;; Make the Cantor graph.
(define (cantor-DAG-make)
  (let* ([t (graph-create "0,0")]
```

(B) From the interpreter after running the definitions we get this.

```
> (define t (binary-tree-make))
> (traverse-dfs t 0 show-node-name)
a
  c
    g
      i
      h
    f
  b
    e
    d
```

(C) We get this.

```
> (define t (binary-tree-make))
> (traverse-bfs t show-node-name)
a
  c
  b
    e
    d
    g
    f
      i
      h
```

Chapter IV: Automata

IV.1.18

(A)

Step	*Machine*
0	a b b a q_0
1	b b a q_1
2	b a q_2
3	a q_2
4	q_1

Because q_1 is an accepting state, the machine accepts this string.

(B)

Step	*Machine*
0	b a b q_0
1	a b q_0
2	b q_1
3	q_2

The machine does not accept this string because q_2 is not an accepting state.

(C)

Step	*Machine*
0	b b a a b b a a q_0
1	b a a b b a a q_0
2	a a b b a a q_0
3	a b b a a q_1
4	b b a a q_1
5	b a a q_2
6	a a q_2
7	a q_1
8	q_1

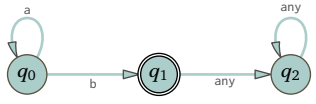
The machine accepts this string.

IV.1.19 False. For instance, Example 1.8's machine recognizes the language consisting of valid decimal representations of integers and thus containing each of these: `0`, `1`, `2`, ...

IV.1.20 Languages do not recognize strings, machines recognize strings. Maybe they mean that they have a language that contains the empty string? Or maybe they mean that what they have is the language of a machine, and this machine accepts the empty string?

IV.1.21 First, "n is infinite" is wrong. That set is a language $\{\mathsf{b}, \mathsf{ab}, \mathsf{a}^2\mathsf{b}, \ldots\}$ and in each string from that language the number of a's is finite.

Second, there certainly is a machine that recognizes that language. This Finite State machine will do.



IV.1.22 Each input character consumed causes one transition. So an input string of length n causes the machine to undergo n transitions. (This includes that the empty string ε causes no transitions.) With n transitions, the machine will have visited $n + 1$ many not necessarily distinct states, including the start state.

IV.1.23

(A) A string is in this language when it consists of two strings of a's of equal length, separated by a single b. Five elements of the language are aba, aabaa, b, aaabaaa, and aaaabaaaa. Five non-elements are ba, aaba, a, abba, and bbabb.

(B) The strings in this language have some number of a's followed by a single b, followed by a number of a's. The two numbers of a's need not be the same, and each may be zero. Five language elements are abaa, aabaa, ab, baaaa, and b. Five non-elements are bab, aabb, bb, aabaab, and a.

(C) The criteria for a string to be in this language is that it must start with b and be followed by some number of a's. That could be zero-many a's. Five elements of the language are baa, ba, b, ba^3, and ba^4. Five non-elements are a, ε, abaaa, baaab, and bb.

(D) A string is in this language if and only if it consists of some number of a's followed by a b, followed by some number of a's, where the second string of a's has two more than the first string. Five members of the language are abaaa, aabaaaa, a^3ba^5, a^4ba^6, and baa. Five non-members are aba, abaaaa, ba, b, and bab.

(E) This language has members that are doubles in that a member consists of the concatenation of two copies of some string. Five members of the language are ε, babbab, aa, bbbbabbbba, and abbaabba. Five non-members are aba, b, aaabaab, babaa, and bbb.

IV.1.24

(A) For Example 1.6 the only accepting state is q_2. In Example 1.8 also, the only accepting state is q_2. For Example 1.9 the accepting states are q_3, q_6, and q_8. For Example 1.10 it is q_0.

(B) Only Example 1.10 accepts the empty string ε because only in this machine is q_0 an accepting state.

(C) For Example 1.6 the shortest accepted string is the length two string `00`. For Example 1.8 it is any single-digit length one string, such as 6. For Example 1.9 any of the length three strings jpg, png, and pdf. For Example 1.10 it is the empty string ε.

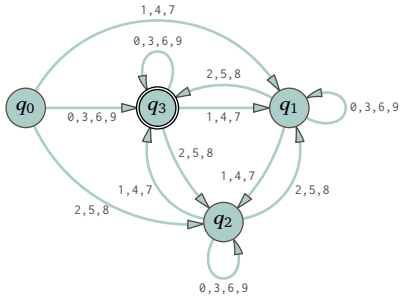
(D) The language of Example 1.6 is the set of strings containing at least two `0`'s and an even number of 1's. That language is infinite because it contains, among other strings, ε, `00`, `0000`, $\texttt{0}^6$, etc. The language of Example 1.8 is infinite because it contains, among other strings, these: `0`, 1, 2, ... The language of Example 1.9 contains exactly three elements, $\mathcal{L} = \{\,$ jpg, png, *and* pdf $\}$. For Example 1.10, the language is infinite because it contains `0`, 3, 6, ...

IV.1.25 This is the computation.

$$\langle q_0, 2332\rangle \vdash \langle q_2, 332\rangle \vdash \langle q_2, 32\rangle \vdash \langle q_2, 2\rangle \vdash \langle q_1, \varepsilon\rangle$$

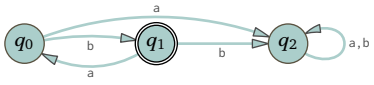
Because q_1 is not an accepting state, the machine rejects 2332.

IV.1.26 We must change the machine so that its initial state is not an accepting state.

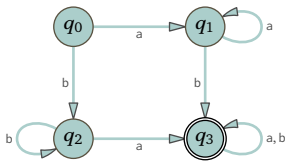


Recall the section's design advice that each state should have an intuitive function that is directed to the machine's purpose, and that is unique. In that machine there are two states, q_0 and q_3, whose intuitive meaning is that the sum of the digits seen so far is congruent to 0 modulo 3. However, they are different because the initial state q_0 is not accepting.

IV.1.27 This is the graph picture.

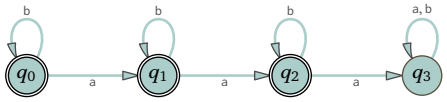


The language is $\{\,\sigma \in \{\mathtt{a},\mathtt{b}\}^* \;\big|\; \sigma = \mathtt{b}(\mathtt{ab})^n \text{ for } n \in \mathbb{N}\}$.



Δ	a	b
q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_3	q_2
$+q_3$	q_3	q_3

FIGURE 80, FOR QUESTION IV.1.29: Finite State machine accepting strings with at least one a and b.



Δ	a	b
$+q_0$	q_1	q_0
$+q_1$	q_2	q_1
$+q_2$	q_3	q_2
q_3	q_3	q_3

FIGURE 81, FOR QUESTION IV.1.29: Finite State machine accepting strings with fewer than three a's.

IV.1.28 The language is the set of strings over $\Sigma = \{\mathsf{a}, \mathsf{b}\}$ that contain exactly one b. Restated, $\mathcal{L}(\mathcal{M}) = \{\mathsf{a}^n\mathsf{b}\mathsf{a}^m \mid n, m \in \mathbb{N}\}$.

IV.1.29 In the section body is the advice to design machines by describing an intuitive meaning for each state. Although the question does not ask for it, for each item here we give those.

(A) Five members are ab, baab, ba, $\mathsf{b}^7\mathsf{a}^3$, and $\mathsf{a}\mathsf{b}^6\mathsf{a}$. Five nonmembers are aaaa, bbbb, a, ε, and b. We take the meaning of the states to be as here.

State	*Description*
q_0	start state
q_1	have seen at least one a, no b's
q_2	have seen at least one b, no a's
q_3	at least one of each

The circle drawing and transition table are Figure 80 on page 121.

(B) Five members are: bb, ε, aba, a, and bbbaab. Five nonmembers are aaa, baaa, a^5, bababa, and a^{10}. We take the states with this meaning.

State	*Description*
q_0	start state, no a's seen yet
q_1	have seen one a, and maybe some number of b's
q_2	have seen two a's
q_3	have seen three or more a's

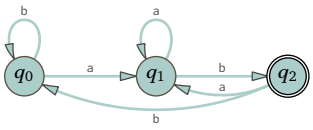
For the circle diagram and transition table, see Figure 81 on page 121.

(C) Five members are ab, bab, aab, baab, and abab. Five nonmembers: a, bbb, ε, b, and abababb. We use the states with this meaning.

State	*Description*
q_0	start state, have not just seen an a
q_1	have seen an a, waiting for a b
q_2	have just seen ab

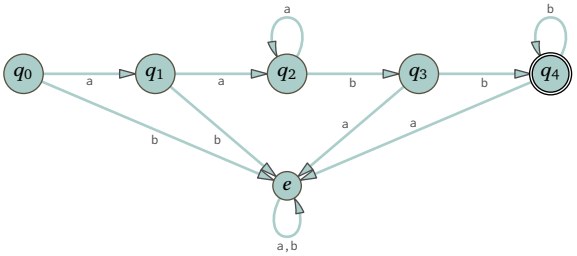
The transition diagram and table are Figure 82 on page 122.

(D) Five members are aabb, aaabb, aabbb, $\mathsf{a}^7\mathsf{b}^9$, and $\mathsf{a}^2\mathsf{b}^{200}$. Five nonmembers are ε, a, ab, ba, and b. We use the states with this meaning.



Δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
$+q_2$	q_1	q_0

FIGURE 82, FOR QUESTION IV.1.29: Finite State machine accepting strings that end in ab.



Δ	a	b
q_0	q_1	e
q_1	q_2	e
q_2	q_2	q_3
q_3	e	q_4
$+q_4$	e	q_4
e	e	e

FIGURE 83, FOR QUESTION IV.1.29: Finite State machine for strings of at least two a's and then at least two b's.

State	*Description*
q_0	start state, have not seen any input
q_1	have seen one character, an a
q_2	have seen at least two a's and no b's
q_3	have seen at least two a's and one b
q_4	have seen at least two a's and at least two b's
$q_5 = e$	error, cannot recover

See Figure 83 on page 122 for the circle diagram and the table.

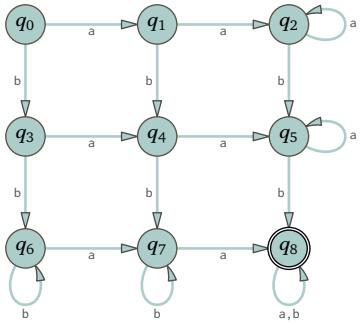
(E) Five members are aabba, aabb, abb, bb, and a^5bb^7. Five nonmembers are ε, bab, aabaa, aabbbaa, and aabbaab. We take the states with this meaning.

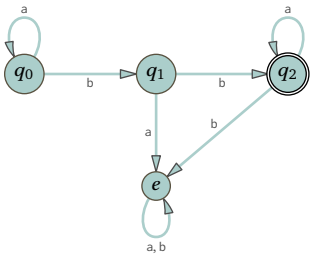
State	*Description*
q_0	have so far seen some number of a's (including zero-many)
q_1	have seen some a's followed by one b
q_2	seen some a's, two b's, and some a's (including zero-many)
e	error

The two representations of the transition function are Figure 84 on page 123.

IV.1.30 The introduction to these exercises suggests thinking through the language description by naming five strings that are in the language and five that are not. The question doesn't ask for that, but as lagniappe here are some: five elements of the language are aabb, bbaa, abab, aababb, and a^3b^4. Five non-elements are ε, a, b, aab, and bbbbabbbb.

Here is the machine.





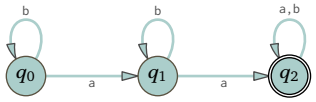
Δ	a	b
q_0	q_0	q_1
q_1	e	q_2
$+q_2$	q_2	e
e	e	e

FIGURE 84, FOR QUESTION IV.1.29: Finite State machine for strings with some a's, then two b's, then some a's.

The intuitive meaning of the states in the first row, q_0, q_1, and q_2, is that when the machine is in those states it has so far seen zero-many b's. States in the second row are where the machine has so far seen one b. The third row contains the states where the machine has seen two b's (or more). Similarly, the first column contains the states where the machine has so far seen no a's, in the second column are states where the machine has seen a single a, and in the final column the machine has see two or more a's.

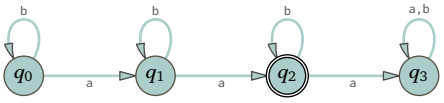
IV.1.31 At the start of these exercises is the advice to think through the language description by listing five members and five nonmembers of that language. Although the question doesn't ask for it, we give that for each. Also, in the section body is the advice to design machines by describing an intuitive meaning for each state. Although that is not a requirement and the question does not ask for it, for each item here we give those.

(A) Five strings that are elements of the language are aa, aaa, aab, aba, and baa. Five that are not are ε, a, b, abbbbbb, and ba. This machine accepts only strings in that language.



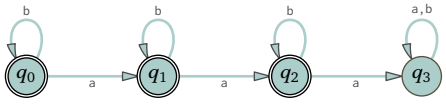
Δ	a	b
q_0	q_1	q_0
q_1	q_2	q_1
$+q_2$	q_2	q_2

(B) Five elements of the language are aa, aab, aba, baa, and aabb. Five that are not in the language are ε, a, ba, aaa, and baaa.



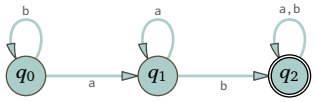
Δ	a	b
q_0	q_1	q_0
q_1	q_2	q_1
$+q_2$	q_3	q_2
q_3	q_3	q_3

(C) Five strings that are elements of the language are ε, a, b, aa, and ab. Five that are not are aaa, aaab, aaba, abaa, and baaa.



Δ	a	b
$+q_0$	q_1	q_0
$+q_1$	q_2	q_1
$+q_2$	q_3	q_2
q_3	q_3	q_3

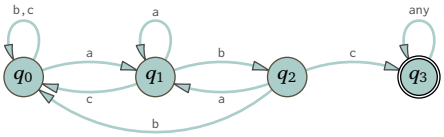
(D) Five from the language are ab, aab, aba, abb, and bab. Five that are not in the language are ε, a, b, ba, and bbbbaaaa.



Δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
$+q_2$	q_2	q_2

IV.1.32 To accept strings containing the substring abc the machine must have a state whose intuitive meaning is "have just seen a, next looking for bc." That state is q_1. The state q_2 means that the machine has just

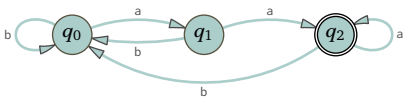
processed a substring ab and is looking for c. And q_3 means the machine has seen the substring abc. Finally, q_0 is the state the machine is in if it has so far not seen even the first character in a potential substring abc. Here is the machine.



Note that if the machine processes ab followed by b then it returns to q_0, but if it processes ab followed by a then it does not return to q_0, it passes to q_1. This follows from the intuitive meaning of q_1.

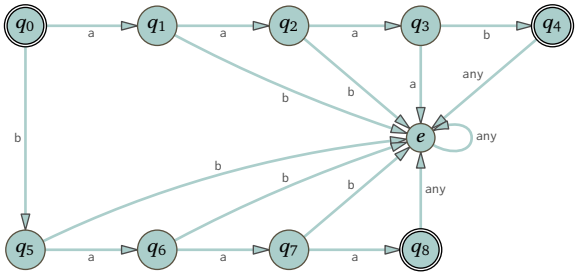
IV.1.33

(A) Five that are in the language are aa, aaa, baa, aaaa, and abaa. Five that are not are ε, a, b, ab ba, and aba. Here is the machine.

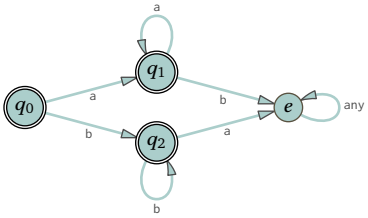


(B) The only string in the language is ε. That is, $\mathcal{L} = \{\varepsilon\}$. Five not in the language are a, b, aa, ab, and ba. The machine is this.

(C) There are only two strings in the language: $\mathcal{L} = \{\text{aaab}, \text{baaa}\}$. Five strings not in the language are ε, a, b, aa, and ab. The machine is this.

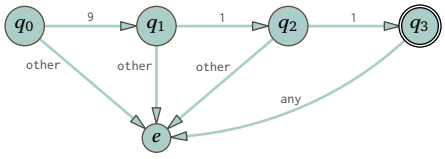


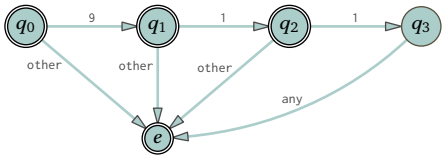
(D) Five strings in the language are ε, a, b, aa, and bb. Five not in the language are ab, ba, aab, aba, and baa. This is the machine.



Note that q_0 is an accepting state, because ε is a member of the language.

IV.1.34 The first machine accepts only 911 while the second accepts anything but 911. Note that in the two machines the sets of accepting states are complementary.





IV.1.35 Here are the length three strings.

Input σ	aaa	aab	aba	abb	baa	bab	bba	bbb
$\hat{\Delta}(\sigma)$	q_1	q_2	q_1	q_2	q_1	q_2	q_1	q_0

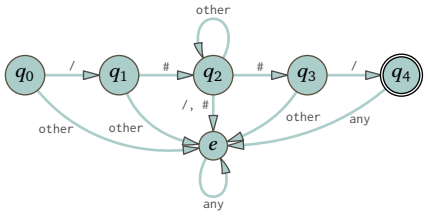
And here are the length four strings.

Input σ	aaaa	aaab	aaba	aabb	abaa	abab	abba	abbb
$\hat{\Delta}(\sigma)$	q_1	q_2	q_1	q_2	q_1	q_2	q_1	q_2

baaa	baab	baba	babb	bbaa	bbab	bbba	bbbb
q_1	q_2	q_1	q_2	q_1	q_2	q_1	q_0

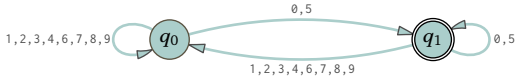
IV.1.36 It outputs the start state, $\hat{\Delta}(\varepsilon) = q_0$.

IV.1.37

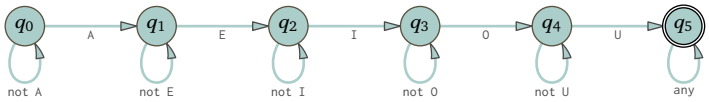


IV.1.38

(A) 

(B) 

IV.1.39 Note that although the string $\sigma =$ `AEAIAOAUA` has an `A` that occurs after an `E`, nonetheless the machine should accept it because there exists a subsequence of string elements consisting of the ordered vowels.

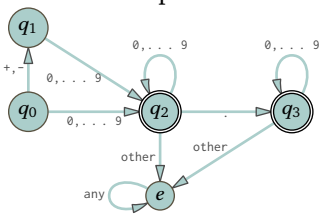


IV.1.40

(A) Five that are: `+1`, `123`, `-123.45`, `-0.0`, and `0`. Five that are not: `+`, `+-`, `..`, `12.3.4`, and `2+3`

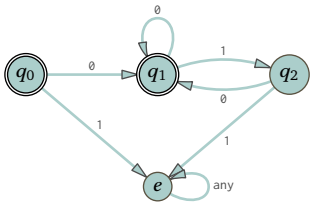
(B) No, the ⟨*posreal*⟩ rule gives that before the period must come at least one digit.

(C) It is the set of representations of real numbers with integer part, a decimal point, and a decimal part.

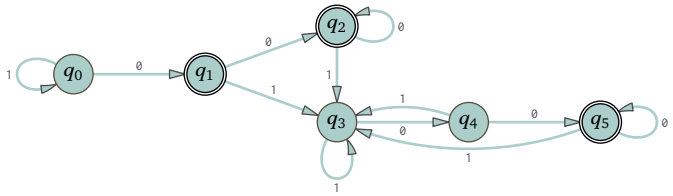
(D) 

IV.1.41 Although the exercise doesn't ask for it, for each language we list five members and five nonmembers.

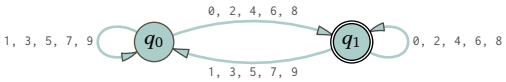
(A) Five that are in the language are `010`, `0010`, `0`, `0101000`, and ε. Five that are not are `10`, `0110`, `0101`, `01001`, and `1`.



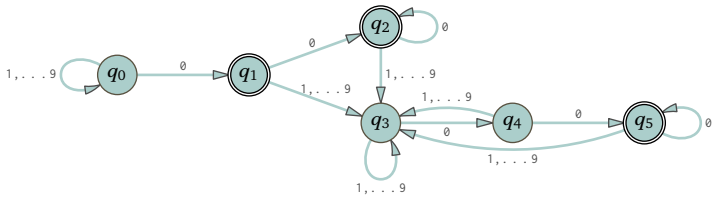
(B) A string is a binary representation of a multiple of 4 if and only if it ends in two zero's, `00`, or it is `0`. Five in the language are `100`, `0100`, `1000`, `10100`, and `0`. Five not in the language are `10`, `1`, `1001`, `001`, and ε.



(C) Five strings in the language are `0`, `20`, `12`, `88`, and `08`. Five that are not are `1`, `01`, `2221`, `17`, and ε.



(D) This is basically the same as the second item. A string is a decimal representation of a multiple of 100 if and only if it ends in two zero's, `00`, or it is `0`. Five in the language are `100`, `0100`, `2000`, `60300`, and `0`. Five not in the language are `10`, `8`, `1009`, `001`, and ε.



IV.1.42 A number is divisible by four if and only if its representation ends in `00`, `04`, . . . `96`, or the single-digit cases where the representation is `0`, `4`, or `8`. That's 28 cases. A picture would be cluttered but this is a finite language and for any finite language there is a Finite State machines that recognizes that language. (We shall see an alternative approach that makes a clearer picture in the section on nondeterminism.)

IV.1.43 Recall the transition function for that example.

Δ	`0`	`1`
q_0	q_1	q_3
q_1	q_2	q_4
+ q_2	q_2	q_5
q_3	q_4	q_0
q_4	q_5	q_1
q_5	q_5	q_2

(A) If the input string is $\sigma =$ `0` then peel off the final character to get $\sigma = \tau\frown$`0` $= \varepsilon\frown$`0` and the definition gives this.

$$\hat{\Delta}(\texttt{0}) = \Delta(\hat{\Delta}(\varepsilon), \texttt{0}) = \Delta(q_0, \texttt{0}) = q_1$$

(In the leftmost expression `0` is a length-one string, while in the rest of the equation `0` is a character. We ignore this uninteresting difference.) Similarly, $\hat{\Delta}(\texttt{1}) = q_3$.

(B) For $\sigma =$ `00`, peel off the final character to get $\sigma = \tau\frown$`0` $=$ `0`$\frown$`0` (here the first `0` is a length one string while the second is a character). Then use the prior item's computation of $\hat{\Delta}(\texttt{0})$.

$$\hat{\Delta}(\texttt{00}) = \Delta(\hat{\Delta}(\texttt{0}), \texttt{0}) = \Delta(q_1, \texttt{0}) = q_2$$

The other three are similar.

$$\hat{\Delta}(\texttt{01}) = q_4 \quad \hat{\Delta}(\texttt{10}) = q_4 \quad \hat{\Delta}(\texttt{11}) = q_0$$

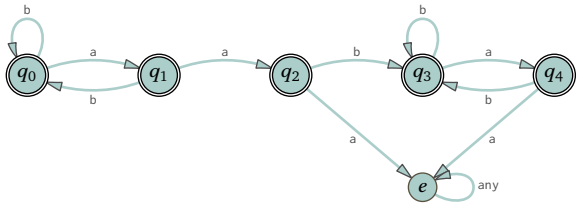
(C) For each of these use the prior item's computation. Here is the first.

$$\hat{\Delta}(\texttt{000}) = \Delta(\hat{\Delta}(\texttt{00}), \texttt{0}) = \Delta(q_2, \texttt{0}) = q_2$$

The others are computed in the same way.

$\hat{\Delta}(000)$	$\hat{\Delta}(001)$	$\hat{\Delta}(010)$	$\hat{\Delta}(011)$	$\hat{\Delta}(100)$	$\hat{\Delta}(101)$	$\hat{\Delta}(110)$	$\hat{\Delta}(111)$
q_2	q_5	q_5	q_1	q_5	q_1	q_1	q_3

IV.1.44



IV.1.45

(A) There is one such machine.

Δ	0	1
q_0	q_0	q_0

(B) This list has 16 machines.

Δ_0	0	1
q_0	q_0	q_0
q_1	q_0	q_0

Δ_1	0	1
q_0	q_0	q_0
q_1	q_0	q_1

Δ_2	0	1
q_0	q_0	q_0
q_1	q_1	q_0

$\ldots$

Δ_{15}	0	1
q_0	q_1	q_1
q_1	q_1	q_1

(C) Fix n states, $Q = \{q_0, \ldots q_{n-1}\}$. Each line of the transition table has two columns so the number of distinct lines is $n \cdot n = n^2$. An overall table chooses n such lines. There are $n^2 \cdot n^2 \cdots n^2 = n^{2n}$ many ways to do that.

(D) For each of the n states, either it is final or it is not. The additional factor of 2^n makes the total number of distinct tables equal to $2^n n^{2n}$.

IV.1.46 The states that the situation can occupy involve having the man, the wolf, the goat, and the cabbage, on the near side or far side. The allowed transitions are: man crosses with wolf, man crosses with goat, man crosses with cabbage, and man crosses alone. If the man leaves the wolf alone with the goat then that is an error state (on either near or far side), as is leaving the goat alone with the cabbage. (Having the man, the wolf, and the goat together is not an error because the man is with them.)

Locations		
Near	*Far*	*Notes*
Man, Wolf, Goat, Cabbage	–	Start state
Man, Wolf, Goat,	Cabbage	
Man, Wolf, Cabbage	Goat	
Man, Goat, Cabbage	Wolf	
Man, Goat	Wolf, Cabbage	
Man, Wolf	Goat, Cabbage	Error
Man, Cabbage	Wolf, Goat	Error
Man	Wolf, Goat, Cabbage	Error
Wolf, Goat, Cabbage	Man	Error
Wolf, Goat	Man, Cabbage	Error
Wolf, Cabbage	Man, Goat	
Wolf	Man, Goat, Cabbage	
Goat, Cabbage	Man, Wolf	Error
Goat	Man, Wolf, Cabbage	
Cabbage	Man, Wolf, Goat	
–	Man, Wolf, Goat, Cabbage	Final state

The Finite State machine shown in Figure 85 on page 128 covers all of these possibilities. For example, the line labeled 'n: mwg,f: c' means that the man is on the near side with the wolf and goat, while the cabbage is alone on the far side. The errors are gathered into a single state. So for instance the table in the figure has no line labeled 'n: wg,f: mc'. The start state is q_0 and the only accepting state is q_9.

In addition to the two errors in the first paragraph, it is an error to transition with the wolf if the man does not start with the wolf, and likewise for the goat and cabbage.

The circle and arrow diagram does not show the error arrows, as it would clog up the picture. With that diagram, finding a solution is easy. One path through the graph is q_0-q_5-q_2-q_8-q_3-q_7-q_4-q_9. In words that path

state	*near and far*	with wolf	with goat	with cabbage	with nothing
q_0	n: mwgc, f: –	e	q_5	e	e
q_1	n: mwg, f: c	q_7	q_6	e	e
q_2	n: mwc, f: g	q_8	e	q_6	q_5
q_3	n: mgc, f: w	e	q_8	q_7	e
q_4	n: mg, f: wc	e	q_9	e	q_7
q_5	n: wc, f: mg	e	q_0	e	q_2
q_6	n: w, f: mgc	e	q_1	q_2	e
q_7	n: g, f: mwc	q_1	e	q_3	q_4
q_8	n: c, f: mwg	q_2	q_3	e	e
+ q_9	n: –, f: mwgc	q_9	q_9	q_9	q_9
e	error	e	e	e	e

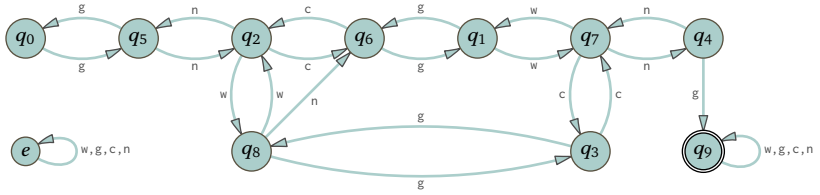


FIGURE 85, FOR QUESTION IV.1.46: Finite state machine for the Wolf-Goat-Cabbage problem.

is: the man first brings the goat to the far side and leaves it there. He goes back to the near side alone. He then brings the wolf with him from the near to the far side. Next he brings back the goat to the near side. He leaves the goat on the near side and takes the cabbage and brings it to the far side. Finally, he goes back alone to the near side, and finishes by taking the goat to the far side.

IV.1.47 For the alphabet with at least two members we write Σ. We fix one character from that alphabet, s.

(A) We will show that over that alphabet there are infinitely many different Finite State machines. The machines below differ both because they have a different number of states, and because they recognize different languages.

Δ	s	*other*
+ q_0	q_0	q_0

Δ	s	*other*
+ q_0	q_1	q_0
q_1	q_0	q_0

Δ	s	*other*
+ q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_0	q_0

. . .

(B) By definition the set of possible states is $Q = \{q_0, q_1, \ldots\}$. Then we can count the Finite State machines because for any collection of finitely many states $\{q_0, \ldots q_k\}$ there are finitely many transition tables. Thus the number of Finite State machines using those states is countable. Then the union of countably many countable sets is countable, and so the number of machines over all collections of finitely many states is countable.

(C) Because Σ has at least two members, the usual diagonalization construction shows that the power set of Σ is not countable.

IV.2.23 The main point is that entries in the table are sets of states. On the left is the machine from Example 2.7 while Example 2.8's machine is on the right.

Δ	a	b
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_3\}$	$\{\}$
q_2	$\{\}$	$\{q_3\}$
+ q_3	$\{q_3\}$	$\{q_3\}$

Δ	a	b	c
+ q_0	$\{q_1\}$	$\{\}$	$\{\}$
q_1	$\{\}$	$\{\}$	$\{q_0\}$

IV.2.24

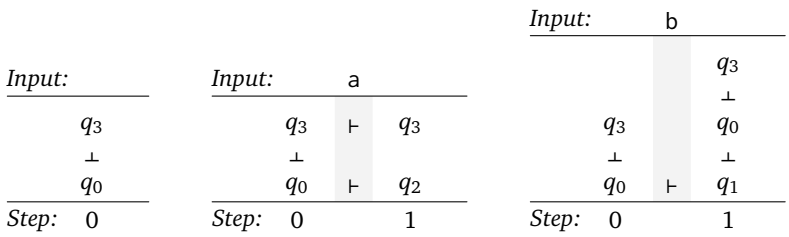


FIGURE 86, FOR QUESTION IV.2.27: Computation trees for the strings ε, a, and b.

(A) Yes. From q_0 an ε transition goes to q_2, which is an accepting state. That is, an accepting maximal path is $\langle q_0, \varepsilon\rangle \vdash \langle q_2, \varepsilon\rangle$, where the transition there is an ε transition.

(B) No. From q_0 the machine can either read the `0` and go to q_1, or take the ε transition to q_2 and then read the `0`, which sends the machine to no-state. Neither q_2 nor no-state is an accepting state.

Said more formally, the ε closure of q_0 is $\{q_0, q_2\}$. Neither state, on reading the character `0`, transitions to a state whose ε closure includes an accepting state.

(C) Yes. With that input the machine can go from q_0 to q_1, then to q_2, and then around the loop to q_2 again. More formally, this is a sequence of allowed transitions that ends in an accepting state.

$$\langle q_0, \texttt{011}\rangle \vdash \langle q_1, \texttt{11}\rangle \vdash \langle q_2, \texttt{1}\rangle \vdash \langle q_2, \varepsilon\rangle$$

(D) No. Starting at q_0 with the character `0` first on the tape, the machine can do two things. If it takes the ε transition then when it reads the `0` it goes to no-state, which is not an accepting state.

Otherwise, the `0` moves the machine to q_1. Next, on the following tape character `1`, the machine moves to q_2. Finally, on the third character `0`, the machine moves to no-state.

(E) There are two: `01111` and `11111`.

IV.2.25 Someone in this class saying that a topic is "just mathematical abstraction" is like someone objecting in an aerodynamics class that this is all "just air." Yes, that's what we are doing.

Said another way, if someone compares the thinking about computation that we are doing here with the thinking about computation that happens in a course on data structures, and holds the latter up as a model of "real" then that is a stretch. "Real" is changing diapers, or grafting fruit trees, or feeding the hungry. All of what we are doing is abstraction of one kind of another.

If that person responds, "OK but abstraction can go too far and get away from what brought us into the program" then that is more measured. One reason that nondeterminism is here, among many, is because it is necessary to understand the material of the fifth chapter, which describes the most important problem in the field today, which will directly impact all practitioners, the *P* versus *NP* question.

Besides, it is fun. Surely that merits at least some consideration.

IV.2.26 Going around forever in a cycle between q_0 and q_1, always taking the ε transitions, will indeed not lead to a successful outcome. But the definition does not require that there must not be any wrong ways to process the input string, only that there must be a right way.

The machine accepts the input string if there is a sequence of transitions that has a halting configuration with an accepting state. The given machine can accept `aab` by starting in q_0, transitioning to q_1 on the first `a`, taking the ε transition to q_0, then going to q_1 on the second `a`, then going to q_2, which is accepting, with the `b`.

$$\langle q_0, \texttt{aab}\rangle \vdash \langle q_1, \texttt{ab}\rangle \vdash \langle q_0, \texttt{ab}\rangle \vdash \langle q_1, \texttt{b}\rangle \vdash \langle q_2, \varepsilon\rangle$$

IV.2.27 For the first three strings see Figure 86 on page 129. For the input strings `aa` and `ab` see Figure 87 on page 130. For `ba` and `bb` see Figure 88 on page 130.

IV.2.28 This is the sequence of transitions.

$$\langle q_0, \texttt{010101110}\rangle \vdash \langle q_0, \texttt{10101110}\rangle \vdash \langle q_0, \texttt{0101110}\rangle \vdash \langle q_1, \texttt{101110}\rangle$$
$$\vdash \langle q_2, \texttt{01110}\rangle \vdash \langle q_3, \texttt{1110}\rangle \vdash \langle q_4, \texttt{110}\rangle \vdash \langle q_5, \texttt{10}\rangle \vdash \langle q_6, \texttt{0}\rangle \vdash \langle q_7, \varepsilon\rangle$$

Input:	a		a	
	q_3	$\vdash$	q_3	$\vdash$ q_3
	$\perp$			
	q_0	$\vdash$	q_2	
Step:	0		1	2

Input:	a		b	
	q_3	$\vdash$	q_3	q_3
	$\perp$			$\perp$
	q_0	$\vdash$	q_2	$\vdash$ q_0
Step:	0		1	2

FIGURE 87, FOR QUESTION IV.2.27: Computation trees for the strings aa and aa.

Input:	b		a	
		q_3	$\vdash$	q_3
		$\perp$		
	q_3	q_0	$\vdash$	q_2
	$\perp$	$\perp$		
	q_0	$\vdash$ q_1		
Step:	0	1		2

Input:	b		b	
				q_3
				$\perp$
		q_3		q_0
		$\perp$		$\perp$
	q_3	q_0	$\vdash$	q_1
	$\perp$	$\perp$		
	q_0	$\vdash$ q_1		
Step:	0	1		2

FIGURE 88, FOR QUESTION IV.2.27: Computation trees for the strings ba and bb.

IV.2.29

(A) Here are the ε closures.

state q	q_0	q_1	q_2
ε *closure* $\hat{E}(q)$	$\{q_0\}$	$\{q_1, q_2\}$	$\{q_2\}$

(B) It does not accept the empty string because the ε closure of q_0 does not contain any final states.

(C) It accepts both of the one-character strings a and b. For instance, on the string a the machine can transition from q_0 to q_1, and then it can make an ε transition to q_2, which is an accepting state.

(D) The given machine can accept aab by starting in q_0, transitioning to q_0 on the first a, then going to q_1 on the second a, taking the ε transition to q_2, then going to q_2 with the b, thereby ending in an accepting state.

$$\langle q_0, \mathsf{aab}\rangle \vdash \langle q_0, \mathsf{ab}\rangle \vdash \langle q_1, \mathsf{b}\rangle \vdash \langle q_2, \mathsf{b}\rangle \vdash \langle q_2, \varepsilon\rangle$$

(E) Five of the shortest strings are: a, b, aa, ab, and aab.

(F) Five that it does not accept are: the empty string ε, ba, baa, bab, and baaa.

IV.2.30

Δ	a	b	ε
q_0	$\{q_0, q_1\}$	$\{q_1\}$	$\{\}$
q_1	$\{\}$	$\{\}$	$\{q_2\}$
+ q_2	$\{\}$	$\{q_0, q_2\}$	$\{\}$

IV.2.31

(A) $\langle q_0, \mathtt{011}\rangle \vdash \langle q_2, \mathtt{11}\rangle \vdash \langle q_2, \mathtt{1}\rangle \vdash \langle q_2, \varepsilon\rangle$

(B) $\langle q_0, \mathtt{00011}\rangle \vdash \langle q_1, \mathtt{0011}\rangle \vdash \langle q_1, \mathtt{011}\rangle \vdash \langle q_1, \mathtt{11}\rangle \vdash \langle q_2, \mathtt{1}\rangle \vdash \langle q_2, \varepsilon\rangle$

(C) Yes, because the state q_0 is accepting.

(D) It accepts `0` because it can go through these steps that end in an accepting state.

$$\langle q_0, \mathtt{0}\rangle \vdash \langle q_2, \varepsilon\rangle$$

It does not accept 1 because there is no such sequence of steps.

(E) Five are ε, `0`, `01`, `011`, and `0111`.

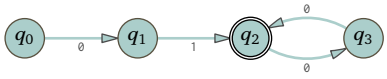
(F) Here are five: 1, 10, 11, 010, and 110.

(G) One description is $\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma = 0^i 1^j \text{ for } i \geq 1 \text{ and } j \geq 0\}$. (Note that $\sigma = \varepsilon$ when $i = j = 0$.)

IV.2.32 State q_0 reaches its limit at column $m = 1$. State q_1 reaches its limit at column 0. State q_2 reaches its limit at column 2, and q_3 reaches its limit at 1.

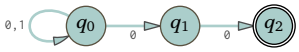
	$m = 0$	1	2	3	$\hat{E}(q)$
q_0	$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$
q_1	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$
q_2	$\{q_2\}$	$\{q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$
q_3	$\{q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_3\}$

IV.2.33 No need for an error state in a nondeterministic machine.

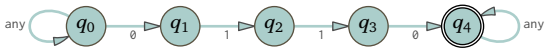


IV.2.34

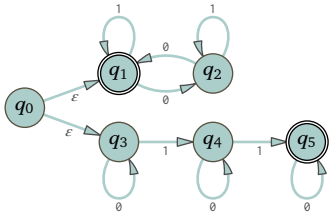
(A)



(B)



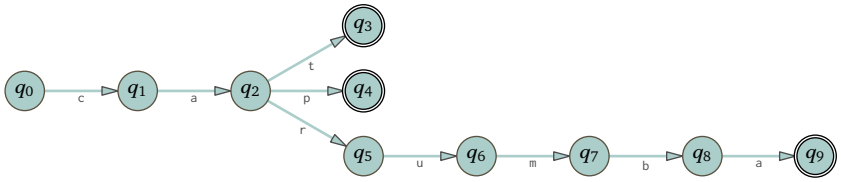
(C)



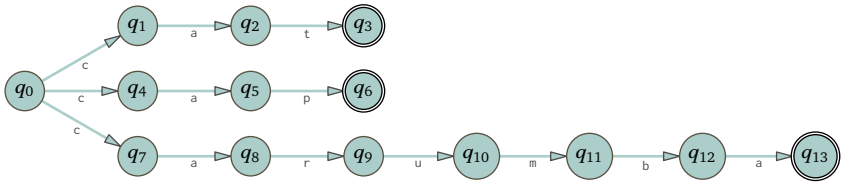
(D)

IV.2.35

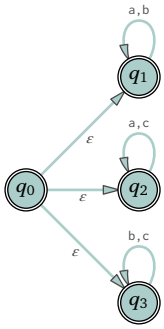
IV.2.36 There are a number of ways to go. This is one.



and this is another.

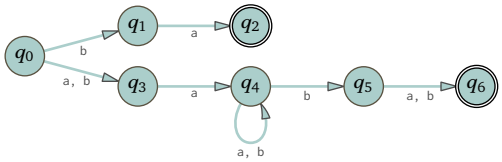


IV.2.37 This language includes the empty string.

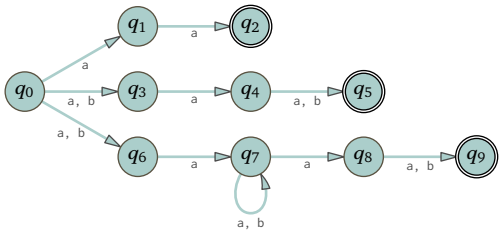


IV.2.38

(A)

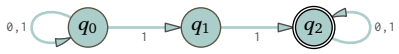


(B)

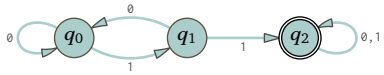


IV.2.39 Any b must be followed by an a, except if it appears at the end of the string, where it must be a string of b's without an a. In short, the language is $\mathcal{L} = \{\sigma \in \{\mathsf{a},\mathsf{b}\}^* \mid \sigma \text{ has no substring } \mathsf{bba}\}$.

IV.2.40 Here is the nondeterministic machine



and this is a deterministic one.



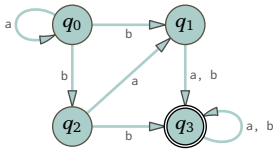
IV.2.41 These are for the machine on the left, and on the right.

Δ_{D_0}	0	1
$r_0 = \{\,\}$	r_0	r_0
$r_1 = \{q_0\}$	r_4	r_0
$r_2 = \{q_1\}$	r_3	r_3
$+\ r_3 = \{q_2\}$	r_0	r_1
$r_4 = \{q_0, q_1\}$	r_7	r_3
$+\ r_5 = \{q_0, q_2\}$	r_4	r_1
$+\ r_6 = \{q_1, q_2\}$	r_3	r_5
$+\ r_7 = \{q_0, q_1, q_2\}$	r_7	r_5

Δ_{D_1}	0	1
$r_0 = \{\,\}$	r_0	r_0
$r_1 = \{q_0\}$	r_7	r_0
$r_2 = \{q_1\}$	r_3	r_3
$+\ r_3 = \{q_2\}$	r_0	r_5
$r_4 = \{q_0, q_1\}$	r_7	r_3
$+\ r_5 = \{q_0, q_2\}$	r_7	r_5
$+\ r_6 = \{q_1, q_2\}$	r_3	r_5
$+\ r_7 = \{q_0, q_1, q_2\}$	r_7	r_5

IV.2.42 The alphabet is $\Sigma = \{\mathsf{a},\mathsf{b}\}$.

(A)



(B) A string is in the language if it has the form $\sigma_0 \frown \sigma_1 \frown \sigma_2$ where $\sigma_0 = \mathsf{a}^k\mathsf{b}$ for some $k \in \mathbb{N}$ and σ_1 is either aa or ab or b, and σ_2 is any member of Σ^*

(C) This is the transition function for the associated deterministic machine.

Δ_D	a	b
$r_0 = \{\,\}$	r_0	r_0
$r_1 = \{q_0\}$	r_1	r_8
$r_2 = \{q_1\}$	r_4	r_4
$+\ r_3 = \{q_2\}$	r_2	r_4
$r_4 = \{q_3\}$	r_4	r_4
$r_5 = \{q_0, q_1\}$	r_7	r_{14}
$+\ r_6 = \{q_0, q_2\}$	r_5	r_{14}
$r_7 = \{q_0, q_3\}$	r_7	r_{14}
$+\ r_8 = \{q_1, q_2\}$	r_9	r_4
$r_9 = \{q_1, q_3\}$	r_4	r_4
$+\ r_{10} = \{q_2, q_3\}$	r_9	r_4
$+\ r_{11} = \{q_0, q_1, q_2\}$	r_{12}	r_{14}
$r_{12} = \{q_0, q_1, q_3\}$	r_7	r_{14}
$+\ r_{13} = \{q_0, q_2, q_3\}$	r_{12}	r_{14}
$+\ r_{14} = \{q_1, q_2, q_3\}$	r_9	r_4
$+\ r_{15} = \{q_0, q_1, q_2, q_3\}$	r_{12}	r_{14}

IV.2.43

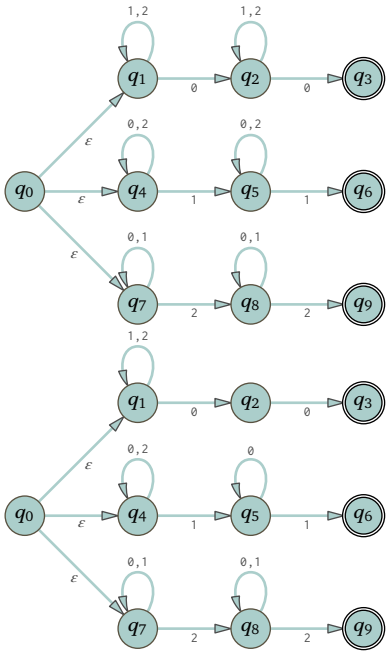
The associated deterministic machine has eight states. Its accepting states are the ones containing q_2.

Δ_D	0	1
$r_0 = \{\,\}$	r_0	r_0
$r_1 = \{q_0\}$	r_0	r_2
$r_2 = \{q_1\}$	r_3	r_0
$+\ r_3 = \{q_2\}$	r_3	r_3
$r_4 = \{q_0, q_1\}$	r_3	r_2
$+\ r_5 = \{q_0, q_2\}$	r_3	r_6
$+\ r_6 = \{q_1, q_2\}$	r_3	r_3
$+\ r_7 = \{q_0, q_1, q_2\}$	r_3	r_6

IV.2.44

(A)

(B)



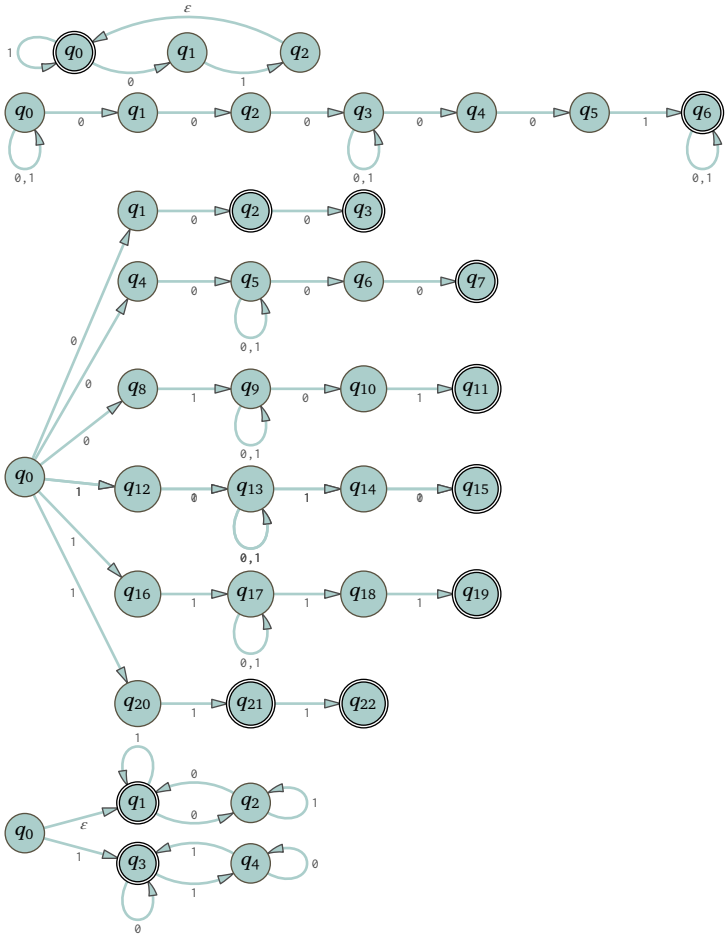
IV.2.45

(A)

(B)

(C)

(D)



IV.2.46 Here are the graphs.

The machines have these tables.

Δ	a	b	c
+ q_0	$\{\,\}$	$\{\,\}$	$\{\,\}$

Δ	a	b	c
q_0	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$
+ q_1	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$

IV.2.47

(A) This is a derivation of abb.

$$S \Rightarrow aA \Rightarrow abB \Rightarrow abb$$

This is a derivation of aabb.

$$S \Rightarrow aA \Rightarrow aaA \Rightarrow aabB \Rightarrow aabb$$

And, this is a derivation of abbb.

$$S \Rightarrow aA \Rightarrow abB \Rightarrow abbB \Rightarrow abbb$$

This verifies that the first is accepted by the machine.

$$\langle S, abb\rangle \vdash \langle A, bb\rangle \vdash \langle B, b\rangle \vdash \langle F, \varepsilon\rangle$$

The second is here.

$$\langle S, aabb\rangle \vdash \langle A, abb\rangle \vdash \langle A, bb\rangle \vdash \langle B, b\rangle \vdash \langle F, \varepsilon\rangle$$

The third is similar.

$$\langle S, abbb\rangle \vdash \langle A, bbb\rangle \vdash \langle B, bb\rangle \vdash \langle B, b\rangle \vdash \langle F, \varepsilon\rangle$$

(B) The language of the machine and the grammar is $\mathcal{L} = \{\, a^i b^j \mid i > 0, j > 1 \,\}$.

IV.2.48 Each of these is solvable. For each we will sketch an algorithm.

(A) Clearly we can write a simulator for deterministic Finite State machines. This simulator is uniform in that it takes in the Δ table and the input string. By Church's Thesis since we can write it on a modern computer, we can write one for a Turing machine. When given an input σ, the machine will consume it in no more than $|\sigma|$ transitions, and so the simulation is sure to halt. Then we can just check whether the ending state is an accepting state.

(B) The section body discusses how to simulate a nondeterministic computation, by dovetailing (that is, time-slicing). Or, we can use the powerset algorithm to convert it to a deterministic machine and then simulate that. We can then answer questions about whether this machine accepts a given input σ just as in the prior item.

IV.2.49

(A) Here is the list.

$E(q,m)$	$m=0$	1	2	3
q_0	$\{q_0\}$	$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_3\}$
q_1	$\{q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$
q_2	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$

So, $\hat{E}(q_0) = \{q_0, q_3\}$, $\hat{E}(q_1) = \{q_0, q_1, q_3\}$, $\hat{E}(q_2) = \{q_2\}$, and $\hat{E}(q_3) = \{q_3\}$.

(B) This is similar.

$E(q,m)$	$m=0$	1	2	3
q_0	$\{q_0\}$	$\{q_0\}$	$\{q_0\}$	$\{q_0\}$
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$

So, $\hat{E}(q_0) = \{q_0\}$, $\hat{E}(q_1) = \{q_1, q_2\}$, and $\hat{E}(q_2) = \{q_2\}$.

IV.3.16 Each one illustrates the match using underbars.

	a*b	a*	∅	ε	b(a|b)a	(a|b)(ε|a)a
ε	*F*	*T*	*F*	*T*	*F*	*F*
a	*F*	*T*	*F*	*F*	*F*	*F*
b	*T*	*F*	*F*	*F*	*F*	*F*
aa	*F*	*T*	*F*	*F*	*F*	*T*
ab	*T*	*F*	*F*	*F*	*F*	*F*
ba	*F*	*F*	*F*	*F*	*F*	*T*
bb	*F*	*F*	*F*	*F*	*F*	*F*
aaa	*F*	*T*	*F*	*F*	*F*	*T*
aab	*T*	*F*	*F*	*F*	*F*	*F*
aba	*F*	*F*	*F*	*F*	*F*	*F*
abb	*F*	*F*	*F*	*F*	*F*	*F*
baa	*F*	*F*	*F*	*F*	*T*	*T*
bab	*F*	*F*	*F*	*F*	*F*	*F*
bba	*F*	*F*	*F*	*F*	*T*	*F*
bbb	*F*	*F*	*F*	*F*	*F*	*F*

FIGURE 89, FOR QUESTION IV.3.19: String matching test.

(A) Yes. One match is $\underset{0^*}{00}\ \underset{1}{1}\ \underset{0}{0}$.

(B) Yes This works: $\underset{1^*}{1}\ \underset{0}{0}\ \underset{1}{1}$.

(C) No. To match a string with a 0, that 0 must be the final character.

(D) Yes. A match is $\underset{1^*}{1}\ \ \underset{(0|1)^*}{01}$.

(E) Yes; this is a case where the star means 'zero' in 'zero or more'. The concatenation of the empty string ε with the string 01 equals the given string 01. So we can write 01 as $\varepsilon^\frown 01$ and recognize this match: $\underset{1^*}{\varepsilon}\ \underset{0}{0}\ \underset{1^*}{1}$.

IV.3.17

(A) Five that match are 0, 01, 011, 0111, and $01^4 = 01111$. Five that do not are ε, 1, 10, 11, and 100.

(B) Five matching strings are ε, 01, 0101, 010101, and $(01)^4$. Five that don't match are 0, 1, 00, 10, and 11.

(C) There are only two matches, 101 and 111. Five that do not match are ε, 0, 1, 00, 01, and 10.

(D) Five matches are 0, 01, 00, 1, and 10. Five that don't match are ε, 001, 0001, 101, and 1101.

(E) Matches: none. Non-matches: ε, 0, 1, 00, 01, and 10.

IV.3.18

(A) The language consists of strings with some number of a's (that number may be zero), followed by a single c, followed by some number of b's.

(B) The language consists of strings of at least one a.

(C) Each string in this language starts with an a, followed by any number of a's and b's, and ends with two b's.

IV.3.19 Figure 89 on page 136 shows the table. Observe that the regular expression ∅ never matches.

IV.3.20 Figure 90 on page 137 shows the table.

IV.3.21 First, (0*1)* matches 01001 because we can decompose it into $01001 = 01^\frown 001$, and both 01 and 001 match (0*1). Second, (0*1)* matches 00000101 because $00000101 = 000001^\frown 01$, and both of those match (0*1). Thus, Kleene star means more like "repeatedly match the inside."

In terms of the definitions, the language of the star is the star of the language: $\mathcal{L}(R^*) = (\mathcal{L}(R))^*$. The definition of the latter is about decomposition, $\mathcal{L}(R)^* = \{\, \sigma_0{}^\frown \cdots {}^\frown \sigma_{k-1} \;\big|\; k \in \mathbb{N} \text{ and } \sigma_0, \ldots, \sigma_{k-1} \in \mathcal{L}(R) \,\}$.

IV.3.22 Regular expressions are not unique. For instance, here are two regular expressions for the language of strings over $\Sigma = \{a, b\}$ that have a substring with at least three a's: (a|b)*aaa(a|b)* and (a|b)*aaaa*(a|b)*.

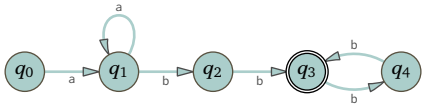
	`0*1`	`1*0`	∅	ε	`0(0|1)*`	`(100)(ε|1)0*`
ε	*F*	*F*	*F*	*T*	*F*	*F*
`0`	*F*	*T*	*F*	*F*	*T*	*F*
`1`	*T*	*F*	*F*	*F*	*F*	*F*
`00`	*F*	*F*	*F*	*F*	*T*	*F*
`01`	*T*	*F*	*F*	*F*	*T*	*F*
`10`	*F*	*T*	*F*	*F*	*F*	*F*
`11`	*F*	*F*	*F*	*F*	*F*	*F*
`000`	*F*	*F*	*F*	*F*	*T*	*F*
`001`	*T*	*F*	*F*	*F*	*T*	*F*
`010`	*F*	*F*	*F*	*F*	*T*	*F*
`011`	*F*	*F*	*F*	*F*	*T*	*F*
`100`	*F*	*F*	*F*	*F*	*F*	*T*
`101`	*F*	*F*	*F*	*F*	*F*	*F*
`110`	*F*	*T*	*F*	*F*	*F*	*F*
`111`	*F*	*F*	*F*	*F*	*F*	*F*

FIGURE 90, FOR QUESTION IV.3.20: String matching tests.

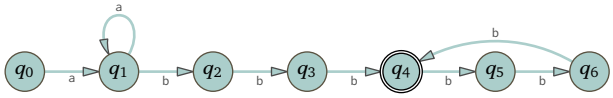
IV.3.23 One is `(0|1)*111(0|1)*`.

IV.3.24 For each, many sets of five strings are possible answers. We list here one set out of many.

(A) Here are five members of $\mathcal{L}_0$: abb, aabb, ab^4, aab^4, ab^6. Here are five that are not: ε, a, b, ab, abba, ba. A regular expression is `aa*bb(bb)*`. Here is a (nondeterministic) Finite State machine that accepts this language.



(B) Five members of $\mathcal{L}_1$ are: abbb, aabbb, ab^6, aab^6, ab^9. Five that are not are: ε, a, b, abbbb, abbba, ba. A regular expression is `aa*bbb(bbb)*`. This is a nondeterministic Finite State machine that accepts $\mathcal{L}_1$.



IV.3.25 `((a|b)*)|((a|c)*)|((b|c)*)`

IV.3.26

(A) `b(a|b)*`

(B) `(a|b)*a(a|b)`

(C) The two characters could come in the order $\sigma = \ldots \mathsf{a} \ldots \mathsf{b} \ldots$ or the order $\sigma = \ldots \mathsf{a} \ldots \mathsf{b} \ldots$ So this works: `(a|b)*((a(a|b)*b)|(b(a|b)*a))(a|b)*`.

(D) The a's must come in triplets: `(b*ab*ab*a)*b*`.

IV.3.27

(A) `aba(a|b)*`

(B) `(a|b)*aba`

(C) `(a|b)*aba(a|b)*`

IV.3.28

(A) There must be at least one 1, and more 1's must come in pairs: `(0*10*1)*0*10*`.

(B) The regular expression `(ε|1)(01)*(ε|0)` will do. In particular, it matches the empty string as well as the string `0` and the string 1.

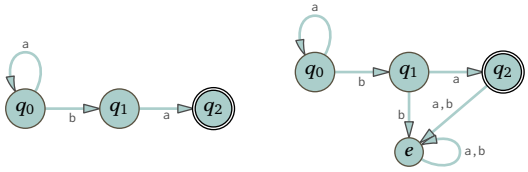


FIGURE 91, FOR QUESTION IV.3.33: Finite State machine for a*ba.

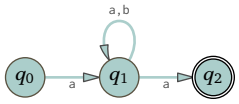


FIGURE 92, FOR QUESTION IV.3.33: Finite State machine for ab*(a|b)*a, which describes the same language as a(a|b)*a.

(C) A multiple of eight in binary ends in three 0's: `(0|1)*000`. That expression matches binary numbers with leading 0's and won't match 0. If you don't like that then instead use `(1(0|1)*000)|0`.

IV.3.29

(A) This is an answer: `((ba)*bb*)*`. Notice that it matches the empty string, and that it matches babab and bbbbab and b.

(B) `(a|b)*((bba*bb)|(bbb))(a|b)*`

IV.3.30

(A) Here the 0's come in pairs: `(1*01*01*)*1*`. Note that it matches ε as well as 1.

(B) Here are three 1's without Kleene star's: `0*10*10*1(0|1)*`.

(C) We can start with the expression from the prior item and for the 0's, substitute the expression from the first item.

```
((1*01*01*)*1*)*1((1*01*01*)*1*)*1((1*01*01*)*1*)*1(((1*01*01*)*1*)|1)*
```

IV.3.31

(A) `(a(a|b)*a)|(b(a|b)*b)`

(B) `((aaa)*b(aaa)*)|(a(aaa)*ba(aaa)*)|(aa(aaa)*baa(aaa)*)`

IV.3.32

(A) This requires the number to have a leading 1: `1(0|1)*0`. To also allow the number 0 we can use the expression `(1(0|1)*0)|0`.

(B) `(1(01)*0)|(0(10)*1)`

(C) `ε|0|1|(0(10)*(1|ε))|(1(01)*(0|ε))`

IV.3.33

(A) A nondeterministic Finite State machine is on the left in Figure 91 on page 138. If you prefer a deterministic machine, use the one on the right.

(B) A nondeterministic machine is in Figure 92 on page 138. Note that `b*(a|b)*` collapses to `(a|b)*`, so that is what the machine handles. We have given a way to convert a nondeterministic Finite State machine to a deterministic one, so if you prefer a deterministic one then take the above nondeterministic machine as an abbreviation.

IV.3.34 We will construct the reachable states, and then the unreachable ones are the others.

(A) The set of reachable states is $S_2 = S_3 = \cdots = \{q_0, q_1, q_2\}$. So the set of unreachable states is $\{q_3\}$.

i	S_i
0	$\{q_0\}$
1	$\{q_0, q_1\}$
2	$\{q_0, q_1, q_2\}$
3	$\{q_0, q_1, q_2\}$

⟨*reg-exp*⟩ ⇒ ⟨*concat*⟩
⇒ ⟨*concat*⟩ ⟨*simple*⟩
⇒ ⟨*concat*⟩ (⟨*reg-exp*⟩)
⇒ ⟨*concat*⟩ (⟨*reg-exp*⟩ '|' ⟨*concat*⟩)
⇒ ⟨*concat*⟩ (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ ⟨*simple*⟩ (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ a (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ a (⟨*simple*⟩ '|' ⟨*concat*⟩)
⇒ a (b '|' ⟨*concat*⟩)
⇒ a (b '|' ⟨*simple*⟩)
⇒ a (b '|' c)

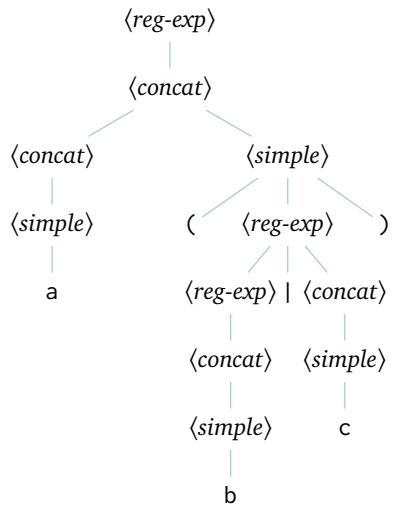


FIGURE 93, FOR QUESTION IV.3.35: Derivation and parse tree for `a(b|c)`.

(B) The set of reachable states is $S_2 = S_3 = \cdots = \{q_0, q_1, q_3\}$. Thus the set of unreachable states is $\{q_2, q_4\}$.

i	S_i
0	$\{q_0\}$
1	$\{q_0, q_1\}$
2	$\{q_0, q_1, q_3\}$
3	$\{q_0, q_1, q_3\}$

IV.3.35

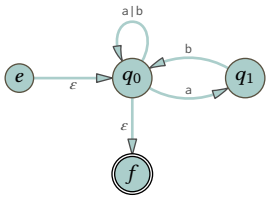
(A) Figure 93 on page 139 has the derivation and the parse tree.

(B) Figure 94 on page 140 shows the derivation and the parse tree.

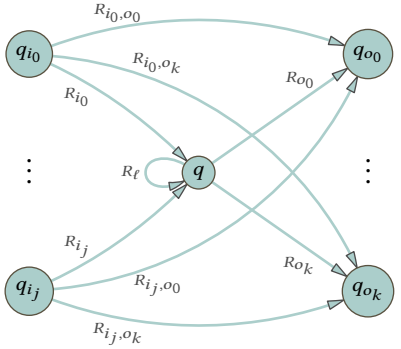
IV.3.36 Figure 95 on page 140 shows the two trees.

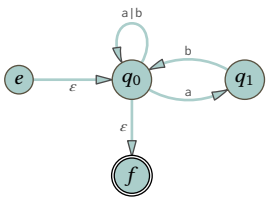
IV.3.37

(A) This is $\hat{\mathcal{M}}$.



(B) For convenience, this is the before half of the diagram from Lemma 3.14's proof, along with $\hat{\mathcal{M}}$.





⟨*reg-exp*⟩ ⇒ ⟨*concat*⟩
⇒ ⟨*concat*⟩ ⟨*simple*⟩
⇒ ⟨*concat*⟩ (⟨*reg-exp*⟩)
⇒ ⟨*concat*⟩ (⟨*reg-exp*⟩ '|' ⟨*concat*⟩)
⇒ ⟨*concat*⟩ (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ ⟨*concat*⟩ ⟨*simple*⟩ (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ ⟨*simple*⟩ ⟨*simple*⟩ (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ a ⟨*simple*⟩ (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ a ⟨*simple*⟩ * (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ a b * (⟨*concat*⟩ '|' ⟨*concat*⟩)
⇒ a b * (⟨*simple*⟩ '|' ⟨*concat*⟩)
⇒ a b * (a '|' ⟨*concat*⟩)
⇒ a b * (a '|' ⟨*simple*⟩)
⇒ a b * (a '|' c)

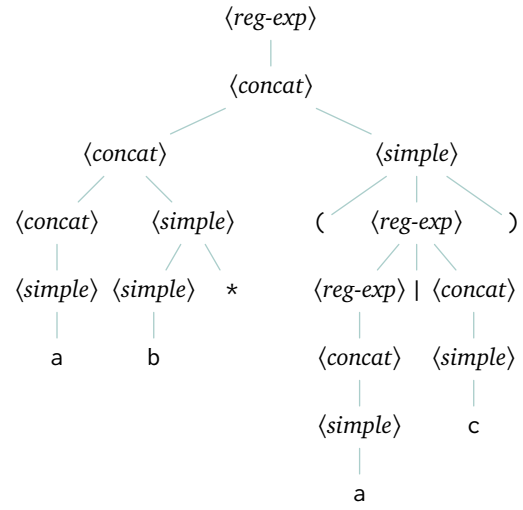


FIGURE 94, FOR QUESTION IV.3.35: Derivation and parse tree for ab*(a|c).

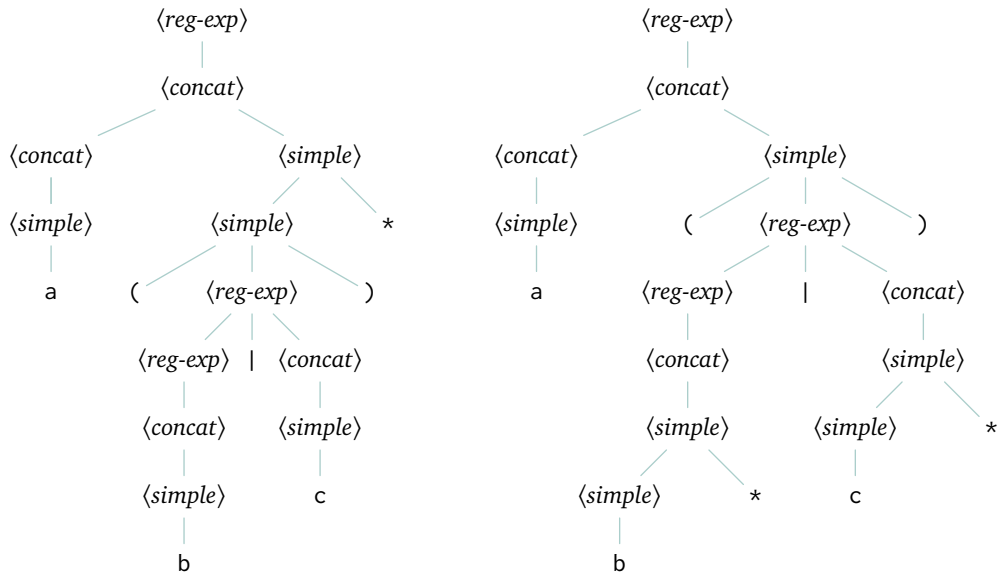


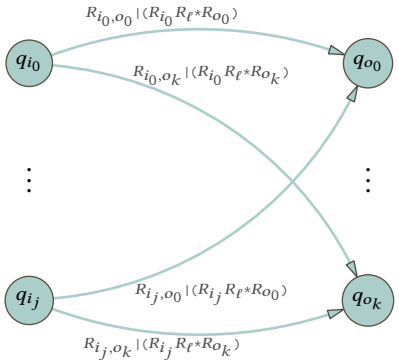
FIGURE 95, FOR QUESTION IV.3.36: Parse trees for a(b|c)* and a(b*|c*).

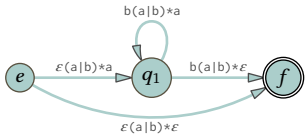
The state q_0 has two states that originate arrows into it, two that receive arrows from it, as well as a loop. So we get this translation of the diagram's notation.

q	q_{i_0}	q_{i_1}	q_{o_0}	q_{o_1}
q_0	e	q_1	q_1	f

R_{i_0}	R_{i_1}	R_{i_0,o_0}	R_{i_0,o_1}	R_{i_1,o_0}	R_{i_1,o_1}	R_ℓ	R_{o_0}	R_{o_1}
ε	b	–	–	–	–	a|b	a	ε

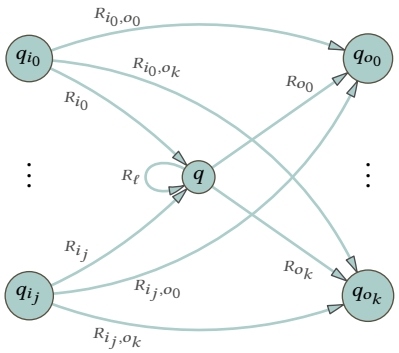
(C) This is the after half diagram of Lemma 3.14's proof, along with the machine $\hat{\mathcal{M}}$ after q_0 is eliminated.

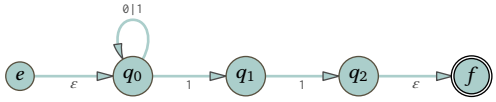




IV.3.38

(A) This is $\hat{\mathcal{M}}$ along with the before half of the diagram from Lemma 3.14's proof.



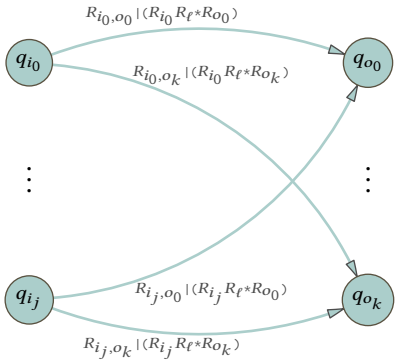


The state q_0 has one state that originates arrows into it, and one that receives arrows from it. So we get this translation of the diagram's notation.

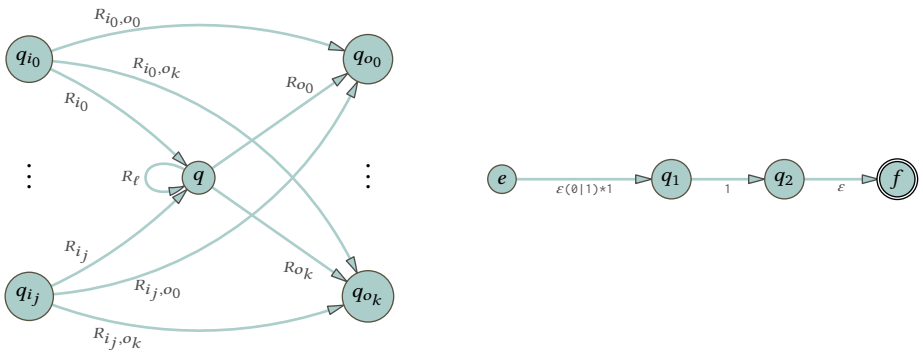
q	q_{i_0}	q_{o_0}
q_0	e	q_1

R_{i_0}	R_{i_0,o_0}	R_ℓ	R_{o_0}
ε	–	0|1	1

This is the after half diagram of Lemma 3.14's proof, along with the machine $\hat{\mathcal{M}}$ after q_0 is eliminated.



(B) This is the machine from the prior answer, along with the before half of the diagram from Lemma 3.14's proof.

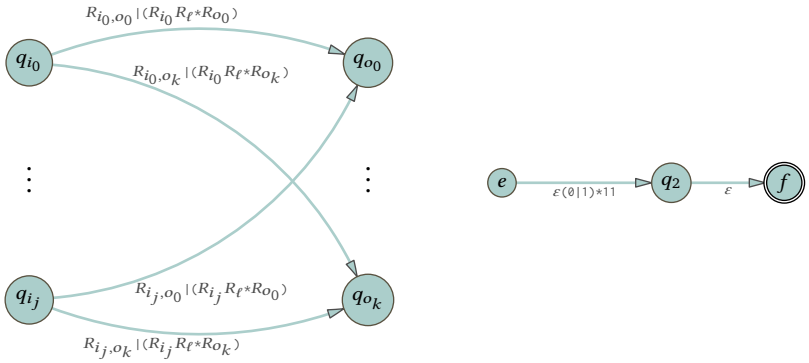


The state q_1 has one state that originates arrows into it, and one that receives arrows from it. So we get this translation of the diagram's notation.

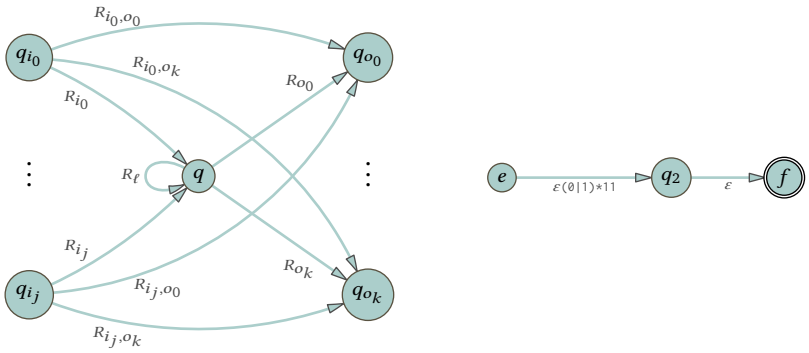
q	q_{i_0}	q_{o_0}
q_1	e	q_2

R_{i_0}	R_{i_0,o_0}	R_ℓ	R_{o_0}
$\varepsilon(0\|1)*1$	–	–	1

This is the after half diagram of Lemma 3.14's proof, along with the machine $\hat{\mathcal{M}}$ after q_1 is eliminated.



(c) This is the machine from the prior answer, along with the before half diagram.

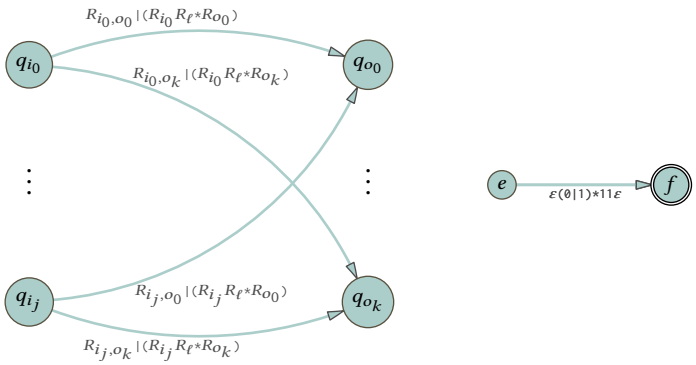


The state q_2 has one state that originates arrows into it, and one that receives arrows from it. So we get this translation of the diagram's notation.

q	q_{i_0}	q_{o_0}
q_2	e	f

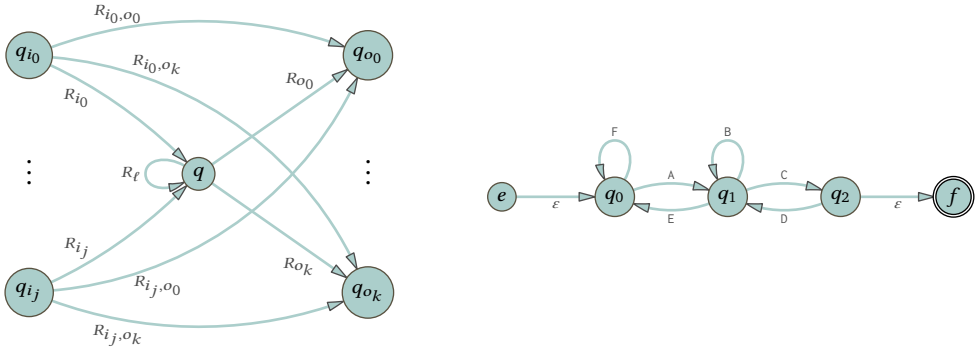
R_{i_0}	R_{i_0,o_0}	R_ℓ	R_{o_0}
$\varepsilon(0\|1)*11$	–	–	ε

This is the after half diagram along with the machine after q_2 is gone.



(D) The regular expression is $\varepsilon(0|1)*11\varepsilon$.

IV.3.39 This is $\hat{\mathcal{M}}$ along with the before half of the diagram from Lemma 3.14's proof for reference.

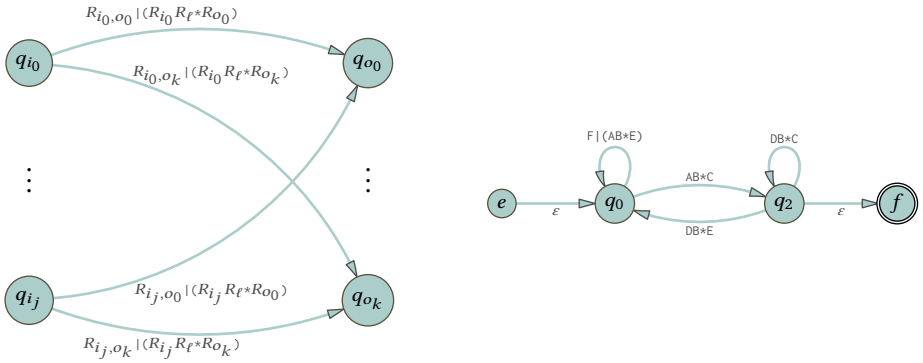


The state q_1 has two states that originate arrows into it, and two that receive arrows from it. So we get this translation of the diagram's notation.

q	q_{i_0}	q_{i_1}	q_{o_0}	q_{o_1}
q_1	q_0	q_2	q_0	q_2

R_{i_0}	R_{i_1}	R_{i_0,o_0}	R_{i_0,o_1}	R_{i_1,o_0}	R_{i_1,o_1}	R_ℓ	R_{o_0}	R_{o_1}
A	D	F	–	–	–	B	E	C

Here is the after diagram, along with the machine $\hat{\mathcal{M}}$ after the elimination of q_1.



IV.3.40 The first is recognized by the same k-state machine, but with the accepting states made nonaccepting and with the nonaccepting made accepting. The second will not be recognized by a k-state machine in the case that $\mathcal{L} = \varnothing$ and $k = 1$.

IV.3.41 We can leverage Kleene's theorem for these.

(A) Transform the machine $\mathcal{M}$ to a new machine $\hat{\mathcal{M}}$ by making the set of final states of $\hat{\mathcal{M}}$ be S.

(B) Transform the machine M to $\hat{\mathcal{M}}$ by making the start state be the given first single state, and also make the set S of the prior item contain only the ending state. Then apply the argument given for the prior item.

IV.3.42 We will show this in two halves: that the set of languages is infinite, and that it is at most countable. Recall that, as defined in Appendix A, alphabets are nonempty and finite.

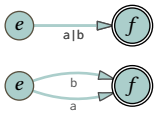
We first show that the number of languages is infinite. Let x be an element of Σ. The one-element language $\mathcal{L}_1 = \{x\}$ is described by the one-character regular expression $R_1 = x$. The one-element language $\mathcal{L}_2 = \{xx\}$ is described by the two-character regular expression $R_2 = xx$. In general, $\mathcal{L}_k$ is a one-element language that is different from all $\mathcal{L}_j$ for $j \neq k$, and is described by a regular expression R_k with k characters. That makes infinitely many different languages described by regular expressions over Σ.

We finish by showing that the set of languages is countable. Every regular expression is associated with only one language and so if we show that there are countably many regular expressions then we will be done. A regular expression is a string over $\Sigma \cup \{\,),\,(,\,|,\,*\}$, which is a finite set. So for any number $n \in \mathbb{N}$ there are finitely many regular expressions of length n. Recall Chapter II's Corollary 2.13, which shows that the countable union of countable sets is countable. Because a finite set is countable, the union over all n of the length n regular expressions is therefore countable.

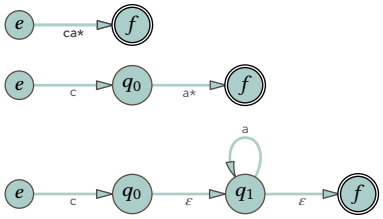
We finish by showing that the set of all languages over Σ, the set of all $\mathcal{L} \subseteq \Sigma^*$, is uncountable. First, Σ^* is countably infinite because there are countably many $\sigma \in \Sigma^*$ of length zero, of length one, etc.. The set of all languages $\mathcal{L} \subseteq \Sigma^*$ is the power set of Σ^* and so has cardinality greater than the cardinality of Σ^*, which makes that set uncountable. So there are languages that are not described by any regular expression.

IV.3.43 Each of these gives the progression of steps.

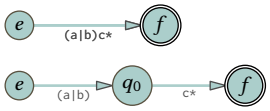
(A)



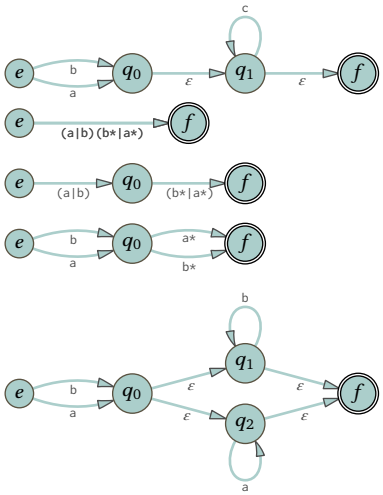
(B) First concatenation and then Kleene star.



(C)



(D)



IV.3.44

(A) Because it accepts the input ε the initial state q_0 must be an accepting state. If q_0 were the only accepting state then this machine accepts a by virtue of a loop from q_0 to itself, which would make the machine accept aa. That is not in the language $\mathcal{L}_1$, so there must be at least one other accepting state.

(B) Because it accepts the input ε the initial state q_0 must be an accepting state. If q_0 were the only accepting state then this machine accepts a by virtue of a loop from q_0 to itself, which would make the machine

accept aaa. That string is not a member of the language $\mathcal{L}_2$ so there must be at least one accepting state other than q_0.

If the machine accepts aa by being in state q_0 then that involves a loop, so the string aaaa would also be accepted by this machine. But that string is not in the language $\mathcal{L}_2$, so the extended transition function on this input, $\hat{E}(\mathtt{aa})$, is not equal to the state q_0. Similarly, if $\hat{E}(\mathtt{aa})$ were equal to $\hat{E}(\mathtt{a})$ then there would be a loop from this state to itself on input a, and so the machine would also accept aaa. But the machine does not accept aaa because that string is not in the language $\mathcal{L}_2$, so there must be a third accepting state.

(c) This is the natural extension of the prior two items. Consider $\mathcal{L}_n = \{\varepsilon, \mathtt{a}, \ldots, \mathtt{a}^n\}$. If $\hat{E}(\mathtt{a}^n)$ were equal to $\hat{E}(\mathtt{a}^i)$ for any $i < n$ then there would be a loop and so the machine would accept additional strings.

IV.4.7 Closure of regular languages under intersection does not imply closure of non-regular languages under intersection. Here is an example. Lemma 4.2 says that over $\Sigma_0 = \{\mathtt{a}, \mathtt{b}\}$ there is a language $\mathcal{L}_0$ that is not regular. Similarly over $\Sigma_1 = \{\mathtt{c}, \mathtt{d}\}$ there is a language $\mathcal{L}_2$ that is not regular. Their intersection is empty because the alphabets do not share characters, and the empty language is regular (we can take it to be a language over $\Sigma_0 \cup \Sigma_1$).

IV.4.8 This is not a well-defined question.

In principle a person could take Σ to be the words in some fixed dictionary, and we could assert that no English sentence has more than a million words (see (Wikipedia contributors 2020)). Then we might claim that English is a regular language because we can list the sentences that we declare allowed, and there are finitely many of them. However, that seems unsatisfactory.

Otherwise we must choose a grammar. But natural languages are not sharply defined: is "My god, it is her." valid even though some would disallow it, sying that instead it should be, "My god, it is she."? (See also `https://cs.stackexchange.com/q/116127/50343`.)

IV.4.9 The finite languages over some fixed alphabet.

For another answer, we can say that a language $\mathcal{L}$ is computably enumerable if there is a Turing machine $\mathcal{P}$ so that if $\sigma \in \mathcal{L}$ then when $\mathcal{P}$ is started with the tape consisting of σ then it halts, while if $\sigma \notin \mathcal{L}$ then it does not halt. It is easy to see that the set of computably enumerable languages is closed under intersection and union but not under complementation.

IV.4.10

(a) False, the empty language is recognized by any Finite State machine that has no accepting states. It is also described by the regular expression $\varnothing$.

(b) False. What would be correct is that the intersection of two regular languages is regular. For example, where $\mathcal{L}$ is not regular, the intersection $\mathcal{L} \cap \mathbb{B}^* = \mathcal{L}$ is not regular.

(c) False. The language $\mathbb{B}^*$ is the set of all strings recognized by a Finite State machine whose states are all accepting states. It is also the language described by the regular expression `(0|1)*`.

(d) False. The infinite language $\{\mathtt{ab}, \mathtt{abb}, \ldots\}$ is described by the regular expression `ab*`, and every pair of strings have a in the first position.

IV.4.11 The first is true and the second is false.

(a) If the language consists of the strings $\sigma_0, \ldots \sigma_{n-1}$ then a regular expression is $\sigma_0 | \cdots | \sigma_{n-1}$. If the language is empty then the regular expression is $\varnothing$.

(b) The language over $\Sigma = \{\mathtt{a}, \mathtt{b}\}$ described by the regular expression `a*` is $\{\varepsilon, \mathtt{a}, \mathtt{aa}, \ldots\}$, and is infinite.

IV.4.12 Yes. In binary, powers of 2 are represented by strings of bits starting with `1` and ending in `0`'s. So the regular expression `10*` describes the language. (If you want to allow leading `0`'s then the regular expression is `0*10*`.)

IV.4.13 One way to show this is to give a regular expression for each.

(a) `a(a|b)*a`

(b) `b*(ab*a)*b*`

IV.4.14 Either that set is $S_0 = \{9^n \mid n < N\}$ for some $N \in \mathbb{N}$, or it is $S_1 = \{9^n \mid n \in \mathbb{N}\}$. Both are regular.

IV.4.15 Because the language is regular it is described by a regular expression, R. Then the regular expression $1R$ describes $\hat{\mathcal{L}}$.

IV.4.16 The size of the set $Q_0 \times Q_1$ is $n_0 \cdot n_1$.

IV.4.17 These are the tables for the machines.

	Δ_0	a	b
+	q_0	q_0	q_1
+	q_1	q_2	q_1
	q_2	q_2	q_0

	Δ_1	a	b
	s_0	s_1	s_0
+	s_1	s_0	s_0

Here is the cross product machine, without specifying accepting states.

Δ	a	b
(q_0, s_0)	(q_0, s_1)	(q_1, s_0)
(q_0, s_1)	(q_0, s_0)	(q_1, s_0)
(q_1, s_0)	(q_2, s_1)	(q_1, s_0)
(q_1, s_1)	(q_2, s_0)	(q_1, s_0)
(q_2, s_0)	(q_2, s_1)	(q_0, s_0)
(q_2, s_1)	(q_2, s_0)	(q_0, s_0)

(A) To have this machine accept the intersection of the two languages, take $F = \{(q_0, s_1), (q_1, s_1)\}$, so that (q_i, s_j) is accepting if and only if both q_i and s_j are accepting.
(B) Take (q_i, s_j) to be accepting if and only if either q_i or s_j is accepting, so $F = \{(q_0, s_0), (q_0, s_1), (q_1, s_0), (q_1, s_1)\}$.

IV.4.18 This is the transition table for $\mathcal{M}$.

	Δ_0	a	b
	s_0	s_0	s_1
+	s_1	s_1	s_0

The product of this machine with itself has four states.

	Δ_1	a	b
	(s_0, s_0)	(s_0, s_0)	(s_1, s_1)
	(s_0, s_1)	(s_0, s_1)	(s_1, s_0)
	(s_1, s_0)	(s_1, s_0)	(s_0, s_1)
+	(s_1, s_1)	(s_1, s_1)	(s_0, s_0)

As shown, the set of accepting states is defined as for the intersection, to $F_\cap = \{(s_1, s_1)\}$. We could have instead used the set for union, $F_\cup = \{(s_1, s_1), (s_0, s_1), (s_1, s_0)\}$ (although the two additional states are unreachable).

IV.4.19 The machine on the left below accepts a string $\sigma \in \mathbb{B}^*$ if it contains at least two `0`'s. The one on the right accepts a string if it contains an even number of `1`'s.

	Δ_0	0	1
	q_0	q_1	q_0
	q_1	q_2	q_1
+	q_2	q_2	q_2

	Δ_1	0	1
+	s_0	s_0	s_1
	s_1	s_1	s_0

The product construction gives this. The set of accepting states is arranged for the intersection of the two languages.

Δ	0	1
(q_0, s_0)	(q_1, s_0)	(q_0, s_1)
(q_0, s_1)	(q_1, s_1)	(q_0, s_0)
(q_1, s_0)	(q_2, s_0)	(q_1, s_1)
(q_1, s_1)	(q_2, s_1)	(q_1, s_0)
(q_2, s_0)	(q_2, s_0)	(q_2, s_1)
(q_2, s_1)	(q_2, s_1)	(q_2, s_0)

IV.4.20

(A) True. Every language is a subset of the language Σ^*.

(B) False. For any alphabet, a language with one element is regular and its only subset is the empty language, which is also regular.

(C) False. Take $\mathcal{L}_0$ to be not regular and take $\mathcal{L}_1$ to be the empty language, $\mathcal{L}_1 = \{\,\}$, which is regular. Their union is $\mathcal{L}_0$.

(D) True. This is part of Lemma 4.3.

IV.4.21 The answer is (B): it might be regular, or might be not regular. Fix an alphabet Σ and a language $\mathcal{L}$ over Σ that is not regular. A concatenation of a regular language with a non-regular one that gives a regular outcome is $\Sigma^* \frown \mathcal{L} = \Sigma^*$. A language concatenation that gives an outcome that is not regular is $\{s\} \frown \mathcal{L}$ where $s \in \Sigma$ (an easy argument gives that regularity of this language implies regularity of $\mathcal{L}$).

IV.4.22 Both are false. The same example suffices for both. For $\mathcal{L}_0$ use the set of all bitstrings, $\mathbb{B}^*$. For $\mathcal{L}_1$ use any non-regular subset of it, the existence of which is guaranteed by Lemma 4.2.

IV.4.23 By Lemma 4.3, regular languages are closed under Kleene star and concatenation.

IV.4.24 Let Σ be the alphabet of $\mathcal{L}$. The collection of even length strings over Σ is regular since there is clearly a Finite State machine that accepts only those strings (or, a regular expression is $((s_0 s_0) \mid (s_0 s_1) \mid \ldots \ (s_n s_n))*$ where $\Sigma = \{s_0, \ldots s_n\}$). The intersection of $\mathcal{L}$ with this collection is an intersection of two regular languages, so it is regular.

IV.4.25 Let the set given as not regular, $\{\mathsf{a}^n\mathsf{b}^n \in \{\mathsf{a},\mathsf{b}\}^* \;\big|\; n \in \mathbb{N}\}$, be $\mathcal{L}_0$. Let the language described by the regular expression a*b* be $\mathcal{L}_1$. And, let the language $\{\sigma \in \{\mathsf{a},\mathsf{b}\}^* \;\big|\; \sigma \text{ contains the same number of a's as b's}\}$ be $\mathcal{L}_2$. If $\mathcal{L}_2$ were regular then because the intersection of regular languages is a regular language, $\mathcal{L}_2 \cap \mathcal{L}_1$ would be regular. But that set is $\mathcal{L}_0$, which is not regular.

IV.4.26 If there was a Finite State machine $\mathcal{M}$ that accepted this language then we could solve the Halting problem with a Turing machine. For, we can obviously write write the finite state machine as a subroutine. Then, given $e \in \mathbb{N}$, feed 1^e to the subroutine. If it accepts, then $e \in K$ and otherwise $e \notin K$.

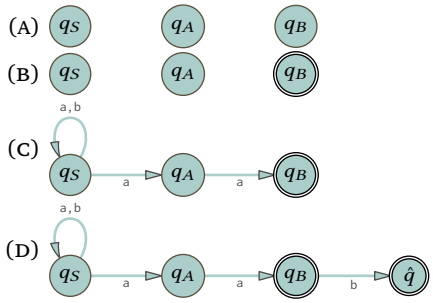
IV.4.27

(A) For every word $\sigma \in \Sigma^*$, the one-word language $\mathcal{L} = \{\sigma\}$ is regular. But any non-regular language is the union of a collection of such languages.

(B) Similarly, for every word $\sigma \in \Sigma^*$, the language $\mathcal{L} = \Sigma^* - \{\sigma\}$ is regular. Any non-regular language $\mathcal{L}$ is the intersection of a collection of such languages, namely the collection over all $\sigma \notin \mathcal{L}$.

IV.4.28 The language of all strings $\mathcal{L} = \mathbb{B}^*$ is regular and is uncountable.

IV.4.29 The result is a nondeterministic Finite State machine.



IV.4.30 To show closure we must show, for instance, that if $\mathcal{L}$ is regular then $\text{pref}(\mathcal{L})$ is also regular.

(A) Fix a Finite State machine $\mathcal{M}$ that recognizes $\mathcal{L}$. Convert to a new machine $\hat{\mathcal{M}}$ with the same states and transitions, but where there are more accepting states. Specifically, find all of the states $q \in \mathcal{M}$ such that from q we can reach an accepting state of $\mathcal{M}$. (That is, there is a sequence of character transitions taking $\mathcal{M}$ from q to some accepting state of $\mathcal{M}$.) In $\hat{\mathcal{M}}$, make every such q an accepting state.

This $\hat{\mathcal{M}}$ recognizes $\text{pref}(\mathcal{L})$. That's because σ is a prefix of some $\alpha \in \mathcal{L}$ if and only if it takes the machine $\mathcal{M}$ from its initial state q_0 to some state q from which the further transitions in τ will lead to $\sigma \frown \tau = \alpha$ getting accepted.

(B) Again, fix a Finite State machine $\mathcal{M}$ that recognizes $\mathcal{L}$. Again we will convert to a new machine $\hat{\mathcal{M}}$ by starting with the same states and transitions.

Let S be the set of states reachable from q_0 in $\mathcal{M}$. Get $\hat{M}$ by introducing a new state r, and defining an ε transition from r to each state in S. This new state r is the start state of $\hat{M}$. Then $\hat{M}$ recognizes $\text{suff}(\mathcal{L})$.

(C) As before, fix a Finite State machine $\mathcal{M}$ that recognizes $\mathcal{L}$. We will convert to a new machine $\hat{\mathcal{M}}$ by starting with the same states and transitions.

Introduce to the new machine a sink state e that is not accepting and from which any input results in a loop back to e. Take every non-accepting state q in $\mathcal{M}$ and replace it with the sink state. That is, all transitions into q now go to e and we omit q with all of its outward transitions.

This machine accepts a string in $\mathcal{L}$ if and only if it accepts all of the prefixes of that string also.

IV.4.31

(A) Lemma 4.3 shows that the collection of regular languages is closed under union.

(B) This identity shows that closure under complement will also demonstrate closure under set difference.

(C) Fix a regular language $\mathcal{L}$ over some alphabet Σ. It is recognized by some deterministic Finite State machine $\mathcal{M}$ whose input alphabet is Σ.

Define a new machine $\hat{\mathcal{M}}$ with the same states and transition function as $\mathcal{M}$ but whose accepting states are the complement, $F_{\hat{\mathcal{M}}} = Q_{\mathcal{M}} - F_{\mathcal{M}}$. We will show that the language of this machine is $\Sigma^* - \mathcal{L}$.

Because $\mathcal{M}$ is deterministic, each input string τ is associated with a unique last state, the state that the machine is in after it consumes τ's last character, namely $\hat{\Delta}(\tau)$. Observe that $\hat{\Delta}(\tau) \in F_{\hat{\mathcal{M}}}$ if and only if $\hat{\Delta}(\tau) \notin F_{\mathcal{M}}$. Thus the language of $\hat{\mathcal{M}}$ is the complement of the language of $\mathcal{M}$ and since this language is recognized by a Finite State machine, it too is regular.

IV.4.32 For one direction suppose that the machine accepts at least one string, σ, of length k where $n \leq k < 2n$. In processing σ the machine must go through at least n-many transitions. But the machine has n states and starting at q_0 and visiting all the other states exactly once would require only $n - 1$ many transitions. Thus, by the Pigeonhole principle, in processing the string σ the machine visits some state q twice. That is, in processing σ the machine makes a loop.

Said another way, the accepted string σ decomposes as $\sigma = \alpha^\frown \beta^\frown \gamma$ where processing the α portion takes the machine from the start to the loop (from state q_0 to q), the β portion takes the machine around the nontrivial loop (so that $\beta \neq \varepsilon$ brings the machine from q to q again) and the γ portion brings the machine to some final state. But then the recognized language is infinite, since clearly the machine also accepts $\alpha^\frown \beta^\frown \beta^\frown \gamma$, etc. (Note that we can take β to be of length less than n because otherwise the size of the machine would imply it has a subloop.)

Conversely, suppose the language of accepted strings is infinite. At least one of them is longer than n; fix a minimal length string that is longer. As in the prior paragraph, because the machine only has n states, by the Pigeonhole principle the accepted string σ decomposes as $\sigma = \alpha^\frown \beta^\frown \gamma$ where the β portion is a nontrivial loop so processing α brings the machine from q_0 to some q, then processing $\beta \neq \varepsilon$ brings the machine from q to q again, and then processing the γ portion brings the machine to some final state. Because the string is of minimal length, $|\beta| < n$. In total then, $|\sigma| < 2n$.

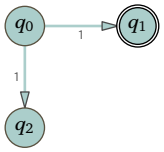
IV.4.33

(A) For the first, $\hat{h}(01) = \hat{h}(0^\frown 1) = \hat{h}(0)^\frown \hat{h}(1) = \mathtt{aba}$. Similarly $\hat{h}(10) = \mathtt{baa}$ and $\hat{h}(101) = \mathtt{baaba}$

(B) A regular expression to describe $\hat{\mathcal{L}} = \{\sigma^\frown 1 \mid \sigma \in \mathbb{B}^*\}$ is `(0|1)*1` A regular expression to describe $\hat{h}(\hat{\mathcal{L}})$ is `(a|ba)*ba`.

(C) Let $\mathcal{L}$ be described by the regular expression R. Apply the function h to each non-meta symbol in R. The result is a regular expression that describes $h(\mathcal{L})$.

IV.4.34 For this machine the language of accepted strings contains 1.



If we complement the set of accepting states then the language of the resulting machine also contains 1.

IV.4.35

(A) Fix σ_0. We prove this by induction on the length of σ_1. For the base step consider both $\sigma_1 = \varepsilon$ and $\sigma_1 = a$ for $a \in \Sigma$. Both of the statements $(\sigma_0{}^\frown \varepsilon)^{\mathsf{R}} = (\sigma_0)^{\mathsf{R}} = (\varepsilon)^{\mathsf{R}\frown}(\sigma_0)^{\mathsf{R}}$ and $(\sigma_0{}^\frown \mathsf{a})^{\mathsf{R}} = (\mathsf{a})^{\mathsf{R}\frown}(\sigma_0)^{\mathsf{R}}$ obviously hold.

For the inductive step assume that the statement is true for any σ_1 when $|\sigma_1| = 0$, $|\sigma_1| = 1$, ... $|\sigma_1| = k$ and consider the $|\sigma_1| = k + 1$ case. In this case $\sigma_1 = \tau^\frown a$ for some character $a \in \Sigma$ and string $\tau \in \Sigma^*$ of length k. Then $(\sigma_0{}^\frown \sigma_1)^R = (\sigma_0{}^\frown \tau^\frown a)^R$. Parenthesize as $\big((\sigma_0{}^\frown \tau)^\frown a\big)^R$ and apply the base case to get $a^R{}^\frown (\sigma_0{}^\frown \tau)^R$. The inductive hypothesis gives $= a^\frown \tau^R{}^\frown \sigma_0{}^R$, and finish by recognizing that as $\sigma_1{}^R{}^\frown \sigma_0{}^R$.

(B) We prove this by induction on the length of the regular expression. There are no regular expressions of length zero so the base case is that $|R| = 1$. If $R = \varnothing$ then by condition (1), R^R is the regular expression $\varnothing$ and $\mathcal{L}(R) = \mathcal{L}(R^R)$ because both equal the empty set, as required. If $R = \varepsilon$ then by condition (2), $R^R = \varepsilon$ and $\mathcal{L}(R) = \{\varepsilon\} = \mathcal{L}(R^R)$, again as required. And, if $R = x$ for some $x \in \Sigma$ then by condition (3), $R^R = x$ and $\mathcal{L}(R) = \{x\} = \mathcal{L}(R^R)$.

Now for the inductive step. Assume that the statement is true for regular expressions R of length $n = 1$, $n = 2$, ... $n = k$, and consider an expression of length $n = k + 1$. We handled the case that it is a concatenation, $R = R_0{}^\frown R_1$, in this question's first item.

If $R = R_0 | R_1$ then $\mathcal{L}(R^R) = \mathcal{L}((R_0|R_1)^R) = \mathcal{L}(R_0{}^R|R_1{}^R)$ by condition (5). By the definition of the language associated with an alternation operation on regular expressions following Definition 3.3, that is $\mathcal{L}(R_0{}^R) \cup \mathcal{L}(R_1{}^R)$. The inductive hypothesis gives that it equals $\mathcal{L}(R_0)^R \cup \mathcal{L}(R_1)^R$, which equals $\{\sigma_0{}^R \mid \sigma_0 \in \mathcal{L}(R_0)\} \cup \{\sigma_1{}^R \mid \sigma_1 \in \mathcal{L}(R_1)\}$ and again by the material following Definition 3.3, that equals $\mathcal{L}(R)^R$.

The final form is similar. If $R = R_0*$ then $\mathcal{L}(R^R) = \mathcal{L}((R_0{}^R)*)$. By the definition following Definition 3.3 that equals $\{\sigma_0{}^\frown \cdots{}^\frown \sigma_m \mid \sigma_i \in \mathcal{L}(R_0{}^R)\}$. Rewriting gives $\{\tau_0{}^R{}^\frown \cdots{}^\frown \tau_m{}^R \mid \tau_j \in \mathcal{L}(R_0)\}$. By this exercise's first item that equals $\{(\tau_m{}^\frown \cdots{}^\frown \tau_0)^R \mid \tau_j \in \mathcal{L}(R_0)\}$, which is the set $(\mathcal{L}(R_0)^*)^R$.

IV.5.8 There is no such regular expression. The syntax of Definition 3.3 does not have the capacity for that construct.

If we fix, say, $n = 2$, then we can make a regular expression $0^2 1^3$. We can use alternation to connect a finite number of such expressions, as in $0|011|0^2 1^3|\ldots 0^9 1^{10}$. But no regular expression describes all of the strings in the language $\mathcal{L}$.

IV.5.9 Compared with $\sigma = \alpha\beta\gamma$, the string $\alpha\gamma$ has had β omitted. By definition σ has balanced parentheses. Because β consists of at least one open parenthesis and has no closed parentheses, the omission of β means that in $\alpha\gamma$ the number of open parentheses is at least one fewer than the number of closed parentheses.

IV.5.10 If a language is finite, $\mathcal{L} = \{\sigma_0, \ldots \sigma_n\}$, then the conclusion of the Pumping Lemma holds by taking the pumping length to be $p > \max(|\sigma_0|, \ldots |\sigma_n|)$.

IV.5.11 It is not correct. The first problem is that it is not sensible. There may be many Finite State machines that recognize the language, so "number of states" for a language is not well-defined.

Even if we grant that they meant to fix a particular machine, saying something like, "where $\mathcal{M}$ accepts the language, if that language has a string of length greater than the number of states in $\mathcal{M}$ then it cannot be a regular language" then it still isn't sensible because if there is a Finite State machine then the language for sure is regular.

And, even if we stretch figuring out what they meant all the way to taking that they meant something like this, "If in an argument we assume that the language is recognized by a Finite State machine and we go on to show that the language has a string that is longer than the number of states in that machine, then we have the desired contradiction" then this statement, while now technically sensible, is wrong. For example, there is a Finite State machine that accepts strings described by a*, and there certainly are strings of that form whose length is greater than the number of states in the machine.

IV.5.12 The Pumping Lemma only allows us to conclude that $\alpha\beta$ consists of ('s, not that this substring got all the ('s. The substring γ indeed got all of the close parentheses, the)'s, but that we know, it may have gotten some of the open parentheses as well.

IV.5.13

(A) (1) $a^p bc^p$, (2) the string $\alpha^\frown \beta$ consists entirely of a's, (3) β consists of at least one a, (4) $\alpha\gamma$, (5) at least one fewer a but the same number of c's. (For (4) and (5) other choices are possible.)

(B) (1) $a^p b^{2p}$, (2) the string $\alpha^\frown \beta$ consists entirely of a's, (3) β consists of at least one a, (4) $\alpha\beta^2\gamma$, (5) at least one extra a but the same number of b's. (For (4) and (5) other choices are possible.)

IV.5.14

(A) Strings are in this language if they consist of a string of a's followed by a string of b's, and the number of b's is two more than the number of a's. Five elements of $\mathcal{L}_0$ are bb, abbb, aabbbb, a^3b^5, and a^4b^6. Five strings that are not elements are ε, a, ab, aabbb, and baabbb.

Assume for contradiction that $\mathcal{L}_0$ is regular. By the Pumping Lemma it has a pumping length, p. Consider $\sigma = a^p b^{p+2}$. This string is a member of the language that is of length greater than or equal to p so the Pumping Lemma gives a decomposition $\sigma = \alpha^\frown \beta^\frown \gamma$ subject to the three conditions. Condition (1) is that $|\alpha\beta| \leq p$, so because the first p-many characters of σ are all a's, the two substrings α and β contain only a's. Condition (2) is that β is nonempty, so it consists of at least one a.

Consider the list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... Its second element $\alpha\beta^2\gamma$ has more a's, but no more b's, than does $\sigma = \alpha\beta\gamma$. That means $\alpha\beta^2\gamma$ is not an element of the language $\mathcal{L}_0$, a contradiction to the Pumping Lemma's condition (3).

(B) In prose, this language consists of a's followed by b's and then followed by c's, with only the restriction that the number of a's equals the number of c's. Thus five elements of $\mathcal{L}_1$ are: ε, ac, b, aabcc, and bb. Five that are not are: aac, acc, ab, bbc, and ca.

Assume for contradiction that $\mathcal{L}_1$ is regular. Then it has a pumping length p. Consider $\sigma = a^p c^p$. Because $\sigma \in \mathcal{L}_1$ and $|\sigma| \geq p$, the Pumping Lemma says that it decomposes, $\sigma = \alpha^\frown \beta^\frown \gamma$, in a way that satisfies the three conditions. By condition (1) the first two substrings together are short, $|\alpha\beta| \leq p$. Thus these two consist entirely of a's. By condition (2) the second substring β is nonempty, so it consists of at least one a. Finally, condition (3) gives that the strings $\alpha\gamma$, $\alpha\beta^2\gamma$, ... are all members of the language $\mathcal{L}_1$. But the string $\alpha\gamma$, when compared with $\sigma = \alpha\beta\gamma$, has at least one fewer a but the same number of c's. That means that the number of a's in σ differs from the number of c's, so it is not a member of the language, which is a contradiction.

(C) In $\mathcal{L}_2$ the strings have a's followed by b's, with the number of a's less than the number of b's. So five strings that are members of $\mathcal{L}_2$ are abb, abbb, abbbb, ab^5, and a^4b^5. Five strings that are not elements are ε, a, ab, aab, and baabbb.

For contradiction suppose that $\mathcal{L}_2$ is regular and let its pumping length be p. The string $\sigma = a^p b^{p+1}$ is an element of the language whose length is greater than or equal to the pumping length. So the Pumping Lemma gives a decomposition, $\sigma = \alpha\beta\gamma$, subject to the three conditions. Condition (1), that $|\alpha\beta| \leq p$, says that these two substrings contain only a's. Condition (2), that β is nonempty, says that it consists of at least one a.

Consider the strings $\alpha\gamma$, $\alpha\beta^2\gamma$, ... Compared with $\sigma = \alpha\beta\gamma$, the string $\alpha\beta^2\gamma$ has more a's, but no more b's. Thus the number of a's is at least $p + 1$ while the number of b's is fixed at $p + 1$. That means that $\alpha\beta^2\gamma$ is not a member of the language $\mathcal{L}_2$, which contradicts the Pumping Lemma's condition (3).

IV.5.15

(A) Five strings that are members are aaa, aaab, aaabb, aaabbb, and aaabbbb. This languages is regular, and a regular expression that describes it is aaab*.

(B) Five strings in this language are bbb, abbbb, aabbbbb, a^3b^6, and a^4b^7. This language is not regular.

To get a contradiction assume that the language is regular. The Pumping Lemma says that the language has a pumping length p. The string $\sigma = a^p b^{p+3}$ is a member of the language and has length greater than or equal to the pumping length. So it decomposes, $\sigma = \alpha\beta\gamma$, in a way that is subject to the three conditions. The first condition is that $|\alpha\beta| \leq p$ and so the two substrings α and β contain only a's. The second condition is that $|\beta| > 0$ and so the substring β consists of at least one a.

Finally, consider the list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... Condition (3) says that any element of that list is a member of the language. However, the list's first element has fewer a's than σ but the same number of b's, and therefore the number of a's is not three less than the number of b's. This is a contradiction.

(C) Five strings in this language are ε, aab, b, aaaabbbb, and abbbb. This language is regular. A regular expression that describes it is a*b*.

(D) Five strings in this language are b^{13}, ab^{14}, b^{14}, a^2b^{15}, and ab^{15}. This language is not regular.

Assume that it is regular, for contradiction. Then the language has a pumping length p. Let $\sigma = a^p b^{p+13}$. By the Pumping Lemma it decomposes into $\sigma = \alpha\beta\gamma$ satisfying the three conditions. A consequence of the first condition is that the first two substrings contain only of a's. A consequence of the second condition is that β consists of at least one a.

Consider $\alpha\gamma$, $\alpha\beta^2\gamma$, ... The string $\alpha\beta^2\gamma$ has at least one more a than does σ. Thus the number of bs' minus the number of a's is not more than 12, which means that it is not a member of the language. That contradicts the Pumping Lemma's third condition.

IV.5.16

(A) The two substrings of σ are: $\alpha\beta$ matches a* while γ matches a*bb ... b where there are $2p$-many b's.

(B) The substring $\alpha\beta$ matches a* while γ matches a*bb ... b with $p + 5$-many b's.

(C) The substring $\alpha\beta$ matches a* while γ matches a*ba ... a with $p + 1$-many a's.

(D) The substring $\alpha\beta$ matches a* and γ matches a*b ... b with p-many b's.

(E) The substring $\alpha\beta$ matches a* while γ matches a*bba ... a with p-many a's.

IV.5.17

(A) Five members of this language are ε, abab, babbab, aaaa, and aaabaaab.

(B) Suppose otherwise, that it is regular. By the Pumping Lemma the language has a pumping length p. Consider $\sigma = \mathtt{a}^p\mathtt{b}^\frown\mathtt{a}^p\mathtt{b}$. It is a member of the language whose length is greater than or equal to p and so it has a decomposition $\sigma = \alpha\beta\gamma$ that satisfies the three conditions. The first condition, $|\alpha\beta| \leq p$, gives that the two substrings contain only a's. The second condition, $|\beta| > 0$, gives that the second substring is nonempty and so consists of at least one a.

Now consider the list from the third condition, $\alpha\gamma$, $\alpha\beta^2\gamma$, ... Compare $\alpha\gamma$ to $\sigma = \alpha\beta\gamma$. Because the β is omitted in $\alpha\gamma$, before its first b it has fewer a's than σ has before its first b. But before the second b the two strings have the same number of a's. So $\alpha\gamma$ is not a member of the language, which contradicts the third condition.

(C) The argument in the prior part has a marker b that ends up in the substring γ. The attempted argument in this part has no marker, so we cannot compare the string σ to the strings in the list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... Your friend needs to pick a string with a marker of some kind.

IV.5.18 Five strings from the language are cb, acbb, aacbbb, aaacbbbb, and aaaacbbbbb.

To show that this language is not regular, assume for contradiction that it is. Then the language has a pumping length p. Consider $\sigma = \mathtt{a}^p\mathtt{c}\mathtt{b}^{p+1}$. Because this string is a member of the language and has length at least p, the Pumping Lemma says that it decomposes into $\sigma = \alpha\beta\gamma$, subject to the three conditions.

The first condition, $|\alpha\beta| \leq p$, implies that the two substrings α and β contain only a's. The second condition, $|\beta| > 0$, implies that β consists of at least one a.

Now, in the list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... consider the first one, $\alpha\gamma$. By the prior paragraph, compared with $\sigma = \alpha\beta\gamma$ this string has at least one fewer a but the same number of b's. So the number of a's is not one less than the number of b's, and this string is not in the language, which contradicts the third condition.

IV.5.19 Let $\mathcal{L} = \{\sigma \in \{\mathtt{a},\mathtt{b}\}^* \mid \text{the number of a's is greater than the number of b's}\}$. Assume for contradiction that it is regular. Then the Pumping Lemma applies and this language has a pumping length p. The string $\sigma = \mathtt{a}^{p+1}\mathtt{b}^p$ is an element of the language whose length is greater than or equal to p. So there is a decomposition $\sigma = \alpha\beta\gamma$ that satisfies the three conditions. Condition (1) gives that $|\alpha\beta| \leq p$ and so these two substrings contain only a's. Condition (2) gives that β is nonempty and thus it consists of at least one a.

The contradiction comes on considering condition (3)'s list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... The first entry in that list, $\alpha\gamma$, when compared with $\sigma = \alpha\beta\gamma$ will have at least one fewer a and the same number of b's. It is therefore not a member of the language $\mathcal{L}$, contradicting condition (3). This contradiction shows that $\mathcal{L}$ is not regular.

IV.5.20 The first is not regular while the second is regular.

(A) For contradiction, suppose that this language is regular. Then it has a pumping length p. Fix the string $\sigma = \mathtt{a}^{p^2}\mathtt{b}^p$. It is a member of the language whose length is greater than or equal to p so it decomposes into $\sigma = \alpha\beta\gamma$ in a way that satisfies the three conditions. Condition (1) implies that both α and β contain only a's. Condition (2) implies that β consists of at least one a.

Now consider condition (3)'s list, $\alpha\gamma$, $\alpha\beta^2\gamma$, ... Compared to $\sigma = \alpha\beta\gamma$, the entry $\alpha\gamma$ has at least one fewer a but the same number of b's. Therefore the number of a's is not the square of the number of b's, and so $\alpha\gamma$ is not in the language. That contradicts condition (3).

(B) A regular expression that describes this language is aaaa*bbbb*.

IV.5.21 The language is regular. This is a trick question: take $\alpha = \varepsilon$ to get that the set equals $\mathbb{B}^*$.

IV.5.22 Assume that it is regular and let the pumping length be p (for the parenthesized comment below it is

convenient to take $p > 1$). Take $\sigma = 1^{p!}$. This string is a member of $\mathcal{L}$ and has length at least p, so the Pumping Lemma says that it decomposes as $\sigma = \alpha\beta\gamma$, subject to the three conditions. The second condition implies that $|\beta| > 0$. Consider the list $\alpha\gamma$, $\alpha\beta^2\gamma$, $\alpha\beta^3\gamma$, ... After the first, the difference between the lengths of successive list elements is always $|\beta|$. But, as the hint notes, the difference between successive factorials is not fixed, instead it grows without bound. (Said another way, the difference in length between $\sigma = 1^{p!}$ and $\hat{\sigma} = 1^{(p+1)!}$ is $(p+1)! - p! = \big((p+1)-1\big) \cdot p! = p \cdot p!$. But the difference in length between $\sigma = \alpha\beta\gamma$ and $\alpha\beta^2\gamma$ is $|\beta|$, which is smaller than $p \cdot p!$ since β is a substring of a length $p!$ string, and we took $p > 1$.) So there is a list element that is not in the language, which contradicts the third condition. Thus the language is not regular.

IV.5.23 The first is regular, and is described by the regular expression `0*10*`. The second is not regular. To prove that, assume for contradiction that it is regular. Then it has a pumping length p. Let $\sigma = \texttt{0}^p\texttt{10}^p$. Then σ is a member of the language and has length at least p so the Pumping Lemma says it decomposes into $\sigma = \alpha\beta\gamma$ in a way that satisfies the three conditions. The first condition implies that the α and β substrings contain only `0`'s. The second condition says that β consists of at least one `0`.

Consider the strings on the third conditions's list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... By the prior paragraph the first, $\alpha\gamma$, has fewer `0`'s before the `1` than does σ, but the same number of `0`'s after the `1`. Hence it is not a member of the language, contradicting the Pumping Lemma's third condition.

IV.5.24 The language $\mathcal{L}_4 = \{\, \mathtt{a}^i\mathtt{b}^j\mathtt{c}^k \;\big|\; i,j,k \in \mathbb{N} \text{ and } i+j=k \text{ and } k<4 \,\}$ is finite and any finite language is regular.

As to the language of all sums, $\mathcal{L} = \{\, \mathtt{a}^i\mathtt{b}^j\mathtt{c}^k \;\big|\; i,j,k \in \mathbb{N} \text{ and } i+j=k \,\}$, suppose that it is regular and fix a pumping length p. Fix $\sigma = a^p b^0 c^p$. Because $|\sigma| \geq p$, it has a decomposition $\sigma = \alpha^\frown \beta^\frown \gamma$ subject to the three conditions of the Pumping Lemma. By the first condition the length $|\alpha\beta|$ is less than or equal to p, and so $\alpha^\frown \beta$ has only a's. By the second condition the string β is nonempty and so it consists of at least one a. The third condition says that $\alpha\gamma$ is also a member of the language, but that is a contradiction since this string has fewer a's than c's, and it has no b's because σ had no b's to start with, and thus $\alpha\gamma$ is not a member of $\mathcal{L}$.

IV.5.25

(A) It is regular because it is finite, $\{\,\texttt{00000}, \texttt{0000}1, \texttt{000}1\texttt{0}, \ldots 1111\texttt{0}\,\}$.

(B) The language $\mathcal{L} = \{\, \sigma \in \mathbb{B}^* \;\big|\; \text{the number of } \texttt{0}\text{'s minus the number of } 1\text{'s is five} \,\}$ is not regular. For, suppose otherwise. Let the language have pumping length p and let $\sigma = \texttt{0}^{p+5}1^p \in \mathcal{L}$. It has length at least p so decompose it into $\sigma = \alpha\beta\gamma$ subject to the three conditions. Condition (1), that $|\alpha\beta| \leq p$, implies that these two substrings have only `0`'s. Condition (2) says that β is not empty so it consists of at least one `0`.

Condition (3) says that every string on the list $\alpha\gamma$, $\alpha\beta^2\gamma$, ... is a member of $\mathcal{L}$, but that is not true here. By the prior paragraph the string $\alpha\gamma$ has at least one fewer `0` than the string $\sigma = \alpha\beta\gamma$, but it has the same number of 1's. It is therefore not an element of $\mathcal{L}$. By this contradiction we have shown that $\mathcal{L}$ is not regular.

IV.5.26 Call this language $\mathcal{L} = \{\, \texttt{0}^m1^n \in \mathbb{B}^* \;\big|\; m \neq n \,\}$. Where $\hat{\mathcal{L}} = \{\, \texttt{0}^m1^m \in \mathbb{B}^* \;\big|\; m \in \mathbb{N} \,\}$, we have $\hat{\mathcal{L}} = \mathbb{B}^* - \mathcal{L}$. By Theorem 4.5, regular languages are closed under set difference. Clearly $\mathbb{B}^*$ is regular — it is described by the regular expression `(0|1)*` — so if $\mathcal{L}$ were regular then it would imply that $\hat{\mathcal{L}}$ is regular also. But $\hat{\mathcal{L}}$ is not regular, since it is the language of balanced parentheses, Example 5.2, with `0`'s substituted for open parentheses and 1's for closed parentheses.

IV.5.27 Following the hint, for a string to be a member of the language $\mathcal{L}$, it must have the same number of transitions in as transitions out. For instance, the string aaa b aaa bbb is not a member of $\mathcal{L}$ because there are two transitions into a sequence of b's from a sequence of a's, but only one transition out of a sequence of b's and back to a sequence of a's.

In short, $\mathcal{L}$ is the set of strings whose last character equals its first character. Thus the regular expression is `(a(a|b)*a)|(b(a|b)*b)`.

IV.5.28 The proof of the Pumping Lemma uses the Pigeonhole Principle to show that there is at least one loop, and it then reasons with that loop. It is unaffected by there being a later loop.

IV.5.29 Assume for contradiction that this language is regular and fix a pumping length p. Consider condition (3)'s list.

$$\alpha\gamma, \alpha\beta^2\gamma, \alpha\beta^3\gamma, \ldots\ \alpha\beta^n\gamma, \ldots \qquad n \geq 2$$

All of the gaps between strings on this list (except for the first gap) have the same size, $|\alpha\beta^{n+1}\gamma| - |\alpha\beta^n\gamma| = |\beta|$.

So for $n \geq 2$ the length of $\alpha\beta^n\gamma$ is $|\alpha\beta^2\gamma| + (n-2)\cdot|\beta|$. Take n to be $|\alpha\beta^2\gamma| + 2$ and note that the length of $\alpha\beta^n\gamma$ is not prime. Thus not every string in condition (3)'s list is a member of the language, which is the desired contradiction.

IV.5.30

(A) The number of c's is the product of the number of a's and the number of b's. Five such strings are abc, abbcc, abbbccc, aabcc, and aabbcccc.

(B) For contradiction assume that this language, $\mathcal{L}$, is regular. The Pumping Lemma says that $\mathcal{L}$ has a pumping length, p. Consider $\sigma = \mathtt{a}^p\mathtt{b}^p\mathtt{c}^{(p^2)}$. That string has more than p characters so it decomposes as $\sigma = \alpha\beta\gamma$, subject to the three conditions. Condition (1) is that $|\alpha\beta| \leq p$ and so both substrings α and β are composed entirely of a's. Condition (2) is that β is not the empty string and so β consists of at least one a.

Condition (3) is that the strings in the list $\alpha\gamma, \alpha\beta^2\gamma, \alpha\beta^3\gamma, \ldots$ are also members of the language. We will get the contradiction from the first one, $\alpha\gamma$. Compared to $\sigma = \alpha\beta\gamma$, in the first one the β is gone. So the number of a's in $\alpha\gamma$ is less than the number in $\sigma = \alpha\beta\gamma$, But the number of b's and the number of c's is the same. Thus in $\alpha\gamma$ the number of a's times the number of b's does not equal the number of c's.

IV.5.31 Recall the string notation defined in Appendix A, that in the string τ, character j-th is $\tau[j]$. The indexing is zero-offset, meaning that the initial character is $\tau[0]$. And, $\tau[j{:}]$ is the substring starting with character j and going to the end. Thus, for $\tau = \mathtt{012345}$ we have $\tau[0] = \mathtt{0}$ and $\tau[1{:}] = \mathtt{12345}$.

(A) The regular expression `(b|c)*|((aaa*)(b|c)*)` describes this language.

(B) Assume that $\mathcal{L}_0$ is regular and fix some pumping length $p > 1$ (the footnote on the Pumping Lemma allows us to set the pumping length to be at least two). Consider $\sigma = \mathtt{a}\mathtt{b}^p\mathtt{c}^p$. It decomposes into $\sigma = \alpha\beta\gamma$ subject to the conditions. The first condition gives that α and β contain only a's and b's. The second condition gives that β consists of at least one character, necessarily not a c. The third condition then gives a contradiction on considering $\alpha\gamma$, which contains the same number of c's as σ but either no a or fewer b's.

(C) If $\mathcal{L}$ were regular then the set difference $\mathcal{L} - \mathcal{L}_1 = \mathcal{L}_0$ would be regular, but the prior item shows that it is not.

(D) The Pumping Lemma only puts a restriction on strings of length at least the pumping length $p = 3$. For a string of the form $\sigma = \mathtt{a}\mathtt{b}^n\mathtt{c}^n$, the condition $|\sigma| \geq p$ implies that $n \geq 1$. With that, we will show that one decomposition satisfying the three conditions is $\alpha = \varepsilon$, and $\beta = \mathtt{a}$, and $\gamma = \mathtt{b}^n\mathtt{c}^n$. The first two conditions, that $|\alpha\beta| \leq 3 = p$ and that β is not the empty string, are clear. To finish we must verify that every string on the list $\alpha\gamma, \alpha\beta^2\gamma, \ldots$ is an element of $\mathcal{L}$. The first is $\alpha\gamma = \mathtt{b}^n\mathtt{c}^n$, which is an element of $\mathcal{L}_1$, where $k = 0$ in its definition. Next is $\alpha\beta^2\gamma = \mathtt{a}\mathtt{a}\mathtt{b}^n\mathtt{c}^n$, which is an element of $\mathcal{L}_1$ where $k = 2$. The rest, $\alpha\beta^i\gamma = \mathtt{a}^i\mathtt{b}^n\mathtt{c}^n$ with $i > 2$, are similarly elements of $\mathcal{L}_1$ where $k = i$.

(E) For a string of the form $\sigma = \tau \in \{\mathtt{b}, \mathtt{c}\}^*$ to satisfy $|\sigma| \geq p = 3$ we need $|\tau| \geq 3$. Here one such decomposition is $\alpha = \varepsilon$, and $\beta = \tau[0] = x$ where $x = \mathtt{b}$ or $x = \mathtt{c}$, and $\gamma = \tau[1{:}]$. Then the first two conditions are satisfied: $|\alpha\beta| \leq p$ and $\beta \neq \varepsilon$. To finish we check that every string on the list $\alpha\gamma, \alpha\beta^2\gamma, \ldots$ is an element of $\mathcal{L}$. The first is $\alpha\gamma = \tau[1{:}]$ which is an element of $\mathcal{L}_1$ where $k = 0$. A string from the rest of the list, $\alpha\beta^i\gamma = x^i{}^\frown\tau[1{:}]$ with $i \geq 2$, is similarly an element of $\mathcal{L}_1$ with $k = 0$.

(F) Consider strings of the form $\sigma = \mathtt{a}^k{}^\frown\tau$ where $k \geq 2$ and $\tau \in \{\mathtt{b}, \mathtt{c}\}^*$. The Pumping Lemma requires only that we consider strings σ with $|\sigma| \geq p = 3$. One decomposition that works is $\alpha = \mathtt{a}\mathtt{a}$, and $\beta = x$ where x is the third character in the string, that is, $x = \sigma[2]$, and $\gamma = \sigma[3{:}]$. Observe that if $x = \mathtt{a}$ then γ matches the regular expression `a*(b|c)*`, while if $x \neq \mathtt{a}$ then γ matches `(b|c)*`.

This decomposition satisfies first two conditions of the Pumping Lemma since clearly $|\alpha\beta| \leq p$ and $\beta \neq \varepsilon$. What remains is to check the third condition, that every string on the list $\alpha\gamma, \alpha\beta^2\gamma, \ldots$ is an element of $\mathcal{L}$.

The first list element is $\alpha\gamma$. Suppose that β's single character x is a. We have already noted that γ matches the regular expression `a*(b|c)*`, and so the concatenation $\alpha\gamma$ matches `aaa*(b|c)*`, and therefore is an element of $\mathcal{L}_1$ where $k \geq 2$. Suppose that x is not a. We have noted that in this case γ matches `(b|c)*` and therefore the concatenation $\alpha\gamma$ matches `aa(b|c)*`, making it an element of $\mathcal{L}_1$ where $k = 2$.

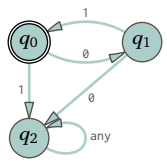
Strings on the rest of the list have the form $\alpha\beta^i\gamma$ with $i \geq 2$. If β's character x is a then $\alpha\beta^i\gamma$ matches the regular expression below on the left. If β's character x is not a then $\alpha\beta^i\gamma$ matches the one on the right.

$$\mathtt{aa}\underbrace{(\mathtt{aa}\ldots\mathtt{a})}_{i\text{-many}}\mathtt{a*(b|c)*} \qquad \mathtt{aa}\underbrace{(x x \ldots x)}_{i\text{-many}}\mathtt{(b|c)*}$$

In either case the string is an element of $\mathcal{L}_1$.

IV.5.32

(A) This machine works.



It is minimal because from q_0 an input of `1` must go to an error state, while an input of `0` must go to a different, but also non-accepting, state. Since q_0 is accepting, that's three states.

(B) The prior item shows that the minimal Finite State machine recognizing $\mathcal{L}$ has three states. So by the proof for the Pumping Lemma, any word of the language that has at least three characters can be pumped, that is, can be decomposed in a way that satisfies the lemma's conditions. The only length 2 word, $\sigma = \texttt{01}$, can be pumped with $\alpha = \varepsilon$, $\beta = \texttt{01}$, and $\gamma = \varepsilon$. So the minimum pumping length is at most 2. As to $p = 1$, the language contains no word of length 1. Therefore the minimum pumping length for the language is 1, because if $\sigma \in \mathcal{L}$ and $|\sigma| \geq 1$ then automatically $|\sigma| \geq 2$, and σ can be pumped.

IV.6.7 The machine with these two instructions just keeps popping $\perp$ off the stack and then pushing it back on again.

Inst	*Input*	*Output*
0	$q_0, \varepsilon, \perp$	q_1, '$\perp$'
1	$q_1, \varepsilon, \perp$	q_0, '$\perp$'

This machine has the stack grow without bound.

Inst	*Input*	*Output*
0	$q_0, \varepsilon, \perp$	q_1, '`g0` $\perp$'
1	$q_1, \varepsilon, \texttt{g0}$	q_0, '`g0 g0`'
2	$q_0, \varepsilon, \texttt{g0}$	q_1, '`g0 g0`'

IV.6.8 Each configuration is a triple of present state, remaining tape tokens, and stack.

(A) We have this.

$$\langle q_0, \texttt{[][]B}, \perp\rangle \vdash \langle q_0, \texttt{][]B}, \texttt{g0}\ \perp\rangle \vdash \langle q_0, \texttt{[]B}, \perp\rangle \vdash \langle q_0, \texttt{]B}, \texttt{g0}\ \perp\rangle \vdash \langle q_0, \texttt{B}, \perp\rangle \vdash \langle q_1, \text{`'}, \perp\rangle$$

The last configuration has an empty tape and the state q_1, so the machine accepts the initial string.

(B) This computation is quite different than the prior one.

$$\langle q_0, \texttt{][][B}, \perp\rangle \vdash \langle q_0, \texttt{[][B}, \text{`'}\rangle$$

At this point the machine has an empty stack. It can't start the next step because each step starts by popping the top character off the stack, but there is no such character. The computation stops, with the input rejected.

IV.6.9 Here is a maximal path through the comptuation tree accepting the input `bacab`.

Step	*Machine*		*Instruction*
0	b a c a b B q_0	⊥	1
1	a c a b B q_0	g1 ⊥	5
2	c a b B q_0	g0 g1 ⊥	8
3	a b B q_1	g0 g1 ⊥	10
4	b B q_1	g1 ⊥	11
5	B q_1	⊥	12
6	 q_3	⊥	

In state q_0, when this machine sees a on the tape then it pushes g0 onto the stack, and for b it pushes g1. Reading the c switches the machine to state q_1. In this phase, if it is reading a and the character on top of the stack is g0 then the machine consumes the a, pops the g0, and goes on. The same happens with b and g1. (But if the tape has a while the stack is g1, or if the tape has b while the stack is g0, then the computation wouild dead-end.) Finally, the machine reaches the end of the input string, including the marker B, in the accepting state q_2.

IV.6.10

(A) When this machine consumes an a it pushes two g0's onto the stack. When it sees the middle marker c it switches to popping. The only accepting state is q_2.

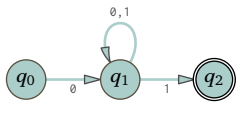
Inst	*Input*	*Output*
0	q_0, a, ⊥	q_0, 'g0 g0 ⊥'
1	q_0, c, ⊥	q_1, '⊥'
2	q_0, a, g0	q_0, 'g0 g0 g0'
3	q_0, c, g0	q_1, 'g0'
4	q_1, b, g0	q_1, ''
5	q_1, B, ⊥	q_2, ⊥

(B) This adaptation of the prior item's machine just throws away the first a. The only accepting state is q_3.

Inst	*Input*	*Output*
0	q_0, a, ⊥	q_1, '⊥'
1	q_1, a, ⊥	q_1, 'g0 g0 ⊥'
2	q_1, c, ⊥	q_2, '⊥'
3	q_1, a, g0	q_1, 'g0 g0 g0'
4	q_1, c, g0	q_2, 'g0'
5	q_2, b, g0	q_2, ''
6	q_2, B, ⊥	q_3, ⊥

IV.6.11 This is a regular language so we don't need the stack. This Pushdown machine implements the shown nondeterministic Finite State machine; its accepting state is q_2.

Inst	*Input*	*Output*
0	q_0, 0, ⊥	q_1, '⊥'
1	q_1, 0, ⊥	q_1, '⊥'
2	q_1, 1, ⊥	q_1, '⊥'
3	q_1, 1, ⊥	q_2, '⊥'



IV.6.12 First this machine reads a's and pushes g0's. When it guesses that the time is right, it transitions to reading a's and popping g0's. The only accepting state is q_2.

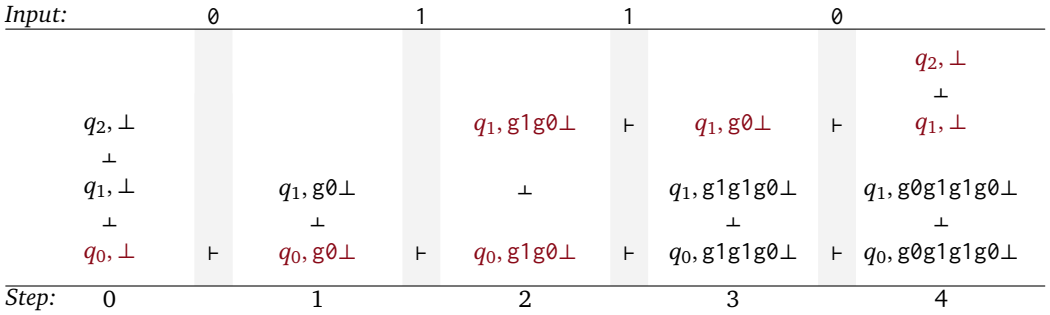
Inst	*Input*	*Output*
0	q_0, B, $\bot$	q_2, '$\bot$'
1	q_0, a, $\bot$	q_0, 'g0 $\bot$'
2	q_0, a, g0	q_0, 'g0 g0'
3	q_0, ε, g0	q_1, 'g0 $\bot$'
2	q_1, a, g0	q_1, ''
3	q_1, B, $\bot$	q_2, '$\bot$'

IV.6.13 This machine pushes a g0 onto the stack for every initial a on the tape. Then when it sees a b it switches to popping two g0's off for every input. Its accepting state is q_4.

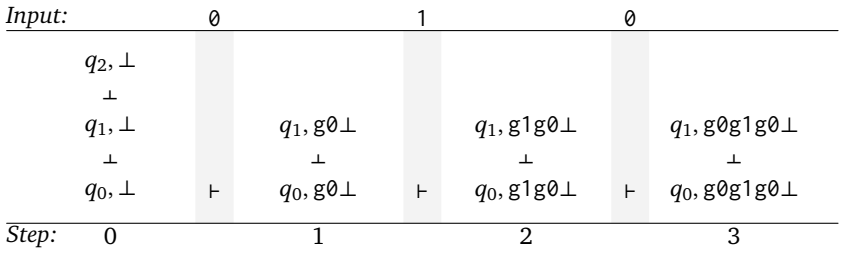
Inst	*Input*	*Output*
2	q_0, B, $\bot$	q_4, '$\bot$'
0	q_0, a, $\bot$	q_1, 'g0 $\bot$'
1	q_1, a, g0	q_1, 'g0 g0'
3	q_1, b, g0	q_3, ''
4	q_2, b, g0	q_3, ''
5	q_3, b, g0	q_2, ''
6	q_2, B, $\bot$	q_4, '$\bot$'

IV.6.14

(A)



(B)



IV.6.15 S $\longrightarrow$ oSo | 1S1 | ε

IV.6.16 The language is $\mathcal{L}_{\text{ELP}} = \{\, \sigma\sigma^{\mathsf{R}} \;\big|\; \sigma \in \mathbb{B}^* \,\}$.

For contradiction, assume that $\mathcal{L}_{\text{ELP}}$ is regular. Then the Pumping Lemma says that it has a pumping length, which we can denote p. Consider the string $\sigma = 0^p 1 1 0^p$.

It is an element of $\mathcal{L}$ of length greater than p. Therefore it decomposes into three substrings $\sigma = \alpha^\frown\beta^\frown\gamma$ that satisfy the conditions. Condition (1) is that the length of the prefix $\alpha^\frown\beta$ is less than or equal to p. Because the first p-many characters of σ are all 0's, this implies that both α and β contain only 0's. Condition (2) is that β is not empty so it consists of at least one 0.

Consider $\alpha\gamma$. Compared with $\sigma = \alpha\beta\gamma$, this string is missing the β, which means that before $\alpha\gamma$'s substring 11 it has fewer 0's than does σ. But it has the same number of 0's after the 11 substring, and so it is not a member of $\mathcal{L}_{\text{ELP}}$. That contradicts condition (3) of the lemma.

IV.6.17

(A) A Pushdown machine is a finite set of five tuples. Each of the five components comes from a countable set. So in total the number of Pushdown machines is countable.

(B) Every machine accepts one language. So the number of languages is at most as large as the number of machines.

(C) There are uncountably many languages.

IV.6.18 We can simulate a Pushdown machine with a Turing machine.

IV.A.21

(A) `.*abc*.*`

(B) `.*abc+.*`

(C) `.*abc2.*`

(D) `.*abc2,5.*`

(E) `.*abc2,.*`

(F) `.*a(b|c).*` or `.*a[bc].*`

IV.A.22

(A) `.*abe.*` Note that in practice you often can just write abe; see your language's documentation.

(B) `[a-zA-Z0-9]*`

(C) `[^]`

IV.A.23

(A) `(a|e|i|o|u).*bc.*`

(B) `(a|e|i|o|u)(bc|.*bc).*`

(C) `.*a.*e.*i.*o.*u.*`

(D) `(.*a.*e.*i.*o.*u.*)|(.*a.*e.*i.*u.*o.*)|. . . |(.*u.*o.*i.*e.*a.*)` There are 5! = 120 clauses. You might use a program to write this regex.

IV.A.24 `.*([.*{|{.*[).*`

IV.A.25 `\d{10}`

IV.A.26

(A) `(\d{3}|\(\d{3}\)) \d{3}-\d{4}`

(B) We must factor out the space as well as the area code: `(\d{3} |\(\d{3}\) )?\d{3}-\d{4}`.

IV.A.28 `\d{2}:\d{2}:\d{2}(.\d +)|\d{2}(:\d{2})?`

IV.A.29 One is `[-\+]?(\d)+\.?|\d)*\.\d+)`

IV.A.30

(A) `4(\d15|\d12)`

(B) `(5[1-5]\d2|222[1-9]|22[3-9]\d|2[3-6]\d2|27[01]\d|2720)\d12`

(C) `3[47][0-9]13`

IV.A.31

(A) We can do the six as alternatives.

```
[A-Z]\d2 \d[A-Z]2|[A-Z]\d \d[A-Z]2|[A-Z]\d[A-Z] \d[A-Z]2|[A-Z]2\d2 \d[A-Z]2|[A-Z]2\d
                          \d[A-Z]2|[A-Z]2\d[A-Z] \d[A-Z]2
```

(B) We can factor out the ending of `[A-Z]2`.

```
[A-Z]\d2|[A-Z]\d|[A-Z]\d[A-Z]|[A-Z]2\d2|[A-Z]2\d|[A-Z]2\d[A-Z]) \d[A-Z]2
```

IV.A.32 A suitable regex is `textasciicircum g.n.{3}i.$` and a match is 'gynaecia', meaning the aggregate of carpels or pistils in a flower.

IV.A.34 `^M0,4(CM|CD|D?C0,3)(XC|XL|L?X0,3)(IX|IV|V?I0,3)$`

IV.A.35 Assume it is a regular language and let its pumping length be p. Consider $\sigma = \mathtt{a}^p\mathtt{ba}^p\mathtt{b}$ It is an element of $\mathcal{L}$ whose length is greater than p so By the Pumping Lemma it decomposes as $\sigma = \alpha^\frown\beta^\frown\gamma$ subject to the

three conditions. By the first condition the length of $\alpha\frown\beta$ is less than p, and so these two substrings consist solely of a's. The second condition is that β is not the empty string. The third condition says that the strings $\alpha\gamma$, $\alpha\beta^2\gamma$, ... are all members of $\mathcal{L}$. Note that the first is not a square because in omitting β it omits at least one a from the prefix but not from the suffix. That's a contradiction.

IV.A.36

(A) We take the alphabet to be $\Sigma = \mathbb{B}$. Assume that it is regular and let its pumping length be p. Consider $\sigma = \mathtt{0}^p\mathtt{1}\mathtt{0}^p$, which is a member of the language. By the Pumping Lemma it therefore decomposes into $\sigma = \alpha\beta\gamma$ subject to the three conditions. By the first and second conditions $\alpha\beta$ consists solely of 0's, and is therefore part of the prefix before the 1, and β is nonempty. By the third condition the string $\alpha\gamma$ is also a member of $\mathcal{L}$. But this is a contradiction because the omission of β means that $\alpha\gamma$ has fewer 0's before the 1 than after.

(B) `(a*)b\1`

IV.A.37

(A) `.*r`

(B) `.*i.*t.*`

(C) `.*foo.*`

IV.A

(A) HELP

(B)

	(A|B|C)\1	(AB|OE|SK)
.*M?O.*	B	O
(AN|FE|BE)	B	E

IV.B.18 Every string in the language $\mathcal{L} = \{\,\mathtt{ab}^n \;\big|\; n \in \mathbb{N}\,\} = \{\,\mathtt{a}, \mathtt{ab}, \mathtt{abb}, \ldots\,\}$ is a member of $\mathcal{E}_{\mathcal{L},1}$. Of the other strings, the only member of $\mathcal{E}_{\mathcal{L},0}$ is ε and all other strings are in $\mathcal{E}_{\mathcal{L},2}$.

(A) $\mathtt{bba} \in \mathcal{E}_{\mathcal{L},2}$

(B) $\mathtt{abb} \in \mathcal{E}_{\mathcal{L},1}$

(C) $\mathtt{babab} \in \mathcal{E}_{\mathcal{L},2}$

(D) $\mathtt{abbbb} \in \mathcal{E}_{\mathcal{L},1}$

IV.B.19

(A) The starting string's class is $\mathtt{bba} \in \mathcal{E}_{\mathcal{L},2}$. After appending to get $\mathtt{bba}\frown\mathtt{a} = \mathtt{bbaa}$ we have $\mathtt{bbaa} \in \mathcal{E}_{\mathcal{L},2}$. This fits with the lower-left entry in the Δ table given in Example B.14 (that is, with the bottom entry in the column headed 'a').

(B) The input class is $\varepsilon \in \mathcal{E}_{\mathcal{L},0}$. Appending gives $\varepsilon\frown\mathtt{a} = \mathtt{a} \in \mathcal{E}_{\mathcal{L},1}$. This fits the upper-left entry in the Δ table.

(C) The starting class is $\mathtt{abbb} \in \mathcal{E}_{\mathcal{L},1}$. Appending results in $\mathtt{abbba} \in \mathcal{E}_{\mathcal{L},2}$, which matches the middle-left entry in the transition table.

(D) The starting class is $\mathtt{aa} \in \mathcal{E}_{\mathcal{L},2}$. Appending results in $\mathtt{aaa} \in \mathcal{E}_{\mathcal{L},2}$, which fits the table's bottom-left entry.

IV.B.20 We will follow the intuition given in the example. Fix some $i \in \{0, 1, 2, 3\}$. We first show that if $\sigma_0, \sigma_1 \in \mathcal{E}_{\mathcal{L},i}$ then they are indistinguishable. Consider a bitstring τ and let $k \in \mathbb{N}$ be the number of 1's in τ. Then the number of 1's in $\sigma_0\frown\tau$ is the remainder on division by four of $i + k$, and that is the same as the number of 1's in $\sigma_1\frown\tau$. Thus $\sigma_0\frown\tau \in \mathcal{L}$ if and only if $\sigma_1\frown\tau \in \mathcal{L}$.

We finish by showing that if $\sigma_0 \in \mathcal{E}_{\mathcal{L},i}$ and $\sigma_1 \in \mathcal{E}_{\mathcal{L},j}$ with $j \neq i$ then they are distinguishable. If $\tau = \mathtt{1}^{4-i}$ then because $i \neq j$ we have $\sigma_0 \in \mathcal{L}$ but $\sigma_1 \notin \mathcal{L}$, so they are distinguishable.

IV.B.21

(A) Appending to the three strings gives $\sigma_0\frown\mathtt{0} = \mathtt{000}$, $\sigma_1\frown\mathtt{0} = \mathtt{110110}$, and $\sigma_2\frown\mathtt{0} = \mathtt{001}^8\mathtt{0}$. All three are members of $\mathcal{E}_{\mathcal{L},0}$. Looking at the machine's transition table, that is indeed the result of starting with the class $\mathcal{E}_{\mathcal{L},0}$ and transitioning on input 0.

(B) Appending gives $\sigma_0\frown\mathtt{1} = \mathtt{001}$, $\sigma_1\frown\mathtt{1} = \mathtt{110111}$, and $\sigma_2\frown\mathtt{1} = \mathtt{001}^8\mathtt{1}$. All three are members of $\mathcal{E}_{\mathcal{L},1}$. The machine's transition table shows this as the result of starting with the class $\mathcal{E}_{\mathcal{L},0}$ and transitioning on input 1.

(C) Of course, many triples of strings are possible but for illustration we can give $\sigma_0 = \mathtt{100}$, and $\sigma_1 = \mathtt{010}$, and

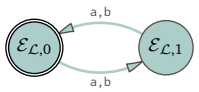
$\sigma_2 = 111101$.

Concatenating 0 gives $\sigma_0 \frown 0 = 1000$, and $\sigma_1 \frown 0 = 0100$, and $\sigma_2 \frown 0 = 1111010$. All three are members of $\mathcal{E}_{\mathcal{L},1}$. The machine's transition table shows that $\mathcal{E}_{\mathcal{L},2}$ is indeed the result of starting with the class $\mathcal{E}_{\mathcal{L},1}$ and transitioning on input 0.

As to concatenating 1, we get $\sigma_0 \frown 1 = 1001 \in \mathcal{E}_{\mathcal{L},2}$, $\sigma_1 \frown 1 = 0101 \in \mathcal{E}_{\mathcal{L},2}$, and $\sigma_2 \frown 1 = 1111011 \in \mathcal{E}_{\mathcal{L},2}$. The machine's transition table agrees, showing that the result of starting with the class $\mathcal{E}_{\mathcal{L},1}$ and transitioning on input 1 is $\mathcal{E}_{\mathcal{L},2}$.

IV.B.22 That example gives the classes as $\mathcal{E}_{\mathcal{L},0} = \{\sigma \mid |\sigma| \text{ is even}\}$ and $\mathcal{E}_{\mathcal{L},1} = \{\sigma \mid |\sigma| \text{ is odd}\}$. We can pick any string from a class to represent that class. As a representative of $\mathcal{E}_{\mathcal{L},0}$ we can use the length zero string ε. As a representative of $\mathcal{E}_{\mathcal{L},1}$ we can use the length one string a. The only accepting state is $\mathcal{E}_{\mathcal{L},0}$ because it contains all of the strings from $\mathcal{L}$. It is also the start state because it contains the empty string.

Δ	a	b
$+\,\varepsilon \in \mathcal{E}_{\mathcal{L},0}$	$a \in \mathcal{E}_{\mathcal{L},1}$	$b \in \mathcal{E}_{\mathcal{L},1}$
$a \in \mathcal{E}_{\mathcal{L},1}$	$aa \in \mathcal{E}_{\mathcal{L},0}$	$ab \in \mathcal{E}_{\mathcal{L},0}$



IV.B.23 The classes are $\mathcal{E}_{\mathcal{L},0} = \{\varepsilon\}$, and $\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \Sigma^* \mid |\sigma| = 1\} = \{a, b\}$, and $\mathcal{E}_{\mathcal{L},2} = \{\sigma \in \Sigma^* \mid |\sigma| = 2\} = \{aa, ab, ba, bb\}$, and $\mathcal{E}_{\mathcal{L},3} = \{\sigma \in \Sigma^* \mid |\sigma| = 3\}$, along with $\mathcal{E}_{\mathcal{L},4} = \{\sigma \mid |\sigma| \geq 4\}$. To determine the transitions, from each class we can pick any string member. As a representative of $\mathcal{E}_{\mathcal{L},0}$ we must use ε. To represent $\mathcal{E}_{\mathcal{L},1}$ we can use the length one string a. To represent $\mathcal{E}_{\mathcal{L},2}$ we can pick aa, to represent $\mathcal{E}_{\mathcal{L},3}$ we can use aaa, and to represent $\mathcal{E}_{\mathcal{L},4}$ we can take aaaa.

Δ	a	b
$\varepsilon \in \mathcal{E}_{\mathcal{L},0}$	$a \in \mathcal{E}_{\mathcal{L},1}$	$b \in \mathcal{E}_{\mathcal{L},1}$
$a \in \mathcal{E}_{\mathcal{L},1}$	$aa \in \mathcal{E}_{\mathcal{L},2}$	$ab \in \mathcal{E}_{\mathcal{L},2}$
$aa \in \mathcal{E}_{\mathcal{L},2}$	$aaa \in \mathcal{E}_{\mathcal{L},3}$	$aab \in \mathcal{E}_{\mathcal{L},3}$
$+\,aaa \in \mathcal{E}_{\mathcal{L},3}$	$aaaa \in \mathcal{E}_{\mathcal{L},4}$	$aaab \in \mathcal{E}_{\mathcal{L},4}$
$aaaa \in \mathcal{E}_{\mathcal{L},4}$	$aaaaa \in \mathcal{E}_{\mathcal{L},4}$	$aaaab \in \mathcal{E}_{\mathcal{L},4}$

Because it contains all of the strings from $\mathcal{L}$, the sole accepting state is $\mathcal{E}_{\mathcal{L},3}$. The start state is the class containing ε, namely $\mathcal{E}_{\mathcal{L},0}$.

IV.B.24 We first verify that all of the strings in $\mathcal{E}_{\mathcal{L},0} = \{\sigma \in \{a, b\}^* \mid \sigma \text{ ends in } b\}$ are $\sim_{\mathcal{L}}$ equivalent. So fix $\sigma_0, \sigma_1 \in \mathcal{E}_{\mathcal{L},0}$. Take $\tau \in \{a, b\}^*$ and consider $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$. There are two cases. If $\tau \neq \varepsilon$ then whether or not $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are both elements of $\mathcal{L}$ is determined only by the final character of τ, so it doesn't matter what σ_0 and σ_1 are, they play no role. If $\tau = \varepsilon$ then $\sigma_0 \frown \tau = \sigma_0$ and $\sigma_1 \frown \tau = \sigma_1$ are both elements of $\mathcal{L}$. In either case $\sigma_0 \frown \tau \in \mathcal{L}$ if and only if $\sigma_1 \frown \tau \in \mathcal{L}$.

The same argument works for $\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \{a, b\}^* \mid \sigma \text{ ends in } a\}$.

IV.B.25 Observe that where $\sigma \in \{a, b\}^*$, if τ is nonempty then $\sigma \frown \tau$ is in the language if and only if τ ends in a.

(A) The observation implies that no pair $\sigma_0, \sigma_1 \in \mathcal{L}$ has a distinguishing extension, since $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are elements of $\mathcal{L}$ just if either τ is empty or τ ends in a. In addition, no string $\sigma \in \mathcal{L}$ is equivalent to a string $\alpha \notin \mathcal{L}$ since σ ends in a and α does not, and therefore a distinguishing extension is ε.

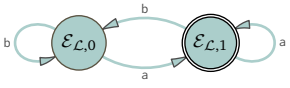
(B) The observation also implies, by the same reasoning, that the pair $\sigma_0, \sigma_1 \in \mathcal{L}^c$ does not have a nonempty distinguishing extension. And, $\tau = \varepsilon$ is also not a distinguishing extension. (We do not need to establish that no string in $\mathcal{L}^c$ is equivalent to a string in $\mathcal{L}$ since we did that in the prior item.)

(C) The initial state is the class containing the empty string. In this machine, the empty string is an element of $\mathcal{E}_{\mathcal{L},0}$.

(D) A class is accepting if it contains a string from $\mathcal{L}$. In this machine, that is only the class $\mathcal{E}_{\mathcal{L},1}$.

(E) As class representatives we can choose $\varepsilon \in \mathcal{E}_{\mathcal{L},0}$ and the length one string $a \in \mathcal{E}_{\mathcal{L},1}$.

Δ	a	b
$\varepsilon \in \mathcal{E}_{\mathcal{L},0}$	$a \in \mathcal{E}_{\mathcal{L},1}$	$b \in \mathcal{E}_{\mathcal{L},0}$
$+\,a \in \mathcal{E}_{\mathcal{L},1}$	$aa \in \mathcal{E}_{\mathcal{L},1}$	$ab \in \mathcal{E}_{\mathcal{L},0}$



The accepted strings are those that end in a, namely a, abba, and bba.

IV.B.26 We follow the discussion before Example B.17, that where the machine is minimal the $\sim_{\mathcal{L}}$ classes are the $\sim_{\mathcal{M}}$ classes. As to finding the $\sim_{\mathcal{M}}$ classes, these four implications are clear.

$\hat{\Delta}(\sigma) = q_0 \iff \sigma = \varepsilon$

$\hat{\Delta}(\sigma) = q_1 \iff \sigma = \mathtt{a}$

$\hat{\Delta}(\sigma) = q_2 \iff \sigma$ matches the regular expression `aab*`

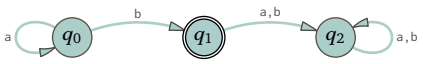
$\hat{\Delta}(\sigma) = q_3 \iff \sigma$ is otherwise (specifically, σ matches `b|ab|(aab*a(a|b)*)`)

Consequently, these are the four $\sim_{\mathcal{L}}$ classes.

Associated with	*Class*
q_0	$\mathcal{E}_{\mathcal{L},0} = \{\varepsilon\}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \{\mathtt{a}\}$
q_2	$\mathcal{E}_{\mathcal{L},2} = \mathcal{L}$
q_3	$\mathcal{E}_{\mathcal{L},3} = \{\sigma \in \Sigma^* \mid \sigma \notin (\mathcal{E}_{\mathcal{L},0} \cup \mathcal{E}_{\mathcal{L},1} \cup \mathcal{E}_{\mathcal{L},2})\}$

The class $\mathcal{E}_{\mathcal{L},2}$ contains strings from $\mathcal{L}$ so it is a final state. The class $\mathcal{E}_{\mathcal{L},0}$ contains the empty string so it is the initial state.

IV.B.27 We follow the discussion before Example B.17, that where the machine is minimal the $\sim_{\mathcal{L}}$ classes are the $\sim_{\mathcal{M}}$ classes. Here is the natural Finite State machine to recognize this language.



It is clearly minimal, so we find the the $\sim_{\mathcal{M}}$ classes. These implications are straightforward.

$\hat{\Delta}(\sigma) = q_0 \iff \sigma$ matches the regular expression `a*`

$\hat{\Delta}(\sigma) = q_1 \iff \sigma$ matches the regular expression `a*b`

$\hat{\Delta}(\sigma) = q_2 \iff \sigma$ is otherwise (specifically, it matches `a*b(a|b)(a|b)*`)

These are the $\sim_{\mathcal{L}}$ classes.

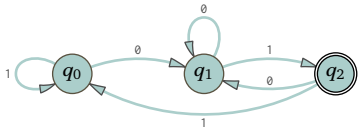
Associated with	*Class*
q_0	$\mathcal{E}_{\mathcal{L},0} = \{\sigma \in \Sigma^* \mid \sigma = \mathtt{a}^n \text{ for } n \in \mathbb{N}\}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \mathcal{L} = \{\sigma \in \Sigma^* \mid \sigma = \mathtt{a}^n\mathtt{b} \text{ for } n \in \mathbb{N}\}$
q_2	$\mathcal{E}_{\mathcal{L},2} = \{\sigma \in \Sigma^* \mid \sigma = \mathtt{a}^n\mathtt{b}^\frown\beta \text{ for } n \in \mathbb{N} \text{ and } \beta \neq \varepsilon\}$

The class $\mathcal{E}_{\mathcal{L},1}$ contains strings from $\mathcal{L}$ so it is a final state. The class $\mathcal{E}_{\mathcal{L},0}$ contains the empty string so it is the initial state.

Comment: while the Finite State machine given certainly appears minimal, strictly speaking we have not shown it to be minimal. We could show that using Extra **??**.

IV.B.28

(A) This is a regular language and the natural Finite State machine to recognize it is below.



We take this machine to be minimal. (We could verify that using Extra **??**.) We follow the discussion before Example B.17, that where the machine is minimal, the $\sim_{\mathcal{L}}$ classes are the $\sim_{\mathcal{M}}$ classes.

These implications are straightforward.

$\hat{\Delta}(\sigma) = q_2 \iff \sigma \in \mathcal{L}$ (that is, it matches the regular expression `(0|1)*01`)

$\hat{\Delta}(\sigma) = q_1 \iff \sigma$ ends in `0` (it matches `(0|1)*0`)

$\hat{\Delta}(\sigma) = q_0 \iff \sigma$ is otherwise

These are the $\sim_{\mathcal{L}}$ classes.

Associated with	*Class*
q_2	$\mathcal{E}_{\mathcal{L},2} = \mathcal{L}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \Sigma^* \mid \sigma \text{ ends in } \texttt{0}\}$
q_0	$\mathcal{E}_{\mathcal{L},0} = \{\sigma \in \Sigma^* \mid \sigma \in \Sigma^* - (\mathcal{E}_{\mathcal{L},2} \cup \mathcal{E}_{\mathcal{L},1})\}$

The class $\mathcal{E}_{\mathcal{L},2}$ is the only one containing strings from $\mathcal{L}$ so it is the only final state. The class $\mathcal{E}_{\mathcal{L},0}$ contains the empty string so it is the initial state.

(B) We are looking at three sets as candidates for the equivalence classes. We are thinking that one of them, here named $\mathcal{E}_{\mathcal{L},2}$, is $\mathcal{L}$. Another, here named $\mathcal{E}_{\mathcal{L},1}$, holds those strings that end in `0`, and the final set must contain all of the other strings, $\mathcal{E}_{\mathcal{L},0} = \Sigma^* - (\mathcal{E}_{\mathcal{L},1} \cup \mathcal{E}_{\mathcal{L},2})$.

First we argue that $\mathcal{E}_{\mathcal{L},2} = \mathcal{L}$ is a $\sim_{\mathcal{L}}$ equivalence class. Assume that $\sigma_0, \sigma_1 \in \mathcal{L}$, aiming to show that $\sigma_0 \sim_{\mathcal{L}} \sigma_1$. Both strings end in `01`. Consider an extension string $\tau \in \mathbb{B}^*$. If $\tau = \varepsilon$ then $\sigma_0 \frown \tau \in \mathcal{L}$ and $\sigma_i \frown \tau \in \mathcal{L}$. If $\tau \neq \varepsilon$ then $\sigma_0 \frown \tau$ and and $\sigma_i \frown \tau$ are both elements of $\mathcal{L}$ if and only if τ ends in `01`. In both of those two cases the pair have no distinguishing extension.

We finish showing that this set is an equivalence class by showing that no string $\sigma \in \mathcal{E}_{\mathcal{L},2} = \mathcal{L}$ is $\sim_{\mathcal{L}}$ equivalent to a string $\alpha \in \mathcal{L}^{\mathsf{c}}$. This holds because ε distinguishes between σ and α.

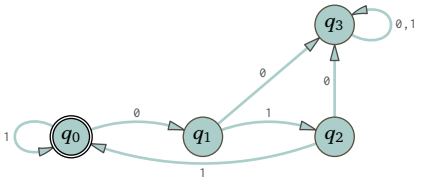
Next we argue that the set of strings ending in `0` is an equivalence class. Consider two such strings $\sigma_0, \sigma_1 \in \mathcal{E}_{\mathcal{L},1}$ along with an extension τ. If $\tau = \varepsilon$ then both $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are not elements of $\mathcal{L}$. If τ has length one then both $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are elements of $\mathcal{L}$ if and only if $\tau = \texttt{1}$. If τ has length greater than one then both $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are elements of $\mathcal{L}$ if and only if τ ends in `01`. In all of those three cases τ does not distinguish between σ_0 and σ_1.

To finish showing that this is an equivalence class, we argue that no $\sigma \in \mathcal{E}_{\mathcal{L},1}$ is $\sim_{\mathcal{L}}$ indistinguishable from some $\alpha \notin \mathcal{E}_{\mathcal{L},1}$. The argument is: no string ending in `0` is equivalent to a string not ending in `0` because a distinguishing extension is `1`.

Finally, consider the remaining strings, those that do not end with `01` and do not end with `0`. For any such $\sigma \in \mathcal{E}_{\mathcal{L},0}$, the extension $\sigma \frown \tau$ is in the language if and only if τ ends with `01`. So there is no extension that distinguishes between a pair of such strings, and they are mutually equivalent.

There is no need to show that a $\sigma \in \mathcal{E}_{\mathcal{L},0}$ cannot be equivalent to an $\alpha \notin \mathcal{E}_{\mathcal{L},0}$ because we have already done so for the earlier two classes.

IV.B.29 We follow the discussion before Example B.17, that where the machine is minimal, the $\sim_{\mathcal{L}}$ classes are the $\sim_{\mathcal{M}}$ classes. Here is the natural Finite State machine to recognize the language.



It is clearly minimal, which means that the $\sim_{\mathcal{L}}$ classes equal the $\sim_{\mathcal{M}}$ classes. These implications are straightforward.

$\hat{\Delta}(\sigma) = q_0 \iff \sigma$ matches the regular expression `(1|011)*`

$\hat{\Delta}(\sigma) = q_1 \iff \sigma$ matches the regular expression `(1|011)*0`

$\hat{\Delta}(\sigma) = q_2 \iff \sigma$ matches the regular expression `(1|011)*01`

$\hat{\Delta}(\sigma) = q_3 \iff \sigma$ is otherwise; it does not match `(1|011)*(`ε`|0|01)`

These are the $\sim_{\mathcal{L}}$ classes.

Associated with	*Class*
q_0	$\mathcal{E}_{\mathcal{L},0} = \mathcal{L}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \Sigma^* \mid \sigma \text{ matches } \texttt{(1|011)*0}\}$
q_2	$\mathcal{E}_{\mathcal{L},2} = \{\sigma \in \Sigma^* \mid \sigma \text{ matches } \texttt{(1|011)*01}\}$
q_3	$\mathcal{E}_{\mathcal{L},3} = \{\sigma \in \Sigma^* \mid \sigma \text{ does not match } \texttt{(1|011)*(}\varepsilon\texttt{|0|01)}\}$

The class $\mathcal{E}_{\mathcal{L},0}$ is the only accepting class because it is the only one containing strings from $\mathcal{L}$. The class $\mathcal{E}_{\mathcal{L},0}$ also contains the empty string so it is the initial class.

IV.B.30 There are of course many possible answers. A reasonable approach is to think of a regular language where to to make the Finite State machine there are two or more natural accepting states.

Consider $\mathcal{L} \subseteq \{\mathsf{a}, \mathsf{b}\}^*$ containing all strings that end in a, and also all strings of length two. Then the strings a and bb are both members of $\mathcal{L}$ but they have a distinguishing extension $\tau = \mathsf{b}$, so the set $\mathcal{L}$ splits into at least two equivalence classes.

IV.B.31 We will show there are infinitely many $\sim_{\mathcal{L}}$ equivalence classes by showing that there are infinitely many non-equivalent strings.

We claim that no two unequal strings of the form a^n are equivalent. As an example consider $\sigma_2 = \mathsf{aa}$ and $\sigma_3 = \mathsf{aaa}$. Take $\tau = \mathsf{baa}$. Because $\sigma_2 \frown \tau \in \mathcal{L}$ and $\sigma_3 \frown \tau \notin \mathcal{L}$, the two are not equivalent. Clearly this works for any two such strings.

IV.B.32 The classes are: the set of strings with the same number of a's as b's. those with one more a, those with one more b, those with two more a's, etc. We don't have to prove that, we need only prove that there are infinitely many distinguishable strings.

So let $n(\sigma)$ be the number of a's minus the number of b's. That could be positive, negative, or zero. Assume that $n(\sigma_0) \neq n(\sigma_0)$ for $\sigma_0, \sigma_1 \in \{\mathsf{a}, \mathsf{b}\}^*$. Then a distinguishing string is any $\tau \in \{\mathsf{a}, \mathsf{b}\}^*$ where $n(\tau) = -1 \cdot n(\sigma_0)$.

IV.B.33 We will show that there are infinitely many non-equivalent strings.

ε 0 00 000 ...

If $i \neq j$ then $\mathtt{0}^i \nsim_{\mathcal{L}} \mathtt{0}^j$ since $\tau = \mathtt{1}^i$ is a distinguishing extension. That's true because $\mathtt{0}^i \frown \mathtt{1}^i$ is a member of $\mathcal{L}$ but $\mathtt{0}^j \frown \mathtt{1}^i$ is not.

IV.B.34 We will show that every string in the language is in its own $\sim_{\mathcal{L}}$ equivalence class, and so there are infinitely many such classes.

Consider $\sigma_0 = \mathtt{0}^{2^n}$ and $\sigma_1 = \mathtt{0}^{2^m}$ where $n \neq m$. One of them must be smaller; suppose that $n < m$. Then the extension string $\tau = \mathtt{0}^{2^n}$ gives $\sigma_0 \frown \tau = \mathtt{0}^{2^n} \frown \mathtt{0}^{2^n} = \mathtt{0}^{2 \cdot 2^n} = \mathtt{0}^{2^{n+1}}$ is an element of the language. But because m is greater than n the string $\sigma_1 \frown \tau$ is not an element of the language (the number of 0's in the string is $2^m + 2^n$, which is strictly less than $2^{2(m+1)}$ because n is strictly less than m). So they have a distinguishing extension.

IV.B.35 For both arguments write the language as $\mathcal{L}$ and the alphabet as Σ.

(A) By definition, $\sigma_0 \sim_{\mathcal{L}} \sigma_1$ if there is no distinguishing extension τ. That holds if and only if either both of $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are members $\mathcal{L}$ or both are not.

The relation $\sim_{\mathcal{L}}$ is reflexive because for any string $\sigma \in \Sigma^*$, either both of $\sigma \frown \tau$ and $\sigma \frown \tau$ are in the language or both are not.

To verify that the relation is symmetric assume that $\sigma_0 \sim \sigma_1$. Then for any $\tau \in \Sigma^*$, either both of $\sigma_0 \frown \tau$ and $\sigma_1 \frown \tau$ are in the language or both are not. By symmetry of the word 'and', either both of $\sigma_1 \frown \tau$ and $\sigma_0 \frown \tau$ and are in the language or both are not. That is equivalent to the statement that $\sigma_1 \sim \sigma_0$.

Finally, for transitivity assume that $\sigma_0 \sim \sigma_1$ and $\sigma_1 \sim \sigma_2$. Let τ be a string. Either $\sigma_1 \frown \tau \in \mathcal{L}$ or not. If $\sigma_1 \frown \tau \in \mathcal{L}$ then $\sigma_0 \sim \sigma_1$ implies that $\sigma_0 \frown \tau \in \mathcal{L}$, and $\sigma_1 \sim \sigma_2$ implies that $\sigma_2 \frown \tau \in \mathcal{L}$, and therefore both of $\sigma_0 \frown \tau$ and $\sigma_2 \frown \tau$ are members of the language. Similarly if $\sigma_1 \frown \tau \notin \mathcal{L}$ then both of both of $\sigma_0 \frown \tau$ and $\sigma_2 \frown \tau$ are not members of the language. Consequently, $\sigma_0 \sim_{\mathcal{L}} \sigma_2$.

(B) By definition, $\sigma_0 \sim_{\mathcal{M}} \sigma_1$ if $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_1)$. The relation $\sim_{\mathcal{M}}$ is reflexive because $\hat{\Delta}(\sigma) = \hat{\Delta}(\sigma)$ for any string $\sigma \in \Sigma^*$.

To verify that the relation is symmetric, assume that $\sigma_0 \sim_{\mathcal{M}} \sigma_1$. Then $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_1)$ and so by symmetry of equality, $\hat{\Delta}(\sigma_1) = \hat{\Delta}(\sigma_0)$. Thus $\sigma_1 \sim_{\mathcal{M}} \sigma_0$.

For transitivity assume that $\sigma_0 \sim_{\mathcal{M}} \sigma_1$ and $\sigma_1 \sim_{\mathcal{M}} \sigma_2$. The first gives $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_1)$ while the second gives $\hat{\Delta}(\sigma_1) = \hat{\Delta}(\sigma_2)$. Therefore $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_2)$ and $\sigma_0 \sim_{\mathcal{M}} \sigma_2$.

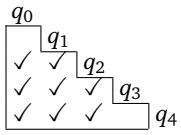
IV.B.36 Two strings $\sigma_0, \sigma_1\Sigma^*$ are $\mathcal{L}$-indistinguishable if and only if for every extension τ, either both of $\sigma_0 \frown \tau$ and $\sigma_0 \frown \tau$ are elements of $\mathcal{L}$ or both of them are elements of $\mathcal{L}^{\mathsf{c}}$. That's equivalent to being $\mathcal{L}^{\mathsf{c}}$-indistinguishable.

IV.C.8 There is a cluster of blank cells in the entries q_0, q_1 and q_0, q_2 and q_1, q_2. There is also a blank in q_3, q_5.

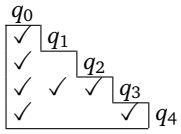
There is no blank associated with q_4 so it is alone in its class. The equivalence classes are $\mathcal{E}_{i,0} = \{q_0, q_1, q_2\}$, and $\mathcal{E}_{i,1} = \{q_3, q_5\}$, and $\mathcal{E}_{i,2} = \{q_4\}$.

IV.C.9

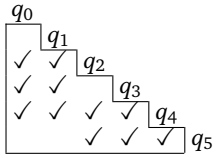
(A) The given classes say that we can i-distinguish q_0 from q_2, q_3, and q_4. So looking down the column headed by q_0 we put checkmarks in rows labeled q_2, q_3, and q_4. The other columns work the same way.



(B)



(C)



IV.C.10

(A) The classes are different for q_0, q_1, and q_0, q_2, and q_1, q_5, and q_2, q_5 (all are different in the b column).

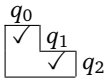
(B) The matching $\sim_1$ classes are $\mathcal{E}_{1,0} = \{q_0, q_5\}$, and $\mathcal{E}_{1,1} = \{q_1, q_2\}$, and $\mathcal{E}_{1,2} = \{q_3, q_4\}$

IV.C.11 By inspection, all of the states are reachable.

Step 0 is to put into separate classes the accepting states and the non-accepting states.

$$\mathcal{E}_{0,0} = \{q_0, q_2\} \qquad \mathcal{E}_{0,1} = \{q_1\}$$

We also make the initial triangular table by checkmarking the q_i, q_j entries where one of q_i and q_j is accepting while the other is not.



The checkmarks denote pairs of states that are 0-distinguishable and the blanks denote pairs of states that are 0-indistinguishable.

Step 1 sees if any of the $\sim_0$ classes split. Here is the computation.

	0	1
q_0, q_2	$q_0 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$

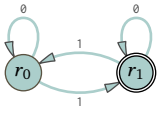
Neither of the two classes split. The first phase of the algorithm stops with two $\sim$-equivalence classes.

$$\mathcal{E}_0 = \{q_0, q_2\} \qquad \mathcal{E}_1 = \{q_1\}$$

Incorporating the information from the original machine gives this minimized one.

	a	b
q_0	$q_0 \in \mathcal{E}_0$	$q_1 \in \mathcal{E}_1$
q_2	$q_2 \in \mathcal{E}_0$	$q_1 \in \mathcal{E}_1$
+ q_1	$q_1 \in \mathcal{E}_1$	$q_2 \in \mathcal{E}_0$

	a	b
r_0	r_0	r_1
+ r_1	r_1	r_0

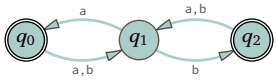


IV.C.12

(A) These are the sets.

Step n	R_n
0	$\{q_0\}$
1	$\{q_0, q_1\}$
2	$\{q_0, q_1, q_2\}$
3	$\{q_0, q_1, q_2\}$

Thus the set of reachable states is $R = \{q_0, q_1, q_2\}$. The set of unreachable states is $Q - R = \{q_3\}$. Here is the machine with only the reachable states.

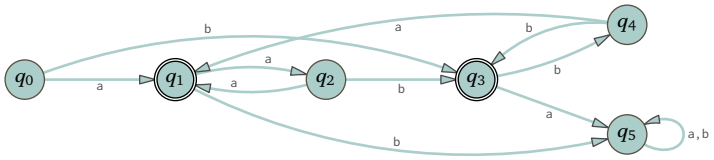


(b) These are the sets.

Step n	R_n
0	$\{q_0\}$
1	$\{q_0, q_1, q_3\}$
2	$\{q_0, q_1, q_3, q_4\}$
3	$\{q_0, q_1, q_3, q_4\}$

The set of reachable states is $R = R_2 = \{q_0, q_1, q_3, q_4\}$. The set of unreachable states is $Q - R = \{q_2, q_5\}$.

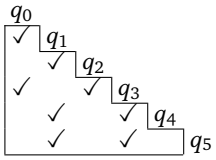
IV.C.13 For reference, here is the machine.



By inspection every state is reachable.

Step 0 separates the states into two $\sim_0$ classes, one where the states are not accepting and one where they are. Also shown is the associated triangular table, marking the 0-disinguishable states, where one of q_i and q_j is accepting while the other is not.

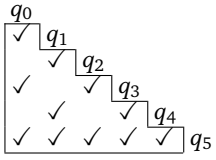
$$\mathcal{E}_{0,0} = \{q_0, q_2, q_4, q_5\} \quad \mathcal{E}_{0,1} = \{q_1, q_3\}$$



Step 1 sees if any of the two $\sim_0$ classes split. Here is the calculation.

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{0,1},\ q_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,1},\ q_3 \in \mathcal{E}_{0,1}$
q_0, q_4	$q_1 \in \mathcal{E}_{0,1},\ q_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,1},\ q_3 \in \mathcal{E}_{0,1}$
q_0, q_5	$q_1 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$
q_2, q_4	$q_1 \in \mathcal{E}_{0,1},\ q_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,1},\ q_3 \in \mathcal{E}_{0,1}$
q_2, q_5	$q_1 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$
q_4, q_5	$q_1 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_2 \in \mathcal{E}_{0,0},\ q_5 \in \mathcal{E}_{0,0}$	$q_5 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,0}$

That gives this triangular table and the $\sim_1$ equivalence classes, showing that $\mathcal{E}_{0,0}$ splits into two.

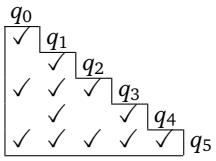


$$\mathcal{E}_{1,0} = \{q_0, q_2, q_4\} \quad \mathcal{E}_{1,1} = \{q_5\} \quad \mathcal{E}_{1,2} = \{q_1, q_3\}$$

Next, step 2.

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{1,2},\ q_1 \in \mathcal{E}_{1,2}$	$q_3 \in \mathcal{E}_{1,2},\ q_3 \in \mathcal{E}_{1,2}$
q_0, q_4	$q_1 \in \mathcal{E}_{1,2},\ q_1 \in \mathcal{E}_{1,2}$	$q_3 \in \mathcal{E}_{1,2},\ q_3 \in \mathcal{E}_{1,2}$
q_2, q_4	$q_1 \in \mathcal{E}_{1,2},\ q_1 \in \mathcal{E}_{1,2}$	$q_3 \in \mathcal{E}_{1,2},\ q_3 \in \mathcal{E}_{1,2}$
q_1, q_3	$q_2 \in \mathcal{E}_{1,0},\ q_5 \in \mathcal{E}_{1,1}$	$q_5 \in \mathcal{E}_{1,1},\ q_4 \in \mathcal{E}_{1,0}$

This triangular table summarizes, showing that $\mathcal{E}_{1,2}$ splits.



$$\mathcal{E}_{2,0} = \{q_0, q_2, q_4\} \quad \mathcal{E}_{2,1} = \{q_5\} \quad \mathcal{E}_{2,2} = \{q_1\} \quad \mathcal{E}_{2,3} = \{q_3\}$$

Step 3's calculation shows no more splitting.

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{2,2},\ q_1 \in \mathcal{E}_{2,2}$	$q_3 \in \mathcal{E}_{2,3},\ q_3 \in \mathcal{E}_{2,3}$
q_0, q_4	$q_1 \in \mathcal{E}_{2,2},\ q_1 \in \mathcal{E}_{2,2}$	$q_3 \in \mathcal{E}_{2,3},\ q_3 \in \mathcal{E}_{2,3}$
q_2, q_4	$q_1 \in \mathcal{E}_{2,2},\ q_1 \in \mathcal{E}_{2,2}$	$q_3 \in \mathcal{E}_{2,3},\ q_3 \in \mathcal{E}_{2,3}$

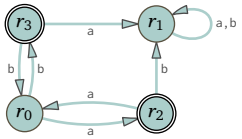
These are the $\sim$ equivalence classes.

$$\mathcal{E}_0 = \{q_0, q_2, q_4\} \quad \mathcal{E}_1 = \{q_5\} \quad \mathcal{E}_2 = \{q_1\} \quad \mathcal{E}_3 = \{q_3\}$$

The second phase of Moore's algorithm uses those as states to produce this minimal machine.

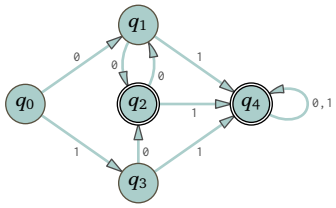
	a	b
q_0	$q_1 \in \mathcal{E}_2$	$q_3 \in \mathcal{E}_3$
q_2	$q_1 \in \mathcal{E}_2$	$q_3 \in \mathcal{E}_3$
q_4	$q_1 \in \mathcal{E}_2$	$q_3 \in \mathcal{E}_3$
q_5	$q_5 \in \mathcal{E}_1$	$q_5 \in \mathcal{E}_1$
$+\ q_1$	$q_4 \in \mathcal{E}_0$	$q_5 \in \mathcal{E}_1$
$+\ q_3$	$q_5 \in \mathcal{E}_1$	$q_4 \in \mathcal{E}_0$

	a	b
r_0	r_2	r_3
r_1	r_1	r_1
$+\ r_2$	r_0	r_1
$+\ r_3$	r_1	r_0



IV.C.14 The answer is "yes." The initial state of the minimal machine is the one that contains q_0. In the minimized machine a state is accepting if and only if it contains any accepting states in the original machine. So if q_0 is accepting then the minimal machine's initial state is also accepting.

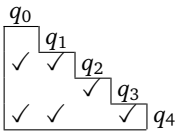
IV.C.15 For reference here is the initial machine.



All of the states are reachable, by inspection.

Step 0 splits the states into two classes, the accepting ones and the others. We also include the initial triangular table, which checkmarks the q_i, q_j entries where one of q_i and q_j is accepting while the other is not.

$$\mathcal{E}_{0,0} = \{q_0, q_1, q_3\}$$
$$\mathcal{E}_{0,1} = \{q_2, q_4\}$$



Next, step 1 sees if any of the $\sim_0$ classes split. First is the calculation.

	0	1
q_0, q_1	$q_1 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$
q_1, q_3	$q_2 \in \mathcal{E}_{0,1},\ q_2 \in \mathcal{E}_{0,1}$	$q_4 \in \mathcal{E}_{0,1},\ q_4 \in \mathcal{E}_{0,1}$
q_2, q_4	$q_1 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$	$q_4 \in \mathcal{E}_{0,1},\ q_4 \in \mathcal{E}_{0,1}$

The splitting is: state pairs that are not 0-distinguishable but are 1-distinguishable are q_0, q_1, and q_0, q_3, and q_2, q_4. The triangular table reflects these changes, and the blank cells show that there are four $\sim_1$ classes.

q_0			
✓	q_1		
✓	✓	q_2	
✓		✓	q_3
✓	✓	✓	✓ q_4

$\mathcal{E}_{1,0} = \{q_0\}$
$\mathcal{E}_{1,1} = \{q_1, q_3\}$
$\mathcal{E}_{1,2} = \{q_2\}$
$\mathcal{E}_{1,3} = \{q_4\}$

Step 2 decides of any of the $\sim_1$ classes split.

	0	1
q_1, q_3	$q_2 \in \mathcal{E}_{1,2},\ q_2 \in \mathcal{E}_{1,2}$	$q_4 \in \mathcal{E}_{1,3},\ q_4 \in \mathcal{E}_{1,3}$

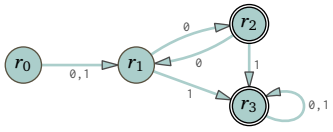
There is no further splitting. These are the $\sim$ equivalence classes.

$\mathcal{E}_0 = \{q_0\}$ $\mathcal{E}_1 = \{q_1, q_3\}$ $\mathcal{E}_2 = \{q_2\}$ $\mathcal{E}_3 = \{q_4\}$

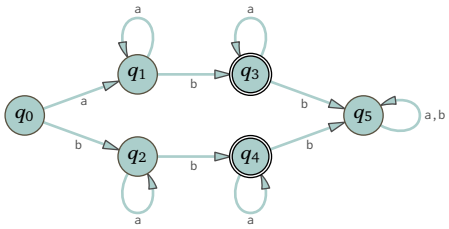
Phase two determines the transitions.

	0	1
q_0	$q_1 \in \mathcal{E}_1$	$q_3 \in \mathcal{E}_1$
q_1	$q_2 \in \mathcal{E}_2$	$q_4 \in \mathcal{E}_3$
q_3	$q_2 \in \mathcal{E}_2$	$q_4 \in \mathcal{E}_3$
+ q_2	$q_1 \in \mathcal{E}_1$	$q_4 \in \mathcal{E}_3$
+ q_4	$q_4 \in \mathcal{E}_3$	$q_4 \in \mathcal{E}_3$

	0	1
r_0	r_1	r_1
r_1	r_2	r_3
+ r_2	r_1	r_3
+ r_3	r_3	r_3



IV.C.16 By eye, all of the states are reachable. This is the starting machine, for reference.



Step 0 splits the collection of all states into the class of accepting state and the class of non-accepting states.

$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_5\}$ $\mathcal{E}_{0,1} = \{q_3, q_4\}$

We also give the initial triangular table by checkmarking the q_i, q_j entries where one of q_i and q_j is accepting while the other is not.

q_0				
	q_1			
		q_2		
✓	✓	✓	q_3	
✓	✓	✓		q_4
			✓ ✓	q_5

Step 1 computes whether any state pairs are 1-distinguishable that are not 0-distinguishable.

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{0,0},\ q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,1}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$
q_0, q_5	$q_1 \in \mathcal{E}_{0,0},\ q_5 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_5 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_1 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1},\ q_4 \in \mathcal{E}_{0,1}$
q_1, q_5	$q_1 \in \mathcal{E}_{0,0},\ q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$
q_2, q_5	$q_2 \in \mathcal{E}_{0,0},\ q_5 \in \mathcal{E}_{0,0}$	$q_4 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$
q_3, q_4	$q_3 \in \mathcal{E}_{0,1},\ q_4 \in \mathcal{E}_{0,1}$	$q_5 \in \mathcal{E}_{0,0},\ q_5 \in \mathcal{E}_{0,0}$

That computation distinguishes q_0, q_1, and q_0, q_2, and q_1, q_5, and q_2, q_5. Here is the updated triangular table, and the $\sim_1$ classes given by its blank cells.

q_0					
✓	q_1				
✓		q_2			
✓	✓	✓	q_3		
✓	✓	✓		q_4	
	✓	✓	✓	✓	q_5

$$\mathcal{E}_{1,0} = \{q_0, q_5\}$$
$$\mathcal{E}_{1,1} = \{q_1, q_2\}$$
$$\mathcal{E}_{1,2} = \{q_3, q_4\}$$

The class $\mathcal{E}_{0,0}$ has split into two.

Iterate. Step 2 computes whether any of those $\sim_1$ equivalence classes split.

	a	b
q_0, q_5	$q_1 \in \mathcal{E}_{1,1},\ q_5 \in \mathcal{E}_{1,0}$	$q_2 \in \mathcal{E}_{1,1},\ q_5 \in \mathcal{E}_{1,0}$
q_1, q_2	$q_1 \in \mathcal{E}_{1,1},\ q_2 \in \mathcal{E}_{1,1}$	$q_3 \in \mathcal{E}_{1,2},\ q_4 \in \mathcal{E}_{1,2}$
q_3, q_4	$q_3 \in \mathcal{E}_{1,2},\ q_4 \in \mathcal{E}_{1,2}$	$q_5 \in \mathcal{E}_{1,0},\ q_5 \in \mathcal{E}_{1,0}$

The only state pair that is 2-distinguishable but was not 1-distinguishable is q_0, q_5. The triangular table reflects this additional checkmark.

q_0					
✓	q_1				
✓		q_2			
✓	✓	✓	q_3		
✓	✓	✓		q_4	
✓	✓	✓	✓	✓	q_5

$$\mathcal{E}_{2,0} = \{q_0\}$$
$$\mathcal{E}_{2,1} = \{q_1, q_2\}$$
$$\mathcal{E}_{2,2} = \{q_3, q_4\}$$
$$\mathcal{E}_{2,3} = \{q_5\}$$

Again. Step 3's calculation is here.

	a	b
q_1, q_2	$q_1 \in \mathcal{E}_{2,1},\ q_2 \in \mathcal{E}_{2,1}$	$q_3 \in \mathcal{E}_{2,2},\ q_4 \in \mathcal{E}_{2,2}$
q_3, q_4	$q_3 \in \mathcal{E}_{2,2},\ q_4 \in \mathcal{E}_{2,2}$	$q_5 \in \mathcal{E}_{2,3},\ q_5 \in \mathcal{E}_{2,3}$

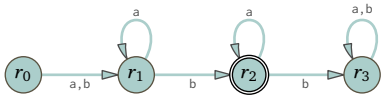
There is no splitting. The first phase of Moore's algorithm terminates. These are the $\sim$ classes.

$$r_0 = \mathcal{E}_0 = \{q_0\} \quad r_1 = \mathcal{E}_1 = \{q_1, q_2\} \quad r_2 = \mathcal{E}_2 = \{q_3, q_4\} \quad r_3 = \mathcal{E}_3 = \{q_5\}$$

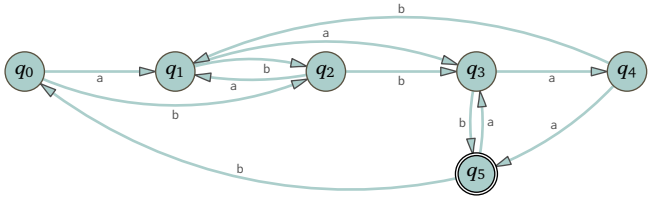
As to the second phase, here is the minimized machine.

	a	b
q_0	$q_1 \in \mathcal{E}_1$	$q_2 \in \mathcal{E}_1$
q_1	$q_1 \in \mathcal{E}_1$	$q_3 \in \mathcal{E}_2$
q_2	$q_2 \in \mathcal{E}_1$	$q_4 \in \mathcal{E}_2$
+ q_3	$q_3 \in \mathcal{E}_2$	$q_5 \in \mathcal{E}_3$
+ q_4	$q_4 \in \mathcal{E}_2$	$q_5 \in \mathcal{E}_3$
q_5	$q_5 \in \mathcal{E}_3$	$q_5 \in \mathcal{E}_3$

	a	b
r_0	r_1	r_1
r_1	r_1	r_2
+ r_2	r_2	r_3
r_3	r_3	r_3



IV.C.17 When we apply the algorithm we will find that all states are indistinguishable. For reference here is the machine.



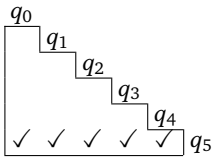
Clearly all states are reachable.

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{0,0}$, $q_3 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}$, $q_2 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0}$, $q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}$, $q_3 \in \mathcal{E}_{0,0}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0}$, $q_4 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$
q_0, q_4	$q_1 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}$, $q_1 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_3 \in \mathcal{E}_{0,0}$, $q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}$, $q_3 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_3 \in \mathcal{E}_{0,0}$, $q_4 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$
q_1, q_4	$q_3 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}$, $q_1 \in \mathcal{E}_{0,0}$
q_2, q_3	$q_1 \in \mathcal{E}_{0,0}$, $q_4 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$
q_2, q_4	$q_1 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}$, $q_1 \in \mathcal{E}_{0,0}$
q_3, q_4	$q_4 \in \mathcal{E}_{0,0}$, $q_5 \in \mathcal{E}_{0,1}$	$q_5 \in \mathcal{E}_{0,1}$, $q_1 \in \mathcal{E}_{0,0}$

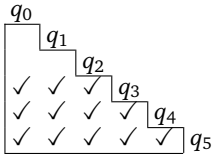
FIGURE 96, FOR QUESTION IV.C.17: Step (1) of Exercise C.17.

Step 0 separates the accepting states from the non-accepting ones.

$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_3, q_4\}$ $\mathcal{E}_{0,1} = \{q_5\}$



Step 1's table is so big that it is broken out as Figure 96 on page 168. That calculation finds that while these pairs of states are 0-indistinguishable they ares 1-distinguishable: q_0, q_3, and q_0, q_4, and q_1, q_3, and q_1, q_4, and q_2, q_3, and q_2, q_4, and q_3, q_4. That calculation gives this triangular table and set of equivalence classes.

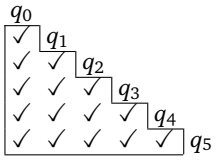


$\mathcal{E}_{1,0} = \{q_0, q_1, q_2\}$ $\mathcal{E}_{1,1} = \{q_3\}$ $\mathcal{E}_{1,2} = \{q_4\}$ $\mathcal{E}_{1,3} = \{q_5\}$

Step 2's calculation is much smaller.

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{1,0}$, $q_3 \in \mathcal{E}_{1,1}$	$q_2 \in \mathcal{E}_{1,0}$, $q_2 \in \mathcal{E}_{1,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{1,0}$, $q_1 \in \mathcal{E}_{1,0}$	$q_2 \in \mathcal{E}_{1,0}$, $q_3 \in \mathcal{E}_{1,1}$
q_1, q_2	$q_3 \in \mathcal{E}_{1,1}$, $q_1 \in \mathcal{E}_{1,0}$	$q_2 \in \mathcal{E}_{1,0}$, $q_3 \in \mathcal{E}_{1,1}$

That calculation 2-distinguishes all three pairs q_0, q_1, and q_0, q_2, and q_1, q_2. The result is that the triangular table is completely filled and each state is in its own equivalence class.



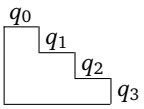
$\mathcal{E}_{2,0} = \{q_0\}$ $\mathcal{E}_{2,1} = \{q_1\}$ $\mathcal{E}_{2,2} = \{q_2\}$ $\mathcal{E}_{2,3} = \{q_3\}$ $\mathcal{E}_{2,4} = \{q_4\}$ $\mathcal{E}_{2,5} = \{q_5\}$

Since there are no indistinguishable states to collapse together, the original machine is minimal.

IV.C.18 We will go through the algorithm but before we do, observe that the answer must be that a machine with no accepting states therefore accepts no string. A minimal such machine has one state, the initial state, and all arrows are loops. So we know what the answer must be before we start and we are just verifying that the algorithm does the right thing.

By inspection all of the states are reachable. Step 0 divides the collection of states into the accepting states and the others. Here we also include the initial triangular table.

$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_3\}$



The point is that there is one $\sim_0$-equivalence class, not the two that we have seen in other minimizations.

Step 1 sees if any of these classes split.

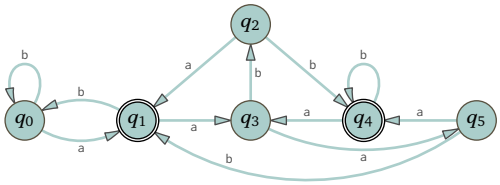
	0	1
q_0, q_1	$q_1 \in \mathcal{E}_{0,0},\ q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0},\ q_0 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0},\ q_0 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_1 \in \mathcal{E}_{0,0},\ q_0 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_1 \in \mathcal{E}_{0,0},\ q_0 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_2, q_3	$q_0 \in \mathcal{E}_{0,0},\ q_0 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$

That computation 1-distinguishes no state pairs (how could it? there is only one class). So the first phase of Moore's algorithm stops.

For the second phase there is only one class so this is the resulting minimized machine, as predicted.

If we instead start with a machine where all states are accepting then a similar thing happens. This machine recognizes the language Σ^* and its minimization has one state, which is both initial and accepting. The algorithm, as above, would never distinguish between pairs because there would be only one class, the accepting states.

IV.C.19 For reference here is the starting machine.

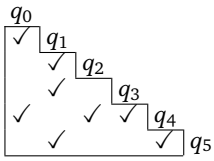


By inspection all of its states are reachable.

Step 0 splits the states into two classes, those that are accepting and those that are not.

$\mathcal{E}_{0,0} = \{q_0, q_2, q_3, q_5\}$ $\mathcal{E}_{0,1} = \{q_1, q_4\}$

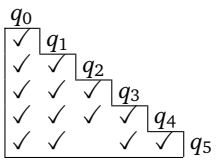
Here is the initial triangular table, with entry q_i, q_j checkmarked when one of q_i and q_j is accepting while the other is not.



Step 1 computes whether any of those classes split.

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{0,1},\ q_1 \in \mathcal{E}_{0,1}$	$q_0 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_0, q_5	$q_1 \in \mathcal{E}_{0,1},\ q_4 \in \mathcal{E}_{0,1}$	$q_0 \in \mathcal{E}_{0,0},\ q_1 \in \mathcal{E}_{0,1}$
q_2, q_3	$q_1 \in \mathcal{E}_{0,1},\ q_5 \in \mathcal{E}_{0,0}$	$q_4 \in \mathcal{E}_{0,1},\ q_2 \in \mathcal{E}_{0,0}$
q_2, q_5	$q_1 \in \mathcal{E}_{0,1},\ q_4 \in \mathcal{E}_{0,1}$	$q_4 \in \mathcal{E}_{0,1},\ q_1 \in \mathcal{E}_{0,1}$
q_3, q_5	$q_5 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0},\ q_1 \in \mathcal{E}_{0,1}$
q_1, q_4	$q_3 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$

In that calculation we find different classes in entries on the lines for q_0, q_2, and q_0, q_3, and q_0, q_5, and q_2, q_3, and q_3, q_5, and q_1, q_4. So this distinguishes is all pairs except for q_2, q_5. Update the triangular table and split the classes.



$$\mathcal{E}_{1,0} = \{q_0\}$$
$$\mathcal{E}_{1,1} = \{q_1\}$$
$$\mathcal{E}_{1,2} = \{q_2, q_5\}$$
$$\mathcal{E}_{1,3} = \{q_3\}$$
$$\mathcal{E}_{1,4} = \{q_4\}$$

Finally, step 2 computes whether the one remaining multi-state class splits.

	a	b
q_2, q_5	$q_1 \in \mathcal{E}_{1,1},\ q_4 \in \mathcal{E}_{1,4}$	$q_4 \in \mathcal{E}_{1,4},\ q_1 \in \mathcal{E}_{1,1}$

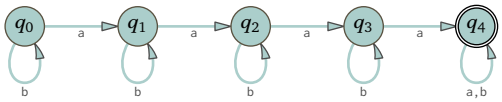
There is a split, and so all of the states are 2-distinguishable. Every $\sim$ class is a singleton.

$$\mathcal{E}_0 = \{q_0\} \quad \mathcal{E}_1 = \{q_1\} \quad \mathcal{E}_2 = \{q_2\} \quad \mathcal{E}_3 = \{q_3\} \quad \mathcal{E}_4 = \{q_4\} \quad \mathcal{E}_5 = \{q_5\}$$

The original machine is minimal.

IV.C.20 We will get a machine that recognizes the same language. The reachable states of this machine will form a minimal set. But there will still be unreachable states (although perhaps fewer of them since it can be that some collapse together.)

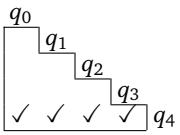
IV.C.21 Here is the starting machine.



By inspection all of the states are reachable.

Step 0 splits the collection of states into two classes, the accepting states and the non-accepting states. The initial triangular table checkmarks the q_i, q_j entries where one of q_i and q_j is accepting but the other is not.

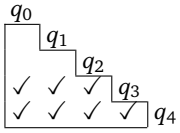
$$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_3\}$$
$$\mathcal{E}_{0,1} = \{q_4\}$$



Step 1 computes whether any of the $\sim_0$ classes split.

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0},\ q_1 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$	$q_0 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_2 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$	$q_1 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_2 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$	$q_1 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$
q_2, q_3	$q_3 \in \mathcal{E}_{0,0},\ q_4 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0},\ q_3 \in \mathcal{E}_{0,0}$

We must put checkmarks in the triangular table cells q_0, q_3, and q_1, q_3, and q_2, q_3. In short, q_3 is 1-distinguishable from all of the other states.



$$\mathcal{E}_{1,0} = \{q_0, q_1, q_2\}$$
$$\mathcal{E}_{1,1} = \{q_3\}$$
$$\mathcal{E}_{1,2} = \{q_4\}$$

Next is step 2.

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{1,0},\ q_2 \in \mathcal{E}_{1,0}$	$q_0 \in \mathcal{E}_{1,0},\ q_1 \in \mathcal{E}_{1,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{1,0},\ q_3 \in \mathcal{E}_{1,1}$	$q_0 \in \mathcal{E}_{1,0},\ q_2 \in \mathcal{E}_{1,0}$
q_1, q_2	$q_2 \in \mathcal{E}_{1,0},\ q_3 \in \mathcal{E}_{1,1}$	$q_1 \in \mathcal{E}_{1,0},\ q_2 \in \mathcal{E}_{1,0}$

We need checkmarks in cells q_0, q_2 and q_1, q_2. So this calculation distinguishes a single state, q_2.

q_0				
	q_1			
✓	✓	q_2		
✓	✓	✓	q_3	
✓	✓	✓	✓	q_4

$$\mathcal{E}_{2,0} = \{q_0, q_1\}$$
$$\mathcal{E}_{2,1} = \{q_2\}$$
$$\mathcal{E}_{2,2} = \{q_3\}$$
$$\mathcal{E}_{2,3} = \{q_4\}$$

There is only one class that could split so step 3 will be the last one, whether or not there is any splitting.

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{2,0},\ q_2 \in \mathcal{E}_{2,1}$	$q_0 \in \mathcal{E}_{2,0},\ q_1 \in \mathcal{E}_{2,0}$

They do split; q_1 is 3-distinguishable from q_0. These are the $\sim_3$ classes.

q_0				
✓	q_1			
✓	✓	q_2		
✓	✓	✓	q_3	
✓	✓	✓	✓	q_4

$$\mathcal{E}_{3,0} = \{q_0\}$$
$$\mathcal{E}_{3,1} = \{q_1\}$$
$$\mathcal{E}_{3,2} = \{q_2\}$$
$$\mathcal{E}_{3,3} = \{q_3\}$$
$$\mathcal{E}_{3,4} = \{q_4\}$$

The first phase of Moore's algorithm stops. The minimized machine has five states; it is the machine that we started with.

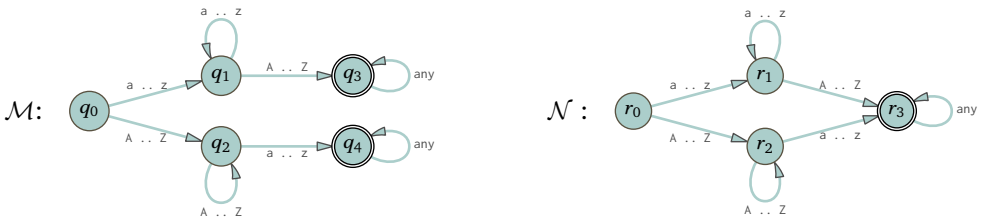
Had we started with a similar machine having a one more state then the algorithm would take one more step. The number of steps is two less than the number of states, that is, the time taken grows linearly with the number of states.

IV.C.22 Recall that to show a binary relation is an equivalence we must show that it is reflexive, symmetric, and transitive.

(A) Fix some $n \in \mathbb{N}$. For reflexivity, obviously any state is n-indistinguishable from itself, for any n. Symmetry is also clear. So suppose that $q_0 \sim_n q_1$ and $q_1 \sim_n q_2$ for some machine. Let σ be a string of length less than or equal to n. There are two cases. The first is that σ takes the machine from q_0 to an accepting state then because $q_0 \sim_n q_1$ it also takes the machine from q_1 to an accepting state, and from that it follows because $q_1 \sim_n q_2$ that it also takes the machine from q_2 to an accepting state. The other case, that σ takes the machine from q_0 to a non-accepting state, is similar.

(B) A Finite Sate machine has only so many states. So only so many states can be distinguished. Once k is larger than the last index n where states can be n-distinguished then $\sim$ is $\sim_k$. The prior item applies.

IV.C.23 For reference here are the two machines.



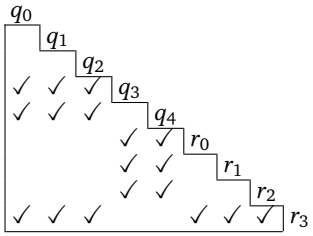
(A) These are the classes.

$$\mathcal{E}_{0,0} = \{q_3, q_4, r_3\} \quad \mathcal{E}_{0,1} = \{q_0, q_1, q_2, r_0, r_1, r_2\}$$

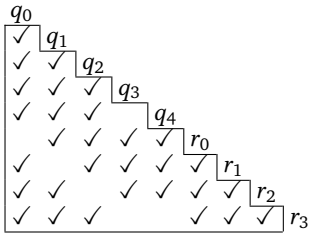
Here is the initial triangular table.

	a	A
q_3, q_4	$q_3 \in \mathcal{E}_{0,0}$, $q_4 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}$, $q_4 \in \mathcal{E}_{0,0}$
q_3, r_3	$q_3 \in \mathcal{E}_{0,0}$, $r_3 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}$, $r_3 \in \mathcal{E}_{0,0}$
q_4, r_3	$q_4 \in \mathcal{E}_{0,0}$, $r_3 \in \mathcal{E}_{0,0}$	$q_4 \in \mathcal{E}_{0,0}$, $r_3 \in \mathcal{E}_{0,0}$
q_0, q_1	$q_1 \in \mathcal{E}_{0,1}$, $q_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}$, $q_3 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,1}$, $q_4 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,1}$, $q_2 \in \mathcal{E}_{0,1}$
q_0, r_0	$q_1 \in \mathcal{E}_{0,1}$, $r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}$, $r_2 \in \mathcal{E}_{0,1}$
q_0, r_1	$q_1 \in \mathcal{E}_{0,1}$, $r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$
q_0, r_2	$q_1 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,1}$, $r_2 \in \mathcal{E}_{0,1}$
q_1, q_2	$q_1 \in \mathcal{E}_{0,1}$, $q_4 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}$, $q_2 \in \mathcal{E}_{0,1}$
q_1, r_0	$q_1 \in \mathcal{E}_{0,1}$, $r_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}$, $r_2 \in \mathcal{E}_{0,1}$
q_1, r_1	$q_1 \in \mathcal{E}_{0,1}$, $r_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}$, $r_3 \in \mathcal{E}_{0,0}$
q_1, r_2	$q_1 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}$, $r_2 \in \mathcal{E}_{0,1}$
q_2, r_0	$q_4 \in \mathcal{E}_{0,0}$, $r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}$, $r_2 \in \mathcal{E}_{0,1}$
q_2, r_1	$q_4 \in \mathcal{E}_{0,0}$, $r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$
q_2, r_2	$q_4 \in \mathcal{E}_{0,0}$, $r_3 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,1}$, $r_2 \in \mathcal{E}_{0,1}$
r_0, r_1	$r_1 \in \mathcal{E}_{0,1}$, $r_1 \in \mathcal{E}_{0,1}$	$r_2 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$
r_0, r_2	$r_1 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$	$r_2 \in \mathcal{E}_{0,1}$, $r_2 \in \mathcal{E}_{0,1}$
r_1, r_2	$r_1 \in \mathcal{E}_{0,1}$, $r_3 \in \mathcal{E}_{0,0}$	$r_3 \in \mathcal{E}_{0,0}$, $r_2 \in \mathcal{E}_{0,1}$

FIGURE 97, FOR QUESTION IV.C.23: Step (1) of Exercise C.23.



(B) Step 1 looks at pairs from the two $\sim_0$ classes, calculating whether character transitions split any $\sim_0$ class. This table is so large that it is Figure 97 on page 172. There are many splits, for example q_0 and q_1 are 1-distinguishable.



There are six empty cells, at q_0, r_0, at q_1, r_1, at q_2, r_2, at q_3, q_4, at q_3, r_3, and at q_4, r_3. We conclude that $\mathcal{E}_{0,0}$ does not split but that $\mathcal{E}_{0,1}$ splits into three.

$$\mathcal{E}_{1,0} = \{q_3, q_4, r_3\} \quad \mathcal{E}_{1,1} = \{q_0, r_0\} \quad \mathcal{E}_{1,2} = \{q_1, r_1\} \quad \mathcal{E}_{1,3} = \{q_2, r_2\}$$

(C) The calculation for Step 2 does no more distinguishings.

(D) Moore's algorithm has grouped q_0 with r_0, and q_1 with r_1, and q_2 with r_2. It has also grouped q_3 and q_4 together with r_3. Since there is one and only one r_i in each class, that machine recognizes the same language as the q_j machine and is minimal.

IV.C.24

(A) The algorithm halts at a step when no class splits. Such a step must happen eventually since these machines have only finitely many states.

(B) If there is a $\sim_n$ class where some character sends two states in that class to different $\sim_n$ classes then the

algorithm does not terminate at that step. Instead it splits that class and goes to the next step. So the algorithm cannot end with an ill-defined transition function.

(C) in Moore's algorithm, step 0 partitions the states into two classes, one with all of $\mathcal{M}$'s final states and one with all of $\mathcal{M}$'s non-final states. At the end the $\sim$ classes are subclasses of those, and so consist entirely of final states or of non-final states.

(D) Consider a string, $\sigma \in \Sigma^*$. We will show that it is accepted by the machine $\mathcal{M}$ if and only if it is accepted by the other machine $\mathcal{N}$. The argument is by induction, although we will present it informally.

The input machine starts in q_0 while the output machine starts in a state $r_0 = \mathcal{E}_0$ that contains q_0. The first character of σ moves $\mathcal{M}$ from q_0 to some state q_{i_1} and moves $\mathcal{N}$ from a state that contains q_0 to a state that contains q_{i_1}. The second character of σ moves $\mathcal{M}$ from q_{i_1} to some q_{i_2}, and moves $\mathcal{N}$ from a state that contains q_{i_1} to a state that contains q_{i_2}. The machines proceed in this way until the string runs out. At the end, $\mathcal{M}$ is in a final state if and only if $\mathcal{N}$ is in a state containing that final state, which is itself a final state of $\mathcal{N}$.

(E) To be clear about the notation: the starting machine is $\mathcal{M}$, the result of running that machine through Moore's algorithm is $\mathcal{N}$, and $\hat{\mathcal{M}}$ is a Finite State machine that recognizes the same language as $\mathcal{M}$. As a preliminary, arrange that $\hat{\mathcal{M}}$ and $\mathcal{N}$ don't share any states by renaming so that the states of $\hat{\mathcal{M}}$ are $q_0, q_1, \ldots$ and the states of $\mathcal{N}$ are $r_0, r_1, \ldots$

Before we start we make two observations about minimal machines. First, in the minimal machine $\mathcal{N}$ all states must be reachable, or else we could eliminate a non-reachable state and get a smaller machine that recognizes the same language. Second, running the minimal machine through Moore's algorithm ends with each $\sim$ class having one and only one member.

We must show that $\hat{\mathcal{M}}$ has at least as many states as $\mathcal{N}$. Begin by performing Moore's algorithm on the union of the two sets of states (as in Exercise C.23).

The two machines $\hat{M}$ and $\mathcal{N}$ recognize the same language, so starting both in their start state with a string $\tau \in \Sigma^*$ on the tape results always in the same outcome: either both machines end in an accepting state or both end in a rejecting state. Therefore there is no string distinguishing between q_0 and r_0, that is, when we run Moore's algorithm on the union of the two sets of states then their start states are indistinguishable, $q_0 \sim r_0$.

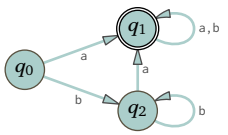
We finish with an induction argument. Suppose that states q and r are indistinguishable. We claim that for all $x \in \Sigma$, the state $\Delta_{\mathcal{N}}(q, x)$ is indistinguishable from the state $\Delta_{\hat{\mathcal{M}}}(r, x)$. That's because if τ were a string distinguishing between $\Delta_{\mathcal{N}}(q, x)$ and $\Delta_{\hat{\mathcal{M}}}(r, x)$ then $x^\frown\tau$ would distinguish between q and r.

The first observation above shows that there is some string $\sigma \in \Sigma^*$ such that $\hat{\Delta}_{\mathcal{N}}(\sigma) = r$. This σ takes the machine $\hat{\mathcal{M}}$ to some state, that is, $\hat{\Delta}_{\hat{\mathcal{M}}}(\sigma)$ is a state in $\hat{\mathcal{M}}$, and by the prior paragraph it is indistinguishable from r.

By the above observation, every $\sim$ class contains one and only one state r of $\mathcal{N}$. The prior paragraph says that each state r of $\mathcal{N}$ is in the same $\sim$ class as at least one state q of $\hat{\mathcal{M}}$. Consequently, there are at least as many states in $\mathcal{M}$ as there are in $\mathcal{N}$.

IV.C.25

(A) Here is the machine, for reference.

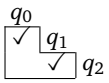


By inspection all of the states are reachable.

Step 0 separates the states into two classes, one with the accepting states and one with the non-accepting states. starts by checkmarking the q_i, q_j entries.

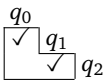
$$\mathcal{E}_{0,0} = \{q_0, q_2\} \quad \mathcal{E}_{0,1} = \{q_1\}$$

Here is the initial triangular table with checkmarks in entry q_i, q_j where one of q_i and q_j is accepting while the other is not.



Next, step 1 calculates whether any of those $\sim_0$ classes split.

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{0,1},\ q_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0},\ q_2 \in \mathcal{E}_{0,0}$



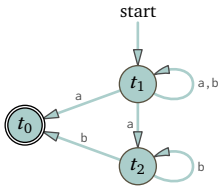
There is no splitting.

These are the $\sim$ classes.

$$r_0 = \mathcal{E}_0 = \{q_0, q_2\} \quad r_1 = \mathcal{E}_1 = \{q_1\}$$

Here is the minimized machine.

(B) Here is the machine with the arrows reversed, the nodes renamed, the start node made final, and the final one made a start.

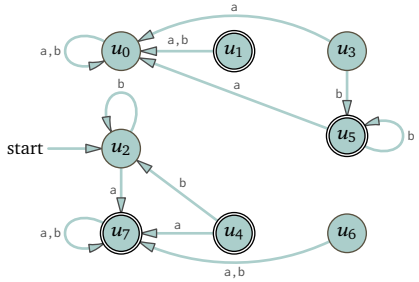


It is nondeterministic; for instance, two edges leaving node t_1 are labeled a.

(C) This is the table to convert that nondeterministic Finite State machine to a deterministic one.

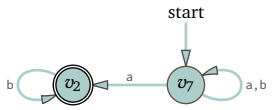
Node name	*Set of nodes*	a	b
u_0	$\varnothing$	u_0	u_0
+ u_1	$\{t_0\}$	u_0	u_0
u_2	$\{t_1\}$	u_7	u_2
u_3	$\{t_2\}$	u_0	u_5
+ u_4	$\{t_0, t_1\}$	u_7	u_2
+ u_5	$\{t_0, t_2\}$	u_0	u_5
u_6	$\{t_1, t_2\}$	u_7	u_7
+ u_7	$\{t_0, t_1, t_2\}$	u_7	u_7

Here is the arrow diagram.



The only reachable states are u_2 and u_7.

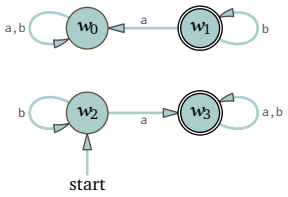
(D) Here is the picture.



(E) Convert that nondeterministic Finite State machine to a deterministic one.

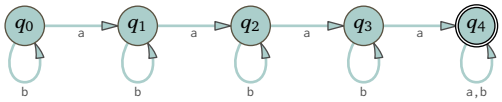
Node name	*Set of nodes*	a	b
w_0	$\emptyset$	w_0	w_0
+ w_1	$\{v_2\}$	w_0	w_1
w_2	$\{v_7\}$	w_3	w_2
+ w_3	$\{v_2, v_7\}$	w_3	w_3

Here is the arrow diagram.



Once we omit unreachable states, this machine is the same (except for state renaming) as the one in the first item.

IV.C.26 The machine of Exercise C.21



has five states and is minimal for the language described by `b*ab*ab*ab*a(a|b)*`. Clearly this generalizes.

Chapter V: Computational Complexity

V.1.31 True. Informally, Big $\mathcal{O}$ means something like "grows at a rate less than or equivalent to." Growing at a rate less than or equivalent to the rate of x^2 implies growing at a rate less than or equivalent to the rate of x^3.

Formally, assume that f is $\mathcal{O}(x^2)$. Then there are constants $N, C \in \mathbb{R}$ so that if $x \geq N$ then $f(x) \leq C \cdot x^2$. Since $x^3 \geq x^2$ for all $x \geq 1$, we have: if $\hat{x} \geq \max(N, 1)$ then $f(\hat{x}) \leq C \cdot \hat{x}^3$. Thus f is $\mathcal{O}(x^3)$.

V.1.32 First, the assertion that "The algorithm has running time $\mathcal{O}(n^2)$" does not mean that the runtime function is quadratic, since if $f(n) = n$ then f is $\mathcal{O}(n^2)$.

Second, even if the running time is quadratic, it may involve constants and lower order terms. Thus, $g_0(n) = 1000 \cdot n^2$ is $\mathcal{O}(n^2)$, as is $g_1(n) = n^2 + 3n + 42$. But neither $g_0(5)$ nor $g_1(5)$ equals 25.

Still another issue is that the the runtime behavior is quadratic only for sufficiently large inputs. So unless we know that 5 is greater than the N of the definition, there is no reason at all to think quadratic.

V.1.33 Perhaps they mean that for their problem the algorithm that they have is $\mathcal{O}(n^3)$. Or perhaps they mean that the best algorithm known is $\mathcal{O}(n^3)$.

V.1.34

(A) Because 5 in binary is 101, it takes three bits.

(B) The number 50 in binary is 11 0010 so it takes six bits.

(C) In binary the number 500 is 1 1111 0100 so it take nine bits.

(D) The binary equivalent of 5 000 is 1 0011 1000 1000, which is thirteen bits.

This table verifies these using the formula $\text{bits}(n) = 1 + \lfloor \lg(n) \rfloor$.

Integer n in decimal	5	50	500	5 000
Binary representation	101	11 0010	1 1111 0100	1 0011 1000 1000
$\lg(n)$	2.322	5.644	8.966	12.288
$1 + \lfloor \lg(n) \rfloor$	3	6	9	13

V.1.35 The second is true while the first is not.

Recall the intuition that 'f is $\mathcal{O}(g)$' means something like "f's growth rate is less than or equivalent to g's," while 'f is $\Theta(g)$' means something like "f's growth rate is equivalent to g's." So the second statement makes sense because if f and g grow at equivalent rates then that implies that f's rate is less than or equivalent to g's. Arguing precisely, the second is true because by definition if f is $\Theta(g)$ then both f is $\mathcal{O}(g)$ and g is $\mathcal{O}(f)$.

As to the first statement, the intuition is that assuming that f's growth is less than or equivalent to g's does not force that the two are equivalent. That is, there are functions f, g so that f is $\mathcal{O}(g)$ but f is not $\Theta(g)$. One such pair is $f(n) = n$ and $g(n) = n^2$.

V.1.36 The only correct answer is 'None'. We can't apply Big $\mathcal{O}$ to a fixed number. Big $\mathcal{O}$ is about how the function grows as the input size grows, and does not apply to individual inputs. Restated, if 61 669 is not greater than the definition's N then the statement of quadratic-ness doesn't apply at all.

Comment. It is true that in practice you will often hear people refer to an algorithm, or the code implementing an algorithm, by saying something like, "the runtime for 60 000 should be in the neighborhood of quadruple the time for 30 000." They mean that they are familiar with the algorithm or code and that the N and C of Big $\mathcal{O}$'s definition play only a minimal role. But with only the information given in this question we cannot say that.

V.1.37 We use the principles for simplifying Big $\mathcal{O}$ expressions, Lemma 1.12 from page **??**.

(A) $\mathcal{O}(n^2)$

(B) $\mathcal{O}(2^{\text{poly}(n)})$

(C) $\mathcal{O}(n^4)$

(D) $\mathcal{O}(\lg n)$

V.1.38

(A) For sufficiently large input arguments n this function is $f(n) = 0$. So it is $\mathcal{O}(1)$.

(B) For sufficiently large arguments n this function is $f(n) = n^2$. So it is $\mathcal{O}(n^2)$.

(C) For sufficiently large n this function is $f(n) = \lg(n)$. So it is $\mathcal{O}(\lg(n))$.

V.1.39 Because we will apply L'Hôpital's Rule, we convert from n's to x's, signaling we must take them to be real number functions.

(A) To apply the theorem consider the limit of the ratio.

$$\lim_{x\to\infty}\frac{f(x)}{g(x)}=\lim_{x\to\infty}\frac{3x^3+2x+4}{\ln(x)+6}$$

This is an "∞/∞" limit. Apply L'Hôpital's Rule.

$$\lim_{x\to\infty}\frac{3x^3+2x+4}{\lg(x)+6}=\lim_{x\to\infty}\frac{9x^2+2}{1/(\ln(2)x)}=\ln(2)\cdot\lim_{x\to\infty}\frac{x\cdot(9x^2+2)}{1}=\infty$$

So g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$. In short, f's growth rate is strictly bigger than g's.

(B) This is of type "∞/∞." Use L'Hôpital's Rule multiple times.

$$\lim_{x\to\infty}\frac{f(x)}{g(x)}=\lim_{x\to\infty}\frac{3x^3+2x+4}{x+5x^3}=\lim_{x\to\infty}\frac{9x^2+2}{1+15x^2}=\lim_{x\to\infty}\frac{18x}{30x}=\lim_{x\to\infty}\frac{18}{30}$$

Conclude that f is $\Theta(g)$ (we can also state that as g is $\Theta(f)$). Briefly, f's growth rate is equivalent to g's.

(C) This is the limit of the ratios.

$$\lim_{x\to\infty}\frac{f(x)}{g(x)}=\lim_{x\to\infty}\frac{(1/2)x^3+12x^2}{x^2\ln(x)}$$

We could start by simplifying the fraction, canceling x^2 from the numerator and the denominator. However, we choose to instead start by applying L'Hôpital's Rule to this "∞/∞" limit. Note the use of the Product Rule in the denominator.

$$\begin{aligned}\lim_{x\to\infty}\frac{(1/2)x^3+12x^2}{x^2\ln(x)}=\lim_{x\to\infty}\frac{(3/2)x^2+24x}{2x\ln(x)+x}=\lim_{x\to\infty}\frac{(3/2)x^2+24x}{x(2\ln(x)+1)}=\lim_{x\to\infty}\frac{(3/2)x+24}{2\ln(x)+1}\\ =\lim_{x\to\infty}\frac{3/2}{2/x}=\lim_{x\to\infty}\frac{3x}{4}=\infty\end{aligned}$$

So g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$.

(D) The limit is "∞/∞." L'Hôpital's Rule gives this.

$$\lim_{x\to\infty}\frac{f(x)}{g(x)}=\lim_{x\to\infty}\frac{\lg(x)}{\ln(x)}=\lim_{x\to\infty}\frac{(1/x)\cdot(1/\ln(2))}{(1/x)}=\lim_{x\to\infty}\frac{1}{\ln(2)}\cdot\frac{(1/x)}{(1/x)}=\frac{1}{\ln(2)}\cdot\lim_{x\to\infty}1$$

So they grow at equivalent rates: f is $\Theta(g)$.

(E) Recognize this as an "∞/∞" limit.

$$\begin{aligned}\lim_{x\to\infty}\frac{f(x)}{g(x)}=\lim_{x\to\infty}\frac{x^2+\lg(x)}{x^4-x^3}=\lim_{x\to\infty}\frac{2x+(1/x)}{4x^3-3x^2}=\lim_{x\to\infty}\frac{2x^2+1}{4x^4-3x^3}\\ =\lim_{x\to\infty}\frac{4x}{16x^3-9x^2}=\lim_{x\to\infty}\frac{4}{48x^2-18x}=0\end{aligned}$$

So f is $\mathcal{O}(g)$ but g is not $\mathcal{O}(f)$.

(F) We consider the ratio of the functions.

$$\lim_{x\to\infty}\frac{f(x)}{g(x)}=\lim_{x\to\infty}\frac{55}{x^2+x}$$

This is not an "∞/∞" limit (it is instead "$55/\infty$") so L'Hôpital's Rule doesn't apply. But it doesn't matter because it is easy. The denominator grows at a faster rate than the numerator because the numerator doesn't grow at all. That is, f is $\mathcal{O}(g)$ but g is not $\mathcal{O}(f)$.

V.1.40 From Table 1.27 we can deduce that it is (D).

To establish $\mathcal{O}(f_2)<\mathcal{O}(f_3)$ independently of the table we have

$$\lim_{x\to\infty}\frac{x^{\ln x}}{x^{\sqrt{x}}}=\lim_{x\to\infty}x^{\ln x-\sqrt{x}}=0$$

because $\lim_{x\to\infty}(\ln x-\sqrt{x})=-\infty$. (One way to prove this limit is to use Lemma 1.12 and Table 1.27. Another way is to multiply $\log x-\sqrt{x}$ by the fraction $(\log x+\sqrt{x})/(\log x+\sqrt{x})$. That gives this.

$$\begin{aligned}\lim_{x\to\infty}\frac{(\ln x)^2-x}{\sqrt{x}+\ln(x)}&=\lim_{x\to\infty}\frac{2\ln x\cdot(1/x)-1}{1/(2x^{1/2})+1/x}=2\cdot\lim_{x\to\infty}\frac{\ln x-x}{\sqrt{x}+2}\\&=2\cdot\lim_{x\to\infty}\frac{(1/x)-1}{1/(2x^{1/2})}=4\cdot\lim_{x\to\infty}\frac{x^{-1/2}-x^{1/2}}{1}=4\cdot\lim_{x\to\infty}\frac{1-x}{\sqrt{x}}\\&=4\cdot\lim_{x\to\infty}\frac{-1}{1/(2x^{1/2})}=4\cdot\lim_{x\to\infty}-2\sqrt{x}\end{aligned}$$

That's $-\infty$.)

For $\mathcal{O}(f_3)<\mathcal{O}(f_1)$ take the limit of the natural logarithm of the numerator and denominator. Since the natural logarithm is an increasing function, this proves the result.

$$\lim_{x\to\infty}\frac{\ln(f_3(x))}{\ln(f_1(x))}=\lim_{x\to\infty}\frac{\ln(x^{\sqrt{x}})}{\ln(10^x)}=\lim_{x\to\infty}\frac{\sqrt{x}\ln(x)}{x\ln(10)}=\lim_{x\to\infty}\frac{1}{\ln(10)}\cdot\frac{\ln(x)}{\sqrt{x}}$$

This is an "∞/∞" type limit. Apply L'HôItal.

$$\lim_{x\to\infty}\frac{1/x}{1/(2x^{1/2})}=2\cdot\lim_{x\to\infty}\frac{\sqrt{x}}{x}=2\cdot\lim_{x\to\infty}\frac{1}{\sqrt{x}}=0$$

V.1.41 Note that $\cos(t)$ is bounded by 1. Also, $\lg(5^n)=n\cdot\lg(5)$. Thus, the ones with a growth rate less than or equivalent to n^2's are the first, second, fourth, and fifth. The third one has a growth rate that is $\Theta(n^3)$, bigger than n^2's.

V.1.42 We can do these is with Theorem 1.17 and Table 1.27. L'Hôpital's Rule also works. For these answers we did the first both ways, and some of the rest with L'Hôpital's Rule.

(A) True. By Theorem 1.17 this is the sum of a function that is $\Theta(n^2)$ and one that is $\Theta(n)$. By the same lemma and Table 1.27 the sum is $\Theta(n^2)$, and therefore is $\mathcal{O}(n^2)$.

The alternate argument with L'Hôpital's Rule gives that $5n^2+2n$ is $\Theta(n^2)$.

$$\lim_{x\to\infty}\frac{5n^2+2n}{n^2}=\lim_{x\to\infty}\frac{10n+2}{2n}=\lim_{x\to\infty}\frac{10}{2}=5$$

In the order of growth hierarchy that lies below n^3.

(B) False. L'Hôpital's Rule shows that $2+4n^3$ is $\Theta(n^3)$.

$$\lim_{x\to\infty}\frac{2+4n^3}{n^3}=\lim_{x\to\infty}\frac{12n^2}{3n^2}=4$$

But in the order of growth hierarchy Table 1.27 that is strictly above $\lg n$.

(C) True. As discussed in the section, all logarithmic functions grow at the same rate. So $\ln n$ is $\Theta(\lg n)$ and hence $\ln n$ is $\mathcal{O}(\lg n)$.

(D) True. L'Hôpital's Rule gives this

$$\lim_{x\to\infty}\frac{x^3+x^2+x}{x^3}=\lim_{x\to\infty}\frac{3x^2+2x+1}{3x^2}=\lim_{x\to\infty}\frac{6x+2}{6x}=\lim_{x\to\infty}\frac{6}{6}=1$$

(these are a sequence of "∞/∞" limits). So n^3+n^2+n is $\Theta(n^3)$, and therefore n^3+n^2+n is $\mathcal{O}(n^3)$.

(E) True. For any polynomial function p, we have that p is $\mathcal{O}(2^{\text{poly}(n)})$, by Lemma 1.25.

V.1.43 *Comment:* In doing these we may have the option of using the algebraic properties in Lemma 1.12 to simplify, or of using L'Hôpital's Rule. The lemma gives more of a sense of using how the expressions are constructed. For instance, in the first item below we strictly speaking do not need the first paragraph because the second paragraph suffices to show $k=4$ is correct. But including only the second paragraph could give a sense to a person who had trouble with the question that the $k=4$ was plucked out of the air.

(A) The smallest power is $k=4$. By the lemma's item A the function $(1/10\,000\,000)n^4$ is $\mathcal{O}(n^4)$ and of course the function n^3 is $\mathcal{O}(n^3)$. By Table 1.27 n^3 is $\mathcal{O}(n^4)$, and thus by item B of the lemma, the function $n^3+(1/10\,000\,000)n^4$ is $\mathcal{O}(n^4)$.

One way to see that no smaller power $k\in\mathbb{N}$ will do is to consider $\lim_{x\to\infty}x^3/(x^3+(x^4/10\,000\,000))$.

$$\begin{aligned}\lim_{x\to\infty}\frac{x^4}{x^3+(x^4/10\,000\,000)}&=\lim_{x\to\infty}\frac{x}{1+(x/10\,000\,000)}=\lim_{x\to\infty}\frac{x}{(10\,000\,000+x)/10\,000\,000)}\\&=\lim_{x\to\infty}\frac{10\,000\,000x}{10\,000\,000+x}=\lim_{x\to\infty}\frac{10\,000\,000}{1}=10\,000\,000\end{aligned}$$

(the next to last equality is from L'Hôpital's Rule). So $n^3+(n^4/10\,000\,000)$ is $\Theta(n^4)$. Thus $k=4$ is the minimum.

(B) The smallest power is $k=4$. Multiplying through gives $n^4+5n^3+6n^2-n^2\lg n-5n\lg n-6\lg n$. Lemma 1.12's item B says that the entire expression is $\mathcal{O}(g)$ where g is the term with the largest order of growth. In this expression Table 1.27 shows that it is n^4.

To check that no smaller power suffices, compare the functions with the limit.

$$\lim_{x\to\infty}\frac{x^4+5x^3+6x^2-x^2\lg x-5x\lg x-6\lg x}{x^4}=\lim_{x\to\infty}\frac{x^4}{x^4}+\frac{5x^3}{x^4}+\frac{6x^2}{x^4}-\frac{x^2\lg x}{x^4}-\frac{5x\lg x}{x^4}-\frac{6\lg x}{x^4}$$

All of those limits are zero except for the first, which is 1. Therefore $x^4+5x^3+6x^2-x^2\lg x-5x\lg x-6\lg x$ is $\Theta(x^4)$.

(C) The answer is $k=3$. The cosine function is bounded by 1. So Lemma 1.12 gives that the entire expression is $\mathcal{O}(n^3)$. We can use Theorem 1.17 to argue that $n=3$ is right with this limit.

$$\lim_{x\to\infty}\frac{5x^3+25+\lceil\cos(x)\rceil}{x^3}\lim_{x\to\infty}\frac{5x^3}{x^3}+\frac{25+\lceil\cos(x)\rceil}{x^3}=5$$

The limit comes to a number intermediate between zero and infinity so $5x^3$ is $\Theta(x^3)$.

(D) This answer is $k=12$. Expanding the expression gives a leading term of $9n^{12}$. Either Lemma 1.12 or Theorem 1.17 then gives the answer.

(E) This is $\mathcal{O}(n^{7/2})$ but since the question specifies that $k\in\mathbb{N}$, the answer is $k=4$.

V.1.44 In these we simplify Big $\mathcal{O}$ expressions using Lemma 1.12.

(A) The answer is 'both'. Both of these functions are $\Theta(n^2)$. So by Lemma 1.15 f is $\Theta(g)$.

(B) The answer is that g is $\mathcal{O}(f)$ but not vice versa. Here, g is a logarithmic function and so is $\Theta(\lg n)$, while and f is a cubic and so is $\Theta(n^3)$. Then the table gives that g is $\mathcal{O}(f)$ but it is not the case that f is $\mathcal{O}(g)$.

(C) The function f is quadratic while g is a square root. Thus g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$.

(D) The function f is $\mathcal{O}(n^{1.2})$ while the function g is $\mathcal{O}(n^{\sqrt{2}})$, which is approximately $\mathcal{O}(n^{1.414})$. Consequently, f is $\mathcal{O}(g)$ but g is not $\mathcal{O}(f)$.

V.1.45 We simplify Big $\mathcal{O}$ expressions using Lemma 1.12.

(A) Lemma 1.25 says that the exponential function $2^{(1/6)n}$ grows faster than the polynomial function n^6. So f is $\mathcal{O}(g)$ but it is not the case the g is $\mathcal{O}(f)$.
(B) Table 1.27 shows that the function 2^n is $\mathcal{O}(3^n)$, while 3^n is not $\mathcal{O}(2^n)$. Thus, g is $\mathcal{O}(f)$ but f is not $\mathcal{O}(g)$.
(C) Recall the logarithm rule that $\lg(3n) = \lg(3) + \lg(n)$. Because $\lg(3)$ is $\Theta(1)$, Lemma 1.12 says that $\lg(3n)$ is $\Theta(\lg n)$. From this it follows that the two have equivalent growth rates, so f is $\Theta(g)$.

V.1.46 We will apply the definition, Definition 1.6, of Big $\mathcal{O}$, to show that Z is $\mathcal{O}(g)$ for any complexity function g.

Because g is a complexity function there is an $N \in \mathbb{N}$ such that if $x \geq N$ then $g(x)$ is defined and nonnegative. Take $C = 1$. Then for $\hat{x} > N$ we have that $|Z(\hat{x})| = 0 \leq C \cdot |g(\hat{x})| = C \cdot g(\hat{x})$.

V.1.47

	$n = 1$	$n = 10$	$n = 100$	$n = 1000$
$\lg n$	–	1.05×10^{-17}	2.10×10^{-17}	3.15×10^{-17}
n	3.16×10^{-18}	3.16×10^{-17}	3.16×10^{-16}	3.16×10^{-15}
$n \lg n$	–	1.05×10^{-16}	2.10×10^{-15}	3.15×10^{-14}
n^2	3.16×10^{-18}	3.16×10^{-16}	3.16×10^{-14}	3.16×10^{-12}
n^3	3.16×10^{-18}	3.16×10^{-15}	3.16×10^{-12}	3.16×10^{-9}
2^n	6.33×10^{-18}	3.24×10^{-15}	4.01×10^{12}	3.39×10^{283}

V.1.48 The number of seconds in a year is $60 \cdot 60 \cdot 24 \cdot 365.25 = 31\,557\,600$. At 10 00, 000 000 ticks per second, that is about 3.16×10^{17} ticks in a year (as also described in Table 1.29).
(A) With $\lg(n) = 3.16 \times 10^{17}$, we have $n = 2^{3.16 \times 10^{17}} = 4.60 \times 10^{94\,997\,841\,911\,656\,529}$.
(B) $(3.16 \times 10^{17})^2 = 9.96 \times 10^{34}$
(C) 3.16×10^{17}
(D) $(3.16 \times 10^{17})^{1/2} = 5.62 \times 10^{8}$
(E) Because $(3.16 \times 10^{11})^{1/3} \approx 680823.684681371$, the answer is 680 823.
(F) We have $\lg(3.16 \times 10^{17}) \approx 58.131$ so the answer is 58.

V.1.49 Solving $100\,000 \cdot n^2 = n^3$ gives $n = 100\,000$. So the first number n where $f(n) = 100 \cdot n^2$ is less than $g(n) = n^3$ is $n = 100\,001$.

V.1.50 Linear. Finite State machines consume one character at each step. So they run in time equal to the length of the input. In the notation of Definition 1.30, $t_{\mathcal{M}}(\sigma) = |\sigma|$.

V.1.51 Let g be the function $g(x) = 1$.
(A) We can directly apply the definition of Big $\mathcal{O}$, Definition 1.6. Take $N = 1$ and $C = 8$. If $x > N = 1$ then $C \cdot |g(x)| = 8 \cdot 1$ while $|f(x)|$, so $|f(x)| \leq C|g(x)|$.
(B) Recall that $\sin(x)$ is bounded between -1 and 1. Take $N = 1$ and $C = 9$. If $x > N$ then $C \cdot |g(x)| = 9$ while $|f(x)| = 7 + \sin(x) \leq 8$.
(C) Fix $N = 1$ and $C = 8$. If $x \geq N = 1$ then $C \cdot |g(x)| = 8$ is greater than or equal to $|f(x)| = 7 + (1/x)$.
(D) We first show that if it is bounded then it is $\mathcal{O}(1)$. Suppose that f is bounded by $K \in \mathbb{R}$. If it is bounded by a smaller number then it is bounded by a larger so we can take $K \in \mathbb{R}^+$. Fix $N = 1$ and $C = K$. If $x \geq N = 1$ then $C \cdot |g(x)| = K \geq |f(x)|$, so f is $\mathcal{O}(g)$.

For the converse, suppose that f is $\mathcal{O}(g)$. Then there exists constants $N, C \in \mathbb{R}^+$ so that $x \geq N$ implies that $C \cdot |g(x)| = C \geq |f(x)|$. Choose $L = \max(C, f(0), \ldots f(N-1))$. Then for any $x \in \mathbb{N}$ we have $f(x) \leq L$.

V.1.52 A function $f\colon \mathbb{R} \to \mathbb{R}$ is $\mathcal{O}(1)$ if it is bounded by a constant. So $g(x) \leq x^{\mathcal{O}(1)}$ says that the function g has at most a polynomial growth rate.

V.1.53 Let $N = 4$ and $C = 1$. Note that $4! = 24 > 2^4 = 16$. With that, if $n \geq N = 4$ then $C \cdot n! = n! = 4! \cdot 5 \cdot 6 \cdots (n-1) \cdot n \geq 2^4 \cdot 2^{n-4} = 2^n$.

V.1.54
(A) This limit equals 0

$$\lim_{x\to\infty} \frac{\left(\log_b(x)\right)^2}{x^d} = \lim_{x\to\infty} \frac{2\log_b(x) \cdot \frac{1}{x\ln(b)}}{dx^{d-1}} = \frac{2}{d\ln(b)} \cdot \lim_{x\to\infty} \frac{\log_b(x)}{x^d}$$

by the calculation in Example 1.22.

(B) This limit equals 0

$$\lim_{x\to\infty}\frac{\big(\log_b(x)\big)^3}{x^d}=\lim_{x\to\infty}\frac{3\big(\log_b(x)\big)^2\cdot\frac{1}{x\ln(b)}}{dx^{d-1}}=\frac{3}{d\ln(b)}\cdot\lim_{x\to\infty}\frac{\big(\log_b(x)\big)^2}{x^d}$$

by the calculation in the prior item.

(C) The prior item suggests an argument by induction. So assume that the statement holds for the powers $n=1$, $n=2$, ... $n=k$ and consider the $n=k+1$ case.

$$\lim_{x\to\infty}\frac{\big(\log_b(x)\big)^{k+1}}{x^d}=\lim_{x\to\infty}\frac{(k+1)\big(\log_b(x)\big)^k\cdot\frac{1}{x\ln(b)}}{dx^{d-1}}=\frac{k+1}{d\ln(b)}\cdot\lim_{x\to\infty}\frac{\big(\log_b(x)\big)^k}{x^d}$$

Finish by applying the $n=k$ inductive hypothesis.

V.1.55 The function

$$K(x)=\begin{cases}0 & \text{- if } \phi_x(x)\!\uparrow\\ 1 & \text{- if } \phi_x(x)\!\downarrow\end{cases}$$

is bounded and so is clearly $\mathcal{O}(1)$.

V.1.56 For the first use Theorem 1.17.

$$\lim_{x\to\infty}\frac{2^x}{3^x}=\lim_{x\to\infty}\left(\frac{2}{3}\right)^x=0$$

For the second, $3^x=2^{\lg(3)\cdot x}$ where $\lg(3)$ is the base 2 logarithm of 3.

V.1.57

(A) With an eye toward Theorem 1.17, consider the ratio.

$$\frac{n!}{n^n}=\frac{1}{n}\cdot\frac{2}{n}\cdots\frac{n}{n}\leq\frac{1}{n}\cdot 1\cdots 1$$

Clearly the limit of the ratio $n!/n^n$ is 0.

(B) No. Don't overlook the e^{-n}.

V.1.58

(A) Apply L'Hôpital's Rule twice, after verifying that they are both "∞/∞" limits.

$$\lim_{x\to\infty}\frac{x^2+5x+1}{x^2}=\lim_{x\to\infty}\frac{2x+5}{2x}=\lim_{x\to\infty}\frac{2}{2}=1$$

(B) Apply L'Hôpital's Rule also here, remembering to use the Chain Rule for the numerator.

$$\lim_{x\to\infty}\frac{\lg(x+1)}{\lg(x)}=\lim_{x\to\infty}\frac{(1/x\ln(2))\cdot 1}{1/x\ln(2)}=1$$

V.1.59 No. If $\mathcal{O}(f)$ is the set of polynomials then f must be a polynomial. However, then f has some degree, and no polynomial of higher degree is a member of $\mathcal{O}(f)$.

V.1.60 We have $x^{\lg x}=(2^{\lg x})^{\lg x}=2^{(\lg x)^2}\in\mathcal{O}(2^{\text{poly}(\lg x)})$ (the first equality comes from the identity $x=2^{\lg x}$).

V.1.61 Recall the logarithm rules that $\log_b(x^a)=a\cdot\log_b x$ and $b^{\log_b(x)}=x$.

(A) Consider the ratio.

$$\lim_{x\to\infty}\frac{k\lg x}{(\lg x)^2}=k\lim_{x\to\infty}\frac{1}{\lg x}=0$$

Theorem 1.17 gives the desired conclusion.

(B) By Theorem 1.17, it suffices to show that this is 0.

$$\lim_{x\to\infty}\frac{x^k}{x^{\lg(x)}}=\lim_{x\to\infty}x^{k-\lg(x)}$$

Rewrite using the identity that $a=2^{\lg(a)}$.

$$=\lim_{x\to\infty}2^{\lg(\,x^{k-\lg(x)}\,)}=\lim_{x\to\infty}2^{(k-\lg(x))\cdot\lg(x)}=\lim_{x\to\infty}2^{k\lg(x)-(\lg(x))^2}$$

The prior item gives that the limit of the exponent is $-\infty$, and so the limit of the entire expression is 0.

(C) L'Hôpital's Rule gives this.

$$\lim_{x\to\infty}\frac{(\lg x)^2}{x}=\lim_{x\to\infty}\frac{2\lg x\cdot(1/x)}{1}=2\lim_{x\to\infty}\frac{\lg x}{x}=2\lim_{x\to\infty}\frac{(1/x)}{1}=2\lim_{x\to\infty}\frac{1}{x}=0$$

(D) Apply Theorem 1.17 along with L'Hôpital's Rule

$$\lim_{x\to\infty}\frac{x^{\lg x}}{2^x}=\lim_{x\to\infty}\frac{(2^{\lg x})^{\lg x}}{2^x}=\lim_{x\to\infty}\frac{2^{(\lg x)^2}}{2^x}=\lim_{x\to\infty}2^{(\lg x)^2-x}$$

(the first equality uses the identity $a=2^{\lg(a)}$). The prior item gives that the limit of the exponent is $\lim_{x\to\infty}(\lg x)^2-x=-\infty$, and so the limit of the entire expression is 0.

V.1.62 We will use the notation from the lemma statement.

(A) Suppose that $a\in\mathbb{R}^+$. Since f is a complexity function there is an $N\in\mathbb{R}$ so that $x\geq N$ implies that f is defined and nonnegative. Fix $C=a$. If $x\geq N$ then $|af(x)|=af\,(x)\leq a\cdot f(x)=a|f(x)|$, as required.

(B) Suppose that f_1 is $\mathcal{O}(f_0)$. By definition there are constants $N,C\in\mathbb{R}$ so that if $x\geq N$ then $|f_1(x)|\leq C\cdot|f_0(x)|$. Because these two functions are complexity functions we can assume that N is so large that both are nonnegative for all inputs x. That gives $f_0(x)+f_1(x)\leq f_0(x)+Cf_1(x)=(1+C)\cdot f_0(x)$, and hence there is a constant, $1-C$, giving that f_0+f_1 is in $\mathcal{O}(f_0)$.

The argument for f_0-f_1 is the same except that the constant is $1-C$.

(C) Take $f_0(x)=x^2$ and $f_1(x)=x$. Then the function f_0f_1 is given by $f_0f_1(x)=x^3$, so f_0f_1 is not in $\mathcal{O}(f_0)$.

V.1.63

(A) We must show that f is in $\mathcal{O}(f)$. Take $N=1$ and $C=1$.

(B) Suppose that f_0 is $\mathcal{O}(f_1)$ and that f_1 is $\mathcal{O}(f_2)$. Then there exists $N_0,N_1,C_0,C_1\in\mathbb{R}$ such that if $x\geq N_0$ then $C_0\cdot f_1(x)\geq f_0(x)$, and if $x\geq N_1$ then $C_1\cdot f_2(x)\geq f_1(x)$. Take $\hat{N}=\max(N_0,N_1)$ and $\hat{C}=C_0\cdot C_1$. Then $x\geq\hat{N}$ implies that $\hat{C}\cdot f_2(x)\geq C_0\cdot f_1(x)\geq f_0(x)$.

V.1.64 Recall that by definition f is $\mathcal{O}(g)$ if and only if there exists numbers $N,C\in\mathbb{R}$ such that if $x\geq N$ then $|f(x)|\leq C\cdot|g(x)|$.

(A) If $\lim_{x\to\infty}f(x)/g(x)$ exists and equals 0 then for any $\varepsilon>0$ there is an $M\in\mathbb{R}$ such that $x\geq M$ implies that $|f(x)/g(x)|<\varepsilon$, and therefore $|f(x)|<\varepsilon\cdot|g(x)|$. Taking N to be M and C to be ε, this is exactly what is required to conclude that f is $\mathcal{O}(g)$.

(B) Suppose otherwise, that g is $\mathcal{O}(f)$. Then by definition there are constants $K,P\in\mathbb{R}$ such that if $x\geq K$ then $|g(x)|\leq P\cdot|f(x)|$, which gives $1/P\leq|f(x)/g(x)|$.

The question's assumption is that $\lim_{x\to\infty}f(x)/g(x)$ exists and equals 0. Fix $\varepsilon=1/(P+1)$. Then there exists a constant $M\in\mathbb{R}$ so that if $x\geq M$ then $|f(x)/g(x)|\leq 1/(P+1)$.

Those two paragraphs conflict for x's larger than the maximum of K and M.

V.1.65 First use L'Hôpital's Rule to get that if $g(x)=a_mx^m+\cdots+a_0$ then g is $\Theta(x^m)$. Then Example 1.22 shows that any logarithmic function is $\mathcal{O}(g)$.

For the other half, fix a base $b>1$ and consider the exponential function $h(x)=b^x$. L'Hôpital's Rule gives this

$$\lim_{x\to\infty}\frac{x^m}{b^x}=\frac{m}{\ln(b)}\cdot\lim_{x\to\infty}\frac{x^{m-1}}{b^x}=\frac{m(m-1)}{(\ln(b))^2}\cdot\lim_{x\to\infty}\frac{x^{m-2}}{b^x}=\cdots=\frac{m!}{(\ln(b))^m}\cdot\lim_{x\to\infty}\frac{1}{b^x}=0$$

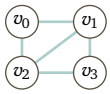
and Theorem 1.17 gives the desired conclusion.

V.2.41 For the octahedron, the path $\langle v_0, v_1, v_5, v_4, v_3, v_2, v_0\rangle$ will do. For the icosahedron, one path is $\langle v_0, v_{10}, v_{11}, v_8, v_9, v_6, v_7, v_2, v_3, v_4, v_5, v_1, v_0\rangle$.

V.2.42 Traversing the Hamiltonian path means that there is one and only one edge leaving each vertex. So if a graph has n vertices, $v_0, \ldots v_{n-1}$, then a Hamiltonian path has the same number of edges, n.

V.2.43

(A) This graph has a Hamiltonian circuit, around the outside.



To include the vertices v_0 and v_3, any circuit must include those four outside edges. In every Hamiltonian circuit the number of edges equals the number of vertices and hence no Hamiltonian circuit can contain the edge v_1v_2, because that would make five edges.

(B) Assume that all the edge weights are positive (if the given graph does not have that then to all weights, add a number large enough to make them all positive). To the edge e assign this weight.

$$-(\text{number of vertices}) \cdot (\text{max weight}) - 1$$

Every Hamiltonian path has the same number of edges, specifically the number of edges equals the number of vertices. So a path through e has a negative total weight, and this is clearly the only way to get a negative total weight, so the Hamiltonian path solving the Traveling Salesman must include e.

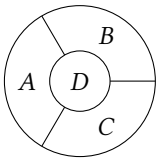
V.2.44 We can do these by eye.

(A) The shortest is q_2 to q_1 to q_7, which has length 14.

(B) The shortest is q_0 to q_3 to q_4 to q_8, of length 23.

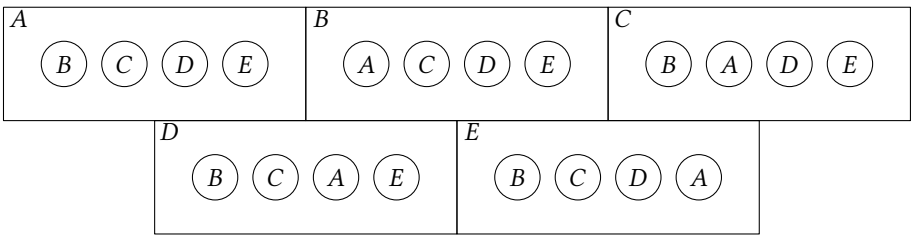
(C) There is no path between these two vertices.

V.2.45 This is the natural one. Every region borders all three others so three colors does not suffice.



V.2.46

(A) This map shows five countries. They are not contiguous; for instance, country A has five separate components.



Because of the portion in the upper left, country A must be a different color than any of the others. The other portions similarly give that all the other countries are different colors than each other. In total there are five countries, each a different color than all the others.

(B) Make a circle and split it into five wedges.

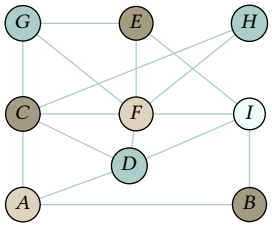
V.2.47

(A) Suppose that a three-coloring suffices. Then the three vertices A, C, and D form a triangle, so they must be different colors. Since CDF is also a triangle, we conclude that A and F are the same colors. So far we have: $\mathcal{C}_0 = \{A, F\}$, $\mathcal{C}_1 = \{C\}$, $\mathcal{C}_2 = \{D\}$.

Next, consider triangle DFI. It's presence gives that C and I are the same colors. So we have $\mathcal{C}_0 = \{A, F\}$, $\mathcal{C}_1 = \{C, I\}$, $\mathcal{C}_2 = \{D\}$. Then triangle EFI leads to $\mathcal{C}_0 = \{A, F\}$, $\mathcal{C}_1 = \{C, I\}$, $\mathcal{C}_2 = \{D, E\}$.

Finally, triangle CFG gives that D and G are the same color, giving $\mathcal{C}_0 = \{A, F\}$, $\mathcal{C}_1 = \{C, I\}$, $\mathcal{C}_2 = \{D, E, G\}$. But that's impossible because G is connected to E.

(B) This is a four-coloring.



$C_1 = \{A, F\}$
$C_2 = \{B, C, E\}$
$C_3 = \{D, G, H\}$
$C_4 = \{I\}$

V.2.48 A formula is satisfiable if there is a T in the truth table's final column.

(A) This formula is satisfiable.

P	Q	R	$P \wedge Q$	$\neg Q$	$\neg Q \wedge R$	$(P \wedge Q) \vee (\neg Q \wedge R)$
F	F	F	F	T	F	F
F	F	T	F	T	T	T
F	T	F	F	F	F	F
F	T	T	F	F	F	F
T	F	F	F	T	F	F
T	F	T	F	T	T	T
T	T	F	T	F	F	T
T	T	T	T	F	F	T

(B) This formula is not satisfiable.

P	Q	$P \to Q$	$P \wedge Q$	$\neg P$	$(P \wedge Q) \vee \neg P$	$\neg(P \wedge Q) \vee \neg P)$	$(P \to Q) \wedge \neg((P \wedge Q) \vee \neg P)$
F	F	T	F	T	T	F	F
F	T	T	F	T	T	F	F
T	F	F	F	F	F	T	F
T	T	T	T	F	T	F	F

V.2.49

(A) There are four lines where the input causes the expression to evaluate to F.

Line	*Clause*
F-T-F	$P \vee \neg Q \vee R$
F-T-T	$P \vee \neg Q \vee \neg R$
T-F-T	$\neg P \vee Q \vee \neg R$
T-T-T	$\neg P \vee \neg Q \vee \neg R$

(B) Any clause where the elements are joined by $\vee$'s will evaluate to T unless every single element is F. These clauses are constructed is to make every element false on the targeted line. On any other line at least one element is true.

(C) The Conjunctive Normal form is this.

$$(P \vee \neg Q \vee R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee \neg R) \wedge (\neg P \vee \neg Q \vee \neg R)$$

This statement evaluates to F if and only if any of its constituent clauses is F. By the prior item, that happens if and only if the input is a targeted line.

V.2.50 It does not imply that. For an example, let the Boolean expression be $P \vee Q$.

P	Q	$P \vee Q$	$\neg(P \vee Q)$
F	F	F	T
F	T	T	F
T	F	T	F
T	T	T	F

Combination of five vertices					*A missing edge*
v_0	v_1	v_2	v_3	v_4	v_1v_3
v_0	v_1	v_2	v_3	v_5	v_2v_5
v_0	v_1	v_2	v_3	v_6	v_2v_6
v_0	v_1	v_2	v_4	v_5	v_0v_4
v_0	v_1	v_2	v_4	v_6	v_0v_6
v_0	v_1	v_2	v_5	v_6	v_0v_6
v_0	v_1	v_3	v_4	v_5	v_0v_4
v_0	v_1	v_3	v_4	v_6	v_0v_6
v_0	v_1	v_3	v_5	v_6	v_0v_6
v_0	v_1	v_4	v_5	v_6	v_0v_6
v_0	v_2	v_3	v_4	v_5	v_0v_4

Combination of five vertices					*A missing edge*
v_0	v_2	v_3	v_4	v_6	v_0v_4
v_0	v_2	v_3	v_5	v_6	v_0v_6
v_0	v_2	v_4	v_5	v_6	v_0v_4
v_0	v_3	v_4	v_5	v_6	v_0v_4
v_1	v_2	v_3	v_4	v_5	v_1v_3
v_1	v_2	v_3	v_4	v_6	v_1v_3
v_1	v_2	v_3	v_5	v_6	v_1v_3
v_1	v_2	v_4	v_5	v_6	v_1v_5
v_1	v_3	v_4	v_5	v_6	v_1v_3
v_2	v_3	v_4	v_5	v_6	v_2v_5

FIGURE 98, FOR QUESTION V.2.58: A check of all possible 5-cliques.

It is satisfiable by taking both variables to be T. Its negation is also satisfiable, by taking both variables to be F

V.2.51 A finite set of propositional logic statements $\{S_0, \ldots S_{k-1}\}$ is satisfiable if and only if the single statement $S_0 \wedge S_1 \ldots \wedge S_{k-1}$ is satisfiable.

V.2.52

(A) AT&T connects to Citigroup, which connects to Ford Motor.

(B) Haliburton connects to Citigroup, which connects to Ford Motor.

(C) There is no connection between Caterpillar and Ford Motor.

(D) Also no connection.

V.2.53 (A) 2 (B) 2 (C) 0 (D) 4 (I was only in the deleted scenes. But all this is just for fun.)

V.2.54

(A) A vertex cover with $B = 2$ elements is $S = \{q_1, q_3\}$. An independent set with $\hat{B} = 4$ elements is $\hat{S} = \{q_0, q_2, q_4, q_5\}$.

(B) A vertex cover with $B = 3$ elements is $S = \{q_1, q_2, q_4\}$. An independent set with $\hat{B} = 3$ elements is $\hat{S} = \{q_0, q_3, q_5\}$.

(C) A vertex cover with $B = 4$ elements is $S = \{q_2, q_5, q_8, q_9\}$. An independent set with $\hat{B} = 6$ elements is $\hat{S} = \{q_0, q_1, q_3, q_4, q_6, q_7\}$.

(D) For any edge, if it has at least one endpoint in S then it has at most one endpoint in the complement $\hat{S} = \mathcal{N} - S$. Conversely, if an edge has at most one endpoint in $\hat{S}$ then it has at least one endpoint in the complement $S = \mathcal{N} - \hat{S}$.

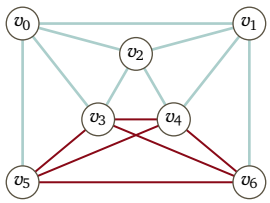
V.2.55 A 3-clique is a triangle. A 2-clique is an edge.

V.2.56 A k-clique has $k(k-1)/2$ edges, one for each combination of two vertices.

V.2.57 We could list all triples of vertices and then check each to see whether it is in the graph. But instead this list is by eye.

$$\{v_0, v_1, v_2\}, \{v_0, v_1, v_6\}, \{v_0, v_2, v_6\}, \{v_2, v_3, v_4\}, \{v_2, v_3, v_5\}, \{v_2, v_4, v_5\}, \{v_3, v_4, v_5\}$$

V.2.58 It has a 4-clique, and thus it necessarily has a 3-clique.



But it has no 5-clique, as checked by exhaustion in Figure 98 on page 186.

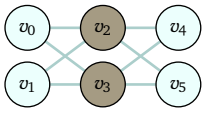
V.2.59 The one on the right does not partition the vertices into two disjoint sets.

V.2.60 This graph cut gives a cut set with eight elements.

21	33	49	42	19	*Total*
N	N	N	N	N	0
N	N	N	N	Y	19
N	N	N	Y	N	42
N	N	N	Y	Y	61
N	N	Y	N	N	49
N	N	Y	N	Y	68
N	N	Y	Y	N	91
N	N	Y	Y	Y	110
N	Y	N	N	N	33
N	Y	N	N	Y	52
N	Y	N	Y	N	75
N	Y	N	Y	Y	94
N	Y	Y	N	N	82
N	Y	Y	N	Y	101
N	Y	Y	Y	N	124
N	Y	Y	Y	Y	143

21	33	49	42	19	*Total*
Y	N	N	N	N	21
Y	N	N	N	Y	40
Y	N	N	Y	N	63
Y	N	N	Y	Y	82
Y	N	Y	N	N	70
Y	N	Y	N	Y	89
Y	N	Y	Y	N	112
Y	N	Y	Y	Y	131
Y	Y	N	N	N	54
Y	Y	N	N	Y	73
Y	Y	N	Y	N	96
Y	Y	N	Y	Y	115
Y	Y	Y	N	N	103
Y	Y	Y	N	Y	122
Y	Y	Y	Y	N	145
Y	Y	Y	Y	Y	164

FIGURE 99, FOR QUESTION V.2.63: All possibilities in the Subset Sum instance.



It is as large as possible because it contains all of the edges.

V.2.61 The definition of a Three-dimensional Matching problem requires three sets, here the instructors, the courses, and the time slots, and that they all have the same size, here five. We must be given as input a set $M \subseteq X \times Y \times Z$, and here that set consists of all triples that the graph shows as a possibility, so $M = \{ (A, 1, \beta), (A, 1, \varepsilon), (A, 2, \gamma), \ldots \}$. We must decide if there is a set $\hat{M} \subseteq M$ containing n elements (here, five elements) such that no two of the triples in $\hat{M}$ agree on their first coordinates, or their second or third coordinates either. That fits the problem.

A suitable matching is $\hat{M} = \{ \langle A, 2, \varepsilon \rangle, \langle B, 0, \alpha \rangle, \langle C, 3, \gamma \rangle, \langle D, 1, \beta \rangle, \langle E, 4, \delta \rangle \}$.

V.2.62

(A) The set $M = X \times Y \times Z$ contains these twenty seven triples $\langle x, y, z \rangle$.

$z =$ a			$z =$ d			$z =$ e		
⟨a, b, a⟩	⟨a, c, a⟩	⟨a, d, a⟩	⟨a, b, d⟩	⟨a, c, d⟩	⟨a, d, d⟩	⟨a, b, e⟩	⟨a, c, e⟩	⟨a, d, e⟩
⟨b, b, a⟩	⟨b, c, a⟩	⟨b, d, a⟩	⟨b, b, d⟩	⟨b, c, d⟩	⟨b, d, d⟩	⟨b, b, e⟩	⟨b, c, e⟩	⟨b, d, e⟩
⟨c, b, a⟩	⟨c, c, a⟩	⟨c, d, a⟩	⟨c, b, d⟩	⟨c, c, d⟩	⟨c, d, d⟩	⟨c, b, e⟩	⟨c, c, e⟩	⟨c, d, e⟩

(B) There are lots of correct answers. One is $\hat{M} = \{ \langle \mathsf{a}, \mathsf{b}, \mathsf{a} \rangle, \langle \mathsf{b}, \mathsf{c}, \mathsf{d} \rangle, \langle \mathsf{c}, \mathsf{d}, \mathsf{e} \rangle \}$.

V.2.63 Figure 99 on page 187 is a check of every possibility.

V.2.64 Figure 100 on page 188 shows the result of writing a small script.

V.2.65 There are a total of 538 electoral votes. A 269-269 tie is easy to concoct. For example, the electoral votes of CA, TX, FL, NY, IL, PA, OH, GA, MI, NC, VA, and VT add to 269.

V.2.66 There are twenty five: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, and 97.

V.2.67

(A) Yes, it is prime.

(B) No, it is not prime. Besides the trivial ones, the divisors of 6 165 are 3, 5, 9, 15, 45, 137, 411, 685, 1 233, and 2 055.

(C) Yes.

(D) No. Besides itself and 1, this number is also divisible by 7 and 601.

a	*b*	*c*	*d*	*e*	*Weight*	*Value*
N	N	N	N	N	0	0
N	N	N	N	Y	19	40
N	N	N	Y	N	42	44
N	N	N	Y	Y	61	84
N	N	Y	N	N	49	34
N	N	Y	N	Y	68	74
N	N	Y	Y	N	91	78
N	N	Y	Y	Y	110	118
N	Y	N	N	N	33	48
N	Y	N	N	Y	52	88
N	Y	N	Y	N	75	92
N	Y	N	Y	Y	94	132
N	Y	Y	N	N	82	82
N	Y	Y	N	Y	101	122
N	Y	Y	Y	N	124	126
N	Y	Y	Y	Y	143	166

a	*b*	*c*	*d*	*e*	*Weight*	*Value*
Y	N	N	N	N	21	50
Y	N	N	N	Y	40	90
Y	N	N	Y	N	63	94
Y	N	N	Y	Y	82	134
Y	N	Y	N	N	70	84
Y	N	Y	N	Y	89	124
Y	N	Y	Y	N	112	128
Y	N	Y	Y	Y	131	168
Y	Y	N	N	N	54	98
Y	Y	N	N	Y	73	138
Y	Y	N	Y	N	96	142
Y	Y	N	Y	Y	115	182
Y	Y	Y	N	N	103	132
Y	Y	Y	N	Y	122	172
Y	Y	Y	Y	N	145	176
Y	Y	Y	Y	Y	164	216

FIGURE 100, FOR QUESTION V.2.64: Output of a brute force script to show that there is no solution of an instance of the Knapsack problem.

(E) No. Besides the trivial divisors of itself and 1, this number also has the divisors 3, 11, 33, 233, 699, and 2 563.

V.2.68

(A) One is 3. The list of nontrivial divisors is 3, 9, 3 469, and 10 407.

(B) One is 2. The list of nontrivial ones is 2, 4, 8, 6 553, 13 106, and 26 212.

(C) One is 2. The list of nontrivial divisors is 2, 3, 4, 5, 6, 8, 10, 12, 15, 16, 20, 24, 25, 30, 32, 40, 48, 50, 60, 64, 75, 80, 96, 100, 120, 128, 150, 160, 192, 200, 240, 300, 320, 384, 400, 480, 600, 640, 800, 960, 1 200, 1 600, 1 920, 2 400, 3 200, and 4 800.

(D) One is 61. The list of nontrivial divisors is 61, 71.

(E) The only ones are 1 and 877. It is prime.

V.2.69 The AKS algorithm that solves Primality answers 'yes' or 'no' without finding divisors.

V.3.9 *Note:* the meanings give here are common usage. But because these terms are not formally defined, different authors may use them in slightly different ways.

(A) A heuristic is a shortcut, an approach to a problem that is more reliable than simple guessing and is faster than a full solution (or perhaps no full solution is possible). But usually the heuristic is not a theoretically correct full solution.

One example is approaching the Traveling Salesman problem by using a greedy algorithm, that is, always going next to the nearest not-yet-visited city. Another example is that antivirus programs use heuristics to try and determine if an executable is malicious, such as comparing the first few bytes to a list of known-bad ones.

A third example is based on the data in Figure 101 on page 189. In the contests for US President since 1900, the taller candidate has won 21 out of 31 times, for a sample proportion of 0.68 (not counting 1992). The chance of such a result by happenstance is less than four percent. You could decide to bet in the next election on the taller candidate. This isn't logical, but it is a method and it is used in practice.

So, a heuristic is a method that, while it may be used in practice, is not guaranteed to be optimal, perfect, or even rational. In contrast, an algorithm for a problem is guaranteed to solve that problem.

(B) Pseudocode is a description of an algorithm. It gives a sketch or or outline, usually at a quite high level. It is often laid out as though it were compilable code but usually it does not conform to any specific language's syntax; for instance it typically does not include details such as declaration of variables.

This is pseudocode for the recursive depth-first traversal of a binary tree.

Year	*Winner*	*Other*	*Tallest?*
1900	W McKinley	W J Bryan	N
1904	T Roosevelt	A B Parker	Y
1908	W H Taft	W J Bryan	Y
1912	W Wilson	–2 others–	N
1916	W Wilson	C E Hughes	Y
1920	W G Harding	J M Cox	Y
1924	C Coolidge	J W Davis	N
1928	H Hoover	A Smith	Y
1932	F D Roosevelt	H Hoover	Y
1936	F D Roosevelt	A Landon	Y
1940	F D Roosevelt	W Willkie	N
1944	F D Roosevelt	T Dewey	Y
1948	H S Truman	T Dewey	Y
1952	D Eisenhower	A Stevenson II	Y
1956	D Eisenhower	A Stevenson II	Y
1960	J F Kennedy	R Nixon	Y
1964	L B Johnson	B Goldwater	Y
1968	R Nixon	H Humphrey	Y
1972	R Nixon	G McGovern	N
1976	J Carter	G Ford	N
1980	R Reagan	J Carter	Y
1984	R Reagan	W Mondale	Y
1990	G H W Bush	M Dukakis	Y
1992	W Clinton	G H W Bush	–same–
1996	W Clinton	R Dole	Y
2000	G W Bush	A Gore	N
2004	G W Bush	J Kerry	N
2008	B Obama	J McCain	Y
2012	B Obama	M Romney	N
2016	D Trump	H Clinton	Y
2020	J Biden	D Trump	N
2024	D Trump	K Harris	Y

FIGURE 101, FOR QUESTION V.3.9: Height comparison of US Presidential winner and main opponent, since 1900.

```
process_subtree(v)
  process_subtree(v's left child)
  process_vertex(v)
  process_subtree(v's right child)
```

Another example describes the most simple algorithm for checking whether a number is prime, by dividing it by each natural smaller than it but bigger than one.

```
is_prime(n)
  for 2 <= i < n
    if i divides n
      return False
  return True
```

We have seen many other examples in this book, often given as flowcharts.

So pseudocode is close to a specification of an algorithm. But in ordinary usage, an algorithm is completely specified so that an implementing programmer does not need to work out any significant details, while pseudocode may leave some details unspecified. (An analogy: an experienced cook may describe making Scalloped Potatoes as, "bake thin-sliced potatoes a white sauce with breadcrumbs on top." That is not complete in that it depends on the implementer's understanding a number of details, but it is headed in the direction of a recipe.)

(C) Of course, in the first section of the first chapter we saw a number of Turing machines, such as one that computes the predecessor function, another that computes the sum of two inputs, and a third that never halts.

A Turing machine is more like a program than an algorithm, in that it is an implementation of an algorithm. For instance, a programmer may have the Bubble Sort algorithm in mind and then implement it as a Turing machine.

(D) A flowchart is graphical pseudocode. It describes an algorithm, typically at a high level (often even higher than that used for pseudocode). We have seen a number of flowcharts in this book, including in this chapter. Flowcharts differ from algorithms in the same way that pseudocode does.

(E) Source code differs from an algorithm in that is an implementation of an algorithm (sometimes a number of algorithms are bundled together to make one source). Source is targeted at a specific compiler or interpreter, and perhaps at a specific hardware and software platform.

For example, there is source code in Racket that implements the algorithm to determine if an input natural number n is prime by trying to divide by all numbers less than n.

(F) An executable is a binary file that can be run on an operating system. It is usually the result of compiling

source code, and so is an implementation of an algorithm. Three examples are the executable that results from compiling (and linking) the "Hello World!" program in C, the executable for your browser, and the executable that performs the typesetting code in LaTeX.

(G) A process is an instance of a program that has been has been loaded into a machine's memory for execution. As such, it is an implementation of an algorithm (or perhaps a number of algorithms). Three examples are the result of calling the executable of the "Hello World!" program, of calling the executable for your browser, and of invoking the executable for LaTeX.

Some processes, such as an operating system shell or a web server, are be designed to run forever, while typically in computational complexity discussions the term 'algorithm' is used for something that is guaranteed to halt.

V.3.10 A solution is an algorithm that acts as the characteristic function of the set.

V.3.11 A decision problem is any one that requires a 'yes' or 'no' answer. A language decision problem is a specialization of that, to 'yes' or 'no' answers about membership in the given language. (However, we often take languages as precise descriptions of the problems that we are working on and the two terms may be casually conflated sometimes.)

V.3.12

(A) The multiplication is easy: $(1/0.01)^2 \cdot 21 \cdot 30 = 6.30 \times 10^6$.

(B) The answer is $\lceil \lg(1\,000\,000) \rceil$. Doing the calculation with natural log (which can be done on a pocket calculator), we get $6 \ln(10)/\ln(2) \approx 19.93$, so the answer is 20 bits per pixel.

(C) Multiply $(6.30 \times 10^6) \cdot 20 = 1.26 \times 10^8$. That's less than 16 megabytes, about eight books from Project Gutenberg.

V.3.13 One is the smell of a baby's head. They have a great smell. But it is hard to see how to compute it.

V.3.14 False, the converse does not hold; two programs computing the same function need not implement the same algorithm. For instance, two programs may both sort input strings but one uses Bubble Sort while the other uses Quick Sort.

V.3.15

(A) Recognizing a correctly sorted string is a different task than finding a good way (or indeed the best way) to sort an unsorted string.

(B) Similarly, recognizing a correct permutation feels different — and much easier — than generating such a permutation when given an unsorted string.

V.3.16 These algorithms signal that the input is accepted by returning 1 and signal that is it not accepted by returning 0.

(A) Because the number 4 is a square that is one greater than a prime, three members are $\langle 0, 4 \rangle$, $\langle 1, 3 \rangle$, and $\langle 2, 2 \rangle$, For an algorithm computing the characteristic function, first input the sequence and unpack it to get $n, m \in \mathbb{N}$. Then add the two numbers. If the sum is both square and one greater than a prime then return 1, otherwise return 0.

(B) Three string members of $\mathcal{L}$ are $\sigma_0 = \texttt{200}$, $\sigma_1 = \texttt{100}$, and $\sigma_2 = \texttt{300}$. The algorithm is: if the input is a single `0` or ends in two `0`'s, then return 1. Otherwise return 0.

(C) The language $\mathcal{L}_2$ contains 11, 110, and 1. A natural algorithm is to scan the input string, keeping a running count of the number of `0`'s and 1's. At the end if there are more 1's then return 1, otherwise return 0.

(D) Recall that strings equal to their reverse are called palindromes. Three palindromes over $\mathbb{B}^*$ are 101, 11, and 1. To compute the characteristic function, read and save the input string. Reverse the string and compare it with the original. If it matches then return 1 and otherwise return 0.

V.3.17 These algorithms return 1 if the input is accepted and 0 if it is not.

(A) The empty language is $\mathcal{L} = \varnothing = \{\,\}$. Its characteristic function is $\mathbb{1}_{\mathcal{L}}(n) = 0$. The algorithm to solve this language decision problem is to, for all input, return 0.

(B) This language contains two strings, $\mathbb{B} = \{\,\texttt{0}, \texttt{1}\,\}$ (technically these are characters, not length one strings, but in this book we don't worry too much about the difference). The characteristic function is this.

$$\mathbb{1}_{\mathbb{B}}(\sigma) = \begin{cases} 1 & \text{– if } \sigma = \texttt{0} \text{ or } \sigma = \texttt{1} \\ 0 & \text{– otherwise} \end{cases}$$

The algorithm computing that function is obvious.

(C) The language $\mathbb{B}^*$ contains all of the bitstrings. The algorithm that solves the decision problem for this language will, given a bitstring, return 1.

V.3.18 These algorithms signal acceptance by returning 1 and signal rejection with 0.

(A) For the algorithm, fix a Finite State machine $\mathcal{M}$ that recognizes the language described by a∗ba∗. Read the input and simulate running $\mathcal{M}$ on that input. This will terminate in a finite number of steps. If $\mathcal{M}$ ends in an accepting state then return 1, otherwise return 0.

(B) The grammar defines the language $\mathcal{L} = \{ \mathsf{a}^n b^m \bigm| n, m \in \mathbb{N} \}$. The algorithm is: input the string and if all of the b's come after all of the a's then return 1. Otherwise, return 0.

V.3.19 These algorithms signal acceptance by returning 1 and signal rejection with 0.

First observe that there is an algorithm that, given a machine $\mathcal{M}$ and a state q in that machine, finds the set of states reachable from q. The algorithm proceeds in steps. At step 1, initialize by taking $R_0 = \{q\}$. Step $i+1$ begins with the set of states that are reachable from q with i-many transitions or fewer. In that step, for each state in R_i, find if any states reachable from it in one transition in R_i. Make the set R_{i+1} by adding them all to R_i. If there are no such new states then the algorithm stops. This algorithm must terminate at some step because the number of states in the machine is finite.

(A) The language is empty if and only if from the start state, no accepting state is reachable.

(B) The language is infinite if and only if there is a state $\hat{q}$ that can be reached from the initial state and is such that $\hat{q}$ can be reached from itself by a nonempty string and some accepting state can be reached from $\hat{q}$.

(C) Given $\mathcal{M}$, obtain a new machine $\hat{\mathcal{M}}$ that recognizes the complement of $\mathcal{L}(\mathcal{M})$ by copying $\mathcal{M}$, except setting each accepting state of $\mathcal{M}$ to be non-accepting in $\hat{\mathcal{M}}$, and setting each non-accepting state of $\mathcal{M}$ to be accepting in $\hat{\mathcal{M}}$. Then $\mathcal{L}(\mathcal{M})$ is empty if and only if $\mathcal{L}(\hat{\mathcal{M}})$ is empty, and we have reduced to this exercise's first item.

V.3.20

(A) See if the input string starts with a 1 bit.

(B) See if the input string has length less than or equal to ten. If the length equals ten then there are a fixed number of cases to do ($1000 - 2^9 = 488$ of them) but that doesn't change that the algorithm is $\mathcal{O}(1)$.

V.3.21 This is a search problem since we need only produce a single bridge.

V.3.22 As remarked in the section, a decision problem involves finding a function — a Boolean function. So in that sense, multiple answers are possible.

(A) This is a yes-or-no question, so it is a decision problem.

(B) Function problem.

(C) Decision problem.

(D) Search problem.

(E) Search problem.

(F) Decision problem.

(G) Language decision problem, for $\mathcal{L} = \{ \sigma \in \mathbb{B}^* \bigm| \sigma \text{ represents a number that in decimal has only odd digits} \}$.

V.3.23

(A) Decision problem.

(B) Search problem.

(C) Function problem.

(D) Function problem (although it is a search problem in that any satisfying assignment at all will do for output). The 'F' in F-SAT stands for 'functional'.

(E) Decision problem.

V.3.24

(A) Search problem.

(B) Decision problem.

(C) Search problem.

(D) Optimization problem.

V.3.25

(A) A language suited to machines with unary input is $\mathcal{L}_0 = \{ 1^n \bigm| n \in \mathbb{N} \text{ is a square} \}$. A language suited to

more everyday-style input is $\mathcal{L}_1 = \{ d \in \{ \texttt{0}, \ldots \texttt{9} \}^* \mid d \text{ represents a perfect square in base } 10 \}$.

(B) One language is the set of triples $\langle x, y, z \rangle \in (\{ \texttt{0}, \ldots \texttt{9} \}^*)^3$ such that where x, y, z represent integers, the squares of the first two integers add to the square of the third.

(C) A language is $\{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents a graph } \mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle \text{ where the number of edges in } \mathcal{E} \text{ is even} \}$.

(D) The language is $\{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents a pair } \langle \mathcal{G}, p \rangle \text{ where } p \text{ is a path with a repeated vertex} \}$.

V.3.26

(A) $\{ n \in \mathbb{N} \mid \text{the factors of } n \text{ sum to more than } 2n \}$.

(B) $\{ \langle \mathcal{P}, i \rangle \mid \text{machine } \mathcal{P} \text{ on input } i \text{ halts in less than ten steps} \}$

(C) $\{ E \mid \text{the logic expression } E \text{ can be satisfied by three distinct assignments} \}$

(D) $\{ \langle \mathcal{G}, B \rangle \mid \text{for all } v_0, v_1 \in \mathcal{G} \text{ there is a path of cost less than } B \}$

V.3.27

(A) The Graph Colorability problem is stated as a decision problem, so we just need to give a suitable language.

$$\mathcal{L} = \{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents } \langle \mathcal{G}, k \rangle, \text{ where } \mathcal{G} \text{ is } k\text{-colorable} \}$$

Often that language would be written in this way

$$\mathcal{L} = \{ \langle \mathcal{G}, k \rangle \mid \mathcal{G} \text{ is } k\text{-colorable} \}$$

although that description does not explicitly mention the string representation asked for in this exercise.

(B) The version given in the text is a search problem. This is a language for the language decision problem version.

$$\{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents a graph that has a path that traverses each edge exactly once} \}$$

(C) The Shortest Path problem is given in the text as a function problem. This is a language for the language decision problem.

$$\{ \sigma \in \mathbb{B}^* \mid \sigma \text{ represents } \langle \mathcal{G}, v_0, v_1, p \rangle, \text{ where } p \text{ is a minimal-length path between the vertices} \}$$

V.3.28 It is about deciding membership in the language $\{ \langle \mathcal{P}_e, e \rangle \mid e \in \mathbb{N} \}$.

V.3.29 Note that this exercise does not call explicitly for language decision problems. It calls for decision problems.

(A) Given a triple $\langle n_0, n_1, p \rangle \in \mathbb{N}^3$, decide if $p = n_0 \cdot n_1$.

(B) Given a triple $\langle \mathcal{G}, v_0, v_1 \rangle$, where $\mathcal{G}$ is a weighted graph and $v_0 \neq v_1$ are vertices in that graph, decide if v_1 is a vertex that is nearest to v_0.

V.3.30 This family

$$\mathcal{L}_B = \{ \langle \mathcal{G}, v_0, v_1 \rangle \mid \text{There is a path in } \mathcal{G} \text{ from } v_0 \text{ to } v_1 \text{ of total weight less than or equal to } B \}$$

is parametrized by the bound B.

V.3.31

(A) We use $\mathcal{B}$ to denote a game board, a 4×4 array containing the numbers 1 through 14 along with a blank. Take $B \in \mathbb{N}$ as the parameter.

$$\mathcal{L}_B = \{ \mathcal{B} \mid \mathcal{B} \text{ can be solved in fewer than } B \text{ slide moves} \}$$

(B) Take $B \in \mathbb{N}$ as a bound. Write $\mathcal{R}$ for a Rubik's cube arrangement, a sequence of six 3×3 arrays populated by nine members of the set $C_0, \ldots$ nine members of the set C_5.

$$\mathcal{L}_B = \{ \mathcal{R} \mid \mathcal{R} \text{ can be solved in fewer than } B \text{ moves} \}$$

(C) Take $B \in \mathbb{R}^+ \cup \{0\}$. We can denote a way to finish a car with a sequence whose elements are jobs $C = \{ j_0, \ldots j_k \}$. With every such sequence there is a total time $t(C)$.

$$\mathcal{L}_B = \{ C \mid t(C) \leq B \}$$

V.3.32 A function version inputs a graph and outputs the set of every Hamiltonian circuit. A search version inputs a graph and outputs a Hamiltonian circuit, if one exists, and some nominal value such as `0` if no circuit exists.

An optimization version outputs a circuit that is best by some criterion. For instance, in a weighted graph it returns the circuit of least weight; this is the Traveling Salesman problem.

V.3.33 The empty set of vertices is independent and so is a set with one vertex. So every graph has a subset of its vertices that is independent.

(A) Because of the observation we can't sensibly say, "Given a graph, decide if it has an independent set." The answer would always be 'Yes'. We can instead use a parameter B: given a graph, decide if it has an independent set with B vertices.

(B) Use the language $\mathcal{L}_B = \{\mathcal{G} \mid \mathcal{G} \text{ has } B \text{ vertices that do not share an edge}\}$.

(C) Given a graph G, find an independent set with B-many vertices. (There may be more than one, or none, but if we find one then we are done.)

(D) Given a graph, return all independent sets of size B, or the string `None`.

(E) Given a graph, return an independent set that is maximal, that it is as large as possible for the graph.

V.3.34 Consider the problem of finding the maximum integer in a nonempty list. The natural decision problem, "Decide if the list has a maximum" is trivial since every list has a maximum (there may be a tie). But the natural search variant, "Find the maximum" appears to require looking through the list.

V.3.35 With an algorithm that can check any two vertices, we could check all pairs.

V.3.36 Imagine that we have a routine `shortest_path` that takes in a weighted graph and two vertices, and returns the shortest path between them. As a side effect, that solves the decision problem of whether there is any path at all.

V.3.37

(A) The smallest nontrivial divisor of a number is its smallest prime divisor.

(B) The search problem can return the smallest prime, so just take B to be n.

V.3.38 The decision variant takes in a set and answers 'Yes' or 'No' whether there is a subset that adds to the target T. Take the numbers in the set $S = \{s_0, \ldots s_{n-1}\}$ one at a time by considering the sequence $S_0 = \{s_0\}$, then $S_1 = \{s_0, s_1\}, \ldots$ When the decision variant finds a subset of $S_j = \{s_0, \ldots s_j\}$ on which it is able to say 'Yes', we know that one of the numbers in a solution set is s_j.

Iterate, using $\hat{S} = S - \{s_j\}$ and $\hat{T} = T - s_j$.

V.4.13 True. Such problems have solution algorithms that are basically a fixed number of `if ... then ...` branches, so there is a maximum number of steps that any input can trigger. Thus, the solution algorithm is $\mathcal{O}(1)$.

V.4.14 The two 'runs in polytime' and 'is in P' are different. Confusing them is a type error — the type of thing that runs in polytime is different than the type of thing that is in P. Algorithms are not in P; rather, problems are in P.

Specifically, a problem is in P if from among all the algorithms that solve it, at least one runs in polynomial time, so that it is $\mathcal{O}(n^c)$ for some constant c. Thus, better than what your coworker said is is to say something like, "I've got a problem whose solution algorithm takes polytime."

V.4.15 An order of growth is a set of functions. A complexity class is a set of problems, language decision problems.

V.4.16 Your friend is observing that the circuit representation may have length exponential in the circuit depth, since an r-deep circuit may have 2^r many vertices. But the definition of polynomial time is that the number of steps taken is a polynomial function of the size of the input representation, not of the number of circuit elements.

V.4.17 The student's machine accepts all inputs. The definition requires a machine that determines whether or not the input string is in the language and this machine doesn't do the 'or not'.

V.4.18 True; in Table 1.27 the logarithmic functions are before the polynomials.

V.4.19 True, although not by the same machine. Let $\mathcal{L}$ be computed by $\mathcal{P}$ so that the machine has this

input-output behavior.

$$\phi_{\mathcal{P}}(\sigma) = \mathbb{1}_{\mathcal{L}}(\sigma) = \begin{cases} 1 & \text{– if } \sigma \in \mathcal{L} \\ 0 & \text{– otherwise} \end{cases}$$

Modify the machine by interchanging the output 0's and 1's, to get a new machine $\hat{\mathcal{P}}$ with the complementary behavior.

$$\phi_{\hat{\mathcal{P}}}(\sigma) = \mathbb{1}_{\mathcal{L}^{\mathsf{C}}}(\sigma) = \begin{cases} 0 & \text{– if } \sigma \in \mathcal{L} \\ 1 & \text{– otherwise} \end{cases}$$

(By Church's Thesis because we can make such a modification in, say, Racket, we can make it on a Turing machine.)

V.4.20 A complexity class is just a set of languages. So the union of two is also a complexity class. The same holds for the intersection, and for the complement, where the complement is taken inside the universe $\mathscr{P}(\mathbb{B}^*)$.

V.4.21 Each computably enumerable language $\mathcal{L}$ satisfies that there is a computable function ϕ_e with $\mathcal{L} = \{k \in \mathbb{N} \;\big|\; k = \phi_e(i) \text{ for some input } i \in \mathbb{N}\}$. The set *RE* is a complexity class because it is a collection of languages.

V.4.22 Any straightforward algorithm will suffice. Here is one: given σ, find its length. Then for $i \in \{0, 1, \ldots |\sigma| - 1\}$, read the characters $\sigma[i]$ and compare them to the corresponding character read from the back, $\sigma[(|\sigma| - 1) - i]$. (We can stop once we get through the halfway point, $i = \lfloor |\sigma|/2 \rfloor$.) If each comparison gives equality then return 1, else return 0. This algorithm takes time polynomial in $|\sigma|$: finding the length takes $|\sigma|$ steps, and the comparison loop is a nested loop so it takes $|\sigma|^2$ steps.

V.4.23 We can express this problem as the language decision problem for $\mathcal{L} = \{\langle m, n\rangle \;\big|\; \text{the two are relatively prime}\}$. One way to solve it is to use Euclid's algorithm to find whether their greatest common divisor is 1. That algorithm runs in polynomial time (in fact, logarithmic time).

V.4.24 In each case we must produce an algorithm that runs in polynomial time.

(A) This is the problem: $\{\sigma \in \mathbb{B}^* \;\big|\; \sigma = \tau^3 = \tau \frown \tau \frown \tau \text{ for some } \tau \in \mathbb{B}^*\}$. Given σ, decide if its length is divisible by three (determining the length takes about $|\sigma|$ steps). If not, return 0. If the length is divisible by three then set τ to be the substring that is the first third of σ (finding τ takes about $|\sigma|$ steps). Then just check, by moving through the string once, whether $\sigma = \tau^3$ (this also takes $|\sigma|$ steps).

(B) Stated as a language decision problem, we have $\{i \in \mathbb{N} \;\big|\; \mathcal{P}_i \text{ halts on the empty string in 10 steps}\}$. There is some overhead in going from the input index i to the Turing machine $\mathcal{P}_i$, simulating it with a universal Turing machine, etc. But putting that aside as a constant overhead, no matter what is the input i, we then just run the simulation for ten steps. So this problem is an element of *P*.

V.4.25

(A) In the Clique problem, the size of the clique is a parameter. We are give a pair $\langle \mathcal{G}, B\rangle$ and asked if the graph has a B-sized clique. Here the size of the clique is fixed.

(B) Given $\mathcal{G}$, enumerate all triples $\langle v_0, v_1, v_2\rangle \in \mathcal{N}^3$ and check whether all three of v_0v_1, v_0v_2, and v_1v_2 are edges, members of $\mathcal{E}$. This takes polynomial time. Specifically, where $|\mathcal{N}| = n$, there are $\binom{n}{2}$ many pairs v_iv_j; that's about n^2 many. We check triples of pairs, so that's $\binom{n^2}{3}$, which is $\mathcal{O}(|\mathcal{N}|^6)$, which is polynomial. We could add a factor of $|\mathcal{E}|$ to do lookups, but in any event it all happens in polytime.

V.4.26

(A) To determine whether the string is in alphabetical order, a single pass through it suffices. For each item except the initial one, we compare it to the prior one. Since the alphabet is given as $\Sigma = \{\mathsf{a}, \ldots \mathsf{z}\}$, comparisons are possible in polytime (for instance, a 26×26 lookup table is enough), and thus overall this one-pass algorithm takes polytime.

(B) As mentioned in Problem 2.40, determining whether a number is composite or prime is known to be possible in polytime.

(C) We can give an algorithm that runs in time polynomial in the length of the graph description. Fix the set $S_0 = \{v_0\}$. Next, follow each edge leading out of v_0, to get the set of vertices that are connected to v_0 in

at most one step, $S_1 = S_0 \cup \{ v \mid \{ v_0, v \} \in \mathcal{E} \}$. After that, for all the vertices in S_1, follow all the edges to get the set $S_2 = S_1 \cup \{ v \mid \{ \hat{v}, v \} \in \mathcal{E} \text{ where } \hat{v} \in S_1 \}$ of all vertices that are connected to v_0 in at most two steps. This process must stop at some point, with $S_{i-1} = S_i$, because the graph is finite. Then, either $v_1 \in \S_i$ or not, which determines whether there is a path from v_0 to v_1. The number of steps in this algorithm, i, is bounded by the number of vertices in the graph, so the algorithm is polynomial in the size of the graph representation.

V.4.27

(A) By Dijkstra's algorithm, the Shortest Path problem can be done in polynomial time. So the problem is in *P*.

(B) Knapsack is believed to be quite hard (in a later section we will see that it is *NP* complete). No one knows of a polynomial algorithm.

(C) Euler Path is easy; we just need to check whether each node has even degree. There are algorithms that are linear.

(D) No one knows of a polynomial algorithm for Hamiltonian Circuit. (In a later section we will see that it is *NP* complete.)

V.4.28 Yes. The algorithm that takes in a single input and returns 0 computes the characteristic function of the empty language, and clearly runs in polytime.

V.4.29

(A) Our definition of complexity class is completely general — it is a collection of languages. Under that definition, the collection of regular languages is a complexity class.

(B) Yes. A language is regular if it is accepted by a Finite State machine. For every regular language we can make a Turing machine that simulates that Finite State machine and returns `0` or `1` to signal rejection or acceptance of the input strings. Each input string σ is processed by its Finite State machine in time $\mathcal{O}(|\sigma|)$ since the machine simply reads the character and passes to the indicated state. So the collection of regular languages is in *P*.

V.4.30 There are countably many Turing machines to use as deciders so *P* is countable.

V.4.31 No. Take $\mathcal{L}_0 = \emptyset$ and $\mathcal{L}_1 = \mathscr{P}(\mathbb{B}^*)$. Because *P* is countable, as there are only countably many Turing machines to use as deciders, and the collection of all languages $\mathscr{P}(\mathbb{B}^*)$ is uncountable, there are uncountably many languages with $\mathcal{L}_0 \subseteq \mathcal{L} \subseteq \mathcal{L}_1$. So at least one is not in *P*.

V.4.32 No. Certainly we can state the Halting problem as a language decision problem, using the language $\mathcal{L} = \{ n \in \mathbb{N} \mid \phi_n(n)\downarrow \}$. But to be a member of *P* there must be a Turing machine that solves the problem (and that runs in polytime). No Turing machine decides Halting problem and so it is not in *P*.

V.4.33 Here is the desired input-output behavior for the circuit.

b_2	b_1	b_0	$b_0 + b_1 + b_2$	$b_0 + b_1 + b_2 \pmod 2$
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	2	0
1	0	0	1	1
1	0	1	2	0
1	1	0	2	0
1	1	1	3	1

This propositional logic formula gives the behavior in that table (this is in Conjunctive Normal form).

$$(\neg b_2 \wedge \neg b_1 \wedge b_0) \vee (\neg b_2 \wedge b_1 \wedge \neg b_0) \vee (b_2 \wedge \neg b_1 \wedge \neg b_0) \vee (b_2 \wedge b_1 \wedge b_0)$$

Figure 102 on page 196 shows the circuit, using the standard notation of $\supset$ for 'and' gates, $\supset\!\!\!\supset$ for 'or' gates, and $\triangleright\!\!\circ$ for 'not' gates. But this circuit does not perfectly match the problem since some gates do not have two inputs and one output. Figure 103 on page 196 modifies the prior diagram to draw the circuit to take advantage of the fact that $x_0 \wedge x_1 \wedge x_2 = (x_0 \wedge x_1) \wedge x_2$, and that a similar relationship holds for '$\vee$'. It also includes an acyclic directed graph, by folding the 'not' gates into the boolean binary functions. These are the Boolean functions used there.

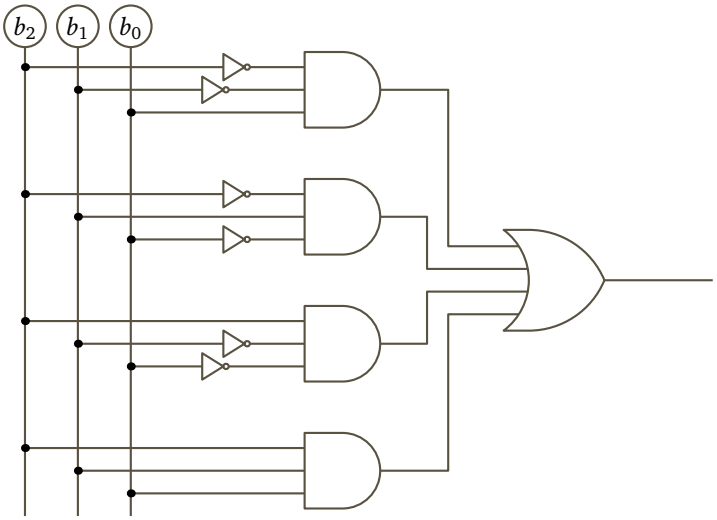


FIGURE 102, FOR QUESTION V.4.33: Initial circuit for the Circuit Evaluation problem.

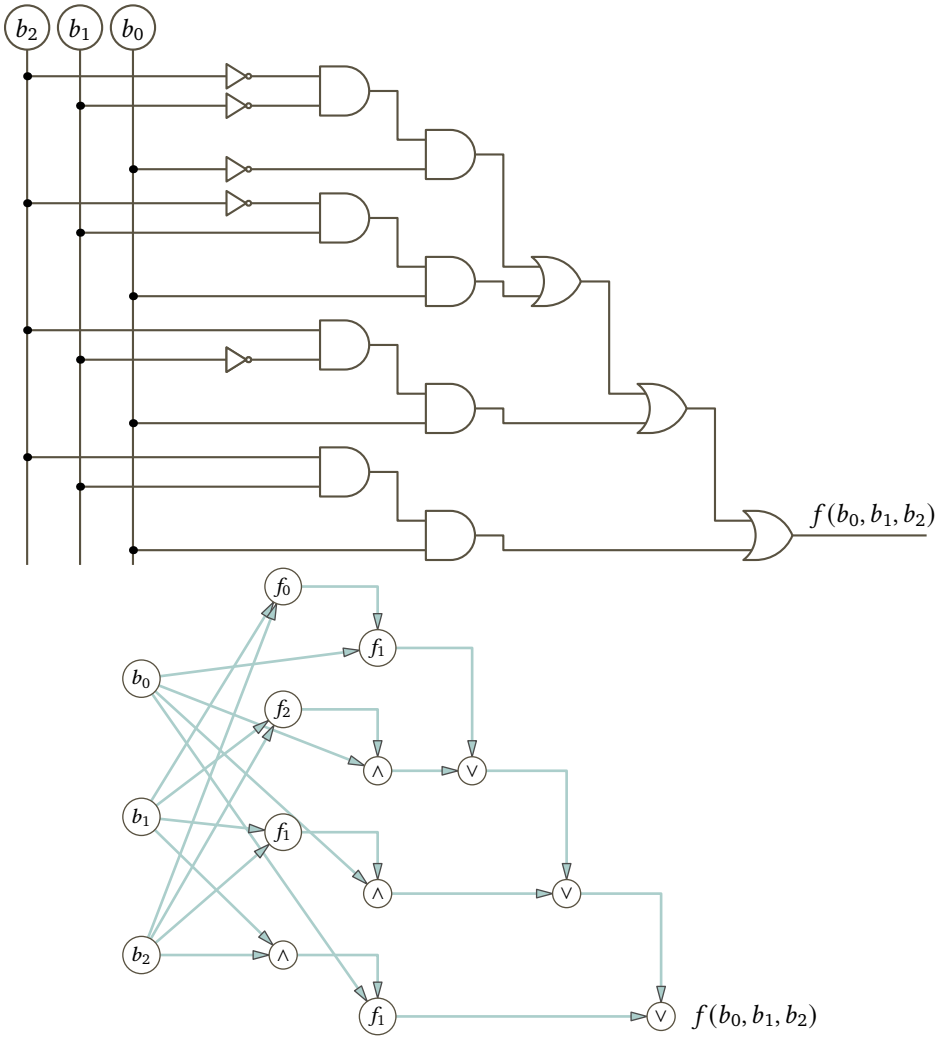


FIGURE 103, FOR QUESTION V.4.33: Revised circuit for the Circuit Evaluation problem and the associated DAG.

P	Q	$\wedge$
0	0	0
0	1	0
1	0	0
1	1	1

P	Q	$\vee$
0	0	0
0	1	1
1	0	1
1	1	1

P	Q	f_0
0	0	1
0	1	0
1	0	0
1	1	0

P	Q	f_1
0	0	0
0	1	0
1	0	1
1	1	0

P	Q	f_2
0	0	0
0	1	1
1	0	0
1	1	0

So, f_0 is equivalent to $\neg P \wedge \neg Q$, and f_1 is equivalent to $P \wedge \neg Q$, and f_2 is equivalent to $\neg P \wedge Q$.

V.4.34

(A) Suppose that $\mathcal{L}_0 \in P$ has the characteristic function ϕ_{e_0} that runs in time $\mathcal{O}(x^{k_0})$, and suppose that $\mathcal{L}_1$'s has the characteristic function ϕ_{e_1} that runs in time $\mathcal{O}(x^{k_1})$. The canonical algorithm for intersection is to first test the input for membership in $\mathcal{L}_0$ and if it passes go on to test for membership in $\mathcal{L}_1$. This algorithm is in $\mathcal{O}(x^k)$ where $k = \max(\{k_0, k_1\})$.

(B) We must prove that if a language $\mathcal{L}$ is in P then so is its set complement $\mathcal{L}^{\mathsf{c}}$. If $\mathcal{L} \in P$ then there is a deterministic Turing machine $\mathcal{P}$ that runs in polynomial time, and that accepts an input string $\sigma \in \mathbb{B}^*$ if and only if $\sigma \in \mathcal{L}$. (Recall that we take 'accepts' to mean that the machine halts, and when it halts there is only a 1 on the tape, while 'rejects' means there is only a 0.) Restated, there is a computable function $f : \mathbb{B}^* \to \mathbb{B}^*$ such that

$$f(\sigma) = \begin{cases} 0 & \text{– if } \sigma \notin \mathcal{L} \\ 1 & \text{– if } \sigma \in \mathcal{L} \end{cases}$$

and such that computing f is $\mathcal{O}(p)$ for some polynomial p. (More precisely, what is $\mathcal{O}(p)$ is the function $t(n)$, defined as the maximum runtime of $f(\tau)$ over all $\tau \in \mathbb{B}^*$ with $n = |\tau|$.)

Then clearly this function is also computable, and also in polytime: $\bar{f}(\sigma) = 1 - f(\sigma)$. Consequently, $\mathcal{L}^{\mathsf{c}} \in P$.

V.4.35 Fix a language $\mathcal{L} \in P$. Then there is a Turing machine that computes the characteristic function $f : \mathbb{B}^* \to \mathbb{B}^*$ of $\mathcal{L}$, with runtime $\mathcal{O}(x^n)$ for some power $n \in \mathbb{N}$. To compute the characteristic function of the reversal, $\mathcal{L}^{\mathsf{R}} = \{\tau^{\mathsf{R}} \mid \tau \in \mathcal{L}\}$, the algorithm is given an input string σ. It constructs the reversal, σ^{R}, and decides whether $\sigma^{\mathsf{R}} \in \mathcal{L}^{\mathsf{R}}$. The runtime analysis is: the algorithm can construct the reversal from the input in one pass through the input, and deciding whether that reversal string is in $\mathcal{L}$ is $\mathcal{O}(x^n)$, to in total the runtime is $\mathcal{O}(x^k)$ where $k = \max(\{1, n\})$.

V.4.36 Fix two languages, $\mathcal{L}_0, \mathcal{L}_1 \in \mathscr{P}(\mathbb{B}^*)$, that are members of P. Then there are powers $n_0, n_1 \in \mathbb{N}$ so that $\mathcal{L}_0$ is accepted by a Turing machine with runtime $\mathcal{O}(x^{n_0})$, and $\mathcal{L}_1$ is accepted by a Turing machine with runtime $\mathcal{O}(x^{n_1})$.

Here is the natural algorithm to decide if a given input string $\sigma \in \mathbb{B}^*$ is a member of $\mathcal{L}_0 \frown \mathcal{L}_1$. We will pass through the string, first decomposing it as $\sigma = \varepsilon \frown \sigma$, then as $\sigma = \sigma[0] \frown \sigma[1:]$, etc. At each stage of this pass we have $\sigma = \alpha \frown \beta$ and we check whether $\alpha \in \mathcal{L}_0$ and $\beta \in \mathcal{L}_1$.

Checking whether $\alpha \in \mathcal{L}_0$ takes time $\mathcal{O}(|\alpha|^{n_0})$, and checking whether $\beta \in \mathcal{L}_1$ takes time $\mathcal{O}(|\beta|^{n_1})$. It simplifies things to overestimate and use $|\sigma|$, rather than worry about the length of the substrings. Hence each step of the pass can be done in time that is $\mathcal{O}(|\sigma|^k)$ where $k = \max(\{n_0, n_1\})$. The pass itself takes $|\sigma| + 1$ many steps, so overall the algorithm runs in polytime, $\mathcal{O}(|\sigma|^m$ where $m = 1 + \max(\{n_0, n_1\})$.

V.4.37 Let two languages $\mathcal{L}_0, \mathcal{L}_1 \in \mathscr{P}(\mathbb{B}^*)$ be elements of P. Then there are computable functions to determine membership in the two with runtimes $\mathcal{O}(x^{n_0})$ and $\mathcal{O}(x^{n_1})$. The obvious algorithm to compute membership in the union gets as input a string $\sigma \in \mathbb{B}^*$. It first checks whether the string is a member of the first language and then checks whether it is a member of the second. Together, that takes a running time of $\mathcal{O}(x^{\max(n_0,n_1)})$, which means that the algorithm runs in polytime.

The argument for the union of any finite number $\mathcal{L}_0, \ldots \mathcal{L}_n$ of languages from P proceeds by induction. The base step is that $\mathcal{L}_0 \in P$ by assumption. The inductive step takes as the hypothesis that the union $\mathcal{L}_0 \cup \cdots \mathcal{L}_i$ is in P, for $0 < i < n$. Consider $\mathcal{L}_0 \cup \cdots \mathcal{L}_{i+1} = (\mathcal{L}_0 \cup \cdots \mathcal{L}_i) \cup \mathcal{L}_{i+1}$. This is the union of two members of P. By the first paragraph it is therefore a member of P.

As to a union of infinitely many, note that a single-element language $\mathcal{L} = \{n\}$ for $n \in \mathbb{N}$ is in P because a decider is basically a single `if .. then ..` statement and so runs in time $\mathcal{O}(1)$. Now any language is the union

of infinitely many single-element languages. This includes any language that is not in P. Hence, P is not closed under the union of infinitely many languages.

V.4.38 Lemma 4.12 says that P is closed under complementation. That is, if $\mathcal{L} \in P$ then $\mathcal{L}^{\mathtt{c}} \in P$. It follows immediately that $|P \subseteq coP$.

It also follows that $coP \subseteq P$ because if $L^{\mathtt{c}} \in P$ then since $\mathcal{L}^{\mathtt{cc}} = \mathcal{L}$, we have $\mathcal{L} \in P$.

V.5.16 That lemma requires that the verifier have the property that if $\sigma \in \mathcal{L}$ then there is a witness leading to acceptance, while if $\sigma \notin \mathcal{L}$ then there does not exists a witness that would cause the verifier to accept. The proposed verifier will accept an input σ if the bit tells it to, even if σ is not in the language.

For a concrete instance, consider the problem $\mathcal{L} = \{ n \in \mathbb{N} \mid n \text{ is even} \}$. The proposed verifier inputs a pair $\langle \sigma, \omega \rangle$, where σ is a number and ω is a bit, and then accepts the pair if $\omega = 1$. Absolutely, given the pair $\langle 5, 0 \rangle$, the verifier will not accept, which is correct here because $5 \notin \mathcal{L}$. However, given $\langle 5, 1 \rangle$, the verifier will accept. So, for $\sigma = 5$ there exists an ω leading to acceptance, which violates the definition of a verifier.

V.5.17 It is 'Every branch rejects'.

V.5.18 The easiest way to check satisfiability is to make a truth table.

(A) The expression $(P \wedge Q) \vee (\neg Q \wedge R)$ is satisfiable.

P	Q	R	$P \wedge Q$	$\neg Q$	$\neg Q \wedge R$	$(P \wedge Q) \vee (\neg Q \wedge R)$
F	F	F	F	T	F	F
F	F	T	F	T	T	T
F	T	F	F	F	F	F
F	T	T	F	F	F	F
T	F	F	F	T	F	F
T	F	T	F	T	T	T
T	T	F	T	F	F	T
T	T	T	T	F	F	T

(B) The expression $(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$ is not satisfiable.

P	Q	$P \rightarrow Q$	$P \wedge Q$	$\neg P$	$(P \wedge Q) \vee \neg P$	$\neg((P \wedge Q) \vee \neg P)$	$(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$
F	F	T	F	T	T	F	F
F	T	T	F	T	T	F	F
T	F	F	F	F	F	T	F
T	T	T	T	F	T	F	F

V.5.19 True, by Lemma 5.8.

V.5.20 This machine does not decide any language. To be a decider, a nondeterministic Turing machine's computation tree must have that all maximal paths are finite.

V.5.21

(A) Backtracking will solve the problem. But it is a deterministic algorithm. (Technically, a deterministic algorithm is a special case of a nondeterministic one, but when you ask about it, your professor says, "The point of this exercise is to demonstrate understanding of nondeterminism, not to quibble.")

(B) Your algorithm chooses, but it does so at random. If there are paths through the maze, but the random choices happens not to fall on such a path then your algorithm will not detect them. A nondeterministic algorithm is required to accept the input maze if there is a path and to not accept the input if there is not.

(C) We will give two answers, one using the unbounded parallelism imagery and one using choosing.

The unbounded parallelism algorithm maintains a computational history that is a tree. Whenever the algorithm finds itself at a place in the maze where it can do more than one thing, it forks child processes to handle each possibility. So it may have lots of branches, and lots of them may go to dead ends, but if there is a way through the maze then this algorithm will find it. So it accepts the input maze if there is a path and never accepts when there is not.

The choosing algorithm chooses. That is, the machine guesses the correct path, and then verifies it against the input maze. By definition, if there is a way for this guess to be right then the algorithm accepts. (Note again that this algorithm is not guessing at random. It is guessing, correctly.)

V.5.22 The algorithm inputs an array $\langle a_0, a_1, \ldots a_{n-1} \rangle$.

In the unbounded parallelism model, the algorithm forks a child process for each index $0 \leq i \leq n$. Child i checks whether $a_i = k$, and if so causes the algorithm to accept.

The guessing model is much the same. It guesses at an index i. If the array entry a_i equals k then it causes the algorithm to accept the input array. This model accepts the same languages as the other because, by definition, a guessing algorithm is right if there is a way for the guess to correctly result in acceptance, which will happen if and only if there is an array entry that equals k.

V.5.23 We recast this as language decision problems for a family of languages parametrized by the bound $B \in \mathbb{N}$.

$$\mathcal{L}_B = \{ \langle x_0, x_1, \ldots x_n \rangle \in \mathbb{Z}^{n+1} \;\big|\; \text{the } x_j \text{ satisfy all the constraints and } F(x_0, x_1, \ldots x_n) \leq B \}$$

An unboundedly parallel algorithm tries all possible assignments. If there is a satisfying assignment then this algorithm will find it and accept the input. If there is no satisfying assignment then there is no way for the input to get accepted.

A guessing algorithm guesses an assignment $\langle x_0, \ldots x_n \rangle$ and then verifies that $F(x_1, \ldots x_n) \leq B$. If there is a satisfying assignment then there is a way to guess it. Thus, by definition, this machine recognizes the language $\mathcal{L}_B$.

V.5.24 The language is $\mathcal{L} = \{ n \in \mathbb{N} \;\big|\; n\text{'s prime factorization has exactly two primes} \}$.

One unbounded parallelism algorithm tries each conceivable prime factorization, simultaneously. For instance, given the input 8, it could check the products $2^0 3^0 5^0 7^0$, $2^1 3^0 5^0 7^0$, $\ldots$ $2^7 3^7 5^7 7^7$, to see whether any of them equals 8 and exactly two primes nave nonzero exponents. Multiplying is fast and checking whether a number is composite or prime can also be done in polytime. Thus each of these checks happens in polytime.

A guessing algorithm is that the machine inputs the number n and guesses a two-prime factorization, or is given one by a demon. For instance, given 45, the machine might guess $3^2 5^1$. (It might instead guess $3^1 5^2$, but that one won't check out.) Again, the verification is polytime because multiplication is polytime, as is checking whether the two numbers are composite or prime. By definition, the machine recognizes the language $\mathcal{L}$ if, given $n \in \mathbb{N}$, when $n \in \mathcal{L}$ then there exists some guess that verifies, and when $n \notin \mathcal{L}$ then there is no way for a guess to verify. So this algorithm recognizes $\mathcal{L}$.

V.5.25

(A) This is the language.

$$\mathcal{L} = \{ M \in X \times Y \times Z \;\big|\; M \text{ has an } n\text{-element subset } \hat{M} \text{ where no triples agree on any coordinate} \}$$

A nondeterministic algorithm, framed in terms of guessing, is: given M, the machine guesses a size n subset $\hat{M}$ meeting the conditions. Verifying the conditions clearly takes only polytime. If there is such an $\hat{M}$ then there is a way for the machine to guess it, and it will verify. If there is no such $\hat{M}$ then there is no way for the machine to guess one that verifies. So by the definition of this model of nondeterminism, the machine recognizes the language in polytime.

(B) Here is the language.

$$\mathcal{L} = \{ A \;\big|\; A \text{ is a multiset of natural numbers with an } \hat{A} \subseteq A \text{ so that } \sum_{a \in \hat{A}} a = \sum_{a \in A - \hat{A}} a \}$$

A nondeterministic algorithm stated in guessing terms is that, when asked to determine if the given A is a member of $\mathcal{L}$, the machine guesses at a $\hat{A}$. (That is, $\hat{A}$ is the witness.) Verifying that $\sum_{a \in \hat{A}} a = \sum_{a \notin \hat{A}} a$ is clearly polytime. If there is such a partition then for this machine there exists a guess that would pass verification. If there is no such partition then no guess will verify. So by the definition, this machine recognizes the language $\mathcal{L}$.

V.5.26 For unbounded parallelism, we can try every possible B-coloring of the graph's nodes. There are a large but finite number of those (where the graph has k vertices, there are B^k-many colorings). So the computation tree has a single node with that many children. Each coloring is easy to check for validity, since we just have to check every pair of vertices to see if they are the same color. If any of the child nodes is OK, then there is a B-coloring.

For guessing, the machine does not spawn lots of branches. Instead, it nondeterministically selects one coloring (that is, it guesses one, or is given one by a demon). It then verifies the guess by iterating through

every vertex pair to see if they are the same color. If there is a way to correctly nondeterministically guess then by definition this algorithm succeeds.

V.5.27

(A) This is the language $\mathcal{L}$.

$$\{\,\sigma \;\big|\; \sigma \text{ represents a Propositional Logic statement where at least two lines of the truth tables end in } T\,\}$$

These are suitable fill-ins: (1) a Propositional Logic statement, (2) a pointer to two lines of the truth table, (3) for those two truth table lines, the statement returns T, (4) two truth table lines that cause the expression σ to evaluate to T, and (5) pair of truth table lines.

(B) Here is the problem cast as the decision problem for a language.

$$\mathcal{L} = \{\,\langle S, T\rangle \;\big|\; \text{a subset of } A \subseteq S \text{ has } \sum_{a\in A} a = T\,\}$$

These are fill-ins: (1) a set and number pair $\langle S, T\rangle$, (2) a set of numbers, (3) that is a subset of the given S and that its elements sum to T, (4) a subset of S whose elements sum to T, and (5) subset of S.

V.5.28 Lemma 5.11 requires that we produce a deterministic Turing machine verifier, $\mathcal{V}$. It must input pairs of the form $\langle \sigma, \omega\rangle$, where $\sigma = \langle s_0, \ldots\ s_5, T\rangle$. It must have the property that if $\sigma \in \mathcal{L}_{\text{CD}}$ then there is a witness ω such that $\mathcal{V}$ accepts the input, while if $\sigma \notin \mathcal{L}_{\text{CD}}$ then there is no such witness. And it must run in time polynomial in $|\sigma|$.

The witness ω is an arithmetic expression (more precisely, a string representation of an arithmetic expression) that evaluates to the target T and that involves a sub-multiset of the six numbers $s_0, \ldots\ s_5$. The verifier checks that the expression does indeed evaluate to the target, and does indeed use each of the six numbers at most once. Clearly those checks can be done in polytime.

If $\sigma \in \mathcal{L}_{\text{CD}}$ then by definition there is an arithmetic expression, and so a witness ω exists that will cause $\mathcal{V}$ to accept the input, namely that expression. If $\sigma \notin \mathcal{L}_{\text{CD}}$ then there is no such arithmetic expression, and therefore no witness ω will cause $\mathcal{V}$ to accept.

V.5.29 We recast the Traveling Salesman problem from an optimization problem by using bounds $B \in \mathbb{N}$ to get this parametrized set of bitstrings.

$$\mathcal{TS}_B = \{\,G \in \mathbb{B}^* \;\big|\; G \text{ is a weighted graph with a circuit of length less than } B\,\}$$

To show that this problem is in *NP*, Lemma 5.11 requires that we produce a deterministic Turing machine verifier, $\mathcal{V}$. It must input pairs of the form $\langle \sigma, \omega\rangle$. The first item in the pair is a graph $\mathcal{G}$ (technically, a bitstring representation of a graph) that is a candidate for membership in the language, and the second is a witness. The verifier must have the property that if $\sigma \in \mathcal{TS}_B$ then there is an ω such that $\mathcal{V}$ accepts the input, while if $\sigma \notin \mathcal{TS}_B$ then there is no such ω. This verifier must run in time polynomial in $|\sigma|$.

For the witness we can use a circuit $\omega = \langle v_{i_0}, v_{i_1}, \ldots\ v_{i_k}\rangle$ that is of total cost less than the bound. The verifier gets an input pair $\langle \sigma, \omega\rangle$ and checks that ω is indeed a Hamiltonian circuit of the graph, and that it costs less than the bound. A verifier can do those checks in time polynomial in $|\sigma|$. If σ is indeed an element of $\mathcal{TS}_B$ then there is an ω allowing the verifier to accept. If σ is not an element of $\mathcal{TS}_B$ then there is no such ω, and so there is not way for the verifier to accept the pair.

V.5.30 The Independent Sets problem is the decision problem for this language.

$$\mathcal{L} = \{\,\langle \mathcal{G}, n\rangle \;\big|\; \text{the graph } \mathcal{G} \text{ has at least } n \text{ independent vertices}\,\}$$

To show that this language decision problem is in *NP*, Lemma 5.11 requires that we produce a deterministic Turing machine verifier, $\mathcal{V}$. This verifier must input pairs of the form $\langle \sigma, \omega\rangle$, where σ is a graph-natural number pair, $\langle \mathcal{G}, n\rangle$. The verifier must satisfy that if $\sigma \in \mathcal{L}$ then there is a ω such that $\mathcal{V}$ accepts the input, while if $\sigma \notin \mathcal{L}$ then there is no such ω. This verifier must run in time polynomial in $|\sigma|$.

For a witness, take a size-n set of independent vertices from the graph, $\omega = \langle v_0, \ldots\ v_{n-1}\rangle$. The verifier gets an input pair $\langle \sigma, \omega\rangle$ where σ is a graph, and checks that the vertices are in the graph and that no two of

them are connected. Those checks take time polynomial in $|\sigma|$ (obviously, this depends on the graph being represented with reasonable efficiency, but we always make that assumption). If the pair σ is indeed an element of $\mathcal{L}$ then there is such a witness ω, so there exists a pair $\langle\sigma,\omega\rangle$ that the verifier can accept. If σ is not an element of $\mathcal{L}$ then there is no such ω, and so there does not exist a pair that the verifier will accept.

V.5.31 The Knapsack problem is the decision problem for this language, $\mathcal{L}$.

$$\{\,\langle(w_0,v_0),\ldots(w_{n-1},v_{n-1}),B,T\rangle \;\big|\; \text{there is } I=\{\,i_0,\ldots i_k\,\}\subseteq\{\,0,\ldots n-1\,\} \text{ so } \sum_{i\in I} w_i \leq B \text{ and } \sum_{i\in I} v_i \geq T\,\}$$

To show that this language decision problem is in *NP*, Lemma 5.11 requires that we produce a deterministic Turing machine verifier, $\mathcal{V}$. The verifier input pairs $\langle\sigma,\omega\rangle$, where $\sigma=\langle(w_0,v_0),\ldots(w_{n-1},v_{n-1}),B,T\rangle$. The verifier must have the property that if $\sigma\in\mathcal{L}$ then there is a ω such that $\mathcal{V}$ accepts the input, while if $\sigma\notin\mathcal{L}$ then there is no such ω. This verifier must also run in time polynomial in $|\sigma|$.

For a witness, take a set $\omega=\{\,i_0,\ldots i_k\,\}\subseteq\{\,0,\ldots n-1\,\}$. The verifier gets an input pair $\langle\sigma,\omega\rangle$ and checks that the sum of the weights of the indicated pairs, $\sum_{i\in\omega} w_i$, is less than or equal to the bound B, and also that the sum of the values of pairs, $\sum_{i\in\omega} v_i$, is greater than or equal to the target T. Those checks clearly take time polynomial in $|\sigma|$.

If σ is indeed an element of $\mathcal{L}$ then there is such a selection of indices, so there is such a witness ω, so there exists a pair $\langle\sigma,\omega\rangle$ that the verifier can accept. If σ is not an element of $\mathcal{L}$ then there is no such ω, and so there does not exist a pair that the verifier will accept.

V.5.32 True. By Lemma 5.8 it suffices to show that the language decision problem is in *P*. But that's clear: given a triple $\langle a,b,c\rangle$, checking whether the first two sum to the third is clearly a polytime task.

V.5.33 Here is the appropriate language.

$$\mathcal{L}=\{\,\langle\mathcal{G},B\rangle \;\big|\; \mathcal{G} \text{ has a simple path of length at least } B\,\}$$

To show that the decision problem for $\mathcal{L}$ is in *NP*, Lemma 5.11 wants a deterministic Turing machine verifier, $\mathcal{V}$. It inputs pairs $\langle\sigma,\omega\rangle$, where $\sigma=\langle\mathcal{G},B\rangle$. The verifier must be such that if $\sigma\in\mathcal{L}$ then there is a ω such that $\mathcal{V}$ accepts the input pair, while if $\sigma\notin\mathcal{L}$ then there is no such ω. Also, the verifier must run in time polynomial in $|\sigma|$.

For a witness, take a path $\omega=\langle v_0,\ldots v_{n-1}\rangle$. The verifier gets an input pair $\langle\sigma,\omega\rangle$ and checks that the path is in $\mathcal{G}$, that no two vertices are equal, and has length at least B. Those checks clearly can be done in time polynomial in $|\sigma|$.

If σ is indeed an element of $\mathcal{L}$ then there is such a path ω, so there exists a pair $\langle\sigma,\omega\rangle$ that the verifier accepts. If σ is not an element of $\mathcal{L}$ then there is no such ω and so there does not exist a pair that the verifier accepts.

V.5.34

(A) Here is an appropriate language.

$$\mathcal{L}_0=\{\,\langle a,b\rangle\in\mathbb{N}^2 \;\big|\; \text{there exists } x\in\mathbb{N} \text{ with } ax+1=b\,\}$$

To satisfy Lemma 5.11, we have to produce a deterministic Turing machine verifier, $\mathcal{V}$. It inputs $\langle\sigma,\omega\rangle$, where σ is the number pair $\langle a,b\rangle$. The verifier must satisfy that if $\sigma\in\mathcal{L}_0$ then there is a witness ω such that $\mathcal{V}$ accepts the input pair, while if $\sigma\notin\mathcal{L}_0$ then there is no such ω. The verifier must run in time polynomial in $|\sigma|$.

For a witness, take a number $\omega=x$. The verifier gets an input pair $\langle\sigma,\omega\rangle$ and checks that $ax+1=b$. That can be done in time polynomial in $|\sigma|$. If σ is indeed an element of $\mathcal{L}_0$ then there is a number $\omega=x$ so that the verifier can accept $\langle\sigma,\omega\rangle$. If σ is not an element of $\mathcal{L}_0$ then there is no such ω, and so there does not exist a pair that the verifier accepts.

(B) This is the associated language.

$$\mathcal{L}_1=\{\,M \;\big|\; \text{there is a set } P\subset\mathbb{Z} \text{ of numbers such that } M \text{ is the multiset of pairwise distances}\,\}$$

Consider a deterministic Turing machine verifier, $\mathcal{V}$, that inputs pairs $\langle \sigma, \omega \rangle$, where σ is a multiset M. For a witness, take a set of positions $\omega = P$. The verifier gets an input pair $\langle \sigma, \omega \rangle$ and checks that the pairwise distances among elements of P equals M. That can be done in time polynomial in $|\sigma|$.

If $\sigma \in \mathcal{L}_1$ then there is such a set of positions $\omega = P$, so there exists a pair $\langle \sigma, \omega \rangle$ that the verifier accepts. If $\sigma \notin \mathcal{L}_1$ then there is no such set ω, and so there does not exist a pair that the verifier accepts.

V.5.35 Countable. A verifier is a deterministic Turing machine, and there are countably many such machines.

V.5.36 We use Lemma 5.11. This is an appropriate language, where the road map is the weighted graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$.

$$\mathcal{L}_B = \{ \mathcal{V} \subseteq \mathcal{V} \;\big|\; \text{there are two suitable cycles} \}$$

To use Lemma 5.11, we must produce a verifier and suitable witnesses. The verifier inputs $\langle \sigma, \omega \rangle$, where $\sigma = V$ (technically, σ is a string representation of V). A witness is the two cycles, $\omega = \langle c_0, c_1 \rangle$. The verifier checks that every vertex is a member of at least one of the two cycles, and that each cycle is of total weight at most B. Clearly that check can be done in time polynomial in $|\sigma|$.

If $\sigma \in \mathcal{L}$ then there is a pair of cycles, so there exists a suitable witness ω such that the verifier accepts the pair $\langle \sigma, \omega \rangle$. If $\sigma \notin \mathcal{L}$ then there are not two such cycles. So there is no such witness ω that checks out, and so the verifier will not accept for any such ω.

V.5.37 The proof is much like Theorem 5.4's. Here also one direction is easy because a deterministic Turing machine is a special case of a nondeterministic one, so if a deterministic machine recognizes a language then a nondeterministic one does also.

For the converse suppose that a nondeterministic Turing machine $\mathcal{P}$ recognizes a language $\mathcal{L}$. We will describe a deterministic machine $\mathcal{Q}$ that does the same.

Fix an input σ. Have the deterministic machine $\mathcal{Q}$ do a breadth-first search of $\mathcal{P}$'s computation tree. If $\mathcal{P}$ accepts σ then there is a maximal path whose terminal configuration has an accepting state, and $\mathcal{Q}$ will eventually find that configuration because the search is breadth-first. If the tree has no accepting maximal path then $\mathcal{Q}$ will never find one, and so will not accept σ.

V.5.38

(A) The natural language is $\mathcal{L} = \{ \langle \mathcal{G}_0, \mathcal{G}_1 \rangle \;\big|\; \text{they are isomorphic} \}$.

(B) We use Lemma 5.11, so we must produce a verifier and suitable witnesses. The verifier inputs $\langle \sigma, \omega \rangle$ where σ is a pair of graphs, $\langle \mathcal{G}_0, \mathcal{G}_1 \rangle$. The witness is a function, a set of ordered pairs $\langle v, f(v) \rangle$. The verifier checks that this function satisfies the conditions, that it is one-to-one and onto, and that $\{ v, \hat{v} \}$ is an edge of $\mathcal{G}_0$ if and only if $\{ f(v), f(\hat{v}) \}$ is an edge of $\mathcal{G}_1$ That check clearly takes time polynomial in $|\sigma|$.

If $\sigma \in \mathcal{L}$ then there is such a function, so there exists a hint ω that will allow the verifier to accept the pair $\langle \sigma, \omega \rangle$. If the two graphs are not isomorphic then no witness ω will verify.

V.5.39

(A) The class *NP* is the set of languages decidable by a nondeterministic machine in polytime. Every problem decidable by a nondeterministic Turing machine is decidable by a deterministic Turing machine. The Halting problem is not decidable by a deterministic Turing machine, so it is not in *NP*.

(B) There is no polynomial bound on the number of steps, and consequently on the length of ω.

V.5.40 *Comment: there are a number of possible different approaches but rather than give no answer here we give one of those.*

A configuration of a nondeterministic Turing machine $\mathcal{P}$ is a four-tuple, $\langle q, s, \tau_L, \tau_R \rangle \in Q \times \Sigma \times \Sigma^* \times \Sigma^*$. The items signify the current state q, the character under the read/write head s, and the tape contents to the left and right of the head τ_L and τ_R. Where the machine's input is the string σ whose first character is s, so that $\sigma = \langle s \rangle^\frown \alpha$, the computation's initial configuration is $C_0 = \langle q_0, s, \varepsilon, \alpha \rangle$.

We next describe under what circumstances a configuration $\hat{C}$ succeeds C, denoted $C \vdash \hat{C}$. Suppose that $C = \langle q, s, \tau_L, \tau_R \rangle$. Compute the set $\Delta(q, s)$. If that set is empty then there is no next configuration $\hat{C}$.

Otherwise that set is not empty. For every instruction $\langle q_p T_p T_n q_n \rangle \in \Delta(q, s)$ there are three possibilities. Exactly one of those three holds and it defines a succeeding configuration $\hat{C}$. (1) If the action symbol T_n is a member of the tape alphabet, $T_n \in \Sigma$, then write it to the tape, $\hat{C} = \langle q_n, T_n, \tau_L, \tau_R \rangle$. (2) If T_n is the character L then move the tape head to the left. That is, $\hat{C} = \langle q_n, \hat{s}, \hat{\tau}_L, \hat{\tau}_R \rangle$ where $\hat{s}$ is the rightmost character of the

string τ_L (if $\tau_L = \varepsilon$ then $\hat{s}$ is the blank character, B), where $\hat{\tau}_L$ is τ_L with its rightmost character omitted (if $\tau_L = \varepsilon$ then $\hat{\tau}_L = \varepsilon$ also), and where $\hat{\tau}_R$ is the string formed by pushing the tape character s onto the front of τ_R, giving $\hat{\tau}_R = \langle s \rangle^\frown \tau_R$. (3) If T_n = R then move the tape head to the right. This is so much like the prior item that we omit the details.

A computation path is a sequence of configurations $C_0 \vdash C_1 \vdash \cdots$, starting with the initial configuration. The union of all such paths forms a directed graph called the computation tree, or simply the computation. If for a given input σ at least one maximal path in that tree ends in a configuration whose state is accepting then we say that the machine accepts σ. If the machine does not accept σ then it rejects σ.

V.6.10 The first way is right: when $\mathcal{L}_1 \leq_p \mathcal{L}_0$ then $\mathcal{L}_1$ reduces to $\mathcal{L}_0$.

V.6.11 No. That intuition fits Cook reduction. Karp reduction calls the black box for A at most one time. This is the '$f(\sigma) \in \mathcal{L}_0$' at the end of Definition 6.1.

V.6.12 It is the first. A fast algorithm for $\mathcal{L}_0$ would give a fast one for $\mathcal{L}_1$.

V.6.13 The assertion about P is immediate from Lemma 6.9's item (4). It also holds for NP by item (5).

V.6.14 If $\mathcal{L}_1 \leq_p \mathcal{L}_0$ via the reduction function f then there can be another language $\mathcal{L}$ where $\mathcal{L}_1 \leq_p \mathcal{L}$, also via the reduction function f. There can be uncountably many such other languages.

Expanding a bit: in the proof of Lemma 6.9's argument for item (3), the reduction function is as below for the input string α, using one member $\sigma \in \mathcal{L}_0$ and one nonmember $\tau \notin \mathcal{L}_0$.

$$f(\alpha) = \begin{cases} \sigma & \text{– if } \alpha \in \mathcal{L}_1 \\ \tau & \text{– otherwise} \end{cases}$$

This definition leaves untouched the question of elementhood for almost all of the codomain. Strings other than σ and τ can be elements or non-elements of $\mathcal{L}$. As a result, this one reduction function relates $\mathcal{L}_1$ to uncountably many other sets.

V.6.15 There is indeed a subset that adds to T, namely $A = \{23, 31, 72\}$.

V.6.16 Lemma 6.9 says that $\leq_p$ is a reflexive relation so the diagonal below contains Y's. By the same lemma, any problem in P is polytime reducible to any other problem at all. So the first, second, and fourth row are all Y's.

	$\mathcal{L}_0$	$\mathcal{L}_1$	$\mathcal{L}_2$	$\mathcal{L}_3$
$\mathcal{L}_0$	Y	Y	Y	Y
$\mathcal{L}_1$	Y	Y	Y	Y
$\mathcal{L}_2$	N	N	Y	N
$\mathcal{L}_3$	Y	Y	Y	Y

Also by the lemma, if a problem is polytime reducible to one in P then it is itself in P. But $\mathcal{L}_2$ is given as not in P, which accounts for the N's in the third row.

V.6.17

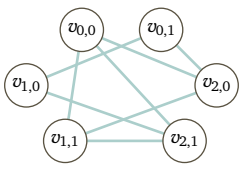
(A) This is wrong. If A is the empty set, which is decidable, and $B = K$, then $A \leq_p B$ but B is not decidable in polytime. The reason that $A \leq_p B$ is that A can be decided in polytime without any reference to B at all; in fact, deciding A is $\mathcal{O}(1)$.

(B) This is true.

V.6.18 Example 6.4 says that we want the graph $\mathcal{G}$ whose vertices are pairs $\langle c, \ell \rangle$ where c is the number of a clause and ℓ is a literal in that clause. Two vertices $v_0 = \langle c_0, \ell_0 \rangle$ and $v_1 = \langle c_1, \ell_1 \rangle$ are connected by an edge if they come from different clauses, $c_0 \neq c_1$, and the literals are not negations of each other, $\ell_0 \neq \neg\ell_1$.

The given propositional logic expression has three clauses, $x_0 \vee x_1$, $\neg x_0 \vee \neg x_1$, and $x_0 \vee \neg x_1$, each with two literals. This is the graph.

$v_{0,0} = \langle 0, x_0 \rangle$
$v_{0,1} = \langle 0, x_1 \rangle$
$v_{1,0} = \langle 1, \neg x_0 \rangle$
$v_{1,1} = \langle 1, \neg x_1 \rangle$
$v_{2,0} = \langle 2, x_0 \rangle$
$v_{2,1} = \langle 2, \neg x_1 \rangle$



V.6.19 For each vertex there is a clause.

$$a_{0,0} \vee b_{0,0} \vee c_{0,0} \quad a_{0,1} \vee b_{0,1} \vee c_{0,1} \quad a_{1,0} \vee b_{1,0} \vee c_{1,0} \quad a_{1,1} \vee b_{1,1} \vee c_{1,1} \quad a_{2,0} \vee b_{2,0} \vee c_{2,0} \quad a_{2,1} \vee b_{2,1} \vee c_{2,1}$$

For each edge there are three clauses. For instance, for the edge between $v_{0,0}$ and $v_{1,1}$ the three clauses are here.

$$(\neg a_{0,0} \vee \neg a_{1,1}) \qquad (\neg b_{0,0} \vee \neg b_{1,1}) \qquad (\neg c_{0,0} \vee \neg c_{1,1})$$

Construct the entire expression,

$$\begin{aligned} E =&(a_{0,0} \vee b_{0,0} \vee c_{0,0}) \wedge (a_{0,1} \vee b_{0,1} \vee c_{0,1}) \wedge (a_{1,0} \vee b_{1,0} \vee c_{1,0}) \\ &\wedge (a_{1,1} \vee b_{1,1} \vee c_{1,1}) \wedge (a_{2,0} \vee b_{2,0} \vee c_{2,0}) \wedge (a_{2,1} \vee b_{2,1} \vee c_{2,1}) \\ &\wedge (\neg a_{0,0} \vee \neg a_{1,1}) \wedge (\neg b_{0,0} \vee \neg b_{1,1}) \wedge (\neg c_{0,0} \vee \neg c_{1,1}) \\ &\wedge (\neg a_{0,0} \vee \neg a_{2,0}) \wedge (\neg b_{0,0} \vee \neg b_{2,0}) \wedge (\neg c_{0,0} \vee \neg c_{2,0}) \\ &\wedge (\neg a_{0,0} \vee \neg a_{2,1}) \wedge (\neg b_{0,0} \vee \neg b_{2,1}) \wedge (\neg c_{0,0} \vee \neg c_{2,1}) \\ &\wedge (\neg a_{0,1} \vee \neg a_{1,0}) \wedge (\neg b_{0,1} \vee \neg b_{1,0}) \wedge (\neg c_{0,1} \vee \neg c_{1,0}) \\ &\wedge (\neg a_{0,1} \vee \neg a_{2,0}) \wedge (\neg b_{0,1} \vee \neg b_{2,0}) \wedge (\neg c_{0,1} \vee \neg c_{2,0}) \\ &\wedge (\neg a_{1,0} \vee \neg a_{2,1}) \wedge (\neg b_{1,0} \vee \neg b_{2,1}) \wedge (\neg c_{1,0} \vee \neg c_{2,1}) \\ &\wedge (\neg a_{1,1} \vee \neg a_{2,0}) \wedge (\neg b_{1,1} \vee \neg b_{2,0}) \wedge (\neg c_{1,1} \vee \neg c_{2,0}) \\ &\wedge (\neg a_{1,1} \vee \neg a_{2,1}) \wedge (\neg b_{1,1} \vee \neg b_{2,1}) \wedge (\neg c_{1,1} \vee \neg c_{2,1}) \end{aligned}$$

by taking the conjunction of the clauses.

V.6.20

(A) Three are: $\beta_0 = \alpha = \texttt{0110010}$, $\beta_1 = \texttt{1100100}$, and $\beta_2 = \texttt{1001001}$.

(B) It is a cyclic shift starting from index $k = 6$.

(C) It is the decision problem for this language.

$$\mathcal{L}_0 = \{ \langle \sigma, \tau \rangle \in \Sigma^2 \;\big|\; \tau \text{ is a substring of } \sigma \}$$

(D) It is the decision problem for this.

$$\mathcal{L}_1 = \{ \langle \alpha, \beta \rangle \in \Sigma^2 \;\big|\; \beta \text{ is a cyclic shift of } \alpha \}$$

(E) Following the hint, the reduction function f inputs $\langle \alpha, \beta \rangle$ and if the two are the same length then it outputs $\langle \alpha^\frown \alpha, \beta \rangle$ (else it outputs some pair sure to fail, such as $\langle \varepsilon, \texttt{0} \rangle$). Clearly f and $\langle \alpha, \beta \rangle \in \mathcal{L}_1$ if and only if $f(\langle \alpha, \beta \rangle) \in \mathcal{L}_0$. Also, f is clearly computable in polytime. Therefore $\mathcal{L}_1 \leq_p \mathcal{L}_0$.

V.6.21 In Example 6.3 this is the language given for the Shortest Path problem.

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, v_0, v_1, B \rangle \;\big|\; \text{there is path from } v_0 \text{ to } v_1 \text{ of total weight at most } B \}$$

(A) The Unweighted Shortest Path problem is an optimization problem, calling for the shortest path if one exists. As with many such problems, to express it as a language decision problem we use a parameter bound B_1 (the graph $\mathcal{G}$ is unweighted.)

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, v_0, v_1, B_1 \rangle \;\big|\; \text{there is path from } v_0 \text{ to } v_1 \text{ having fewer than } B_1 \text{ edges} \}$$

(In contrast, Vertex-to-Vertex Path is a decision problem: it just wants a 'yes there is a path' or 'no there is not' answer.)

For the reduction, the function takes input $\langle \mathcal{G}_1, v_0, v_1, B_1 \rangle$ for the Unweighted Shortest Path problem, that is, $\mathcal{G}_1$ is unweighted. It makes a weighted graph $\mathcal{H}_1$ by using $\mathcal{G}_1$'s vertices and giving each of its edges a weight of one. Then the output of the reduction function is $\langle \mathcal{H}_1, v_0, v_1, B_1 \rangle$. If there is a path from $\mathcal{H}_1$'s vertex v_0 to its v_1 of total weight B_1 then there is an unweighted path with at most B_1-many edges.

(B) It is an optimization problem so this is the canonical language.

$$\mathcal{L}_2 = \{\, \langle \mathcal{G}, v_0, v_1, B_2 \rangle \;\big|\; \text{there is path from } v_0 \text{ to } v_1 \text{ having total weight at least as large as } B_2 \,\}$$

The reduction function inputs $\langle \mathcal{G}, v_0, v_1, B_2 \rangle$, where $\mathcal{G}$ is a weighted graph. It converts to a new graph $\mathcal{H}$ with the same vertices but where the weights of all edges have been made the negative of the weight of $\mathcal{G}$'s edges. Then the output of the reduction function is $\langle \mathcal{H}, v_0, v_1, -B_2 \rangle$. The idea is that a path that is short for the negatives of the weights is long for the original weights.

V.6.22

(A) The Hamiltonian Circuit problem is the decision problem for this language.

$$\mathcal{L}_0 = \{\, \mathcal{H} \;\big|\; \text{the unweighted graph contains a circuit that includes each vertex exactly once} \,\}$$

The Traveling Salesman problem is this decision problem.

$$\mathcal{L}_1 = \{\, \langle \mathcal{G}, B \rangle \;\big|\; \text{some circuit of the weighted graph } \mathcal{G} \text{ that includes each vertex has total cost at most } B \,\}$$

(B) The reduction function inputs an instance of Hamiltonian Circuit, a graph $\mathcal{H} = \langle \mathcal{N}, \mathcal{E} \rangle$ whose edges are unweighted. It returns the instance of Traveling Salesman that uses the vertex set $\mathcal{N}$ as cities and that takes the distances between the cities to be: $d(v_i, v_j) = 1$ if $v_i v_j$ is an edge of $\mathcal{G}$ and $d(v_i, v_j) = 2$ if $v_i v_j \notin \mathcal{E}$, and such that the bound B is the number of vertices $|\mathcal{N}|$.

Because of this bound, there will be a Traveling Salesman solution if and only if there is a Hamiltonian Circuit solution, namely the Traveling Salesman solution uses the edges of the Hamiltonian circuit. That is, $\mathcal{H} \in \mathcal{L}_0$ if and only if $f(\mathcal{H}) \in \mathcal{L}_1$.

(C) The number of edges is less than twice the number of vertices, so that polytime in the input graph size is the same as polytime in the number of vertices. To output the Traveling Salesman instance, the algorithm examines all the pairs of vertices, which is a nested loop and so takes time that is quadratic in the number of vertices. So f runs in polytime.

V.6.23

(A) This is the language for the 3-SAT problem.

$$\{\, E \;\big|\; E \text{ is a satisfiable Boolean expression in CNF where each clause has at most three literals} \,\}$$

This is the language for Strict 3-Satisfiability.

$$\{\, E \;\big|\; E \text{ is a satisfiable Boolean expression in CNF where each clause has exactly three literals} \,\}$$

(B) Any instance of Strict 3-Satisfiability is an instance of 3-SAT. So the reduction function is the identity function.

(C) We are asked to show that the two-literal Propositional Logic expression $E_0 = P \vee Q$ is equivalent to the the expression $E_1 = (P \vee Q \vee R) \wedge (P \vee Q \vee \neg R)$ where each clause has three literals.

The most straightforward thing is to use a truth table to compare the two expressions.

P	Q	R	E_0	$P \vee Q \vee R$	$P \vee Q \vee \neg R$	E_1
F	F	F	F	F	T	F
F	F	T	F	T	F	F
F	T	F	T	T	T	T
F	T	T	T	T	T	T
T	F	F	T	T	T	T
T	F	T	T	T	T	T
T	T	F	T	T	T	T
T	T	T	T	T	T	T

The same technique works for the one-literal expression $E_2 = P$. It is equivalent to the expression $E_3 = (P \vee Q \vee R) \wedge (P \vee \neg Q \vee R) \wedge (P \vee Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R)$ where each clause has three literals.

P	Q	R	E_2	$(P \vee Q \vee R)$	$(P \vee \neg Q \vee R)$	$(P \vee Q \vee \neg R)$	$(P \vee \neg Q \vee \neg R)$	E_3
F	F	F	F	F	T	T	T	F
F	F	T	F	T	T	F	T	F
F	T	F	F	T	F	T	T	F
F	T	T	F	T	T	T	F	F
T	F	F	T	T	T	T	T	T
T	F	T	T	T	T	T	T	T
T	T	F	T	T	T	T	T	T
T	T	T	T	T	T	T	T	T

(D) The reduction function inputs expressions in Conjunctive Normal form $E = \sigma_0 \wedge \sigma_1 \wedge \ldots \sigma_k$, where the σ_i clauses are disjunctions of three or fewer literals. It must output equivalent propositional logic expressions that also are in Conjunctive Normal form but where the clauses are disjunctions of exactly three literals. The prior item shows how the function converts the one to the other. Obviously the function can run in polytime.

V.6.24

(A) The Independent Set problem is the decision problem for this language.

$$\mathcal{L}_0 = \{\, \langle \mathcal{G}, \mathcal{B} \rangle \;\big|\; \text{there is } I \subseteq \mathcal{N}(_{\mathcal{G}} \text{ with } B \leq |I| \text{ such that no edge contains two vertices from } I \,\}$$

Vertex Cover is the decision problem for this language.

$$\mathcal{L}_1 = \{\, \langle \mathcal{H}, D \rangle \;\big|\; \text{there is } C \subseteq \mathcal{N}_{\mathcal{H}} \text{ with } |C| \leq D \text{ such that every edge contains at least one vertex from } C \,\}$$

(B) A vertex cover with $k = 4$ elements is $S = \{\, q_2, q_5, q_8, q_9 \,\}$.

(C) An independent set with $\hat{k} = 6$ elements is $\hat{S} = \{\, q_0, q_1, q_3, q_4, q_6, q_7 \,\}$.

(D) By definition, a set I of vertices is independent if every edge contains either zero elements or one element of I. Also by definition, a set C of vertices is a vertex cover if every edge contains either one element or two elements of C. Thus a set S of vertices is independent if and only if its complement $\mathcal{N} - S$ is a vertex cover.

(E) We will do Independent Set $\leq_p$ Vertex Cover; the other reduction is similar. By the prior item, there is a relationship between independent sets and vertex covers. In particular, if there is an independent set of size $\hat{k}$ then there is a vertex cover of size $|\mathcal{N}| - \hat{k}$.

The definition requires that we produce a polytime computable function f. Given an instance of Independent Set $\sigma = \langle \mathcal{G}, \hat{k} \rangle$, the function converts it to an instance of Vertex Cover by adjusting the bound, $f(\sigma) = \langle \mathcal{G}, |\mathcal{G}| - \hat{k} \rangle$.

V.6.25

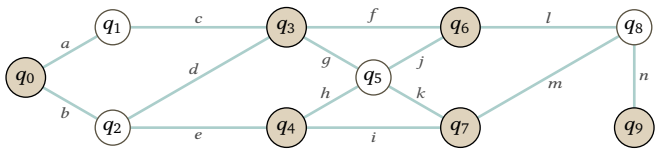
(A) Vertex Cover is the decision problem for this language.

$$\mathcal{L}_0 = \{\, \langle \mathcal{G}, B \rangle \;\big|\; \text{there is } C \subseteq \mathcal{N}(\mathcal{G}) \text{ with } |C| \leq B \text{ such that every edge contains a vertex from } C \,\}$$

Set Cover is the decision problem for this language.

$$\mathcal{L}_1 = \{\, \langle S, S_0, \ldots S_{k-1}, D \rangle \;\big|\; \text{each } S_j \text{ is a subset of } S, \text{ and } k \leq D, \text{ and } S_0 \cup \cdots \cup S_{k-1} = S \,\}$$

(B) One vertex cover is $C = \{\, q_0, q_3, q_4, q_6, q_7, q_9 \,\}$.



(C) The set of edges is $S = \{\, a, b, \ldots n \,\}$. The subsets are these.

$$S_0 = \{\, a, b \,\},\ S_1 = \{\, a, c \,\},\ S_2 = \{\, b, d, e \,\},\ S_3 = \{\, c, d, f, g \,\},\ S_4 = \{\, e, h, i \,\},$$
$$S_5 = \{\, g, h, j, k \,\},\ S_6 = \{\, f, j, l \,\},\ S_7 = \{\, i, k, m \,\},\ S_8 = \{\, l, m, n \,\},\ S_9 = \{\, n \,\}$$

To find a set cover we can take the subsets associated with vertices in the vertex cover.

$$S_{q_0} \cup S_{q_3} \cup S_{q_4} \cup S_{q_6} \cup S_{q_7} \cup S_{q_9} = \{\, a, b \,\} \cup \{\, c, d, f, g \,\} \cup \{\, e, h, i \,\} \cup \{\, f, j, l \,\} \cup \{\, i, k, m \,\} \cup \{\, n \,\} = S$$

(D) We must produce a reduction function f. It inputs an instance $\langle \mathcal{G}, B \rangle$ of the Vertex Cover problem. The computation is: let $S = \mathcal{E}$, the set of of all edges. For each vertex v, construct $S_v \subseteq S$ consisting of the edges incident on v. Then the output of the function is an instance of the Set Cover problem, $f(\langle \mathcal{G}, B \rangle) = \langle S, S_{v_0}, \ldots B \rangle$. Clearly we can compute this function in polytime, and clearly also $\langle \mathcal{G}, B \rangle \in \mathcal{L}_0$ if and only if $\langle S, S_{v_0}, \ldots B \rangle \in \mathcal{L}_1$.

V.6.26

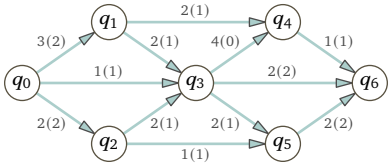
(A) Max-Flow has this language.

$$\mathcal{L}_0 = \{ \langle F, \mathcal{G}, B \rangle \;\big|\; \text{the flow } F \text{ meets the constraints of } \mathcal{G} \text{ and the flow into the sink is at least } B \}$$

This is the language for the Linear Programming optimization problem.

$$\mathcal{L}_1 = \{ \langle \vec{x}, C, Z, D \rangle \;\big|\; \text{the point } \vec{x} \text{ meets the constraints } C \text{ and makes } Z(\vec{x}) \geq D \}$$

(B) There are algorithms to solve these problems, but for small ones we can do it by eye. The maximum flow is 5, with each edge's numbers given in parentheses.



(C) The variables are $x_{0,1}, \ldots x_{5,6}$. These are the capacity constraints.

$$\begin{aligned} &0 \leq x_{0,1} \leq 3, \quad 0 \leq x_{0,2} \leq 2, \quad 0 \leq x_{0,3} \leq 1, \quad 0 \leq x_{1,3} \leq 2, \quad 0 \leq x_{1,4} \leq 2, \\ &\quad 0 \leq x_{2,3} \leq 2, \quad 0 \leq x_{2,5} \leq 1, \quad 0 \leq x_{3,4} \leq 4, \quad 0 \leq x_{3,5} \leq 2, \quad 0 \leq x_{3,6} \leq 2, \\ &\quad 0 \leq x_{4,6} \leq 1, \quad \text{and} \quad 0 \leq x_{5,6} \leq 2 \end{aligned}$$

(These fit the definition of the problem because each is a combination of two linear constraints. For instance, $0 \leq x_{0,1} \leq 3$ is a combination of $0 \leq x_{0,1}$ and $x_{0,1} \leq 3$.) These are the flow constraints.

$$\begin{aligned} &x_{0,1} = x_{1,3} + x_{1,4}, \quad x_{0,2} = x_{2,3} + x_{2,5}, \quad x_{0,3} + x_{1,3} + x_{2,3} = x_{3,4} + x_{3,5} + x_{3,6}, \\ &\quad x_{1,4} + x_{3,4} = x_{4,6}, \quad \text{and} \quad x_{2,5} + x_{3,5} = x_{5,6} \end{aligned}$$

(Here also, these fit the definition of the problem because each is a combination of two linear constraints. For instance, $x_{0,1} = x_{1,3} + x_{1,4}$ is a combination of $0 \leq x_{0,1} - x_{1,3} - x_{1,4}$ and $x_{0,1} - x_{1,3} - x_{1,4} \leq 0$.) We want to maximize $Z = x_{3,6} + x_{4,6} + x_{5,6}$.

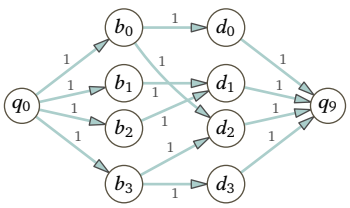
(D) We must construct a reduction function f. The input is a triple $\langle F, \mathcal{G}, B \rangle$. Use that information to construct variables, capacity constraints and flow constraints C, and an optimization expression Z. In particular, the flow F (shown in parentheses in the network above) gives an associated point $\vec{x}$ in the Linear Programming instance.

Clearly we can do that construction in polytime. Clearly also $\langle F, \mathcal{G}, B \rangle \in \mathcal{L}_0$ if and only if $\langle \vec{x}, C, Z, B \rangle \in \mathcal{L}_1$.

V.6.27

(A) The maximum number of matches is three, as in (b_0, d_0), (b_1, d_1), and (b_3, d_2).

(B) Here is a network diagram. Each edge has weight 1.



(C) The Max-Flow problem has this language, where W is a bound.

$$\mathcal{L}_0 = \{ \langle F, \mathcal{G}, W \rangle \mid \text{the flow } F \text{ meets the constraints of } \mathcal{G} \text{ and the flow into the sink is at least } W \}$$

The Matching problem has this, where X is a bound. The pairs (b_j, d_{i_j}) are band-drummer matches.

$$\mathcal{L}_1 = \{ \langle B_0, \ldots B_k, (b_0, d_{i_0}), \ldots (b_m, d_{i_m}), X \rangle \mid \text{each } d_{i_j} \in B_j \text{ and } m \geq X \}$$

(D) A reduction function f is given a sequence $\langle B_0, \ldots B_k, (b_0, d_{i_0}), \ldots (b_m, d_{i_m}), X \rangle$. It constructs a network that, from left to right, has a source node, a bank of band nodes, a bank of drummer nodes, and a sink node. Every edge is directed, and every edge has weight 1. The pairs (b, d) in the input give a flow F. So the output is the triple $f(\langle B_0, \ldots B_k, (b_0, d_{i_0}), \ldots (b_m, d_{i_m}), X \rangle) = \langle F, \mathcal{G}, X \rangle$. Clearly we can compute this f in polytime. Also clearly, $\langle B_0, \ldots B_k, (b_0, d_{i_0}), \ldots (b_m, d_{i_m}), X \rangle \in \mathcal{L}_1$ if and only if $\langle F, \mathcal{G}, X \rangle \in \mathcal{L}_0$.

V.6.28 This is the language for the 3-SAT problem.

$$\{ E \mid E \text{ is a satisfiable Boolean expression in CNF where each clause has at most three literals} \}$$

This is the language for SAT.

$$\{ E \mid E \text{ is a satisfiable Boolean expression in CNF} \}$$

(A) The reduction function f must input an instance of 3-SAT, a satisfiable propositional logic expression in Conjunctive Normal form where the clauses have at most three literals. It must output an instance of SAT, a satisfiable propositional logic expression in Conjunctive Normal form. So take f to be the identity $f(\sigma) = \sigma$. Obviously it runs in polytime.

(B) First suppose that there is an assignment of the truth values T or F to the variables $P_0, \ldots P_4$ such that C is true. Then at least one of the literals $p_0, \ldots p_4$ evaluates to T. If p_o or p_1 is T then setting A to F makes $E = (A \vee p_0 \vee p_1) \wedge (\neg A \vee p_2 \vee p_3 \vee p_4)$ evaluate to T. Otherwise setting A to T makes $E = T$.

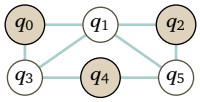
Now suppose that no assignment of the truth values to the variables $P_0, \ldots P_4$ makes C evaluate to T. Then no matter what truth value we assign to A, one of E's two clauses $(A \vee p_0 \vee p_1)$ and $(\neg A \vee p_2 \vee p_3 \vee p_4)$ is F. Hence for every assignment of truth values to $P_0, \ldots P_4$ and A the expression E gives F.

(C) The reduction function inputs statements S in Conjunctive Normal form. It must output Conjunctive Normal form statements where the clauses have three or fewer literals.

The prior item suggests how the reduction function does that. It replaces each clause C having more than three literals with a number of clauses having three or fewer. For instance starting with input $p_0 \vee p_1 \vee p_2 \vee p_3 \vee p_4$, the reduction function can go to $(A \vee p_0 \vee p_1) \wedge (\neg A \vee p_2 \vee p_3 \vee p_4)$, and then to the output $(A \vee p_0 \vee p_1) \wedge (B \vee \neg A \vee p_2) \wedge (\neg B \vee p_3 \vee p_4)$. Clearly that transformation can be done in polytime.

V.6.29

(A) The natural independent set is this.



(B) This is the language for it as a language decision problem.

$$\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ has at least } B \text{ vertices that do not share an edge} \}$$

(C) This is the language for the 3-SAT problem.

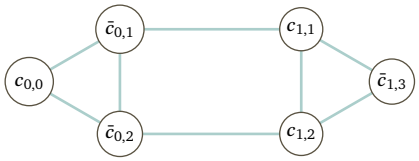
$$\{ E \mid E \text{ is a satisfiable Boolean expression in CNF where each clause has at most three literals} \}$$

(D) Here is the truth table.

P_0	P_1	P_2	P_3	$P_0 \vee \neg P_1 \vee \neg P_2$	$P_1 \vee P_2 \vee \neg P_3$	E
F	F	F	F	T	T	T
F	F	F	T	T	F	F
F	F	T	F	T	T	T
F	F	T	T	T	T	T
F	T	F	F	T	T	T
F	T	F	T	T	T	T
F	T	T	F	F	T	F
F	T	T	T	F	T	F
T	F	F	F	T	T	T
T	F	F	T	T	F	F
T	F	T	F	T	T	T
T	F	T	T	T	T	T
T	T	F	F	T	T	T
T	T	F	T	T	T	T
T	T	T	F	T	T	T
T	T	T	T	T	T	T

Any line ending in T shows that it is satisfiable.

(E) Here is the graph.



(F) The development in Example 6.4 points out that a CNF formula is satisfiable if and only if at least one literal in each clause evaluates to T. Literals from different clauses are compatible if they are not based on the same Boolean variable P_i, or if they are based on the same P_i and either both are 'P_i' or both are '$\neg P_i$'.

So we make a graph whose vertices are constructed from the literals. The vertices associated with clause i are: $c_{i,j}$ if clause i has a literal P_j, and $\bar{c}_{i,j}$ if clause i has $\neg P_j$.

As to edges, for any clause C_i connect together all of the vertices $c_{i,j}$ from that clause. In addition, where $i \neq \hat{\imath}$ connect all vertices of the forms $c_{i,j}$ and $\bar{c}_{\hat{\imath},j}$, since they arise from literals that are incompatible (P_j and $\neg P_j$).

Recall that an independent set $I \subseteq \mathcal{N}$ is one where for each edge at most one vertex is in I. Because the vertices associated with each clause are all connected, the fact that a set is independent implies that at most one vertex in that grouping is in that set.

So assume that the graph has an independent set I with as many members as there are clauses. Then I contains exactly one vertex for each clause. Also, because our construction connects vertices associated with incompatible literals, the set I contains only vertices associate with compatible literals.

Therefore the graph has an independent set containing as many vertices as there are clauses if and only if the underlying expression is satisfiable.

V.6.30

(A) These constraints ensure in this integer programming context that each variable is either 0 or 1.

$$0 \leq x_0, \quad x_0 \leq 1, \quad 0 \leq x_1, \quad x_1 \leq 1, \quad 0 \leq x_2, \quad x_2 \leq 1,$$

As to the expression, use $1 \leq x_0 + (1 - x_1) + (1 - x_2)$.

(B) We must produce a reduction function f. It inputs a propositional logic expression. For each variable P_i in that expression, create an integer variable x_i. To force each x_i to be either 0 or 1, include the constraints $0 \leq x_i$ and $x_i \leq 1$. For every clause in the propositional logic expression, add a constraint to the Integer Linear Programming instance as in the prior item.

$$P_i \vee \neg P_j \vee \neg P_k \qquad \Longleftrightarrow \qquad 1 \leq x_i + (1 - x_j) + (1 - x_k)$$

Clearly we can write f to run in polytime.

V.6.31

(A) It is the decision problem for this language.

$$D = \{\, p \mid p \text{ is a polynomial and there is } \vec{c} \in \mathbb{Z}^n \text{ so that } p(\vec{c}) = 0 \,\}$$

(B) Suppose first that all three polynomials in $S_{E_0} = \{\, x_0(1-x_0), x_1(1-x_1), x_0(1-x_1) \,\}$ have a value of 0. The first polynomial $x_0(1-x_0)$ evaluates to 0 if and only if x_0 is either 0 or 1. The second polynomial gives the same for x_1. The third evaluates to 0 if and only if either $0 = x_0$ or $1 = x_1$.

The converse, that if both variables equal 0 or 1 and either $0 = x_0$ or $1 = x_1$ then all three polynomials evaluate to 0, is clear.

(C) For $E_1 = (P_0 \vee \neg P_1 \vee \neg P_2) \wedge (P_1 \vee P_2 \vee \neg P_3)$ the analogous set is this.

$$S_{E_1} = \{\, x_0(1-x_0),\ x_1(1-x_1),\ x_2(1-x_2), x_3(1-x_3),\ x_0(1-x_1)(1-x_2),\ x_1x_2(1-x_3) \,\}$$

(D) Take the sum of squares.

$$\begin{aligned} p(x_0,x_1,x_2,x_3) = {} & [x_0(1-x_0)]^2 + [x_1(1-x_1)]^2 + [x_2(1-x_2)]^2 + [x_3(1-x_3)]^2 \\ & + [x_0(1-x_1)(1-x_2)]^2 + [x_1x_2(1-x_3)]^2 \end{aligned}$$

(E) We must produce the reduction function f, that inputs an instance of 3-Satisfiability, a Boolean expression E in which all clauses have three or fewer literals, and outputs an instance p of D. We must verify that E is satisfiable if and only of p has zero for some integer n-tuple. (In addition, it will be clear that f runs in polytime.)

This function constructs the polynomial p following the pattern outlined in the prior items. For each variable P_i used in the input expression E, put $x_i(1-x_1)$ into the set S_E. In addition, for each clause $L_{i_0} \vee L_{i_1} \vee L_{i_2}$ where L_i is a literal, add a polynomial: if $L_i = P_i$ then it has the factor x_i, while for a negation $L_i = \neg P_i$ it has the factor $(1-x_i)$. Finally, the polynomial p is the sum of the squares of the polynomials from the set S_E.

For one direction of the verification, suppose that E is satisfiable. For all atoms P_i that are assigned T, take $x_i = 0$, while if P_i is assigned F then take $x_i = 1$. As a result, each polynomial member of S_E has the value of 0. Consequently the sum of squares polynomial p has a value of 0, and so the tuple of x_i's is a zero of that polynomial.

Now for the other direction. Suppose that we have a tuple of integers $\vec{c}$ such that $p(\vec{c}) = 0$. Because p is a sum of squares, each term must have a value of 0. The terms of the form $x_i(1-x_i)$ imply that $x_i = 0$ or $x_i = 1$. The terms that match the form of a clause imply that there is a matching assignment to the Boolean variables P_i that makes the clause true (namely, $x_i = 0$ is associated with $P_i = T$, and $x_i = 1$ is associated with $P_i = F$).

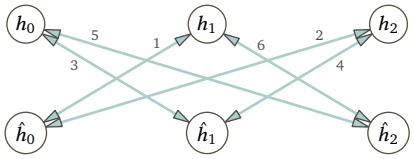
V.6.32

(A) The reduction function must take in an instance of the symmetric problem and return an instance of the asymmetric problem. But a symmetric instance is a special case of an asymmetric instance. So the reduction function can be the identity function.

(B) This is the graph matrix for the first stage of constructing $\mathcal{H}$. It is symmetric along the upper left to lower right diagonal.

	h_0	h_1	h_2	$\hat{h}_0$	$\hat{h}_1$	$\hat{h}_2$
h_0					3	5
h_1				1		6
h_2				2	4	
$\hat{h}_0$		1	2			
$\hat{h}_1$	3		4			
$\hat{h}_2$	5	6				

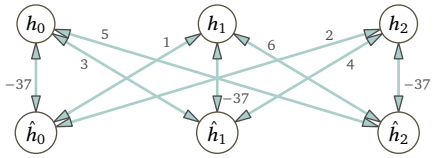
This is the graph of the first stage of $\mathcal{H}$ showing the symmetric edges with arrows on both ends.



We won't distinguish between that and an undirected graph.

(C) The adjusted matrix, the graph matrix for $\mathcal{H}$, is also symmetric.

$$\begin{array}{c|cccccc} & h_0 & h_1 & h_2 & \hat{h}_0 & \hat{h}_1 & \hat{h}_2 \\ h_0 & & & & -37 & 3 & 5 \\ h_1 & & & & 1 & -37 & 6 \\ h_2 & & & & 2 & 4 & -37 \\ \hat{h}_0 & -37 & 1 & 2 & & & \\ \hat{h}_1 & 3 & -37 & 4 & & & \\ \hat{h}_2 & 5 & 6 & -37 & & & \end{array}$$



(D) We first cover the case where the asymmetric instance is the pictured graph $\mathcal{G}$ and the symmetric instance is the derived graph $\mathcal{H}$. To make a circuit in $\mathcal{G}$, the only possibilities are going clockwise and going counterclockwise. Clockwise costs 11, so the least-expensive circuit is $g_0 \to g_1 \to g_2 \to g_0$, of total cost 10.

Observe that twice the number of vertices in $\mathcal{G}$ equals the number of vertices in $\mathcal{H}$, and so w is the number of edges in $\mathcal{H}$ multiplied by the maximum edge weight in $\mathcal{H}$. For $\mathcal{H}$, the large negative weights of $-w-1$ ensure that those edges must be part of a circuit.

To prove that, first observe that a to visit every vertex, a circuit must have a number of edges equal to the number of vertices, which here is six edges. A circuit that does not use a edge with weight -37 has total weight at least $6 \cdot 1$. Replacing one positive weight with -37 means that any circuit now has total weight at most $-37 \cdot 1 + 6 \cdot 5 = -7$, so that lowers the total weight. From there, every circuit with one -37 edge has total weight at least $(-37) \cdot 1 + 1 \cdot 5 = -32$, while replacing another positive weight edge means that the most a circuit can weigh is $(-37) \cdot 2 + 6 \cdot 4 = -50$, an improvement. Finally, every circuit containing two -37 edges totals at least $(-37) \cdot 2 + 1 \cdot 4 = -70$, while replacing a positive weight edge means that a circuit can weigh at most $(-37) \cdot 3 + 6 \cdot 3 = -93$. So three -37's is unbeatable, and by inspection there is such a circuit.

Therefore $\mathcal{H}$'s optimal solution is $h_0 \to \hat{h}_0 \to h_1 \to \hat{h}_1 \to h_2 \to \hat{h}_2 \to \hat{h}_2 \to h_0$. It alternates between edges of the form $h_i\hat{h}_i$ and those of the form $\hat{h}_i h_j$, where edges with matching subscripts $g_i g_j$ are in $\mathcal{G}$'s optimal circuit.

The argument generalizes to any number of edges in $\mathcal{G}$. Let v be the number of vertices in $\mathcal{G}$ and let m be its maximum edge weight, so that $w = vm$. Replacing a positive weight edge will change what a circuit can weight by $-w-1 = -vm-1$, and the remaining edges in any circuit using this replaced edge have a total weight less than vm, so there is a net decrease. And there is a circuit using v-may edges of weight $-w-1$.

The reduction function can clearly do the change from $\mathcal{G}$ to $\mathcal{H}$ in polytime.

V.6.33

(A) A little time playing with the table numbers

$C(t_i, w_j)$	w_0	w_1	w_2	w_3
t_0	13	4	7	6
t_1	1	11	5	4
t_2	6	7	2	8
t_3	1	3	5	9

shows that the optimum solution is to pair t_0, w_1, along with t_1, w_3, and t_2, w_2, and finally t_3, w_0. We can check that with a routine.

```
(define cost-table
  #[#[13 4 7 6]
    #[1 11 5 4]
    #[6 7 2 8]
```

```
    #[1 3 5 9]])

(define (get-cost i j cost-table)
  (vector-ref (vector-ref cost-table i) j))

;; Find minimum cost of matching workers with tasks
;;   For every permutation of task indices, make a list of costs,
;;   then add, and then find the min.
(define (minimize-assignment cost-table)
  (apply min
         (for*/list ([task-perm (in-permutations '(0 1 2 3))])
           (apply + (map (lambda (i j) (get-cost i j cost-table))
                         '(0 1 2 3)
                         task-perm)))))
```

The total cost is 11.

```
> (minimize-assignment cost-table)
11
```

(B) The language for the Assignment problem is this.

$$\mathcal{L}_0 = \{\, \langle W, T, C, B\rangle \;\big|\; W \text{ and } T \text{ are same-sized sets and } C\colon W\times T\to\mathbb{R}, \text{ and there is an assignment } a\colon W\to T \text{ costing no more than } B \,\}$$

This is the language for the Asymmetric Traveling Salesman problem.

$$\mathcal{L}_1 = \{\, \langle \mathcal{G}, D\rangle \;\big|\; \mathcal{G} \text{ is a directed weighted graph having a circuit having total weight no more than } D \,\}$$

A reduction function inputs a $\langle W, T, C, B\rangle$ and outputs a $\langle \mathcal{G}, D\rangle$. As in the hint, take the output graph $\mathcal{G}$ to have vertices labeled with each w_i and t_j, but there are no inter-w edges or inter-t edges. For each pair w_i, t_j there is a directed edge in each direction. Give the edge from w_i to t_j a weight of $C(w_i, t_j)$. Give the edge from t_j to w_i a weight of 0.

If the input $\langle W, T, C, B\rangle$ is an element of $\mathcal{L}_0$ then the total cost of some assignment a does not exceed B. Let that assignment be $w_0 \mapsto t_{i_0}$, $w_1 \mapsto t_{i_1}$, ... Then in $\mathcal{G}$ the circuit starting at the vertex labeled w_0 and traveling to the vertex t_{i_0}, then to w_1 and t_{i_1}, etc. until returning back to w_0, will be of total weight less than $D = B$. That is, $\langle \mathcal{G}, B\rangle$ is an element of $\mathcal{L}_1$.

Clearly we can write reduction function associating this input and output to run in polytime.

V.6.34

(A) This has nothing to do with the 'polytime'. If they were reducible then there would be a function f such that $n \in \mathbb{N}$ if and only if $f(n) \in \emptyset$. But the former is always true while the latter is always false.

(B) Suppose that there is a computable function f such that $x \in A$ if and only if $f(x) \in B$, and suppose that B is computably enumerable. Then computably enumerate the set A by dovetailing an enumeration of B with an enumeration of the range of f: $f(0), f(1), \ldots$; when an element appears on both lists, $f(i) = b_j$, then enumerate i into A.

As to $K^{\mathsf{c}} \not\leq_p K$, the set K is computably enumerable but K^{c} is not.

(C) Assume that $\mathcal{L} \leq_p \mathcal{L}^{\mathsf{c}}$. Then there is a polytime f so that $\sigma \in \mathcal{L}$ if and only if $f(\sigma) \in \mathcal{L}^{\mathsf{c}}$. That's equivalent to $\sigma \notin \mathcal{L}$ if and only if $f(\sigma) \notin \mathcal{L}^{\mathsf{c}}$, which holds when $\sigma \in \mathcal{L}^{\mathsf{c}}$ if and only if $f(\sigma) \in \mathcal{L}$.

V.6.35

(A) Reflexivity is easy; we must show that $\mathcal{L} \leq_p \mathcal{L}$ for any $\mathcal{L} \in \mathscr{P}(\Sigma^*)$. Take the reduction function to be the identity, $f(\sigma) = \sigma$. Clearly it can be computed in polytime, and just as clearly $\sigma \in \mathcal{L}$ if and only if $f(\sigma) \in \mathcal{L}$.

For transitivity let $\mathcal{L}_0 \leq_p \mathcal{L}_1$ via the function f and let $\mathcal{L}_1 \leq_p \mathcal{L}_2$ via the function g, which are polytime computable. Then for all $\sigma \in \Sigma^*$ we have $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$, and $f(\sigma) \in \mathcal{L}_1$ if and only if $g \circ f(\sigma) \in \mathcal{L}_2$. Because the two functions are polytime, so is their composition.

(B) Suppose that $\mathcal{L} \leq_p \emptyset$. Then there is a function f such that $\sigma \in \mathcal{L}$ if and only if $f(\sigma) \in \emptyset$. Since $f(\sigma)$ is never an element of the empty set, σ is never an element of $\mathcal{L}$. That is, any language that polytime reduces to the empty set is empty.

The argument that $\mathcal{L} \leq_p \mathbb{B}^*$ implies that $\mathcal{L} = \mathbb{B}^*$ is just as easy.

(C) Assume that $\mathcal{L}_0 \leq_p \mathcal{L}_1$, so that there is a polytime function $f\colon \Sigma^* \to \Sigma^*$ such that $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$. We will use this reduction function to show that $\mathcal{L}_1 \in NP$ implies that $\mathcal{L}_0 \in NP$.

Assume that $\mathcal{L}_1 \in NP$. Then there is a nondeterministic Turing machine $\mathcal{P}_1$ that decides $\mathcal{L}_1$. For a machine $\mathcal{P}_0$ that decides $\mathcal{L}_0$, start with an input candidate string $\tau \in \Sigma^*$. Compute $f(\tau)$. Run $\mathcal{P}_1$ on input $f(\tau)$. If it accepts then $\mathcal{P}_0$ accepts, while if it rejects then $\mathcal{P}_0$ rejects.

V.6.36

(A) As described in the section, $B \leq_m A$ if there is a total computable function f such that we make one oracle call, asking whether $f(x) \in A$, and then returning the result of that call.

(B) Suppose that $B \leq_m A$ via the total computable function f, and suppose also that B is computably enumerable. As we enumerate $B = \{b_0, b_1 \ldots\}$, we can also enumerate $A = \{f(b_0), f(b_1) \ldots\}$. Therefore A is computably enumerable.

Now for the conclusion. If a set and its complement are both computably enumerable then both are computable. But of course K is not computable, so the prior paragraph means that $K \not\leq_m K^{\mathsf{c}}$.

V.6.37 We answer each with specific examples, but there are many other correct answers. For instance, for the first we can use any two members of P where one is a proper subset of the other.

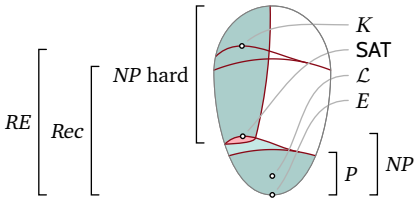
(A) Take $\mathcal{L}_0$ to be the language of strings whose length is a multiple of four, and $\mathcal{L}_1$ to be the language of strings whose length is even. Clearly $\mathcal{L}_0 \subset \mathcal{L}_1$, and both languages are in P so $\mathcal{L}_0 \leq_p \mathcal{L}_1$.

(B) Take $\mathcal{L}_0$ to be the language of even-length strings, and $\mathcal{L}_1$ to be the language of odd-length strings. Clearly both languages are in P, so $\mathcal{L}_0 \leq_p \mathcal{L}_1$.

(C) Recall that the only set that polytime reduces to $\mathbb{B}^*$ is $\mathbb{B}^*$ itself. Take $\mathcal{L}_0$ to be the language of even-length strings and $\mathcal{L}_1 = \mathbb{B}^*$.

(D) Recall that the only set that polytime reduces to $\varnothing$ is $\varnothing$ itself. Take $\mathcal{L}_0$ to be the language of even-length strings and take $\mathcal{L}_1 = \varnothing$.

V.7.14 Here is the figure with the sets.



(A) Every computably enumerable set reduces to K. So the language K is at the top of RE.

(B) Every set can compute the set E of (representations of) even numbers in the time that it takes to read in the input (for that matter, just in the time it takes to read in the least significant bit of the input). So it is is all the way at the bottom.

(C) There are a variety of algorithms for the shortest path problem, but all are fast (quadratic or faster), so $\mathcal{L}$ is in the middle of P.

(D) The SAT problem is *NP* complete so it is at the top of the *NP* problems.

Each trivial sets is $\leq_p$ related to no set other than itself, so they would sit off to the side, as just two lone points.

V.7.15 The number one mistake is that "not computable in polynomial time" isn't the criteria for membership in *NP*. Instead, the criteria for a problem being a member of *NP* is that it has a polytime verifier.

Another thing wrong is that the statement seems to confuse *NP* with the set difference $NP - P$. That is, the speaker seems to mean something like, "experts suspect that Satisfiability is a problem in *NP* but not in *P*." (Along with that, the speaker may well mean *NP* complete, not just an element of *NP*.)

One more thing wrong is that the statement seems to say that the problem is in that class because there is no algorithm yet known. It is not a question of whether we know of such an algorithm, it is a question of whether one exists.

V.7.16 This particular algorithm is indeed not polynomial. But how to prove that there does not exist another algorithm, and that algorithm is polynomial? No one knows.

V.7.17 In saying that the problem has a polytime algorithm, the commenter is saying that the problem is in *P*. But $P \subseteq NP$ and so this problem is in *NP*.

V.7.18 Computational classes such as *NP* do not contain algorithms, they contain languages. (We don't

distinguish too sharply between languages and problems, so may say that a problem is in a complexity class, but we do distinguish between a language and an algorithms to decide that language.)

V.7.19

(A) The statement "*NP* is a subset of the *NP* complete sets" is false. The truth is the other way around: *NP* complete is a subset of *NP*. On the other hand, the contention that *NP* complete is a subset of *NP* hard is true. It is the intersection of *NP* hard and *NP*.

(B) First, nondeterministic machines are a not a specialization of deterministic ones. Rather, nondeterministic machines are a generalization, that is, any deterministic machine is a nondeterministic one.

The clause about subset is trickier. Most experts guess that *NP* is not a subset of *P* but as described in the section, we have no proof.

V.7.20

(A) True. Because Primality $\in$ *NP*, it can be solved on a nondeterministic Turing machine. We can simulate such a machine with a deterministic one, and so we can at least in principle run that algorithm on currently-existing physical computers.

(B) False; we cannot infer this from the given facts. It may be that there is an efficient algorithm, but we don't know. (We don't know from the statement as given, and it is also not known to working researchers today.)

(C) False. The Traveling Salesman problem is *NP* complete. Thus, from Prime Factorization $\in$ *NP*, we can deduce that Prime Factorization $\leq_p$ Traveling Salesman, and from an efficient algorithm for Traveling Salesman we would get an efficient algorithm for Prime Factorization. But this is the reverse of what the question asks. The question says that Prime Factorization is not known to be *NP* complete, so we don't know the other direction.

V.7.21

(A) False; for example, the brute force algorithm of checking all circuits solves all instances.

(B) True, where we use Cobham's Thesis to take 'quickly' to mean polytime and remembering this exercise's assumption that $P \neq NP$.

(C) False. If Traveling Salesman $\in P$ then every *NP* problem is in *P*, because every *NP* problem reduces to Traveling Salesman. That contradicts the $P \neq NP$ assumption.

(D) False. There is at least one algorithm, brute force, so this would imply that Traveling Salesman is in *P*, which the prior item says is impossible.

V.7.22

(A) No, from $\mathcal{L}_1 \leq_p \mathcal{L}_0$ and $\mathcal{L}_0$ is *NP* complete, we cannot conclude that $\mathcal{L}_1$ is *NP* complete. An example is where $\mathcal{L}_0$ is the Satisfiability problem and $\mathcal{L}_1$ is the problem of determining whether an input string contains an even number of 1's.

(B) False; it may be that $\mathcal{L}_0$ is not a member of *NP*. An example is $\mathcal{L}_1$ = SAT and this set.

$$\mathcal{L}_0 = \{\sigma \in \mathbb{B}^* \;\big|\; \sigma \text{ represents } n \text{ in binary and } \phi_n(n)\!\downarrow\}$$

(C) False. This is a reprise of the first item: from $\mathcal{L}_1 \leq_p \mathcal{L}_0$ and $\mathcal{L}_0$ is *NP* complete, you cannot conclude that $\mathcal{L}_1$ is *NP* complete, even knowing that $\mathcal{L}_1 \in NP$. An example is where $\mathcal{L}_0$ is the Satisfiability problem, and $\mathcal{L}_1$ is the problem of determining whether an input number is even.

(D) True; this is Lemma 7.4. If $\mathcal{L}_1$ is *NP* complete then any $\mathcal{L} \in NP$ is reducible to $\mathcal{L}_1$, so $\mathcal{L} \leq_p \mathcal{L}_1$, and transitivity of $\leq_p$ gives that $\mathcal{L} \leq \mathcal{L}_0$. That, along with the statement that $\mathcal{L}_0 \in NP$, gives that $\mathcal{L}_0$ is *NP* complete.

(E) False. A problem is *NP* complete if it is in *NP* and every problem $\mathcal{L}$ in *NP* reduces to it. So two *NP* complete problems reduce to each other. For example, any of the problems on the Basic List are reducible to any of the others, or to Satisfiability.

(F) False. An example is that $\mathcal{L}_1$ is the problem of determining whether a number is even, and $\mathcal{L}_0$ is a problem whose fastest solution algorithm is exponential (such as chess).

(G) True. This is Lemma 6.9, that *P* is closed downward.

V.7.23 We just give one answer for each but other answers might be correct. (If $P = NP$ then all nontrivial sets are *NP* hard, but these answers apply also if $P \neq NP$.)

(A) SAT

(B) $E = \{\sigma \in \mathbb{B}^* \;\big|\; \sigma \text{ represents in binary an even number}\}$

(C) SAT

(D) $\{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents in binary a number } n \text{ where } \phi_n(n)\downarrow\}$

(E) The complement of the set in the prior item.

(F) $E = \{\sigma^n \in \mathbb{B}^* \mid \sigma \text{ represents in binary an even number}\}$

V.7.24 Assume that $P = NP$. Fix a language $\mathcal{L} \in P$ that is nontrivial, so $\mathcal{L} \neq \emptyset$ and $\mathcal{L} \neq \mathbb{N}$. Because $P \subseteq NP$ we have $\mathcal{L} \in NP$.

What remains is to show that $\mathcal{L}$ is NP hard, that every language $\hat{\mathcal{L}} \in NP$ satisfies that $\hat{\mathcal{L}} \leq_p \mathcal{L}$. But with $P = NP$ then $\hat{\mathcal{L}} \in P$, and Lemma 6.9 says that $\hat{\mathcal{L}} \leq_p \mathcal{L}$.

V.7.25

(A) The language is the set of strings $\{\sigma \in \mathbb{B}^* \mid \sigma \text{ represents an even number in binary}\}$. This set is in P (σ represents an even number if and only if the final bit is `0`). Thus it is in NP. Therefore if $\mathcal{L}$ is NP then $P = NP$.

(B) This is like the prior item, except that it is not as trivially a member of P. To see that this problem is a member of P, assume that we are given a graph $\mathcal{G}$. We can check for a size four vertex cover (a set of vertices that includes at least one endpoint for every edge) by checking all 4-tuples of vertices. That takes a loop with $\binom{k}{4}$ iterations, where k is the number of vertices in $\mathcal{G}$.

V.7.26 Move the head left, write a 1, and halt. This is a constant time algorithm.

V.7.27 The natural thing to try is to prove that 3-Satisfiability $\leq_p$ 4-Satisfiability.

The reduction function is the identity function. It inputs a clause with at most three literals, and outputs the same clause (which trivially has at most four literals). Clearly the input is satisfiable if and only if the output is satisfiable. Clearly also this reduction function runs in polytime.

V.7.28

(A) The sum of the elements of $\hat{S} = \{4, 6, 7, 13\}$ is $T = 30$.

(B) The sum of elements in $\hat{A} = \{6, 7, 19\}$ equals the sum of elements in $A - \hat{A} = \{3, 4, 12, 13\}$.

(C) Suppose that we can partition a set $A \subset \mathbb{N}$ into $\hat{A} \subset A$ and $A - \hat{A}$, with equal sums. Then the total of all of the elements of A is twice the sum of the elements of $\hat{A}$, and so is even.

(D) The Subset Sum problem is the language decision problem for this language.

$$\mathcal{L}_0 = \{\langle S, T\rangle \mid \text{a subset } \hat{S} \subseteq S \text{ has } \sum_{s\in\hat{S}} s = T\}$$

The Partition problem is the language decision problem for this language.

$$\mathcal{L}_1 = \{\langle A\rangle \mid \text{there is } \hat{A} \subset A \text{ with } \sum_{a\in\hat{A}} a = \sum_{a\in A-\hat{A}} a\}$$

(In $\mathcal{L}_0$, the elements are strings representing multisets S and targets T, while in $\mathcal{L}_1$ elements are strings representing multisets A. But we will minimize this technicality.)

(E) We must produce a polytime computable function that takes in $\sigma = \langle A\rangle$ and outputs $f(\sigma) = \langle S, T\rangle$, such that $\sigma \in \mathcal{L}_0$ if and only if $f(\sigma) \in \mathcal{L}_1$.

Given $\sigma = \langle A\rangle$, consider the sum of its elements $x = \sum_{a\in A} a$. Where

$$f(\sigma) = \begin{cases} \langle S, x/2\rangle & \text{– if } x \text{ is even} \\ \langle S, x+1\rangle & \text{– otherwise} \end{cases}$$

we have that $\sigma \in \mathcal{L}_1$ if and only if $f(\sigma) \in \mathcal{L}_0$, as required, and clearly f is computable in polytime.

(F) The Partition problem is one of the NP complete sets listed in Theorem 7.5. So Subset Sum is NP hard. But Subset Sum is in NP since as a witness ω we can use the subset. So it is NP complete.

V.7.29

(A) There is a Hamiltonian path: $\langle q_3, q_6, q_4, q_7, q_8, q_5, q_0, q_1, q_2\rangle$.

(B) Take $B \in \mathbb{N}$ to be the parameter and consider this family of languages.

$$\mathcal{L} = \{\langle \mathcal{G}, B\rangle \mid \mathcal{G} \text{ has a simple path of length at least } B\}$$

(C) Given an instance $\mathcal{G}$ of the Hamiltonian Path problem, let the reduction function convert it into the pair $\langle G, |G|-1\rangle$ (where $|\mathcal{G}|$ is the number of vertices in $\mathcal{G}$). A simple path containing $|G|-1$ edges must be Hamiltonian. Obviously that computation can be done in polytime.

(D) The prior exercise shows that the Hamiltonian Path problem is *NP* complete and that, along with the other parts of this exercise, gives that Longest Path is *NP* complete.

V.7.30

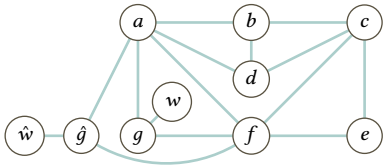
(A) In short, the witness ω is the path.

At greater length, to apply Lemma 5.11, we will produce a deterministic Turing machine verifier, $\mathcal{V}$. It inputs pairs $\langle \mathcal{G}, \omega\rangle$, and must satisfy that if $\mathcal{G}$ has a Hamiltonian path then there is a witness ω such that $\mathcal{V}$ accepts the input pair, while if $\mathcal{G}$ does not have a Hamiltonian path then no witness will result in the pair being accepted. The verifier must run in polytime.

The verifier here interprets the witness ω to be a Hamiltonian path through the graph, so it checks that it is indeed a path in the given graph and that every vertex is in the path. If there is such a path then there is a witness that the routine can verify. If there is no such path then no string ω will suffice. The verifier clearly runs in polytime (the size of a graph is polynomial in the number of vertices since the number of edges is $\mathcal{O}(|\mathcal{N}|)$).

(B) A Hamiltonian path is $\langle v_0, v_7, v_1, v_2, v_3, v_6, v_5, v_4, v_8\rangle$. The two v_0 and v_8 must be the start and end of the path because they have degree 1.

(C) In the starting graph a Hamiltonian circuit is $\langle g, a, d, b, c, e, f, g\rangle$. Here is the enhanced graph.



A Hamiltonian path is $\langle w, g, a, d, b, c, e, f, \hat{g}, \hat{w}\rangle$.

(D) We must produce a reduction function f. It inputs a graph $\mathcal{G}$ and outputs a graph $f(\mathcal{G}) = \mathcal{H}$ with the property that $\mathcal{G}$ has a Hamiltonian circuit if and only if $\mathcal{H}$ has a Hamiltonian path.

As in the prior item, given $\mathcal{G}$, we form $\mathcal{H}$ by adding three vertices. First, fix a vertex $v \in \mathcal{G}$. Add a new vertex $\hat{v} \in \mathcal{H}$ that has the same connections as v, meaning that for every edge $v_i v$ add $\hat{v}v_i$ and for every edge of the form vv_j add $\hat{v}v_j$. Also add two vertices of degree one: w connects only to v, and $\hat{w}$ connects only to $\hat{v}$.

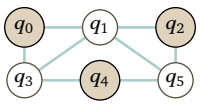
We must show that there is a Hamiltonian path in $\mathcal{H}$ if and only if there is a Hamiltonian circuit in $\mathcal{G}$. If there is a Hamiltonian circuit in $\mathcal{G}$ then the Hamiltonian path in $\mathcal{H}$ is derived by splitting the circuit at v, and hooking it to a path in $\mathcal{H}$ that starts at w, goes to v, then through the circuit, out to $\hat{v}$, and then to $\hat{w}$.

For the other direction suppose that there is a Hamiltonian path in H. Then its start and end must be w and $\hat{w}$ because these vertices have degree 1. It must then proceed to v and $\hat{v}$. Because $\hat{v}$ is a copy of v, a Hamiltonian path in $\mathcal{H}$ that starts off at these vertices corresponds to a Hamiltonian circuit in $\mathcal{G}$.

(E) The Hamiltonian Circuit problem is on the list of Basic *NP* Complete Problems, so the fact that it reduces to Hamiltonian Path means that Hamiltonian Path is *NP* hard. Because this exercise's first item shows that Hamiltonian Path is a member of *NP*, it is *NP* complete.

V.7.31

(A) The natural independent set is this.



(B) This is the language.

$$\mathcal{L}_0 = \{\, \langle \mathcal{G}, B\rangle \;\big|\; \mathcal{G} \text{ has at least } B\text{-many vertices that do not share an edge} \,\}$$

(C) This is the language for 3-SAT.

$$\mathcal{L}_1 = \{\, \langle E\rangle \;\big|\; E \text{ is a satisfiable CNF expression with at most 3 literals per clause} \,\}$$

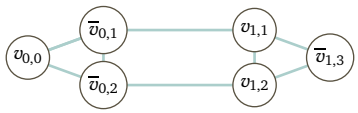
P_0	P_1	P_2	P_3	$P_0 \vee \neg P_1 \vee \neg P_2$	$P_1 \vee P_2 \vee \neg P_3$	E
F	F	F	F	T	T	T
F	F	F	T	T	F	F
F	F	T	F	T	T	T
F	F	T	T	T	T	T
F	T	F	F	T	T	T
F	T	F	T	T	T	T
F	T	T	F	F	T	F
F	T	T	T	F	T	F
T	F	F	F	T	T	T
T	F	F	T	T	F	F
T	F	T	F	T	T	T
T	F	T	T	T	T	T
T	T	F	F	T	T	T
T	T	F	T	T	T	T
T	T	T	F	T	T	T
T	T	T	T	T	T	T

FIGURE 104, FOR QUESTION V.7.31: Truth table for $E = (P_0 \vee \neg P_1 \vee \neg P_2) \wedge (P_1 \vee P_2 \vee \neg P_3)$.

(The role played by the parameter B in $\mathcal{L}_0$ will be played here by the number of clauses in the expression E.)

(D) See Figure 104 on page 217 for E's truth table. One way to satisfy E is by taking $P_0 = T$ and $P_1 = P_2 = P_3 = F$.

(E) The graph associated with the expression is this.



(F) We must produce a reduction function, f. It inputs an instance of the 3-Satisfiability problem, E, and constructs a graph $\mathcal{G}$ as in the prior item. That is, for the j-th clause it adds three vertices, labeled with $v_{j,i}$ if the literal is P_i and with $\overline{v}_{j,i}$ if the literal is $\neg P_i$. It connects these three vertices completely, making a triangle. The reduction function also includes an edge for any two vertices labeled with the same second index i and where one has an overline and the other does not, and thus possibly making inter-triangular edges. Clearly this construction of $\mathcal{G}$ can happen in polytime.

We claim that an expression E with B-many clauses (and with at most three literals per clause) is satisfiable if and only $\langle \mathcal{G}, B\rangle \in \mathcal{L}_0$ where $\mathcal{G} = f(E)$.

First suppose that E is satisfiable. Then in every clause of E there is at least one literal that is T. Pick one such literal from each clause, denote the one from clause i with $\ell(i)$ and consider the set of associated vertices.

$$S = \{\, v_{i,\ell(i)} \text{ or } \overline{v}_{i,\ell(i)} \;\big|\; \text{the associated literal in clause } i \text{ is } T \,\}$$

We must verify that S has B distinct elements and is an independent set. It has B elements because each member is associated with a different clause of E, and so falls into a different triangle. It is an independent set because no two members are connected by an edge: in the construction there are two kinds of edges, those within a triangle and inter-triangular ones. No two members of S are in the same triangle so the first case is out. As for inter-triangular ones, they are between vertices associated with a literal and its negation. But a literal and its negation cannot both be T, so the associated vertices are not both members of S.

To finish, suppose that $\langle \mathcal{G}, B\rangle \in \mathcal{L}_0$ and so $\mathcal{G}$ contains a size B independent set of vertices $\hat{S}$. We claim that taking the associated set of literals to be T will satisfy the expression E. Note that because the construction of $\mathcal{G}$ is such that there is an edge between vertices associated with a literal and its negation, we can set all of the associated literals to be T. Also note that because all vertices inside a triangle are connected, the

B many members of $\hat{S}$ come from different triangles and so are associated with literals from the B many different clauses. So each clause has a literal that evaluates to T, and thus E is satisfied.

V.7.32 No. If we knew of a problem in $NP - P$ then we'd know that $P \neq NP$. We don't know that. (Experts do suspect that some problems are in this area, including the Vertex-to-Vertex Path problem. But until it is proved, that remains speculation.)

V.7.33 Take $\mathcal{L}_1 = \{ 0 \frown \sigma \mid \sigma \in \Sigma^* \}$. Deciding membership in this language is just a matter of deciding whether the first character is 0, so $\mathcal{L}_1$ is an element of P.

For the other two, let $\mathcal{L}$ be any NP complete language of bitstrings. Take $\mathcal{L}_2 = \{ 0 \frown \sigma \mid \sigma \in \mathcal{L} \}$ and $\mathcal{L}_0 = \mathcal{L}_1 \cup \{ 1 \frown \sigma \mid \sigma \in \mathcal{L} \}$.

V.7.34 Assume that $\mathcal{L}_0$ is NP complete. Assume also that $\mathcal{L}_0 \leq_p \mathcal{L}_1$ and $\mathcal{L}_1 \in NP$. To verify that $\mathcal{L}_1$ is NP complete we must show that it is NP hard, that for every $\mathcal{L} \in NP$ we have $\mathcal{L} \leq_p \mathcal{L}_1$.

The assumption that $\mathcal{L}_0$ is NP complete gives $\mathcal{L} \leq_p \mathcal{L}_0$. Thus $\mathcal{L} \leq_p \mathcal{L}_0 \leq_p \mathcal{L}_1$ and transitivity of polytime reduction from Lemma 6.9 gives that $\mathcal{L} \leq_p \mathcal{L}_1$.

V.7.35

(A) Fix $\mathcal{L}, \hat{\mathcal{L}} \in NP$. There are associated polytime verifiers $\mathcal{V}$ and $\hat{\mathcal{V}}$. A verifier for the union can just run those two on the input, and accept if either accepts.

(B) Fix $\mathcal{L}, \hat{\mathcal{L}} \in NP$. There are associated polytime verifiers $\mathcal{V}$ and $\hat{\mathcal{V}}$. A verifier for the concatenation inputs a string σ, and loops over splitting it into substrings $\sigma = \sigma[0 : i] \frown \sigma[i :]$ for $i \in [0 .. |\sigma|)$, testing whether both verifiers accept. Each verifier is polytime, and running the loop keeps the overall algorithm runtime inside of polytime.

(C) The class P is closed under complement. Consequently, if NP is not closed under complement then $P \neq NP$. (But our present state of knowledge is that it is possible that NP is closed under complement but is still different than P).)

V.7.36 Countable. It is a subset of NP, which is countable because a verifier is a deterministic Turing machine, and there are countably many such machines.

V.A.12 They are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, and 97.

V.A.13

(A) $n = pq = 134$, $(p-1) \cdot (q-1) = 120$

(B) Use $e = 7$ (the prior three 2, 3, and 5 are not relatively prime).

(C) The multiplicative inverse of $e = 7$ modulo $n = 134$ is $d = 103$.

(D) The encryption is $m^e \bmod n = 9^7 \bmod 143 = 48$. The decryption is $48^{103} \bmod 143 = 9$.

V.A.14

(A) With two factors $n = k_0 \cdot k_1$ if one is greater than $\sqrt{n}$ then the other must be less.

(B) If n is a perfect square $n = k \cdot k$ then its first nontrivial factor is $k = \sqrt{n}$.

(C) The square root of 10^{12} is $10^6 = 1\,000\,000$, a million.

(D) The input n has size about $\lg(n)$ bits. Thus $\sqrt{n} \approx \sqrt{2^{\text{number digits of } n}} \approx 2^{(1/2)\cdot\text{number digits of } n}$.

V.B.1 The main point relevant to this section is that for an input as small as size 100 we have very capable solvers, which will return the answer quite quickly. The fact that a problem is NP hard does not mean that most of its instances are slow to solve.

V.B.2 There are $n!$ circuits.

V.B.3 See Figure 105 on page 219. Basically, you can go around the outside for 14 or go through the middle for 15.

V.C.1 We can adjust the initial board to match the new one. Here are the first few lines.

```
; Exercise
(define INITIAL-CLAUSES
  (list (list '(1 2 3)) ; there is a 3 in position (1,2)
        (list '(1 3 4))
        (list '(1 4 5))
        (list '(1 6 6))
        (list '(1 7 9))
```

Then hitting the Run button in Dr Racket gives the output of all the clauses. This command drops them all to

Vertex sequence	*First edge*	*Second*	*Third*	*Return*	*Total*
v_0-v_1-v_2-v_3	3	1	4	7	15
v_0-v_1-v_3-v_2	3	5	4	2	14
v_0-v_2-v_1-v_3	2	1	5	7	15
v_0-v_2-v_3-v_1	2	4	5	3	14
v_0-v_3-v_1-v_2	7	5	1	2	15
v_0-v_3-v_2-v_1	7	4	1	3	15
v_1-v_0-v_2-v_3	3	2	4	5	14
v_1-v_0-v_3-v_2	3	7	4	1	15
v_1-v_2-v_0-v_3	1	2	7	5	15
v_1-v_2-v_3-v_0	1	4	7	3	15
v_1-v_3-v_0-v_2	5	7	2	1	15
v_1-v_3-v_2-v_0	5	4	2	3	14
v_2-v_0-v_1-v_3	2	3	5	4	14
v_2-v_0-v_3-v_1	2	7	5	1	15
v_2-v_1-v_0-v_3	1	3	7	4	15
v_2-v_1-v_3-v_0	1	5	7	2	15
v_2-v_3-v_0-v_1	4	7	3	1	15
v_2-v_3-v_1-v_0	4	5	3	2	14
v_3-v_0-v_1-v_2	7	3	1	4	15
v_3-v_0-v_2-v_1	7	2	1	5	15
v_3-v_1-v_0-v_2	5	3	2	4	14
v_3-v_1-v_2-v_0	5	1	2	7	15
v_3-v_2-v_0-v_1	4	2	3	5	14
v_3-v_2-v_1-v_0	4	1	3	7	15

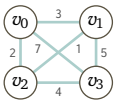


FIGURE 105, FOR QUESTION V.B.3: An exhaustive search for the Traveling Salesman problem.

soduku.cfg.

```
> (dump-to-file)
```

With the data in the external file we can run the command to implement the SAT solver.

```
jim@millstone:~/Documents/computing/src/scheme/complexity$ ./run-sat-solver.sh
WARNING: for repeatability, setting FPU to use double precision
============================[ Problem Statistics ]=============================
|                                                                             |
|  Number of variables:           729                                         |
|  Number of clauses:            2467                                         |
|  Parse time:                   0.00 s                                       |
|  Eliminated clauses:           0.00 Mb                                      |
|  Simplification time:          0.00 s                                       |
|                                                                             |
============================[ Search Statistics ]==============================
| Conflicts |          ORIGINAL         |          LEARNT          | Progress |
|           |    Vars  Clauses Literals |    Limit  Clauses Lit/Cl |          |
===============================================================================
|       100 |     414     1928     6686 |      706      100     12 | 30.864 % |
|       250 |     414     1928     6686 |      777      250     12 | 30.864 % |
|       475 |     414     1928     6686 |      855      475     13 | 30.864 % |
|       812 |     414     1928     6686 |      940      812     13 | 30.864 % |
|      1318 |     414     1928     6686 |     1035      717     13 | 30.865 % |
|      2077 |     413     1921     6659 |     1138     1254     14 | 31.002 % |
|      3216 |     411     1914     6635 |     1252      980     13 | 31.277 % |
|      4924 |     400     1854     6458 |     1377      775     12 | 32.785 % |
|      7486 |     378     1780     6256 |     1515     1620     11 | 35.803 % |
|     11330 |     368     1664     5938 |     1666     1784     12 | 37.174 % |
===============================================================================
restarts              : 60
conflicts             : 13476          (255867 /sec)
decisions             : 21367          (0.00 % random) (405692 /sec)
propagations          : 240090         (4558555 /sec)
conflict literals     : 166762         (18.93 % deleted)
Memory used           : 11.00 MB
CPU time              : 0.052668 s

SATISFIABLE
```

Finally, in the file sat-solver.rkt are some more routines, including this one.

```
> (show-solved-board)
'#(#(7 3 4 5 1 6 9 2 8)
   #(8 1 5 4 9 2 7 6 3)
   #(6 9 2 3 8 7 4 1 5)
   #(9 4 6 8 2 3 1 5 7)
   #(1 2 8 7 5 4 3 9 6)
   #(5 7 3 1 6 9 2 8 4)
   #(2 8 7 6 3 1 5 4 9)
   #(3 5 1 9 4 8 6 7 2)
   #(4 6 9 2 7 5 8 3 1))
```

Appendices

A.1

(A) $\sigma^\frown\tau = \texttt{10110}^\frown\texttt{110111} = \texttt{10110110111}$.

(B) $\sigma^\frown\tau^\frown\sigma = \texttt{10110}^\frown\texttt{110111}^\frown\texttt{10110} = \texttt{1011011011110110}$

(C) $\sigma^{\mathsf{R}} = \texttt{10110}^{\mathsf{R}} = \texttt{01101}$

(D) $\sigma^3 = \texttt{10110}^\frown\texttt{10110}^\frown\texttt{10110} = \texttt{101101011010110}$

(E) $\texttt{0}^{3\frown}\sigma = \texttt{000}^\frown\texttt{10110} = \texttt{00010110}$

A.2

(A) abbca

(B) ababbcabca

(C) baacb

(D) ababab

A.3 There are $2^4 = 16$ bit strings of length 4 and half of them start with `0`, so there are 8 strings in the language.

A.4

(A) Yes.

(B) Yes.

(C) No.

(D) Yes.

A.5 The length of a concatenation is the sum of the two lengths: $|\sigma^\frown\tau| = |\langle s_0, \ldots\ s_{i-1}, t_0, \ldots\ t_{j-1}\rangle| = i + j = |\sigma| + |\tau|$.

A.6

(A) True. The concatenation of the first is $\alpha^\frown\varepsilon = \langle a_0, \ldots\ a_{i-1}\rangle^\frown\langle\rangle = \langle a_0, \ldots\ a_{i-1}\rangle = \alpha$. The second is much the same.

(B) False. A counterexample is the two bitstrings $\alpha = \texttt{0}$ and $\beta = 1$.

(C) False. As with the prior item, a counterexample is the two bitstrings $\alpha = \texttt{0}$ and $\beta = 1$.

(D) True. Where $\alpha = \langle a_0, \ldots\ a_{i-1}\rangle$ we have $\alpha^{\mathsf{R}} = \langle a_{i-1}, \ldots\ a_0\rangle$ and so $\alpha^{\mathsf{R}^{\mathsf{R}}} = \langle a_0, \ldots\ a_{i-1}\rangle$.

(E) False. A counterexample is the bitstring $\alpha = \texttt{01}$ with the power $i = 1$.

B.1

(A) For one-to-one, suppose that $f(x_0) = f(x_1)$ for $x_0, x_1 \in \mathbb{R}$. Then $3x_0 + 1 = 3x_1 + 1$. Subtract the 1's and divide by 3 to conclude that $x_0 = x_1$.

For onto, fix a member of the codomain, $c \in \mathbb{R}$. Observe that $d = (c-1)/3$ is a member of the domain and that $f(d) = 3 \cdot ((c-1)/3) + 1 = (c-1) + 1 = c$. Thus f is onto.

(B) The function g is not one-to-one because $g(2) = g(-2)$. It is not onto because no element of the domain $\mathbb{R}$ is mapped by g to the element 0 of the codomain.

B.2

(A) The function g is a left inverse of f because $g \circ f$ has the action $(x, y) \mapsto (x, y, 0) \mapsto (x, y)$ for all x and y. It is not a right inverse of f because $f \circ g$ has the action $(x, y, z) \mapsto (x, y) \mapsto (x, y, 0)$, which is not the identity function; for instance, $(1, 2, 3) \mapsto (1, 2) \mapsto (1, 2, 0)$.

(B) Observe that $f(2) = f(-2) = 4$. Any left inverse would have to map 4 to 2, and also to map 4 to -2. That would be not well-defined, so there is no function that is the left inverse.

(C) One right inverse is $g_0 : C \to D$ given by $10 \mapsto 0$, $11 \mapsto 1$. It is a right inverse because $f \circ g_0\,(10) = f(g_0(10)) = f(0) = 10$ and $f \circ g_0\,(11) = f(g_0(11)) = f(1) = 11$. A second right inverse is $g_1 : C \to D$ given by $10 \mapsto 2$, $11 \mapsto 3$.

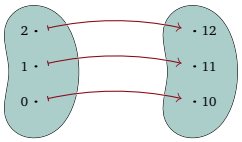
B.3

(A) We must show that it is a two-sided inverse. First, the domain of each equals the codomain of the other so both compositions $f \circ g$ and $g \circ f$ are defined. Next, g is a left inverse of f because $g \circ f\,(a) = g(f(a)) = g(a+3) = (a+3) - 3 = a$ is the identity map. Similarly, it is a right inverse because $f \circ g\,(a) = (a-3) + 3$ is also the identity map.

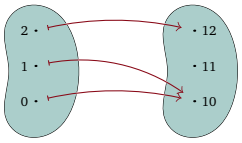
(B) The inverse of h is itself. The domain and codomain are equal, as required. In the case that n is odd, $h \circ h\,(n) = h(n-1) = (n-1) + 1 = n$ (the second equality holds because in this case $n - 1$ is even). Similarly if n is even then $h \circ h\,(n) = (n+1) - 1 = n$.

(C) The inverse of s is the map $r\colon \mathbb{R}^+ \to \mathbb{R}^+$ given my $r(x) = \sqrt{x}$ (that is, the positive root). The domain of each is the codomain of the other, and each of $s \circ r$ and $r \circ s$ is the identity function.

B.4 This is a bean diagram of the function f



and this is a diagram of g.



(A) By inspection f is both one-to-one and onto.

(B) Reverse the association made by f so that $f^{-1}(10) = 0$, $f^{-1}(11) = 1$, and $f^{-1}(12) = 2$.

(C) The map g is not one-to-one since $g(0) = g(1)$. (An alternative correct answer is that g is not onto since no input is sent to $11 \in C$.)

(D) If a h were the inverse of g then we would have both $h(10) = 0$ and $h(10) = 1$, which would violate the 'well-defined' condition in the definition of a function.

B.5

(A) Let $f\colon D \to C$ and $g\colon C \to B$ be one-to-one. Aiming to verify that the composition is one-to-one, suppose that $g \circ f\,(x_0) = g \circ f\,(x_1)$ for some elements $x_0, x_1 \in D$. Then $g(f(x_0)) = g(f(x_1))$. The function g is one-to-one and so, since the two values $g(f(x_0))$ and $g(f(x_1))$ are equal, the arguments $f(x_0)$ and $f(x_1)$ are also equal. The function f is one-to-one and so $f(x_0)$ and $f(x_1)$ gives that $x_0 = x_1$. Because $g \circ f\,(x_0) = g \circ f\,(x_1)$ implies that $x_0 = x_1$, the composition is one-to-one.

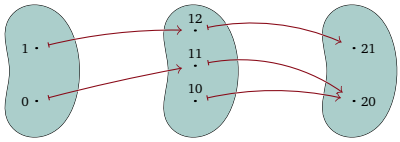
(B) Let $f\colon D \to C$ and $g\colon C \to B$ be onto. If B is the empty set then by the definition of function, C is empty also and, again by the definition of function, in turn $D = \emptyset$, and so the composition is onto, vacuously. Otherwise, fix an element of the codomain of the composition, $b \in B$. The function g is onto so there is a $c \in C$ such that $g(c) = b$. The function f is onto so there is a $d \in D$ such that $f(d) = c$. Then $g \circ f\,(d) = b$, and thus the composition is onto.

(C) Let $f\colon D \to C$ and $g\colon C \to B$ be such that the composition $g \circ f$ is one-to-one. Suppose that f is not one-to-one. Then there are $d_0, d_1 \in D$ with $d_0 \neq d_1$ such that $f(d_0) = f(d_1)$. But this implies that $g(f(d_0)) = g(f(d_1))$, contradicting that the composition is one-to-one.

(D) Let $f\colon D \to C$ and $g\colon C \to B$ be such that the composition $g \circ f$ is onto. Suppose for a contradiction that g is not onto. Then for some $b \in B$ there is no $c \in C$ with $g(c) = b$. But this means that there is no $d \in D$ such that $b = g(f(d))$, contradicting that the composition is onto. So g must be onto.

(E) The answer to both questions is "no."

Let $D = \{0, 1\}$, $C = \{10, 11, 12\}$, and $B = \{20, 21\}$. Let $f\colon D \to C$ and $g\colon C \to B$ be as shown below. Then $g \circ f$ is onto because $g \circ f\,(0) = 20$ and $g \circ f\,(1) = 21$. But f is not onto because no element of its domain D is mapped to the element 10 of its codomain C.



The same function is an example of a composition $g \circ f$ that is one-to-one but where g is not one-to-one. In particular, $g \circ f$ is one-to-one because only one element of its domain D is mapped to it's codomain element 20. Similarly only one element of its domain is mapped to 21.

B.6

(A) We first prove that if the function $f : D \to C$ has an inverse $f^{-1} : C \to D$ then f must be a correspondence. To verify that f is one-to-one, suppose that $d_0, d_1 \in D$ are such that $f(d_0) = f(d_1)$. Let $c \in C$ be $c = f(d_0) = f(d_1)$ and consider $f^{-1}(c)$. Because $f^{-1} \circ f$ is the identity map on D we have both that $f^{-1}(c) = f^{-1}(f(d_0)) = d_0$ and that $f^{-1}(c) = f^{-1}(f(d_1)) = d_1$. Thus $d_0 = d_1$ and so f is one-to-one.

Next we verify that f is onto. Suppose otherwise, so that there is a $c \in C$ that is not associated with any $d \in D$. But that gives a contradiction because the map $f \circ f^{-1}$ cannot satisfy $c = f \circ f^{-1}(c) = f(f^{-1}(c))$ as $c \notin \text{ran}(f)$. Therefore f must be onto.

To finish, we prove that if a map $f : D \to C$ is a correspondence then it has an inverse. We will consider an inverse association g relating elements of D with elements of C, and show that this relationship is a function.

For each $c \in C$, let g associates c with the element $d \in D$ such that $f(d) = c$. There is such an element d because f is onto. Also, this element is unique because f is one-to-one. Therefore, the relationship g is well-defined, that is, it is a function. That the compositions $g \circ f$ and $f \circ g$ are identity maps is clear.

(B) Suppose that $f : D \to C$ has two inverses $g_0, g_1 : C \to D$. We will show that the two same associate the same elements of the domain and codomain, that is, we will show that they have the same graph and are thus the same function.

Let c be any element of the domain C of the two g_0 and g_1. The prior item of this exercise shows that f is onto so there exists a $d \in D$ such that $f(d) = c$. The prior item also shows that f is one-to-one, so there is only one d with that property: $d = g_0(c)$ and $d = g_1(c)$. Therefore the two map the input c to the same output $g_0(c) = g_1(c)$.

(C) Observe that f^{-1} is a function that has an inverse, namely f. This is simply because in the equations $g \circ f^{-1} = \text{id}$ and $f^{-1} \circ g = \text{id}$ we can take g to be f. Then by the first item of this exercise, f^{-1} is a correspondence.

(D) Here is the verification that composition in one order gives the identity function.

$$(f^{-1} \circ g^{-1}) \circ (g \circ f)\,(x) = (f^{-1} \circ g^{-1})\,(g(f(x))) = f^{-1}(g^{-1}(g(f(x)))) = f^{-1}(f(x)) = x$$

Verification of the other order is similar.

B.7

(A) We will do induction on the number of elements in the range. Before the proof we give an illustration. Fix $D = \{0, 1, 2, 3\}$ and $C = \{10, 11, 12\}$. Consider the function $f : D \to C$ given here, which is such that $\text{ran}(f) = C$.

$$0 \mapsto 10 \quad 1 \mapsto 10 \quad 2 \mapsto 11 \quad 3 \mapsto 12$$

To do the induction, we pick an element of the range and omit it; suppose that we omit 10. We then want to consider the restriction map that is onto $C - \{10\}$. For this map to be well-defined, we must omit from its domain all of the elements that map to 10, the set $f^{-1}(10) = \{0, 1\}$. We then have the function $\hat{f}$ with domain $\hat{D} = \{2, 3\}$ and codomain $\hat{C} = \{11, 12\}$ given here.

$$2 \mapsto 11 \quad 3 \mapsto 12$$

The range of this restriction map $\hat{f}$ is $\hat{C}$. The inductive hypothesis therefore gives that $\hat{f}$'s domain, $\hat{D}$, has at least as many elements as its range, $\hat{C}$. To finish the argument, we will add back the omitted elements

to get f, adding back two elements from f's domain D and one from its range C, and conclude that its domain has at least as many elements as its range.

With that example in mind we can go to the induction proof. The base step is to consider a function f whose range is empty. The domain must also be empty since the definition of function requires that every element of the domain is mapped somewhere. Hence $|\operatorname{ran}(f)| \leq |D|$ because both of them are zero.

For the inductive step assume that the statement holds for all functions between finite sets where the range has 0 elements, or 1 element, ... or k elements. Consider $f\colon D \to C$ with $C = \operatorname{ran}(f)$ and $|C| = k+1$.

Fix some $c_0 \in C$ (the range is not empty because $k+1 > 0$). Let $\hat{C} = C - \{c_0\}$. Because c_0 is in the range of f, the set $f^{-1}(c_0) = \{d \in D \mid f(d) = c_0\}$ has at least one element. Restated, where $\hat{D} = D - f^{-1}(c_0)$ we have $|\hat{D}| < |D|$, with a strict inequality. Now the key point: the map $\hat{f}\colon \hat{D} \to \hat{C}$ is onto, as any element $c \in \operatorname{ran}(f)$ is the image of some $d \in D$ and so any such element with $c \neq c_0$ must be the image of some $d \in \hat{D}$. Therefore the inductive hypothesis applies, giving that $|\hat{C}| \leq |\hat{D}|$. Finish by adding one, $|\operatorname{ran}(f)| = |\hat{C}| + 1 \leq |\hat{D}| + 1 \leq |D|$.

Alternate proof. We can instead do induction on the number of elements in the domain D. As above, we first illustrate the argument. Fix $D = \{0,1,2,3\}$ and codomain $C = \{10,11,12,13\}$. To do induction, we will omit an element of the domain, in this example 0, giving $\hat{D} = D - \{0\}$. There are two cases. The first is that the omitted element 0 maps to the same member of the codomain as at least one other, not omitted, element of the domain. This happens with this function f.

$$0 \mapsto 10 \quad 1 \mapsto 10 \quad 2 \mapsto 11 \quad 3 \mapsto 12$$

Consider the restriction, $\hat{f}\colon \hat{D} \to C$ given by $\hat{f}(d) = f(d)$. The inductive hypothesis applies, giving $|\operatorname{ran}(\hat{f})| \leq |\hat{D}|$. In this first case, 0 is not the only element of D associated with 10, so while $|\hat{D}| < |D|$ we have $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})|$ and the desired conclusion is immediate, $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})| \leq |\hat{D}| < |D|$.

The other case is that the omitted domain element 0 does not map to the same member of the codomain as any other domain element. That happens for this function f.

$$0 \mapsto 10 \quad 1 \mapsto 11 \quad 2 \mapsto 12 \quad 3 \mapsto 13$$

To do the induction, we omit 0 from the domain, giving $\hat{D} = D - \{0\}$. Consider the restriction map $\hat{f}\colon \hat{D} \to C$ defined by $\hat{f}(d) = f(d)$. The inductive hypothesis applies, so $|\operatorname{ran}(\hat{f})| \leq |\hat{D}|$. Then adding 1 to both sides gives $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})| + 1 \leq |\hat{D}| + 1 = |D|$.

We are ready for the proof. The base step is to consider a function f whose domain D has no elements at all. The only function with an empty domain is the empty function, whose range is empty, and so $0 = |\operatorname{ran}(f)| \leq |D| = 0$.

For the inductive step, assume that the statement is true for functions between finite sets when the domain of that function has 0 elements, or 1 element, ... or k elements. Consider $f\colon D \to C$ where $|D| = k+1$.

Fix some $d_0 \in D$ (there must be such an element since the domain is not empty, as $k+1 > 0$). Let $\hat{D} = D - \{d_0\}$ and let $\hat{f}\colon \hat{D} \to C$ be the restriction of f to $\hat{D}$, defined by $\hat{f}(d) = f(d)$. The inductive hypothesis gives $|\operatorname{ran}(\hat{f})| \leq |\hat{D}|$.

Consider the codomain element $c_0 = f(d_0)$. There are two cases: either (1) c_0 is also the image of another domain element, so that there is a $d \in \hat{D}$ with $d \neq d_0$ and $f(d) = c_0$, or else (2) it is not the image of another such element. In the first case, $\operatorname{ran}(f) = \operatorname{ran}(\hat{f})$ and we have $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})| \leq |\hat{D}| < |D|$. In the second case, $\operatorname{ran}(f) = \operatorname{ran}(\hat{f}) + 1$ and we have $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})| + 1 \leq |\hat{D}| + 1 = |D|$.

(B) We will will do induction on the number of elements in the range. The base step is that the range is empty. The only function with an empty range is the empty function, which has an empty domain, giving $0 = |\operatorname{ran}(f)| = |D| = 0$.

For the inductive step, assume that the statement is true for any one-to-one function between finite sets with whose range has 0 elements, or 1 element, ... or k elements. Consider a one-to-one $f\colon D \to C$ with $|\operatorname{ran}(f)| = k+1$.

Fix $c_0 \in \operatorname{ran}(f)$ (such an element exists because $k+1 > 0$). Let $\hat{C} = C - \{c_0\}$. Because c_0 is an element of the range of f, there is a domain element that maps to it. Because f is one-to-one, there is only one such element. Call it d_0 and let $\hat{D} = D - \{d_0\}$.

The restriction function $\hat{f}\colon \hat{D} \to \hat{C}$ is defined by $\hat{f}(d) = f(d)$. This map is also one-to-one: thinking of f as a set of ordered pairs, the assumption that it is one-to-one means that no two pairs have the same second element, and the restriction $\hat{f}$ is a subset of f so it also has no two pairs sharing the same second element. By the inductive hypothesis, $|\operatorname{ran}(\hat{f})| = |\hat{D}|$. Adding 1 gives $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})| + 1 = |\hat{D}| + 1 = |D|$.

Alternate proof. We can instead do induction on the number of elements in the domain D. The base step is that the domain is empty, $|D| = 0$. The only function with an empty domain is the empty function and we have $0 = |\operatorname{ran}(f)| = |D| = 0$.

For the inductive step, assume that the statement is true for any one-to-one function between finite sets with whose domain has 0 elements, ... k elements. Consider a one-to-one function $f\colon D \to C$ whose domain has $|D| = k + 1$.

Fix some $d_0 \in D$ (the set D is not empty because $k + 1 > 0$) and let $\hat{D} = D - \{d_0\}$. Consider $c_0 = f(d_0)$. Because f is one-to-one, c_0 is not the image of any element of the domain other than d_0. So the restriction map $\hat{f}\colon \hat{D} \to C$ has one less element in its range, $\operatorname{ran}(f) = \operatorname{ran}(\hat{f}) + 1$.

The key point is that because f is one-to-one, $\hat{f}$ is also one-to-one: thinking of f as a set of ordered pairs, one-to-one-ness means that no two pairs share the same second element, and $\hat{f}$ is a subset of f so it has the same property. Therefore the inductive hypothesis applies, giving $|\operatorname{ran}(\hat{f})| = |\hat{D}|$. Add 1 to get the desired conclusion, that $|\operatorname{ran}(f)| = |\operatorname{ran}(\hat{f})| + 1 = |\hat{D}| + 1 = |D|$.

C.1

(A) This is a table for $\neg(P \vee Q)$.

P	Q	$P \vee Q$	$\neg(P \vee Q)$
F	F	F	T
F	T	T	F
T	F	T	F
T	T	T	F

(B) The same for $\neg P \wedge \neg Q$.

P	Q	$\neg P$	$\neg Q$	$\neg P \wedge \neg Q$
F	F	T	T	T
F	T	T	F	F
T	F	F	T	F
T	T	F	F	F

It has the same final column as the prior item.

(C) This is a table for $\neg(P \wedge Q)$.

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$
F	F	F	T
F	T	F	T
T	F	F	T
T	T	T	F

(D) This is the one for $\neg P \vee \neg Q$.

P	Q	$\neg P$	$\neg Q$	$\neg P \vee \neg Q$
F	F	T	T	T
F	T	T	F	T
T	F	F	T	T
T	T	F	F	F

The final column is the same as the final column of the prior item.

C.2

(A) This is a table for $(P \wedge Q) \wedge R$.

P	Q	R	$P \wedge Q$	$(P \wedge Q) \wedge R$
F	F	F	F	F
F	F	T	F	F
F	T	F	F	F
F	T	T	F	F
T	F	F	F	F
T	F	T	F	F
T	T	F	T	F
T	T	T	T	T

(B) And this is a table for $P \wedge (Q \wedge R)$. It has the same final column as the one from the prior item.

P	Q	R	$Q \wedge R$	$P \wedge (Q \wedge R)$
F	F	F	F	F
F	F	T	F	F
F	T	F	F	F
F	T	T	T	F
T	F	F	F	F
T	F	T	F	F
T	T	F	F	F
T	T	T	T	T

(C) This is a table for $P \wedge (Q \vee R)$.

P	Q	R	$Q \vee R$	$P \wedge (Q \vee R)$
F	F	F	F	F
F	F	T	T	F
F	T	F	T	F
F	T	T	T	F
T	F	F	F	F
T	F	T	T	T
T	T	F	T	T
T	T	T	T	T

(D) And this is a table for $(P \wedge Q) \vee (P \wedge R)$. It has the same final column as the prior item.

P	Q	R	$P \wedge Q$	$P \wedge R$	$(P \wedge Q) \vee (P \wedge R)$
F	F	F	F	F	F
F	F	T	F	F	F
F	T	F	F	F	F
F	T	T	F	F	F
T	F	F	F	F	F
T	F	T	F	T	T
T	T	F	T	F	T
T	T	T	T	T	T

C.3 In Disjunctive Normal form the clauses are separated by disjuncts.

(A) The second, third and sixth lines show T. The associated clauses are $\neg P \wedge \neg Q \wedge R$, $\neg P \wedge Q \wedge \neg R$, and $P \wedge \neg Q \wedge R$. The resulting CNF expression is this.

$$(\neg P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge R)$$

(B) The T lines give four clauses.

$$(\neg P \wedge \neg Q \wedge \neg R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R)$$

C.4 In Conjunctive Normal form the clauses are separated by conjuncts.

(A) The F lines are first, fourth, fifth, seventh, and eighth.

$$E_0 \equiv (P \vee Q \vee R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee R) \wedge (\neg P \vee \neg Q \vee R) \wedge (\neg P \vee \neg Q \vee \neg R)$$

(B) Find the F lines.

$$E_1 \equiv (P \vee Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee R) \wedge (\neg P \vee Q \vee \neg R)$$

C.5 Here are the sixteen.

P	Q	O_0
F	F	F
F	T	F
T	F	F
T	T	F

P	Q	O_1
F	F	F
F	T	F
T	F	F
T	T	T

P	Q	O_2
F	F	F
F	T	F
T	F	T
T	T	F

P	Q	O_3
F	F	F
F	T	F
T	F	T
T	T	T

P	Q	O_4
F	F	F
F	T	T
T	F	F
T	T	F

P	Q	O_5
F	F	F
F	T	T
T	F	F
T	T	T

P	Q	O_6
F	F	F
F	T	T
T	F	T
T	T	F

P	Q	O_7
F	F	F
F	T	T
T	F	T
T	T	T

P	Q	O_8
F	F	T
F	T	F
T	F	F
T	T	F

P	Q	O_9
F	F	T
F	T	F
T	F	F
T	T	T

P	Q	O_{10}
F	F	T
F	T	F
T	F	T
T	T	F

P	Q	O_{11}
F	F	T
F	T	F
T	F	T
T	T	T

P	Q	O_{12}
F	F	T
F	T	T
T	F	F
T	T	F

P	Q	O_{13}
F	F	T
F	T	T
T	F	F
T	T	T

P	Q	O_{14}
F	F	T
F	T	T
T	F	T
T	T	F

P	Q	O_{15}
F	F	T
F	T	T
T	F	T
T	T	T

Here are the operators.

Operator	*Expression*	*Operator*	*Expression*
O_0	F (Called 'falsum', sometimes written $\bot$)	O_8	$\neg(P \vee Q)$
O_1	$P \wedge Q$	O_9	$P \leftrightarrow Q$
O_2	$\neg(P \rightarrow Q)$	O_{10}	$\neg Q$
O_3	P	O_{11}	$Q \rightarrow P$
O_4	$\neg(Q \rightarrow P)$	O_{12}	$\neg P$
O_5	Q	O_{13}	$P \rightarrow Q$
O_6	$\neg(P \leftrightarrow Q)$	O_{14}	$\neg(P \wedge Q)$
O_7	$P \vee Q$	O_{15}	T (sometimes written $\top$)